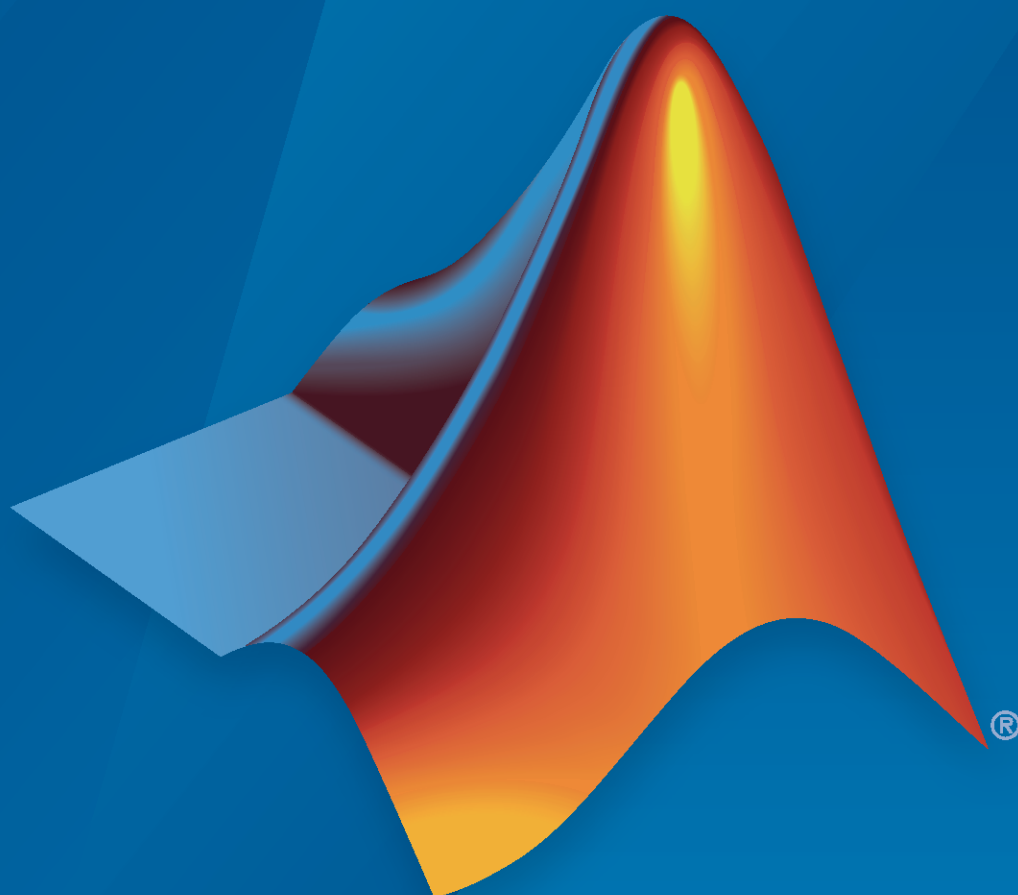


Financial Toolbox™

User's Guide



MATLAB®

R2025b



How to Contact MathWorks



Latest news: www.mathworks.com
Sales and services: www.mathworks.com/sales_and_services
User community: www.mathworks.com/matlabcentral
Technical support: www.mathworks.com/support/contact_us



Phone: 508-647-7000



The MathWorks, Inc.
1 Apple Hill Drive
Natick, MA 01760-2098

Financial Toolbox™ User's Guide

© COPYRIGHT 1995-2025 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

Trademarks

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See www.mathworks.com/trademarks for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Patents

MathWorks products are protected by one or more U.S. patents. Please see www.mathworks.com/patents for more information.

Revision History

October 1995	First printing	
January 1998	Second printing	Revised for Version 1.1
January 1999	Third printing	Revised for Version 2.0 (Release 11)
November 2000	Fourth printing	Revised for Version 2.1.2 (Release 12)
May 2003	Online only	Revised for Version 2.3 (Release 13)
June 2004	Online only	Revised for Version 2.4 (Release 14)
August 2004	Online only	Revised for Version 2.4.1 (Release 14+)
September 2005	Fifth printing	Revised for Version 2.5 (Release 14SP3)
March 2006	Online only	Revised for Version 3.0 (Release 2006a)
September 2006	Sixth printing	Revised for Version 3.1 (Release 2006b)
March 2007	Online only	Revised for Version 3.2 (Release 2007a)
September 2007	Online only	Revised for Version 3.3 (Release 2007b)
March 2008	Online only	Revised for Version 3.4 (Release 2008a)
October 2008	Online only	Revised for Version 3.5 (Release 2008b)
March 2009	Online only	Revised for Version 3.6 (Release 2009a)
September 2009	Online only	Revised for Version 3.7 (Release 2009b)
March 2010	Online only	Revised for Version 3.7.1 (Release 2010a)
September 2010	Online only	Revised for Version 3.8 (Release 2010b)
April 2011	Online only	Revised for Version 4.0 (Release 2011a)
September 2011	Online only	Revised for Version 4.1 (Release 2011b)
March 2012	Online only	Revised for Version 4.2 (Release 2012a)
September 2012	Online only	Revised for Version 5.0 (Release 2012b)
March 2013	Online only	Revised for Version 5.1 (Release 2013a)
September 2013	Online only	Revised for Version 5.2 (Release 2013b)
March 2014	Online only	Revised for Version 5.3 (Release 2014a)
October 2014	Online only	Revised for Version 5.4 (Release 2014b)
March 2015	Online only	Revised for Version 5.5 (Release 2015a)
September 2015	Online only	Revised for Version 5.6 (Release 2015b)
March 2016	Online only	Revised for Version 5.7 (Release 2016a)
September 2016	Online only	Revised for Version 5.8 (Release 2016b)
March 2017	Online only	Revised for Version 5.9 (Release 2017a)
September 2017	Online only	Revised for Version 5.10 (Release 2017b)
March 2018	Online only	Revised for Version 5.11 (Release 2018a)
September 2018	Online only	Revised for Version 5.12 (Release 2018b)
March 2019	Online only	Revised for Version 5.13 (Release 2019a)
September 2019	Online only	Revised for Version 5.14 (Release 2019b)
March 2020	Online only	Revised for Version 5.15 (Release 2020a)
September 2020	Online only	Revised for Version 6.0 (Release 2020b)
March 2021	Online only	Revised for Version 6.1 (Release 2021a)
September 2021	Online only	Revised for Version 6.2 (Release 2021b)
March 2022	Online only	Revised for Version 6.3 (Release 2022a)
September 2022	Online only	Revised for Version 6.4 (Release 2022b)
March 2023	Online only	Revised for Version 6.5 (Release 2023a)
September 2023	Online only	Revised for Version 23.2 (R2023b)
March 2024	Online only	Revised for Version 24.1 (R2024a)
September 2024	Online only	Revised for Version 24.2 (R2024b)
March 2025	Online only	Revised for Version 25.1 (R2025a)
September 2025	Online only	Rereleased for Version 25.2 (R2025b)

1

Getting Started

Financial Toolbox Product Description	1-2
Expected Users	1-3
Analyze Sets of Numbers Using Matrix Functions	1-4
Introduction	1-4
Key Definitions	1-4
Referencing Matrix Elements	1-4
Transposing Matrices	1-5
Matrix Algebra Refresher	1-7
Introduction	1-7
Adding and Subtracting Matrices	1-7
Multiplying Matrices	1-8
Dividing Matrices	1-11
Solving Simultaneous Linear Equations	1-11
Operating Element by Element	1-13
Using Input and Output Arguments with Functions	1-15
Input Arguments	1-15
Output Arguments	1-16

2

Performing Common Financial Tasks

Handle and Convert Dates	2-2
Date Formats	2-2
Date Conversions	2-3
Current Date and Time	2-7
Determining Specific Dates	2-8
Determining Holidays	2-8
Determining Cash-Flow Dates	2-9
Analyzing and Computing Cash Flows	2-11
Introduction	2-11
Interest Rates/Rates of Return	2-11
Present or Future Values	2-12
Depreciation	2-13
Annuities	2-13

Pricing and Computing Yields for Fixed-Income Securities	2-15
Introduction	2-15
Fixed-Income Terminology	2-15
Framework	2-18
Default Parameter Values	2-18
Coupon Date Calculations	2-20
Yield Conventions	2-21
Pricing Functions	2-21
Yield Functions	2-21
Fixed-Income Sensitivities	2-22
Treasury Bills Defined	2-25
Computing Treasury Bill Price and Yield	2-26
Introduction	2-26
Treasury Bill Repurchase Agreements	2-26
Treasury Bill Yields	2-27
Term Structure of Interest Rates	2-29
Returns with Negative Prices	2-32
Negative Price Conversion	2-32
Analysis of Negative Price Returns	2-33
Visualization of Complex Returns	2-35
Conclusion	2-38
Pricing and Analyzing Equity Derivatives	2-39
Introduction	2-39
Sensitivity Measures	2-39
Analysis Models	2-40
About Life Tables	2-44
Life Tables Theory	2-44
Case Study for Life Tables Analysis	2-46
Machine Learning for Statistical Arbitrage: Introduction	2-48
Machine Learning for Statistical Arbitrage I: Data Management and Visualization	2-50
Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development	2-59
Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction	2-69
Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data	2-78

Analyzing Portfolios	3-2
Portfolio Optimization Functions	3-3
Portfolio Construction Examples	3-5
Introduction	3-5
Efficient Frontier Example	3-5
Portfolio Selection and Risk Aversion	3-7
Introduction	3-7
Optimal Risky Portfolio	3-8
portopt Migration to Portfolio Object	3-11
Migrate portopt Without Output Arguments	3-11
Migrate portopt with Output Arguments	3-12
Migrate portopt for Target Returns Within Range of Efficient Portfolio Returns	3-13
Migrate portopt for Target Return Outside Range of Efficient Portfolio Returns	3-14
Migrate portopt Using portcons Output for ConSet	3-15
Integrate Output from portcons, pcalims, pcglims, and pcgcomp with a Portfolio Object	3-17
Constraint Specification Using a Portfolio Object	3-19
Constraints for Efficient Frontier	3-19
Linear Constraint Equations	3-21
Specifying Group Constraints	3-24
Active Returns and Tracking Error Efficient Frontier	3-27

Mean-Variance Portfolio Optimization Tools

Portfolio Optimization Theory	4-4
Portfolio Optimization Problems	4-4
Portfolio Problem Specification	4-4
Return Proxy	4-5
Risk Proxy	4-6
Supported Constraints for Portfolio Optimization Using Portfolio Objects	4-9
Linear Inequality Constraints	4-9
Linear Equality Constraints	4-10
'Simple' Bound Constraints	4-10
'Conditional' Bound Constraints	4-11
Budget Constraints	4-11
Conditional Budget Constraints	4-12
Group Constraints	4-12

Group Ratio Constraints	4-13
Average Turnover Constraints	4-14
One-Way Turnover Constraints	4-14
Tracking Error Constraints	4-15
Cardinality Constraints	4-16
Default Portfolio Problem	4-17
Portfolio Object Workflow	4-18
Portfolio Object	4-20
Portfolio Object Properties and Functions	4-20
Working with Portfolio Objects	4-20
Setting and Getting Properties	4-20
Displaying Portfolio Objects	4-21
Saving and Loading Portfolio Objects	4-21
Estimating Efficient Portfolios and Frontiers	4-21
Arrays of Portfolio Objects	4-22
Subclassing Portfolio Objects	4-23
Conventions for Representation of Data	4-23
Creating the Portfolio Object	4-25
Syntax	4-25
Portfolio Problem Sufficiency	4-25
Portfolio Function Examples	4-26
Common Operations on the Portfolio Object	4-33
Naming a Portfolio Object	4-33
Configuring the Assets in the Asset Universe	4-33
Setting Up a List of Asset Identifiers	4-33
Truncating and Padding Asset Lists	4-35
Setting Up an Initial or Current Portfolio	4-37
Setting Up a Tracking Portfolio	4-40
Asset Returns and Moments of Asset Returns Using Portfolio Object	4-42
Assignment Using the Portfolio Function	4-42
Assignment Using the setAssetMoments Function	4-43
Scalar Expansion of Arguments	4-44
Estimating Asset Moments from Prices or Returns	4-45
Estimating Asset Moments with Missing Data	4-47
Estimating Asset Moments from Time Series Data	4-49
Working with a Riskless Asset	4-52
Working with Transaction Costs	4-54
Setting Transaction Costs Using the Portfolio Function	4-54
Setting Transaction Costs Using the setCosts Function	4-54
Setting Transaction Costs with Scalar Expansion	4-56
Working with Portfolio Constraints Using Defaults	4-58
Setting Default Constraints for Portfolio Weights Using Portfolio Object	4-58

Working with 'Simple' Bound Constraints Using Portfolio Object	4-62
Setting 'Simple' Bounds Using the Portfolio Function	4-62
Setting 'Simple' Bounds Using the setBounds Function	4-62
Setting 'Simple' Bounds Using the Portfolio Function or setBounds Function	4-63
Working with Budget Constraints Using Portfolio Object	4-65
Setting Budget Constraints Using the Portfolio Function	4-65
Setting Budget Constraints Using the setBudget Function	4-65
Working with Conditional Budget Constraints Using Portfolio Object . .	4-67
Setting Conditional Budget Constraints Using the Portfolio Function . . .	4-67
Setting Conditional Budget Constraints Using the setConditionalBudget Function	4-67
Working with Group Constraints Using Portfolio Object	4-69
Setting Group Constraints Using the Portfolio Function	4-69
Setting Group Constraints Using the setGroups and addGroups Functions	4-69
Working with Group Ratio Constraints Using Portfolio Object	4-72
Setting Group Ratio Constraints Using the Portfolio Function	4-72
Setting Group Ratio Constraints Using the setGroupRatio and addGroupRatio Functions	4-73
Working with Linear Equality Constraints Using Portfolio Object	4-75
Setting Linear Equality Constraints Using the Portfolio Function	4-75
Setting Linear Equality Constraints Using the setEquality and addEquality Functions	4-75
Working with Linear Inequality Constraints Using Portfolio Object	4-78
Setting Linear Inequality Constraints Using the Portfolio Function	4-78
Setting Linear Inequality Constraints Using the setInequality and addInequality Functions	4-78
Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects	4-81
Setting 'Conditional' BoundType Constraints Using the setBounds Function	4-81
Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function	4-82
Working with Average Turnover Constraints Using Portfolio Object . . .	4-85
Setting Average Turnover Constraints Using the Portfolio Function	4-85
Setting Average Turnover Constraints Using the setTurnover Function . .	4-85
Working with One-Way Turnover Constraints Using Portfolio Object . . .	4-88
Setting One-Way Turnover Constraints Using the Portfolio Function	4-88
Setting Turnover Constraints Using the setOneWayTurnover Function . .	4-88
Working with Tracking Error Constraints Using Portfolio Object	4-91
Setting Tracking Error Constraints Using the Portfolio Function	4-91
Setting Tracking Error Constraints Using the setTrackingError Function	4-91

Validate the Portfolio Problem for Portfolio Object	4-94
Validating a Portfolio Set	4-94
Validating Portfolios	4-95
Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object	4-98
Obtaining Portfolios Along the Entire Efficient Frontier	4-99
Obtaining Endpoints of the Efficient Frontier	4-102
Obtaining Efficient Portfolios for Target Returns	4-105
Obtaining Efficient Portfolios for Target Risks	4-108
Efficient Portfolio That Maximizes Sharpe Ratio	4-111
Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization	4-114
Using 'lcprog' and 'quadprog'	4-114
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	4-115
Solver Guidelines for Portfolio Objects	4-115
Solver Guidelines for Custom Objective Problems Using Portfolio Objects	4-119
Estimate Efficient Frontiers for Portfolio Object	4-122
Obtaining Portfolio Risks and Returns	4-122
Plotting the Efficient Frontier for a Portfolio Object	4-125
Postprocessing Results to Set Up Tradable Portfolios	4-130
When to Use Portfolio Objects Over Optimization Toolbox	4-132
Always Use Portfolio, PortfolioCVaR, or PortfolioMAD Object	4-134
Preferred Use of Portfolio, PortfolioCVaR, or PortfolioMAD Object	4-135
Use Optimization Toolbox	4-136
Comparison of Methods for Covariance Estimation	4-138
Choose MINLP Solvers for Portfolio Problems	4-140
Troubleshooting Portfolio Optimization Results	4-145
Portfolio Object Destroyed When Modifying	4-145
Optimization Fails with "Bad Pivot" Message	4-145
Speed of Optimization	4-145
Matrix Incompatibility and "Non-Conformable" Errors	4-145
Missing Data Estimation Fails	4-145
mv_optim_transform Errors	4-145
solveContinuousCustomObjProb or solveMICustomObjProb Errors	4-146
Efficient Portfolios Do Not Make Sense	4-146
Efficient Frontiers Do Not Make Sense	4-146
Troubleshooting estimateCustomObjectivePortfolio	4-148
Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints	4-148

Role of Convexity in Portfolio Problems	4-157
Examples of Convex Functions	4-158
Examples of Concave Functions	4-159
Examples of Nonconvex Functions	4-159
Portfolio Optimization Examples Using Financial Toolbox	4-161
Asset Allocation Case Study	4-180
Portfolio Optimization with Semicontinuous and Cardinality Constraints	4-190
Portfolio Optimization Against a Benchmark	4-202
Portfolio Analysis with Turnover Constraints	4-211
Leverage in Portfolio Optimization with a Risk-Free Asset	4-217
Black-Litterman Portfolio Optimization Using Financial Toolbox	4-222
Portfolio Optimization Using Factor Models	4-231
Backtest Investment Strategies Using Financial Toolbox	4-238
Backtest Investment Strategies with Trading Signals	4-251
Portfolio Optimization Using Social Performance Measure	4-264
Diversify ESG Portfolios	4-271
Risk Budgeting Portfolio	4-286
Backtest Using Risk-Based Equity Indexation	4-291
Create Hierarchical Risk Parity Portfolio	4-296
Backtest Strategies Using Deep Learning	4-302
Backtest with Brinson Attribution to Evaluate Portfolio Performance	4-315
Analyze Performance Attribution Using Brinson Model	4-323
Diversify Portfolios Using Custom Objective	4-331
Solve Tracking Error Portfolio Problems	4-343
Solve Problem for Minimum Tracking Error with Net Return Constraint	4-349
Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty	4-351
Risk Parity or Budgeting with Constraints	4-357

Single Period Goal-Based Wealth Management	4-362
Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods	4-367
Multiperiod Goal-Based Wealth Management Using Reinforcement Learning	4-379
Compare Performance of Covariance Denoising with Factor Modeling Using Backtesting	4-394
Mixed-Integer Mean-Variance Portfolio Optimization Problem	4-402
Deep Reinforcement Learning for Optimal Trade Execution	4-407
Backtest Investment Strategies Using datetime and calendarDuration	4-451
Adding Constraints to Satisfy UCITS Directive	4-457

CVaR Portfolio Optimization Tools

5

Portfolio Optimization Theory	5-3
Portfolio Optimization Problems	5-3
Portfolio Problem Specification	5-3
Return Proxy	5-4
Risk Proxy	5-5
Supported Constraints for Portfolio Optimization Using PortfolioCVaR	
Object	5-8
Linear Inequality Constraints	5-8
Linear Equality Constraints	5-9
'Simple' Bound Constraints	5-9
'Conditional' Bound Constraints	5-10
Budget Constraints	5-10
Conditional Budget Constraints	5-11
Group Constraints	5-11
Group Ratio Constraints	5-12
Average Turnover Constraints	5-13
One-way Turnover Constraints	5-13
Cardinality Constraints	5-14
Default Portfolio Problem	5-15
PortfolioCVaR Object Workflow	5-16
PortfolioCVaR Object	5-17
PortfolioCVaR Object Properties and Functions	5-17
Working with PortfolioCVaR Objects	5-17
Setting and Getting Properties	5-18
Displaying PortfolioCVaR Objects	5-18

Saving and Loading PortfolioCVaR Objects	5-18
Estimating Efficient Portfolios and Frontiers	5-18
Arrays of PortfolioCVaR Objects	5-19
Subclassing PortfolioCVaR Objects	5-20
Conventions for Representation of Data	5-20
Creating the PortfolioCVaR Object	5-22
Syntax	5-22
PortfolioCVaR Problem Sufficiency	5-22
PortfolioCVaR Function Examples	5-23
Common Operations on the PortfolioCVaR Object	5-29
Naming a PortfolioCVaR Object	5-29
Configuring the Assets in the Asset Universe	5-29
Setting Up a List of Asset Identifiers	5-29
Truncating and Padding Asset Lists	5-31
Setting Up an Initial or Current Portfolio	5-33
Asset Returns and Scenarios Using PortfolioCVaR Object	5-36
How Stochastic Optimization Works	5-36
What Are Scenarios?	5-36
Setting Scenarios Using the PortfolioCVaR Function	5-37
Setting Scenarios Using the setScenarios Function	5-38
Estimating the Mean and Covariance of Scenarios	5-38
Simulating Normal Scenarios	5-39
Simulating Normal Scenarios from Returns or Prices	5-39
Simulating Normal Scenarios with Missing Data	5-40
Simulating Normal Scenarios from Time Series Data	5-42
Simulating Normal Scenarios with Mean and Covariance	5-44
Working with a Riskless Asset	5-46
Working with Transaction Costs	5-47
Setting Transaction Costs Using the PortfolioCVaR Function	5-47
Setting Transaction Costs Using the setCosts Function	5-47
Setting Transaction Costs with Scalar Expansion	5-49
Working with CVaR Portfolio Constraints Using Defaults	5-51
Setting Default Constraints for Portfolio Weights Using PortfolioCVaR Object	5-51
Working with 'Simple' Bound Constraints Using PortfolioCVaR Object	5-55
Setting 'Simple' Bounds Using the PortfolioCVaR Function	5-55
Setting 'Simple' Bounds Using the setBounds Function	5-55
Setting 'Simple' Bounds Using the PortfolioCVaR Function or setBounds Function	5-56
Working with Budget Constraints Using PortfolioCVaR Object	5-58
Setting Budget Constraints Using the PortfolioCVaR Function	5-58
Setting Budget Constraints Using the setBudget Function	5-58

Working with Conditional Budget Constraints Using PortfolioCvAR Object	
.....	5-60
Setting Conditional Budget Constraints Using the PortfolioCvAR Function	
.....	5-60
Setting Conditional Budget Constraints Using the setConditionalBudget	
Function	5-60
Working with Group Constraints Using PortfolioCvAR Object	5-62
Setting Group Constraints Using the PortfolioCvAR Function	5-62
Setting Group Constraints Using the setGroups and addGroups Functions	
.....	5-62
Working with Group Ratio Constraints Using PortfolioCvAR Object	5-65
Setting Group Ratio Constraints Using the PortfolioCvAR Function	5-65
Setting Group Ratio Constraints Using the setGroupRatio and	
addGroupRatio Functions	5-66
Working with Linear Equality Constraints Using PortfolioCvAR Object	
.....	5-68
Setting Linear Equality Constraints Using the PortfolioCvAR Function ..	5-68
Setting Linear Equality Constraints Using the setEquality and addEquality	
Functions	5-68
Working with Linear Inequality Constraints Using PortfolioCvAR Object	
.....	5-70
Setting Linear Inequality Constraints Using the PortfolioCvAR Function	
.....	5-70
Setting Linear Inequality Constraints Using the setInequality and	
addInequality Functions	5-70
Working with 'Conditional' BoundType, MinNumAssets, and	
 MaxNumAssets Constraints Using PortfolioCvAR Objects	5-72
Setting 'Conditional' BoundType Constraints Using the setBounds Function	
.....	5-72
Setting the Limits on the Number of Assets Invested Using the	
setMinMaxNumAssets Function	5-73
Working with Average Turnover Constraints Using PortfolioCvAR Object	
.....	5-75
Setting Average Turnover Constraints Using the PortfolioCvAR Function	
.....	5-75
Setting Average Turnover Constraints Using the setTurnover Function ..	5-75
Working with One-Way Turnover Constraints Using PortfolioCvAR Object	
.....	5-78
Setting One-Way Turnover Constraints Using the PortfolioCvAR Function	
.....	5-78
Setting Turnover Constraints Using the setOneWayTurnover Function ..	5-78
Validate the CVaR Portfolio Problem	5-81
Validating a CVaR Portfolio Set	5-81
Validating CVaR Portfolios	5-82
Estimate Efficient Portfolios for Entire Frontier for PortfolioCvAR Object	
.....	5-85

Obtaining Portfolios Along the Entire Efficient Frontier	5-86
Obtaining Endpoints of the Efficient Frontier	5-89
Obtaining Efficient Portfolios for Target Returns	5-92
Obtaining Efficient Portfolios for Target Risks	5-95
Choosing and Controlling the Solver for PortfolioCVaR Optimizations .	5-98
Using 'TrustRegionCP', 'ExtendedCP', and 'cuttingplane' SolverTypes . . .	5-98
Using 'fmincon' SolverType	5-99
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	5-100
Solver Guidelines for PortfolioCVaR Objects	5-100
Estimate Efficient Frontiers for PortfolioCVaR Object	5-105
Obtaining CVaR Portfolio Risks and Returns	5-105
Obtaining Portfolio Standard Deviation and VaR	5-106
Plotting the Efficient Frontier for a PortfolioCVaR Object	5-109
Postprocessing Results to Set Up Tradable Portfolios	5-115
Working with Other Portfolio Objects	5-118
Troubleshooting CVaR Portfolio Optimization Results	5-121
PortfolioCVaR Object Destroyed When Modifying	5-121
Matrix Incompatibility and "Non-Conformable" Errors	5-121
CVaR Portfolio Optimization Warns About "Max Iterations"	5-121
CVaR Portfolio Optimization Errors with "Could Not Solve" Message . .	5-122
Missing Data Estimation Fails	5-122
cvar_optim_transform Errors	5-122
Efficient Portfolios Do Not Make Sense	5-123
Hedging Using CVaR Portfolio Optimization	5-125
Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio	5-137
Mixed-Integer CVaR Portfolio Optimization Problem	5-141
Bond Portfolio CVaR Optimization Using Diebold-Li Model	5-146

MAD Portfolio Optimization Tools

6

Portfolio Optimization Theory	6-3
Portfolio Optimization Problems	6-3
Portfolio Problem Specification	6-3
Return Proxy	6-4
Risk Proxy	6-5

Supported Constraints for Portfolio Optimization Using PortfolioMAD	
Object	6-8
Linear Inequality Constraints	6-8
Linear Equality Constraints	6-9
'Simple' Bound Constraints	6-9
'Conditional' Bound Constraints	6-10
Budget Constraints	6-10
Conditional Budget Constraints	6-11
Group Constraints	6-11
Group Ratio Constraints	6-12
Average Turnover Constraints	6-13
One-way Turnover Constraints	6-13
Cardinality Constraints	6-14
Default Portfolio Problem	6-15
PortfolioMAD Object Workflow	6-16
PortfolioMAD Object	6-17
PortfolioMAD Object Properties and Functions	6-17
Working with PortfolioMAD Objects	6-17
Setting and Getting Properties	6-18
Displaying PortfolioMAD Objects	6-18
Saving and Loading PortfolioMAD Objects	6-18
Estimating Efficient Portfolios and Frontiers	6-18
Arrays of PortfolioMAD Objects	6-19
Subclassing PortfolioMAD Objects	6-20
Conventions for Representation of Data	6-20
Creating the PortfolioMAD Object	6-22
Syntax	6-22
PortfolioMAD Problem Sufficiency	6-22
PortfolioMAD Function Examples	6-23
Common Operations on the PortfolioMAD Object	6-29
Naming a PortfolioMAD Object	6-29
Configuring the Assets in the Asset Universe	6-29
Setting Up a List of Asset Identifiers	6-29
Truncating and Padding Asset Lists	6-31
Setting Up an Initial or Current Portfolio	6-33
Asset Returns and Scenarios Using PortfolioMAD Object	6-35
How Stochastic Optimization Works	6-35
What Are Scenarios?	6-35
Setting Scenarios Using the PortfolioMAD Function	6-36
Setting Scenarios Using the setScenarios Function	6-37
Estimating the Mean and Covariance of Scenarios	6-37
Simulating Normal Scenarios	6-38
Simulating Normal Scenarios from Returns or Prices	6-38
Simulating Normal Scenarios with Missing Data	6-39
Simulating Normal Scenarios from Time Series Data	6-41
Simulating Normal Scenarios with Mean and Covariance	6-43
Working with a Riskless Asset	6-45

Working with Transaction Costs	6-46
Setting Transaction Costs Using the PortfolioMAD Function	6-46
Setting Transaction Costs Using the setCosts Function	6-46
Setting Transaction Costs with Scalar Expansion	6-48
Working with MAD Portfolio Constraints Using Defaults	6-50
Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object	6-50
Working with 'Simple' Bound Constraints Using PortfolioMAD Object .	6-54
Setting 'Simple' Bounds Using the PortfolioMAD Function	6-54
Setting 'Simple' Bounds Using the setBounds Function	6-54
Setting 'Simple' Bounds Using the PortfolioMAD Function or setBounds Function	6-55
Working with Budget Constraints Using PortfolioMAD Object	6-57
Setting Budget Constraints Using the PortfolioMAD Function	6-57
Setting Budget Constraints Using the setBudget Function	6-57
Working with Conditional Budget Constraints Using PortfolioMAD Object	6-59
Setting Conditional Budget Constraints Using the PortfolioMAD Function	6-59
Setting Conditional Budget Constraints Using the setConditionalBudget Function	6-59
Working with Group Constraints Using PortfolioMAD Object	6-61
Setting Group Constraints Using the PortfolioMAD Function	6-61
Setting Group Constraints Using the setGroups and addGroups Functions	6-61
Working with Group Ratio Constraints Using PortfolioMAD Object	6-64
Setting Group Ratio Constraints Using the PortfolioMAD Function	6-64
Setting Group Ratio Constraints Using the setGroupRatio and addGroupRatio Functions	6-65
Working with Linear Equality Constraints Using PortfolioMAD Object	6-67
Setting Linear Equality Constraints Using the PortfolioMAD Function ..	6-67
Setting Linear Equality Constraints Using the setEquality and addEquality Functions	6-67
Working with Linear Inequality Constraints Using PortfolioMAD Object	6-69
Setting Linear Inequality Constraints Using the PortfolioMAD Function .	6-69
Setting Linear Inequality Constraints Using the setInequality and addInequality Functions	6-69
Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects	6-71
Setting 'Conditional' BoundType Constraints Using the setBounds Function	6-71
Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function	6-72

Working with Average Turnover Constraints Using PortfolioMAD Object	6-74
Setting Average Turnover Constraints Using the PortfolioMAD Function	6-74
Setting Average Turnover Constraints Using the setTurnover Function	6-74
Working with One-Way Turnover Constraints Using PortfolioMAD Object	6-77
Setting One-Way Turnover Constraints Using the PortfolioMAD Function	6-77
Setting Turnover Constraints Using the setOneWayTurnover Function	6-77
Validate the MAD Portfolio Problem	6-80
Validating a MAD Portfolio Set	6-80
Validating MAD Portfolios	6-81
Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object	6-84
Obtaining Portfolios Along the Entire Efficient Frontier	6-85
Obtaining Endpoints of the Efficient Frontier	6-87
Mixed-Integer MAD Portfolio Optimization Problem	6-89
Obtaining Efficient Portfolios for Target Returns	6-93
Obtaining Efficient Portfolios for Target Risks	6-96
Choosing and Controlling the Solver for PortfolioMAD Optimizations	6-99
Using 'TrustRegionCP' and 'ExtendedCP' SolverTypes	6-99
Using 'fmincon' SolverType	6-100
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	6-101
Solver Guidelines for PortfolioMAD Objects	6-101
Estimate Efficient Frontiers for PortfolioMAD Object	6-105
Obtaining MAD Portfolio Risks and Returns	6-105
Obtaining the PortfolioMAD Standard Deviation	6-106
Plotting the Efficient Frontier for a PortfolioMAD Object	6-108
Postprocessing Results to Set Up Tradable Portfolios	6-113
Working with Other Portfolio Objects	6-115
Troubleshooting MAD Portfolio Optimization Results	6-118
PortfolioMAD Object Destroyed When Modifying	6-118
Matrix Incompatibility and "Non-Conformable" Errors	6-118
Missing Data Estimation Fails	6-118
mad_optim_transform Errors	6-118
Efficient Portfolios Do Not Make Sense	6-119

Performance Metrics Overview	7-2
Performance Metrics Types	7-2
Performance Metrics Illustration	7-3
Using the Sharpe Ratio	7-5
Using the Information Ratio	7-7
Using Tracking Error	7-8
Using Risk-Adjusted Return	7-9
Using Sample and Expected Lower Partial Moments	7-11
Introduction	7-11
Sample Lower Partial Moments	7-11
Expected Lower Partial Moments	7-12
Using Maximum and Expected Maximum Drawdown	7-14
Introduction	7-14
Maximum Drawdown	7-14
Expected Maximum Drawdown	7-16

Estimation of Transition Probabilities	8-2
Introduction	8-2
Estimate Transition Probabilities	8-2
Estimate Transition Probabilities for Different Rating Scales	8-4
Working with a Transition Matrix Containing NR Rating	8-6
Estimate Point-in-Time and Through-the-Cycle Probabilities	8-10
Estimate t-Year Default Probabilities	8-12
Estimate Bootstrap Confidence Intervals	8-13
Group Credit Ratings	8-15
Work with Nonsquare Matrices	8-17
Remove Outliers	8-18
Estimate Probabilities for Different Segments	8-19
Work with Large Datasets	8-20
Forecasting Corporate Default Rates	8-23
Credit Quality Thresholds	8-45
Introduction	8-45
Compute Credit Quality Thresholds	8-45
Visualize Credit Quality Thresholds	8-46

About Credit Scorecards	8-49
What Is a Credit Scorecard?	8-49
Credit Scorecard Development Process	8-50
Credit Scorecard Modeling Workflow	8-52
Credit Scorecard Modeling Using Observation Weights	8-55
Credit Scorecard Modeling with Missing Values	8-57
Troubleshooting Credit Scorecard Results	8-64
Predictor Name Is Unspecified and the Parser Returns an Error	8-64
Using bininfo or plotbins Before Binning	8-64
If Categorical Data Is Given as Numeric	8-66
NaNs Returned When Scoring a “Test” Dataset	8-68
Case Study for Credit Scorecard Analysis	8-71
Credit Scorecards with Constrained Logistic Regression Coefficients ..	8-91
Credit Default Swap (CDS)	8-100
Bootstrapping a Default Probability Curve	8-101
Finding Breakeven Spread for New CDS Contract	8-104
Valuing an Existing CDS Contract	8-107
Converting from Running to Upfront	8-109
Bootstrapping from Inverted Market Curves	8-111
Visualize Transitions Data for transprob	8-114
Impute Missing Data in the Credit Scorecard Workflow Using the k-Nearest Neighbors Algorithm	8-121
Impute Missing Data in the Credit Scorecard Workflow Using the Random Forest Algorithm	8-127
Treat Missing Data in a Credit Scorecard Workflow Using MATLAB fillmissing	8-132

Regression with Missing Data

9

Multivariate Normal Regression	9-2
Introduction	9-2
Multivariate Normal Linear Regression	9-2
Maximum Likelihood Estimation	9-3
Special Case of Multiple Linear Regression Model	9-4

Least-Squares Regression	9-4
Mean and Covariance Estimation	9-4
Convergence	9-4
Fisher Information	9-4
Statistical Tests	9-5
Maximum Likelihood Estimation with Missing Data	9-7
Introduction	9-7
ECM Algorithm	9-7
Standard Errors	9-8
Data Augmentation	9-8
Multivariate Normal Regression Functions	9-10
Multivariate Normal Regression Without Missing Data	9-11
Multivariate Normal Regression With Missing Data	9-11
Least-Squares Regression With Missing Data	9-11
Multivariate Normal Parameter Estimation With Missing Data	9-12
Support Functions	9-12
Multivariate Normal Regression Types	9-13
Regressions	9-13
Multivariate Normal Regression	9-13
Multivariate Normal Regression Without Missing Data	9-13
Multivariate Normal Regression With Missing Data	9-14
Least-Squares Regression	9-14
Least-Squares Regression Without Missing Data	9-14
Least-Squares Regression With Missing Data	9-14
Covariance-Weighted Least Squares	9-14
Covariance-Weighted Least Squares Without Missing Data	9-15
Covariance-Weighted Least Squares With Missing Data	9-15
Feasible Generalized Least Squares	9-15
Feasible Generalized Least Squares Without Missing Data	9-15
Feasible Generalized Least Squares With Missing Data	9-16
Seemingly Unrelated Regression	9-16
Seemingly Unrelated Regression Without Missing Data	9-17
Seemingly Unrelated Regression With Missing Data	9-17
Mean and Covariance Parameter Estimation	9-17
Troubleshooting Multivariate Normal Regression	9-18
Biased Estimates	9-18
Requirements	9-18
Slow Convergence	9-18
Nonrandom Residuals	9-19
Nonconvergence	9-19
Portfolios with Missing Data	9-21
Valuation with Missing Data	9-26
Introduction	9-26
Capital Asset Pricing Model	9-26
Estimation of the CAPM	9-27
Estimation with Missing Data	9-27
Estimation of Some Technology Stock Betas	9-27
Grouped Estimation of Some Technology Stock Betas	9-30
References	9-32

Capital Asset Pricing Model with Missing Data	9-34
---	------

Solving Sample Problems

10

Sensitivity of Bond Prices to Interest Rates	10-2
Bond Portfolio for Hedging Duration and Convexity	10-6
Bond Prices and Yield Curve Parallel Shifts	10-9
Bond Prices and Yield Curve Nonparallel Shifts	10-12
Greek-Neutral Portfolios of European Stock Options	10-14
Term Structure Analysis and Interest-Rate Swaps	10-18
Plotting an Efficient Frontier Using portopt	10-22
Plotting Sensitivities of an Option	10-25
Plotting Sensitivities of a Portfolio of Options	10-27
Bond Portfolio Optimization Using Portfolio Object	10-30
Hedge Options Using Reinforcement Learning Toolbox	10-40
Hedge Using Monte Carlo Simulation	10-49
Using Extreme Value Theory and Copula Fitting to Generate Synthetic Data	10-61

Using Financial Timetables

11

Convert Financial Time Series Objects (fints) to Timetables	11-2
Create Time Series	11-2
Index an Object	11-3
Transform Time Series	11-3
Convert Time Series	11-4
Merge Time Series	11-5
Analyze Time Series	11-5
Data Extraction	11-6
Use Timetables in Finance	11-7

12

Trading Calendars User Interface	12-2
UICalendar User Interface	12-4
Using UICalendar in Standalone Mode	12-4
Using UICalendar with an Application	12-4

Technical Analysis**13**

Technical Indicators	13-2
-----------------------------------	-------------

Stochastic Differential Equations**14**

SDEs	14-2
SDE Modeling	14-2
Trials vs. Paths	14-3
NTrials, NPeriods, and NSteps	14-3
SDE Class Hierarchy	14-5
SDE Models	14-7
Introduction	14-7
Creating SDE Objects	14-7
Drift and Diffusion	14-10
Available SDE Models	14-11
SDE Simulation and Interpolation Methods	14-13
Base SDE Models	14-16
Overview	14-16
Specify Base Stochastic Differential Equation (SDE) Model	14-16
Drift and Diffusion Models	14-19
Overview	14-19
Specify Drift and Diffusion Rate Functions	14-19
Specify SDEDDO with Drift and Diffusion Functions	14-20
Linear Drift Models	14-22
Overview	14-22
Specify SDELD Model	14-22
Parametric Models	14-24
Creating Brownian Motion (BM) Models	14-24
Specify Brownian Motion Model	14-24
Creating Constant Elasticity of Variance (CEV) Models	14-25

Creating Geometric Brownian Motion (GBM) Models	14-25
Creating Stochastic Differential Equations from Mean-Reverting Drift (SDEMRD) Models	14-26
Creating Cox-Ingersoll-Ross (CIR) Square Root Diffusion Models	14-27
Creating Hull-White/Vasicek (HWV) Gaussian Diffusion Models	14-28
Creating Heston Stochastic Volatility Models	14-29
Simulating Equity Prices	14-31
Simulating Multidimensional Market Models	14-31
Induce Dependence and Correlation Between States	14-41
Dynamic Behavior of Market Parameters	14-43
Price European Stock Options Using Monte Carlo Simulation	14-47
Simulating Interest Rates	14-50
Simulate Interest Rates Through Interpolation	14-50
Simulate Positive Interest Rates	14-54
Stratified Sampling	14-58
Quasi-Monte Carlo Simulation	14-63
Performance Considerations	14-65
Managing Memory	14-65
Enhancing Performance	14-66
Optimizing Accuracy: About Solution Precision and Error	14-66
Price American Basket Options Using Standard Monte Carlo and Quasi- Monte Carlo Simulation	14-71
Volatility Modeling for Soft Commodities	14-88
Improving Performance of Monte Carlo Simulation with Parallel Computing	14-111

Functions

15

Bibliography

A

Bibliography	A-2
Bond Pricing and Yields	A-2
Term Structure of Interest Rates	A-2
Derivatives Pricing and Yields	A-3
Portfolio Analysis	A-3
Investment Performance Metrics	A-3
Financial Statistics	A-4
Standard References	A-4
Credit Risk Analysis	A-5

Credit Derivatives	A-5
Portfolio Optimization	A-5
Stochastic Differential Equations	A-6
Life Tables	A-6

Getting Started

- “Financial Toolbox Product Description” on page 1-2
- “Expected Users” on page 1-3
- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Matrix Algebra Refresher” on page 1-7
- “Using Input and Output Arguments with Functions” on page 1-15

Financial Toolbox Product Description

Analyze financial data and develop financial models

Financial Toolbox provides functions for the mathematical modeling and statistical analysis of financial data. You can analyze, backtest, and optimize investment portfolios taking into account turnover, transaction costs, semi-continuous constraints, and minimum or maximum number of assets. The toolbox enables you to estimate risk, model credit scorecards, analyze yield curves, price fixed-income instruments and European options, and measure investment performance.

Stochastic differential equation (SDE) tools let you model and simulate a variety of stochastic processes. Time series analysis functions let you perform transformations or regressions with missing data and convert between different trading calendars and day-count conventions.

Expected Users

In general, this guide assumes experience working with financial derivatives and some familiarity with the underlying models.

In designing Financial Toolbox documentation, we assume that your title is like one of these:

- Analyst, quantitative analyst
- Risk manager
- Portfolio manager
- Asset allocator
- Financial engineer
- Trader
- Student, professor, or other academic

We also assume that your background, education, training, and responsibilities match some aspects of this profile:

- Finance, economics, perhaps accounting
- Engineering, mathematics, physics, other quantitative sciences
- Focus on quantitative approaches to financial problems

Analyze Sets of Numbers Using Matrix Functions

In this section...

"Introduction" on page 1-4

"Key Definitions" on page 1-4

"Referencing Matrix Elements" on page 1-4

"Transposing Matrices" on page 1-5

Introduction

Many financial analysis procedures involve *sets* of numbers; for example, a portfolio of securities at various prices and yields. Matrices, matrix functions, and matrix algebra are the most efficient ways to analyze sets of numbers and their relationships. Spreadsheets focus on individual cells and the relationships between cells. While you can think of a set of spreadsheet cells (a range of rows and columns) as a matrix, a matrix-oriented tool like MATLAB® software manipulates sets of numbers more quickly, easily, and naturally. For more information, see "Matrix Algebra Refresher" on page 1-7.

Key Definitions

Matrix

A rectangular array of numeric or algebraic quantities subject to mathematical operations; the regular formation of elements into rows and columns. Described as a "*m*-by-*n*" matrix, with *m* the number of rows and *n* the number of columns. The description is always "row-by-column." For example, here is a 2-by-3 matrix of two bonds (the rows) with different par values, coupon rates, and coupon payment frequencies per year (the columns) entered using MATLAB notation:

```
Bonds = [1000    0.06    2
          500    0.055   4]
```

Vector

A matrix with only one row or column. Described as a "1-by-*n*" or "*m*-by-1" matrix. The description is always "row-by-column." For example, here is a 1-by-4 vector of cash flows in MATLAB notation:

```
Cash = [1500    4470    5280   -1299]
```

Scalar

A 1-by-1 matrix; that is, a single number.

Referencing Matrix Elements

To reference specific matrix elements, use (row, column) notation. For example:

```
Bonds(1,2)
```

```
ans =
```

```
0.06
```

```
Cash(3)
```

```
ans =
```

```
5280.00
```

You can enlarge matrices using small matrices or vectors as elements. For example,

```
AddBond = [1000 0.065 2];
Bonds = [Bonds; AddBond]
```

adds another row to the matrix and creates

```
Bonds =
```

```
1000 0.06 2
500 0.055 4
1000 0.065 2
```

Likewise,

```
Prices = [987.50
475.00
995.00]
```

```
Bonds = [Prices, Bonds]
```

adds another column and creates

```
Bonds =
```

```
987.50 1000 0.06 2
475.00 500 0.055 4
995.00 1000 0.065 2
```

Finally, the colon (:) is important in generating and referencing matrix elements. For example, to reference the par value, coupon rate, and coupon frequency of the second bond:

```
BondItems = Bonds(2, 2:4)
```

```
BondItems =
```

```
500.00 0.055 4
```

Transposing Matrices

Sometimes matrices are in the wrong configuration for an operation. In MATLAB, the apostrophe or prime character (') transposes a matrix: columns become rows, rows become columns. For example,

```
Cash = [1500 4470 5280 -1299]'
```

produces

```
Cash =
```

```
1500
4470
5280
-1299
```

See Also

More About

- “Matrix Algebra Refresher” on page 1-7
- “Using Input and Output Arguments with Functions” on page 1-15

Matrix Algebra Refresher

In this section...

"Introduction" on page 1-7
 "Adding and Subtracting Matrices" on page 1-7
 "Multiplying Matrices" on page 1-8
 "Dividing Matrices" on page 1-11
 "Solving Simultaneous Linear Equations" on page 1-11
 "Operating Element by Element" on page 1-13

Introduction

The explanations in the sections that follow should help refresh your skills for using matrix algebra and using MATLAB functions.

In addition, *Macro-Investment Analysis* by William Sharpe also provides an excellent explanation of matrix algebra operations using MATLAB. It is available on the web at:

<https://www.stanford.edu/~wfsharpe/mia/mia.htm>

Tip When you are setting up a problem, it helps to "talk through" the units and dimensions associated with each input and output matrix. In the example under "Multiplying Matrices" on page 1-8, one input matrix has five days' closing prices for three stocks, the other input matrix has shares of three stocks in two portfolios, and the output matrix therefore has five days' closing values for two portfolios. It also helps to name variables using descriptive terms.

Adding and Subtracting Matrices

Matrix addition and subtraction operate element-by-element. The two input matrices must have the same dimensions. The result is a new matrix of the same dimensions where each element is the sum or difference of each corresponding input element. For example, consider combining portfolios of different quantities of the same stocks ("shares of stocks A, B, and C [the rows] in portfolios P and Q [the columns] plus shares of A, B, and C in portfolios R and S").

```
Portfolios_PQ = [100    200
                 500    400
                 300    150];
```

```
Portfolios_RS = [175    125
                 200    200
                 100    500];
```

```
NewPortfolios = Portfolios_PQ + Portfolios_RS
```

```
NewPortfolios =
```

```
    275    325
    700    600
    400    650
```

Adding or subtracting a scalar and a matrix is allowed and also operates element-by-element.

```
SmallerPortf = NewPortfolios-10
```

```
SmallerPortf =
    265.00    315.00
    690.00    590.00
    390.00    640.00
```

Multiplying Matrices

Matrix multiplication does *not* operate element-by-element. It operates according to the rules of linear algebra. In multiplying matrices, it helps to remember this key rule: the inner dimensions must be the same. That is, if the first matrix is m -by- 3 , the second must be 3 -by- n . The resulting matrix is m -by- n . It also helps to “talk through” the units of each matrix, as mentioned in “Analyze Sets of Numbers Using Matrix Functions” on page 1-4.

Matrix multiplication also is *not* commutative; that is, it is not independent of order. $A*B$ does *not* equal $B*A$. The dimension rule illustrates this property. If A is 1 -by- 3 matrix and B is 3 -by- 1 matrix, $A*B$ yields a scalar (1 -by- 1) matrix but $B*A$ yields a 3 -by- 3 matrix.

Multiplying Vectors

Vector multiplication follows the same rules and helps illustrate the principles. For example, a stock portfolio has three different stocks and their closing prices today are:

```
ClosePrices = [42.5    15    78.875]
```

The portfolio contains these numbers of shares of each stock.

```
NumShares = [100
             500
             300]
```

To find the value of the portfolio, multiply the vectors

```
PortfValue = ClosePrices * NumShares
```

which yields:

```
PortfValue =
    3.5413e+004
```

The vectors are 1 -by- 3 and 3 -by- 1 ; the resulting vector is 1 -by- 1 , a scalar. Multiplying these vectors thus means multiplying each closing price by its respective number of shares and summing the result.

To illustrate order dependence, switch the order of the vectors

```
Values = NumShares * ClosePrices
```

```
Values =
    1.0e+004 *
    0.4250    0.1500    0.7887
    2.1250    0.7500    3.9438
    1.2750    0.4500    2.3663
```

which shows the closing values of 100, 500, and 300 shares of each stock, not the portfolio value, and this is meaningless for this example.

Computing Dot Products of Vectors

In matrix algebra, if X and Y are vectors of the same length

$$Y = [y_1, y_2, \dots, y_n]$$

$$X = [x_1, x_2, \dots, x_n]$$

then the dot product

$$X \cdot Y = x_1y_1 + x_2y_2 + \dots + x_ny_n$$

is the scalar product of the two vectors. It is an exception to the commutative rule. To compute the dot product in MATLAB, use `sum(X .* Y)` or `sum(Y .* X)`. Be sure that the two vectors have the same dimensions. To illustrate, use the previous vectors.

```
Value = sum(NumShares .* ClosePrices')
```

```
Value =
```

```
3.5413e+004
```

```
Value = sum(ClosePrices .* NumShares')
```

```
Value =
```

```
3.5413e+004
```

As expected, the value in these cases matches the `PortfValue` computed previously.

Multiplying Vectors and Matrices

Multiplying vectors and matrices follows the matrix multiplication rules and process. For example, a portfolio matrix contains closing prices for a week. A second matrix (vector) contains the stock quantities in the portfolio.

```
WeekClosePr = [42.5      15      78.875
               42.125   15.5     78.75
               42.125   15.125   79
               42.625   15.25    78.875
               43       15.25    78.625];
PortQuan = [100
            500
            300];
```

To see the closing portfolio value for each day, simply multiply

```
WeekPortValue = WeekClosePr * PortQuan
```

```
WeekPortValue =
```

```
1.0e+004 *
```

```
3.5412
```

```
3.5587
```

```

3.5475
3.5550
3.5513

```

The prices matrix is 5-by-3, the quantity matrix (vector) is 3-by-1, so the resulting matrix (vector) is 5-by-1.

Multiplying Two Matrices

Matrix multiplication also follows the rules of matrix algebra. In matrix algebra notation, if A is an m -by- n matrix and B is an n -by- p matrix

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & \cdots & b_{1j} & \cdots & b_{1p} \\ b_{21} & \cdots & b_{2j} & \cdots & b_{2p} \\ \vdots & & \vdots & & \vdots \\ b_{n1} & \cdots & b_{nj} & \cdots & b_{np} \end{bmatrix}$$

then $C = A*B$ is an m -by- p matrix; and the element c_{ij} in the i th row and j th column of C is

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}.$$

To illustrate, assume that there are two portfolios of the same three stocks previously mentioned but with different quantities.

```

Portfolios = [100    200
               500    400
               300    150];

```

Multiplying the 5-by-3 week's closing prices matrix by the 3-by-2 portfolios matrix yields a 5-by-2 matrix showing each day's closing value for both portfolios.

```
PortfolioValues = WeekClosePr * Portfolios
```

```
PortfolioValues =
```

```
1.0e+004 *
```

```

3.5412    2.6331
3.5587    2.6437
3.5475    2.6325
3.5550    2.6456
3.5513    2.6494

```

Monday's values result from multiplying each Monday closing price by its respective number of shares and summing the result for the first portfolio, then doing the same for the second portfolio. Tuesday's values result from multiplying each Tuesday closing price by its respective number of shares and summing the result for the first portfolio, then doing the same for the second portfolio. And so on, through the rest of the week. With one simple command, MATLAB quickly performs many calculations.

Multiplying a Matrix by a Scalar

Multiplying a matrix by a scalar is an exception to the dimension and commutative rules. It just operates element-by-element.

```
Portfolios = [100  200
              500  400
              300  150];
```

```
DoublePort = Portfolios * 2
```

```
DoublePort =
    200    400
   1000    800
    600    300
```

Dividing Matrices

Matrix division is useful primarily for solving equations, and especially for solving simultaneous linear equations (see “Solving Simultaneous Linear Equations” on page 1-11). For example, you want to solve for X in $A*X = B$.

In ordinary algebra, you would divide both sides of the equation by A , and X would equal B/A . However, since matrix algebra is not commutative ($A*X \neq X*A$), different processes apply. In formal matrix algebra, the solution involves matrix inversion. MATLAB, however, simplifies the process by providing two matrix division symbols, left and right ($\backslash$ and $/$). In general,

$X = A \backslash B$ solves for X in $A*X = B$ and

$X = B/A$ solves for X in $X*A = B$.

In general, matrix A must be a nonsingular square matrix; that is, it must be invertible and it must have the same number of rows and columns. (Generally, a matrix is invertible if the matrix times its inverse equals the identity matrix. To understand the theory and proofs, consult a textbook on linear algebra such as *Elementary Linear Algebra* by Hill listed in “Bibliography” on page A-2.) MATLAB gives a warning message if the matrix is singular or nearly so.

Solving Simultaneous Linear Equations

Matrix division is especially useful in solving simultaneous linear equations. Consider this problem: Given two portfolios of mortgage-based instruments, each with certain yields depending on the prime rate, how do you weight the portfolios to achieve certain annual cash flows? The answer involves solving two linear equations.

A linear equation is any equation of the form

$$a_1x + a_2y = b,$$

where a_1 , a_2 , and b are constants (with a_1 and a_2 not both 0), and x and y are variables. (It is a linear equation because it describes a line in the xy -plane. For example, the equation $2x + y = 8$ describes a line such that if $x = 2$, then $y = 4$.)

A system of linear equations is a set of linear equations that you usually want to solve at the same time; that is, simultaneously. A basic principle for exact answers in solving simultaneous linear equations requires that there be as many equations as there are unknowns. To get exact answers for x and y , there must be two equations. For example, to solve for x and y in the system of linear equations

$$\begin{aligned} 2x + y &= 13 \\ x - 3y &= -18, \end{aligned}$$

there must be two equations, which there are. Matrix algebra represents this system as an equation involving three matrices: A for the left-side constants, X for the variables, and B for the right-side constants

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix}, \quad X = \begin{bmatrix} x \\ y \end{bmatrix}, \quad B = \begin{bmatrix} 13 \\ -18 \end{bmatrix},$$

where $A \cdot X = B$.

Solving the system simultaneously means solving for X . Using MATLAB,

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix};$$

$$B = \begin{bmatrix} 13 \\ -18 \end{bmatrix};$$

$$X = A \setminus B$$

solves for X in $A \cdot X = B$.

$$X = \begin{bmatrix} 3 & 7 \end{bmatrix}$$

So $x = 3$ and $y = 7$ in this example. In general, you can use matrix algebra to solve any system of linear equations such as

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

$$\vdots$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

by representing them as matrices

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}, \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

and solving for X in $A \cdot X = B$.

To illustrate, consider this situation. There are two portfolios of mortgage-based instruments, M1 and M2. They have current annual cash payments of \$100 and \$70 per unit, respectively, based on today's prime rate. If the prime rate moves down one percentage point, their payments would be \$80 and \$40. An investor holds 10 units of M1 and 20 units of M2. The investor's receipts equal cash payments times units, or $R = C \cdot U$, for each prime-rate scenario. As word equations:

	M1	M2
Prime flat:	\$100 * 10 units	+ \$70 * 20 units = \$2400 receipts
Prime down:	\$80 * 10 units	+ \$40 * 20 units = \$1600 receipts

As MATLAB matrices:

```

Cash = [100  70
        80  40];

Units = [10
         20];

Receipts = Cash * Units

Receipts =

    2400
    1600

```

Now the investor asks this question: Given these two portfolios and their characteristics, how many units of each should they hold to receive \$7000 if the prime rate stays flat and \$5000 if the prime drops one percentage point? Find the answer by solving two linear equations.

	M1	M2
Prime flat:	\$100 * x units	+ \$70 * y units = \$7000 receipts
Prime down:	\$80 * x units	+ \$40 * y units = \$5000 receipts

In other words, solve for U (units) in the equation $R(\text{receipts}) = C(\text{cash}) * U(\text{units})$. Using MATLAB left division

```

Cash = [100  70
        80  40];

Receipts = [7000
           5000];

Units = Cash \ Receipts

Units =

    43.7500
    37.5000

```

The investor should hold 43.75 units of portfolio M1 and 37.5 units of portfolio M2 to achieve the annual receipts desired.

Operating Element by Element

Finally, element-by-element arithmetic operations are called operations. To indicate a MATLAB array operation, precede the operator with a period (.). Addition and subtraction, and matrix multiplication and division by a scalar, are already array operations so no period is necessary. When using array operations on two matrices, the dimensions of the matrices must be the same. For example, given vectors of stock dividends and closing prices

```

Dividends = [1.90  0.40  1.56  4.50];
Prices = [25.625  17.75  26.125  60.50];

Yields = Dividends ./ Prices

```

Yields =

0.0741 0.0225 0.0597 0.0744

See Also

More About

- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Using Input and Output Arguments with Functions” on page 1-15

Using Input and Output Arguments with Functions

In this section...

"Input Arguments" on page 1-15

"Output Arguments" on page 1-16

Input Arguments

Vector and Matrix Input

By design, MATLAB software can efficiently perform repeated operations on collections of data stored in vectors and matrices. MATLAB code that is written to operate simultaneously on different arrays is said to be vectorized. Vectorized code is not only clean and concise, but is also efficiently processed by MATLAB.

Because MATLAB is optimized for processing vectorized code, many Financial Toolbox functions accept either vector or matrix input arguments, rather than single (scalar) values.

One example of such a function is the `irr` function, which computes the internal rate of return of a cash flow stream. If you input a vector of cash flows from a single cash flow stream, then `irr` returns a scalar rate of return. If you input a matrix of cash flows from multiple cash flow streams, where each matrix column represents a different stream, then `irr` returns a vector of internal rates of return, where the columns correspond to the columns of the input matrix. Many other Financial Toolbox functions work similarly.

As an example, suppose that you make an initial investment of \$100, from which you then receive by a series of annual cash receipts of \$10, \$20, \$30, \$40, and \$50. This cash flow stream is stored in a vector

```
CashFlows = [-100 10 20 30 40 50]'
```

```
CashFlows =
-100
  10
  20
  30
  40
  50
```

Use the `irr` function to compute the internal rate of return of the cash flow stream.

```
Rate = irr(CashFlows)
```

```
Rate =
0.1201
```

For the single cash flow stream `CashFlows`, the function returns a scalar rate of return of 0.1201, or 12.01%.

Now, use the `irr` function to compute internal rates of return for multiple cash flow streams.

```
Rate = irr([CashFlows CashFlows CashFlows])
```

```
Rate =  
    0.1201    0.1201    0.1201
```

MATLAB performs the same computation on all the assets at once. For the three cash flow streams, the `irr` function returns a vector of three internal rates of return.

In the Financial Toolbox context, vectorized programming is useful in portfolio management. You can organize multiple assets into a single collection by placing data for each asset in a different matrix column or row, then pass the matrix to a Financial Toolbox function.

Character Vector Input

Enter MATLAB character vectors surrounded by single quotes ('character vector').

A character vector is stored as a character array, one ASCII character per element. Thus, the date character vector is

```
DateCharacterVector = '9/16/2017'
```

This date character vector is actually a 1-by-9 vector. If you create a vector or matrix of character vectors, each character vector must have the same length. Using a column vector to create a vector of character vectors can allow you to visually check that all character vectors are the same length. If your character vectors are not the same length, use spaces or zeros to make them the same length, as in the following code.

```
DateFields = ['01/12/2017'  
              '02/14/2017'  
              '03/03/2017'  
              '06/14/2017'  
              '12/01/2017'];
```

`DateFields` is a 5-by-10 array of character vectors.

You cannot mix numbers and character vectors in a vector or matrix. If you input a vector or matrix that contains a mix of numbers and character vectors, MATLAB treats every entry as a character. As an example, input the following code

```
Item = [83  90  99  '14-Sep-1999']
```

```
Item =
```

```
SZc14-Sep-1999
```

The software understands the input not as a 1-by-4 vector, but as a 1-by-14 character array with the value `SZc14-Sep-1999`.

Output Arguments

Some functions return no arguments, some return just one, and some return multiple arguments. Functions that return multiple arguments use the syntax

```
[A, B, C] = function(input_arguments...)
```

to return arguments A, B, and C. If you omit all but one, the function returns the first argument. Thus, for this example if you use the syntax

```
X = function(input_arguments...)
```

the function returns a value for A, but not for B or C.

Some functions that return vectors accept only scalars as arguments. Such functions cannot accept vectors as arguments and return matrices, where each column in the output matrix corresponds to an entry in the input. Output vectors can be variable length.

For example, most functions that require asset life as an input, and return values corresponding to different periods over the asset life, cannot handle vectors or matrices as input arguments. These functions include `amortize`, `depfixdb`, `depgendb`, and `depsoyd`. For example, consider a car for which you want to compute the depreciation schedule. Use the `depfixdb` function to compute a stream of declining-balance depreciation values for the asset. Set the initial value of the asset and the lifetime of the asset. Note that in the returned vector, the asset lifetime determines the number of rows. Now consider a collection of cars with different lifetimes. Because `depfixdb` cannot output a matrix with an unequal number of rows in each column, `depfixdb` cannot accept a single input vector with values for each asset in the collection.

See Also

Related Examples

- “Matrices and Arrays”

More About

- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Matrix Algebra Refresher” on page 1-7

Performing Common Financial Tasks

- “Handle and Convert Dates” on page 2-2
- “Analyzing and Computing Cash Flows” on page 2-11
- “Pricing and Computing Yields for Fixed-Income Securities” on page 2-15
- “Treasury Bills Defined” on page 2-25
- “Computing Treasury Bill Price and Yield” on page 2-26
- “Term Structure of Interest Rates” on page 2-29
- “Returns with Negative Prices” on page 2-32
- “Pricing and Analyzing Equity Derivatives” on page 2-39
- “About Life Tables” on page 2-44
- “Case Study for Life Tables Analysis” on page 2-46
- “Machine Learning for Statistical Arbitrage: Introduction” on page 2-48
- “Machine Learning for Statistical Arbitrage I: Data Management and Visualization” on page 2-50
- “Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development” on page 2-59
- “Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction” on page 2-69
- “Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data” on page 2-78

Handle and Convert Dates

In this section...
"Date Formats" on page 2-2
"Date Conversions" on page 2-3
"Current Date and Time" on page 2-7
"Determining Specific Dates" on page 2-8
"Determining Holidays" on page 2-8
"Determining Cash-Flow Dates" on page 2-9

Date Formats

Virtually all financial data derives from a time series, functions in Financial Toolbox have extensive date-handling capabilities. The toolbox functions support date or date-and-time formats as character vectors, datetime arrays, or serial date numbers.

- Date character vectors are text that represent date and time, which you can use with multiple formats. For example, 'dd-mmm-yyyy HH:MM:SS', 'dd-mmm-yyyy', and 'mm/dd/yyyy' are all supported text formats for a date character vector. Most often, you work with date character vectors (such as 14-Sep-1999) when dealing with dates.
- Datetime arrays, created using `datetime`, are the best data type for representing points in time. `datetime` values have flexible display formats and up to nanosecond precision, and can account for time zones, daylight saving time, and leap seconds. When `datetime` objects are used as inputs to other Financial Toolbox functions, the format of the input `datetime` object is preserved. For example:

```
originalDate = datetime('now','Format','yyyy-MM-dd HH:mm:ss');  
% Find the next business day  
b = busdate(originalDate)
```

```
b =
```

```
datetime
```

```
2021-05-04 15:59:34
```

- Serial date numbers represent a calendar date as the number of days that have passed since a fixed base date. In MATLAB software, serial date number 1 is January 1,0000 A.D. Financial Toolbox works internally with serial date numbers (such as, 730377). MATLAB also uses serial time to represent fractions of days beginning at midnight. For example, 6 p.m. equals 0.75 serial days, so 6:00 p.m. on 14-Sep-1999, in MATLAB, is serial date number 730377.75

Note If you specify a two-digit year, MATLAB assumes that the year lies within the 100-year period centered on the current year. See the function `datenum` for specific information. MATLAB internal date handling and calculations generate no ambiguous values. However, whenever possible, use serial date numbers or date character vectors containing four-digit years.

Many Financial Toolbox functions that require dates as input arguments accept date character vectors, datetime arrays, or serial date numbers. If you are dealing with a few dates at the MATLAB command-line level, date character vectors are more convenient. If you are using Financial Toolbox functions on large numbers of dates, as in analyzing large portfolios or cash flows, performance improves if you use datetime arrays or serial date numbers. For more information, see “Represent Dates and Times in MATLAB”.

Date Conversions

Financial Toolbox provides functions that convert date character vectors to or from serial date numbers. In addition, you can convert character vectors or serial date numbers to datetime arrays.

Functions that convert between date formats are:

<code>datedisp</code>	Displays a numeric matrix with date entries formatted as date character vectors.
<code>datenum</code>	Converts a date character vector to a serial date number.
<code>datestr</code>	Converts a serial date number to a date character vector.
<code>datetime</code>	Converts from date character vectors or serial date numbers to create a datetime array.
<code>datevec</code>	Converts a serial date number or date character vector to a date vector whose elements are [Year Month Day Hour Minute Second].
<code>m2xdate</code>	Converts MATLAB serial date number to Excel® serial date number.
<code>x2mdate</code>	Converts Microsoft® Excel serial date number to MATLAB serial date number.

For more information, see “Convert Between Text and datetime or duration Values”.

Convert Between Datetime Arrays and Character Vectors

A date can be a character vector composed of fields related to a specific date and time. There are several ways to represent dates and times in several text formats. For example, all the following are character vectors represent August 23, 2010 at 04:35:42 PM:

```
'23-Aug-2010 04:35:06 PM'
'Wednesday, August 23'
'08/23/10 16:35'
'Aug 23 16:35:42.946'
```

A date character vector includes characters that separate the fields, such as the hyphen, space, and colon used here:

```
d = '23-Aug-2010 16:35:42'
```

Convert one or more date character vectors to a `datetime` array using the `datetime` function. For the best performance, specify the format of the input character vectors as an input to `datetime`.

Note The specifiers that `datetime` uses to describe date and time formats differ from the specifiers that the `datestr`, `datevec`, and `datenum` functions accept.

```
t = datetime(d, 'InputFormat', 'dd-MMM-yyyy HH:mm:ss')
t =
    23-Aug-2010 16:35:42
```

Although the date string, `d`, and the `datetime` scalar, `t`, look similar, they are not equal. View the size and data type of each variable.

```
whos d t
```

Name	Size	Bytes	Class	Attributes
d	1x20	40	char	
t	1x1	121	datetime	

Convert a `datetime` array to a character vector that uses `char` or `cellstr`. For example, convert the current date and time to a timestamp to append to a file name.

```
t = datetime('now', 'Format', 'yyyy-MM-dd' 'T' 'HHmmss')
t =
    datetime
    2016-12-11T125628

S = char(t);
filename = ['myTest_', S]
filename =
    'myTest_2016-12-11T125628'
```

Convert Serial Date Numbers to Datetime Arrays

Serial time can represent fractions of days beginning at midnight. For example, 6 p.m. equals 0.75 serial days, so the character vector '31-Oct-2003, 6:00 PM' in MATLAB is date number 731885.75.

Convert one or more serial date numbers to a `datetime` array using the `datetime` function. Specify the type of date number that is being converted:

```
t = datetime(731885.75, 'ConvertFrom', 'datenum')
t =
    datetime
    31-Oct-2003 18:00:00
```

Convert Datetime Arrays to Numeric Values

Some MATLAB functions accept numeric data types but not `datetime` values as inputs. To apply these functions to your date and time data, first, convert `datetime` values to meaningful numeric values, and then call the function. For example, the `log` function accepts `double` inputs but not `datetime` inputs. Suppose that you have a `datetime` array of dates spanning the course of a research study or experiment.


```
t = datetime(2014,6,18) + calmonths(1:4)
t =
    1×4 datetime array

    18-Jul-2014    18-Aug-2014    18-Sep-2014    18-Oct-2014
```

Subtract the origin value. For example, the origin value can be the starting day of an experiment.

```
dt = t - datetime(2014,7,1)
dt =
    1×4 duration array

    408:00:00    1152:00:00    1896:00:00    2616:00:00
```

`dt` is a `duration` array. Convert `dt` to a double array of values in units of years, days, hours, minutes, or seconds by using the `years`, `days`, `hours`, `minutes`, or `seconds` function, respectively.

```
x = hours(dt)
x =
    408    1152    1896    2616
```

Pass the double array as the input to the `log` function.

```
y = log(x)
y =
    6.0113    7.0493    7.5475    7.8694
```

Input Conversions with `datenum`

The `datenum` function is important for using Financial Toolbox software efficiently. `datenum` takes an input date character vector in any of several formats, with `'dd-mmm-yyyy'`, `'mm/dd/yyyy'`, or `'dd-mmm-yyyy, hh:mm:ss.ss'` formats being the most common. The input date character vector can have up to six fields formed by letters and numbers separated by any other characters, such that:

- The day field is an integer from 1 through 31.
- The month field is either an integer from 1 through 12 or an alphabetical character vector with at least three characters.
- The year field is a nonnegative integer. If only two numbers are specified, then the year is assumed to lie within the 100-year period centered on the current year. If the year is omitted, the current year is the default.
- The hours, minutes, and seconds fields are optional. They are integers separated by colons or followed by `'am'` or `'pm'`.

For example, if the current year is 1999, then all these dates are equivalent:

```
'17-May-1999'
'17-May-99'
'17-may'
'May 17, 1999'
```

```
'5/17/99'  
'5/17'
```

Also, both of these formats represent the same time.

```
'17-May-1999, 18:30'  
'5/17/99/6:30 pm'
```

The default format for numbers-only input follows the US convention. Therefore, 3/6 is March 6, not June 3.

With `datenum`, you can convert dates into serial date format, store them in a matrix variable, and then later pass the variable to a function. Alternatively, you can use `datenum` directly in a function input argument list.

For example, consider the function `bndprice` that computes the price of a bond given the yield to maturity. First set up variables for the yield to maturity, coupon rate, and the necessary dates.

```
Yield      = 0.07;  
CouponRate = 0.08;  
Settle     = datenum('17-May-2000');  
Maturity   = datenum('01-Oct-2000');
```

Then call the function with the variables.

```
bndprice(Yield,CouponRate,Settle,Maturity)
```

```
ans =  
  
100.3503
```

Alternatively, convert date character vectors to serial date numbers directly in the function input argument list.

```
bndprice(0.07,0.08,datenum('17-May-2000'),...  
datenum('01-Oct-2000'))
```

```
ans =  
  
100.3503
```

`bndprice` is an example of a function designed to detect the presence of date character vectors and make the conversion automatically. For functions like `bndprice`, date character vectors can be passed directly.

```
bndprice(0.07,0.08,'17-May-2000','01-Oct-2000')
```

```
ans =  
  
100.3503
```

The decision to represent dates as either date character vectors or serial date numbers is often a matter of convenience. For example, when formatting data for visual display or for debugging date-handling code, you can view dates more easily as date character vectors because serial date numbers are difficult to interpret. Alternately, serial date numbers are just another type of numeric data, which you can place in a matrix along with any other numeric data for convenient manipulation.

Remember that if you create a vector of input date character vectors, use a column vector, and be sure that all character vectors are the same length. To ensure that the character vectors are the same

length, fill the character vectors with spaces or zeros. For more information, see “Character Vector Input” on page 1-16.

Output Conversions with `datestr`

The `datestr` function converts a serial date number to one of 19 different date character vector output formats showing date, time, or both. The default output for dates is a day-month-year character vector, for example, 24-Aug-2000. The `datestr` function is useful for preparing output reports.

datestr Format	Description
01-Mar-2000 15:45:17	day-month-year hour:minute:second
01-Mar-2000	day-month-year
03/01/00	month/day/year
Mar	month, three letters
M	month, single letter
3	month number
03/01	month/day
1	day of month
Wed	day of week, three letters
W	day of week, single letter
2000	year, four numbers
99	year, two numbers
Mar01	month year
15:45:17	hour:minute:second
03:45:17 PM	hour:minute:second AM or PM
15:45	hour:minute
03:45 PM	hour:minute AM or PM
Q1-99	calendar quarter-year
Q1	calendar quarter

Current Date and Time

The `today` and `now` functions return serial date numbers for the current date, and the current date and time, respectively.

`today`

`ans =`

736675

`now`

`ans =`

7.3668e+05

The MATLAB function `date` returns a character vector for the current date.

```
date
ans =
    '11-Dec-2016'
```

Determining Specific Dates

Financial Toolbox provides many functions for determining specific dates. For example, assume that you schedule an accounting procedure for the last Friday of every month. Use the `lweekdate` function to return those dates for the year 2000. The input argument 6 specifies Friday.

```
Fridates = lweekdate(6,2000,1:12);
Fridays = datestr(Fridates)
```

```
Fridays =
    12x11 char array
    '28-Jan-2000'
    '25-Feb-2000'
    '31-Mar-2000'
    '28-Apr-2000'
    '26-May-2000'
    '30-Jun-2000'
    '28-Jul-2000'
    '25-Aug-2000'
    '29-Sep-2000'
    '27-Oct-2000'
    '24-Nov-2000'
    '29-Dec-2000'
```

Another example of needing specific dates could be that your company closes on Martin Luther King Jr. Day, which is the third Monday in January. You can use the `nweekdate` function to determine those specific dates for 2011 through 2014.

```
MLKDates = nweekdate(3,2,2011:2014,1);
MLKDays = datestr(MLKDates)
```

```
MLKDays =
    4x11 char array
    '17-Jan-2011'
    '16-Jan-2012'
    '21-Jan-2013'
    '20-Jan-2014'
```

Determining Holidays

Accounting for holidays and other nontrading days is important when you examine financial dates. Financial Toolbox provides the `holidays` function, which contains holidays and special nontrading days for the New York Stock Exchange from 1950 through 2030, inclusive. In addition, you can use `nyseclosures` to evaluate all known or anticipated closures of the New York Stock Exchange from

January 1, 1885, to December 31, 2050. `nyseclosures` returns a vector of serial date numbers corresponding to market closures between the dates `StartDate` and `EndDate`, inclusive.

In this example, use `holidays` to determine the standard holidays in the last half of 2012.

```
LHHDates = holidays('1-Jul-2012','31-Dec-2012');
LHHDays = datestr(LHHDates)
```

```
LHHDays =
```

```
6×11 char array
```

```
'04-Jul-2012'
'03-Sep-2012'
'29-Oct-2012'
'30-Oct-2012'
'22-Nov-2012'
'25-Dec-2012'
```

You can then use the `busdate` function to determine the next business day in 2012 after these holidays.

```
LHNextDates = busdate(LHHDates);
LHNextDays = datestr(LHNextDates)
```

```
LHNextDays =
```

```
6×11 char array
```

```
'05-Jul-2012'
'04-Sep-2012'
'31-Oct-2012'
'31-Oct-2012'
'23-Nov-2012'
'26-Dec-2012'
```

Determining Cash-Flow Dates

To determine cash-flow dates for securities with periodic payments, use `cfdates`. This function accounts for the coupons per year, the day-count basis, and the end-of-month rule. For example, you can determine the cash-flow dates for a security that pays four coupons per year on the last day of the month using an `actual/365` day-count basis. To do so, enter the settlement date, the maturity date, and the parameters for `Period`, `Basis`, and `EndMonthRule`.

```
PayDates = cfdates('14-Mar-2000','30-Nov-2001',4,3,1);
PayDays = datestr(PayDates)
```

```
PayDays =
```

```
7×11 char array
```

```
'31-May-2000'
'31-Aug-2000'
'30-Nov-2000'
'28-Feb-2001'
'31-May-2001'
'31-Aug-2001'
'30-Nov-2001'
```

See Also

`datedisp` | `datenum` | `datestr` | `datetime` | `datevec` | `format` | `date` | `holidays` | `nyseclosures` | `busdate` | `cfdates` | `addBusinessCalendar`

Related Examples

- “Convert Between Text and datetime or duration Values”
- “Read Collection or Sequence of Spreadsheet Files”
- “Trading Calendars User Interface” on page 12-2
- “UICalendar User Interface” on page 12-4

More About

- “Convert Dates Between Microsoft Excel and MATLAB” ([Spreadsheet Link](#))

External Websites

- Automated Data Cleaning and Preparation in MATLAB (43 min)

Analyzing and Computing Cash Flows

In this section...

"Introduction" on page 2-11
 "Interest Rates/Rates of Return" on page 2-11
 "Present or Future Values" on page 2-12
 "Depreciation" on page 2-13
 "Annuities" on page 2-13

Introduction

Financial Toolbox cash-flow functions compute interest rates and rates of return, present or future values, depreciation streams, and annuities.

Some examples in this section use this income stream: an initial investment of \$20,000 followed by three annual return payments, a second investment of \$5,000, then four more returns. Investments are negative cash flows, return payments are positive cash flows.

```
Stream = [-20000, 2000, 2500, 3500, -5000, 6500, ...
          9500, 9500, 9500];
```

Interest Rates/Rates of Return

This example shows how to compute the internal rate of return of the cash stream using `irr`.

Specify the income stream as an initial investment of \$20,000 followed by three annual return payments, a second investment of \$5,000, then four more returns. Investments are negative cash flows, return payments are positive cash flows.

```
Stream = [-20000, 2000, 2500, 3500, -5000, 6500, ...
          9500, 9500, 9500];
```

Use `irr` to compute the internal rate of return of the cash stream.

```
ROR = irr(Stream)
```

```
ROR =
0.1172
```

The rate of return is 11.72%.

The internal rate of return of a cash flow may not have a unique value. Every time the sign changes in a cash flow, the equation defining `irr` can give up to two additional answers. An `irr` computation requires solving a polynomial equation, and the number of real roots of such an equation can depend on the number of sign changes in the coefficients. The equation for internal rate of return is

$$\frac{cf_1}{(1+r)} + \frac{cf_2}{(1+r)^2} + \dots + \frac{cf_n}{(1+r)^n} + Investment = 0,$$

where *Investment* is a (negative) initial cash outlay at time 0, *cf_n* is the cash flow in the *n*th period, and *n* is the number of periods. `irr` finds the rate *r* such that the present value of the cash flow

equals the initial investment. If all the cf_n s are positive there is only one solution. Every time there is a change of sign between coefficients, up to two additional real roots are possible.

Another toolbox rate function, `effrr`, calculates the effective rate of return given an annual interest rate (also known as nominal rate or annual percentage rate, APR) and number of compounding periods per year. To find the effective rate of a 9% APR compounded monthly, enter

```
Rate = effrr(0.09, 12)
```

```
Rate =  
0.0938
```

The Rate is 9.38%.

A companion function `nomrr` computes the nominal rate of return given the effective annual rate and the number of compounding periods.

Present or Future Values

This example shows how to compute the present or future value of cash flows at regular or irregular time intervals with equal or unequal payments.

To compute the present or future value, you can use the following functions: `fvfix`, `fvvar`, `pvfix`, and `pvvar`. The `-fix` functions assume equal cash flows at regular intervals, while the `-var` functions allow irregular cash flows at irregular periods.

Specify the income stream as an initial investment of \$20,000 followed by three annual return payments, a second investment of \$5,000, then four more returns. Investments are negative cash flows, return payments are positive cash flows.

```
Stream = [-20000, 2000, 2500, 3500, -5000, 6500, ...  
          9500, 9500, 9500];
```

Use `irr` to compute the internal rate of return of the cash stream.

```
ROR = irr(Stream)
```

```
ROR =  
0.1172
```

Compute the net present value of the sample income stream for which you computed the internal rate of return. This exercise also serves as a check on that calculation because the net present value of a cash stream at its internal rate of return should be zero. Enter

```
NPV = pvvar(Stream, ROR)
```

```
NPV =  
5.9117e-12
```

The NPV is very close to zero. The answer usually is not *exactly* zero due to rounding errors and the computational precision of the computer. Note, other toolbox functions behave similarly. The functions that compute a bond's yield, for example, often must solve a nonlinear equation. If you then use that yield to compute the net present value of the bond's income stream, it usually does not *exactly* equal the purchase price, but the difference is negligible for practical applications.

Depreciation

This example shows how to compute standard depreciation schedules using `depgendb`.

The following code depreciates an automobile worth \$15,000 over five years with a salvage value of \$1,500. It computes the general declining balance using two different depreciation rates: 50% (or 1.5), and 100% (or 2.0, also known as double declining balance).

```
Decline1 = depgendb(15000, 1500, 5, 1.5)
```

```
Decline1 = 1×5  
103 ×
```

```
4.5000    3.1500    2.2050    1.5435    2.1015
```

```
Decline2 = depgendb(15000, 1500, 5, 2.0)
```

```
Decline2 = 1×5  
103 ×
```

```
6.0000    3.6000    2.1600    1.2960    0.4440
```

These results indicate the actual depreciation amount for the first four years and the remaining depreciable value as the entry for the fifth year.

Annuities

This example shows how to work with annuities using `annurate`.

The following code shows how to compute the interest rate associated with a series of loan payments when only the payment amounts and principal are known. For a loan whose original value was \$5000.00 and which was paid back monthly over four years at \$130.00/month:

```
Rate = annurate(4*12, 130, 5000, 0, 0)
```

```
Rate =  
0.0094
```

The function returns a rate of 0.0094 monthly, or about 11.28% annually.

You can use a present-value function (`pvinfos`) to compute the initial principal when the payment and rate are known. For a loan paid at \$300.00/month over four years at 11% annual interest:

```
Principal = pvinfos(0.11/12, 4*12, 300, 0, 0)
```

```
Principal =  
1.1607e+04
```

The function returns the original principal value of \$11,607.43.

You can compute an amortization schedule using `amortize` for a loan or annuity. For example, the original value was \$5000.00 and was paid back over 12 months at an annual rate of 9%.

```
[Prpmt, Intpmt, Balance, Payment] = amortize(0.09/12, 12, 5000, 0, 0)
```

```
Prpmt = 1×12
```

399.7574	402.7556	405.7762	408.8196	411.8857	414.9748	418.0872	421.2228	424.3820	427.5
----------	----------	----------	----------	----------	----------	----------	----------	----------	-------

```
Intpmt = 1×12
```

37.5000	34.5018	31.4812	28.4378	25.3717	22.2825	19.1702	16.0346	12.8754	9.
---------	---------	---------	---------	---------	---------	---------	---------	---------	----

```
Balance = 1×12  
103 ×
```

4.6002	4.1975	3.7917	3.3829	2.9710	2.5560	2.1379	1.7167	1.2923	0.8
--------	--------	--------	--------	--------	--------	--------	--------	--------	-----

```
Payment =  
437.2574
```

See Also

`irr` | `effrr` | `nomrr` | `fvfix` | `fvvar` | `pvfix` | `pvvar`

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Pricing and Computing Yields for Fixed-Income Securities” on page 2-15

Pricing and Computing Yields for Fixed-Income Securities

In this section...

“Introduction” on page 2-15
 “Fixed-Income Terminology” on page 2-15
 “Framework” on page 2-18
 “Default Parameter Values” on page 2-18
 “Coupon Date Calculations” on page 2-20
 “Yield Conventions” on page 2-21
 “Pricing Functions” on page 2-21
 “Yield Functions” on page 2-21
 “Fixed-Income Sensitivities” on page 2-22

Introduction

The Financial Toolbox product provides functions for computing accrued interest, price, yield, convexity, and duration of fixed-income securities. Various conventions exist for determining the details of these computations. The Financial Toolbox software supports conventions specified by the Securities Industry and Financial Markets Association (SIFMA), used in the US markets, the International Capital Market Association (ICMA), used mainly in the European markets, and the International Swaps and Derivatives Association (ISDA). For historical reasons, SIFMA is referred to in Financial Toolbox documentation as SIA and ISMA is referred to as International Capital Market Association (ICMA). Financial Instruments Toolbox™ supports additional functionality for pricing fixed-income securities. For more information, see “Price Interest-Rate Instruments” (Financial Instruments Toolbox).

Fixed-Income Terminology

Since terminology varies among texts on this subject, here are some basic definitions that apply to these Financial Toolbox functions.

The *settlement date* of a bond is the date when money first changes hands; that is, when a buyer pays for a bond. It need not coincide with the *issue date*, which is the date a bond is first offered for sale.

The *first coupon date* and *last coupon date* are the dates when the first and last coupons are paid, respectively. Although bonds typically pay periodic annual or semiannual coupons, the length of the first and last coupon periods may differ from the standard coupon period. The toolbox includes price and yield functions that handle these odd first and/or last periods.

Successive *quasi-coupon dates* determine the length of the standard coupon period for the fixed income security of interest, and do not necessarily coincide with actual coupon payment dates. The toolbox includes functions that calculate both actual and quasi-coupon dates for bonds with odd first and/or last periods.

Fixed-income securities can be purchased on dates that do not coincide with coupon payment dates. In this case, the bond owner is not entitled to the full value of the coupon for that period. When a bond is purchased between coupon dates, the buyer must compensate the seller for the pro-rata share of the coupon interest earned from the previous coupon payment date. This pro-rata share of

the coupon payment is called *accrued interest*. The *purchase price*, the price paid for a bond, is the quoted market price plus accrued interest.

The *maturity date* of a bond is the date when the issuer returns the final face value, also known as the *redemption value* or *par value*, to the buyer. The *yield-to-maturity* of a bond is the nominal compound rate of return that equates the present value of all future cash flows (coupons and principal) to the current market price of the bond.

Period

The period of a bond refers to the frequency with which the issuer of a bond makes coupon payments to the holder.

Period of a Bond

Period Value	Payment Schedule
0	No coupons (Zero coupon bond)
1	Annual
2	Semiannual
3	Tri-annual
4	Quarterly
6	Bi-monthly
12	Monthly

Basis

The basis of a bond refers to the basis or day-count convention for a bond. Day count basis determines how interest accrues over time for various instruments and the amount transferred on interest payment dates. Basis is normally expressed as a fraction in which the numerator determines the number of days between two dates, and the denominator determines the number of days in the year.

For example, the numerator of *actual/actual* means that when determining the number of days between two dates, count the actual number of days; the denominator means that you use the actual number of days in the given year in any calculations (either 365 or 366 days depending on whether the given year is a leap year). The calculation of accrued interest for dates between payments also uses day count basis. Day count basis is a fraction of **Number of interest accrual days / Days in the relevant coupon period**.

Note Although the concept of day count sounds deceptively simple, the actual calculation of day counts can be complex. You can find a good discussion of day counts and the formulas for calculating them in Chapter 5 of Stigum and Robinson, *Money Market and Bond Calculations* in “Bibliography” on page A-2. For more information on Basis, see EMU and Market Conventions: Recent Developments.

Supported day count conventions and basis values are:

Basis Value	Day Count Convention
0	actual/actual (default) — Number of days in both a period and a year is the actual number of days. Also, another common actual/actual basis is basis 12.
1	30/360 SIA — Year fraction is calculated based on a 360 day year with 30-day months, after applying the following rules: If the first date and the second date are the last day of February, the second date is changed to the 30th. If the first date falls on the 31st or is the last day of February, it is changed to the 30th. If after the preceding test, the first day is the 30th and the second day is the 31st, then the second day is changed to the 30th.
2	actual/360 — Number of days in a period is equal to the actual number of days, however the number of days in a year is 360.
3	actual/365 — Number of days in a period is equal to the actual number of days, however the number of days in a year is 365 (even in a leap year).
4	30/360 PSA — Number of days in every month is set to 30 (including February). If the start date of the period is either the 31st of a month or the last day of February, the start date is set to the 30th, while if the start date is the 30th of a month and the end date is the 31st, the end date is set to the 30th. The number of days in a year is 360.
5	30/360 ISDA — Number of days in every month is set to 30, except for February where it is the actual number of days. If the start date of the period is the 31st of a month, the start date is set to the 30th while if the start date is the 30th of a month and the end date is the 31st, the end date is set to the 30th. The number of days in a year is 360.
6	30E /360 — Number of days in every month is set to 30 except for February where it is equal to the actual number of days. If the start date or the end date of the period is the 31st of a month, that date is set to the 30th. The number of days in a year is 360.
7	actual/365 Japanese — Number of days in a period is equal to the actual number of days, except for leap days (29th February) which are ignored. The number of days in a year is 365 (even in a leap year).
8	actual/actual ICMA — Number of days in both a period and a year is the actual number of days and the compounding frequency is annual.
9	actual/360 ICMA — Number of days in a period is equal to the actual number of days, however the number of days in a year is 360 and the compounding frequency is annual.
10	actual/365 ICMA — Number of days in a period is equal to the actual number of days, however the number of days in a year is 365 (even in a leap year) and the compounding frequency is annual.
11	30/360 ICMA — Number of days in every month is set to 30, except for February where it is equal to the actual number of days. If the start date or the end date of the period is the 31st of a month, that date is set to the 30th. The number of days in a year is 360 and the compounding frequency is annual.
12	actual/365 ISDA — The day count fraction is calculated using the following formula: (Actual number of days in period that fall in a leap year / 366) + (Actual number of days in period that fall in a normal year / 365). Basis 12 is also referred to as actual/actual ISDA.
13	bus/252 — The number of days in a period is equal to the actual number of business days. The number of business days in a year is 252.

End-of-Month Rule

The *end-of-month rule* affects a bond's coupon payment structure. When the rule is in effect, a security that pays a coupon on the last actual day of a month will always pay coupons on the last day of the month. This means, for example, that a semiannual bond that pays a coupon on February 28 in nonleap years will pay coupons on August 31 in all years and on February 29 in leap years.

End-of-Month Rule

End-of-Month Rule Value	Meaning
1 (default)	Rule in effect.
0	Rule not in effect.

Framework

Although not all Financial Toolbox functions require the same input arguments, they all accept the following common set of input arguments.

Common Input Arguments

Input	Meaning
Settle	Settlement date
Maturity	Maturity date
Period	Coupon payment period
Basis	Day-count basis
EndMonthRule	End-of-month payment rule
IssueDate	Bond issue date
FirstCouponDate	First coupon payment date
LastCouponDate	Last coupon payment date

Of the common input arguments, only `Settle` and `Maturity` are required. All others are optional. They are set to the default values if you do not explicitly set them. By default, the `FirstCouponDate` and `LastCouponDate` are nonapplicable. In other words, if you do not specify `FirstCouponDate` and `LastCouponDate`, the bond is assumed to have no odd first or last coupon periods. In this case, the bond is a standard bond with a coupon payment structure based solely on the maturity date.

Default Parameter Values

To illustrate the use of default values in Financial Toolbox functions, consider the `cfdates` function, which computes actual cash flow payment dates for a portfolio of fixed income securities regardless of whether the first and/or last coupon periods are normal, long, or short.

The complete calling syntax with the full input argument list is

```
CFlowDates = cfdates(Settle, Maturity, Period, Basis, ...
    EndMonthRule, IssueDate, FirstCouponDate, LastCouponDate)
```

while the minimal calling syntax requires only settlement and maturity dates

```
CFlowDates = cfdates(Settle, Maturity)
```

Single Bond Example

As an example, suppose that you have a bond with these characteristics:

```
Settle      = '20-Sep-1999'
Maturity    = '15-Oct-2007'
Period      = 2
Basis       = 0
EndMonthRule = 1
IssueDate   = NaN
FirstCouponDate = NaN
LastCouponDate = NaN
```

Period, Basis, and EndMonthRule are set to their default values, and IssueDate, FirstCouponDate, and LastCouponDate are set to NaN.

Formally, a NaN is an IEEE® arithmetic standard for *Not-a-Number* and is used to indicate the result of an undefined operation (for example, zero divided by zero). However, NaN is also a convenient placeholder. In the SIA functions of Financial Toolbox software, NaN indicates the presence of a nonapplicable value. It tells the Financial Toolbox functions to ignore the input value and apply the default. Setting IssueDate, FirstCouponDate, and LastCouponDate to NaN in this example tells cfdates to assume that the bond has been issued before settlement and that no odd first or last coupon periods exist.

Having set these values, all these calls to cfdates produce the same result.

```
cfdates(Settle, Maturity)
cfdates(Settle, Maturity, Period)
cfdates(Settle, Maturity, Period, [])
cfdates(Settle, Maturity, [], Basis)
cfdates(Settle, Maturity, [], [])
cfdates(Settle, Maturity, Period, [], EndMonthRule)
cfdates(Settle, Maturity, Period, [], NaN)
cfdates(Settle, Maturity, Period, [], [], IssueDate)
cfdates(Settle, Maturity, Period, [], [], IssueDate, [], [])
cfdates(Settle, Maturity, Period, [], [], [], [], LastCouponDate)
cfdates(Settle, Maturity, Period, Basis, EndMonthRule, ...
IssueDate, FirstCouponDate, LastCouponDate)
```

Thus, leaving a particular input unspecified has the same effect as passing an empty matrix ([]) or passing a NaN - all three tell cfdates (and other Financial Toolbox functions) to use the default value for a particular input parameter.

Bond Portfolio Example

Since the previous example included only a single bond, there was no difference between passing an empty matrix or passing a NaN for an optional input argument. For a portfolio of bonds, however, using NaN as a placeholder is the only way to specify default acceptance for some bonds while explicitly setting nondefault values for the remaining bonds in the portfolio.

Now suppose that you have a portfolio of two bonds.

```
Settle      = '20-Sep-1999'
Maturity    = ['15-Oct-2007'; '15-Oct-2010']
```

These calls to cfdates all set the coupon period to its default value (Period = 2) for both bonds.

```
cfdates(Settle, Maturity, 2)
cfdates(Settle, Maturity, [2 2])
cfdates(Settle, Maturity, [])
cfdates(Settle, Maturity, NaN)
cfdates(Settle, Maturity, [NaN NaN])
cfdates(Settle, Maturity)
```

The first two calls explicitly set `Period = 2`. Since `Maturity` is a 2-by-1 vector of maturity dates, `cfdates` knows that you have a two-bond portfolio.

The first call specifies a single (that is, scalar) 2 for `Period`. Passing a scalar tells `cfdates` to apply the scalar-valued input to all bonds in the portfolio. This is an example of implicit scalar-expansion. The settlement date has been implicit scalar-expanded as well.

The second call also applies the default coupon period by explicitly passing a two-element vector of 2's. The third call passes an empty matrix, which `cfdates` interprets as an invalid period, for which the default value is used. The fourth call is similar, except that a `NaN` has been passed. The fifth call passes two `NaN`'s, and has the same effect as the third. The last call passes the minimal input set.

Finally, consider the following calls to `cfdates` for the same two-bond portfolio.

```
cfdates(Settle, Maturity, [4 NaN])
cfdates(Settle, Maturity, [4 2])
```

The first call explicitly sets `Period = 4` for the first bond and implicitly sets the default `Period = 2` for the second bond. The second call has the same effect as the first but explicitly sets the periodicity for both bonds.

The optional input `Period` has been used for illustrative purpose only. The default-handling process illustrated in the examples applies to any of the optional input arguments.

Coupon Date Calculations

Calculating coupon dates, either actual or quasi dates, is notoriously complicated. Financial Toolbox software follows the SIA conventions in coupon date calculations.

The first step in finding the coupon dates associated with a bond is to determine the reference, or synchronization date (the *sync date*). Within the SIA framework, the order of precedence for determining the sync date is:

- 1 The first coupon date
- 2 The last coupon date
- 3 The maturity date

In other words, a Financial Toolbox function first examines the `FirstCouponDate` input. If `FirstCouponDate` is specified, coupon payment dates and quasi-coupon dates are computed with respect to `FirstCouponDate`; if `FirstCouponDate` is unspecified, empty (`[]`), or `NaN`, then the `LastCouponDate` is examined. If `LastCouponDate` is specified, coupon payment dates and quasi-coupon dates are computed with respect to `LastCouponDate`. If both `FirstCouponDate` and `LastCouponDate` are unspecified, empty (`[]`), or `NaN`, the `Maturity` (a required input argument) serves as the synchronization date.

Yield Conventions

There are two yield and time factor conventions that are used in the Financial Toolbox software – these are determined by the input `basis`. Specifically, bases 0 to 7 are assumed to have semiannual compounding, while bases 8 to 12 are assumed to have annual compounding regardless of the period of the bond's coupon payments (including zero-coupon bonds). In addition, any yield-related sensitivity (that is, duration and convexity), when quoted on a periodic basis, follows this same convention. (See `bndconvp`, `bndconvy`, `bnddurp`, `bnddury`, and `bndkrdur`.)

Pricing Functions

This example shows how to compute the price of a bond with an odd first period using `bndprice`.

Assume that you have a bond with these characteristics:

```
Settle          = '11-Nov-1992';
Maturity        = '01-Mar-2005';
IssueDate       = '15-Oct-1992';
FirstCouponDate = '01-Mar-1993';
CouponRate      = 0.0785;
Yield           = 0.0625;
```

Allow coupon payment period (`Period = 2`), day-count basis (`Basis = 0`), and end-of-month rule (`EndMonthRule = 1`) to assume the default values. Also, assume that there is no odd last coupon date and that the face value of the bond is \$100. Calling the function:

```
[Price, AccruedInt] = bndprice(Yield, CouponRate, Settle, ...
Maturity, [], [], [], IssueDate, FirstCouponDate)
```

```
Price =
113.5977
```

```
AccruedInt =
0.5855
```

`bndprice` returns a price of \$113.60 and accrued interest of \$0.59.

Note, `bndprice` uses nonlinear formulas to compute the price of a security. For this reason, Financial Toolbox™ software uses Newton's method when solving for an independent variable within a formula.

Yield Functions

This example shows how to use `bndyield` compute the yield of a bond that has odd first and last periods and settlement in the first period.

Set up variables for settlement, maturity date, issue, first coupon, and a last coupon date.

```
Settle          = '12-Jan-2000';
Maturity        = '01-Oct-2001';
IssueDate       = '01-Jan-2000';
FirstCouponDate = '15-Jan-2000';
LastCouponDate  = '15-Apr-2000';
```

Assume a face value of \$100. Specify a purchase price of \$95.70, a coupon rate of 4%, quarterly coupon payments, and a 30/360 day-count convention (`Basis = 1`).

```
Price      = 95.7;
CouponRate = 0.04;
Period     = 4;
Basis      = 1;
EndMonthRule = 1;
```

Call the `bndyield` function.

```
Yield = bndyield(Price, CouponRate, Settle, Maturity, Period, ...
Basis, EndMonthRule, IssueDate, FirstCouponDate, LastCouponDate)
```

```
Yield =
0.0659
```

The function returns a `Yield = 0.0659` (6.60%).

Fixed-Income Sensitivities

Financial Toolbox software supports the following options for managing interest-rate risk for one or more bonds:

- `bnddurp` and `bnddury` support duration and convexity analysis based on market quotes and assume parallel shifts in the bond yield curve.
- `bndkrdur` supports key rate duration based on a market yield curve and can model nonparallel shifts in the bond yield curve.

Calculating Duration and Convexity for Bonds

This example shows how to compute the annualized Macaulay and modified durations and the periodic Macaulay duration for a bond.

The Macaulay duration of an income stream, such as a coupon bond, measures how long, on average, the owner waits before receiving a payment. It is the weighted average of the times payments are made, with the weights at time T equal to the present value of the money received at time T . The modified duration is the Macaulay duration discounted by the per-period interest rate; that is, divided by $(1 + \text{rate}/\text{frequency})$. The *Macaulay duration* is a measure of price sensitivity to yield changes. This duration is measured in years and is a weighted average-time-to-maturity of an instrument.

To illustrate, the following code computes the annualized Macaulay and modified durations and the periodic Macaulay duration for a bond with settlement (12-Jan-2000) and maturity (01-Oct-2001) dates, a 5% coupon rate, and a 4.5% yield to maturity. For simplicity, any optional input arguments assume default values (that is, semiannual coupons, and day-count basis = 0 (actual/actual), coupon payment structure synchronized to the maturity date, and end-of-month payment rule in effect).

```
CouponRate = 0.05;
Yield = 0.045;
Settle = datetime(2000,1,12);
Maturity = datetime(2001,10,1);

[ModDuration, YearDuration, PerDuration] = bnddury(Yield, ...
CouponRate, Settle, Maturity)
```

```
ModDuration =
1.6107
```

```
YearDuration =
1.6470
```

```
PerDuration =
3.2940
```

The durations are:

- ModDuration = 1.6107 (years)
- YearDuration = 1.6470 (years)
- PerDuration = 3.2940 (semiannual periods)

Note that the semiannual periodic Macaulay duration (PerDuration) is twice the annualized Macaulay duration (YearDuration).

Calculating Key Rate Durations for Bonds

This example shows how to compute the key rate duration of the US Treasury Bond.

Key rate duration enables you to evaluate the sensitivity and price of a bond to nonparallel changes in the spot or zero curve by decomposing the interest rate risk along the spot or zero curve. Key rate duration refers to the process of choosing a set of key rates and computing a duration for each rate. Specifically, for each key rate, while the other rates are held constant, the key rate is shifted up and down (and intermediate cash flow dates are interpolated), and then the present value of the security given the shifted curves is computed.

The calculation of `bndkrdur` supports

$$krdur_i = \frac{(PV_{down} - PV_{up})}{(PV \times ShiftValue \times 2)}$$

Where *PV* is the current value of the instrument, *PV_{up}* and *PV_{down}* are the new values after the discount curve has been shocked, and *ShiftValue* is the change in interest rate. For example, if key rates of 3 months, 1, 2, 3, 5, 7, 10, 15, 20, 25, 30 years were chosen, then a 30-year bond might have corresponding key rate durations of:

3M	1Y	2Y	3Y	5Y	7Y	10Y	15Y	20Y	25Y	30Y
.01	.04	.09	.21	.4	.65	1.27	1.71	1.68	1.83	7.03

The key rate durations add up to approximately equal the duration of the bond.

Compute the key rate duration of the US Treasury Bond with maturity date of August 15, 2028 and coupon rate of 5.5%.

```
Settle = datenum('18-Nov-2008');
CouponRate = 5.500/100;
Maturity = datenum('15-Aug-2028');
Price = 114.83;
```

For the ZeroData information on the current spot curve for this bond, refer to <https://www.treasury.gov/resource-center/data-chart-center/interest-rates/Pages/TextView.aspx?data=yield>.

```
ZeroDates = daysadd(Settle , [30 90 180 360 360*2 360*3 360*5 ...  
360*7 360*10 360*20 360*30]);  
ZeroRates = ([0.06 0.12 0.81 1.08 1.22 1.53 2.32 2.92 3.68 4.42 4.20]/100)';
```

Compute the key rate duration using `bndkrdur` for a specific set of rates (choose this based on the maturities of the available hedging instruments).

```
krd = bndkrdur([ZeroDates ZeroRates], CouponRate, Settle, Maturity, 'keyrates', [2 5 10 20])
```

```
krd = 1×4
```

```
0.2865    0.8729    2.6451    8.5778
```

Note, the sum of the key rate durations approximately equals the duration of the bond.

```
[sum(krd) bnddurp(Price, CouponRate, Settle, Maturity)]
```

```
ans = 1×2
```

```
12.3823    12.3919
```

See Also

`bndconvp` | `bndconvy` | `bnddurp` | `bnddury` | `bndkrdur`

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Term Structure of Interest Rates” on page 2-29
- “Computing Treasury Bill Price and Yield” on page 2-26

More About

- “Treasury Bills Defined” on page 2-25

Treasury Bills Defined

Treasury bills are short-term securities (issued with maturities of one year or less) sold by the United States Treasury. Sales of these securities are frequent, usually weekly. From time to time, the Treasury also offers longer duration securities called Treasury notes and Treasury bonds.

A Treasury bill is a discount security. The holder of the Treasury bill does not receive periodic interest payments. Instead, at the time of sale, a percentage discount is applied to the face value. At maturity, the holder redeems the bill for full face value.

The basis for Treasury bill interest calculation is actual/360. Under this system, interest accrues on the actual number of elapsed days between purchase and maturity, and each year contains 360 days.

See Also

`tbilldisc2yield` | `tbillprice` | `tbillrepo` | `tbillyield` | `tbillyield2disc` | `tbillval01` | `tbl2bond` | `tr2bonds` | `zbtprice` | `zbtyield`

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Term Structure of Interest Rates” on page 2-29
- “Computing Treasury Bill Price and Yield” on page 2-26

Computing Treasury Bill Price and Yield

In this section...

“Introduction” on page 2-26
 “Treasury Bill Repurchase Agreements” on page 2-26
 “Treasury Bill Yields” on page 2-27

Introduction

Financial Toolbox software provides the following suite of functions for computing price and yield on Treasury bills.

Treasury Bill Functions

Function	Purpose
<code>tbilldisc2yield</code>	Convert discount rate to yield.
<code>tbillprice</code>	Price Treasury bill given its yield or discount rate.
<code>tbillrepo</code>	Break-even discount of repurchase agreement.
<code>tbillyield</code>	Yield and discount of Treasury bill given its price.
<code>tbillyield2disc</code>	Convert yield to discount rate.
<code>tbillval01</code>	The value of 1 basis point (one hundredth of one percentage point, or 0.0001) given the characteristics of the Treasury bill, as represented by its settlement and maturity dates. You can relate the basis point to discount, money-market, or bond-equivalent yield.

For all functions with yield in the computation, you can specify yield as money-market or bond-equivalent yield. The functions all assume a face value of \$100 for each Treasury bill.

Treasury Bill Repurchase Agreements

This example shows how to compute the break-even discount rate. This is the rate that correctly prices the Treasury bill such that the profit from selling the bill equals 0.

```
Maturity = '26-Dec-2002';
InitialDiscount = 0.0161;
PurchaseDate = '26-Sep-2002';
SaleDate = '26-Oct-2002';
RepoRate = 0.0149;

BreakevenDiscount = tbillrepo(RepoRate, InitialDiscount, ...
PurchaseDate, SaleDate, Maturity)

BreakevenDiscount =
0.0167
```

You can check the result of this computation by examining the cash flows in and out from the repurchase transaction. First compute the price of the Treasury bill on the purchase date (September 26).

```
PriceOnPurchaseDate = tbillprice(InitialDiscount, ...
PurchaseDate, Maturity, 3)
```

```
PriceOnPurchaseDate =
99.5930
```

Compute the interest due on the repurchase agreement.

```
RepoInterest = ...
RepoRate*PriceOnPurchaseDate*days360(PurchaseDate, SaleDate)/360
```

```
RepoInterest =
0.1237
```

RepoInterest for a 1.49% 30-day term repurchase agreement (30/360 basis) is **0.1237**.

Compute the price of the Treasury bill on the sale date (October 26).

```
PriceOnSaleDate = tbillprice(BreakevenDiscount, SaleDate, ...
Maturity, 3)
```

```
PriceOnSaleDate =
99.7167
```

Examining the cash flows, observe that the break-even discount causes the sum of the price on the purchase date plus the accrued 30-day interest to be equal to the price on sale date. The following table shows the cash flows:

Cash Flows from Repurchase Agreement

Date	Cash Out Flow		Cash In Flow	
9/26/2002	Purchase T-bill	99.593	Repo money	99.593
10/26/2002	Payment of repo	99.593	Sell T-bill	99.7168
	Repo interest	0.1238		
	Total	199.3098		199.3098

Treasury Bill Yields

This example shows how to convert a Treasury bill discount to an equivalent yield.

You can examine the money-market and bond-equivalent yields of a Treasury bill at the time of purchase and sale. The function `tbilldisc2yield` can perform both computations at one time.

```
Maturity = '26-Dec-2002';
InitialDiscount = 0.0161;
PurchaseDate = '26-Sep-2002';
SaleDate = '26-Oct-2002';
RepoRate = 0.0149;
BreakevenDiscount = tbillrepo(RepoRate, InitialDiscount, ...
PurchaseDate, SaleDate, Maturity)
```

```
BreakevenDiscount =
0.0167
```

```
[BEYield, MMYield] = ...  
tbilldisc2yield([InitialDiscount; BreakevenDiscount], ...  
[PurchaseDate; SaleDate], Maturity)
```

```
BEYield = 2×1
```

```
0.0164  
0.0170
```

```
MMYield = 2×1
```

```
0.0162  
0.0168
```

For the short Treasury bill (fewer than 182 days to maturity), the money-market yield is 360/365 of the bond-equivalent yield, as this example shows.

See Also

[tbilldisc2yield](#) | [tbillprice](#) | [tbillrepo](#) | [tbillyield](#) | [tbillyield2disc](#) | [tbillval01](#) | [tbl2bond](#) | [tr2bonds](#) | [zbtprice](#) | [zbtyield](#)

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Term Structure of Interest Rates” on page 2-29

More About

- “Treasury Bills Defined” on page 2-25

Term Structure of Interest Rates

This example shows how to derive and analyze interest-rate curves, including data conversion and extrapolation, bootstrapping, and interest-rate curve conversions.

One of the first problems in analyzing the term structure of interest rates is dealing with market data reported in different formats. Treasury bills, for example, are quoted with bid and asked bank-discount rates. Treasury notes and bonds, on the other hand, are quoted with bid and asked prices based on \$100 face value. To examine the full spectrum of Treasury securities, analysts must convert data to a single format. Financial Toolbox™ functions ease this conversion. The following code uses only one security each; analysts often use 30, 100, or more of each.

First, capture Treasury bill quotes and Treasury bond quotes in their reported format.

```
%      Maturity      Days  Bid    Ask    AskYield
TBill = [datetime('12/26/2000') 53    0.0503  0.0499  0.0510];

%      Coupon  Maturity      Bid    Ask    AskYield
TBond = [0.08875  datetime(2001,11,5) 103+4/32  103+6/32  0.0564];
```

Note that these quotes are based on a November 3, 2000 settlement date.

```
Settle = datetime('3-Nov-2000');
```

Use the `tbl2bond` to convert the Treasury bill data to Treasury bond format.

```
TBTBond = tbl2bond(TBill)

TBTBond = 1×5
105 ×

      0      7.3085      0.0010      0.0010      0.0000
```

The second element of `TBTBond` is the serial date number for December 26, 2000.

Combine short-term (Treasury bill) with long-term (Treasury bond) data to set up the overall term structure.

```
TBondsAll = [TBTBond; TBond]

TBondsAll = 2×5
105 ×

      0      7.3085      0.0010      0.0010      0.0000
0.0000      7.3116      0.0010      0.0010      0.0000
```

Use `tr2bonds` to convert the bond data into a form ready for the bootstrapping functions. `tr2bonds` generates a matrix of bond information sorted by maturity date, plus vectors of prices and yields.

```
[Bonds, Prices, Yields] = tr2bonds(TBondsAll)

Bonds = 2×6
105 ×
```

```
7.3085      0      0.0010      0      0      0.0000
7.3116      0.0000      0.0010      0.0000      0      0.0000
```

```
Prices = 2×1
```

```
99.2654
103.1875
```

```
Yields = 2×1
```

```
0.0510
0.0564
```

Use a bootstrapping function to derive an implied zero curve. Bootstrapping is a process whereby you begin with known data points and solve for unknown data points using an underlying arbitrage theory. Every coupon bond can be valued as a package of zero-coupon bonds which mimic its cash flow and risk characteristics. By mapping yields-to-maturity for each theoretical zero-coupon bond, to the dates spanning the investment horizon, you can create a theoretical zero-rate curve. The Financial Toolbox™ software provides two bootstrapping functions: `zbtprice` derives a zero curve from bond data and *prices*, and `zbtyield` derives a zero curve from bond data and *yields*. Using `zbtprice`

```
[ZeroRates, CurveDates] = zbtprice(Bonds, Prices, Settle)
```

```
ZeroRates = 2×1
```

```
0.0516
0.0558
```

```
CurveDates = 2×1
```

```
730846
731160
```

`CurveDates` gives the investment horizon.

```
datestr(CurveDates)
```

```
ans = 2×11 char array
'26-Dec-2000'
'05-Nov-2001'
```

Use the functions `zero2disc`, `zero2fwd`, and `zero2pyld` to construct discount, forward, and par yield curves from the zero curve, and vice versa.

```
[DiscRates, CurveDates] = zero2disc(ZeroRates, CurveDates, Settle)
```

```
DiscRates = 2×1
```

```
0.9926
0.9462
```

```
CurveDates = 2×1
```

```
730846  
731160
```

```
[FwdRates, CurveDates] = zero2fwd(ZeroRates, CurveDates, Settle)
```

```
FwdRates = 2×1
```

```
0.0516  
0.0565
```

```
CurveDates = 2×1
```

```
730846  
731160
```

```
[PYldRates, CurveDates] = zero2pyld(ZeroRates, CurveDates, Settle)
```

```
PYldRates = 2×1
```

```
0.0522  
0.0557
```

```
CurveDates = 2×1
```

```
730846  
731160
```

See Also

tbilldisc2yield | tbillprice | tbillrepo | tbillyield | tbillyield2disc | tbillval01 |
tbl2bond | tr2bonds | zbtprice | zbtyield

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Computing Treasury Bill Price and Yield” on page 2-26

More About

- “Treasury Bills Defined” on page 2-25

Returns with Negative Prices

Once considered a mathematical impossibility, negative prices have become an established aspect of many financial markets. Negative prices arise in situations where investors determine that holding an asset entails more risk than the current value of the asset. For example, energy futures see negative prices because of costs associated with overproduction and limited storage capacity. In a different setting, central banks impose negative interest rates when national economies become deflationary, and the pricing of interest rate derivatives, traditionally based on positivity, have to be rethought (see “Work with Negative Interest Rates Using Functions” (Financial Instruments Toolbox)). A negative price encourages the buyer to take something from the seller, and the seller pays a fee for the service of divesting.

MathWorks® Computational Finance products support several functions for converting between price series $p(t)$ and return series $r(t)$. Price positivity is not a requirement. The returns computed from input negative prices can be unexpected, but they have mathematical meaning that can help you to understand price movements.

Negative Price Conversion

Financial Toolbox functions `ret2tick` and `tick2ret` support converting between price series $p(t)$ and return series $r(t)$.

For simple returns (default), the functions implement the formulas

$$\begin{aligned} r_s(t) &= \frac{p_s(t)}{p_s(t-1)} - 1 \\ p_s(t) &= p_s(t-1)(r_s(t) + 1). \end{aligned}$$

For continuous returns, the functions implement the formulas

$$\begin{aligned} r_c(t) &= \log\left(\frac{p_c(t)}{p_c(t-1)}\right) \\ p_c(t) &= p_c(t-1)e^{r_c(t)}. \end{aligned}$$

The functions `price2ret` and `ret2price` implement the same formulas, but they divide by Δt in the return formulas and they multiply by Δt in the price formulas. A positive factor of Δt (enforced by required monotonic observation times) does not affect the behavior of the functions. Econometrics Toolbox™ calls simple returns *periodic*, and continuous returns are the default. Otherwise, the functionality between the set of functions is identical. This example concentrates on the Financial Toolbox functions.

In the simple return formula, $r_s(t)$ is the percentage change (*PC*) in $p_s(t-1)$ over the interval $[t-1, t]$

$$\begin{aligned} PC &= \frac{p_s(t)}{p_s(t-1)} - 1 \\ p_s(t) &= p_s(t-1) + PC \cdot p_s(t-1). \end{aligned}$$

For positive prices, the range of *PC* is $(-1, \infty)$, that is, anything from a 100% loss ($p_s: p_s(t-1) \rightarrow 0$) to unlimited gain. The recursion in the second equation gives the subsequent prices; $p_s(t)$ is computed from $p_s(t-1)$ by adding a percentage of $p_s(t-1)$.

Furthermore, you can aggregate simple returns through time using the formula

$$\frac{p_s(T)}{p_s(1)} - 1 = \prod_{t=2}^T (r_s(t) + 1) - 1,$$

where the left-hand side represents the simple return over the entire interval $[0, T]$.

Continuous returns add 1 to PC to move the range to $(0, \infty)$, the domain of the real logarithm. Continuous returns have the time aggregation property

$$\log\left(\frac{p_c(T)}{p_c(1)} - 1\right) = \log\left(\prod_{t=2}^T \left(\frac{p_c(t)}{p_c(t-1)}\right)\right) = \sum_{t=2}^T \log\left(\frac{p_c(t)}{p_c(t-1)}\right) = \sum_{t=2}^T r_c(t).$$

This transformation ensures additivity of compound returns.

If negative prices are allowed, the range of simple returns PC expands to $(-\infty, \infty)$, that is, anything from unlimited loss to unlimited gain. In the formula for continuous returns, logarithms of negative numbers are unavoidable. The logarithm of a negative number is not a mathematical problem because the complex logarithm (the MATLAB default) interprets negative numbers as complex numbers with phase angle $\pm\pi$, so that, for example,

$$\begin{aligned} -2 &= 2e^{i\pi} \\ \log(-2) &= 2 + i\pi \end{aligned}$$

If $x < 0$, $\log(x) = \log(|x|) \pm i\pi$. The log of a negative number has an imaginary part of $\pm\pi$. The log of 0 is undefined because the range of the exponential $e^{i\theta}$ is positive. Therefore, zero prices (that is, free exchanges) are unsupported.

Analysis of Negative Price Returns

To illustrate negative price inputs, consider the following price series and its simple returns.

```
p = [3; 1; -2; -1; 1]
```

```
p =
```

```
3
1
-2
-1
1
```

```
rs = tick2ret(p)
```

```
rs =
```

```
-0.6667
-3.0000
-0.5000
-2.0000
```

This table summarizes the recursions.

$p_s(t-1)$	$p_s(t)$	$r_s(t)$
+3	+1	-0.6667
+1	-2	-3.0000
-2	-1	-0.5000
-1	+1	-2.0000

The returns have the correct size (66%, 300%, 50%, 200%), but do they have the correct sign? If you interpret negative returns as losses, as with the positive price series, the signs seem wrong—the last two returns should be gains (that is, if you interpret less negative to be a gain). However, if you interpret the negative returns by the formula

$$p_s(t) = p_s(t-1) + PC \cdot p_s(t-1),$$

which requires the signs, the last two negative percentage changes multiply *negative* prices $p_s(t-1)$, which produces *positive* additions to $p_s(t-1)$. Briefly, a negative return on a negative price produces a positive price movement. The returns are correct.

The round trip produced by `ret2tick` returns the original price series.

```
ps = ret2tick(rs,StartPrice=3)
```

```
ps =
```

```
3
1
-2
-1
1
```

Also, the following computations shows that time aggregation holds.

```
p(5)/p(1) - 1
```

```
ans =
```

```
-0.6667
```

```
prod(rs + 1) - 1
```

```
ans =
```

```
-0.6667
```

For continuous returns, negative price ratios $p_c(t)/p_c(t-1)$ are interpreted as complex numbers with phase angles $\pm\pi$, and the complex logarithm is invoked.

```
rc = tick2ret(p,Method="continuous")
```

```
rc =
```

```
-1.0986 + 0.0000i
0.6931 + 3.1416i
-0.6931 + 0.0000i
0.0000 - 3.1416i
```

This table summarizes the recursions.

$p_c(t-1)$	$p_c(t)$	$r_c(t)$
+3	+1	$-1.0986 + 0i$
+1	-2	$+0.6931 + \pi i$
-2	-1	$-0.6931 + 0i$
-1	+1	$0.0000 - \pi i$

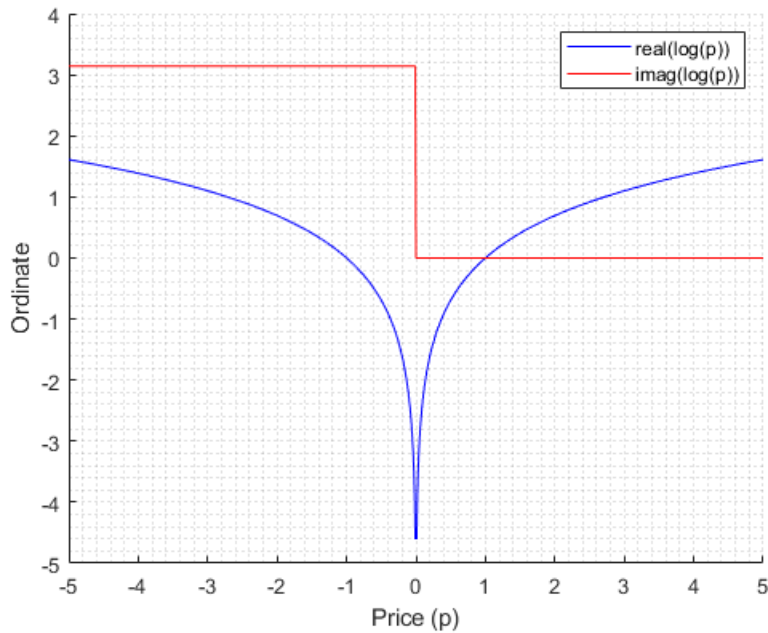
The real part shows the trend in the absolute price series. When $|p_c(t-1)| < |p_c(t)|$, that is, when prices move away from zero, $r_c(t)$ has a positive real part. When $|p_c(t-1)| > |p_c(t)|$, that is, when prices move toward zero, $r_c(t)$ has a negative real part. When $|p_c(t-1)| = |p_c(t)|$, that is, when the absolute size of the prices is unchanged, $r_c(t)$ has a zero real part. For positive price series, where the absolute series is the same as the series itself, the real part has its usual meaning.

The imaginary part shows changes of sign in the price series. When $p_c(t-1) > 0$ and $p_c(t) < 0$, that is, when prices move from investments to divestments, $r_c(t)$ has a positive imaginary part ($+\pi$). When $p_c(t-1) < 0$ and $p_c(t) > 0$, that is, when prices move from divestments to investments, $r_c(t)$ has a negative imaginary part ($-\pi$). When the sign of the prices is unchanged, $r_c(t)$ has a zero imaginary part. For positive price series, changes of sign are irrelevant, and the imaginary part conveys no information (0).

Visualization of Complex Returns

Complex continuous returns contain a lot of information. Visualizing the information can help you to interpret the complex returns. The following code plots the real and imaginary parts of the logarithm on either side of zero.

```
p = -5:0.01:5;
hold on
plot(p,real(log(p)),"b")
plot(p,imag(log(p)),"r")
xticks(-5:5)
xlabel("Price (p)")
ylabel("Ordinate")
legend(["real(log(p))" "imag(log(p))"],AutoUpdate=false)
grid minor
```



Due to the following identity

$$\begin{aligned}
 r_c(t) &= \log\left(\frac{p_c(t)}{p_c(t-1)}\right) \\
 &= \log(p_c(t)) - \log(p_c(t-1)) \\
 &= \underbrace{[\text{real}(\log(p_c(t)) - \text{real}(\log(p_c(t-1))))]}_{\text{blue curve}} + \underbrace{[i(\text{imag}(\log(p_c(t))) - \text{imag}(\log(p_c(t-1))))]}_{\text{red curve}} \cdot i,
 \end{aligned}$$

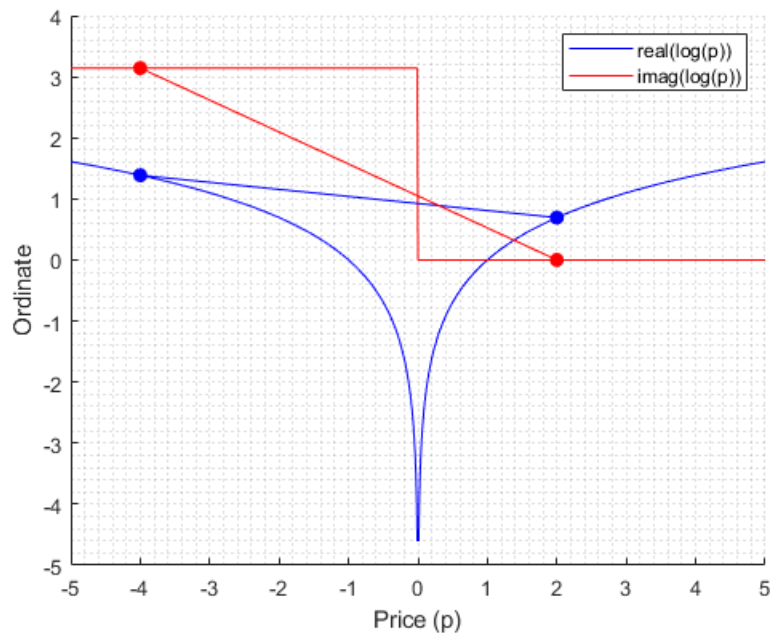
you can read the real part of a continuous return as a difference in ordinates on the blue graph, and you can read the imaginary part as a difference in ordinates on the red graph. Absolute price movements toward zero result in a negative real part and absolute price movements away from zero result in a positive real part. Likewise, changes of sign result in a jump of $\pm\pi$ in the imaginary part, with the sign change depending on the direction of the move.

For example, the plot below superimposes the real and imaginary parts of the logarithm at prices $p = -4$ and $p = 2$, with lines to help visualize their differences.

```

p = [-4; 2];
plot(p, real(log(p)), "bo-", MarkerFaceColor="b")
plot(p, imag(log(p)), "ro-", MarkerFaceColor="r")
hold off

```

If $p_c(t-1) = -4$ and $p_c(t) = 2$, the real part of $\log(p_c(t)) - \log(p_c(t-1))$ is negative (line slopes down), corresponding to a decrease in absolute price. The imaginary part is $0 - \pi = -\pi$, corresponding to a change of sign from negative to positive. If the direction of the price movement is reversed, so that $p_c(t-1) = 2$ and $p_c(t) = -4$, the positive difference in the real part corresponds to an increase in absolute price, and the positive difference in the imaginary part corresponds to a change of sign from positive to negative.

If you convert the continuous returns $r_c(t) = \log(p_c(t)/p_c(t-1))$ to simple returns $r_s = (p_s(t)/p_s(t-1) - 1)$ by the following computation, the result is the same simple returns series as before.

```
rs = exp(rc) - 1
```

```
rs =
```

```
-0.6667 + 0.0000i
-3.0000 + 0.0000i
-0.5000 + 0.0000i
-2.0000 - 0.0000i
```

You can complete the round trip, which results in the expected price series, by the computation

```
pc = ret2tick(rc,Method="continuous",StartPrice=3)
```

```
pc =
```

```
3.0000 + 0.0000i
1.0000 + 0.0000i
-2.0000 + 0.0000i
-1.0000 + 0.0000i
1.0000 + 0.0000i
```

Conclusion

Complex continuous returns are a necessary intermediary when considering logarithms of negative price ratios. `tick2ret` computes a continuous complex extension of the function on the positive real axis. The logarithm maintains the additivity property, used when computing multiperiod returns.

Because of the extensible logarithm implemented in MATLAB, current implementations of Computational Finance tools that accept prices and returns behave logically with negative prices. The interpretation of complex-valued results can be unfamiliar at first, but as shown, the results are meaningful and explicable.

See Also

`tick2ret` | `ret2tick`

More About

- “Work with Negative Interest Rates Using Functions” (Financial Instruments Toolbox)

Pricing and Analyzing Equity Derivatives

In this section...

"Introduction" on page 2-39

"Sensitivity Measures" on page 2-39

"Analysis Models" on page 2-40

Introduction

These toolbox functions compute prices, sensitivities, and profits for portfolios of options or other equity derivatives. They use the Black-Scholes model for European options and the binomial model for American options. Such measures are useful for managing portfolios and for executing collars, hedges, and straddles:

- A collar is an interest-rate option that guarantees that the rate on a floating-rate loan will not exceed a certain upper level nor fall below a lower level. It is designed to protect an investor against wide fluctuations in interest rates.
- A hedge is a securities transaction that reduces or offsets the risk on an existing investment position.
- A straddle is a strategy used in trading options or futures. It involves simultaneously purchasing put and call options with the same exercise price and expiration date, and it is most profitable when the price of the underlying security is very volatile.

Sensitivity Measures

There are six basic sensitivity measures associated with option pricing: delta, gamma, lambda, rho, theta, and vega — the "greeks." The toolbox provides functions for calculating each sensitivity and for implied volatility.

Delta

Delta of a derivative security is the rate of change of its price relative to the price of the underlying asset. It is the first derivative of the curve that relates the price of the derivative to the price of the underlying security. When delta is large, the price of the derivative is sensitive to small changes in the price of the underlying security.

Gamma

Gamma of a derivative security is the rate of change of delta relative to the price of the underlying asset; that is, the second derivative of the option price relative to the security price. When gamma is small, the change in delta is small. This sensitivity measure is important for deciding how much to adjust a hedge position.

Lambda

Lambda, also known as the elasticity of an option, represents the percentage change in the price of an option relative to a 1% change in the price of the underlying security.

Rho

Rho is the rate of change in option price relative to the risk-free interest rate.

Theta

Theta is the rate of change in the price of a derivative security relative to time. Theta is usually small or negative since the value of an option tends to drop as it approaches maturity.

Vega

Vega is the rate of change in the price of a derivative security relative to the volatility of the underlying security. When vega is large the security is sensitive to small changes in volatility. For example, options traders often must decide whether to buy an option to hedge against vega or gamma. The hedge selected usually depends upon how frequently one rebalances a hedge position and also upon the standard deviation of the price of the underlying asset (the volatility). If the standard deviation is changing rapidly, balancing against vega is preferable.

Implied Volatility

The implied volatility of an option is the standard deviation that makes an option price equal to the market price. It helps determine a market estimate for the future volatility of a stock and provides the input volatility (when needed) to the other Black-Scholes functions.

Analysis Models

Toolbox functions for analyzing equity derivatives use the Black-Scholes model for European options and the binomial model for American options. The Black-Scholes model makes several assumptions about the underlying securities and their behavior. The Black-Scholes model was the first complete mathematical model for pricing options, developed by Fischer Black and Myron Scholes. It examines market price, strike price, volatility, time to expiration, and interest rates. It is limited to only certain kinds of options.

The binomial model, on the other hand, makes far fewer assumptions about the processes underlying an option. A binomial model is a method of pricing options or other equity derivatives in which the probability over time of each possible price follows a binomial distribution. The basic assumption is that prices can move to only two values (one higher and one lower) over any short time period. For further explanation, see *Options, Futures, and Other Derivatives* by John Hull in “Bibliography” on page A-2.

Black-Scholes Model

This example shows how to compute the call and put prices of a European option and its delta, gamma, lambda, and implied volatility.

Using the Black-Scholes model entails several assumptions:

- The prices of the underlying asset follow an Ito process. (See “Derivatives Pricing and Yields” on page A-3, page 222.)
- The option can be exercised only on its expiration date (European option).
- Short selling is permitted.
- There are no transaction costs.
- All securities are divisible.
- There is no riskless arbitrage (where *arbitrage* is the purchase of securities on one market for immediate resale on another market to profit from a price or currency discrepancy).

- Trading is a continuous process.
- The risk-free interest rate is constant and remains the same for all maturities.

If any of these assumptions is untrue, Black-Scholes may not be an appropriate model.

To illustrate toolbox Black-Scholes functions (`blsprice`, `blsdelta`, `blsgamma`, `blsvega`, and `blslambda`) this example computes the call and put prices of a European option and its delta, gamma, lambda, and implied volatility. The asset price is \$100.00, the exercise price is \$95.00, the risk-free interest rate is 10%, the time to maturity is 0.25 years, the volatility is 0.50, and the dividend rate is 0.

```
[OptCall, OptPut] = blsprice(100, 95, 0.10, 0.25, 0.50, 0)
```

```
OptCall =  
13.6953
```

```
OptPut =  
6.3497
```

```
[CallVal, PutVal] = blsdelta(100, 95, 0.10, 0.25, 0.50, 0)
```

```
CallVal =  
0.6665
```

```
PutVal =  
-0.3335
```

```
GammaVal = blsgamma(100, 95, 0.10, 0.25, 0.50, 0)
```

```
GammaVal =  
0.0145
```

```
VegaVal = blsvega(100, 95, 0.10, 0.25, 0.50, 0)
```

```
VegaVal =  
18.1843
```

```
[LamCall, LamPut] = blslambda(100, 95, 0.10, 0.25, 0.50, 0)
```

```
LamCall =  
4.8664
```

```
LamPut =  
-5.2528
```

To summarize:

- The option call price `OptCall` = \$13.70
- The option put price `OptPut` = \$6.35
- delta for a call `CallVal` = 0.6665 and delta for a put `PutVal` = -0.3335
- gamma `GammaVal` = 0.0145
- vega `VegaVal` = 18.1843
- lambda for a call `LamCall` = 4.8664 and lambda for a put `LamPut` = -5.2528

As a computation check, find the implied volatility of the option using the call option price from `blsprice`.

```
Volatility = blsimpv(100, 95, 0.10, 0.25, OptCall)
```

```
Volatility =  
0.5000
```

The function returns an implied volatility of 0.500, the original `blsprice` input.

Binomial Model

This example shows how to price an American call option using a binomial model.

The binomial model for pricing options or other equity derivatives assumes that the probability over time of each possible price follows a binomial distribution. The basic assumption is that prices can move to only two values, one up and one down, over any short time period. Plotting the two values, and then the subsequent two values each, and then the subsequent two values each, and so on over time, is known as "building a binomial tree." This model applies to American options, which can be exercised any time up to and including their expiration date.

This example prices an American call option using a binomial model. The asset price is \$100.00, the exercise price is \$95.00, the risk-free interest rate is 10%, and the time to maturity is 0.25 years. The function `binprice` computes the tree in increments of 0.05 years, so there are $0.25/0.05 = 5$ periods in the example. The volatility is 0.50, this is a call (`flag = 1`), the dividend rate is 0, and it pays a dividend of \$5.00 after three periods (an ex-dividend date).

```
[StockPrice, OptionPrice] = binprice(100, 95, 0.10, 0.25, ...  
0.05, 0.50, 1, 0, 5.0, 3)
```

```
StockPrice = 6×6
```

100.0000	111.2713	123.8732	137.9629	148.6915	166.2807
0	89.9677	100.0495	111.3211	118.8981	132.9629
0	0	80.9994	90.0175	95.0744	106.3211
0	0	0	72.9825	76.0243	85.0175
0	0	0	0	60.7913	67.9825
0	0	0	0	0	54.3608

```
OptionPrice = 6×6
```

12.1011	19.1708	29.3470	42.9629	54.1653	71.2807
0	5.3068	9.4081	16.3211	24.3719	37.9629
0	0	1.3481	2.7402	5.5698	11.3211
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

The output from the binomial function is a binary tree. Read the `StockPrice` matrix this way: column 1 shows the price for period 0, column 2 shows the up and down prices for period 1, column 3 shows the up-up, up-down, and down-down prices for period 2, and so on. Ignore the zeros. The `OptionPrice` matrix gives the associated option value for each node in the price tree. Ignore the zeros that correspond to a zero in the price tree.

See Also

`blsprice` | `binprice` | `blkimpv` | `blkprice` | `blsdelta` | `blsgamma` | `blsimpv` | `blslambda` | `blsrho` | `blstheta` | `blsvega` | `opprofit`

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Greek-Neutral Portfolios of European Stock Options” on page 10-14
- “Plotting Sensitivities of an Option” on page 10-25
- “Plotting Sensitivities of a Portfolio of Options” on page 10-27

About Life Tables

Life tables are used for life insurance and work with the probability distribution of human mortality. This distribution, which is age-dependent, has several characteristic features that are consequences of biological, cultural, and behavioral factors. Usually, the practitioners of life studies use life tables that contain age-dependent series for specific demographics. The tables are in a standard format with standard notation that is specific to the life studies field. An example of a life table is shown in Table 1 from CDC life tables for the United States.

Table 1. Life table for the total population: United States, 2009

Age (years)	Probability of dying between ages x and $x + 1$	Number surviving to age x	Number dying between ages x and $x + 1$	Person-years lived between ages x and $x + 1$	Total number of person-years lived above age x	Expectation of life at age x
	q_x	l_x	d_x	L_x	T_x	e_x
0-1	0.006372	100,000	637	99,444	7,946,926	78.5
1-2	0.000407	99,363	40	99,343	7,747,481	78.0
2-3	0.000274	99,322	27	99,309	7,548,139	77.0
3-4	0.000209	99,295	21	99,285	7,348,830	76.0
4-5	0.000160	99,274	16	99,266	7,149,545	75.0
5-6	0.000150	99,259	15	99,251	6,950,279	74.1
6-7	0.000135	99,244	13	99,237	6,751,028	73.1
7-8	0.000122	99,230	12	99,224	6,551,791	72.1
8-9	0.000109	99,215	11	99,213	6,352,566	71.1
9-10	0.000095	99,207	9	99,203	6,153,354	70.1
10-11	0.000087	99,198	9	99,194	5,954,151	69.1
11-12	0.000083	99,189	9	99,185	5,754,957	68.1
12-13	0.000127	99,180	13	99,174	5,555,773	67.1
13-14	0.000160	99,167	16	99,158	5,356,599	66.1
14-15	0.000279	99,148	28	99,134	5,157,441	65.1
15-16	0.000370	99,121	37	99,102	4,958,307	64.1
16-17	0.000454	99,084	45	99,061	4,759,205	63.2
17-18	0.000537	99,039	53	99,012	4,560,143	62.2
18-19	0.000615	98,986	61	98,955	4,361,131	61.2
19-20	0.000691	98,925	68	98,891	4,162,175	60.3

Often, these life tables can have numerous variations such as abridged tables (which pose challenges due to the granularity of the data) and different termination criteria (that can make it difficult to compare tables or to compute life expectancies).

Most raw life tables have one or more of the first three series in this table (q_x , l_x , and d_x) and the notation for these three series is standard in the field.

- The q_x series is basically the discrete hazard function for human mortality.
- The l_x series is the survival function multiplied by a radix of 100,000.
- The d_x series is the discrete probability density for the distribution as a function of age.

Financial Toolbox can handle arbitrary life table data supporting several standard models of mortality and provides various interpolation methods to calibrate and analyze the life table data.

Although primarily designed for life insurance applications, the life tables functions (`lifetableconv`, `lifetablefit`, and `lifetablegen`) can also be used by social scientists, behavioral psychologists, public health officials, and medical researchers.

Life Tables Theory

Life tables are based on hazard functions and survival functions which are, in turn, derived from probability distributions. Specifically, given a continuous probability distribution, its cumulative distribution function is $F(x)$ and its probability density function is $f(x) = dF(x)/dx$.

For the analysis of mortality, the random variable of interest X is the distribution of ages at which individuals die within a population. So, the probability that someone dies by age x is

$$\Pr[X \leq x] = F(x)$$

The survival function, $(s(x))$, which characterizes the probability that an individual lives beyond a specified age $x > 0$, is

$$\begin{aligned}s(x) &= \Pr[X > x] \\ &= 1 - F(x)\end{aligned}$$

For a continuous probability distribution, the hazard function is a function of the survival function with

$$\begin{aligned}h(x) &= \lim_{\Delta x \rightarrow 0} \frac{\Pr[x \leq X < x + \Delta x | X \geq x]}{\Delta x} \\ &= -\frac{1}{s(x)} \frac{d(s(x))}{dx}\end{aligned}$$

and the survival functions is a function of the hazard function with

$$s(x) = \exp\left(-\int_0^x h(\xi) d\xi\right)$$

Life table models generally specify either the hazard function or the survival function. However, life tables are discrete and work with discrete versions of the hazard and survival functions. Three series are used for life tables and the notation is the convention. The discrete hazard function is denoted as

$$\begin{aligned}q_x &\approx h(x) \\ &= 1 - \frac{s(x+1)}{s(x)}\end{aligned}$$

which is the probability a person at age x dies by age $x + 1$ (where x is in years). The discrete survival function is presented in terms of an initial number of survivors at birth called the life table radix (which is usually 100,000 individuals) and is denoted as

$$l_x = l_0 s(x)$$

with radix $l_0 = 100000$. This number, l_x , represents the number of individuals out of 100,000 at birth who are still alive at age x .

A third series is related to the probability density function which is the number of "standardized" deaths in a given year denoted as

$$d_x = l_x - l_{x+1}$$

Based on a few additional rules about how to initialize and terminate these series, any one series can be derived from any of the other series.

See Also

[lifetableconv](#) | [lifetablefit](#) | [lifetablegen](#)

Related Examples

- "Case Study for Life Tables Analysis" on page 2-46

Case Study for Life Tables Analysis

This example shows how to use the basic workflow for life tables.

Load the life table data file.

```
load us_lifetable_2009
```

Calibrate life table from survival data with the default heligman-pollard parametric model.

```
a = lifetablefit(x, lx);
```

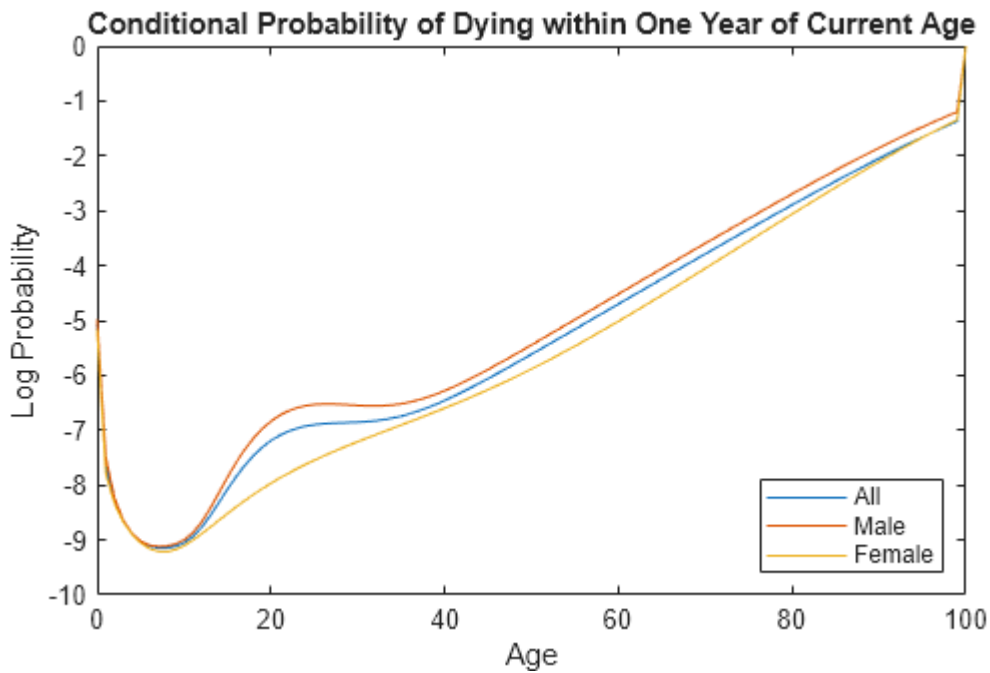
Generate life table series from the calibrated mortality model.

```
qx = lifetablegen((0:100), a);  
display(qx(1:40,:))
```

0.0063	0.0069	0.0057
0.0005	0.0006	0.0004
0.0002	0.0003	0.0002
0.0002	0.0002	0.0002
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0002	0.0002	0.0001
0.0002	0.0002	0.0002
0.0002	0.0003	0.0002
0.0003	0.0004	0.0002
0.0004	0.0005	0.0002
0.0005	0.0006	0.0003
0.0006	0.0008	0.0003
0.0007	0.0009	0.0003
0.0008	0.0011	0.0003
0.0008	0.0012	0.0004
0.0009	0.0013	0.0004
0.0009	0.0014	0.0005
0.0010	0.0014	0.0005
0.0010	0.0015	0.0005
0.0010	0.0015	0.0006
0.0010	0.0015	0.0006
0.0010	0.0015	0.0007
0.0010	0.0014	0.0007
0.0011	0.0014	0.0007
0.0011	0.0014	0.0008
0.0011	0.0014	0.0008
0.0011	0.0014	0.0009
0.0011	0.0014	0.0009
0.0012	0.0015	0.0010
0.0012	0.0015	0.0011
0.0013	0.0016	0.0011
0.0014	0.0017	0.0012
0.0015	0.0018	0.0013

Plot the q_x series and display the legend. The series q_x is the conditional probability that a person at age x will die between age x and the next age in the series

```
plot((0:100), log(qx));
legend(series, 'location', 'southeast');
title('Conditional Probability of Dying within One Year of Current Age');
xlabel('Age');
ylabel('Log Probability');
```



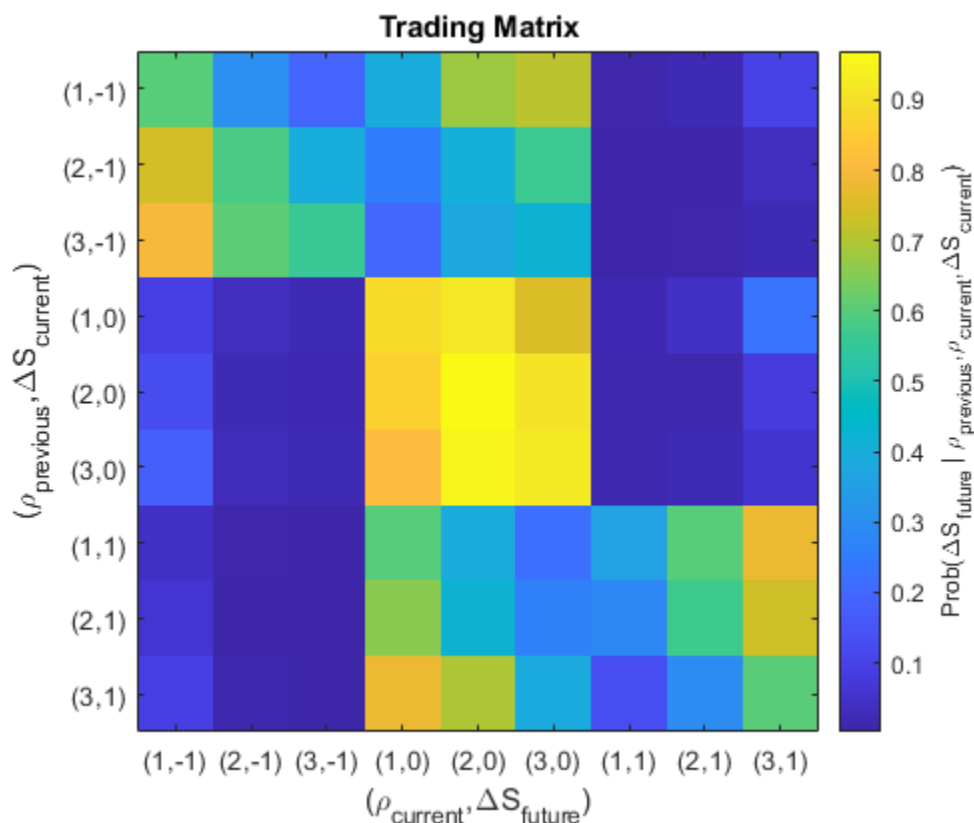
See Also

[lifetableconv](#) | [lifetablefit](#) | [lifetablegen](#)

More About

- "About Life Tables" on page 2-44

Machine Learning for Statistical Arbitrage: Introduction



Machine learning techniques for processing large amounts of data are broadly applicable in computational finance. The series of examples introduced in this topic provides a general workflow, illustrating how capabilities in MATLAB apply to a specific problem in financial engineering. The workflow is problem-oriented, exploratory, and guided by the data and the resulting analysis. The overall approach, however, is useful for constructing applications in many areas.

The workflow consists of these actions:

- Formulate a simple approach to algorithmic trading, through an analysis of market microstructure, with the goal of identifying real-time arbitrage opportunities.
- Use a large sample of exchange data to track order dynamics of a single security on a single day, selectively processing the data to develop relevant statistical measures.
- Create a model of intraday dynamics conditioned on a selection of hyperparameters introduced during feature engineering and development.
- Evaluate hyperparameter tunings using a supervising objective that computes cash returned on a model-based trading strategy.
- Optimize the trading strategy using different machine learning algorithms.
- Suggest modifications for further development.

The workflow is separated into three examples:

- 1** “Machine Learning for Statistical Arbitrage I: Data Management and Visualization” on page 2-50
- 2** “Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development” on page 2-59
- 3** “Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction” on page 2-69

For more information about general workflows for machine learning, see:

- “Machine Learning in MATLAB”
- “Supervised Learning Workflow and Algorithms”

Machine Learning for Statistical Arbitrage I: Data Management and Visualization

This example shows techniques for managing, processing, and visualizing large amounts of financial data in MATLAB®. It is part of a series of related examples on machine learning for statistical arbitrage (see “Machine Learning Applications”).

Working with Big Data

Financial markets, with electronic exchanges such as NASDAQ executing orders on a timescale of milliseconds, generate vast amounts of data. Data streams can be mined for statistical arbitrage opportunities, but traditional methods for processing and storing dynamic analytic information can be overwhelmed by big data. Fortunately, new computational approaches have emerged, and MATLAB has an array of tools for implementing them.

Main computer memory provides high-speed access but limited capacity, whereas external storage offers low-speed access but potentially unlimited capacity. Computation takes place in memory. The computer recalls data and results from external storage.

Data Files

This example uses one trading day of NASDAQ exchange data [2] on one security (INTC) in a sample provided by LOBSTER [1] and included with Financial Toolbox™ documentation in the zip file `LOBSTER_SampleFile_INTC_2012-06-21_5.zip`. Extract the contents of the zip file into your current folder. The expanded files, including two CSV files of data and the text file `LOBSTER_SampleFiles_ReadMe.txt`, consume 93.7 MB of memory.

```
unzip("LOBSTER_SampleFile_INTC_2012-06-21_5.zip");
```

The data describes the intraday evolution of the *limit order book* (LOB), which is the record of *market orders* (best price), *limit orders* (designated price), and resulting buys and sells. The data includes the precise time of these events, with orders tracked from arrival until cancellation or execution. At each moment in the trading day, orders on both the buy and sell side of the LOB exist at various *levels* away from the midprice between the lowest ask (order to sell) and the highest bid (order to buy).

Level 5 data (five levels away from the midprice on either side) is contained in two CSV files. Extract the trading date from the message file name.

```
MSGFileName = "INTC_2012-06-21_34200000_57600000_message_5.csv"; % Message file (description of  
LOBFileName = "INTC_2012-06-21_34200000_57600000_orderbook_5.csv"; % Data file  
  
[ticker,rem] = strtok(MSGFileName,'_');  
date = strtok(rem,'_');
```

Data Storage

Daily data streams accumulate and need to be stored. A *datastore* is a repository for collections of data that are too big to fit in memory.

Use `tabularTextDatastore` to create datastores for the message and data files. Because the files contain data with different formats, create the datastores separately. Ignore generic column headers (for example, `VarName1`) by setting the `'ReadVariableNames'` name-value argument to `false`. Replace the headers with descriptive variable names obtained from

LOBSTER_SampleFiles_ReadMe.txt. Set the 'ReadSize' name-value argument to 'file' to allow similarly formatted files to be appended to existing datastores at the end of each trading day.

```
DSMSG = tabularTextDatastore(MSGFileName, 'ReadVariableNames', false, 'ReadSize', 'file');
DSMSG.VariableNames = ["Time", "Type", "OrderID", "Size", "Price", "Direction"];
```

```
DSL0B = tabularTextDatastore(LOBFileName, 'ReadVariableNames', false, 'ReadSize', 'file');
DSL0B.VariableNames = ["AskPrice1", "AskSize1", "BidPrice1", "BidSize1", ...
    "AskPrice2", "AskSize2", "BidPrice2", "BidSize2", ...
    "AskPrice3", "AskSize3", "BidPrice3", "BidSize3", ...
    "AskPrice4", "AskSize4", "BidPrice4", "BidSize4", ...
    "AskPrice5", "AskSize5", "BidPrice5", "BidSize5"];
```

Create a combined datastore by selecting Time and the level 3 data.

```
TimeVariable = "Time";
DSMSG.SelectedVariableNames = TimeVariable;

LOB3Variables = ["AskPrice1", "AskSize1", "BidPrice1", "BidSize1", ...
    "AskPrice2", "AskSize2", "BidPrice2", "BidSize2", ...
    "AskPrice3", "AskSize3", "BidPrice3", "BidSize3"];
DSL0B.SelectedVariableNames = LOB3Variables;

DS = combine(DSMSG, DSL0B);
```

You can preview the first few rows in the combined datastore without loading data into memory.

```
DSPreview = preview(DS);
LOBPreview = DSPreview(:, 1:5)
```

```
LOBPreview=8x5 table
    Time    AskPrice1    AskSize1    BidPrice1    BidSize1
    _____    _____    _____    _____    _____
    34200    2.752e+05         66    2.751e+05         400
    34200    2.752e+05        166    2.751e+05         400
    34200    2.752e+05        166    2.751e+05         400
    34200    2.752e+05        166    2.751e+05         400
    34200    2.752e+05        166    2.751e+05         300
    34200    2.752e+05        166    2.751e+05         300
    34200    2.752e+05        166    2.751e+05         300
    34200    2.752e+05        166    2.751e+05         300
```

The preview shows asks and bids *at the touch*, meaning the level 1 data, which is closest to the midprice. Time units are seconds after midnight, price units are dollar amounts times 10,000, and size units are the number of shares (see LOBSTER_SampleFiles_ReadMe.txt).

Tall Arrays and Timetables

Tall arrays work with out-of-memory data backed by a datastore using the MapReduce technique (see “Tall Arrays for Out-of-Memory Data”). When you use MapReduce, tall arrays remain unevaluated until you execute specific computations that use the data.

Set the execution environment for MapReduce to the local MATLAB session, instead of using Parallel Computing Toolbox™, by calling `mapreducer(0)`. Then, create a tall array from the datastore DS by using `tall`. Preview the data in the tall array.

```
mapreducer(0)
DT = tall(DS);
```

```
DTPreview = DT(:,1:5)
```

```
DTPreview =
```

M×5 tall table

Time	AskPrice1	AskSize1	BidPrice1	BidSize1
34200	2.752e+05	66	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300
:	:	:	:	:
:	:	:	:	:

Timetables allow you to perform operations specific to time series (see “Create Timetables”). Because the LOB data consists of concurrent time series, convert DT to a tall timetable.

```
DT.Time = seconds(DT.Time); % Cast time as a duration from midnight.
DTT = table2timetable(DT);
```

```
DTTPreview = DTT(:,1:4)
```

```
DTTPreview =
```

M×4 tall timetable

Time	AskPrice1	AskSize1	BidPrice1	BidSize1
34200 sec	2.752e+05	66	2.751e+05	400
34200 sec	2.752e+05	166	2.751e+05	400
34200 sec	2.752e+05	166	2.751e+05	400
34200 sec	2.752e+05	166	2.751e+05	400
34200 sec	2.752e+05	166	2.751e+05	300
34200 sec	2.752e+05	166	2.751e+05	300
34200 sec	2.752e+05	166	2.751e+05	300
34200 sec	2.752e+05	166	2.751e+05	300
:	:	:	:	:
:	:	:	:	:

Display all variables in the MATLAB workspace.

```
whos
```

Name	Size	Bytes	Class	Attributes
DS	1x1	8	matlab.io.datastore.CombinedDatastore	
DSL0B	1x1	8	matlab.io.datastore.TabularTextDatastore	
DSMSG	1x1	8	matlab.io.datastore.TabularTextDatastore	
DSPreview	8x13	4899	table	

DT	Mx13	5292	tall
DTPreview	Mx5	2926	tall
DTT	Mx12	5056	tall
DTTPreview	Mx4	2704	tall
LOB3Variables	1x12	952	string
LOBFileName	1x1	262	string
LOBPreview	8x5	2331	table
MSGFileName	1x1	246	string
TimeVariable	1x1	166	string
date	1x1	182	string
rem	1x1	246	string
ticker	1x1	166	string

Because all the data is in the datastore, the workspace uses little memory.

Preprocess and Evaluate Data

Tall arrays allow preprocessing, or *queuing*, of computations before they are evaluated, which improves memory management in the workspace.

Midprice S and imbalance index I are used to model LOB dynamics. To queue their computations, define them, and the time base, in terms of DTT.

```
timeBase = DTT.Time;
MidPrice = (DTT.BidPrice1 + DTT.AskPrice1)/2;

% LOB level 3 imbalance index:

lambda = 0.5; % Hyperparameter
weights = exp(-(lambda)*[0 1 2]);
VAsk = weights(1)*DTT.AskSize1 + weights(2)*DTT.AskSize2 + weights(3)*DTT.AskSize3;
VBid = weights(1)*DTT.BidSize1 + weights(2)*DTT.BidSize2 + weights(3)*DTT.BidSize3;
ImbalanceIndex = (VBid-VAsk)./(VBid+VAsk);
```

The imbalance index is a weighted average of ask and bid volumes on either side of the midprice [3]. The imbalance index is a potential indicator of future price movements. The variable `lambda` is a *hyperparameter*, which is a parameter specified before training rather than estimated by the machine learning algorithm. A hyperparameter can influence the performance of the model. *Feature engineering* is the process of choosing domain-specific hyperparameters to use in machine learning algorithms. You can tune hyperparameters to optimize a trading strategy.

To bring preprocessed expressions into memory and evaluate them, use the `gather` function. This process is called *deferred evaluation*.

```
[t,S,I] = gather(timeBase,MidPrice,ImbalanceIndex);
```

```
Evaluating tall expression using the Local MATLAB Session:
- Pass 1 of 1: Completed in 4.5 sec
Evaluation completed in 5 sec
```

A single call to `gather` evaluates multiple preprocessed expressions with a single pass through the datastore.

Determine the sample size, which is the number of *ticks*, or updates, in the data.

```
numTicks = length(t)
```

```
numTicks =
581030
```

The daily LOB data contains 581,030 ticks.

Checkpoint Data

You can save both unevaluated and evaluated data to external storage for later use.

Prepend the time base with the date, and cast the result as a datetime array. Save the resulting datetime array, `MidPrice`, and `ImbalanceIndex` to a MAT-file in a specified location.

```
dateTimeBase = datetime(date) + timeBase;
Today = timetable(dateTimeBase, MidPrice, ImbalanceIndex)
```

Today =

581,030×2 tall timetable

dateTimeBase	MidPrice	ImbalanceIndex
21-Jun-2012 09:30:00	2.7515e+05	-0.205
21-Jun-2012 09:30:00	2.7515e+05	-0.26006
21-Jun-2012 09:30:00	2.7515e+05	-0.26006
21-Jun-2012 09:30:00	2.7515e+05	-0.086772
21-Jun-2012 09:30:00	2.7515e+05	-0.15581
21-Jun-2012 09:30:00	2.7515e+05	-0.35382
21-Jun-2012 09:30:00	2.7515e+05	-0.19084
21-Jun-2012 09:30:00	2.7515e+05	-0.19084
:	:	:
:	:	:

```
location = fullfile(pwd, "ExchangeData", ticker, date);
write(location, Today, 'FileType', 'mat')
```

```
Writing tall data to folder C:\TEMP\tp5b508469\finance-ex97702880\ExchangeData\INTC\2012-06-21
Evaluating tall expression using the Local MATLAB Session:
- Pass 1 of 1: Completed in 5.8 sec
Evaluation completed in 6.7 sec
```

The file is written once, at the end of each trading day. The code saves the data to a file in a date-stamped folder. The series of `ExchangeData` subfolders serves as a historical data repository.

Alternatively, you can save workspace variables evaluated with `gather` directly to a MAT-file in the current folder.

```
save("LOBVars.mat", "t", "S", "I")
```

In preparation for model validation later on, evaluate and add market order prices to the same file.

```
[MOBid, MOAsk] = gather(DTT.BidPrice1, DTT.AskPrice1);
```

```
Evaluating tall expression using the Local MATLAB Session:
- Pass 1 of 1: Completed in 4.1 sec
Evaluation completed in 4.2 sec
```

```
save("LOBVars.mat", "MOBid", "MOAsk", "-append")
```

The remainder of this example uses only the unevaluated tall timetable DTT. Clear other variables from the workspace.

```
clearvars -except DTT
whos
```

Name	Size	Bytes	Class	Attributes
DTT	581,030x12	5056	tall	

Data Visualization

To visualize large amounts of data, you must summarize, bin, or sample the data in some way to reduce the number of points plotted on the screen.

LOB Snapshot

One method of visualization is to evaluate only a selected subsample of the data. Create a snapshot of the LOB at a specific time of day (11 AM).

```
sampleTimeTarget = seconds(11*60*60); % Seconds after midnight
sampleTimes = withtol(sampleTimeTarget,seconds(1)); % 1 second tolerance
sampleLOB = DTT(sampleTimes,:);
```

```
numTimes = gather(size(sampleLOB,1))
```

Evaluating tall expression using the Local MATLAB Session:

- Pass 1 of 1: Completed in 4.1 sec

Evaluation completed in 4.4 sec

```
numTimes =
23
```

There are 23 ticks within one second of 11 AM. For the snapshot, use the tick closest to the midtime.

```
sampleLOB = sampleLOB(round(numTimes/2),:);
sampleTime = sampleLOB.Time;
```

```
sampleBidPrices = [sampleLOB.BidPrice1,sampleLOB.BidPrice2,sampleLOB.BidPrice3];
sampleBidSizes = [sampleLOB.BidSize1,sampleLOB.BidSize2,sampleLOB.BidSize3];
sampleAskPrices = [sampleLOB.AskPrice1,sampleLOB.AskPrice2,sampleLOB.AskPrice3];
sampleAskSizes = [sampleLOB.AskSize1,sampleLOB.AskSize2,sampleLOB.AskSize3];
```

```
[sampleTime,sampleBidPrices,sampleBidSizes,sampleAskPrices,sampleAskSizes] = ...
gather(sampleTime,sampleBidPrices,sampleBidSizes,sampleAskPrices,sampleAskSizes);
```

Evaluating tall expression using the Local MATLAB Session:

- Pass 1 of 2: Completed in 3.7 sec

- Pass 2 of 2: Completed in 4 sec

Evaluation completed in 8.6 sec

Visualize the limited data sample returned by gather by using bar.

```
figure
hold on

bar((sampleBidPrices/10000),sampleBidSizes,'r')
bar((sampleAskPrices/10000),sampleAskSizes,'g')
hold off
```

```

xlabel("Price (Dollars)")
ylabel("Number of Shares")
legend(["Bid", "Ask"], 'Location', 'North')
title(strcat("Level 3 Limit Order Book: ", datestr(sampleTime, "HH:MM:SS")))

```



Depth of Market

Some visualization functions work directly with tall arrays and do not require the use of `gather` (see “Visualization of Tall Arrays”). The functions automatically sample data to decrease pixel density. Visualize the level 3 intraday *depth of market*, which shows the time evolution of liquidity, by using `plot` with the tall timetable `DTT`.

```

figure
hold on

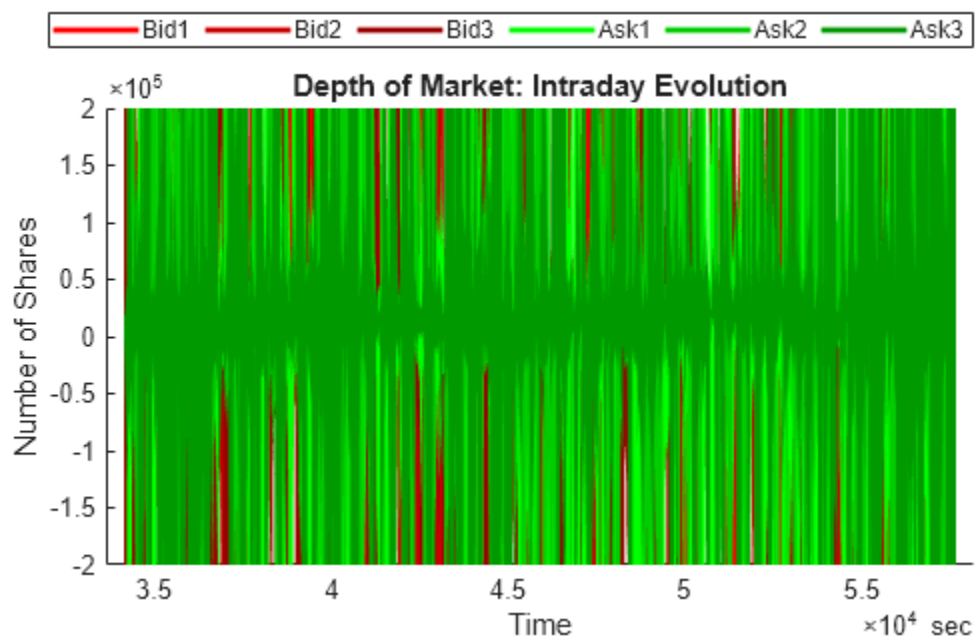
plot(DTT.Time, -DTT.BidSize1, 'Color', [1.0 0 0], 'LineWidth', 2)
plot(DTT.Time, -DTT.BidSize2, 'Color', [0.8 0 0], 'LineWidth', 2)
plot(DTT.Time, -DTT.BidSize3, 'Color', [0.6 0 0], 'LineWidth', 2)

plot(DTT.Time, DTT.AskSize1, 'Color', [0 1.0 0], 'LineWidth', 2)
plot(DTT.Time, DTT.AskSize2, 'Color', [0 0.8 0], 'LineWidth', 2)
plot(DTT.Time, DTT.AskSize3, 'Color', [0 0.6 0], 'LineWidth', 2)

hold off

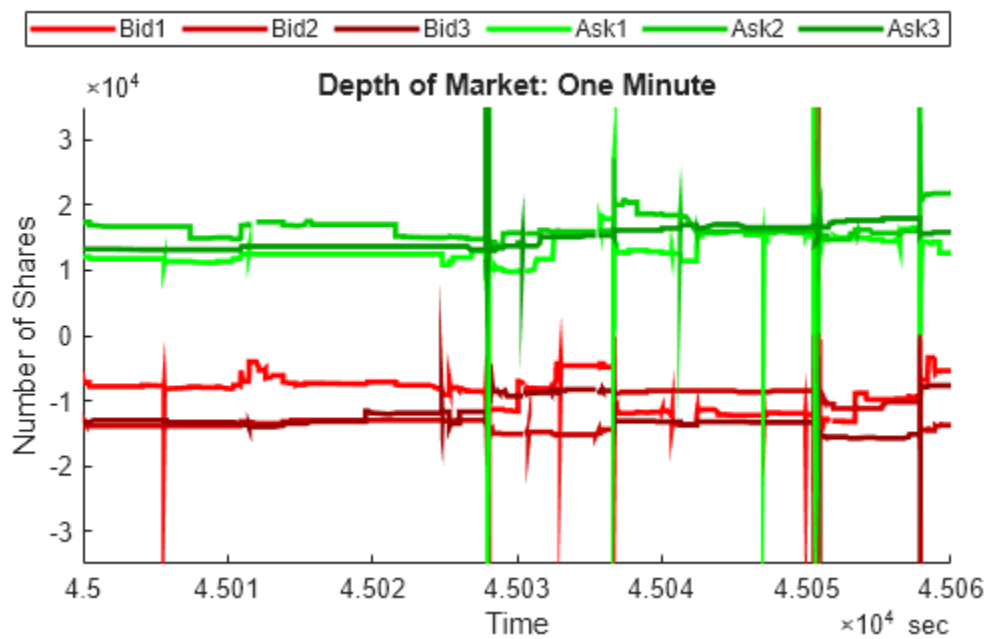
xlabel("Time")
ylabel("Number of Shares")
title("Depth of Market: Intraday Evolution")
legend(["Bid1", "Bid2", "Bid3", "Ask1", "Ask2", "Ask3"], 'Location', 'NorthOutside', 'Orientation', 'Horizontal')

```



To display details, limit the time interval.

```
xlim(seconds([45000 45060]))
ylim([-35000 35000])
title("Depth of Market: One Minute")
```



Summary

This example introduces the basics of working with big data, both in and out of memory. It shows how to set up, combine, and update external datastores, then create tall arrays for preprocessing data without allocating variables in the MATLAB workspace. The `gather` function transfers data into the workspace for computation and further analysis. The example shows how to visualize the data through data sampling or by MATLAB plotting functions that work directly with out-of-memory data.

References

- [1] LOBSTER Limit Order Book Data. Berlin: frishedaten UG (haftungsbeschränkt).
- [2] NASDAQ Historical TotalView-ITCH Data. New York: The Nasdaq, Inc.
- [3] Rubisov, Anton D. "Statistical Arbitrage Using Limit Order Book Imbalance." Master's thesis, University of Toronto, 2015.

See Also

More About

- "Machine Learning for Statistical Arbitrage: Introduction" on page 2-48
- "Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development" on page 2-59
- "Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction" on page 2-69

Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development

This example creates a continuous-time Markov model of limit order book (LOB) dynamics, and develops a strategy for algorithmic trading based on patterns observed in the data. It is part of a series of related examples on machine learning for statistical arbitrage (see “Machine Learning Applications”).

Exploratory Data Analysis

To predict the future behavior of a system, you need to discover patterns in historical data. The vast amount of data available from exchanges, such as NASDAQ, poses computational challenges while offering statistical opportunities. This example explores LOB data by looking for indicators of price momentum, following the approach in [4].

Raw Data

Load `LOBVars.mat`, the preprocessed LOB data set of the NASDAQ security INTC, which is included with the Financial Toolbox™ documentation.

```
load LOBVars
```

The data set contains the following information for each order: the arrival time `t` (seconds from midnight), level 1 asking price `MOAsk`, level 1 bidding price `MOBid`, midprice `S`, and imbalance index `I`.

Create a plot that shows the intraday evolution of the LOB imbalance index `I` and midprice `S`.

```
figure

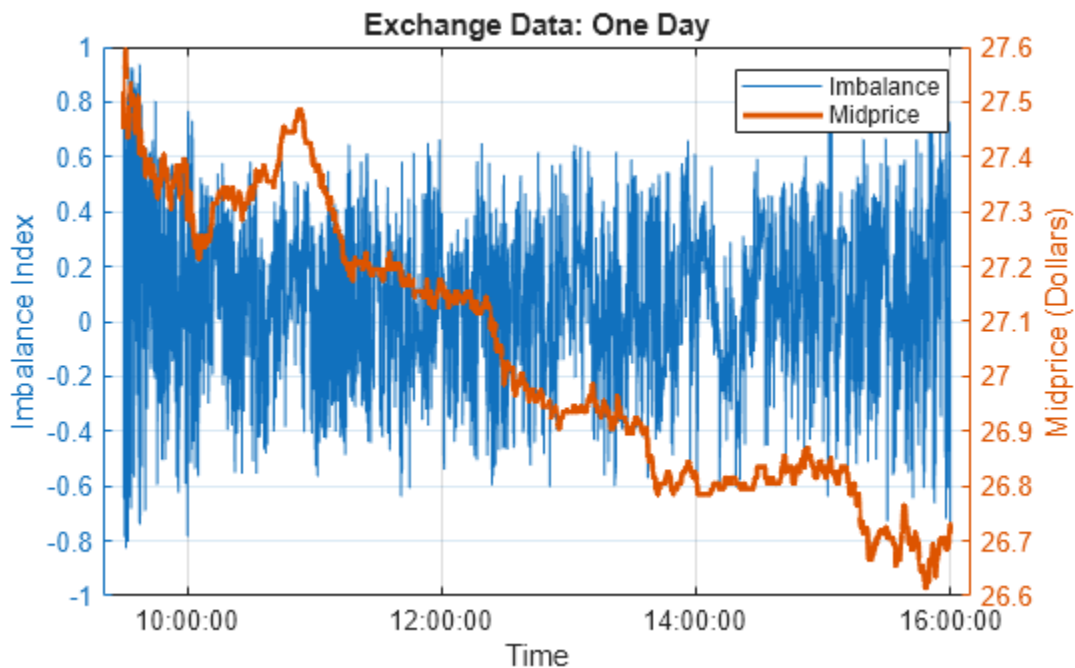
t.Format = "hh:mm:ss";

yyaxis left
plot(t,I)
ylabel("Imbalance Index")

yyaxis right
plot(t,S/10000,'LineWidth',2)
ylabel("Midprice (Dollars)")

xlabel("Time")

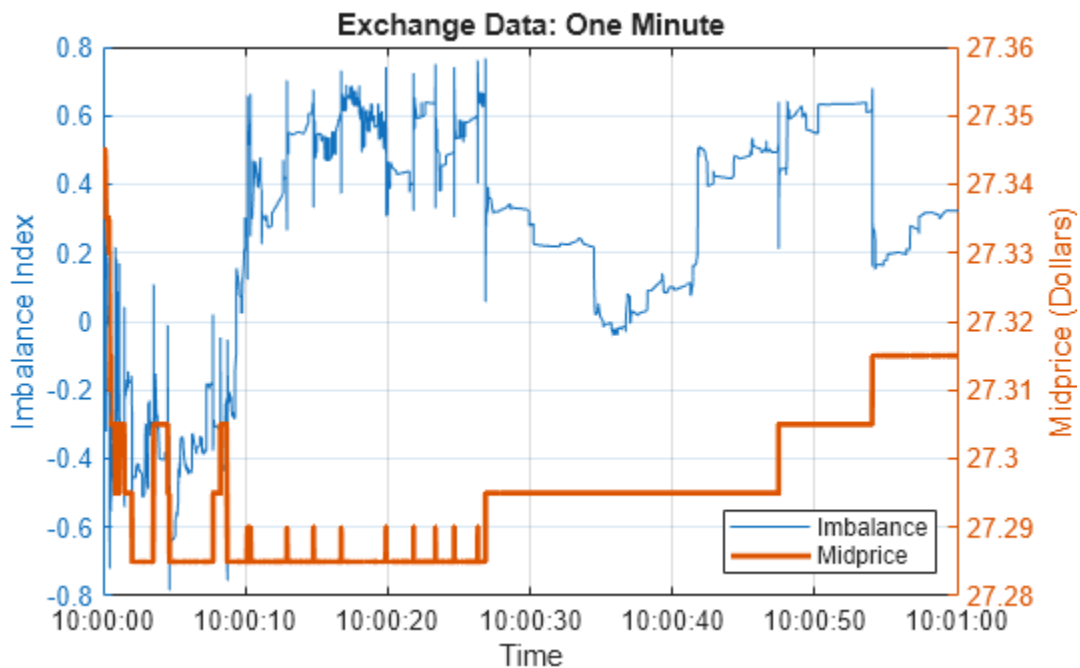
title('Exchange Data: One Day')
legend(["Imbalance","Midprice"],'Location','NE')
grid on
```



At this scale, the imbalance index gives no indication of future changes in the midprice.

To see more detail, limit the time scale to one minute.

```
timeRange = seconds([36000 36060]); % One minute after 10 AM, when prices were climbing
xlim(timeRange)
legend('Location','SE')
title("Exchange Data: One Minute")
```

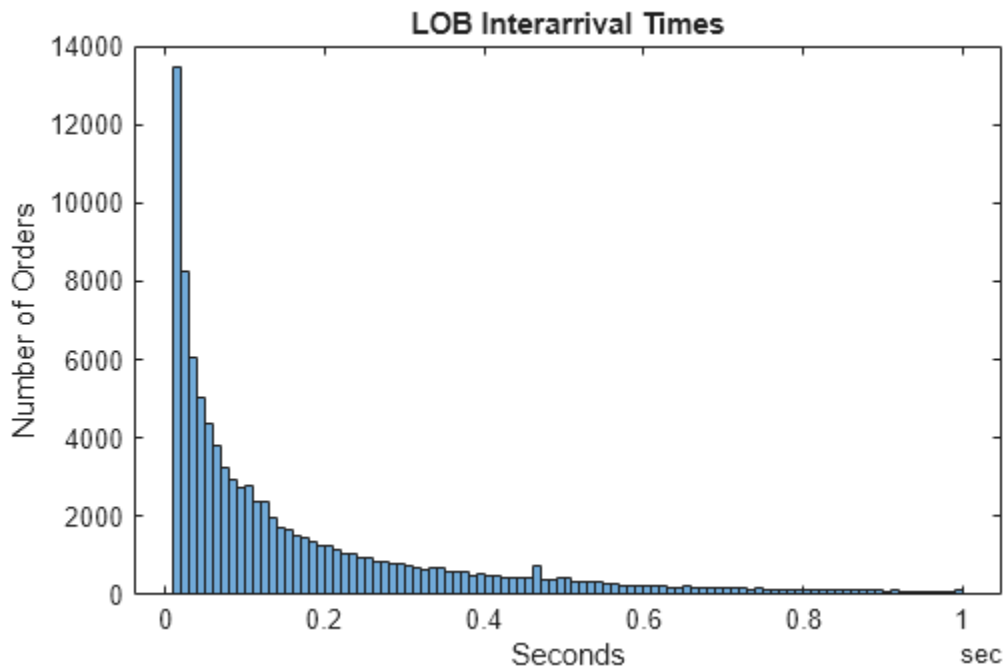


At this scale, sharp departures in the imbalance index align with corresponding departures in the midprice. If the relationship is predictive, meaning imbalances of a certain size forecast *future* price movements, then quantifying the relationship can provide statistical arbitrage opportunities.

Plot a histogram of the interarrival times in the LOB.

```
DT = diff(t); % Interarrival Times
DT.Format = "s";
```

```
figure
binEdges = seconds(0.01:0.01:1);
histogram(DT,binEdges)
xlabel("Seconds")
ylabel("Number of Orders")
title("LOB Interarrival Times")
```



Interarrival times follow the characteristic pattern of a Poisson process.

Compute the average wait time between orders by fitting an exponential distribution to the interarrival times.

```
DTAvg = expfit(DT)
```

```
DTAvg = duration
0.040273 sec
```

Smoothed Data

The raw imbalance series I is erratic. To identify the most significant dynamic shifts, introduce a degree of smoothing dI , which is the number of backward ticks used to average the raw imbalance series.

```
dI = 10; % Hyperparameter
dTI = dI*DTAvg
```

```
dTI = duration
0.40273 sec
```

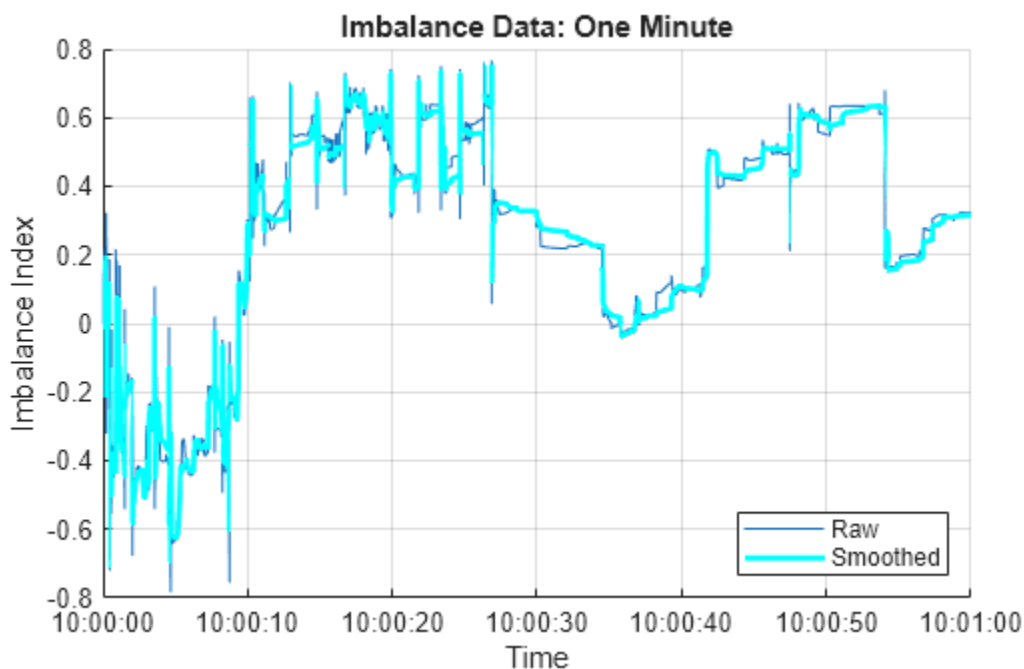
The setting corresponds to an interval of 10 ticks, or about 0.4 seconds on average. Smooth the imbalance indices over a trailing window.

```
sI = smoothdata(I, 'movmean', [dI 0]);
```

Visualize the degree of smoothing to assess the volatility lost or retained.

```
figure
hold on
plot(t,I)
plot(t,sI,'c','LineWidth',2)
hold off

xlabel("Time")
xlim(timeRange)
ylabel("Imbalance Index")
title("Imbalance Data: One Minute")
legend(["Raw", "Smoothed"], 'Location', 'SE')
grid on
```



Discretized Data

To create a Markov model of the dynamics, collect the smoothed imbalance index sI into bins, discretizing it into a finite collection of states ρ (ρ). The number of bins `numBins` is a hyperparameter.

```
numBins = 3; % Hyperparameter
binEdges = linspace(-1,1,numBins+1);
rho = discretize(sI,binEdges);
```

To model forecast performance, aggregate prices over a leading window. The number of ticks in a window dS is a hyperparameter.

```
dS = 20; % Hyperparameter
dTS = dS*DTAvg
```

```
dTS = duration
      0.80547 sec
```

The setting corresponds to an interval of 20 ticks, or about 0.8 seconds on average. Discretize price movements into three states DS (ΔS) given by the sign of the forward price change.

```
DS = NaN(size(S));
shiftS = S(dS+1:end);
DS(1:end-dS) = sign(shiftS-S(1:end-dS));
```

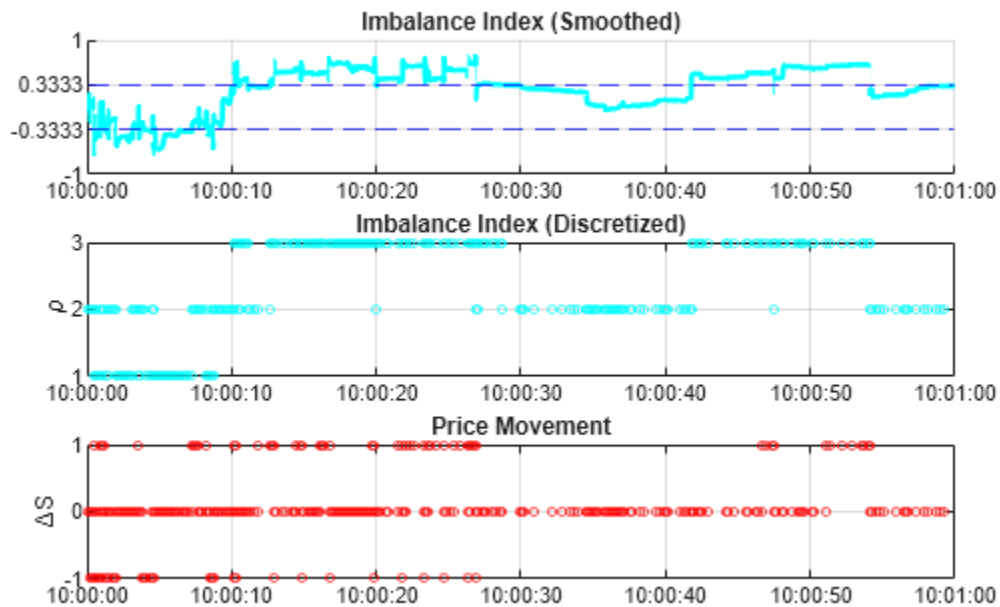
Visualize the discretized data.

figure

```
subplot(3,1,1)
hold on
plot(t,sI,'c','LineWidth',2)
for i = 2:numBins
    yline(binEdges(i),'b--');
end
hold off
xlim(timeRange)
ylim([-1 1])
yticks(binEdges)
title("Imbalance Index (Smoothed)")
grid on

subplot(3,1,2)
plot(t,rho,'co','MarkerSize',3)
xlim(timeRange)
ylim([1 numBins])
yticks(1:numBins)
ylabel("\rho")
title("Imbalance Index (Discretized)")
grid on

subplot(3,1,3)
plot(t,DS,'ro','MarkerSize',3)
xlim(timeRange)
ylim([-1 1])
yticks([-1 0 1])
ylabel("\Delta S")
title("Price Movement")
grid on
```



Continuous Time Markov Process

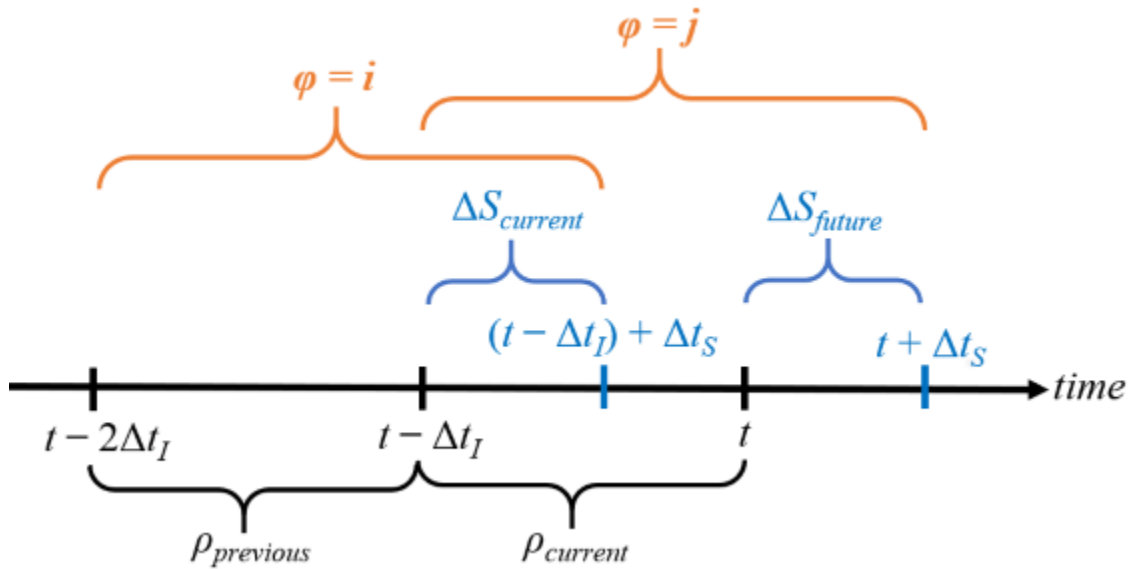
Together, the state of the LOB imbalance index ρ (ρ) and the state of the forward price movement DS (ΔS) describe a two-dimensional continuous-time Markov chain (CTMC). The chain is modulated by the Poisson process of order arrivals, which signals any transition among the states.

To simplify the description, give the two-dimensional CTMC a one-dimensional encoding into states ϕ ($\phi = (\rho, \Delta S)$).

```
numStates = 3*numBins; % numStates(DS)*numStates(rho)

phi = NaN(size(t));
for i = 1:length(t)
    switch DS(i)
        case -1
            phi(i) = rho(i);
        case 0
            phi(i) = rho(i) + numBins;
        case 1
            phi(i) = rho(i) + 2*numBins;
    end
end
```

Successive states of ϕ , and the component states ρ and ΔS , proceed as follows.



Hyperparameters dI (Δt_I) and dS (Δt_S) determine the size of a rolling state characterizing the dynamics. At time t , the process transitions from $\varphi = (\rho_{previous}, \Delta S_{current}) = i$ to $\varphi = (\rho_{current}, \Delta S_{future}) = j$ (or holds in the same state if $i = j$).

Estimate Process Parameters

Execution of the trading strategy at any time t is based on the probability of ΔS_{future} being in a particular state, conditional on the current and previous values of the other states. Following [3] and [4], determine empirical transition probabilities, and then assess them for predictive power.

% Transition counts

```
C = zeros(numStates);
for i = 1:length(phi)-dS-1
    C(phi(i),phi(i+1)) = C(phi(i),phi(i+1))+1;
end
```

% Holding times

```
H = diag(C);
```

% Transition rate matrix (infinitesimal generator)

```
G = C./H;
v = sum(G,2);
G = G + diag(-v);
```

% Transition probability matrix (stochastic for all dI)

```
P = expm(G*dI); % Matrix exponential
```

To obtain a trading matrix Q containing $\text{Prob}(\Delta S_{future} | \rho_{previous}, \rho_{current}, \Delta S_{current})$ as in [4], apply Bayes' rule,

$$\text{Prob}(\Delta S_{\text{future}} | \rho_{\text{previous}}, \rho_{\text{current}}, \Delta S_{\text{current}}) = \frac{\text{Prob}(\rho_{\text{current}}, \Delta S_{\text{future}} | \rho_{\text{previous}}, \Delta S_{\text{current}})}{\text{Prob}(\rho_{\text{current}} | \rho_{\text{previous}}, \Delta S_{\text{current}})}.$$

The numerator is the transition probability matrix P . Compute the denominator P_{Cond} .

```
PCond = zeros(size(P));
phiNums = 1:numStates;
modNums = mod(phiNums,numBins);
for i = phiNums
    for j = phiNums
        idx = (modNums == modNums(j));
        PCond(i,j) = sum(P(i,idx));
    end
end

Q = P./PCond;
```

Display Q in a table. Label the rows and columns with composite states $\varphi = (\rho, \Delta S)$.

```
binNames = string(1:numBins);
stateNames = ["("+binNames+",-1)","("+binNames+",0)","("+binNames+",1)"];
QTable = array2table(Q,'RowNames',stateNames,'VariableNames',stateNames)
```

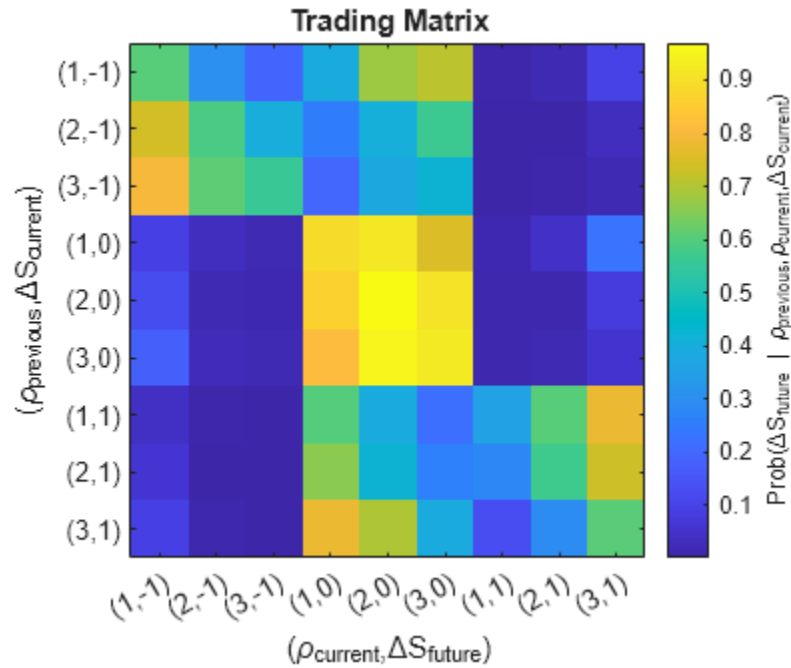
QTable=9×9 table

	(1,-1)	(2,-1)	(3,-1)	(1,0)	(2,0)	(3,0)	(1,1)
(1,-1)	0.59952	0.30458	0.19165	0.39343	0.67723	0.7099	0.0070457
(2,-1)	0.74092	0.58445	0.40023	0.25506	0.41003	0.56386	0.0040178
(3,-1)	0.79895	0.60866	0.55443	0.19814	0.385	0.42501	0.0029096
(1,0)	0.094173	0.036014	0.019107	0.88963	0.91688	0.75192	0.016195
(2,0)	0.12325	0.017282	0.015453	0.86523	0.96939	0.9059	0.011525
(3,0)	0.1773	0.02616	0.018494	0.81155	0.95359	0.92513	0.011154
(1,1)	0.041132	0.0065127	0.0021313	0.59869	0.39374	0.21787	0.36017
(2,1)	0.059151	0.0053554	0.0027769	0.65672	0.42325	0.26478	0.28413
(3,1)	0.095832	0.010519	0.0051565	0.7768	0.6944	0.3906	0.12736

Rows are indexed by $(\rho_{\text{previous}}, \Delta S_{\text{current}})$. Conditional probabilities for each of the three possible states of ΔS_{future} are read from the corresponding column, conditional on ρ_{current} .

Represent Q with a heatmap.

```
figure
imagesc(Q)
axis equal tight
hCB = colorbar;
hCB.Label.String = "Prob(\Delta S_{future} | \rho_{previous}, \rho_{current}, \Delta S_{current})";
xticks(phiNums)
xticklabels(stateNames)
xlabel("\rho_{current}, \Delta S_{future}")
yticks(phiNums)
yticklabels(stateNames)
ylabel("\rho_{previous}, \Delta S_{current}")
title("Trading Matrix")
```



The bright, central 3 x 3 square shows that in most transitions, tick to tick, no price change is expected ($\Delta S_{\text{future}} = 0$). Bright areas in the upper-left 3 x 3 square (downward price movements $\Delta S_{\text{future}} = -1$) and lower-right 3 x 3 square (upward price movements $\Delta S_{\text{future}} = +1$) show evidence of momentum, which can be leveraged in a trading strategy.

You can find arbitrage opportunities by thresholding Q above a specified trigger probability. For example:

```
trigger = 0.5;
QPattern = (Q > trigger)
```

$Q\text{Pattern} = 9 \times 9$ logical array

1	0	0	0	1	1	0	0	0
1	1	0	0	0	1	0	0	0
1	1	1	0	0	0	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	1	0	0	0	1	1
0	0	0	1	0	0	0	1	1
0	0	0	1	1	0	0	0	1

The entry in the (1,1) position shows a chance of more than 50% that a downward price movement ($\Delta S_{\text{current}} = -1$) will be followed by another downward price movement ($\Delta S_{\text{future}} = -1$), provided that the previous and current imbalance states ρ are both 1.

A Trading Strategy?

Q is constructed on the basis of both the available exchange data and the hyperparameter settings. Using Q to inform future trading decisions depends on the market continuing in the same statistical

pattern. Whether the market exhibits momentum in certain states is a test of the weak-form *Efficient Market Hypothesis* (EMH). For heavily traded assets, such as the one used in this example (INTC), the EMH is likely to hold over extended periods, and arbitrage opportunities quickly disappear. However, failure of EMH can occur in some assets over short time intervals. A working trading strategy divides a portion of the trading day, short enough to exhibit a degree of statistical equilibrium, into a training period for estimating Q , using optimal hyperparameter settings and a validation period on which to trade. For an implementation of such a strategy, see “Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction” on page 2-69.

Summary

This example begins with raw data on the LOB and transforms it into a summary (the Q matrix) of statistical arbitrage opportunities. The analysis uses the mathematics of continuous-time Markov chain models, first in recognizing the Poisson process of LOB interarrival times, then by discretizing data into two-dimensional states representing the instantaneous position of the market. A description of state transitions, derived empirically, leads to the possibility of an algorithmic trading strategy.

References

- [1] Cartea, Álvaro, Sebastian Jaimungal, and Jason Ricci. "Buy Low, Sell High: A High-Frequency Trading Perspective." *SIAM Journal on Financial Mathematics* 5, no. 1 (January 2014): 415–44. <https://doi.org/10.1137/130911196>.
- [2] Guilbaud, Fabien, and Huyen Pham. "Optimal High-Frequency Trading with Limit and Market Orders." *Quantitative Finance* 13, no. 1 (January 2013): 79–94. <https://doi.org/10.1080/14697688.2012.708779>.
- [3] Norris, J. R. *Markov Chains*. Cambridge, UK: Cambridge University Press, 1997.
- [4] Rubisov, Anton D. "Statistical Arbitrage Using Limit Order Book Imbalance." Master's thesis, University of Toronto, 2015.

See Also

More About

- “Machine Learning for Statistical Arbitrage: Introduction” on page 2-48
- “Machine Learning for Statistical Arbitrage I: Data Management and Visualization” on page 2-50
- “Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction” on page 2-69

Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction

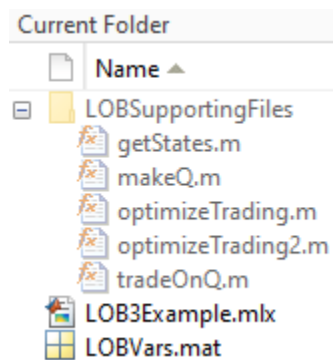
This example uses Bayesian optimization to tune hyperparameters in the algorithmic trading model, supervised by the end-of-day return. It is part of a series of related examples on machine learning for statistical arbitrage (see “Machine Learning Applications”).

Load `LOBVars.mat`, the preprocessed LOB data set of the NASDAQ security INTC, which is included with the Financial Toolbox™ documentation.

```
load LOBVars
```

The data set contains the following information for each order: the arrival time `t` (seconds from midnight), level 1 asking price `MOAsk`, level 1 bidding price `MOBid`, midprice `S`, and imbalance index `I`.

This example includes several supporting functions. To view them, open the example, and then expand the `LOBSupportingFiles` folder in the **Current Folder** pane.



Access the files by adding them to the search path.

```
addpath("LOBSupportingFiles")
```

Trading Strategy

The trading matrix `Q` contains probabilities of future price movements, given current and previous states `rho` of the limit order book (LOB) imbalance index `I` and the latest observed direction in prices `DS`.

View the supporting function `tradeOnQ.m`, which implements a simple trading strategy based on the pattern in `Q`.

```
function cash = tradeOnQ(Data,Q,n,N)
```

```
% Reference: Machine Learning for Statistical Arbitrage
%             Part II: Feature Engineering and Model Development
```

```
% Data
```

```
t = Data.t;
MOBid = Data.MOBid;
```

```
MOAsk = Data.MOAsk;

% States
[rho,DS] = getStates(Data,n,N);

% Start of trading

cash = 0;
assets = 0;

% Active trading
T = length(t);
for tt = 2:T-N % Trading ticks

    % Get Q row, column indices of current state

    row = rho(tt-1)+n*(DS(tt-1)+1);
    downColumn = rho(tt);
    upColumn = rho(tt) + 2*n;

    % If predicting downward price move
    if Q(row,downColumn) > 0.5

        cash = cash + MOBid(tt); % Sell
        assets = assets - 1;

    % If predicting upward price move
    elseif Q(row,upColumn) > 0.5

        cash = cash - MOAsk(tt); % Buy
        assets = assets + 1;

    end
end

% End of trading (liquidate position)
if assets > 0

    cash = cash + assets*MOBid(T); % Sell off
elseif assets < 0

    cash = cash + assets*MOAsk(T); % Buy back
end
```

The algorithm uses predictions from Q to make decisions about trading at each tick. It illustrates the general mechanism of any optimized machine learning algorithm.

This strategy seeks to profit from expected price changes using market orders (best offer at the touch) of a single share at each tick, if an arbitrage opportunity arises. The strategy can be scaled up

to larger trading volumes. Using the conditional probabilities obtained from Q , the `tradeOnQ` function takes one of these actions:

- Executes a buy if the probability of an upward forward price change is greater than 0.5.
- Executes a sell if the probability of a downward forward price change is greater than 0.5.

At the end of the trading day, the function liquidates the position at the touch.

The strategy requires `Data` with tick times t and the corresponding market order bid and ask prices `MOBid` and `MOAsk`, respectively. In real-time trading, data is provided by the exchange. This example evaluates the strategy by dividing the historical sample into *training* (calibration) and *validation* subsamples. The validation subsample serves as a proxy for real-time trading data. The strategy depends on Q , the trading matrix itself, which you estimate after you make a number of hyperparameter choices. The inputs n and N are hyperparameters to tune when you optimize the strategy.

Hyperparameters

The continuous-time Markov model and the resulting trading matrix Q depend on the values of four hyperparameters:

- `lambda` — The weighting parameter used to compute the imbalance index I
- `dI` — The number of backward ticks used to average I during smoothing
- `numBins` — The number of bins used to partition smoothed I for discretization
- `dS` — The number of forward ticks used to convert the prices S to discrete DS

In general, all four hyperparameters are tunable. However, to facilitate visualization, the example reduces the number of dimensions by tuning only `numBins` and N . The example:

- Fixes `lambda`
- Leaves `numBins = n`, where n is free to vary
- Equalizes the window lengths `dI = dS = N`, where N is free to vary

The restrictions do not significantly affect optimization outcomes. The optimization algorithm searches over the two-dimensional parameter space (n, N) for the configuration yielding the maximum return on trading.

Training and Validation Data

Machine learning requires a subsample on which to estimate Q and another subsample on which to evaluate the hyperparameter selections.

Specify a breakpoint to separate the data into training and validation subsamples. The breakpoint affects evaluation of the objective function, and is essentially another hyperparameter. However, because you do not tune the breakpoint, it is external to the optimization process.

```
bp = round((0.80)*length(t)); % Use 80% of data for training
```

Collect data in a timetable to pass to `tradeOnQ`.

```
Data = timetable(t,S,I,MOBid,MOAsk);
TData = Data(1:bp,:); % Training data
VData = Data(bp+1:end,:); % Validation data
```

Cross-Validation

Cross-validation describes a variety of techniques to assess how training results (here, computation of Q) generalize, with predictive reliability, to independent validation data (here, profitable trading). The goal of cross-validation is to flag problems in training results, like bias and overfitting. In the context of the trading strategy, overfitting refers to the time dependence, or nonstationarity, of Q . As Q changes over time, it becomes less effective in predicting future price movements. The key diagnostic issue is the degree to which Q changes, and at what rate, over a limited trading horizon.

With training and validation data in place, specify the hyperparameters and compare Q in the two subsamples. The supporting function `makeQ.m` provides the steps for making Q .

```
% Set specific hyperparameters
```

```
n = 3; % Number of bins for I
N = 20; % Window lengths
```

```
% Compare Qs
```

```
QT = makeQ(TData,n,N);
QV = makeQ(VData,n,N);
QTVDiff = QT - QV
```

```
QTVDiff = 9×9
```

```
    0.0070    0.0182    0.1198   -0.0103   -0.0175   -0.0348    0.0034   -0.0007   -0.0851
   -0.0009    0.0176    0.2535   -0.0010   -0.0233   -0.2430    0.0019    0.0058   -0.0106
    0.0184    0.0948    0.0835   -0.0195   -0.1021   -0.1004    0.0011    0.0073    0.0168
    0.0462    0.0180    0.0254   -0.0512   -0.0172    0.0417    0.0050   -0.0009   -0.0671
    0.0543    0.0089    0.0219   -0.0556   -0.0169   -0.0331    0.0013    0.0080    0.0112
    0.1037    0.0221    0.0184   -0.1043   -0.0401   -0.0479    0.0006    0.0180    0.0295
    0.0266    0.0066    0.0054   -0.0821   -0.0143   -0.0116    0.0555    0.0077    0.0062
    0.0615    0.0050    0.0060   -0.0189   -0.0207   -0.0262   -0.0426    0.0157    0.0203
    0.0735    0.0103    0.0090   -0.0788   -0.1216   -0.0453    0.0053    0.1113    0.0362
```

Differences between QT and QV appear minor, although they vary based on their position in the matrix. Identify trading inefficiencies, which result from indices (market states) where one matrix gives a trading cue (probability value > 0.5) and the other does not.

```
Inhomogeneity = (QT > 0.5 & QV < 0.5 ) | (QT < 0.5 & QV > 0.5 )
```

```
Inhomogeneity = 9×9 logical array
```

```
    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0
```

No significant inhomogeneities appear in the data with the given hyperparameter settings.

The severity of proceeding with a homogeneity assumption is not known *a priori*, and can emerge only from more comprehensive backtesting. Statistical tests are available, as described in [4] and [5], for example. During real-time trading, a rolling computation of Q over trailing training data of suitable size can provide the most reliable cues. Such an approach acknowledges inherent nonstationarity in the market.

Machine Learning

Machine learning refers to the general approach of effectively performing a task (for example, trading) in an automated fashion by detecting patterns (for example, computing Q) and making inferences based on available data. Often, data is dynamic and big enough to require specialized computational techniques. The evaluation process—tuning hyperparameters to describe the data and direct performance of the task—is ongoing.

In addition to the challenges of working with big data, the process of evaluating complex, sometimes black-box, objective functions is also challenging. Objective functions supervise hyperparameter evaluation. The trading strategy evaluates hyperparameter tunings by first computing Q on a training subsample, and then trading during an evaluation (real-time) subsample. The objective is to maximize profit, or minimize negative cash returned, over a space of suitably constrained configurations (n, N). This objective is a prototypical "expensive" objective function. *Bayesian optimization* is a type of machine learning suited to such objective functions. One of its principal advantages is the absence of costly derivative evaluations. To implement Bayesian optimization, use the Statistics and Machine Learning Toolbox™ function `bayesopt`.

The supporting function `optimizeTrading.m` uses `bayesopt` to optimize the trading strategy in `tradeOnQ`.

```
function results = optimizeTrading(TData,VData)

% Optimization variables

n = optimizableVariable('numBins',[1 10],'Type','integer');
N = optimizableVariable('numTicks',[1 50],'Type','integer');

% Objective function handle

f = @(x)negativeCash(x,TData,VData);

% Optimize

results = bayesopt(f,[n,N],...
    'IsObjectiveDeterministic',true,...
    'AcquisitionFunctionName','expected-improvement-plus',...
    'MaxObjectiveEvaluations',25,...
    'ExplorationRatio',2,...
    'Verbose',0);

end % optimizeTrading

% Objective (local)
function loss = negativeCash(x,TData,VData)

n = x.numBins;
N = x.numTicks;

% Make trading matrix Q
```

```
Q = makeQ(TData,n,N);

% Trade on Q

cash = tradeOnQ(VData,Q,n,N);

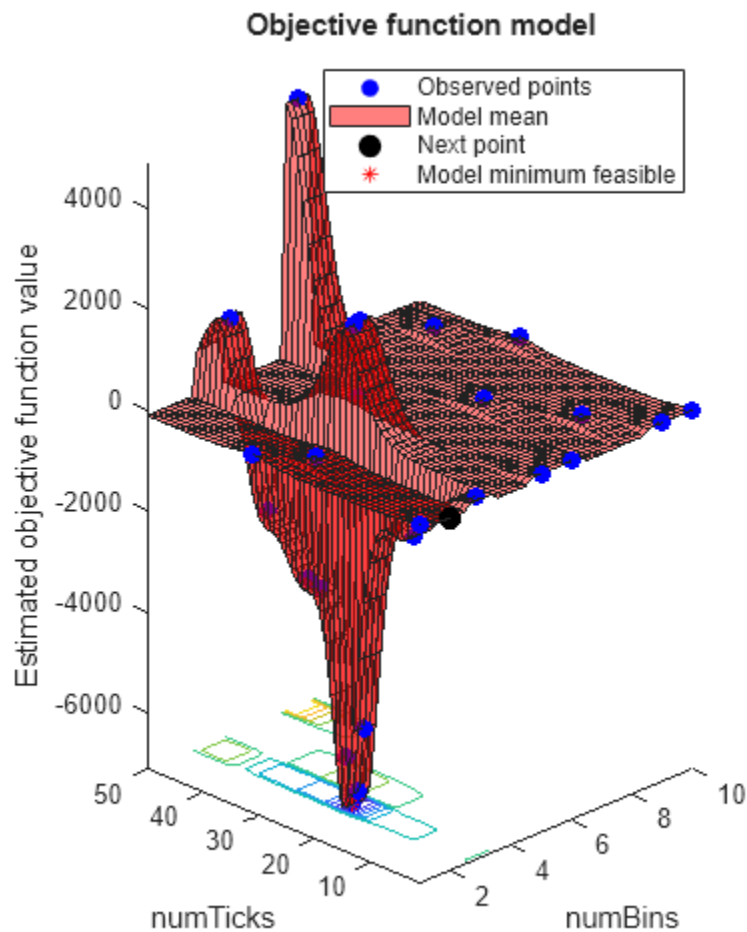
% Objective value

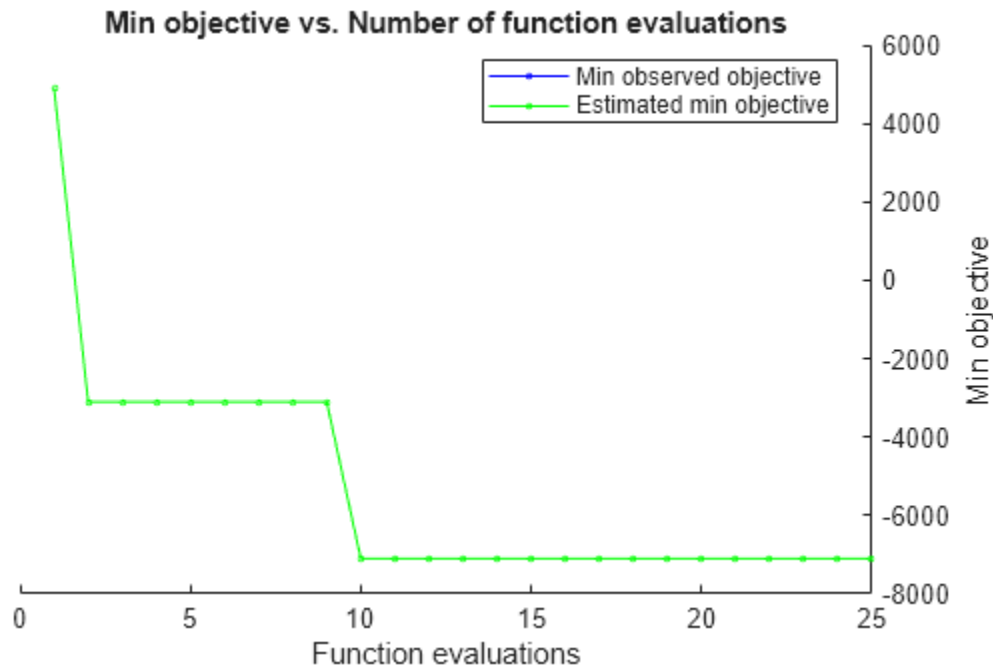
loss = -cash;

end % negativeCash
```

Optimize the trading strategy by passing the training and validation data to `optimizeTrading`.

```
rng(0) % For reproducibility
results = optimizeTrading(TData,VData);
```





The estimated minimum objective coincides with the minimum observed objective (the search is monotonic). Unlike derivative-based algorithms, `bayesopt` does not converge. As it tries to find the global minimum, `bayesopt` continues exploring until it reaches the specified number of iterations (25).

Obtain the best configuration by passing the results to `bestPoint`.

```
[Calibration,negReturn] = bestPoint(results,'Criterion','min-observed')
```

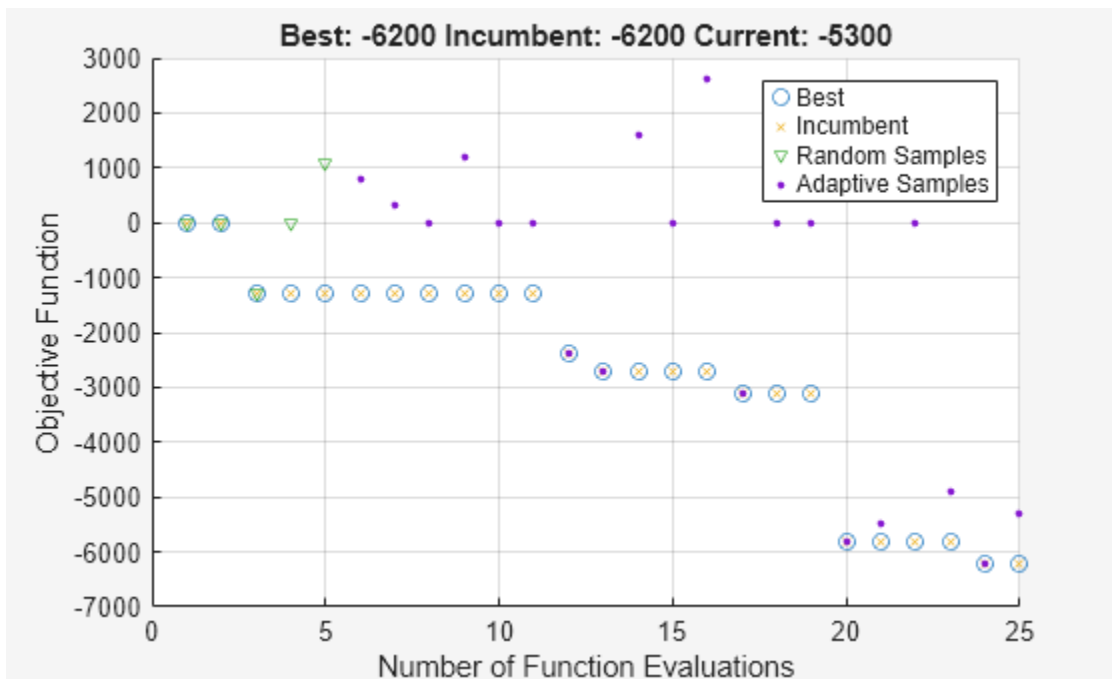
```
Calibration=1x2 table
    numBins    numTicks
    -----
         3         24
```

```
negReturn =
-7100
```

Trading one share per tick, as directed by `Q`, the optimal strategy using $(n,N) = (3,24)$ returns \$0.71 over the final 20% of the trading day. Modifying the trading volume scales the return.

Another optimizer designed for expensive objectives is `surrogateopt` (Global Optimization Toolbox). It uses a different search strategy and can locate optima more quickly, depending on the objective. The supporting function `optimizeTrading2.m` uses `surrogateopt` instead of `bayesopt` to optimize the trading strategy in `tradeOnQ`.

```
rng(0) % For reproducibility
results2 = optimizeTrading2(TData,VData)
```



```
results2 = 1×2
```

```
3      25
```

The results obtained with `surrogateopt` are the same as the `bayesopt` results. The plot contains information about the progress of the search that is specific to the `surrogateopt` algorithm.

Compute `Q` by passing the optimal hyperparameters and the entire data set to `makeQ`.

```
bestQ = makeQ(Data,3,24)
```

```
bestQ = 9×9
```

0.3933	0.1868	0.1268	0.5887	0.7722	0.6665	0.0180	0.0410	0.2068
0.5430	0.3490	0.2716	0.4447	0.6379	0.6518	0.0123	0.0131	0.0766
0.6197	0.3897	0.3090	0.3705	0.5954	0.6363	0.0098	0.0150	0.0547
0.1509	0.0440	0.0261	0.8217	0.8960	0.6908	0.0273	0.0601	0.2831
0.1900	0.0328	0.0280	0.7862	0.9415	0.8316	0.0238	0.0257	0.1404
0.2370	0.0441	0.0329	0.7391	0.9221	0.8745	0.0239	0.0338	0.0925
0.1306	0.0234	0.0101	0.7861	0.6566	0.4168	0.0833	0.3200	0.5731
0.1276	0.0169	0.0118	0.7242	0.6505	0.4712	0.1482	0.3326	0.5171
0.1766	0.0282	0.0186	0.7216	0.7696	0.6185	0.1018	0.2023	0.3629

The trading matrix `bestQ` can be used as a starting point for the next trading day.

Summary

This example implements the optimized trading strategy developed in the first two related examples. Available data is split into training and validation subsamples and used, respectively, to compute the trading matrix `Q` and execute the resulting trading algorithm. The process is repeated over a space of hyperparameter settings using the global optimizers `bayesopt` and `surrogateopt`, both of which

identify an optimal strategy yielding a positive return. The approach has many options for further customization.

References

- [1] Bull, Adam D. "Convergence Rates of Efficient Global Optimization Algorithms." *Journal of Machine Learning Research* 12, (November 2011): 2879-904.
- [2] Rubisov, Anton D. "Statistical Arbitrage Using Limit Order Book Imbalance." Master's thesis, University of Toronto, 2015.
- [3] Snoek, Jasper, Hugo Larochelle, and Ryan P. Adams. "Practical Bayesian Optimization of Machine Learning Algorithms." In *Advances in Neural Information Processing Systems 25*, F. Pereira et. al. editors, 2012.
- [4] Tan, Barış, and Kamil Yılmaz. "Markov Chain Test for Time Dependence and Homogeneity: An Analytical and Empirical Evaluation." *European Journal of Operational Research* 137, no. 3 (March 2002): 524-43. [https://doi.org/10.1016/S0377-2217\(01\)00081-9](https://doi.org/10.1016/S0377-2217(01)00081-9).
- [5] Weißbach, Rafael, and Ronja Walter. "A Likelihood Ratio Test for Stationarity of Rating Transitions." *Journal of Econometrics* 155, no. 2 (April 2010): 188-94. <https://doi.org/10.1016/j.jeconom.2009.10.016>.

See Also

More About

- "Machine Learning for Statistical Arbitrage: Introduction" on page 2-48
- "Machine Learning for Statistical Arbitrage I: Data Management and Visualization" on page 2-50
- "Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development" on page 2-59
- "Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data" on page 2-78
- "Deep Reinforcement Learning for Optimal Trade Execution" on page 4-407

Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data

This example applies artificial intelligence (AI) techniques and backtesting to find trading opportunities from imbalances in limit order book (LOB) data of a security. Specifically, the example applies a backtest trading strategy to measure the performance of a long short-term memory (LSTM) neural network. The goal of the model is to predict future forward price movements from current and past on ask and bid volume imbalances and forward price movements, which are extracted from the raw LOB data. Given a forward price movement prediction, a trading decision can be made. This example uses the following trading strategy and assumptions:

- Trading strategy: "All-in/all-out"; all cash is invested when the model predicts positive price movements and cash out when the model predicts no or negative cash movements.
- Transactions are free.
- Current trading actions do not affect future price movements.
- The risk-free rate is 0%.

This example builds on the developments in the "Machine Learning for Statistical Arbitrage: Introduction" on page 2-48 example series and uses this procedure:

- 1 Load and preprocess the raw limit order book data.
- 2 Engineer features by transforming the raw LOB data to obtain the discretized imbalance index series ρ_t and forward price movement series ΔS_t . The response is the discretized future price movement $\Delta S_{t+\Delta t_S}$.
- 3 Prepare the data for the classification LSTM neural network.
- 4 Partition the data into training, validation, and backtest sets.
- 5 Configure the LSTM neural network.
- 6 Train and validate the LSTM neural network.
- 7 Assess the quality of the trained model.
- 8 Backtest the trained LSTM neural network. Compare the performance of the model with a model that has knowledge of all future price movements (maximum profit model).

Load and Preprocess Raw LOB Data

This example uses the level 3 limit order book data from one trading day of NASDAQ exchange data [3] on one security (INTC) in a sample provided by LOBSTER [2] and included with the Financial Toolbox™ documentation in the zip file `LOBSTER_SampleFile_INTC_2012-06-21_5.zip`.

Extract the contents of the zip file into your current folder.

```
unzip("LOBSTER_SampleFile_INTC_2012-06-21_5.zip");
MSGFileName = "INTC_2012-06-21_34200000_57600000_message_5.csv";
LOBFileName = "INTC_2012-06-21_34200000_57600000_orderbook_5.csv";
```

The "Machine Learning for Statistical Arbitrage I: Data Management and Visualization" on page 2-50 example describes LOB data. To summarize:

- LOB data is composed of two CSV files: the order book `LOBFileName` and a message file `MSGFileName`.

- The order book data describes the intraday evolution of the limit order book of the security, which includes market and limit orders, resulting buys and sells, and corresponding times of all events.
- The message file describes each event.
- Many order events can occur during a trading day, therefore such data sets are typically large and difficult or impossible to fit in memory. This example uses a `tabularTextDatastore` and tall timetables to store and operate on the sets.

Extract the trading date from the message file name.

```
[ticker,rem] = strtok(MSGFileName,"_");
date = strtok(rem,"_");
```

Create separate datastores for the message and data files by using `tabularTextDatastore`. Ignore the generic column headers by setting `ReadVariableNames=false`. To allow similarly formatted files to be appended to existing datastores at the end of each trading day `ReadSize="file"`. Set descriptive variables names to each datastore.

```
DSMSG = tabularTextDatastore(MSGFileName,ReadVariableNames=false,ReadSize="file");
DSMSG.VariableNames = ["Time","Type","OrderID","Size","Price","Direction"];

DSL0B = tabularTextDatastore(LOBFileName,ReadVariableNames=false,ReadSize="file");
DSL0B.VariableNames = ["AskPrice1" "AskSize1" "BidPrice1" "BidSize1" ...
    "AskPrice2" "AskSize2" "BidPrice2" "BidSize2" ...
    "AskPrice3" "AskSize3" "BidPrice3" "BidSize3" ...
    "AskPrice4" "AskSize4" "BidPrice4" "BidSize4" ...
    "AskPrice5" "AskSize5" "BidPrice5" "BidSize5"];
```

Create a combined datastore by selecting the time variable of the message data and all levels in the limit order data.

```
DSMSG.SelectedVariableNames = ["Time" "Type"];
DSL0B.SelectedVariableNames = DSL0B.VariableNames;
```

```
CombinedDS = combine(DSMSG,DSL0B);
```

Engineer Features and Prepare Response Variable

AI techniques can accommodate data in a variety of forms, from high-dimensional raw data, in which it's difficult for humans to find patterns and structure, through a lower dimensional space where the variables in the raw data are transformed to more practical, interpretable measurements. For example, Kolm, et al. 2021 [1] train a convolutional neural network (CNN) and LSTM on raw LOB data to forecast for trading opportunities. Regardless of form, with careful tuning, the layers of a deep network can find structure in the data that might inform predictions.

As in [4] and “Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development” on page 2-59, this example uses the current imbalance index I_t and the sign of the forward price movements ΔS_t to build a predictive model for future price movements. The following summarizes these measurements; for more details, see “Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development” on page 2-59.

Initialize MapReduce for the tall array calculations.

```
mapreducer(0)
```

Convert the tall table into a tall timetable.

```
DT = tall(CombinedDS);
DT.Time = seconds(DT.Time);
DTT = table2timetable(DT);
```

The timestamps are durations, seconds from midnight on June 21, 2012 when the corresponding event occurred. Obtain timestamps that are dates and times of events instead of durations.

```
strt = datetime(201,6,21,0,0,0,34200000);
t = datetime(strt,Format="dd-MMM-yyyy HH:mm:ss.SSSSSSSSSSSSS") + DTT.Time;
```

Imbalance Index I_t

The level 3 imbalance index at time t I_t is the ratio of the weighted averages of the first three levels of ask and bid volumes on either side of the midprice at time t [4]. Symbolically,

$$I_t = \frac{V_{bt} - V_{at}}{V_{bt} + V_{at}},$$

where the level 3 weighted average of bid volumes $V_{bt} = \sum_{j=1}^3 e^{-\lambda(j-1)} V_{bjt}$ and V_{bjt} is the level j bid volume at time t . V_{at} is similarly defined.

The variable $\lambda > 0$ is a hyperparameter controlling the influence of levels 2 through 3 on the average volume. You choose the value of λ before training or you can tune it by, for example, performing cross-validation. This example sets λ to 0.5.

I_t is in the interval $[-1, 1]$; a positive value indicates a larger bid volume than ask volume and a negative value indicates a larger ask volume than bid volume. A magnitude close to 1 suggests an extreme volume imbalance.

Compute level 3 imbalance index values.

```
lvl = 3;           % Hyperparameter
lambda = 0.5;      % Hyperparameter
weights = exp(-(lambda)*(0:(lvl-1)))';
VAsk = DTT{:, "AskSize" + string(1:lvl)}*weights;
VBid = DTT{:, "BidSize" + string(1:lvl)}*weights;
DTT.I = (VBid-VAsk)./(VBid+VAsk);
```

Midprice S_t

The midprice at time t S_t is the average of the level 1 bid and ask prices, $S_t = (V_{b1t} + V_{a1t})/2$. Price changes are measured with respect to changes in midprices.

Compute the midprices.

```
DTT.Midprices = (DTT.BidPrice1 + DTT.AskPrice1)/2;
```

Focus on Order Executions

Prices are far more likely to change when orders are executed, which correspond to event types 4 and 5 in the message data. Focus the sample on only those events.

```
executionIdx = DTT.Type == 4 | DTT.Type == 5; % By assumption
I = DTT.I(executionIdx);
```

```
midprices = DTT.Midprices(executionIdx);
t = t(executionIdx);
```

Initiate all calculations on the tall arrays.

```
[I,midprices,t,DTT] = gather(I,midprices,t,DTT);
```

Evaluating tall expression using the Local MATLAB Session:

- Pass 1 of 1: Completed in 15 sec

Evaluation completed in 17 sec

```
t = t + milliseconds(1:numel(midprices))'; % Make timestamps unique
```

Forward Price Movement ΔS_t

The forward price movement at time t ΔS_t is the difference between the midprice S_t at time t and time $t - \Delta t_S$.

The variable $\Delta t_S > 0$ is a hyperparameter controlling the baseline midprice to compare the current forward price. You can set it to the value informed by theory or practice, or you can tune it.

Following the procedure in “Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development” on page 2-59 to determine a value for Δt_S , compute the average time for arrivals of order executions.

```
dt = diff(t,1,1);
dt.Format = "s";
dtav = expfit(dt)
```

```
dtav = duration
      0.7214 sec
```

```
dtS = 1; % Hyperparameter
```

The average arrival time of order executions is 0.72 seconds. The setting $\Delta t_S = 1$ corresponds to an interval of 0.72 seconds, on average, which is close to the figure from the example. Therefore, $\Delta S_t = S_t - S_{t-1}$.

ΔS_t indicates the magnitude and direction of the current price change from the baseline $S_{t-\Delta t_S}$.

Although use of the magnitude might inform future price changes, this example discretizes the ΔS_t such that it uses only the direction of the price change.

Compute the forward price movements between execution orders.

```
DS = NaN(size(midprices));
shiftS = midprices((dtS+1):end);
DS(1:(end-dtS)) = sign(shiftS-midprices(1:(end-dtS)));
```

The difference operation introduces Δt_S missing values at the tail of DS. Remove the missing values and synchronize all series.

```
DS = DS(1:end-dtS,:);
I = I(dtS+1:end,:);
midprices = midprices(dtS+1:end);
t = t(dtS+1:end,:);
X = [I DS];
```

Response Variable y_t

The goal of the analysis is to build a model that accurately predicts forward price direction $\Delta S_{t+\Delta t_S}$ from historical forward price directions ΔS_j and imbalance index values I_j , $j \leq t$.

Prepare the response data. Cast it as a categorical variable.

```
classes = ["loss" "const" "gain"];
y = DS((dtS+1):end);
y = categorical(y, [-1 0 1], classes);
n = numel(y);
numClasses = numel(classes);
```

The predictor data X and responses y are prepared for many MATLAB® machine learning functions. However, for richer model, the LSTM neural network can associate a time series of predictors with each response.

Display the distribution of y_t .

```
tabulate(y)
```

Value	Count	Percent
loss	1497	4.61%
const	29651	91.29%
gain	1333	4.10%

The class distribution is severely imbalanced—a majority of the forward price changes are constant. A naive model that always predicts into the constant class, or a mathematical model trained to predict into that way, has about 91% accuracy. However, such a model does not predict price changes for trading opportunities. Therefore, because the goal is to exploit imbalances to make money, the model needs to be trained to learn how to predict losses and gains well, likely at the expense of misclassifying the constant class.

Prepare Data for LSTM Neural Network

A long short-term memory neural network is a special case of a recurrent neural network (RNN) that associates a time series of features with a response. Its goal is to learn long-term associations between the response and features, while "forgetting" less useful associations. Symbolically,

$$y_t = f(X_t, X_{t-1}, \dots, X_{t-b}; W),$$

where:

- y_t is the response at time t .
- X_t is the collection of features at time t , which can include lagged response values.
- X_{t-1} is the collection of features at time $t-1$.
- b is the outlook hyperparameter.
- W is the collection of neural network weights and biases.
- f specifies the LSTM architecture.

For more details, see “Long Short-Term Memory Neural Networks” (Deep Learning Toolbox).

Prepare the data for the LSTM neural network by associating each response with the past $b = 100$ forward price movements and imbalance index values (b is arbitrarily chosen in this example). Store each successive feature series in a cell vector.

```
b = 100;    % Hyperparameter
n = n - b + 1;
XCell = cell(n,1);

for k = 1:n
    tmp = zeros(width(X),b);
    for j = 1:b
        tmp(:,j) = X(j+k-1,:)' ;
    end
    XCell{k} = tmp;
end
```

Because the first predictor observation contains measurements from times 1 through 100, the response data must start at time 101. In other words, the setting b causes the sample size reduction $n - b$.

Synchronize the series.

```
XCell = XCell(1:(end-1));
y = y(b:(end-1));
midprices = midprices(b-1:end-2);
n = numel(y);
head(table(XCell,y))
```

XCell	y
{2×100 double}	const
{2×100 double}	const
{2×100 double}	const
{2×100 double}	const
{2×100 double}	const
{2×100 double}	const
{2×100 double}	const
{2×100 double}	const

Each response is associated with a 2-by-100 matrix of predictor data. In the first row of the table display, $XCell\{1, : \}$ contains imbalance index values during times $t(1:100)$ and $XCell\{2, : \}$ contains forward price movements in the same period. $y(1)$ is future forward price movement during time(101) (recall $\Delta t_S = 1$). Symbolically, the model is

$$y_{t+1} = f(I_t, I_{t-1}, \dots, I_{t-100}, \Delta S_t, \Delta S_{t-1}, \dots, \Delta S_{t-100}; W).$$

Partition Data

Partition the data into training, validation, and backtesting sets. Because the data are time series, you can use the `tspartition` function, which, among other features, cuts the data, rather than shuffles it before cutting it. Reserve half the data for backtesting, and partition the other half of the data so that 85% is for training the model and 15% is for validation.

```
pBT = 0.5;
tspFull = tspartition(n, Holdout=pBT);
idxTV = training(tspFull); % Training and validation indices
```

```

idxBT = test(tspFull);           % Backtesting indices
XTV = XCell(idxTV);
yTV = y(idxTV);
XBT = XCell(idxBT);
yBT = y(idxBT);
tBT = t(idxBT);
mpBT = midprices(idxBT);
nBT = sum(idxBT);

pV = 0.15;
nTV = sum(idxTV);
tspTV = tspartition(nTV, Holdout=pV);
idxT = training(tspTV);         % Training indices
idxV = test(tspTV);             % Validation indices
XT = XCell(idxT);
yT = y(idxT);
XV = XCell(idxV);
yV = y(idxV);

```

Configure LSTM Neural Networks

Consider training the following two LSTM neural networks:

- A deep learning model that ignores the severely imbalanced class distribution of the response data.
- A deep learning model that, despite the risk, addresses imbalanced class distribution by applying a weighting scheme.

For both deep learning models, this example uses the following LSTM architecture in this order. For more details, see “Long Short-Term Memory Neural Networks” (Deep Learning Toolbox).

- `sequenceInputLayer` — Sequence input layer for the two feature variables with $b = \text{MinLength} = 100$.
- `batchNormalizationLayer` — Batch normalization layer, which normalizes a mini-batch of data across all observations to speed up training and reduce the sensitivity to network initialization.
- `lstmLayer` — Long short-term memory layer with 64 hidden units and configured to return only the last step sequence. The number of hidden layers is a hyperparameter.
- `fullyConnectedLayer` — Fully connected layer, which combines W and inputs, and reduces the number of neurons to 10.
- `leakyReluLayer` — Leaky ReLU layer, which reduces negative outputs of the fully connected layer to 10% of their value.
- `fullyConnectedLayer` — Fully connected layer, which combines W and inputs, and reduces the number of neurons to 3.
- `softmaxLayer` — Softmax layer, which outputs a length 3 vector of class probabilities by applying the softmax function to the outputs of the previous fully connected layer.

The risky model applies the following class weighting scheme to increase the penalty on the loss for predicting into the gain and loss classes:

$$u_j = \frac{n_T}{3c_j},$$

where n_T is the number of training observations and c_j is the number of responses in class j . This weighting scheme forces the classifier to pay more attention to the classes with lower frequency.

Compute the class weights.

```
nTP = tspTV.TrainSize;
classFreqT = countcats(yT);
classWtsT = nTP./(numClasses*classFreqT);
```

For each LSTM neural network, create an array specifying the architecture. Note that the LSTM architecture itself is a hyperparameter, as are most options taken at their default.

```
% Hyperparameters
numHiddenUnits = 64;
numNeuronsOutFC1 = 10;
thresholdScaleLR = 0.1;

layersLSTM = [
    sequenceInputLayer(2,Name="Sequence",MinLength=b)
    batchNormalizationLayer(Name="BN")
    lstmLayer(numHiddenUnits,Name="LSTM",OutputMode="last")
    fullyConnectedLayer(numNeuronsOutFC1,Name="FC1")
    leakyReluLayer(thresholdScaleLR,Name="LR")
    fullyConnectedLayer(3,Name="FC2")
    softmaxLayer(Name="Softmax")];

lossFcn = @(Y,T) crossentropy(Y,T, ...
    classWtsT, ...
    Reduction = "sum",...
    WeightsFormat="C",...
    ClassificationMode="single-label",...
    NormalizationFactor="none")*numClasses;
```

Create the LSTM architectures and visualize one of them.

```
figure
plot(dlnetwork(layersLSTM))
```



Train and Validate LSTM Neural Network

Set the following training options by using the `trainingOptions` function:

- Use the stochastic gradient descent with momentum solver.
- Plot the training progress.
- Monitor the solver's progress by printing numeric results, in real time, every 100 iterations.
- Set the initial learning rate to `1e-1` and specify a piecewise learning rate schedule. These are hyperparameters.
- Specify the validation data in a cell vector.
- Validate every 100 iterations and set the validation patience to 5.
- Shuffle the data every epoch.

```
VData = {XV,yV};
```

```
% Hyperparameters
```

```
lr = 1e-4;
```

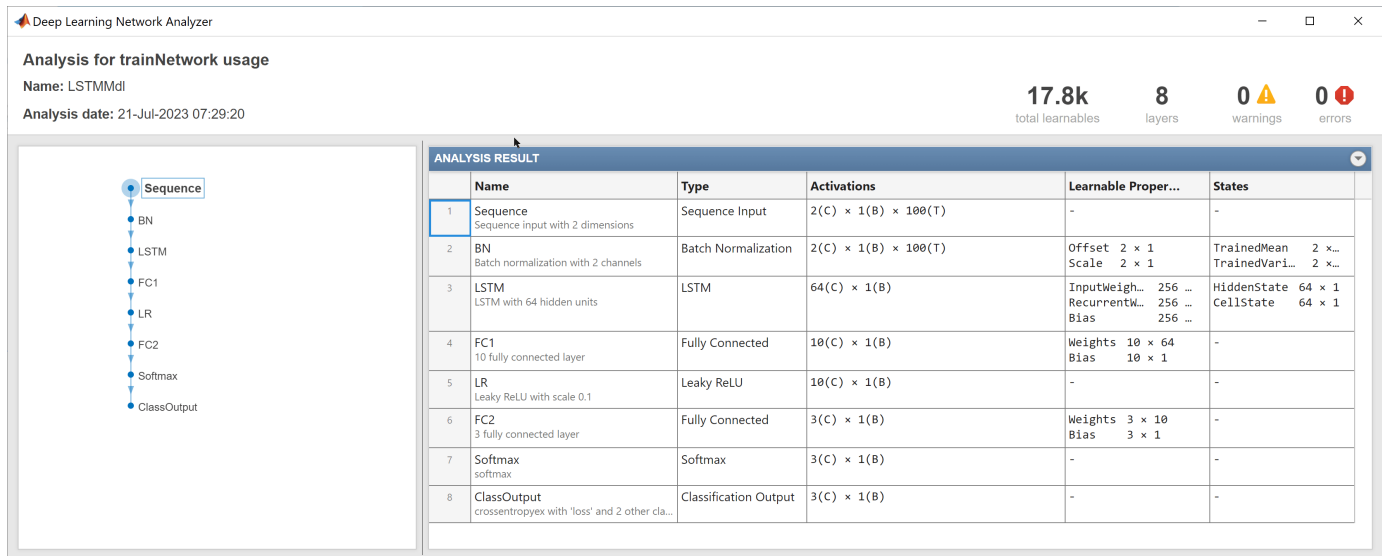
```
lrs = "piecewise";
```

```
options = trainingOptions("sgdm", ...
    Plots='training-progress', ...
    InputDataFormats="CTB",...
    Metric="accuracy",...
    Verbose=1,VerboseFrequency=100, ...
    InitialLearnRate=lr,LearnRateSchedule=lrs, ...
    ValidationData=VData,ValidationFrequency=100,ValidationPatience=5, ...
    Shuffle="every-epoch");
```

To train and validate the LSTM neural networks, set `trainLSTM` to `true`. Otherwise, this example loads the pretrained LSTM neural networks `LSTMmdl` and `RiskyLSTMmdl` in the MAT-file

PretrainedLOBLSTMLv13.mat, which is included with the Financial Toolbox documentation. Analyze the trained risky model.

```
trainLSTM = false;
if trainLSTM
    rng(1,"twister")
    LSTMmdl = trainnet(XT,yT,layersLSTM,"crossentropy",options);
    RiskyLSTMmdl = trainnet(XT,yT,layersLSTM,lossFcn,options);
else
    load PretrainedLOBLSTMLv13
end
analyzeNetwork(RiskyLSTMmdl)
```



Assess Trained LSTM Model Quality

For each trained LSTM model, compute the model accuracy on the validation data and plot a confusion chart by following this procedure:

- 1 Predict responses and classification scores for the validation data by passing the feature validation data and the trained LSTM model to `classify`.
- 2 Pass the observed and predicted responses to `confusionchart`. Add column and row summaries to the chart.

```
[predScoresLSTM] = minibatchpredict(LSTMmdl,XV,InputDataFormats="CTB");
```

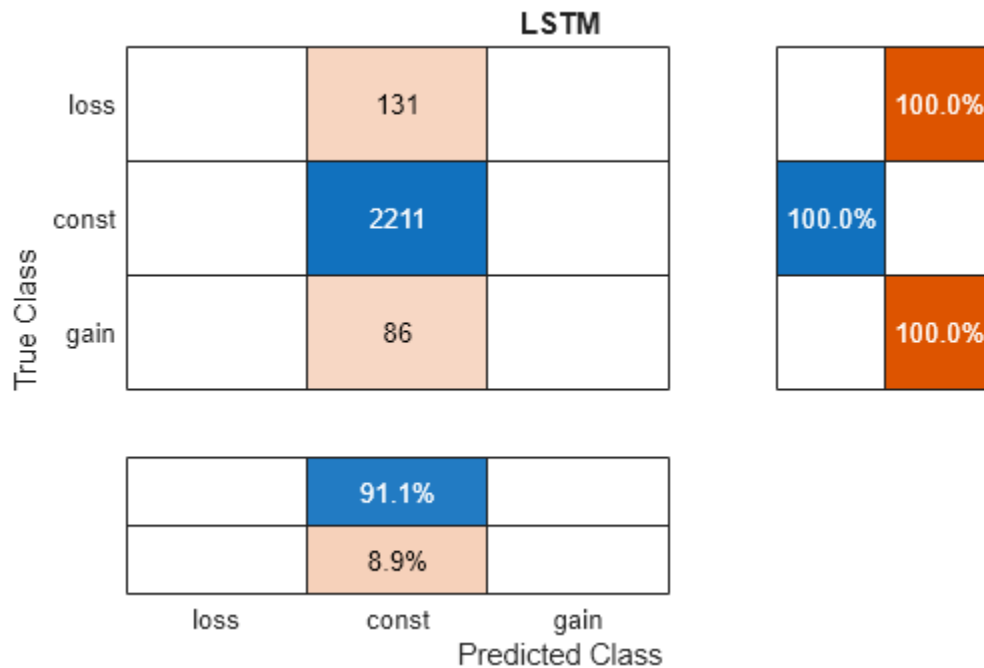
Warning: Support for GPU devices with compute capability 7.0 will be removed in a future MATLAB

```
predY LSTM = scores2label(predScoresLSTM,classes);
accV = 100*mean(predY LSTM == yV)
```

```
accV =
91.0626
```

```
figure
cm = confusionchart(yV,predY LSTM);
cm.NormalizedValues;
cm.RowSummary = "row-normalized";
```

```
cm.ColumnSummary = "column-normalized";
cm.Title = "LSTM";
```

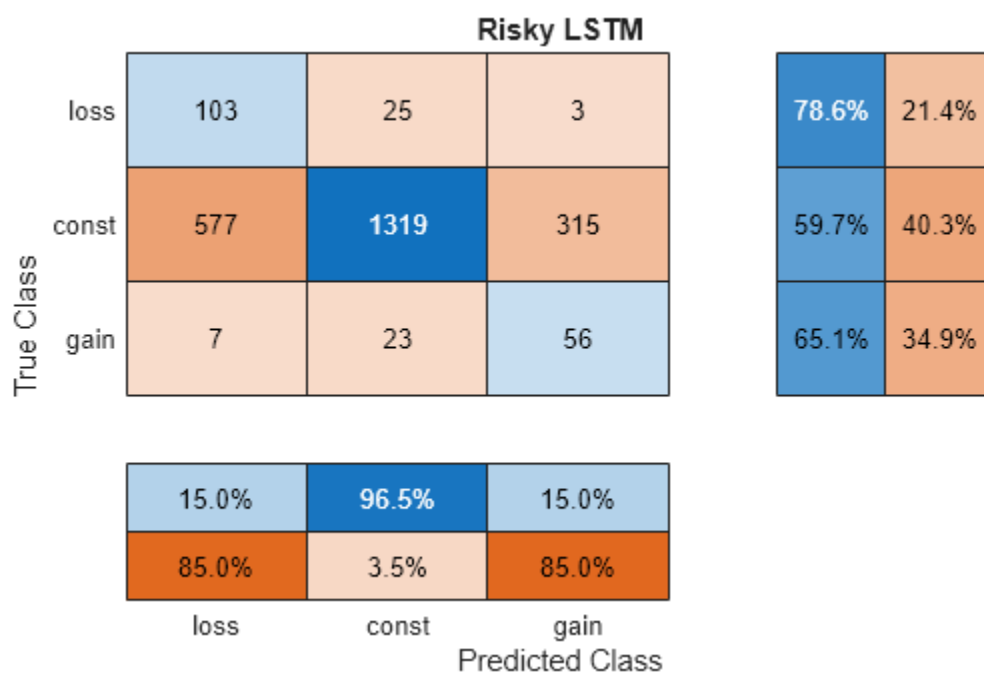


The conservative LSTM neural network predicts into the constant class for all validation observations. The 91% accuracy comes for free because this model acts like the naive model discussed in Engineer Features and Prepare Response Variable on page 2-79, which only ever predicts into the constant class. Such a model does not make money.

```
[predScoresRLSTM] = minibatchpredict(RiskyLSTMMdl,XV,InputDataFormats="CTB");
predYRLSTM = scores2label(predScoresRLSTM,classes);
accRiskyV = 100*mean(predYRLSTM == yV)
```

```
accRiskyV =
60.8731
```

```
figure
cm = confusionchart(yV,predYRLSTM);
cm.NormalizedValues;
cm.RowSummary = "row-normalized";
cm.ColumnSummary = "column-normalized";
cm.Title = "Risky LSTM";
```



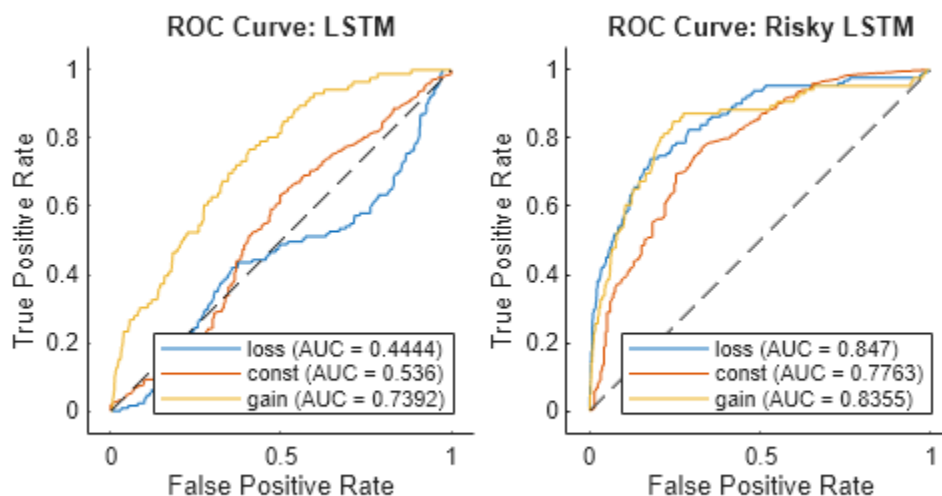
The risky LSTM model accuracy is about 61%. Despite this low value, the confusion chart suggests the following other aspects about the model's performance as it pertains to the trading strategy.

- It correctly predicted about 65% of the 86 true gains, which makes money.
- It correctly predicted about 79% of the 131 true losses, which prevents money loss.
- It incorrectly predicted about 35% of the true gains as losses or constant, which results in a missed opportunity to make money.
- It incorrectly predicted about 2% of the true losses as gains, which results in a money loss.

Plot ROC curves for both deep learning models.

```
rocLSTM = rocmetrics(yV,predScoresLSTM,classes);
rocRLSTM = rocmetrics(yV,predScoresRLSTM,classes);
```

```
figure
tiledlayout(1,2,TileSpacing="compact")
nexttile
plot(rocLSTM,ShowModelOperatingPoint=false)
title("ROC Curve: LSTM")
nexttile
plot(rocRLSTM,ShowModelOperatingPoint=false)
title("ROC Curve: Risky LSTM")
```



In general, each curve represents the ability of the model to distinguish observations that are in the corresponding class from the other two classes at various classification decision thresholds. For example, the blue curve corresponds to the ability of the classifiers to correctly distinguish losses from constants or gains. The diagonal dashed line is a hypothetical classifier that assigns observations into classes at random, which is a baseline for comparison. The curve of a good classifier increases very quickly and then levels off, making the area under the curve (AUC) close to 1. In other words, a good classifier has a high true positive rate for all classification decision thresholds, regardless of false positive rate levels produced. Note that the ROC curve does not consider false negative rates.

In this case, the risky LSTM is better at correctly distinguishing each class from the other two than the conservative LSTM; the AUCs of the risky LSTM are larger than the corresponding AUCs of the conservative LSTM.

Backtest Risky LSTM Model

Backtesting a predictive financial model dispatches it, and a trading strategy based on its predictions, on historical data. This action determines how well the model performs, with respect to the strategy. A backtest requires:

- Historical data not used to train or validate the model (see Partition Data on page 2-83)
- The target predictive model (The risky LSTM neural network `RiskyLSTMMdl`)
- The model's predictions (signals) on the historical data
- At least one trading strategy

Classify the forward price movements in the backtest data XBT using the risky LSTM neural network.

```
[predScoresRLSTMBT] = minibatchpredict(RiskyLSTMMdl,XBT,InputDataFormats="CTB");
predYRLSTMBT = scores2label(predScoresRLSTMBT,classes);
```

A trading strategy requires a way to predict outcomes and what financial decisions to make based on the outcomes. This example uses the following trading decision logic:

- If the forward price direction is predicted to be a gain, invest all available money.
- If the forward price direction is predicted to be a loss, cash out.
- Otherwise, do nothing.

The `rebalance` function, located in this example in Local Functions on page 2-93, defines these predictive models:

- Maximum Profit Model `MaxProfit`: A baseline model that perfectly predicts the next forward price direction, and therefore it always makes the right trading decisions.
- Risky LSTM Neural Network `RiskyLSTM`: A model that predicts the next forward price direction by using the trained risky LSTM neural network `RiskyLSTMmdl`, with trading decisions based on its predictions.

The inputs of `rebalance` are:

- Required asset weights `w`, which are the proportion of capital to invest across assets
- Required asset prices `p`, which are not used in this example
- Signal data `s`, optional data used to inform trading decisions
- Optional user data structure to print backtest iteration information
- Optional column of signal data to operate on for the strategy
- Optional strategy name

Define the backtest strategies by passing the a function handle to `rebalance` to “Create backtestStrategy” on page 15-233. Set the following properties:

- `LookbackWindow=1`, which indicates that the previous data point is available to the `rebalance` function during backtesting.
- `UserData=user_data`, where `user_data` is a structure array containing the fields `Counter`, set initially to 0, and `NumTestPoints`, set to the backtest sample size.

Store the strategies in a vector.

```
user_data = struct("Counter",0,"NumTestPoints",nBT);
maxMoneyStrat = backtestStrategy("MaxProfit",@(w,p,s,u)rebalance(w,p,s,u,1,"MaxProfit"), ...
    LookbackWindow=1,UserData=user_data);
RiskyLSTMStrat = backtestStrategy("RiskyLSTM",@(w,p,s,u)rebalance(w,p,s,u,2,"RiskyLSTM"), ...
    LookbackWindow=1,UserData=user_data);
strategies = [maxMoneyStrat RiskyLSTMStrat];
```

`strategies` is a vector of `backtestStrategy` objects storing the backtest strategies.

Prepare the backtest engine by supplying the vector of `backtestStrategy` objects to `backtestEngine`.

```
bte = backtestEngine(strategies);
```

`bte` is a `backtestEngine` object specifying parameters for the backtest, including the risk-free rate `RiskFreeRate`, which is 0% by default.

Store the backtest forward price direction data and risky LSTM predictions in a timetable.

```
signalTT = timetable(yBT,predYRLSTMBT,RowTimes=tBT);
```

Store the backtest midprices in a timetable.

```
mpTT = timetable(mpBT,RowTimes=tBT);
```

Run the backtest by passing the backtest engine, midprice data, and signal data to `runBacktest`.

```
bte = runBacktest(bte,mpTT,signalTT);
```

```
Backtesting MaxProfit. Total progress: 5000 of 16190.  
Backtesting MaxProfit. Total progress: 10000 of 16190.  
Backtesting MaxProfit. Total progress: 15000 of 16190.  
Backtesting RiskyLSTM. Total progress: 5000 of 16190.  
Backtesting RiskyLSTM. Total progress: 10000 of 16190.  
Backtesting RiskyLSTM. Total progress: 15000 of 16190.
```

`bte` is a `backtestEngine` object containing the results of the backtest.

Display a summary of the results by passing the completed backtest engine object to `summary`.

```
summary(bte)
```

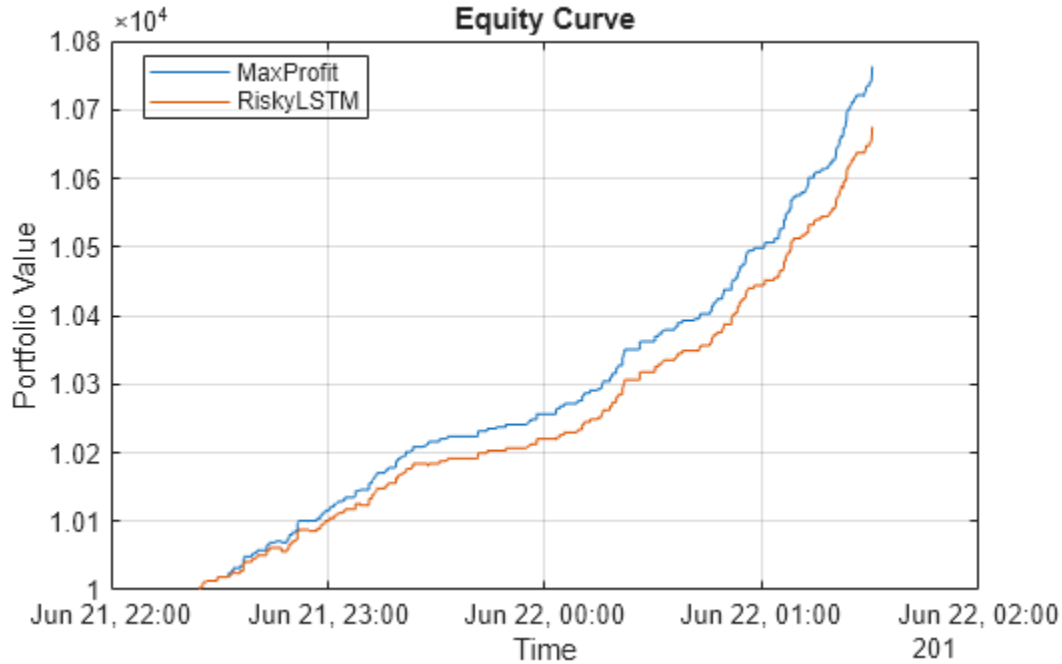
```
ans=9×2 table
```

	MaxProfit	RiskyLSTM
TotalReturn	0.076328	0.067524
SharpeRatio	0.15212	0.13715
Volatility	2.9873e-05	2.9433e-05
AverageTurnover	0.010624	0.0090802
MaxTurnover	0.5	0.5
AverageReturn	4.544e-06	4.0366e-06
MaxDrawdown	0.00018751	0.00055683
AverageBuyCost	0	0
AverageSellCost	0	0

The total nonannualized returns of the maximum profit and risky LSTM model are 7.6% and 6.8%, respectively. Therefore, the LSTM neural network performs well over the historical data.

Plot equity curves of the strategies by passing the completed backtest engine object to `equityCurve`.

```
figure  
equityCurve(bte)
```

The equity curves indicate both strategies performed well, and are in close correspondence with each other.

Local Function

The rebalance function directs the backtest framework to perform the following trades:

- When the model predicts a price gain, invest all available cash.
- When the model predicts a price loss, cash out entirely.
- When the model predicts a constant price, take no actions.

You can use this function only from in this script.

```
function [new_weights,user_data] = rebalance(weights,~,signalData,user_data,col,id)
    new_weights = weights;

    direction = signalData(:,col);
    if direction == "gain"
        new_weights = 1;    % Expect a gain --> Fully invest
    end
    if direction == "loss"
        new_weights = 0;    % Expect loss --> Cash out, with risk-free rate of 0%
    end

    % Print iteration information at command line
    user_data.Counter = user_data.Counter + 1;
    if (mod(user_data.Counter,5000) == 0)
        displayText = "Backtesting " + id + ". Total progress: " + num2str(user_data.Counter) + "
        disp(displayText);
```

end
end

References

- [1] Kolm, Petter, N., Jeremy Turiel, and Nicholas Westray. "Deep Order Flow Imbalance: Extracting Alpha at Multiple Horizons from the Limit Order Book." *SSRN* (August 2021): https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3900141.
- [2] LOBSTER Limit Order Book Data. Berlin: frishedaten UG (haftungsbeschränkt).
- [3] NASDAQ Historical TotalView-ITCH Data. New York: The Nasdaq, Inc.
- [4] Rubisov, Anton D. "Statistical Arbitrage Using Limit Order Book Imbalance." Master's thesis, University of Toronto, 2015.

See Also

More About

- "Machine Learning for Statistical Arbitrage: Introduction" on page 2-48
- "Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development" on page 2-59
- "Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction" on page 2-69
- "Backtest Investment Strategies Using Financial Toolbox" on page 4-238
- "Backtest Strategies Using Deep Learning" on page 4-302
- "Backtest Investment Strategies with Trading Signals" on page 4-251
- "Deep Reinforcement Learning for Optimal Trade Execution" on page 4-407

Portfolio Analysis

- “Analyzing Portfolios” on page 3-2
- “Portfolio Optimization Functions” on page 3-3
- “Portfolio Construction Examples” on page 3-5
- “Portfolio Selection and Risk Aversion” on page 3-7
- “portopt Migration to Portfolio Object” on page 3-11
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Active Returns and Tracking Error Efficient Frontier” on page 3-27

Analyzing Portfolios

Portfolio managers concentrate their efforts on achieving the best possible trade-off between risk and return. For portfolios constructed from a fixed set of assets, the risk/return profile varies with the portfolio composition. Portfolios that maximize the return, given the risk, or, conversely, minimize the risk for the given return, are called *optimal*. Optimal portfolios define a line in the risk/return plane called the *efficient frontier*.

A portfolio may also have to meet additional requirements to be considered. Different investors have different levels of risk tolerance. Selecting the adequate portfolio for a particular investor is a difficult process. The portfolio manager can hedge the risk related to a particular portfolio along the efficient frontier with partial investment in risk-free assets. The definition of the capital allocation line, and finding where the final portfolio falls on this line, if at all, is a function of:

- The risk/return profile of each asset
- The risk-free rate
- The borrowing rate
- The degree of risk aversion characterizing an investor

Financial Toolbox software includes a set of portfolio optimization functions designed to find the portfolio that best meets investor requirements.

Warning `frontcon` has been removed. Use `Portfolio` instead.

`portopt` has been partially removed and will no longer accept `ConSet` or `varargin` arguments. `portopt` will only solve the portfolio problem for long-only fully invested portfolios. Use `Portfolio` instead.

See Also

`portalloc` | `frontier` | `portopt` | `Portfolio` | `portcons` | `portvrisk` | `pcalims` | `pcgcomp` | `pcglims` | `pcpval` | `abs2active` | `active2abs`

Related Examples

- “Portfolio Optimization Functions” on page 3-3
- “Portfolio Construction Examples” on page 3-5
- “Portfolio Selection and Risk Aversion” on page 3-7
- “Active Returns and Tracking Error Efficient Frontier” on page 3-27
- “Plotting an Efficient Frontier Using `portopt`” on page 10-22

More About

- “Portfolio Object Workflow” on page 4-18

Portfolio Optimization Functions

The portfolio optimization functions assist portfolio managers in constructing portfolios that optimize risk and return.

Capital Allocation	Description
<code>portalloc</code>	Computes the optimal risky portfolio on the efficient frontier, based on the risk-free rate, the borrowing rate, and the investor's degree of risk aversion. Also generates the capital allocation line, which provides the optimal allocation of funds between the risky portfolio and the risk-free asset.
Efficient Frontier Computation	Description
<code>frontier</code>	Computes portfolios along the efficient frontier for a given group of assets. Generates a surface of efficient frontiers showing how asset allocation influences risk and return over time.
<code>portopt</code>	Computes portfolios along the efficient frontier for a given group of assets. The computation is based on a set of user-specified linear constraints. Typically, these constraints are generated using the constraint specification functions described below. Warning <code>portopt</code> has been partially removed and will no longer accept <code>ConSet</code> or <code>varargin</code> arguments. <code>portopt</code> will only solve the portfolio problem for long-only fully invested portfolios. Use <code>Portfolio</code> instead. For more information on migrating <code>portopt</code> code to <code>Portfolio</code>, see “<code>portopt</code> Migration to <code>Portfolio</code> Object” on page 3-11.
Constraint Specification	Description
<code>portcons</code>	Generates the portfolio constraints matrix for a portfolio of asset investments using linear inequalities. The inequalities are of the type $A \cdot Wts' \leq b$, where Wts is a row vector of weights.
<code>portvrisk</code>	Portfolio value at risk (VaR) returns the maximum potential loss in the value of a portfolio over one period of time, given the loss probability level <code>RiskThreshold</code> .
<code>pcalims</code>	Asset minimum and maximum allocation. Generates a constraint set to fix the minimum and maximum weight for each individual asset.
<code>pcgcomp</code>	Group-to-group ratio constraint. Generates a constraint set specifying the maximum and minimum ratios between pairs of groups.
<code>pcglims</code>	Asset group minimum and maximum allocation. Generates a constraint set to fix the minimum and maximum total weight for each defined group of assets.
<code>pcpval</code>	Total portfolio value. Generates a constraint set to fix the total value of the portfolio.

Constraint Conversion	Description
abs2active	Transforms a constraint matrix expressed in absolute weight format to an equivalent matrix expressed in active weight format.
active2abs	Transforms a constraint matrix expressed in active weight format to an equivalent matrix expressed in absolute weight format.

Note An alternative to using these portfolio optimization functions is to use the Portfolio object (Portfolio) for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using Portfolio objects, see “Portfolio Object Workflow” on page 4-18.

See Also

portalloc | frontier | portopt | Portfolio | portcons | portvrisk | pcalims | pcgcomp | pcglims | pcpval | abs2active | active2abs

Related Examples

- “Portfolio Construction Examples” on page 3-5
- “Portfolio Selection and Risk Aversion” on page 3-7
- “Active Returns and Tracking Error Efficient Frontier” on page 3-27
- “Plotting an Efficient Frontier Using portopt” on page 10-22
- “portopt Migration to Portfolio Object” on page 3-11

More About

- “Analyzing Portfolios” on page 3-2
- “Portfolio Object Workflow” on page 4-18

Portfolio Construction Examples

In this section...

“Introduction” on page 3-5

“Efficient Frontier Example” on page 3-5

Introduction

The efficient frontier computation functions require information about each asset in the portfolio. This data is entered into the function via two matrices: an expected return vector and a covariance matrix. The expected return vector contains the average expected return for each asset in the portfolio. The covariance matrix is a square matrix representing the interrelationships between pairs of assets. This information can be directly specified or can be estimated from an asset return time series with the function `ewstats`.

Note An alternative to using these portfolio optimization functions is to use the Portfolio object (`Portfolio`) for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using Portfolio objects, see “Portfolio Object Workflow” on page 4-18.

Efficient Frontier Example

`frontcon` has been removed. To model the efficient frontier, use the `Portfolio` object instead. For example, using the `Portfolio` object, you can model an efficient frontier:

- “Obtaining Portfolios Along the Entire Efficient Frontier” on page 4-99
- “Obtaining Endpoints of the Efficient Frontier” on page 4-102
- “Obtaining Efficient Portfolios for Target Returns” on page 4-105
- “Obtaining Efficient Portfolios for Target Risks” on page 4-108
- “Efficient Portfolio That Maximizes Sharpe Ratio” on page 4-111
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Plotting the Efficient Frontier for a Portfolio Object” on page 4-125

See Also

`portalloc` | `frontier` | `portopt` | `Portfolio` | `portcons` | `portvrisk` | `pcalims` | `pcgcomp` | `pcglims` | `pcpval` | `abs2active` | `active2abs`

Related Examples

- “Portfolio Optimization Functions” on page 3-3
- “Portfolio Selection and Risk Aversion” on page 3-7
- “Active Returns and Tracking Error Efficient Frontier” on page 3-27
- “Plotting an Efficient Frontier Using `portopt`” on page 10-22
- “`portopt` Migration to Portfolio Object” on page 3-11

More About

- “Analyzing Portfolios” on page 3-2
- “Portfolio Object Workflow” on page 4-18

Portfolio Selection and Risk Aversion

In this section...

"Introduction" on page 3-7

"Optimal Risky Portfolio" on page 3-8

Introduction

One of the factors to consider when selecting the optimal portfolio for a particular investor is the degree of risk aversion. This level of aversion to risk can be characterized by defining the investor's indifference curve. This curve consists of the family of risk/return pairs defining the trade-off between the expected return and the risk. It establishes the increment in return that a particular investor requires to make an increment in risk worthwhile. Typical risk aversion coefficients range from 2.0 through 4.0, with the higher number representing lesser tolerance to risk. The equation used to represent risk aversion in Financial Toolbox software is

$$U = E(r) - 0.005 * A * \text{sig}^2$$

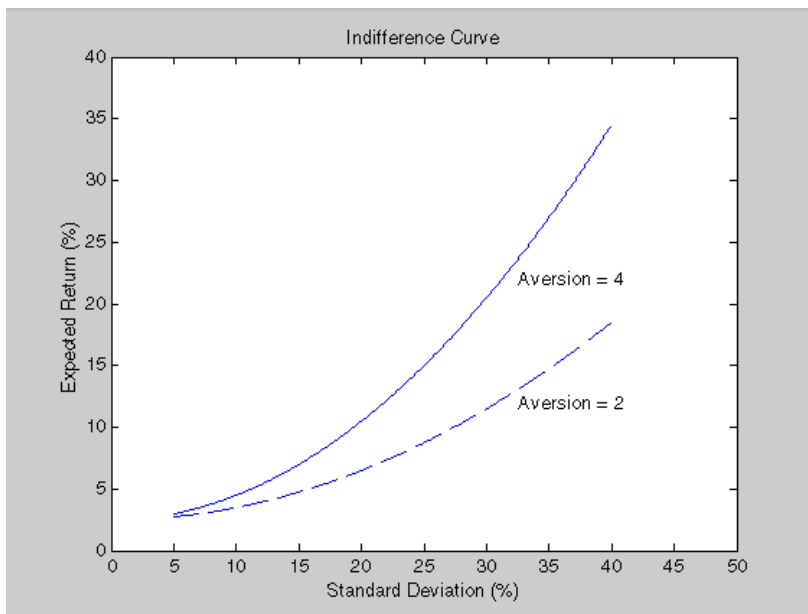
where:

U is the utility value.

$E(r)$ is the expected return.

A is the index of investor's aversion.

sig is the standard deviation.



Note An alternative to using these portfolio optimization functions is to use the Portfolio object (Portfolio) for mean-variance portfolio optimization. This object supports gross or net portfolio

returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using Portfolio objects, see “Portfolio Object Workflow” on page 4-18.

Optimal Risky Portfolio

This example shows how to compute the optimal risky portfolio on the efficient frontier based on the risk-free rate, the borrowing rate, and the investor's degree of risk aversion.

You do this with the function `portalloc`. First generate the efficient frontier data using `portopt`.

```
ExpReturn = [0.1 0.2 0.15];

ExpCovariance = [ 0.005   -0.010    0.004;
                 -0.010    0.040   -0.002;
                 0.004   -0.002    0.023];

NumPorts = 20; % Consider 20 different points along the efficient frontier.

[PortRisk, PortReturn, PortWts] = portopt(ExpReturn,...
ExpCovariance, NumPorts);
```

Calling `portopt`, while specifying output arguments, returns the corresponding vectors and arrays representing the risk, return, and weights for each of the portfolios along the efficient frontier. Use these as the first three input arguments to the function `portalloc`.

Find the optimal risky portfolio and the optimal allocation of funds between the risky portfolio and the risk-free asset, using these values for the risk-free rate, borrowing rate, and investor's degree of risk aversion.

```
RisklessRate = 0.08

RisklessRate =
0.0800

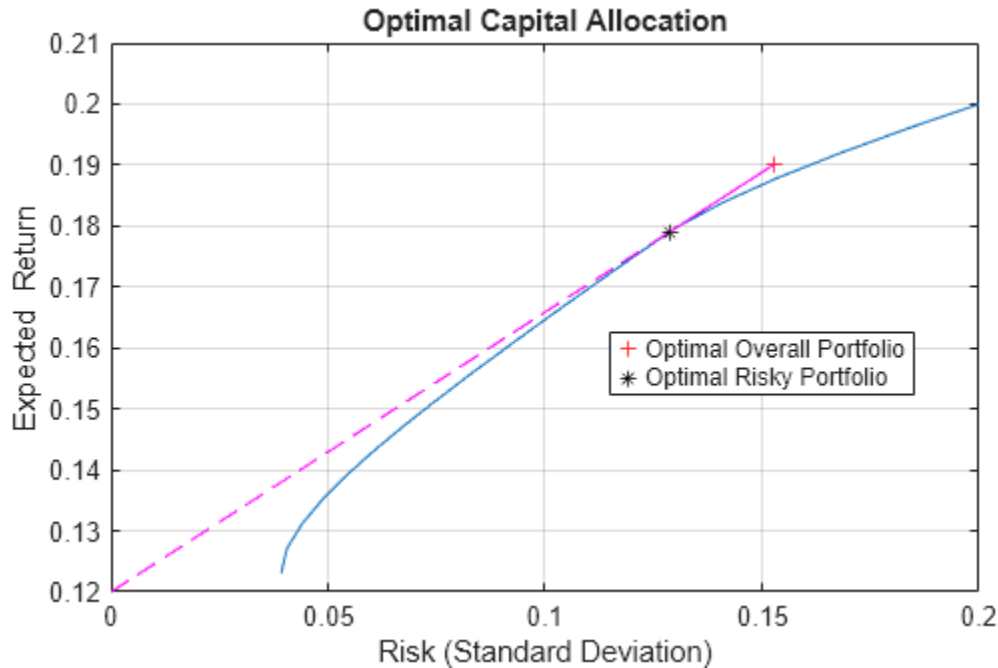
BorrowRate = 0.12

BorrowRate =
0.1200

RiskAversion = 3

RiskAversion =
3

portalloc(PortRisk, PortReturn, PortWts, RisklessRate,...
BorrowRate, RiskAversion);
```



Calling `portalloc` while specifying the output arguments returns the variance (`RiskyRisk`), the expected return (`RiskyReturn`), and the weights (`RiskyWts`) allocated to the optimal risky portfolio. It also returns the fraction (`RiskyFraction`) of the complete portfolio allocated to the risky portfolio, and the variance (`OverallRisk`) and expected return (`OverallReturn`) of the optimal overall portfolio. The overall portfolio combines investments in the risk-free asset and in the risky portfolio. The actual proportion assigned to each of these two investments is determined by the degree of risk aversion characterizing the investor.

```
[RiskyRisk, RiskyReturn, RiskyWts, RiskyFraction, OverallRisk, ...
OverallReturn] = portalloc (PortRisk, PortReturn, PortWts, ...
RisklessRate, BorrowRate, RiskAversion)
```

```
RiskyRisk =
0.1288
```

```
RiskyReturn =
0.1791
```

```
RiskyWts = 1×3
    0.0057    0.5879    0.4064
```

```
RiskyFraction =
1.1869
```

```
OverallRisk =
0.1529
```

```
OverallReturn =
0.1902
```

The value of `RiskyFraction` exceeds 1 (100%), implying that the risk tolerance specified allows borrowing money to invest in the risky portfolio, and that no money is invested in the risk-free asset.

This borrowed capital is added to the original capital available for investment. In this example, the customer tolerates borrowing 18.69% of the original capital amount.

See Also

`portalloc` | `frontier` | `portopt` | `Portfolio` | `portcons` | `portvrisk` | `pcalims` | `pcgcomp` | `pcglims` | `pcpval` | `abs2active` | `active2abs`

Related Examples

- “Portfolio Optimization Functions” on page 3-3
- “Active Returns and Tracking Error Efficient Frontier” on page 3-27
- “Plotting an Efficient Frontier Using `portopt`” on page 10-22
- “`portopt` Migration to Portfolio Object” on page 3-11

More About

- “Analyzing Portfolios” on page 3-2
- “Portfolio Object Workflow” on page 4-18

portopt Migration to Portfolio Object

In this section...

“Migrate portopt Without Output Arguments” on page 3-11

“Migrate portopt with Output Arguments” on page 3-12

“Migrate portopt for Target Returns Within Range of Efficient Portfolio Returns” on page 3-13

“Migrate portopt for Target Return Outside Range of Efficient Portfolio Returns” on page 3-14

“Migrate portopt Using portcons Output for ConSet” on page 3-15

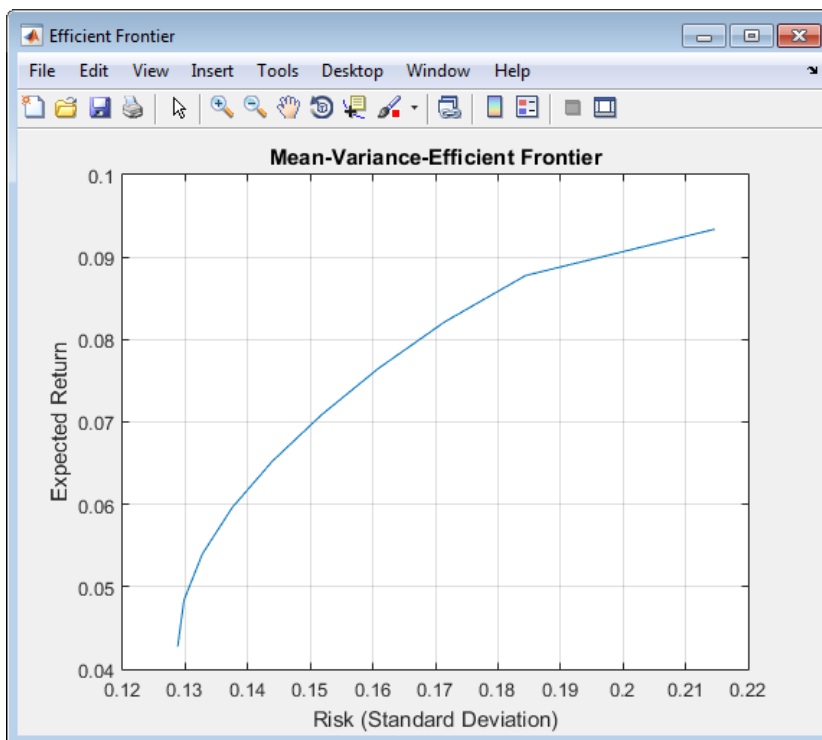
“Integrate Output from portcons, pcalims, pcglims, and pcgcomp with a Portfolio Object” on page 3-17

Migrate portopt Without Output Arguments

This example shows how to migrate portopt without output arguments to a Portfolio object.

The basic portopt functionality is represented as:

```
ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];
ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];
NumPorts = 10;
portopt(ExpReturn, ExpCovariance, NumPorts);
```



To migrate a portopt syntax without output arguments to a Portfolio object:

```

ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];

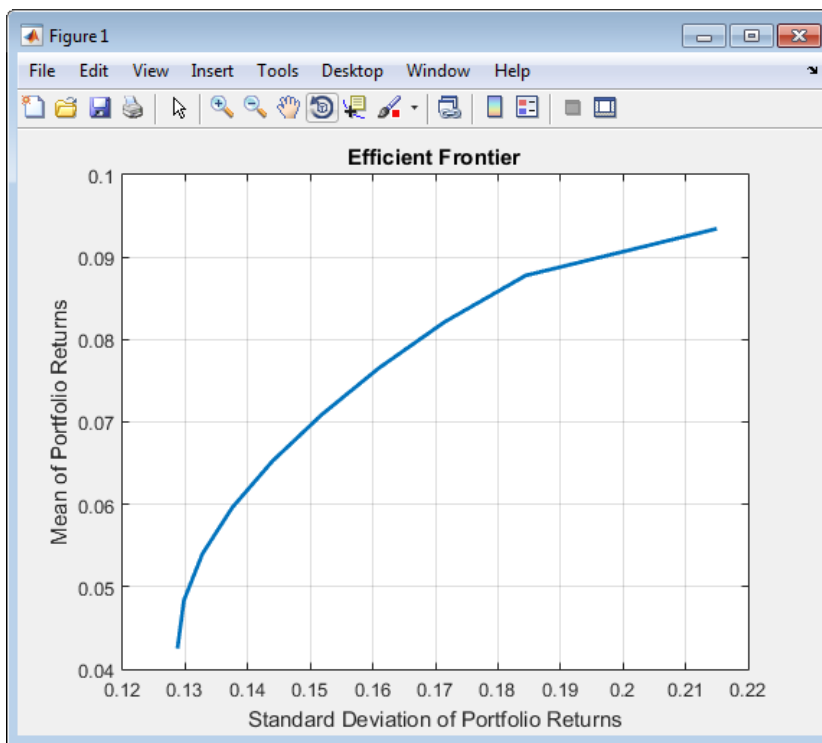
ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
  0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
  0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
  0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
  0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];

NumPorts = 10;

p = Portfolio;
p = setAssetMoments(p, ExpReturn, ExpCovariance);
p = setDefaultConstraints(p);

plotFrontier(p, NumPorts);

```



Without output arguments, `portopt` plots the efficient frontier. The `Portfolio` object has similar behavior although the `Portfolio` object writes to the current figure window rather than create a new window each time a plot is generated.

Migrate `portopt` with Output Arguments

This example shows how to migrate `portopt` with output arguments to a `Portfolio` object.

With output arguments, the basic functionality of `portopt` returns portfolio moments and weights. Once the `Portfolio` object is set up, moments and weights are obtained in separate steps.

```

ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];

ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
  0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
  0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
  0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
  0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];

```

```

0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];

NumPorts = 10;

[PortRisk, PortReturn, PortWts] = portopt(ExpReturn, ExpCovariance, NumPorts);

display(PortWts);

PortWts =

    0.2103    0.2746    0.1157    0.1594    0.2400
    0.1744    0.2657    0.1296    0.2193    0.2110
    0.1386    0.2567    0.1436    0.2791    0.1821
    0.1027    0.2477    0.1575    0.3390    0.1532
    0.0668    0.2387    0.1714    0.3988    0.1242
    0.0309    0.2298    0.1854    0.4587    0.0953
         0     0.2168    0.1993    0.5209    0.0629
         0     0.1791    0.2133    0.5985    0.0091
         0     0.0557    0.2183    0.7260         0
         0         0         0     1.0000         0
    
```

To migrate a `portopt` syntax with output arguments:

```

ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];

ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];

NumPorts = 10;

p = Portfolio;
p = setAssetMoments(p, ExpReturn, ExpCovariance);
p = setDefaultConstraints(p);

PortWts = estimateFrontier(p, NumPorts);
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);

display(PortWts);

PortWts =

    0.2103    0.1744    0.1386    0.1027    0.0668    0.0309         0         0         0         0
    0.2746    0.2657    0.2567    0.2477    0.2387    0.2298    0.2168    0.1791    0.0557         0
    0.1157    0.1296    0.1436    0.1575    0.1714    0.1854    0.1993    0.2133    0.2183         0
    0.1594    0.2193    0.2791    0.3390    0.3988    0.4587    0.5209    0.5985    0.7260    1.0000
    0.2400    0.2110    0.1821    0.1532    0.1242    0.0953    0.0629    0.0091         0         0
    
```

The Portfolio object returns `PortWts` with portfolios going down columns, not across rows. Portfolio risks and returns are still in column format.

Migrate portopt for Target Returns Within Range of Efficient Portfolio Returns

This example shows how to migrate `portopt` target returns within range of efficient portfolio returns to a Portfolio object.

`portopt` can obtain portfolios with specific targeted levels of return but requires that the targeted returns fall within the range of efficient returns. The Portfolio object handles this by selecting portfolios at the ends of the efficient frontier.

```

ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];
    
```

```
ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];

NumPorts = 10;

TargetReturn = [ 0.05; 0.06; 0.07; 0.08; 0.09 ];

[PortRisk, PortReturn, PortWts] = portopt(ExpReturn, ExpCovariance, [], TargetReturn);

disp(' Efficient Target');
disp([PortReturn, TargetReturn]);

Efficient Target
0.0500 0.0500
0.0600 0.0600
0.0700 0.0700
0.0800 0.0800
0.0900 0.0900
```

To migrate a `portopt` syntax for target returns within range of efficient portfolio returns to a Portfolio object:

```
ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];

ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];

NumPorts = 10;

TargetReturn = [ 0.05; 0.06; 0.07; 0.08; 0.09 ];

p = Portfolio;
p = setAssetMoments(p, ExpReturn, ExpCovariance);
p = setDefaultConstraints(p);

PortWts = estimateFrontierByReturn(p, TargetReturn);
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);

disp(' Efficient Target');
disp([PortReturn, TargetReturn]);

Efficient Target
0.0500 0.0500
0.0600 0.0600
0.0700 0.0700
0.0800 0.0800
0.0900 0.0900
```

Migrate portopt for Target Return Outside Range of Efficient Portfolio Returns

This example shows how to migrate `portopt` target returns outside of range of efficient portfolio returns to a Portfolio object.

When the target return is outside of the range of efficient portfolio returns, `portopt` generates an error. The Portfolio object handles this effectively by selecting portfolios at the ends of the efficient frontier.


```

ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];
ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];
NumPorts = 10;
TargetReturn = [ 0.05; 0.06; 0.07; 0.08; 0.09; 0.10 ];
[PortRisk, PortReturn, PortWts] = portopt(ExpReturn, ExpCovariance, [], TargetReturn);
disp(' Efficient      Target');
disp([PortReturn, TargetReturn]);
> In portopt at 85
Error using portopt (line 297)
One or more requested returns are greater than the maximum achievable return of 0.093400.

```

To migrate a `portopt` syntax for target returns outside of the range of efficient portfolio returns to a `Portfolio` object:

```

ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];
ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];
NumPorts = 10;
TargetReturn = [ 0.05; 0.06; 0.07; 0.08; 0.09; 0.10 ];

p = Portfolio;
p = setAssetMoments(p, ExpReturn, ExpCovariance);
p = setDefaultConstraints(p);

PortWts = estimateFrontierByReturn(p, TargetReturn);
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);

disp(' Efficient      Target');
disp([PortReturn, TargetReturn]);

Warning: One or more target return values are outside the feasible range [
0.0427391, 0.0934 ].
Will return portfolios associated with endpoints of the range for these
values.
> In Portfolio/estimateFrontierByReturn (line 106)
Efficient      Target
0.0500         0.0500
0.0600         0.0600
0.0700         0.0700
0.0800         0.0800
0.0900         0.0900
0.0934         0.1000

```

Migrate portopt Using portcons Output for ConSet

This example shows how to migrate `portopt` when the `ConSet` output from `portcons` is used with `portopt`.

`portopt` accepts as input the outputs from `portcons`, `pcalims`, `pcglims`, and `pcgcomp`. This example focuses on `portcons`. `portcons` sets up linear constraints for `portopt` in the form $A \cdot \text{Port} \leq b$. In a matrix `ConSet = [ A, b ]` and break into separate `A` and `b` arrays with `A = ConSet(:,1:end-1);` and `b = ConSet(:,end);`. In addition, to illustrate default problem with additional group constraints, consider three groups. Assets 2, 3, and 4 can constitute up to 80% of portfolio, Assets 1 and 2 can constitute up to 70% of portfolio, and Assets 3, 4, and 5 can constitute up to 90% of portfolio.

```

ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];

```

```
ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;  
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;  
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;  
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;  
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];  
  
NumPorts = 10;  
  
Groups = [ 0 1 1 1 0; 1 1 0 0 0; 0 0 1 1 1 ];  
GroupBounds = [ 0, 0.8; 0, 0.7; 0, 0.9 ];  
  
LowerGroup = GroupBounds(:,1);  
UpperGroup = GroupBounds(:,2);  
  
ConSet = portcons('default', 5, 'grouplims', Groups, LowerGroup, UpperGroup);  
  
[PortRisk, PortReturn, PortWts] = portopt(ExpReturn, ExpCovariance, NumPorts, [], ConSet);  
  
disp([PortRisk, PortReturn]);  
  
Error using portopt (line 83)  
In the current and future releases, portopt will no longer accept ConSet or varargin arguments.  
'It will only solve the portfolio problem for long-only fully-invested portfolios.  
To solve more general problems, use the Portfolio object.  
See the release notes for details, including examples to make the conversion.
```

To migrate portopt to a Portfolio object when the ConSet output from portcons is used with portopt:

```
ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];  
  
ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;  
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;  
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;  
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;  
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];  
  
NumPorts = 10;  
  
Groups = [ 0 1 1 1 0; 1 1 0 0 0; 0 0 1 1 1 ];  
GroupBounds = [ 0, 0.8; 0, 0.7; 0, 0.9 ];  
  
LowerGroup = GroupBounds(:,1);  
UpperGroup = GroupBounds(:,2);  
  
ConSet = portcons('default', 5, 'grouplims', Groups, LowerGroup, UpperGroup);  
  
A = ConSet(:,1:end-1);  
b = ConSet(:,end);  
  
p = Portfolio;  
p = setAssetMoments(p, ExpReturn, ExpCovariance);  
p = setInequality(p, A, b); % implement group constraints here  
  
PortWts = estimateFrontier(p, NumPorts);  
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);  
  
disp([PortRisk, PortReturn]);  
  
0.1288    0.0427  
0.1292    0.0465  
0.1306    0.0503  
0.1328    0.0540  
0.1358    0.0578  
0.1395    0.0615  
0.1440    0.0653  
0.1504    0.0690  
0.1590    0.0728  
0.1806    0.0766
```

The constraints are entered directly into the Portfolio object with the `setInequality` or `addInequality` functions.

Integrate Output from `portcons`, `pcalims`, `pcglims`, and `pcgcomp` with a Portfolio Object

This example shows how to integrate output from `pcalims`, `pcalims`, `pcglims`, or `pcgcomp` with a Portfolio object implementation.

`portcons`, `pcalims`, `pcglims`, and `pcgcomp` setup linear constraints for `portopt` in the form $A \cdot \text{Port} \leq b$. Although some functions permit two outputs, assume that the output is a single matrix `ConSet`. Break into separate `A` and `b` arrays with:

- `A = ConSet(:,1:end-1);`
- `b = ConSet(:,end);`

In addition, to illustrate default problem with additional group constraints, consider three groups:

- Assets 2, 3, and 4 can constitute up to 80% of portfolio.
- Assets 1 and 2 can constitute up to 70% of portfolio.
- Assets 3, 4, and 5 can constitute up to 90% of portfolio.

```
Groups = [ 0 1 1 1 0; 1 1 0 0 0; 0 0 1 1 1 ];
GroupBounds = [ 0, 0.8; 0, 0.7; 0, 0.9 ];
```

To integrate the `ConSet` output of `portcons` with a Portfolio object implementation:

```
ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];

ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];

NumPorts = 10;

Groups = [ 0 1 1 1 0; 1 1 0 0 0; 0 0 1 1 1 ];
GroupBounds = [ 0, 0.8; 0, 0.7; 0, 0.9 ];

LowerGroup = GroupBounds(:,1);
UpperGroup = GroupBounds(:,2);

ConSet = portcons('default', 5, 'grouplims', Groups, LowerGroup, UpperGroup);

A = ConSet(:,1:end-1);
b = ConSet(:,end);

p = Portfolio;
p = setAssetMoments(p, ExpReturn, ExpCovariance);
p = setDefaultConstraints(p); % implement default constraints here
p = setInequality(p, A, b); % implement group constraints here

PortWts = estimateFrontier(p, NumPorts);
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);

disp([PortRisk, PortReturn]);

    0.1288    0.0427
    0.1292    0.0465
    0.1306    0.0503
    0.1328    0.0540
    0.1358    0.0578
    0.1395    0.0615
```

```
0.1440    0.0653
0.1504    0.0690
0.1590    0.0728
0.1806    0.0766
```

To integrate the output of `pcalims` and `pcglims` with a `Portfolio` object implementation:

```
ExpReturn = [ 0.0054; 0.0531; 0.0779; 0.0934; 0.0130 ];

ExpCovariance = [ 0.0569, 0.0092, 0.0039, 0.0070, 0.0022;
    0.0092, 0.0380, 0.0035, 0.0197, 0.0028;
    0.0039, 0.0035, 0.0997, 0.0100, 0.0070;
    0.0070, 0.0197, 0.0100, 0.0461, 0.0050;
    0.0022, 0.0028, 0.0070, 0.0050, 0.0573 ];

NumPorts = 10;

Groups = [ 0 1 1 1 0; 1 1 0 0 0; 0 0 1 1 1 ];
GroupBounds = [ 0, 0.8; 0, 0.7; 0, 0.9 ];

LowerGroup = GroupBounds(:,1);
UpperGroup = GroupBounds(:,2);

AssetMin = [ 0; 0; 0; 0; 0 ];
AssetMax = [ 0.8; 0.8; 0.8; 0.8; 0.8 ];

[Aa, ba] = pcalims(AssetMin, AssetMax);
[Ag, bg] = pcglims(Groups, LowerGroup, UpperGroup);

p = Portfolio;
p = setAssetMoments(p, ExpReturn, ExpCovariance);
p = setDefaultConstraints(p); % implement default constraints first
p = addInequality(p, Aa, ba); % implement bound constraints here
p = addInequality(p, Ag, bg); % implement group constraints here

PortWts = estimateFrontier(p, NumPorts);
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);

disp([PortRisk, PortReturn]);

0.1288    0.0427
0.1292    0.0465
0.1306    0.0503
0.1328    0.0540
0.1358    0.0578
0.1395    0.0615
0.1440    0.0653
0.1504    0.0690
0.1590    0.0728
0.1806    0.0766
```

See Also

`Portfolio` | `portopt` | `portcons` | `pcalims` | `pcglims` | `pcgcomp` | `estimatePortMoments` | `setInequality` | `setDefaultConstraints` | `addInequality` | `setAssetMoments` | `estimateFrontier` | `estimateFrontierByReturn`

More About

- “Portfolio Object Workflow” on page 4-18

Constraint Specification Using a Portfolio Object

When constructing the efficient frontier, several constraints are typically considered. You can use linear constraints that define the limits within which the portfolio optimization problem must operate and you can also use group constraints to limit the exposure to certain sectors or groups.

Constraints for Efficient Frontier

This example shows how to compute the efficient frontier of portfolios consisting of three different assets, INTC, XON, and RD, given a list of constraints.

The expected returns for INTC, XON, and RD are respectively as follows:

```
ExpReturn = [0.1 0.2 0.15];
```

The covariance matrix is

```
ExpCovariance = [ 0.005  -0.010  0.004;
                  -0.010  0.040  -0.002;
                  0.004  -0.002  0.023];
```

- **Constraint 1:** Allow short selling up to 10% of the portfolio value in any asset, but limit the investment in any one asset to 110% of the portfolio value.
- **Constraint 2:** Consider two different sectors, technology and energy, with the following table indicating the sector each asset belongs to.

Asset	INTC	XON	RD
Sector	Technology	Energy	Energy

Constrain the investment in the Energy sector to 80% of the portfolio value, and the investment in the Technology sector to 70%. To solve this problem, use `Portfolio`, passing in a list of asset constraints. Consider eight different portfolios along the efficient frontier:

```
NumPorts = 8;
```

To introduce the asset bounds constraints specified in **Constraint 1**, create the matrix `AssetBounds`, where each column represents an asset. The upper row represents the lower bounds, and the lower row represents the upper bounds. Since the bounds are the same for each asset, only one pair of bounds is needed because of scalar expansion.

```
AssetBounds = [-0.1, 1.1];
```

Constraint 2 must be entered in two parts, the first part defining the groups, and the second part defining the constraints for each group. Given the information above, you can build a matrix of 1s and 0s indicating whether a specific asset belongs to a group. Each column represents an asset, and each row represents a group. This example has two groups: the technology group, and the energy group. Create the matrix `Groups` as follows:

```
Groups = [0 1 1;
          1 0 0];
```

The `GroupBounds` matrix allows you to specify an upper and lower bound for each group. Each row in this matrix represents a group. The first column represents the minimum allocation, and the

second column represents the maximum allocation to each group. Since the investment in the Energy sector is capped at 80% of the portfolio value, and the investment in the Technology sector is capped at 70%, create the `GroupBounds` matrix using this information.

```
GroupBounds = [0    0.80;
               0    0.70];
```

Use the `Portfolio` object to obtain the vectors and arrays representing the risk, return, and weights for each of the eight portfolios computed along the efficient frontier. A budget constraint is added to ensure that the portfolio weights sum to 1.

```
p = Portfolio('AssetMean', ExpReturn, 'AssetCovar', ExpCovariance);
p = setBounds(p, AssetBounds(1), AssetBounds(2));
p = setBudget(p, 1, 1);
p = setGroups(p, Groups, GroupBounds(:,1), GroupBounds(:,2));
```

```
PortWts = estimateFrontier(p, NumPorts);
```

```
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);
```

PortRisk

```
PortRisk = 8×1
```

```
0.0416
0.0499
0.0624
0.0767
0.0920
0.1100
0.1378
0.1716
```

PortReturn

```
PortReturn = 8×1
```

```
0.1279
0.1361
0.1442
0.1524
0.1605
0.1687
0.1768
0.1850
```

PortWts

```
PortWts = 3×8
```

```
0.7000    0.6031    0.4864    0.3696    0.2529    0.2000    0.2000    0.2000
0.2582    0.3244    0.3708    0.4172    0.4636    0.5738    0.7369    0.9000
0.0418    0.0725    0.1428    0.2132    0.2835    0.2262    0.0631   -0.1000
```

The outputs are represented as columns for the portfolio's risk and return. Portfolio weights are identified as corresponding column vectors in a matrix.

Linear Constraint Equations

This example shows how to use the `Portfolio` object to specify the minimum and maximum investment in various groups.

While the `Portfolio` object allows you to enter a fixed set of constraints related to minimum and maximum values for groups and individual assets, you often need to specify a larger and more general set of constraints when finding the optimal risky portfolio. `Portfolio` also addresses this need, by accepting an arbitrary set of constraints.

Maximum and Minimum Group Exposure

Group	Minimum Exposure	Maximum Exposure
North America	0.30	0.75
Europe	0.10	0.55
Latin America	0.20	0.50
Asia	0.50	0.50

The minimum and maximum exposure in Asia is the same. This means that you require a fixed exposure for this group.

Also assume that the portfolio consists of three different funds. The correspondence between funds and groups is shown in the table below.

Group Membership

Group	Fund 1	Fund 2	Fund 3
North America	X	X	
Europe			X
Latin America	X		
Asia		X	X

Using the information in these two tables, build a mathematical representation of the constraints represented. Assume that the vector of weights representing the exposure of each asset in a portfolio is called $\mathbf{Wts} = [W1 \ W2 \ W3]$.

Specifically

1.	$W1 + W2$	$\geq$	0.30
2.	$W1 + W2$	$\leq$	0.75
3.	$W3$	$\geq$	0.10
4.	$W3$	$\leq$	0.55
5.	$W1$	$\geq$	0.20
6.	$W1$	$\leq$	0.50
7.	$W2 + W3$	$=$	0.50

Since you must represent the information in the form $A \cdot W \leq b$, multiply equation numbers 1, 3 and 5 by -1. Also turn equation number 7 into a set of two inequalities: $W_2 + W_3 \geq 0.50$ and $W_2 + W_3 \leq 0.50$. (The intersection of these two inequalities is the equality itself.) Thus

1.	$-W_1 - W_2$	$\leq$	-0.30
2.	$W_1 + W_2$	$\leq$	0.75
3.	$-W_3$	$\leq$	-0.10
4.	W_3	$\leq$	0.55
5.	$-W_1$	$\leq$	-0.20
6.	W_1	$\leq$	0.50
7.	$-W_2 - W_3$	$\leq$	-0.50
8.	$W_2 + W_3$	$\leq$	0.50

Bringing these equations into matrix notation gives the following:

$$A = \begin{bmatrix} -1 & -1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & -1 \\ 0 & 0 & 1 \\ -1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & -1 & -1 \\ 0 & 1 & 1 \end{bmatrix}$$

$$A = 8 \times 3$$

$$\begin{bmatrix} -1 & -1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & -1 \\ 0 & 0 & 1 \\ -1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & -1 & -1 \\ 0 & 1 & 1 \end{bmatrix}$$

$$b = \begin{bmatrix} -0.30; \\ 0.75; \\ -0.10; \\ 0.55; \\ -0.20; \\ 0.50; \\ -0.50; \\ 0.50 \end{bmatrix}$$

$$b = 8 \times 1$$

$$\begin{bmatrix} -0.3000 \\ 0.7500 \\ -0.1000 \\ 0.5500 \\ -0.2000 \\ 0.5000 \\ -0.5000 \\ 0.5000 \end{bmatrix}$$


```
-0.5000
0.5000
```

One approach to solving this portfolio problem is to explicitly use the `setInequality` function with a `Portfolio` object.

```
ExpReturn = [0.1 0.2 0.15];
ExpCovariance = [ 0.005  -0.010  0.004;
                 -0.010  0.040  -0.002;
                 0.004  -0.002  0.023];

NumPorts = 8;
AssetBounds = [-0.1, 1.1];

p = Portfolio('AssetMean', ExpReturn, 'AssetCovar', ExpCovariance);
p = setBounds(p, AssetBounds(1), AssetBounds(2));
p = setBudget(p, 1, 1);
p = setInequality(p, A, b);
PortWts = estimateFrontier(p, NumPorts);
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);
```

PortRisk

PortRisk = 8×1

```
0.0586
0.0586
0.0586
0.0586
0.0586
0.0586
0.0586
0.0586
```

PortReturn

PortReturn = 8×1

```
0.1375
0.1375
0.1375
0.1375
0.1375
0.1375
0.1375
0.1375
```

PortWts

PortWts = 3×8

```
0.5000  0.5000  0.5000  0.5000  0.5000  0.5000  0.5000  0.5000
0.2500  0.2500  0.2500  0.2500  0.2500  0.2500  0.2500  0.2500
0.2500  0.2500  0.2500  0.2500  0.2500  0.2500  0.2500  0.2500
```

In this case, the constraints allow only one optimum portfolio. Since eight portfolios were requested, all eight portfolios are the same. Note that the solution to this portfolio problem using the

`setInequality` function is the same as using the `setGroups` function in the example “Specifying Group Constraints” on page 3-24.

Specifying Group Constraints

This example shows how to solve a portfolio problem using a `Portfolio` object with group constraints.

The example “Linear Constraint Equations” on page 3-21 defines a constraint matrix that specifies a set of typical scenarios. It defines groups of assets, specifies upper and lower bounds for total allocation in each of these groups, and it sets the total allocation of one group to a fixed value. Constraints like these are common occurrences. The `Portfolio` object enables you to simplify the creation of the constraint matrix for these and other common portfolio requirements.

An alternative approach for solving the portfolio problem is to use the `Portfolio` object to define:

- A `Group` matrix, indicating the assets that belong to each group.
- A `GroupMin` vector, indicating the minimum bounds for each group.
- A `GroupMax` vector, indicating the maximum bounds for each group.

Based on the table Group Membership, build the `Group` matrix, with each row representing a group, and each column representing an asset.

Group Membership

Group	Fund 1	Fund 2	Fund 3
North America	X	X	
Europe			X
Latin America	X		
Asia		X	X

```
Group = [1    1    0;
         0    0    1;
         1    0    0;
         0    1    1];
```

The table Maximum and Minimum Group Exposure has the information to build `GroupMin` and `GroupMax`.

Maximum and Minimum Group Exposure

Group	Minimum Exposure	Maximum Exposure
North America	0.30	0.75
Europe	0.10	0.55
Latin America	0.20	0.50
Asia	0.50	0.50

```
GroupMin = [0.30  0.10  0.20  0.50];
GroupMax = [0.75  0.55  0.50  0.50];
```

Use the `Portfolio` object with the `setInequality` function to obtain the vectors and arrays representing the risk, return, and weights for the portfolios computed along the efficient frontier.

```
ExpReturn = [0.1 0.2 0.15];
ExpCovariance = [ 0.005  -0.010   0.004;
                  -0.010   0.040  -0.002;
                  0.004  -0.002   0.023];
NumPorts = 8;
AssetBounds = [-0.1, 1.1];

p = Portfolio('AssetMean', ExpReturn, 'AssetCovar', ExpCovariance);
p = setBounds(p, AssetBounds(1), AssetBounds(2));
p = setBudget(p, 1, 1);
p = setGroups(p, Group, GroupMin, GroupMax);
PortWts = estimateFrontier(p, NumPorts);
[PortRisk, PortReturn] = estimatePortMoments(p, PortWts);
```

PortRisk

PortRisk = 8×1

```
0.0586
0.0586
0.0586
0.0586
0.0586
0.0586
0.0586
0.0586
```

PortReturn

PortReturn = 8×1

```
0.1375
0.1375
0.1375
0.1375
0.1375
0.1375
0.1375
0.1375
```

PortWts

PortWts = 3×8

```
0.5000  0.5000  0.5000  0.5000  0.5000  0.5000  0.5000  0.5000
0.2500  0.2500  0.2500  0.2500  0.2500  0.2500  0.2500  0.2500
0.2500  0.2500  0.2500  0.2500  0.2500  0.2500  0.2500  0.2500
```

In this case, the constraints allow only one optimum portfolio. Since eight portfolios were requested, all eight portfolios are the same. Note that the solution to this portfolio problem using the `setGroups` function is the same as using the `setInequality` function in the example “Linear Constraint Equations” on page 3-21.

See Also

`Portfolio` | `estimateFrontier` | `estimatePortMoments` | `setInequality` | `setGroups`

Related Examples

- “Setting Default Constraints for Portfolio Weights Using Portfolio Object” on page 4-58
- “Working with 'Simple' Bound Constraints Using Portfolio Object” on page 4-62
- “Working with Budget Constraints Using Portfolio Object” on page 4-65
- “Working with Group Constraints Using Portfolio Object” on page 4-69
- “Working with Group Ratio Constraints Using Portfolio Object” on page 4-72
- “Working with Linear Equality Constraints Using Portfolio Object” on page 4-75
- “Working with Linear Inequality Constraints Using Portfolio Object” on page 4-78
- “Working with Average Turnover Constraints Using Portfolio Object” on page 4-85
- “Working with One-Way Turnover Constraints Using Portfolio Object” on page 4-88
- “Working with Tracking Error Constraints Using Portfolio Object” on page 4-91
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

More About

- Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

Active Returns and Tracking Error Efficient Frontier

This example shows how to minimize the variance of the difference in returns with respect to a given target portfolio.

Suppose that you want to identify an efficient set of portfolios that minimize the variance of the difference in returns with respect to a given target portfolio, subject to a given expected excess return. The mean and standard deviation of this excess return are often called the *active return* and *active risk*, respectively. Active risk is sometimes referred to as the tracking error. Since the objective is to track a given target portfolio as closely as possible, the resulting set of portfolios is sometimes referred to as the *tracking error efficient frontier*.

Specifically, assume that the target portfolio is expressed as an index weight vector, such that the index return series may be expressed as a linear combination of the available assets. This example illustrates how to construct a frontier that minimizes the active risk (tracking error) subject to attaining a given level of return. That is, it computes the tracking error efficient frontier.

One way to construct the tracking error efficient frontier is to explicitly form the target return series and subtract it from the return series of the individual assets. In this manner, you specify the expected mean and covariance of the active returns, and compute the efficient frontier subject to the usual portfolio constraints.

This example works directly with the mean and covariance of the absolute (unadjusted) returns but converts the constraints from the usual absolute weight format to active weight format.

Consider a portfolio of five assets with the following expected returns, standard deviations, and correlation matrix based on absolute weekly asset returns.

```
NumAssets      = 5;

ExpReturn      = [0.2074  0.1971  0.2669  0.1323  0.2535]/100;

Sigmas         = [2.6570  3.6297  3.9916  2.7145  2.6133]/100;

Correlations = [1.0000  0.6092  0.6321  0.5833  0.7304
                0.6092  1.0000  0.8504  0.8038  0.7176
                0.6321  0.8504  1.0000  0.7723  0.7236
                0.5833  0.8038  0.7723  1.0000  0.7225
                0.7304  0.7176  0.7236  0.7225  1.0000];
```

Convert the correlations and standard deviations to a covariance matrix using `corr2cov`.

```
ExpCovariance = corr2cov(Sigmas, Correlations);
```

Assume that the target index portfolio is an equally weighted portfolio formed from the five assets. The sum of index weights equals 1, satisfying the standard full investment budget equality constraint.

```
Index = ones(NumAssets, 1)/NumAssets;
```

Generate an asset constraint matrix using `portcons`. The constraint matrix `AbsConSet` is expressed in absolute format (unadjusted for the index), and is formatted as $[A \ b]$, corresponding to constraints of the form $A*w \leq b$. Each row of `AbsConSet` corresponds to a constraint, and each column corresponds to an asset. Allow no short-selling and full investment in each asset (lower and upper bounds of each asset are 0 and 1, respectively). In particular, note that the first two rows

correspond to the budget equality constraint; the remaining rows correspond to the upper/lower investment bounds.

```
AbsConSet = portcons('PortValue', 1, NumAssets, ...
    'AssetLims', zeros(NumAssets,1), ones(NumAssets,1));
```

Transform the absolute constraints to active constraints with `abs2active`.

```
ActiveConSet = abs2active(AbsConSet, Index);
```

An examination of the absolute and active constraint matrices reveals that they differ only in the last column (the columns corresponding to the b in $A \cdot w \leq b$).

```
[AbsConSet(:,end) ActiveConSet(:,end)]
```

```
ans = 12×2
```

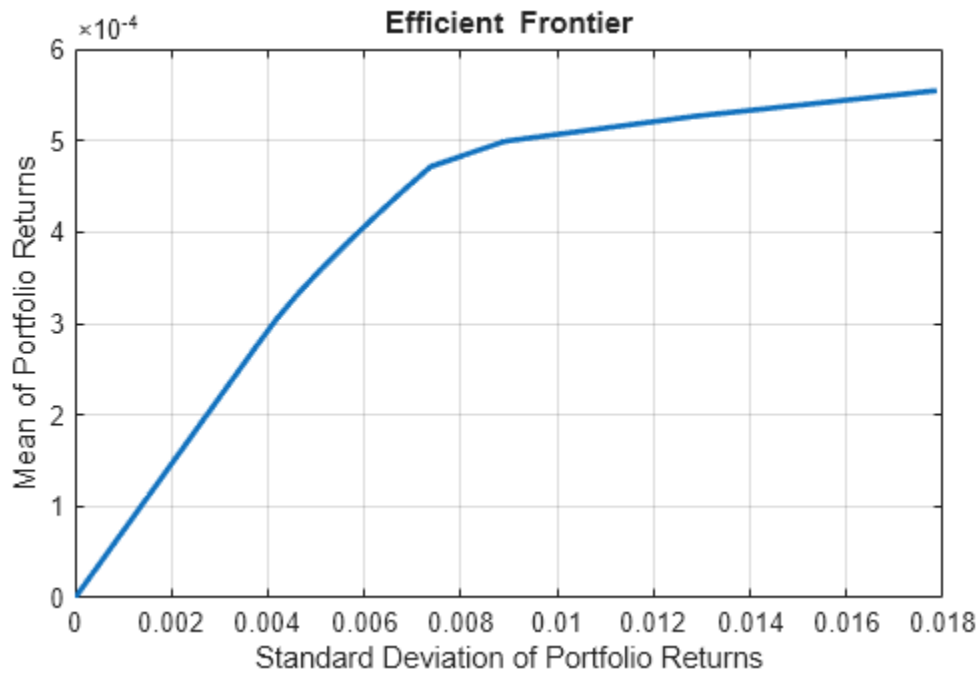
```
    1.0000    0
   -1.0000    0
    1.0000    0.8000
    1.0000    0.8000
    1.0000    0.8000
    1.0000    0.8000
    1.0000    0.8000
    1.0000    0.8000
         0    0.2000
         0    0.2000
         0    0.2000
         0    0.2000
         0    0.2000
         :
```

In particular, note that the sum-to-one absolute budget constraint becomes a sum-to-zero active budget constraint. The general transformation is as follows:

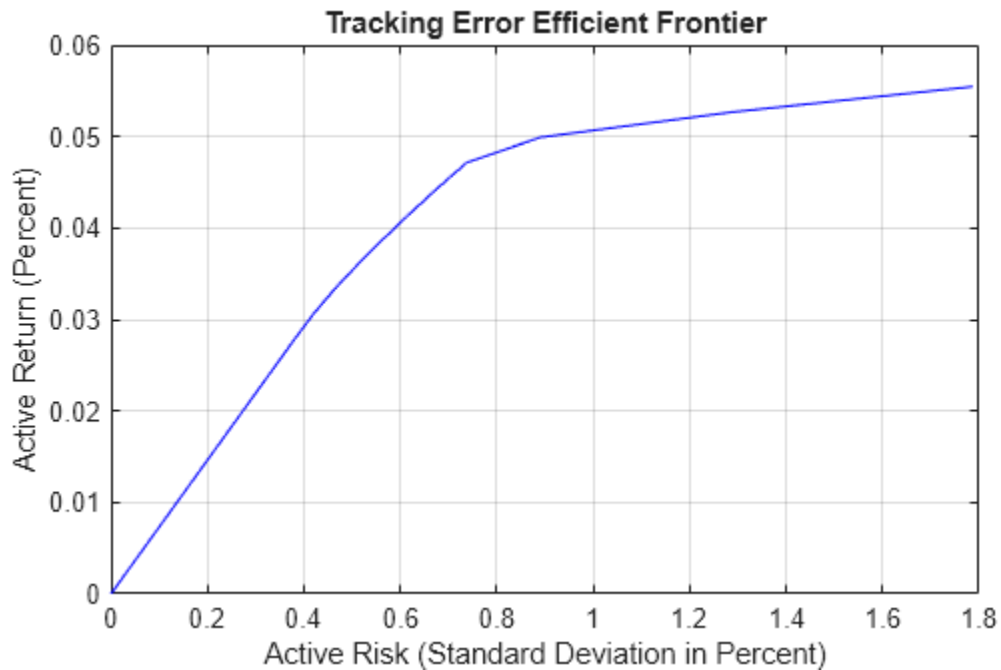
$$b_{\text{active}} = b_{\text{absolute}} - A \times \text{Index}.$$

Construct the `Portfolio` object and plot the tracking error efficient frontier with 21 portfolios.

```
p = Portfolio('AssetMean', ExpReturn, 'AssetCovar', ExpCovariance);
p = p.setInequality(ActiveConSet(:,1:end-1), ActiveConSet(:,end));
[ActiveRisk, ActiveReturn] = p.plotFrontier(21);
```



```
plot(ActiveRisk*100, ActiveReturn*100, 'blue')
grid('on')
xlabel('Active Risk (Standard Deviation in Percent)')
ylabel('Active Return (Percent)')
title('Tracking Error Efficient Frontier')
```



Of particular interest is the lower-left portfolio along the frontier. This zero-risk/zero-return portfolio has a practical economic significance. It represents a full investment in the index portfolio itself. Each

tracking error efficient portfolio (each row in the array `ActiveWeights`) satisfies the active budget constraint, and thus represents portfolio investment allocations with respect to the index portfolio. To convert these allocations to absolute investment allocations, add the index to each efficient portfolio.

```
ActiveWeights = p.estimateFrontier(21)
```

```
ActiveWeights = 5×21
```

-0.0000	-0.0038	-0.0075	-0.0113	-0.0151	-0.0188	-0.0226	-0.0264	-0.0302	-0.0340	-0.0378	-0.0416	-0.0454	-0.0492	-0.0530	-0.0568	-0.0606	-0.0644	-0.0682	-0.0720	-0.0758	-0.0796	-0.0834	-0.0872	-0.0910	-0.0948	-0.0986	-0.1024	-0.1062	-0.1100	-0.1138	-0.1176	-0.1214	-0.1252	-0.1290	-0.1328	-0.1366	-0.1404	-0.1442	-0.1480	-0.1518	-0.1556	-0.1594	-0.1632	-0.1670	-0.1708	-0.1746	-0.1784	-0.1822	-0.1860	-0.1898	-0.1936	-0.1974	-0.2012	-0.2050	-0.2088	-0.2126	-0.2164	-0.2202	-0.2240	-0.2278	-0.2316	-0.2354	-0.2392	-0.2430	-0.2468	-0.2506	-0.2544	-0.2582	-0.2620	-0.2658	-0.2696	-0.2734	-0.2772	-0.2810	-0.2848	-0.2886	-0.2924	-0.2962	-0.3000	-0.3038	-0.3076	-0.3114	-0.3152	-0.3190	-0.3228	-0.3266	-0.3304	-0.3342	-0.3380	-0.3418	-0.3456	-0.3494	-0.3532	-0.3570	-0.3608	-0.3646	-0.3684	-0.3722	-0.3760	-0.3798	-0.3836	-0.3874	-0.3912	-0.3950	-0.3988	-0.4026	-0.4064	-0.4102	-0.4140	-0.4178	-0.4216	-0.4254	-0.4292	-0.4330	-0.4368	-0.4406	-0.4444	-0.4482	-0.4520	-0.4558	-0.4596	-0.4634	-0.4672	-0.4710	-0.4748	-0.4786	-0.4824	-0.4862	-0.4900	-0.4938	-0.4976	-0.5014	-0.5052	-0.5090	-0.5128	-0.5166	-0.5204	-0.5242	-0.5280	-0.5318	-0.5356	-0.5394	-0.5432	-0.5470	-0.5508	-0.5546	-0.5584	-0.5622	-0.5660	-0.5698	-0.5736	-0.5774	-0.5812	-0.5850	-0.5888	-0.5926	-0.5964	-0.6002	-0.6040	-0.6078	-0.6116	-0.6154	-0.6192	-0.6230	-0.6268	-0.6306	-0.6344	-0.6382	-0.6420	-0.6458	-0.6496	-0.6534	-0.6572	-0.6610	-0.6648	-0.6686	-0.6724	-0.6762	-0.6800	-0.6838	-0.6876	-0.6914	-0.6952	-0.6990	-0.7028	-0.7066	-0.7104	-0.7142	-0.7180	-0.7218	-0.7256	-0.7294	-0.7332	-0.7370	-0.7408	-0.7446	-0.7484	-0.7522	-0.7560	-0.7598	-0.7636	-0.7674	-0.7712	-0.7750	-0.7788	-0.7826	-0.7864	-0.7902	-0.7940	-0.7978	-0.8016	-0.8054	-0.8092	-0.8130	-0.8168	-0.8206	-0.8244	-0.8282	-0.8320	-0.8358	-0.8396	-0.8434	-0.8472	-0.8510	-0.8548	-0.8586	-0.8624	-0.8662	-0.8700	-0.8738	-0.8776	-0.8814	-0.8852	-0.8890	-0.8928	-0.8966	-0.9004	-0.9042	-0.9080	-0.9118	-0.9156	-0.9194	-0.9232	-0.9270	-0.9308	-0.9346	-0.9384	-0.9422	-0.9460	-0.9498	-0.9536	-0.9574	-0.9612	-0.9650	-0.9688	-0.9726	-0.9764	-0.9802	-0.9840	-0.9878	-0.9916	-0.9954	-0.9992	-1.0030	-1.0068	-1.0106	-1.0144	-1.0182	-1.0220	-1.0258	-1.0296	-1.0334	-1.0372	-1.0410	-1.0448	-1.0486	-1.0524	-1.0562	-1.0600	-1.0638	-1.0676	-1.0714	-1.0752	-1.0790	-1.0828	-1.0866	-1.0904	-1.0942	-1.0980	-1.1018	-1.1056	-1.1094	-1.1132	-1.1170	-1.1208	-1.1246	-1.1284	-1.1322	-1.1360	-1.1398	-1.1436	-1.1474	-1.1512	-1.1550	-1.1588	-1.1626	-1.1664	-1.1702	-1.1740	-1.1778	-1.1816	-1.1854	-1.1892	-1.1930	-1.1968	-1.2006	-1.2044	-1.2082	-1.2120	-1.2158	-1.2196	-1.2234	-1.2272	-1.2310	-1.2348	-1.2386	-1.2424	-1.2462	-1.2500	-1.2538	-1.2576	-1.2614	-1.2652	-1.2690	-1.2728	-1.2766	-1.2804	-1.2842	-1.2880	-1.2918	-1.2956	-1.2994	-1.3032	-1.3070	-1.3108	-1.3146	-1.3184	-1.3222	-1.3260	-1.3298	-1.3336	-1.3374	-1.3412	-1.3450	-1.3488	-1.3526	-1.3564	-1.3602	-1.3640	-1.3678	-1.3716	-1.3754	-1.3792	-1.3830	-1.3868	-1.3906	-1.3944	-1.3982	-1.4020	-1.4058	-1.4096	-1.4134	-1.4172	-1.4210	-1.4248	-1.4286	-1.4324	-1.4362	-1.4400	-1.4438	-1.4476	-1.4514	-1.4552	-1.4590	-1.4628	-1.4666	-1.4704	-1.4742	-1.4780	-1.4818	-1.4856	-1.4894	-1.4932	-1.4970	-1.5008	-1.5046	-1.5084	-1.5122	-1.5160	-1.5198	-1.5236	-1.5274	-1.5312	-1.5350	-1.5388	-1.5426	-1.5464	-1.5502	-1.5540	-1.5578	-1.5616	-1.5654	-1.5692	-1.5730	-1.5768	-1.5806	-1.5844	-1.5882	-1.5920	-1.5958	-1.5996	-1.6034	-1.6072	-1.6110	-1.6148	-1.6186	-1.6224	-1.6262	-1.6300	-1.6338	-1.6376	-1.6414	-1.6452	-1.6490	-1.6528	-1.6566	-1.6604	-1.6642	-1.6680	-1.6718	-1.6756	-1.6794	-1.6832	-1.6870	-1.6908	-1.6946	-1.6984	-1.7022	-1.7060	-1.7098	-1.7136	-1.7174	-1.7212	-1.7250	-1.7288	-1.7326	-1.7364	-1.7402	-1.7440	-1.7478	-1.7516	-1.7554	-1.7592	-1.7630	-1.7668	-1.7706	-1.7744	-1.7782	-1.7820	-1.7858	-1.7896	-1.7934	-1.7972	-1.8010	-1.8048	-1.8086	-1.8124	-1.8162	-1.8200	-1.8238	-1.8276	-1.8314	-1.8352	-1.8390	-1.8428	-1.8466	-1.8504	-1.8542	-1.8580	-1.8618	-1.8656	-1.8694	-1.8732	-1.8770	-1.8808	-1.8846	-1.8884	-1.8922	-1.8960	-1.8998	-1.9036	-1.9074	-1.9112	-1.9150	-1.9188	-1.9226	-1.9264	-1.9302	-1.9340	-1.9378	-1.9416	-1.9454	-1.9492	-1.9530	-1.9568	-1.9606	-1.9644	-1.9682	-1.9720	-1.9758	-1.9796	-1.9834	-1.9872	-1.9910	-1.9948	-1.9986	-2.0024	-2.0062	-2.0100	-2.0138	-2.0176	-2.0214	-2.0252	-2.0290	-2.0328	-2.0366	-2.0404	-2.0442	-2.0480	-2.0518	-2.0556	-2.0594	-2.0632	-2.0670	-2.0708	-2.0746	-2.0784	-2.0822	-2.0860	-2.0898	-2.0936	-2.0974	-2.1012	-2.1050	-2.1088	-2.1126	-2.1164	-2.1202	-2.1240	-2.1278	-2.1316	-2.1354	-2.1392	-2.1430	-2.1468	-2.1506	-2.1544	-2.1582	-2.1620	-2.1658	-2.1696	-2.1734	-2.1772	-2.1810	-2.1848	-2.1886	-2.1924	-2.1962	-2.2000	-2.2038	-2.2076	-2.2114	-2.2152	-2.2190	-2.2228	-2.2266	-2.2304	-2.2342	-2.2380	-2.2418	-2.2456	-2.2494	-2.2532	-2.2570	-2.2608	-2.2646	-2.2684	-2.2722	-2.2760	-2.2798	-2.2836	-2.2874	-2.2912	-2.2950	-2.2988	-2.3026	-2.3064	-2.3102	-2.3140	-2.3178	-2.3216	-2.3254	-2.3292	-2.3330	-2.3368	-2.3406	-2.3444	-2.3482	-2.3520	-2.3558	-2.3596	-2.3634	-2.3672	-2.3710	-2.3748	-2.3786	-2.3824	-2.3862	-2.3900	-2.3938	-2.3976	-2.4014	-2.4052	-2.4090	-2.4128	-2.4166	-2.4204	-2.4242	-2.4280	-2.4318	-2.4356	-2.4394	-2.4432	-2.4470	-2.4508	-2.4546	-2.4584	-2.4622	-2.4660	-2.4698	-2.4736	-2.4774	-2.4812	-2.4850	-2.4888	-2.4926	-2.4964	-2.5002	-2.5040	-2.5078	-2.5116	-2.5154	-2.5192	-2.5230	-2.5268	-2.5306	-2.5344	-2.5382	-2.5420	-2.5458	-2.5496	-2.5534	-2.5572	-2.5610	-2.5648	-2.5686	-2.5724	-2.5762	-2.5800	-2.5838	-2.5876	-2.5914	-2.5952	-2.5990	-2.6028	-2.6066	-2.6104	-2.6142	-2.6180	-2.6218	-2.6256	-2.6294	-2.6332	-2.6370	-2.6408	-2.6446	-2.6484	-2.6522	-2.6560	-2.6598	-2.6636	-2.6674	-2.6712	-2.6750	-2.6788	-2.6826	-2.6864	-2.6902	-2.6940	-2.6978	-2.7016	-2.7054	-2.7092	-2.7130	-2.7168	-2.7206	-2.7244	-2.7282	-2.7320	-2.7358	-2.7396	-2.7434	-2.7472	-2.7510	-2.7548	-2.7586	-2.7624	-2.7662	-2.7700	-2.7738	-2.7776	-2.7814	-2.7852	-2.7890	-2.7928	-2.7966	-2.8004	-2.8042	-2.8080	-2.8118	-2.8156	-2.8194	-2.8232	-2.8270	-2.8308	-2.8346	-2.8384	-2.8422	-2.8460	-2.8498	-2.8536	-2.8574	-2.8612	-2.8650	-2.8688	-2.8726	-2.8764	-2.8802	-2.8840	-2.8878	-2.8916	-2.8954	-2.8992	-2.9030	-2.9068	-2.9106	-2.9144	-2.9182	-2.9220	-2.9258	-2.9296	-2.9334	-2.9372	-2.9410	-2.9448	-2.9486	-2.9524	-2.9562	-2.9600	-2.9638	-2.9676	-2.9714	-2.9752	-2.9790	-2.9828	-2.9866	-2.9904	-2.9942	-2.9980	-3.0018	-3.0056	-3.0094	-3.0132	-3.0170	-3.0208	-3.0246	-3.0284	-3.0322	-3.0360	-3.0398	-3.0436	-3.0474	-3.0512	-3.0550	-3.0588	-3.0626	-3.0664	-3.0702	-3.0740	-3.0778	-3.0816	-3.0854	-3.0892	-3.0930	-3.0968	-3.1006	-3.1044	-3.1082	-3.1120	-3.1158	-3.1196	-3.1234	-3.1272	-3.1310	-3.1348	-3.1386	-3.1424	-3.1462	-3.1500	-3.1538	-3.1576	-3.1614	-3.1652	-3.1690	-3.1728	-3.1766	-3.1804	-3.1842	-3.1880	-3.1918	-3.1956	-3.1994	-3.2032	-3.2070	-3.2108	-3.2146	-3.2184	-3.2222	-3.2260	-3.2298	-3.2336	-3.2374	-3.2412	-3.2450	-3.2488	-3.2526	-3.2564	-3.2602	-3.2640	-3.2678	-3.2716	-3.2754	-3.2792	-3.2830	-3.2868	-3.2906	-3.2944	-3.2982	-3.3020	-3.3058	-3.3096	-3.3134	-3.3172	-3.3210	-3.3248	-3.3286	-3.3324	-3.3362	-3.3400	-3.3438	-3.3476	-3.3514	-3.3552	-3.3590	-3.3628	-3.3666	-3.3704	-3.3742	-3.3780	-3.3818	-3.3856	-3.3894	-3.3932	-3.3970	-3.4008	-3.4046	-3.4084	-3.4122	-3.4160	-3.4198	-3.4236	-3.4274	-3.4312	-3.4350	-3.4388	-3.4426	-3.4464	-3.4502	-3.4540	-3.4578	-3.4616	-3.4654	-3.4692	-3.4730	-3.4768	-3.4806	-3.4844	-3.4882	-3.4920	-3.4958	-3.4996	-3.5034	-3.5072	-3.5110	-3.5148	-3.5186	-3.5224	-3.5262	-3.5300	-3.5338	-3.5376	-3.5414	-3.5452	-3.5490	-3.5528	-3.5566	-3.5604	-3.5642	-3.5680	-3.5718	-3.5756	-3.5794	-3.5832	-3.5870	-3.5908	-3.5946	-3.5984	-3.6022	-3.6060	-3.6098	-3.6136	-3.6174	-3.6212	-3.6250	-3.6288	-3.6326	-3.6364	-3.6402	-3.6440	-3.6478	-3.6516	-3.6554	-3.6592	-3.6630	-3.6668	-3.6706	-3.6744	-3.6782	-3.6820	-3.6858	-3.6896	-3.6934	-3.6972	-3.7010	-3.7048	-3.7086	-3.7124	-3.7162	-3.7200	-3.7238	-3.7276	-3.7314	-3.7352	-3.7390	-3.7428	-3.7466	-3.7504	-3.7542	-3.7580	-3.7618	-3.7656	-3.7694	-3.7732	-3.7770	-3.7808	-3.7846	-3.7884	-3.7922	-3.7960	-3.7998	-3.8036	-3.8074	-3.8112	-3.8150	-3.8188	-3.8226	-3.8264	-3.8302	-3.8340	-3.8378	-3.8416	-3.8454	-3.8492	-3.8530	-3.8568	-3.8606	-3.8644	-3.8682	-3.8720	-3.8758	-3.8796	-3.8834	-3.8872	-3.8910	-3.8948	-3.8986	-3.9024	-3.9062	-3.9100	-3.9138	-3.9176	-3.9214	-3.9252	-3.9290	-3.9328	-3.9366	-3.9404	-3.9442	-3.9480	-3.9518	-3.9556	-3.9594	-3.9632	-3.9670	-3.9708	-3.9746	-3.9784	-3.9822	-3.9860	-3.9898	-3.9936	-3.9974	-4.0012	-4.0050	-4.0088	-4.0126	-4.0164	-4.0202	-4.0240	-4.0278	-4.0316	-4.0354	-4.0392	-4.0430	-4.0468	-4.0506	-4.0544	-4.0582	-4.0620	-4.0658	-4.0696	-4.0734	-4.0772	-4.0810	-4.0848	-4.0886	-4.0924	-4.0962	-4.1000	-4.1038	-4.1076	-4.1114	-4.1152	-4.1190	-4.1228	-4.1266	-4.1304	-4.1342	-4.1380	-4.1418	-4.1456	-4.1494	-4.1532	-4.1570	-4.1608	-4.1646	-4.1684	-4.1722	-4.1760	-4.1798	-4.1836	-4.1874
---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------

Mean-Variance Portfolio Optimization Tools

- “Portfolio Optimization Theory” on page 4-4
- “Supported Constraints for Portfolio Optimization Using Portfolio Objects” on page 4-9
- “Default Portfolio Problem” on page 4-17
- “Portfolio Object Workflow” on page 4-18
- “Portfolio Object” on page 4-20
- “Creating the Portfolio Object” on page 4-25
- “Common Operations on the Portfolio Object” on page 4-33
- “Setting Up an Initial or Current Portfolio” on page 4-37
- “Setting Up a Tracking Portfolio” on page 4-40
- “Asset Returns and Moments of Asset Returns Using Portfolio Object” on page 4-42
- “Working with a Riskless Asset” on page 4-52
- “Working with Transaction Costs” on page 4-54
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Working with 'Simple' Bound Constraints Using Portfolio Object” on page 4-62
- “Working with Budget Constraints Using Portfolio Object” on page 4-65
- “Working with Conditional Budget Constraints Using Portfolio Object” on page 4-67
- “Working with Group Constraints Using Portfolio Object” on page 4-69
- “Working with Group Ratio Constraints Using Portfolio Object” on page 4-72
- “Working with Linear Equality Constraints Using Portfolio Object” on page 4-75
- “Working with Linear Inequality Constraints Using Portfolio Object” on page 4-78
- “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81
- “Working with Average Turnover Constraints Using Portfolio Object” on page 4-85
- “Working with One-Way Turnover Constraints Using Portfolio Object” on page 4-88
- “Working with Tracking Error Constraints Using Portfolio Object” on page 4-91
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Obtaining Portfolios Along the Entire Efficient Frontier” on page 4-99
- “Obtaining Endpoints of the Efficient Frontier” on page 4-102
- “Obtaining Efficient Portfolios for Target Returns” on page 4-105
- “Obtaining Efficient Portfolios for Target Risks” on page 4-108
- “Efficient Portfolio That Maximizes Sharpe Ratio” on page 4-111
- “Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization” on page 4-114

- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Plotting the Efficient Frontier for a Portfolio Object” on page 4-125
- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “When to Use Portfolio Objects Over Optimization Toolbox” on page 4-132
- “Comparison of Methods for Covariance Estimation” on page 4-138
- “Choose MINLP Solvers for Portfolio Problems” on page 4-140
- “Troubleshooting Portfolio Optimization Results” on page 4-145
- “Role of Convexity in Portfolio Problems” on page 4-157
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Portfolio Analysis with Turnover Constraints” on page 4-211
- “Leverage in Portfolio Optimization with a Risk-Free Asset” on page 4-217
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Backtest Investment Strategies Using Financial Toolbox” on page 4-238
- “Backtest Investment Strategies with Trading Signals” on page 4-251
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify ESG Portfolios” on page 4-271
- “Risk Budgeting Portfolio” on page 4-286
- “Backtest Using Risk-Based Equity Indexation” on page 4-291
- “Create Hierarchical Risk Parity Portfolio” on page 4-296
- “Backtest Strategies Using Deep Learning” on page 4-302
- “Backtest with Brinson Attribution to Evaluate Portfolio Performance” on page 4-315
- “Analyze Performance Attribution Using Brinson Model” on page 4-323
- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Solve Tracking Error Portfolio Problems” on page 4-343
- “Solve Problem for Minimum Tracking Error with Net Return Constraint” on page 4-349
- “Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty” on page 4-351
- “Risk Parity or Budgeting with Constraints” on page 4-357
- “Single Period Goal-Based Wealth Management” on page 4-362
- “Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods” on page 4-367
- “Multiperiod Goal-Based Wealth Management Using Reinforcement Learning” on page 4-379
- “Compare Performance of Covariance Denoising with Factor Modeling Using Backtesting” on page 4-394
- “Mixed-Integer Mean-Variance Portfolio Optimization Problem” on page 4-402
- “Deep Reinforcement Learning for Optimal Trade Execution” on page 4-407

- “Backtest Investment Strategies Using datetime and calendarDuration” on page 4-451
- “Adding Constraints to Satisfy UCITS Directive” on page 4-457

Portfolio Optimization Theory

In this section...

“Portfolio Optimization Problems” on page 4-4
 “Portfolio Problem Specification” on page 4-4
 “Return Proxy” on page 4-5
 “Risk Proxy” on page 4-6

Portfolio Optimization Problems

Portfolio optimization problems involve identifying portfolios that satisfy three criteria:

- Minimize a proxy for risk.
- Match or exceed a proxy for return.
- Satisfy basic feasibility requirements.

Portfolios are points from a feasible set of assets that constitute an asset universe. A portfolio specifies either holdings or weights in each individual asset in the asset universe. The convention is to specify portfolios in terms of weights, although the portfolio optimization tools work with holdings as well.

The set of feasible portfolios is necessarily a nonempty, closed, and bounded set. The proxy for risk is a function that characterizes either the variability or losses associated with portfolio choices. The proxy for return is a function that characterizes either the gross or net benefits associated with portfolio choices. The terms “risk” and “risk proxy” and “return” and “return proxy” are interchangeable. The fundamental insight of Markowitz (see “Portfolio Optimization” on page A-5) is that the goal of the portfolio choice problem is to seek minimum risk for a given level of return and to seek maximum return for a given level of risk. Portfolios satisfying these criteria are efficient portfolios and the graph of the risks and returns of these portfolios forms a curve called the efficient frontier.

Portfolio Problem Specification

To specify a portfolio optimization problem, you need the following:

- Proxy for portfolio return (μ)
- Proxy for portfolio risk (σ)
- Set of feasible portfolios (X), called a portfolio set

Financial Toolbox has three objects to solve specific types of portfolio optimization problems:

- The `Portfolio` object supports mean-variance portfolio optimization (see Markowitz [46], [47] at “Portfolio Optimization” on page A-5). This object has either gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set.
- The `PortfolioCVaR` object implements what is known as conditional value-at-risk portfolio optimization (see Rockafellar and Uryasev [48], [49] at “Portfolio Optimization” on page A-5), which is referred to as CVaR portfolio optimization. CVaR portfolio optimization works with the

same return proxies and portfolio sets as mean-variance portfolio optimization but uses conditional value-at-risk of portfolio returns as the risk proxy.

- The `PortfolioMAD` object implements what is known as mean-absolute deviation portfolio optimization (see Konno and Yamazaki [50] at “Portfolio Optimization” on page A-5), which is referred to as MAD portfolio optimization. MAD portfolio optimization works with the same return proxies and portfolio sets as mean-variance portfolio optimization but uses mean-absolute deviation portfolio returns as the risk proxy.

Return Proxy

The proxy for portfolio return is a function $\mu: X \rightarrow R$ on a portfolio set $X \subset R^n$ that characterizes the rewards associated with portfolio choices. Usually, the proxy for portfolio return has two general forms, gross and net portfolio returns. Both portfolio return forms separate the risk-free rate r_0 so that the portfolio $x \in X$ contains only risky assets.

Regardless of the underlying distribution of asset returns, a collection of S asset returns $y_1, \dots, y_S$ has a mean of asset returns

$$m = \frac{1}{S} \sum_{s=1}^S y_s,$$

and (sample) covariance of asset returns

$$C = \frac{1}{S-1} \sum_{s=1}^S (y_s - m)(y_s - m)^T.$$

These moments (or alternative estimators that characterize these moments) are used directly in mean-variance portfolio optimization to form proxies for portfolio risk and return.

Gross Portfolio Returns

The gross portfolio return for a portfolio $x \in X$ is

$$\mu(x) = r_0 + (m - r_0 \mathbf{1})^T x,$$

where:

r_0 is the risk-free rate (scalar).

m is the mean of asset returns (n vector).

If the portfolio weights sum to 1, the risk-free rate is irrelevant. The properties in the `Portfolio` object to specify gross portfolio returns are:

- `RiskFreeRate` for r_0
- `AssetMean` for m

Net Portfolio Returns

The net portfolio return for a portfolio $x \in X$ is

$$\mu(x) = r_0 + (m - r_0 \mathbf{1})^T x - b^T \max\{0, x - x_0\} - s^T \max\{0, x_0 - x\},$$

where:

r_0 is the risk-free rate (scalar).

m is the mean of asset returns (n vector).

b is the proportional cost to purchase assets (n vector).

s is the proportional cost to sell assets (n vector).

You can incorporate fixed transaction costs in this model also. Though in this case, it is necessary to incorporate prices into such costs. The properties in the `Portfolio` object to specify net portfolio returns are:

- `RiskFreeRate` for r_0
- `AssetMean` for m
- `InitPort` for x_0
- `BuyCost` for b
- `SellCost` for s

Risk Proxy

The proxy for portfolio risk is a function $\sigma: X \rightarrow R$ on a portfolio set $X \subset R^n$ that characterizes the risks associated with portfolio choices.

Mean Variance

The mean variance of portfolio returns for a portfolio $x \in X$ is

$$\text{Variance}(x) = x^T C x$$

where C is the covariance of asset returns (n -by- n positive-semidefinite matrix).

The property in the `Portfolio` object to specify the variance of portfolio returns is `AssetCovar` for C .

Although the risk proxy in mean-variance portfolio optimization is the variance of portfolio returns, the square root, which is the standard deviation of portfolio returns, is often reported and displayed. Moreover, this quantity is often called the “risk” of the portfolio. For details, see Markowitz (“Portfolio Optimization” on page A-5).

Conditional Value-at-Risk

The conditional value-at-risk for a portfolio $x \in X$, which is also known as expected shortfall, is defined as

$$CVaR_\alpha(x) = \frac{1}{1 - \alpha} \int_{f(x, y) \geq VaR_\alpha(x)} f(x, y) p(y) dy,$$

where:

α is the probability level such that $0 < \alpha < 1$.

$f(x, y)$ is the loss function for a portfolio x and asset return y .

$p(y)$ is the probability density function for asset return y .

VaR_α is the value-at-risk of portfolio x at probability level α .

The value-at-risk is defined as

$$VaR_\alpha(x) = \min\{\gamma: \Pr[f(x, Y) \leq \gamma] \geq \alpha\}.$$

An alternative formulation for CVaR has the form:

$$CVaR_\alpha(x) = VaR_\alpha(x) + \frac{1}{1-\alpha} \int_{R^n} \max\{0, (f(x, y) - VaR_\alpha(x))\} p(y) dy$$

The choice for the probability level α is typically 0.9 or 0.95. Choosing α implies that the value-at-risk $VaR_\alpha(x)$ for portfolio x is the portfolio return such that the probability of portfolio returns falling below this level is $(1 - \alpha)$. Given $VaR_\alpha(x)$ for a portfolio x , the conditional value-at-risk of the portfolio is the expected loss of portfolio returns above the value-at-risk return.

Note Value-at-risk is a positive value for losses so that the probability level α indicates the probability that portfolio returns are below the negative of the value-at-risk.

To describe the probability distribution of returns, the `PortfolioCVaR` object takes a finite sample of return scenarios y_s , with $s = 1, \dots, S$. Each y_s is an n vector that contains the returns for each of the n assets under the scenario s . This sample of S scenarios is stored as a scenario matrix of size S -by- n . Then, the risk proxy for CVaR portfolio optimization, for a given portfolio $x \in X$ and $\alpha \in (0, 1)$, is computed as

$$CVaR_\alpha(x) = VaR_\alpha(x) + \frac{1}{(1-\alpha)S} \sum_{s=1}^S \max\{0, -y_s^T x - VaR_\alpha(x)\}$$

The value-at-risk, $VaR_\alpha(x)$, is estimated whenever the CVaR is estimated. The loss function is $f(x, y_s) = -y_s^T x$, which is the portfolio loss under scenario s .

Under this definition, VaR and CVaR are sample estimators for VaR and CVaR based on the given scenarios. Better scenario samples yield more reliable estimates of VaR and CVaR.

For more information, see Rockafellar and Uryasev [48], [49], and Cornuejols and Tütüncü, [51], at "Portfolio Optimization" on page A-5.

Mean Absolute-Deviation

The mean-absolute-deviation (MAD) for a portfolio $x \in X$ is defined as

$$MAD(x) = \frac{1}{S} \sum_{s=1}^S |(y_s - m)^T x|$$

where:

y_s are asset returns with scenarios $s = 1, \dots, S$ (S collection of n vectors).

$f(x,y)$ is the loss function for a portfolio x and asset return y .

m is the mean of asset returns (n vector).

such that

$$m = \frac{1}{S} \sum_{s=1}^S y_s$$

For more information, see Konno and Yamazaki [50] at “Portfolio Optimization” on page A-5.

See Also

[Portfolio](#) | [PortfolioCVaR](#) | [PortfolioMAD](#)

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

More About

- [Portfolio](#)
- “Portfolio Object Workflow” on page 4-18
- Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9
- “Default Portfolio Problem” on page 4-17
- “Role of Convexity in Portfolio Problems” on page 4-157

External Websites

- [Getting Started with Portfolio Optimization \(4 min 13 sec\)](#)
- [MATLAB for Advanced Portfolio Construction and Stock Selection Models \(30 min 28 sec\)](#)

Supported Constraints for Portfolio Optimization Using Portfolio Objects

The final element for a complete specification of a portfolio optimization problem is the set of feasible portfolios, which is called a portfolio set. A portfolio set $X \subset R^n$ is specified by construction as the intersection of sets formed by a collection of constraints on portfolio weights. A portfolio set necessarily and sufficiently must be a nonempty, closed, and bounded set.

When setting up your portfolio set, ensure that the portfolio set satisfies these conditions. The most basic or “default” portfolio set requires portfolio weights to be nonnegative (using the lower-bound constraint) and to sum to 1 (using the budget constraint). The most general portfolio set handled by the portfolio optimization tools (`Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` objects) can have any of these constraints:

- Linear inequality constraints
- Linear equality constraints
- 'Simple' Bound constraints
- 'Conditional' Bound constraints
- Budget constraints
- Conditional budget constraints
- Group constraints
- Group ratio constraints
- Average turnover constraints
- One-way turnover constraints
- Tracking error constraints (only for `Portfolio` object)
- Cardinality constraints

Linear Inequality Constraints

Linear inequality constraints are general linear constraints that model relationships among portfolio weights that satisfy a system of inequalities. Use `setInequality` to set linear inequality constraints. Linear inequality constraints take the form

$$A_I x \leq b_I$$

where:

x is the portfolio (n vector).

A_I is the linear inequality constraint matrix (n_I -by- n matrix).

b_I is the linear inequality constraint vector (n_I vector).

n is the number of assets in the universe and n_I is the number of constraints.

`Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` object properties to specify linear inequality constraints are:

- `AInequality` for A_I
- `bInequality` for b_I
- `NumAssets` for n

The default is to ignore these constraints.

Linear Equality Constraints

Linear equality constraints are general linear constraints that model relationships among portfolio weights that satisfy a system of equalities. Use `setEquality` to set linear equality constraints.

Linear equality constraints take the form

$$A_E x = b_E$$

where:

x is the portfolio (n vector).

A_E is the linear equality constraint matrix (n_E -by- n matrix).

b_E is the linear equality constraint vector (n_E vector).

n is the number of assets in the universe and n_E is the number of constraints.

`Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` object properties to specify linear equality constraints are:

- `AEquality` for A_E
- `BEquality` for b_E
- `NumAssets` for n

The default is to ignore these constraints.

'Simple' Bound Constraints

'Simple' Bound constraints are specialized linear constraints that confine portfolio weights to fall either above or below specific bounds. Use `setBounds` to specify bound constraints with a `'Simple' BoundType`. Since every portfolio set must be bounded, it is often a good practice, albeit not necessary, to set explicit bounds for the portfolio problem. To obtain explicit 'Simple' bounds for a given portfolio set, use the `estimateBounds` function. Bound constraints take the form

$$l_B \leq x \leq u_B$$

where:

x is the portfolio (n vector).

l_B is the lower-bound constraint (n vector).

u_B is the upper-bound constraint (n vector).

n is the number of assets in the universe.

Portfolio, PortfolioCVaR, and PortfolioMAD object properties to specify bound constraints are:

- LowerBound for l_B
- UpperBound for u_B
- NumAssets for n

The default is to ignore these constraints.

The default portfolio optimization problem (see “Default Portfolio Problem” on page 4-17) has $l_B = 0$ with u_B set implicitly through a budget constraint.

'Conditional' Bound Constraints

'Conditional' Bound constraints, also called semicontinuous constraints, are mixed-integer linear constraints that confine portfolio weights to fall either above or below specific bounds *if* the asset is selected; otherwise, the value of the asset is zero. Use `setBounds` to specify bound constraints with a 'Conditional' BoundType. To mathematically formulate this type of constraints, a binary variable v_i is needed. $v_i = 0$ indicates that asset i is not selected and v_i indicates that the asset was selected. Thus

$$l_i v_i \leq x_i \leq u_i v_i$$

where

x is the portfolio (n vector).

l is the conditional lower-bound constraint (n vector).

u is the conditional upper-bound constraint (n vector).

n is the number of assets in the universe.

Portfolio, PortfolioCVaR, and PortfolioMAD object properties to specify the bound constraint are:

- LowerBound for l_B
- UpperBound for u_B
- NumAssets for n

The default is to ignore this constraint.

Budget Constraints

Budget constraints are specialized linear constraints that confine the sum of portfolio weights to fall either above or below specific bounds. Use `setBudget` to set budget constraints. The constraints take the form

$$l_S \leq 1^T x \leq u_S$$

where:

x is the portfolio (n vector).

$\mathbf{1}$ is the vector of ones (n vector).

l_S is the lower-bound budget constraint (scalar).

u_S is the upper-bound budget constraint (scalar).

n is the number of assets in the universe.

`Portfolio`, `PortfolioCVar`, and `PortfolioMAD` object properties to specify budget constraints are:

- `LowerBudget` for l_S
- `UpperBudget` for u_S
- `NumAssets` for n

The default is to ignore this constraint.

The default portfolio optimization problem (see “Default Portfolio Problem” on page 4-17) has $l_S = u_S = 1$, which means that the portfolio weights sum to 1. If the portfolio optimization problem includes possible movements in and out of cash, the budget constraint specifies how far portfolios can go into cash. For example, if $l_S = 0$ and $u_S = 1$, then the portfolio can have 0-100% invested in cash. If cash is to be a portfolio choice, set `RiskFreeRate` (r_0) to a suitable value (see “Return Proxy” on page 4-5 and “Working with a Riskless Asset” on page 4-52).

Conditional Budget Constraints

Conditional budget constraints is a logical constraint that bounds the maximum amount of the portfolio that can be invested in assets exceeding a given threshold. This constraint supports the “Undertakings for Collective Investment in Transferable Securities” on page 15-1452 (UCITS) directive. Use `setConditionalBudget` to set conditional budget constraints. To mathematically formulate this type of constraints, a binary variable w_i is needed. If $w_i = 0$ the asset is below the specified threshold, otherwise the asset is above that threshold which is represented as

$$w_i = 0 \rightarrow x_i \leq \alpha_i$$

$$\sum_i x_i w_i \leq \gamma$$

where:

x is an n vector representing the portfolio.

α is a scalar or n representing the threshold above which assets are accounted for in the conditional budget (scalar or n vector).

γ is a scalar representing the maximum amount of the portfolio that can be invested in assets exceeding α .

Group Constraints

Group constraints are specialized linear constraints that enforce “membership” among groups of assets. Use `setGroups` to set group constraints. The constraints take the form

$$l_G \leq Gx \leq u_G$$

where:

x is the portfolio (n vector).

l_G is the lower-bound group constraint (n_G vector).

u_G is the upper-bound group constraint (n_G vector).

G is the matrix of group membership indexes (n_G -by- n matrix).

Each row of G identifies which assets belong to a group associated with that row. Each row contains either 0s or 1s with 1 indicating that an asset is part of the group or 0 indicating that the asset is not part of the group.

`Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` object properties to specify group constraints are:

- `GroupMatrix` for G
- `LowerGroup` for l_G
- `UpperGroup` for u_G
- `NumAssets` for n

The default is to ignore these constraints.

Group Ratio Constraints

Group ratio constraints are specialized linear constraints that enforce relationships among groups of assets. Use `setGroupRatio` to set group ratio constraints. The constraints take the form

$$l_{Ri}(G_B x)_i \leq (G_A x)_i \leq u_{Ri}(G_B x)_i$$

for $i = 1, \dots, n_R$ where:

x is the portfolio (n vector).

l_R is the vector of lower-bound group ratio constraints (n_R vector).

u_R is the vector matrix of upper-bound group ratio constraints (n_R vector).

G_A is the matrix of base group membership indexes (n_R -by- n matrix).

G_B is the matrix of comparison group membership indexes (n_R -by- n matrix).

n is the number of assets in the universe and n_R is the number of constraints.

Each row of G_A and G_B identifies which assets belong to a base and comparison group associated with that row.

Each row contains either 0s or 1s with 1 indicating that an asset is part of the group or 0 indicating that the asset is not part of the group.

`Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` object properties to specify group ratio constraints are:

- GroupA for G_A
- GroupB for G_B
- LowerRatio for l_R
- UpperRatio for u_R
- NumAssets for n

The default is to ignore these constraints.

Average Turnover Constraints

Turnover constraint is a linear absolute value constraint that ensures estimated optimal portfolios differ from an initial portfolio by no more than a specified amount. Although portfolio turnover is defined in many ways, the turnover constraints implemented in Financial Toolbox compute portfolio turnover as the average of purchases and sales. Use `setTurnover` to set average turnover constraints. Average turnover constraints take the form

$$\frac{1}{2} \mathbf{1}^T |x - x_0| \leq \tau$$

where:

x is the portfolio (n vector).

$\mathbf{1}$ is the vector of ones (n vector).

x_0 is the initial portfolio (n vector).

τ is the upper bound for turnover (scalar).

n is the number of assets in the universe.

`Portfolio`, `PortfolioCVar`, and `PortfolioMAD` object properties to specify the average turnover constraint are:

- Turnover for τ
- InitPort for x_0
- NumAssets for n

The default is to ignore this constraint.

One-Way Turnover Constraints

One-way turnover constraints ensure that estimated optimal portfolios differ from an initial portfolio by no more than specified amounts according to whether the differences are purchases or sales. Use `setOneWayTurnover` to set one-way turnover constraints. The constraints take the forms

$$\mathbf{1}^T \max\{0, x - x_0\} \leq \tau_B$$

$$\mathbf{1}^T \max\{0, x_0 - x\} \leq \tau_S$$

where:

x is the portfolio (n vector)

$\mathbf{1}$ is the vector of ones (n vector).

x_0 is the Initial portfolio (n vector).

τ_B is the upper bound for turnover constraint on purchases (scalar).

τ_S is the upper bound for turnover constraint on sales (scalar).

To specify one-way turnover constraints, use the following properties in the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object:

- `BuyTurnover` for τ_B
- `SellTurnover` for τ_S
- `InitPort` for x_0

The default is to ignore this constraint.

Note The average turnover constraint (see “Working with Average Turnover Constraints Using Portfolio Object” on page 4-85) with τ is not a combination of the one-way turnover constraints with $\tau = \tau_B = \tau_S$.

Tracking Error Constraints

Tracking error constraint, within a portfolio optimization framework, is an additional constraint to specify the set of feasible portfolios known as a portfolio set. Use `setTrackingError` to set tracking error constraints. The tracking-error constraint has the form

$$(x - x_T)^T C (x - x_T) \leq \tau_T^2$$

where:

x is the portfolio (n vector).

x_T is the tracking portfolio against which risk is to be measured (n vector).

C is the covariance of asset returns.

τ_T is the upper bound for tracking error (scalar).

n is the number of assets in the universe.

`Portfolio` object properties to specify the average turnover constraint are:

- `TrackingPort` for x_T
- `TrackingError` for τ_T

The default is to ignore this constraint.

Note The tracking error constraints can be used with any of the other supported constraints in the `Portfolio` object without restrictions. However, since the portfolio set necessarily and sufficiently

must be a non-empty compact set, the application of a tracking error constraint may result in an empty portfolio set. Use `estimateBounds` to confirm that the portfolio set is non-empty and compact.

Cardinality Constraints

Cardinality constraint limits the number of assets in the optimal allocation for an `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. Use `setMinMaxNumAssets` to specify the 'MinNumAssets' and 'MaxNumAssets' constraints. To mathematically formulate this type of constraints, a binary variable v_i is needed. $v_i = 0$ indicates that asset i is not selected and $v_i = 1$ indicates that the asset was selected. Thus

$$\text{MinNumAssets} \leq \sum_{i=1}^{\text{NumAssets}} v_i \leq \text{MaxNumAssets}$$

The default is to ignore this constraint.

See Also

`Portfolio` | `PortfolioCVaR` | `PortfolioMAD`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231

More About

- `Portfolio`
- “Portfolio Object Workflow” on page 4-18
- “Default Portfolio Problem” on page 4-17

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Default Portfolio Problem

The default portfolio optimization problem has a risk and return proxy associated with a given problem, and a portfolio set that specifies portfolio weights to be nonnegative and to sum to 1. The lower bound combined with the budget constraint is sufficient to ensure that the portfolio set is nonempty, closed, and bounded. The default portfolio optimization problem characterizes a long-only investor who is fully invested in a collection of assets.

- For mean-variance portfolio optimization, it is sufficient to set up the default problem. After setting up the problem, data in the form of a mean and covariance of asset returns are then used to solve portfolio optimization problems.
- For conditional value-at-risk portfolio optimization, the default problem requires the additional specification of a probability level that must be set explicitly. Generally, “typical” values for this level are 0.90 or 0.95. After setting up the problem, data in the form of scenarios of asset returns are then used to solve portfolio optimization problems.
- For MAD portfolio optimization, it is sufficient to set up the default problem. After setting up the problem, data in the form of scenarios of asset returns are then used to solve portfolio optimization problems.

See Also

[Portfolio](#) | [PortfolioCVaR](#) | [PortfolioMAD](#)

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- [Portfolio](#)
- [Supported Constraints for Portfolio Optimization Using Portfolio Objects](#) on page 4-9
- [“Portfolio Object Workflow”](#) on page 4-18

External Websites

- [Getting Started with Portfolio Optimization](#) (4 min 13 sec)
- [MATLAB for Advanced Portfolio Construction and Stock Selection Models](#) (30 min 28 sec)

Portfolio Object Workflow

The Portfolio object workflow for creating and modeling a mean-variance portfolio is:

1 Create a Portfolio.

Create a `Portfolio` object for mean-variance portfolio optimization. For more information, see “Creating the Portfolio Object” on page 4-25.

2 Estimate the mean and covariance for returns.

Evaluate the mean and covariance for portfolio asset returns, including assets with missing data and financial time series data. For more information, see “Asset Returns and Moments of Asset Returns Using Portfolio Object” on page 4-42.

3 Specify the Portfolio Constraints.

Define the constraints for portfolio assets such as linear equality and inequality, bound, budget, conditional budget, group, group ratio, turnover, tracking error, 'Conditional' `BoundType`, and `MinNumAssets`, `MaxNumAssets` constraints. For more information, see “Working with Portfolio Constraints Using Defaults” on page 4-58 and “Working with 'Conditional' `BoundType`, `MinNumAssets`, and `MaxNumAssets` Constraints Using Portfolio Objects” on page 4-81.

4 Validate the Portfolio.

Identify errors for the portfolio specification. For more information, see “Validate the Portfolio Problem for Portfolio Object” on page 4-94.

5 Estimate the efficient portfolios and frontiers.

Analyze the efficient portfolios and efficient frontiers for a portfolio. For more information, see “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98 and “Estimate Efficient Frontiers for Portfolio Object” on page 4-122.

Alternatively, you can estimate an optimal portfolio with a user-defined objective function for a `Portfolio` object. For details on using `estimateCustomObjectivePortfolio` to specify a user-defined objective function, see “Solver Guidelines for Custom Objective Problems Using Portfolio Objects” on page 4-119.

6 Postprocess the results.

Use the efficient portfolios and efficient frontiers results to set up trades. For more information, see “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130.

For an example of this workflow, see “Asset Allocation Case Study” on page 4-180 and “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161.

See Also

Related Examples

- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222

- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Optimization Theory” on page 4-4
- “Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization” on page 4-114

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Portfolio Object

In this section...

“Portfolio Object Properties and Functions” on page 4-20
 “Working with Portfolio Objects” on page 4-20
 “Setting and Getting Properties” on page 4-20
 “Displaying Portfolio Objects” on page 4-21
 “Saving and Loading Portfolio Objects” on page 4-21
 “Estimating Efficient Portfolios and Frontiers” on page 4-21
 “Arrays of Portfolio Objects” on page 4-22
 “Subclassing Portfolio Objects” on page 4-23
 “Conventions for Representation of Data” on page 4-23

Portfolio Object Properties and Functions

The `Portfolio` object implements mean-variance portfolio optimization. Every property and function of the `Portfolio` object is public, although some properties and functions are hidden. See `Portfolio` for the properties and functions of the `Portfolio` object. The `Portfolio` object is a value object where every instance of the object is a distinct version of the object. Since the `Portfolio` object is also a MATLAB object, it inherits the default functions associated with MATLAB objects.

Working with Portfolio Objects

The `Portfolio` object and its functions are an interface for mean-variance portfolio optimization. So, almost everything you do with the `Portfolio` object can be done using the associated functions. The basic workflow is:

- 1 Design your portfolio problem.
- 2 Use `Portfolio` to create the `Portfolio` object or use the various `set` functions to set up your portfolio problem.
- 3 Use estimate functions to solve your portfolio problem.

In addition, functions are available to help you view intermediate results and to diagnose your computations. Since MATLAB features are part of a `Portfolio` object, you can save and load objects from your workspace and create and manipulate arrays of objects. After settling on a problem, which, in the case of mean-variance portfolio optimization, means that you have either data or moments for asset returns and a collection of constraints on your portfolios, use `Portfolio` to set the properties for the `Portfolio` object. `Portfolio` lets you create an object from scratch or update an existing object. Since the `Portfolio` object is a value object, it is easy to create a basic object, then use functions to build upon the basic object to create new versions of the basic object. This is useful to compare a basic problem with alternatives derived from the basic problem. For details, see “Creating the Portfolio Object” on page 4-25.

Setting and Getting Properties

You can set properties of a `Portfolio` object using either `Portfolio` or various `set` functions.

Note Although you can also set properties directly, it is not recommended since error-checking is not performed when you set a property directly.

The `Portfolio` object supports setting properties with name-value pair arguments such that each argument name is a property and each value is the value to assign to that property. For example, to set the `AssetMean` and `AssetCovar` properties in an existing `Portfolio` object `p` with the values `m` and `C`, use the syntax:

```
p = Portfolio(p, 'AssetMean', m, 'AssetCovar', C);
```

In addition to `Portfolio`, which lets you set individual properties one at a time, groups of properties are set in a `Portfolio` object with various “set” and “add” functions. For example, to set up an average turnover constraint, use the `setTurnover` function to specify the bound on portfolio average turnover and the initial portfolio. To get individual properties from a `Portfolio` object, obtain properties directly or use an assortment of “get” functions that obtain groups of properties from a `Portfolio` object. The `Portfolio` object and the `set` functions have several useful features:

- `Portfolio` and the `set` functions try to determine the dimensions of your problem with either explicit or implicit inputs.
- `Portfolio` and the `set` functions try to resolve ambiguities with default choices.
- `Portfolio` and the `set` functions perform scalar expansion on arrays when possible.
- The associated `Portfolio` object functions try to diagnose and warn about problems.

Displaying Portfolio Objects

The `Portfolio` object uses the default display functions provided by MATLAB, where `display` and `disp` display a `Portfolio` object and its properties with or without the object variable name.

Saving and Loading Portfolio Objects

Save and load `Portfolio` objects using the MATLAB `save` and `load` commands.

Estimating Efficient Portfolios and Frontiers

Estimating efficient portfolios and efficient frontiers is the primary purpose of the portfolio optimization tools. An efficient portfolio are the portfolios that satisfy the criteria of minimum risk for a given level of return and maximum return for a given level of risk. A collection of “estimate” and “plot” functions provide ways to explore the efficient frontier. The “estimate” functions obtain either efficient portfolios or risk and return proxies to form efficient frontiers. At the portfolio level, a collection of functions estimates efficient portfolios on the efficient frontier with functions to obtain efficient portfolios:

- At the endpoints of the efficient frontier
- That attains targeted values for return proxies
- That attains targeted values for risk proxies
- Along the entire efficient frontier

These functions also provide purchases and sales needed to shift from an initial or current portfolio to each efficient portfolio. At the efficient frontier level, a collection of functions plot the efficient

frontier and estimate either risk or return proxies for efficient portfolios on the efficient frontier. You can use the resultant efficient portfolios or risk and return proxies in subsequent analyses.

Arrays of Portfolio Objects

Although all functions associated with a `Portfolio` object are designed to work on a scalar `Portfolio` object, the array capabilities of MATLAB enable you to set up and work with arrays of `Portfolio` objects. The easiest way to do this is with the `repmat` function. For example, to create a 3-by-2 array of `Portfolio` objects:

```
p = repmat(Portfolio, 3, 2);
disp(p)

disp(p)
3x2 Portfolio array with properties:
```

```
BuyCost
SellCost
RiskFreeRate
AssetMean
AssetCovar
TrackingError
TrackingPort
Turnover
BuyTurnover
SellTurnover
Name
NumAssets
AssetList
InitPort
AInequality
bInequality
AEquality
bEquality
LowerBound
UpperBound
LowerBudget
UpperBudget
GroupMatrix
LowerGroup
UpperGroup
GroupA
GroupB
LowerRatio
UpperRatio
MinNumAssets
MaxNumAssets
ConditionalBudgetThreshold
ConditionalUpperBudget
BoundType
```

After setting up an array of `Portfolio` objects, you can work on individual `Portfolio` objects in the array by indexing. For example:

```
p(i,j) = Portfolio(p(i,j), ... );
```

This example calls `Portfolio` for the (i,j) element of a matrix of `Portfolio` objects in the variable `p`.

If you set up an array of `Portfolio` objects, you can access properties of a particular `Portfolio` object in the array by indexing so that you can set the lower and upper bounds `lb` and `ub` for the (i,j,k) element of a 3-D array of `Portfolio` objects with

```
p(i,j,k) = setBounds(p(i,j,k),lb, ub);
```

and, once set, you can access these bounds with

```
[lb, ub] = getBounds(p(i,j,k));
```

`Portfolio` object functions work on only one `Portfolio` object at a time.

Subclassing Portfolio Objects

You can subclass the `Portfolio` object to override existing functions or to add new properties or functions. To do so, create a derived class from the `Portfolio` class. This gives you all the properties and functions of the `Portfolio` class along with any new features that you choose to add to your subclassed object. The `Portfolio` class is derived from an abstract class called `AbstractPortfolio`. Because of this, you can also create a derived class from `AbstractPortfolio` that implements an entirely different form of portfolio optimization using properties and functions of the `AbstractPortfolio` class.

Conventions for Representation of Data

The portfolio optimization tools follow these conventions regarding the representation of different quantities associated with portfolio optimization:

- Asset returns or prices are in matrix form with samples for a given asset going down the rows and assets going across the columns. In the case of prices, the earliest dates must be at the top of the matrix, with increasing dates going down.
- The mean and covariance of asset returns are stored in a vector and a matrix and the tools have no requirement that the mean must be either a column or row vector.
- Portfolios are in vector or matrix form with weights for a given portfolio going down the rows and distinct portfolios going across the columns.
- Constraints on portfolios are formed in such a way that a portfolio is a column vector.
- Portfolio risks and returns are either scalars or column vectors (for multiple portfolio risks and returns).

See Also

`Portfolio`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222

- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

Creating the Portfolio Object

In this section...

“Syntax” on page 4-25

“Portfolio Problem Sufficiency” on page 4-25

“Portfolio Function Examples” on page 4-26

To create a fully specified mean-variance portfolio optimization problem, instantiate the `Portfolio` object using `Portfolio`. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Syntax

Use `Portfolio` to create an instance of an object of the `Portfolio` class. You can use `Portfolio` in several ways. To set up a portfolio optimization problem in a `Portfolio` object, the simplest syntax is:

```
p = Portfolio;
```

This syntax creates a `Portfolio` object, `p`, such that all object properties are empty.

The `Portfolio` object also accepts collections of argument name-value pair arguments for properties and their values. The `Portfolio` object accepts inputs for public properties with the general syntax:

```
p = Portfolio('property1', value1, 'property2', value2, ... );
```

If a `Portfolio` object already exists, the syntax permits the first (and only the first argument) of `Portfolio` to be an existing object with subsequent argument name-value pair arguments for properties to be added or modified. For example, given an existing `Portfolio` object in `p`, the general syntax is:

```
p = Portfolio(p, 'property1', value1, 'property2', value2, ... );
```

Input argument names are not case-sensitive, but must be completely specified. In addition, several properties can be specified with alternative argument names (see “Shortcuts for Property Names” on page 4-30). The `Portfolio` object detects problem dimensions from the inputs and, once set, subsequent inputs can undergo various scalar or matrix expansion operations that simplify the overall process to formulate a problem. In addition, a `Portfolio` object is a value object so that, given portfolio `p`, the following code creates two objects, `p` and `q`, that are distinct:

```
q = Portfolio(p, ...)
```

Portfolio Problem Sufficiency

A mean-variance portfolio optimization is completely specified with the `Portfolio` object if these two conditions are met:

- The moments of asset returns must be specified such that the property `AssetMean` contains a valid finite mean vector of asset returns and the property `AssetCovar` contains a valid symmetric positive-semidefinite matrix for the covariance of asset returns.

The first condition is satisfied by setting the properties associated with the moments of asset returns.

- The set of feasible portfolios must be a nonempty compact set, where a compact set is closed and bounded.

The second condition is satisfied by an extensive collection of properties that define different types of constraints to form a set of feasible portfolios. Since such sets must be bounded, either explicit or implicit constraints can be imposed, and several functions, such as `estimateBounds`, provide ways to ensure that your problem is properly formulated.

Although the general sufficiency conditions for mean-variance portfolio optimization go beyond these two conditions, the `Portfolio` object implemented in Financial Toolbox implicitly handles all these additional conditions. For more information on the Markowitz model for mean-variance portfolio optimization, see “Portfolio Optimization” on page A-5.

Portfolio Function Examples

If you create a `Portfolio` object, `p`, with no input arguments, you can display it using `disp`:

```
p = Portfolio;  
disp(p)
```

Portfolio with properties:

```
BuyCost: []  
SellCost: []  
RiskFreeRate: []  
AssetMean: []  
AssetCovar: []  
TrackingError: []  
TrackingPort: []  
Turnover: []  
BuyTurnover: []  
SellTurnover: []  
Name: []  
NumAssets: []  
AssetList: []  
InitPort: []  
AInequality: []  
bInequality: []  
AEquality: []  
bEquality: []  
LowerBound: []  
UpperBound: []  
LowerBudget: []  
UpperBudget: []  
GroupMatrix: []  
LowerGroup: []  
UpperGroup: []  
GroupA: []  
GroupB: []  
LowerRatio: []  
UpperRatio: []  
MinNumAssets: []  
MaxNumAssets: []  
ConditionalBudgetThreshold: []
```

```
ConditionalUpperBudget: []
BoundType: []
```

The approaches listed provide a way to set up a portfolio optimization problem with the `Portfolio` object. The `set` functions offer additional ways to set and modify collections of properties in the `Portfolio` object.

Using the Portfolio Function for a Single-Step Setup

You can use the `Portfolio` object to directly set up a “standard” portfolio optimization problem, given a mean and covariance of asset returns in the variables `m` and `C`:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio('assetmean', m, 'assetcovar', C, ...
             'lowerbudget', 1, 'upperbudget', 1, 'lowerbound', 0)
```

`p =`

Portfolio with properties:

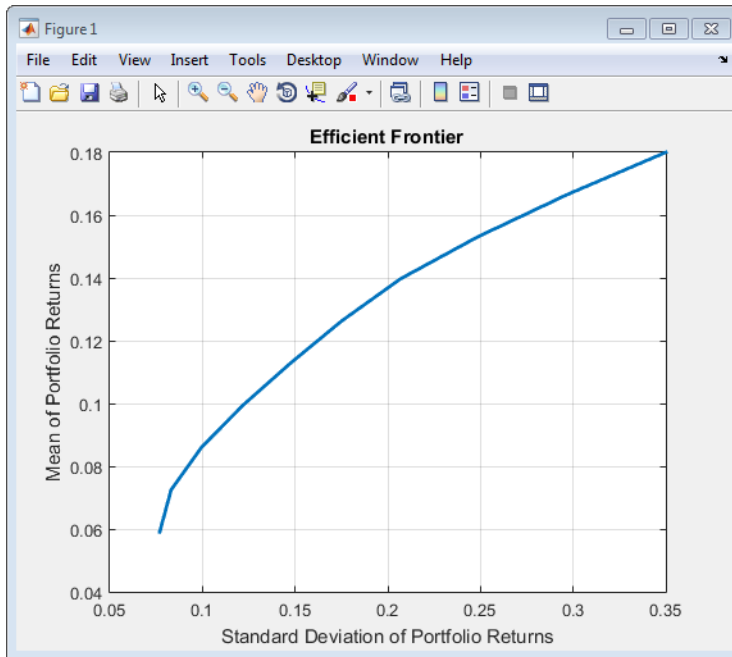
```
BuyCost: []
SellCost: []
RiskFreeRate: []
AssetMean: [4×1 double]
AssetCovar: [4×4 double]
TrackingError: []
TrackingPort: []
Turnover: []
BuyTurnover: []
SellTurnover: []
Name: []
NumAssets: 4
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [4×1 double]
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
```

```
ConditionalUpperBudget: []
BoundType: []
```

The `LowerBound` property value undergoes scalar expansion since `AssetMean` and `AssetCovar` provide the dimensions of the problem.

You can use dot notation with the function `plotFrontier`.

```
p.plotFrontier
```



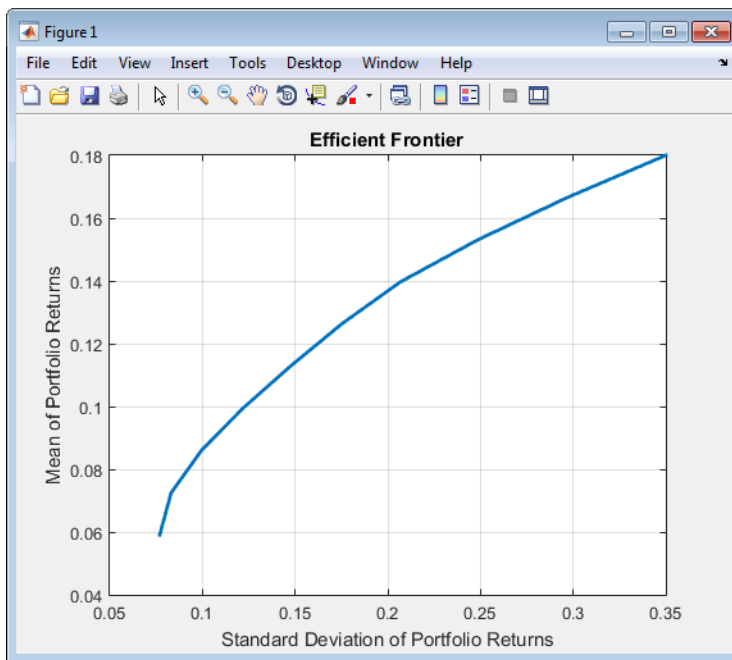
Using the Portfolio Function with a Sequence of Steps

An alternative way to accomplish the same task of setting up a “standard” portfolio optimization problem, given a mean and covariance of asset returns in the variables `m` and `C` (which also illustrates that argument names are not case-sensitive):

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = Portfolio(p, 'assetmean', m, 'assetcovar', C);
p = Portfolio(p, 'lowerbudget', 1, 'upperbudget', 1);
p = Portfolio(p, 'lowerbound', 0);

plotFrontier(p)
```

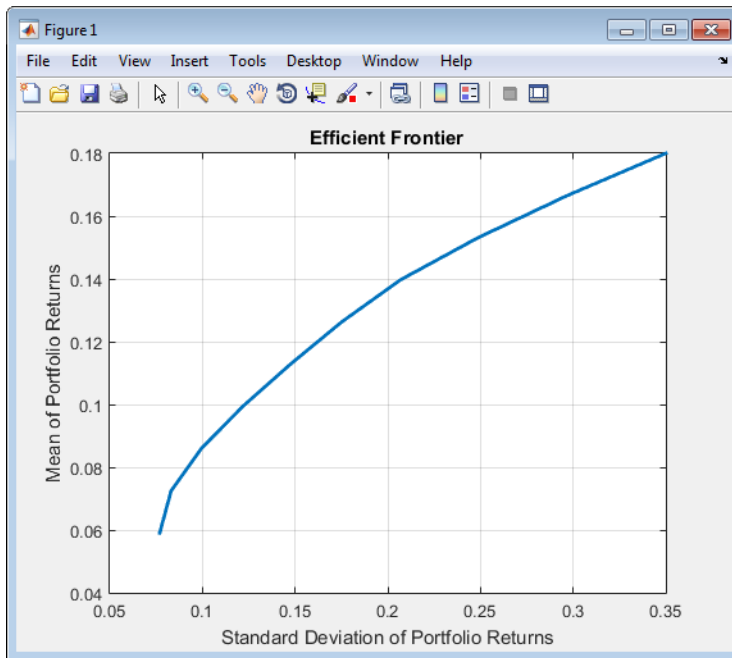


This way works because the calls to `Portfolio` are in this particular order. In this case, the call to initialize `AssetMean` and `AssetCovar` provides the dimensions for the problem. If you were to do this step last, you would have to explicitly dimension the `LowerBound` property as follows:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = Portfolio(p, 'LowerBound', zeros(size(m)));
p = Portfolio(p, 'LowerBudget', 1, 'UpperBudget', 1);
p = Portfolio(p, 'AssetMean', m, 'AssetCovar', C);

plotFrontier(p)
```



If you did not specify the size of `LowerBound` but, instead, input a scalar argument, the `Portfolio` object assumes that you are defining a single-asset problem and produces an error at the call to set asset moments with four assets.

Shortcuts for Property Names

The `Portfolio` object has shorter argument names that replace longer argument names associated with specific properties of the `Portfolio` object. For example, rather than enter `'assetcovar'`, the `Portfolio` object accepts the case-insensitive name `'covar'` to set the `AssetCovar` property in a `Portfolio` object. Every shorter argument name corresponds with a single property in the `Portfolio` object. The one exception is the alternative argument name `'budget'`, which signifies both the `LowerBudget` and `UpperBudget` properties. When `'budget'` is used, then the `LowerBudget` and `UpperBudget` properties are set to the same value to form an equality budget constraint.

Shortcuts for Property Names

Shortcut Argument Name	Equivalent Argument / Property Name
ae	AEquality
ai	AInequality
covar	AssetCovar
assetnames or assets	AssetList
mean	AssetMean
be	bEquality
bi	bInequality
group	GroupMatrix
lb	LowerBound
n or num	NumAssets
rfr	RiskFreeRate
ub	UpperBound
budget	UpperBudget and LowerBudget

For example, this call `Portfolio` uses these shortcuts for properties and is equivalent to the previous examples:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio('mean', m, 'covar', C, 'budget', 1, 'lb', 0);
plotFrontier(p)
```

Direct Setting of Portfolio Object Properties

Although not recommended, you can set properties directly, however no error-checking is done on your inputs:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p.NumAssets = numel(m);
p.AssetMean = m;
p.AssetCovar = C;
p.LowerBudget = 1;
p.UpperBudget = 1;
p.LowerBound = zeros(size(m));

plotFrontier(p)
```

See Also

Portfolio | estimateBounds

Related Examples

- “Common Operations on the Portfolio Object” on page 4-33
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Using Social Performance Measure” on page 4-264

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Common Operations on the Portfolio Object

In this section...

“Naming a Portfolio Object” on page 4-33
 “Configuring the Assets in the Asset Universe” on page 4-33
 “Setting Up a List of Asset Identifiers” on page 4-33
 “Truncating and Padding Asset Lists” on page 4-35

Naming a Portfolio Object

To name a `Portfolio` object, use the `Name` property. `Name` is informational and has no effect on any portfolio calculations. If the `Name` property is nonempty, `Name` is the title for the efficient frontier plot generated by `plotFrontier`. For example, if you set up an asset allocation fund, you could name the `Portfolio` object `Asset Allocation Fund`:

```
p = Portfolio('Name', 'Asset Allocation Fund');
disp(p.Name)
```

```
Asset Allocation Fund
```

Configuring the Assets in the Asset Universe

The fundamental quantity in the `Portfolio` object is the number of assets in the asset universe. This quantity is maintained in the `NumAssets` property. Although you can set this property directly, it is derived from other properties such as the mean of asset returns and the initial portfolio. In some instances, the number of assets may need to be set directly. This example shows how to set up a `Portfolio` object that has four assets:

```
p = Portfolio('NumAssets', 4);
disp(p.NumAssets)
```

```
4
```

After setting the `NumAssets` property, you cannot modify it (unless no other properties are set that depend on `NumAssets`). The only way to change the number of assets in an existing `Portfolio` object with a known number of assets is to create a new `Portfolio` object.

Setting Up a List of Asset Identifiers

When working with portfolios, you must specify a universe of assets. Although you can perform a complete analysis without naming the assets in your universe, it is helpful to have an identifier associated with each asset as you create and work with portfolios. You can create a list of asset identifiers as a cell vector of character vectors in the property `AssetList`. You can set up the list using the next two functions.

Setting Up Asset Lists Using the Portfolio Function

Suppose that you have a `Portfolio` object, `p`, with assets with symbols `'AA'`, `'BA'`, `'CAT'`, `'DD'`, and `'ETR'`. You can create a list of these asset symbols in the object using the `Portfolio` object:

```
p = Portfolio('assetlist', {'AA', 'BA', 'CAT', 'DD', 'ETR'});
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

Notice that the property `AssetList` is maintained as a cell array that contains character vectors, and that it is necessary to pass a cell array into the `Portfolio` object to set `AssetList`. In addition, notice that the property `NumAssets` is set to 5 based on the number of symbols used to create the asset list:

```
disp(p.NumAssets)
```

```
5
```

Setting Up Asset Lists Using the `setAssetList` Function

You can also specify a list of assets using the `setAssetList` function. Given the list of asset symbols 'AA', 'BA', 'CAT', 'DD', and 'ETR', you can use `setAssetList` with:

```
p = Portfolio;  
p = setAssetList(p, { 'AA', 'BA', 'CAT', 'DD', 'ETR' });  
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

`setAssetList` also enables you to enter symbols directly as a comma-separated list without creating a cell array of character vectors. For example, given the list of assets symbols 'AA', 'BA', 'CAT', 'DD', and 'ETR', use `setAssetList`:

```
p = Portfolio;  
p = setAssetList(p, 'AA', 'BA', 'CAT', 'DD', 'ETR');  
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

`setAssetList` has many additional features to create lists of asset identifiers. If you use `setAssetList` with just a `Portfolio` object, it creates a default asset list according to the name specified in the hidden public property `defaultforAssetList` (which is 'Asset' by default). The number of asset names created depends on the number of assets in the property `NumAssets`. If `NumAssets` is not set, then `NumAssets` is assumed to be 1.

For example, if a `Portfolio` object `p` is created with `NumAssets = 5`, then this code fragment shows the default naming behavior:

```
p = Portfolio('numassets',5);  
p = setAssetList(p);  
disp(p.AssetList)
```

```
'Asset1'      'Asset2'      'Asset3'      'Asset4'      'Asset5'
```

Suppose that your assets are, for example, ETFs and you change the hidden property `defaultforAssetList` to 'ETF', you can then create a default list for ETFs:

```
p = Portfolio('numassets',5);  
p.defaultforAssetList = 'ETF';  
p = setAssetList(p);  
disp(p.AssetList)
```

```
'ETF1'      'ETF2'      'ETF3'      'ETF4'      'ETF5'
```

Truncating and Padding Asset Lists

If the `NumAssets` property is already set and you pass in too many or too few identifiers, the `Portfolio` object, and the `setAssetList` function truncate or pad the list with numbered default asset names that use the name specified in the hidden public property `defaultforAssetList`. If the list is truncated or padded, a warning message indicates the discrepancy. For example, assume that you have a `Portfolio` object with five ETFs and you only know the first three CUSIPs '921937835', '922908769', and '922042775'. Use this syntax to create an asset list that pads the remaining asset identifiers with numbered 'UnknownCUSIP' placeholders:

```
p = Portfolio('numassets',5);
p.defaultforAssetList = 'UnknownCUSIP';
p = setAssetList(p,'921937835', '922908769', '922042775');
disp(p.AssetList)
```

Warning: Input list of assets has 2 too few identifiers. Padding with numbered assets.
 > In Portfolio.setAssetList at 121
 '921937835' '922908769' '922042775' 'UnknownCUSIP4' 'UnknownCUSIP5'

Alternatively, suppose that you have too many identifiers and need only the first four assets. This example illustrates truncation of the asset list using the `Portfolio` object:

```
p = Portfolio('numassets',4);
p = Portfolio(p, 'assetlist', { 'AGG', 'EEM', 'MDY', 'SPY', 'VEU' });
disp(p.AssetList)
```

Warning: AssetList has 1 too many identifiers. Using first 4 assets.
 > In Portfolio.checkarguments at 434
 In Portfolio.Portfolio>Portfolio.Portfolio at 171
 'AGG' 'EEM' 'MDY' 'SPY'

The hidden public property `uppercaseAssetList` is a Boolean flag to specify whether to convert asset names to uppercase letters. The default value for `uppercaseAssetList` is `false`. This example shows how to use the `uppercaseAssetList` flag to force identifiers to be uppercase letters:

```
p = Portfolio;
p.uppercaseAssetList = true;
p = setAssetList(p,{ 'aa', 'ba', 'cat', 'dd', 'etr' });
disp(p.AssetList)
```

'AA' 'BA' 'CAT' 'DD' 'ETR'

See Also

`Portfolio` | `setAssetList` | `setInitPort` | `setTrackingPort` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Setting Up an Initial or Current Portfolio” on page 4-37
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Returns and Moments of Asset Returns Using Portfolio Object” on page 4-42
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190

- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Setting Up an Initial or Current Portfolio

In many applications, creating a new optimal portfolio requires comparing the new portfolio with an initial or current portfolio to form lists of purchases and sales. The `Portfolio` object property `InitPort` lets you identify an initial or current portfolio. The initial portfolio also plays an essential role if you have either transaction costs or turnover constraints. The initial portfolio need not be feasible within the constraints of the problem. This can happen if the weights in a portfolio have shifted such that some constraints become violated. To check if your initial portfolio is feasible, use the `checkFeasibility` function described in “Validating Portfolios” on page 4-95. Suppose that you have an initial portfolio in `x0`, then use the `Portfolio` object to set up an initial portfolio:

```
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = Portfolio('InitPort', x0);
disp(p.InitPort)

0.3000
0.2000
0.2000
0
```

As with all array properties, you can set `InitPort` with scalar expansion. This is helpful to set up an equally weighted initial portfolio of, for example, 10 assets:

```
p = Portfolio('NumAssets', 10, 'InitPort', 1/10);
disp(p.InitPort)

0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
```

To clear an initial portfolio from your `Portfolio` object, use either the `Portfolio` object or the `setInitPort` function with an empty input for the `InitPort` property. If transaction costs or turnover constraints are set, it is not possible to clear the `InitPort` property in this way. In this case, to clear `InitPort`, first clear the dependent properties and then clear the `InitPort` property.

The `InitPort` property can also be set with `setInitPort` which lets you specify the number of assets if you want to use scalar expansion. For example, given an initial portfolio in `x0`, use `setInitPort` to set the `InitPort` property:

```
p = Portfolio;
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = setInitPort(p, x0);
disp(p.InitPort)

0.3000
0.2000
0.2000
0
```

To create an equally weighted portfolio of four assets, use `setInitPort`:

```
p = Portfolio;  
p = setInitPort(p, 1/4, 4);  
disp(p.InitPort)  
  
0.2500  
0.2500  
0.2500  
0.2500
```

Portfolio object functions that work with either transaction costs or turnover constraints also depend on the `InitPort` property. So, the `set` functions for transaction costs or turnover constraints permit the assignment of a value for the `InitPort` property as part of their implementation. For details, see “Working with Average Turnover Constraints Using Portfolio Object” on page 4-85, “Working with One-Way Turnover Constraints Using Portfolio Object” on page 4-88, and “Working with Transaction Costs” on page 4-54 for details. If either transaction costs or turnover constraints are used, then the `InitPort` property must have a nonempty value. Absent a specific value assigned through the Portfolio object or various `set` functions, the Portfolio object sets `InitPort` to 0 and warns if `BuyCost`, `SellCost`, or `Turnover` properties are set. The following example illustrates what happens if an average turnover constraint is specified with an initial portfolio:

```
p = Portfolio('Turnover', 0.3, 'InitPort', [ 0.3; 0.2; 0.2; 0.0 ]);  
disp(p.InitPort)  
  
0.3000  
0.2000  
0.2000  
0
```

In contrast, this example shows what happens if an average turnover constraint is specified without an initial portfolio:

```
p = Portfolio('Turnover', 0.3);  
disp(p.InitPort)  
  
Warning: InitPort and NumAssets are empty and either transaction costs or turnover constraints specified.  
Will set NumAssets = 1 and InitPort = 0.  
> In Portfolio.checkarguments at 367  
In Portfolio.Portfolio>Portfolio.Portfolio at 171  
0
```

See Also

Portfolio | setAssetList | setInitPort | estimateBounds | checkFeasibility

Related Examples

- “Setting Up a Tracking Portfolio” on page 4-40
- “Common Operations on the Portfolio Object” on page 4-33
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Returns and Moments of Asset Returns Using Portfolio Object” on page 4-42
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222

- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

Setting Up a Tracking Portfolio

Given a benchmark or tracking portfolio, you can ensure that the risk of a portfolio relative to the benchmark portfolio is no greater than a specified amount. The `Portfolio` object property `TrackingPort` lets you identify a tracking portfolio. For more information on using a tracking portfolio with tracking error constraints, see “Working with Tracking Error Constraints Using Portfolio Object” on page 4-91.

The tracking error constraints can be used with any of the other supported constraints in the `Portfolio` object without restrictions. However, since the portfolio set necessarily and sufficiently must be a non-empty compact set, the application of a tracking error constraint can result in an empty portfolio set. Use `estimateBounds` to confirm that the portfolio set is non-empty and compact.

Suppose that you have an initial portfolio in `x0`, then use the `Portfolio` object to set up a tracking portfolio:

```
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = Portfolio('TrackingPort', x0);
disp(p.TrackingPort)

0.3000
0.2000
0.2000
0
```

As with all array properties, you can set `TrackingPort` with scalar expansion. This is helpful to set up an equally weighted tracking portfolio of, for example, 10 assets:

```
p = Portfolio('NumAssets', 10, 'TrackingPort', 1/10);
disp(p.TrackingPort)

0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
```

To clear a tracking portfolio from your `Portfolio` object, use either the `Portfolio` object or the `setTrackingPort` function with an empty input for the `TrackingPort` property. If transaction costs or turnover constraints are set, it is not possible to clear the `TrackingPort` property in this way. In this case, to clear `TrackingPort`, first clear the dependent properties and then clear the `TrackingPort` property.

The `TrackingPort` property can also be set with `setTrackingPort` which lets you specify the number of assets if you want to use scalar expansion. For example, given an initial portfolio in `x0`, use `setTrackingPort` to set the `TrackingPort` property:

```
p = Portfolio;
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = setTrackingPort(p, x0);
disp(p.TrackingPort)
```



```
0.3000
0.2000
0.2000
0
```

To create an equally weighted portfolio of four assets, use `setTrackingPort`:

```
p = Portfolio;
p = setTrackingPort(p, 1/4, 4);
disp(p.TrackingPort)
```

```
0.2500
0.2500
0.2500
0.2500
```

See Also

`Portfolio` | `setAssetList` | `setInitPort` | `setTrackingPort` | `setTrackingError` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Setting Up an Initial or Current Portfolio” on page 4-37
- “Common Operations on the Portfolio Object” on page 4-33
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Returns and Moments of Asset Returns Using Portfolio Object” on page 4-42
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

Asset Returns and Moments of Asset Returns Using Portfolio Object

In this section...

“Assignment Using the Portfolio Function” on page 4-42
 “Assignment Using the setAssetMoments Function” on page 4-43
 “Scalar Expansion of Arguments” on page 4-44
 “Estimating Asset Moments from Prices or Returns” on page 4-45
 “Estimating Asset Moments with Missing Data” on page 4-47
 “Estimating Asset Moments from Time Series Data” on page 4-49

Since mean-variance portfolio optimization problems require estimates for the mean and covariance of asset returns, the `Portfolio` object has several ways to set and get the properties `AssetMean` (for the mean) and `AssetCovar` (for the covariance). In addition, the return for a riskless asset is kept in the property `RiskFreeRate` so that all assets in `AssetMean` and `AssetCovar` are risky assets. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Assignment Using the Portfolio Function

Suppose that you have a mean and covariance of asset returns in variables `m` and `C`. The properties for the moments of asset returns are set using the `Portfolio` object:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;
p = Portfolio('AssetMean', m, 'AssetCovar', C);
disp(p.NumAssets)
disp(p.AssetMean)
disp(p.AssetCovar)

```

4

```

0.0042
0.0083
0.0100
0.0150

```

```

0.0005    0.0003    0.0002         0
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
0         0.0010    0.0028    0.0102

```

Notice that the `Portfolio` object determines the number of assets in `NumAssets` from the moments. The `Portfolio` object enables separate initialization of the moments, for example:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;

```

```

    0.00408 0.0289 0.0204 0.0119;
    0.00192 0.0204 0.0576 0.0336;
    0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

p = Portfolio;
p = Portfolio(p, 'AssetMean', m);
p = Portfolio(p, 'AssetCovar', C);
[assetmean, assetcovar] = p.getAssetMoments

assetmean =

    0.0042
    0.0083
    0.0100
    0.0150

assetcovar =

    0.0005    0.0003    0.0002         0
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102

```

The `getAssetMoments` function lets you get the values for `AssetMean` and `AssetCovar` properties at the same time.

Assignment Using the `setAssetMoments` Function

You can also set asset moment properties using the `setAssetMoments` function. For example, given the mean and covariance of asset returns in the variables `m` and `C`, the asset moment properties can be set:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

p = Portfolio;
p = setAssetMoments(p, m, C);
[assetmean, assetcovar] = getAssetMoments(p)

assetmean =

    0.0042
    0.0083
    0.0100
    0.0150

assetcovar =

    0.0005    0.0003    0.0002         0

```

```
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
0         0.0010    0.0028    0.0102
```

Scalar Expansion of Arguments

Both the `Portfolio` object and the `setAssetMoments` function perform scalar expansion on arguments for the moments of asset returns. When using the `Portfolio` object, the number of assets must be already specified in the variable `NumAssets`. If `NumAssets` has not already been set, a scalar argument is interpreted as a scalar with `NumAssets` set to 1. `setAssetMoments` provides an additional optional argument to specify the number of assets so that scalar expansion works with the correct number of assets. In addition, if either a scalar or vector is input for the covariance of asset returns, a diagonal matrix is formed such that a scalar expands along the diagonal and a vector becomes the diagonal. This example demonstrates scalar expansion for four jointly independent assets with a common mean 0.1 and common variance 0.03:

```
p = Portfolio;
p = setAssetMoments(p, 0.1, 0.03, 4);
[assetmean, assetcovar] = getAssetMoments(p)
```

```
assetmean =
```

```
0.1000
0.1000
0.1000
0.1000
```

```
assetcovar =
```

```
0.0300    0    0    0
0    0.0300    0    0
0    0    0.0300    0
0    0    0    0.0300
```

If at least one argument is properly dimensioned, you do not need to include the additional `NumAssets` argument. This example illustrates a constant-diagonal covariance matrix and a mean of asset returns for four assets:

```
p = Portfolio;
p = setAssetMoments(p, [ 0.05; 0.06; 0.04; 0.03 ], 0.03);
[assetmean, assetcovar] = getAssetMoments(p)
```

```
assetmean =
```

```
0.0500
0.0600
0.0400
0.0300
```

```
assetcovar =
```

```
0.0300    0    0    0
0    0.0300    0    0
0    0    0.0300    0
0    0    0    0.0300
```

In addition, scalar expansion works with the `Portfolio` object if `NumAssets` is known, or is deduced from the inputs.

Estimating Asset Moments from Prices or Returns

This example shows how to set the moments of asset returns by using the `estimateAssetMoments` function which accepts either prices or returns and estimates the mean and covariance of asset returns. Either prices or returns are stored as matrices with samples going down the rows and assets going across the columns. In addition, prices or returns can be stored in a table or timetable (see “Estimating Asset Moments from Time Series Data” on page 4-49).

To illustrate using `estimateAssetMoments`, generate random samples of 120 observations of asset returns for four assets from the mean and covariance of asset returns in the variables `m` and `C` with `portsim`. The default behavior of `portsim` creates simulated data with estimated mean and covariance identical to the input moments `m` and `C`. In addition to a return series created by `portsim` in the variable `X`, a price series is created in the variable `Y`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;
X = portsim(m', C, 120);
Y = ret2tick(X);
```

Note that Portfolio optimization requires that you use total returns and not just price returns. So, "returns" should be total returns and "prices" should be total return prices.

Given asset returns and prices in variables `X` and `Y`, the following sequence of code demonstrates equivalent ways to estimate asset moments for the `Portfolio` object. A `Portfolio` object is created in `p` with the moments of asset returns set directly in the `Portfolio` object, and a second `Portfolio` object is created in `q` to obtain the mean and covariance of asset returns from asset return data in `X` using `estimateAssetMoments`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

X = portsim(m', C, 120);
p = Portfolio('mean', m, 'covar', C);
q = Portfolio;
q = estimateAssetMoments(q, X);
```

```
[passetmean, passetcovar] = getAssetMoments(p)
```

```
passetmean = 4×1
```

```
0.0042
0.0083
```

```

0.0100
0.0150

passetcovar = 4×4

    0.0005    0.0003    0.0002         0
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102

[qassetmean, qassetcovar] = getAssetMoments(q)

qassetmean = 4×1

    0.0042
    0.0083
    0.0100
    0.0150

qassetcovar = 4×4

    0.0005    0.0003    0.0002   -0.0000
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0028
   -0.0000    0.0010    0.0028    0.0102

```

Notice how either approach has the same moments. The default behavior of `estimateAssetMoments` is to work with asset returns. If, instead, you have asset prices in the variable `Y`, `estimateAssetMoments` accepts a name-value pair argument name `'DataFormat'` with a corresponding value set to `'prices'` to indicate that the input to the function is in the form of asset prices and not returns (the default value for the `'DataFormat'` argument is `'returns'`). The following code compares direct assignment of moments in the `Portfolio` object `p` with estimated moments from asset price data in `Y` in the `Portfolio` object `q`.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

X = portsim(m', C, 120);
Y = ret2tick(X);

p = Portfolio('mean',m,'covar',C);

q = Portfolio;
q = estimateAssetMoments(q, Y, 'dataformat', 'prices');

[passetmean, passetcovar] = getAssetMoments(p)

passetmean = 4×1

```

```

0.0042
0.0083
0.0100
0.0150

passetcovar = 4×4

    0.0005    0.0003    0.0002         0
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102

[qassetmean, qassetcovar] = getAssetMoments(q)

qassetmean = 4×1

    0.0042
    0.0083
    0.0100
    0.0150

qassetcovar = 4×4

    0.0005    0.0003    0.0002   -0.0000
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0028
   -0.0000    0.0010    0.0028    0.0102

```

Estimating Asset Moments with Missing Data

This example shows how to work with missing data indicated by NaN values in your return or price data. Although “Multivariate Normal Regression” on page 9-2 goes into detail about regression with missing data, the `estimateAssetMoments` function has a name-value pair argument name `'MissingData'` that indicates with a Boolean value whether to use the missing data capabilities of Financial Toolbox™ software. The default value for `'MissingData'` is `false` which removes all samples with NaN values. If, however, `'MissingData'` is set to `true`, `estimateAssetMoments` uses the ECM algorithm to estimate asset moments. The following code illustrates how this works on price data with missing values:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

X = portsim(m', C, 120);
Y = ret2tick(X);
Y(1:20,1) = NaN;
Y(1:12,4) = NaN;

```

```

p = Portfolio('mean',m,'covar',C);

q = Portfolio;
q = estimateAssetMoments(q, Y, 'dataformat', 'prices');

r = Portfolio;
r = estimateAssetMoments(r, Y, 'dataformat', 'prices', 'missingdata', true);

[passetmean, passetcovar] = getAssetMoments(p)

passetmean = 4×1

    0.0042
    0.0083
    0.0100
    0.0150

passetcovar = 4×4

    0.0005    0.0003    0.0002         0
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102

[qassetmean, qassetcovar] = getAssetMoments(q)

qassetmean = 4×1

    0.0039
    0.0128
    0.0153
    0.0238

qassetcovar = 4×4

    0.0006    0.0004    0.0001    0.0001
    0.0004    0.0023    0.0015    0.0005
    0.0001    0.0015    0.0043    0.0024
    0.0001    0.0005    0.0024    0.0084

[rassetmean, rassetcovar] = getAssetMoments(r)

rassetmean = 4×1

    0.0033
    0.0083
    0.0100
    0.0176

rassetcovar = 4×4

    0.0006    0.0004    0.0001    0.0001
    0.0004    0.0024    0.0017    0.0007

```



```
0.0001    0.0017    0.0048    0.0025
0.0001    0.0007    0.0025    0.0083
```

The `Portfolio` object `p` contains raw moments, the `Portfolio` object `q` contains estimated moments in which NaN values are discarded, and the `Portfolio` object `r` contains raw moments that accommodate missing values. Each time you run this code, you will get different estimates for the moments in `q` and `r`, and these will also differ from the moments in `p`.

Estimating Asset Moments from Time Series Data

This example shows how the `estimateAssetMoments` function also accepts asset returns or prices stored in a `table` or `timetable`. `estimateAssetMoments` implicitly works with matrices of data or data in a `table` or `timetable` object using the same rules for whether the data are returns or prices.

To illustrate the use of a `table` and `timetable`, use `array2table` and `array2timetable` to create a `table` and a `timetable` that contain asset returns generated with `portsim` (see “Estimating Asset Moments from Prices or Returns” on page 4-45). Two portfolio objects are then created with the `AssetReturns` based on a `table` and a `timetable` object.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

assetRetnScenarios = portsim(m', C, 120);
dates = datetime(datenum(2001,1:120,31), 'ConvertFrom', 'datenum');
assetsName = {'Bonds', 'LargeCap', 'SmallCap', 'Emerging'};
assetRetnTimeTable = array2timetable(assetRetnScenarios, 'RowTimes', dates, 'VariableNames', assetsName);
assetRetnTable = array2table(assetRetnScenarios, 'VariableNames', assetsName);

% Create two Portfolio objects:
% p with predefined mean and covar and q with asset return scenarios to estimate mean and covar.
p = Portfolio('mean', m, 'covar', C);
q = Portfolio;

% Estimate asset moments using a timetable
q = estimateAssetMoments(q, assetRetnTimeTable);
[passetmean, passetcovar] = getAssetMoments(p)

passetmean = 4x1

    0.0042
    0.0083
    0.0100
    0.0150

passetcovar = 4x4
```

```

0.0005    0.0003    0.0002         0
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102

```

```
[qassetmean, qassetcovar] = getAssetMoments(q)
```

```
qassetmean = 4×1
```

```

0.0042
0.0083
0.0100
0.0150

```

```
qassetcovar = 4×4
```

```

0.0005    0.0003    0.0002   -0.0000
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
-0.0000    0.0010    0.0028    0.0102

```

```
% Estimate asset moments using a table
```

```
q = estimateAssetMoments(q, assetRetnTable);
[passetmean, passetcovar] = getAssetMoments(p)
```

```
passetmean = 4×1
```

```

0.0042
0.0083
0.0100
0.0150

```

```
passetcovar = 4×4
```

```

0.0005    0.0003    0.0002         0
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102

```

```
[qassetmean, qassetcovar] = getAssetMoments(q)
```

```
qassetmean = 4×1
```

```

0.0042
0.0083
0.0100
0.0150

```

```
qassetcovar = 4×4
```

```

0.0005    0.0003    0.0002   -0.0000
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028

```

```
-0.0000    0.0010    0.0028    0.0102
```

As you can see, the moments match between the two portfolios. In addition, `estimateAssetMoments` also extracts asset names or identifiers from a table or timetable when the argument name `'GetAssetList'` is set to `true` (its default value is `false`). If the `'GetAssetList'` value is `true`, the identifiers are used to set the `AssetList` property of the Portfolio object. To show this, the formation of the Portfolio object `q` is repeated from the previous code with the `'GetAssetList'` flag set to `true` and this extracts the column labels from a table or timetable object.

```
q = estimateAssetMoments(q,assetRetnTable,'GetAssetList',true);
disp(q.AssetList)

{'Bonds'}    {'LargeCap'}    {'SmallCap'}    {'Emerging'}
```

Note if you set the `'GetAssetList'` flag set to `true` and your input data is in a matrix, `estimateAssetMoments` uses the default labeling scheme from `setAssetList` described in “Setting Up a List of Asset Identifiers” on page 4-33.

See Also

Portfolio | `setAssetMoments` | `estimateAssetMoments` | `getAssetMoments` | `setCosts`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with a Riskless Asset

You can specify a riskless asset with the mean and covariance of asset returns in the `AssetMean` and `AssetCovar` properties such that the riskless asset has variance of 0 and is completely uncorrelated with all other assets. In this case, the `Portfolio` object uses a separate `RiskFreeRate` property that stores the rate of return of a riskless asset. Thus, you can separate your universe into a riskless asset and a collection of risky assets. For example, assume that your riskless asset has a return in the scalar variable `r0`, then the property for the `RiskFreeRate` is set using the `Portfolio` object:

```
r0 = 0.01/12;  
m = [ 0.05; 0.1; 0.12; 0.18 ];  
C = [ 0.0064 0.00408 0.00192 0;  
      0.00408 0.0289 0.0204 0.0119;  
      0.00192 0.0204 0.0576 0.0336;  
      0.0119 0.0336 0.1225 0];  
  
p = Portfolio('RiskFreeRate', r0, 'AssetMean', m, 'AssetCovar', C);  
disp(p.RiskFreeRate)  
  
8.3333e-004
```

Note If your problem has a budget constraint such that your portfolio weights must sum to 1, then the riskless asset is irrelevant.

See Also

`Portfolio` | `setAssetMoments` | `estimateAssetMoments` | `getAssetMoments`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

Working with Transaction Costs

The difference between net and gross portfolio returns is transaction costs. The net portfolio return proxy has distinct proportional costs to purchase and to sell assets which are maintained in the `Portfolio` object properties `BuyCost` and `SellCost`. Transaction costs are in units of total return and, as such, are proportional to the price of an asset so that they enter the model for net portfolio returns in return form. For example, suppose that you have a stock currently priced \$40 and your usual transaction costs are five cents per share. Then the transaction cost for the stock is $0.05/40 = 0.00125$ (as defined in “Net Portfolio Returns” on page 4-5). Costs are entered as positive values and credits are entered as negative values.

Setting Transaction Costs Using the `Portfolio` Function

To set up transaction costs, you must specify an initial or current portfolio in the `InitPort` property. If the initial portfolio is not set when you set up the transaction cost properties, `InitPort` is 0. The properties for transaction costs can be set using the `Portfolio` object. For example, assume that purchase and sale transaction costs are in the variables `bc` and `sc` and an initial portfolio is in the variable `x0`, then transaction costs are set:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = Portfolio('BuyCost', bc, 'SellCost', sc, 'InitPort', x0);
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
0.0013
0.0013
0.0013
0.0013
0.0013
0.0013
0.0070
0.0013
0.0013
0.0024
0.4000
0.2000
0.2000
0.1000
0.1000
```

Setting Transaction Costs Using the `setCosts` Function

You can also set the properties for transaction costs using `setCosts`. Assume that you have the same costs and initial portfolio as in the previous example. Given a `Portfolio` object `p` with an initial portfolio already set, use `setCosts` to set up transaction costs:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
```

```
p = Portfolio('InitPort', x0);
p = setCosts(p, bc, sc);
```

```
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
0.0013
0.0013
0.0013
0.0013
0.0013
0.0013
0.0070
0.0013
0.0013
0.0024
0.4000
0.2000
0.2000
0.1000
0.1000
```

You can also set up the initial portfolio's `InitPort` value as an optional argument to `setCosts` so that the following is an equivalent way to set up transaction costs:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
```

```
p = Portfolio;
p = setCosts(p, bc, sc, x0);
```

```
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
0.0013
0.0013
0.0013
0.0013
0.0013
0.0013
0.0070
0.0013
0.0013
```

```
0.0013
0.0024

0.4000
0.2000
0.2000
0.1000
0.1000
```

For an example of setting costs, see “Portfolio Analysis with Turnover Constraints” on page 4-211.

Setting Transaction Costs with Scalar Expansion

Both the `Portfolio` object and the `setCosts` function implement scalar expansion on the arguments for transaction costs and the initial portfolio. If the `NumAssets` property is already set in the `Portfolio` object, scalar arguments for these properties are expanded to have the same value across all dimensions. In addition, `setCosts` lets you specify `NumAssets` as an optional final argument. For example, assume that you have an initial portfolio `x0` and you want to set common transaction costs on all assets in your universe. You can set these costs in any of these equivalent ways:

```
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = Portfolio('InitPort', x0, 'BuyCost', 0.002, 'SellCost', 0.002);
```

or

```
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = Portfolio('InitPort', x0);
p = setCosts(p, 0.002, 0.002);
```

or

```
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = Portfolio;
p = setCosts(p, 0.002, 0.002, x0);
```

To clear costs from your `Portfolio` object, use either the `Portfolio` object or `setCosts` with empty inputs for the properties to be cleared. For example, you can clear sales costs from the `Portfolio` object `p` in the previous example:

```
p = Portfolio(p, 'SellCost', []);
```

See Also

`Portfolio` | `setAssetMoments` | `estimateAssetMoments` | `getAssetMoments` | `setCosts`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Portfolio Analysis with Turnover Constraints” on page 4-211

- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Portfolio Constraints Using Defaults

The final element for a complete specification of a portfolio optimization problem is the set of feasible portfolios, which is called a portfolio set. A portfolio set $X \subset R^n$ is specified by construction as the intersection of sets formed by a collection of constraints on portfolio weights. A portfolio set necessarily and sufficiently must be a nonempty, closed, and bounded set.

When setting up your portfolio set, ensure that the portfolio set satisfies these conditions. The most basic or “default” portfolio set requires portfolio weights to be nonnegative (using the lower-bound constraint) and to sum to 1 (using the budget constraint). For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Setting Default Constraints for Portfolio Weights Using Portfolio Object

The “default” portfolio problem has two constraints on portfolio weights:

- Portfolio weights must be nonnegative.
- Portfolio weights must sum to 1.

Implicitly, these constraints imply that portfolio weights are no greater than 1, although this is a superfluous constraint to impose on the problem.

Setting Default Constraints Using the Portfolio Function

Given a portfolio optimization problem with `NumAssets` = 20 assets, use the `Portfolio` object to set up a default problem and explicitly set bounds and budget constraints:

```
p = Portfolio('NumAssets', 20, 'LowerBound', 0, 'Budget', 1);  
disp(p)
```

Portfolio with properties:

```
BuyCost: []  
SellCost: []  
RiskFreeRate: []  
AssetMean: []  
AssetCovar: []  
TrackingError: []  
TrackingPort: []  
Turnover: []  
BuyTurnover: []  
SellTurnover: []  
Name: []  
NumAssets: 20  
AssetList: []  
InitPort: []  
AInequality: []  
bInequality: []  
AEquality: []  
bEquality: []  
LowerBound: [20x1 double]  
UpperBound: []  
LowerBudget: 1
```

```

        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
            GroupA: []
            GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
        BoundType: []

```

Setting Default Constraints Using the `setDefaultConstraints` Function

An alternative approach is to use the `setDefaultConstraints` function. If the number of assets is already known in a `Portfolio` object, use `setDefaultConstraints` with no arguments to set up the necessary bound and budget constraints. Suppose that you have 20 assets to set up the portfolio set for a default problem:

```

p = Portfolio('NumAssets', 20);
p = setDefaultConstraints(p);
disp(p)

```

Portfolio with properties:

```

        BuyCost: []
        SellCost: []
RiskFreeRate: []
        AssetMean: []
        AssetCovar: []
TrackingError: []
TrackingPort: []
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
        Name: []
        NumAssets: 20
        AssetList: []
        InitPort: []
AInequality: []
bInequality: []
        AEquality: []
        bEquality: []
LowerBound: [20×1 double]
UpperBound: []
LowerBudget: 1
UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
            GroupA: []
            GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []

```

```
ConditionalBudgetThreshold: []  
ConditionalUpperBudget: []  
BoundType: [20×1 categorical]
```

If the number of assets is unknown, `setDefaultConstraints` accepts `NumAssets` as an optional argument to form a portfolio set for a default problem. Suppose that you have 20 assets:

```
p = Portfolio;  
p = setDefaultConstraints(p, 20);  
disp(p)
```

Portfolio with properties:

```
BuyCost: []  
SellCost: []  
RiskFreeRate: []  
AssetMean: []  
AssetCovar: []  
TrackingError: []  
TrackingPort: []  
Turnover: []  
BuyTurnover: []  
SellTurnover: []  
Name: []  
NumAssets: 20  
AssetList: []  
InitPort: []  
AInequality: []  
bInequality: []  
AEquality: []  
bEquality: []  
LowerBound: [20×1 double]  
UpperBound: []  
LowerBudget: 1  
UpperBudget: 1  
GroupMatrix: []  
LowerGroup: []  
UpperGroup: []  
GroupA: []  
GroupB: []  
LowerRatio: []  
UpperRatio: []  
MinNumAssets: []  
MaxNumAssets: []  
ConditionalBudgetThreshold: []  
ConditionalUpperBudget: []  
BoundType: [20×1 categorical]
```

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Working with ‘Simple’ Bound Constraints Using Portfolio Object” on page 4-62

- “Working with Budget Constraints Using Portfolio Object” on page 4-65
- “Working with Group Constraints Using Portfolio Object” on page 4-69
- “Working with Group Ratio Constraints Using Portfolio Object” on page 4-72
- “Working with Linear Equality Constraints Using Portfolio Object” on page 4-75
- “Working with Linear Inequality Constraints Using Portfolio Object” on page 4-78
- “Working with Average Turnover Constraints Using Portfolio Object” on page 4-85
- “Working with One-Way Turnover Constraints Using Portfolio Object” on page 4-88
- “Working with Tracking Error Constraints Using Portfolio Object” on page 4-91
- “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81
- “Creating the Portfolio Object” on page 4-25
- “Adding Constraints to Satisfy UCITS Directive” on page 4-457
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with 'Simple' Bound Constraints Using Portfolio Object

'Simple' bound constraints are optional linear constraints that maintain upper and lower bounds on portfolio weights (see "Simple' Bound Constraints" on page 4-10). Although every portfolio set must be bounded, it is not necessary to specify a portfolio set with explicit bound constraints. For example, you can create a portfolio set with an implicit upper bound constraint or a portfolio set with average turnover constraints. The bound constraints have properties `LowerBound` for the lower-bound constraint and `UpperBound` for the upper-bound constraint. Set default values for these constraints using the `setDefaultConstraints` function (see "Setting Default Constraints for Portfolio Weights Using Portfolio Object" on page 4-58).

Setting 'Simple' Bounds Using the Portfolio Function

The properties for bound constraints are set through the `Portfolio` object. Suppose that you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. The bound constraints for a balanced fund are set with:

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];
p = Portfolio('LowerBound', lb, 'UpperBound', ub, 'BoundType', 'Simple');
disp(p.NumAssets)
disp(p.LowerBound)
disp(p.UpperBound)

2

0.5000
0.2500

0.7500
0.5000
```

To continue with this example, you must set up a budget constraint. For details, see "Working with Budget Constraints Using Portfolio Object" on page 4-65.

Setting 'Simple' Bounds Using the setBounds Function

You can also set the properties for bound constraints using `setBounds`. Suppose that you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. Given a `Portfolio` object `p`, use `setBounds` to set the bound constraints:

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];
p = Portfolio;
p = setBounds(p, lb, ub, 'BoundType', 'Simple');
disp(p.NumAssets)
disp(p.LowerBound)
disp(p.UpperBound)

2

0.5000
0.2500
```

```
0.7500
0.5000
```

Setting 'Simple' Bounds Using the Portfolio Function or setBounds Function

Both the `Portfolio` object and `setBounds` function implement scalar expansion on either the `LowerBound` or `UpperBound` properties. If the `NumAssets` property is already set in the `Portfolio` object, scalar arguments for either property expand to have the same value across all dimensions. In addition, `setBounds` lets you specify `NumAssets` as an optional argument. Suppose that you have a universe of 500 assets and you want to set common bound constraints on all assets in your universe. Specifically, you are a long-only investor and want to hold no more than 5% of your portfolio in any single asset. You can set these bound constraints in any of these equivalent ways:

```
p = Portfolio('NumAssets', 500, 'LowerBound', 0, 'UpperBound', 0.05, 'BoundType', 'Simple');
```

or

```
p = Portfolio('NumAssets', 500);
p = setBounds(p, 0, 0.05, 'BoundType', 'Simple');
```

or

```
p = Portfolio;
p = setBounds(p, 0, 0.05, 'NumAssets', 500, 'BoundType', 'Simple');
```

To clear bound constraints from your `Portfolio` object, use either the `Portfolio` object or `setBounds` with empty inputs for the properties to be cleared. For example, to clear the upper-bound constraint from the `Portfolio` object `p` in the previous example:

```
p = Portfolio(p, 'UpperBound', []);
```

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190

- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Budget Constraints Using Portfolio Object

The budget constraint is an optional linear constraint that maintains upper and lower bounds on the sum of portfolio weights (see “Budget Constraints” on page 4-11). Budget constraints have properties `LowerBudget` for the lower budget constraint and `UpperBudget` for the upper budget constraint. If you set up a portfolio optimization problem that requires portfolios to be fully invested in your universe of assets, you can set `LowerBudget` to be equal to `UpperBudget`. These budget constraints can be set with default values equal to 1 using `setDefaultConstraints` (see “Setting Default Constraints for Portfolio Weights Using Portfolio Object” on page 4-58).

Setting Budget Constraints Using the Portfolio Function

The properties for the budget constraint can also be set using the `Portfolio` object. Suppose that you have an asset universe with many risky assets and a riskless asset and you want to ensure that your portfolio never holds more than 1% cash, that is, you want to ensure that you are 99-100% invested in risky assets. The budget constraint for this portfolio can be set with:

```
p = Portfolio('LowerBudget', 0.99, 'UpperBudget', 1);
disp(p.LowerBudget)
disp(p.UpperBudget)

0.9900

1
```

Setting Budget Constraints Using the setBudget Function

You can also set the properties for a budget constraint using `setBudget`. Suppose that you have a fund that permits up to 10% leverage which means that your portfolio can be from 100% to 110% invested in risky assets. Given a `Portfolio` object `p`, use `setBudget` to set the budget constraints:

```
p = Portfolio;
p = setBudget(p, 1, 1.1);
disp(p.LowerBudget)
disp(p.UpperBudget)

1

1.1000
```

If you were to continue with this example, then set the `RiskFreeRate` property to the borrowing rate to finance possible leveraged positions. For details on the `RiskFreeRate` property, see “Working with a Riskless Asset” on page 4-52. To clear either bound for the budget constraint from your `Portfolio` object, use either the `Portfolio` object or `setBudget` with empty inputs for the properties to be cleared. For example, clear the upper-budget constraint from the `Portfolio` object `p` in the previous example with:

```
p = Portfolio(p, 'UpperBudget', []);
```

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Conditional Budget Constraints Using Portfolio Object

The conditional budget constraint supports the Undertakings for Collective Investment in Transferable Securities (UCITS) directive for `Portfolio` objects. The UCITS asset allocation regulation states that funds can only invest up to 10% in a single issuer, and that investments in excess of 5% must not exceed 40% of the total portfolio. This is known as the 5/10/40 rule. For more information, see “Undertakings for Collective Investment in Transferable Securities” on page 15-1452 and “Adding Constraints to Satisfy UCITS Directive” on page 4-457.

Setting Conditional Budget Constraints Using the Portfolio Function

The properties for the conditional budget constraint can also be set using the `Portfolio` object. Suppose that you have an asset universe with many risky assets and a riskless asset and you want to ensure that your portfolio never holds more than 1% cash, that is, you want to ensure that you are 99-100% invested in risky assets. The conditional budget constraint for this portfolio can be set with:

```
p = Portfolio('ConditionalBudgetThreshold', 0.99, 'ConditionalUpperBudget', 1);
disp(p.ConditionalBudgetThreshold)
disp(p.ConditionalUpperBudget)

0.9900

1
```

Setting Conditional Budget Constraints Using the setConditionalBudget Function

You can also set the properties for a conditional budget constraint using `setConditionalBudget`. Suppose that you have a fund that permits up to 10% leverage which means that your portfolio can be from 100% to 110% invested in risky assets. Given a `Portfolio` object `p`, use `setConditionalBudget` to set the conditional budget constraints:

```
p = Portfolio;
p = setConditionBudget(p, .40, 1.5);
disp(p.threshold)
disp(p.upperBudget)

.40

1.5000
```

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58

- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Group Constraints Using Portfolio Object

Group constraints are optional linear constraints that group assets together and enforce bounds on the group weights (see “Group Constraints” on page 4-12). Although the constraints are implemented as general constraints, the usual convention is to form a group matrix that identifies membership of each asset within a specific group with Boolean indicators (either `true` or `false` or with 1 or 0) for each element in the group matrix. Group constraints have properties `GroupMatrix` for the group membership matrix, `LowerGroup` for the lower-bound constraint on groups, and `UpperGroup` for the upper-bound constraint on groups.

Setting Group Constraints Using the Portfolio Function

The properties for group constraints are set through the `Portfolio` object. Suppose that you have a portfolio of five assets and want to ensure that the first three assets constitute no more than 30% of your portfolio, then you can set group constraints:

```
G = [ 1 1 1 0 0 ];
p = Portfolio('GroupMatrix', G, 'UpperGroup', 0.3);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.UpperGroup)
```

5

1 1 1 0 0

0.3000

The group matrix `G` can also be a logical matrix so that the following code achieves the same result.

```
G = [ true true true false false ];
p = Portfolio('GroupMatrix', G, 'UpperGroup', 0.3);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.UpperGroup)
```

5

1 1 1 0 0

0.3000

Setting Group Constraints Using the `setGroups` and `addGroups` Functions

You can also set the properties for group constraints using `setGroups`. Suppose that you have a portfolio of five assets and want to ensure that the first three assets constitute no more than 30% of your portfolio. Given a `Portfolio` object `p`, use `setGroups` to set the group constraints:

```
G = [ true true true false false ];
p = Portfolio;
p = setGroups(p, G, [], 0.3);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.UpperGroup)
```

```

5
1      1      1      0      0
0.3000

```

In this example, you would set the `LowerGroup` property to be empty (`[]`).

Suppose that you want to add another group constraint to make odd-numbered assets constitute at least 20% of your portfolio. Set up an augmented group matrix and introduce infinite bounds for unconstrained group bounds or use the `addGroups` function to build up group constraints. For this example, create another group matrix for the second group constraint:

```

p = Portfolio;
G = [ true true true false false ]; % group matrix for first group constraint
p = setGroups(p, G, [], 0.3);
G = [ true false true false true ]; % group matrix for second group constraint
p = addGroups(p, G, 0.2);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.LowerGroup)
disp(p.UpperGroup)

5
1      1      1      0      0
1      0      1      0      1

-Inf
0.2000

0.3000
Inf

```

`addGroups` determines which bounds are unbounded so you only need to focus on the constraints that you want to set.

The `Portfolio` object and `setGroups` and `addGroups` implement scalar expansion on either the `LowerGroup` or `UpperGroup` properties based on the dimension of the group matrix in the property `GroupMatrix`. Suppose that you have a universe of 30 assets with six asset classes such that assets 1–5, assets 6–12, assets 13–18, assets 19–22, assets 23–27, and assets 28–30 constitute each of your six asset classes and you want each asset class to fall from 0% to 25% of your portfolio. Let the following group matrix define your groups and scalar expansion define the common bounds on each group:

```

p = Portfolio;
G = blkdiag(true(1,5), true(1,7), true(1,6), true(1,4), true(1,5), true(1,3));
p = setGroups(p, G, 0, 0.25);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.LowerGroup)
disp(p.UpperGroup)

30

Columns 1 through 16

1      1      1      1      1      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      1      1      1      1      1      1      1      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      1      1      1
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0

Columns 17 through 30

0      0      0      0      0      0      0      0      0      0      0      0      0      0

```

```

0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 1 1 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 1 1 1 1 0 0 0
0 0 0 0 0 0 0 0 0 0 1 1 1 1

0
0
0
0
0
0

0.2500
0.2500
0.2500
0.2500
0.2500
0.2500

```

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Group Ratio Constraints Using Portfolio Object

Group ratio constraints are optional linear constraints that maintain bounds on proportional relationships among groups of assets (see “Group Ratio Constraints” on page 4-13). Although the constraints are implemented as general constraints, the usual convention is to specify a pair of group matrices that identify membership of each asset within specific groups with Boolean indicators (either `true` or `false` or with 1 or 0) for each element in each of the group matrices. The goal is to ensure that the ratio of a base group compared to a comparison group fall within specified bounds. Group ratio constraints have properties:

- `GroupA` for the base membership matrix
- `GroupB` for the comparison membership matrix
- `LowerRatio` for the lower-bound constraint on the ratio of groups
- `UpperRatio` for the upper-bound constraint on the ratio of groups

Setting Group Ratio Constraints Using the Portfolio Function

The properties for group ratio constraints are set using the `Portfolio` object. For example, assume that you want the ratio of financial to nonfinancial companies in your portfolios to never go above 50%. Suppose that you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). To set group ratio constraints:

```
GA = [ 1 1 1 0 0 0 ]; % financial companies
GB = [ 0 0 0 1 1 1 ]; % nonfinancial companies
p = Portfolio('GroupA', GA, 'GroupB', GB, 'UpperRatio', 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

```
6
1      1      1      0      0      0
0      0      0      1      1      1
0.5000
```

Group matrices `GA` and `GB` in this example can be logical matrices with `true` and `false` elements that yield the same result:

```
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = Portfolio('GroupA', GA, 'GroupB', GB, 'UpperRatio', 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

```
6
1      1      1      0      0      0
0      0      0      1      1      1
0.5000
```


Setting Group Ratio Constraints Using the `setGroupRatio` and `addGroupRatio` Functions

You can also set the properties for group ratio constraints using `setGroupRatio`. For example, assume that you want the ratio of financial to nonfinancial companies in your portfolios to never go above 50%. Suppose that you have six assets with three financial companies (assets 1–3) and three nonfinancial companies (assets 4–6). Given a `Portfolio` object `p`, use `setGroupRatio` to set the group constraints:

```
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = Portfolio;
p = setGroupRatio(p, GA, GB, [], 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

6

```
1      1      1      0      0      0
0      0      0      1      1      1
```

0.5000

In this example, you would set the `LowerRatio` property to be empty (`[]`).

Suppose that you want to add another group ratio constraint to ensure that the weights in odd-numbered assets constitute at least 20% of the weights in nonfinancial assets your portfolio. You can set up augmented group ratio matrices and introduce infinite bounds for unconstrained group ratio bounds, or you can use the `addGroupRatio` function to build up group ratio constraints. For this example, create another group matrix for the second group constraint:

```
p = Portfolio;
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = setGroupRatio(p, GA, GB, [], 0.5);

GA = [ true false true false true false ]; % odd-numbered companies
GB = [ false false false true true true ]; % nonfinancial companies
p = addGroupRatio(p, GA, GB, 0.2);
```

```
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.LowerRatio)
disp(p.UpperRatio)
```

6

```
1      1      1      0      0      0
1      0      1      0      1      0

0      0      0      1      1      1
0      0      0      1      1      1
```

-Inf
0.2000

0.5000
Inf

Notice that `addGroupRatio` determines which bounds are unbounded so you only need to focus on the constraints you want to set.

The `Portfolio` object, `setGroupRatio`, and `addGroupRatio` implement scalar expansion on either the `LowerRatio` or `UpperRatio` properties based on the dimension of the group matrices in `GroupA` and `GroupB` properties.

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Linear Equality Constraints Using Portfolio Object

Linear equality constraints are optional linear constraints that impose systems of equalities on portfolio weights (see “Linear Equality Constraints” on page 4-10). Linear equality constraints have properties `AEquality`, for the equality constraint matrix, and `bEquality`, for the equality constraint vector.

Setting Linear Equality Constraints Using the Portfolio Function

The properties for linear equality constraints are set using the `Portfolio` object. Suppose that you have a portfolio of five assets and want to ensure that the first three assets are 50% of your portfolio. To set this constraint:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = Portfolio('AEquality', A, 'bEquality', b);
disp(p.NumAssets)
disp(p.AEquality)
disp(p.bEquality)
```

5

1 1 1 0 0

0.5000

Setting Linear Equality Constraints Using the `setEquality` and `addEquality` Functions

You can also set the properties for linear equality constraints using `setEquality`. Suppose that you have a portfolio of five assets and want to ensure that the first three assets are 50% of your portfolio. Given a `Portfolio` object `p`, use `setEquality` to set the linear equality constraints:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = Portfolio;
p = setEquality(p, A, b);
disp(p.NumAssets)
disp(p.AEquality)
disp(p.bEquality)
```

5

1 1 1 0 0

0.5000

Suppose that you want to add another linear equality constraint to ensure that the last three assets also constitute 50% of your portfolio. You can set up an augmented system of linear equalities or use `addEquality` to build up linear equality constraints. For this example, create another system of equalities:

```
p = Portfolio;
A = [ 1 1 1 0 0 ];      % first equality constraint
```

```

b = 0.5;
p = setEquality(p, A, b);

A = [ 0 0 1 1 1 ];    % second equality constraint
b = 0.5;
p = addEquality(p, A, b);

disp(p.NumAssets)
disp(p.AEquality)
disp(p.bEquality)

5

1      1      1      0      0
0      0      1      1      1

0.5000
0.5000

```

The Portfolio object, `setEquality`, and `addEquality` implement scalar expansion on the `bEquality` property based on the dimension of the matrix in the `AEquality` property.

See Also

Portfolio | setDefaultConstraints | setBounds | setBudget | setConditionalBudget | setGroups | setGroupRatio | setEquality | setInequality | setTurnover | setOneWayTurnover | setTrackingPort | setTrackingError

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Linear Inequality Constraints Using Portfolio Object

Linear inequality constraints are optional linear constraints that impose systems of inequalities on portfolio weights (see “Linear Inequality Constraints” on page 4-9). Linear inequality constraints have properties `AInequality` for the inequality constraint matrix, and `bInequality` for the inequality constraint vector.

Setting Linear Inequality Constraints Using the Portfolio Function

The properties for linear inequality constraints are set using the `Portfolio` object. Suppose that you have a portfolio of five assets and you want to ensure that the first three assets are no more than 50% of your portfolio. To set up these constraints:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = Portfolio('AInequality', A, 'bInequality', b);
disp(p.NumAssets)
disp(p.AInequality)
disp(p.bInequality)
```

5

1 1 1 0 0

0.5000

Setting Linear Inequality Constraints Using the `setInequality` and `addInequality` Functions

You can also set the properties for linear inequality constraints using `setInequality`. Suppose that you have a portfolio of five assets and you want to ensure that the first three assets constitute no more than 50% of your portfolio. Given a `Portfolio` object `p`, use `setInequality` to set the linear inequality constraints:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = Portfolio;
p = setInequality(p, A, b);
disp(p.NumAssets)
disp(p.AInequality)
disp(p.bInequality)
```

5

1 1 1 0 0

0.5000

Suppose that you want to add another linear inequality constraint to ensure that the last three assets constitute at least 50% of your portfolio. You can set up an augmented system of linear inequalities or use the `addInequality` function to build up linear inequality constraints. For this example, create another system of inequalities:

```

p = Portfolio;
A = [ 1 1 1 0 0 ];    % first inequality constraint
b = 0.5;
p = setInequality(p, A, b);

A = [ 0 0 -1 -1 -1 ];    % second inequality constraint
b = -0.5;
p = addInequality(p, A, b);

disp(p.NumAssets)
disp(p.AInequality)
disp(p.bInequality)

5

1      1      1      0      0
0      0     -1     -1     -1

0.5000
-0.5000

```

The Portfolio object, `setInequality`, and `addInequality` implement scalar expansion on the `bInequality` property based on the dimension of the matrix in the `AInequality` property.

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4

- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects

When any one, or any combination of 'Conditional' BoundType, MinNumAssets, or MaxNumAssets constraints are active, the portfolio problem is formulated by adding NumAssets binary variables, where 0 indicates not invested, and 1 is invested. For example, to explain the 'Conditional' BoundType and MinNumAssets and MaxNumAssets constraints, assume that your portfolio has a universe of 100 assets that you want to invest:

- 'Conditional' BoundType (also known as semicontinuous constraints), set by `setBounds`, is often used in situations where you do not want to invest small values. A standard example is a portfolio optimization problem where many small allocations are not attractive because of transaction costs. Instead, you prefer fewer instruments in the portfolio with larger allocations. This situation can be handled using 'Conditional' BoundType constraints for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object.

For example, the weight you invest in each asset is either 0 or between $[0.01, 0.5]$. Generally, a semicontinuous variable x is a continuous variable between bounds $[lb, ub]$ that also can assume the value 0, where $lb > 0$, $lb \leq ub$. Applying this to portfolio optimization requires that very small or large positions should be avoided, that is values that fall in $(0, lb)$ or are more than ub .

- `MinNumAssets` and `MaxNumAssets` (also known as cardinality constraints), set by `setMinMaxNumAssets`, limit the number of assets in a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For example, if you have 100 assets in your portfolio and you want the number of assets allocated in the portfolio to be from 40 through 60. Using `MinNumAssets` and `MaxNumAssets` you can limit the number of assets in the optimized portfolio, which allows you to limit transaction and operational costs or to create an index tracking portfolio.

Setting 'Conditional' BoundType Constraints Using the `setBounds` Function

Use `setBounds` with a 'conditional' BoundType to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i=1, \dots, \text{NumAssets}$:

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ]; p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3)
```

Portfolio with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
AssetMean: [3x1 double]
AssetCovar: [3x3 double]
TrackingError: []
TrackingPort: []
Turnover: []
BuyTurnover: []
```

```

SellTurnover: []
    Name: []
    NumAssets: 3
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [3×1 double]
    UpperBound: [3×1 double]
    LowerBudget: []
    UpperBudget: []
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
        GroupA: []
        GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
    BoundType: [3×1 categorical]

```

Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function

You can also set the `MinNumAssets` and `MaxNumAssets` properties to define a limit on the number of assets invested using `setMinMaxNumAssets`. For example, by setting `MinNumAssets=MaxNumAssets=2`, only two of the three assets are invested in the portfolio.

```

AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setMinMaxNumAssets(p, 2, 2)

p =

```

Portfolio with properties:

```

    BuyCost: []
    SellCost: []
RiskFreeRate: []
    AssetMean: [3×1 double]
    AssetCovar: [3×3 double]
TrackingError: []
TrackingPort: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
        Name: []
        NumAssets: 3

```

```
AssetList: []  
InitPort: []  
AInequality: []  
bInequality: []  
AEquality: []  
bEquality: []  
LowerBound: []  
UpperBound: []  
LowerBudget: []  
UpperBudget: []  
GroupMatrix: []  
LowerGroup: []  
UpperGroup: []  
GroupA: []  
GroupB: []  
LowerRatio: []  
UpperRatio: []  
MinNumAssets: 2  
MaxNumAssets: 2  
ConditionalBudgetThreshold: []  
ConditionalUpperBudget: []  
BoundType: []
```

See Also

`Portfolio` | `setBounds` | `setMinMaxNumAssets` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4

- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Average Turnover Constraints Using Portfolio Object

The turnover constraint is an optional linear absolute value constraint (see “Average Turnover Constraints” on page 4-14) that enforces an upper bound on the average of purchases and sales. The turnover constraint can be set using the `Portfolio` object or the `setTurnover` function. The turnover constraint depends on an initial or current portfolio, which is assumed to be zero if not set when the turnover constraint is set. The turnover constraint has properties `Turnover`, for the upper bound on average turnover, and `InitPort`, for the portfolio against which turnover is computed.

Setting Average Turnover Constraints Using the Portfolio Function

The properties for the turnover constraints are set using the `Portfolio` object. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and you want to ensure that average turnover is no more than 30%. To set this turnover constraint:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio('Turnover', 0.3, 'InitPort', x0);
disp(p.NumAssets)
disp(p.Turnover)
disp(p.InitPort)
```

```
10
```

```
0.3000
```

```
0.1200
```

```
0.0900
```

```
0.0800
```

```
0.0700
```

```
0.1000
```

```
0.1000
```

```
0.1500
```

```
0.1100
```

```
0.0800
```

```
0.1000
```

Note if the `NumAssets` or `InitPort` properties are not set before or when the turnover constraint is set, various rules are applied to assign default values to these properties (see “Setting Up an Initial or Current Portfolio” on page 4-37).

Setting Average Turnover Constraints Using the setTurnover Function

You can also set properties for portfolio turnover using `setTurnover` to specify both the upper bound for average turnover and an initial portfolio. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and want to ensure that average turnover is no more than 30%. Given a `Portfolio` object `p`, use `setTurnover` to set the turnover constraint with and without the initial portfolio being set previously:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio('InitPort', x0);
p = setTurnover(p, 0.3);
```

```
disp(p.NumAssets)
```

```
disp(p.Turnover)
```

```
disp(p.InitPort)
```

```
10

0.3000

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000

or

x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio;
p = setTurnover(p, 0.3, x0);
disp(p.NumAssets)
disp(p.Turnover)
disp(p.InitPort)

10

0.3000

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

For an example of setting turnover, see “Portfolio Analysis with Turnover Constraints” on page 4-211.

`setTurnover` implements scalar expansion on the argument for the initial portfolio. If the `NumAssets` property is already set in the `Portfolio` object, a scalar argument for `InitPort` expands to have the same value across all dimensions. In addition, `setTurnover` lets you specify `NumAssets` as an optional argument. To clear turnover from your `Portfolio` object, use the `Portfolio` object or `setTurnover` with empty inputs for the properties to be cleared.

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58

- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Portfolio Analysis with Turnover Constraints” on page 4-211
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Analysis with Turnover Constraints” on page 4-211
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with One-Way Turnover Constraints Using Portfolio Object

One-way turnover constraints are optional constraints (see “One-Way Turnover Constraints” on page 4-14) that enforce upper bounds on net purchases or net sales. One-way turnover constraints can be set using the `Portfolio` object or the `setOneWayTurnover` function. One-way turnover constraints depend upon an initial or current portfolio, which is assumed to be zero if not set when the turnover constraints are set. One-way turnover constraints have properties `BuyTurnover`, for the upper bound on net purchases, `SellTurnover`, for the upper bound on net sales, and `InitPort`, for the portfolio against which turnover is computed.

Setting One-Way Turnover Constraints Using the Portfolio Function

The Properties for the one-way turnover constraints are set using the `Portfolio` object. Suppose that you have an initial portfolio with 10 assets in a variable `x0` and you want to ensure that turnover on purchases is no more than 30% and turnover on sales is no more than 20% of the initial portfolio. To set these turnover constraints:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio('BuyTurnover', 0.3, 'SellTurnover', 0.2, 'InitPort', x0);
disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)
```

10

0.3000

0.2000

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000

If the `NumAssets` or `InitPort` properties are not set before or when the turnover constraint is set, various rules are applied to assign default values to these properties (see “Setting Up an Initial or Current Portfolio” on page 4-37).

Setting Turnover Constraints Using the `setOneWayTurnover` Function

You can also set properties for portfolio turnover using `setOneWayTurnover` to specify the upper bounds for turnover on purchases (`BuyTurnover`) and sales (`SellTurnover`) and an initial portfolio. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and want to ensure that turnover on purchases is no more than 30% and that turnover on sales is no more than 20% of the initial portfolio. Given a `Portfolio` object `p`, use `setOneWayTurnover` to set the turnover constraints with and without the initial portfolio being set previously:


```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio('InitPort', x0);
p = setOneWayTurnover(p, 0.3, 0.2);
```

```
disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)
```

```
10
0.3000
0.2000
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

or

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio;
p = setOneWayTurnover(p, 0.3, 0.2, x0);
disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)
```

```
10
0.3000
0.2000
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

`setOneWayTurnover` implements scalar expansion on the argument for the initial portfolio. If the `NumAssets` property is already set in the `Portfolio` object, a scalar argument for `InitPort` expands to have the same value across all dimensions. In addition, `setOneWayTurnover` lets you specify `NumAssets` as an optional argument. To remove one-way turnover from your `Portfolio` object, use the `Portfolio` object or `setOneWayTurnover` with empty inputs for the properties to be cleared.

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Tracking Error Constraints Using Portfolio Object

Tracking error constraints are optional constraints (see “Tracking Error Constraints” on page 4-15) that measure the risk relative to a portfolio called a tracking portfolio. Tracking error constraints can be set using the `Portfolio` object or the `setTrackingError` function.

The tracking error constraint is an optional quadratic constraint that enforces an upper bound on tracking error, which is the relative risk between a portfolio and a designated tracking portfolio. For more information, see “Tracking Error Constraints” on page 4-15.

The tracking error constraint can be set using the `Portfolio` object or the `setTrackingPort` and `setTrackingError` functions. The tracking error constraint depends on a tracking portfolio, which is assumed to be zero if not set when the tracking error constraint is set. The tracking error constraint has properties `TrackingError`, for the upper bound on tracking error, and `TrackingPort`, for the portfolio against which tracking error is computed.

Note The initial portfolio in the `Portfolio` object property `InitPort` is distinct from the tracking portfolio in the `Portfolio` object property `TrackingPort`.

Setting Tracking Error Constraints Using the Portfolio Function

The properties for the tracking error constraints are set using the `Portfolio` object. Suppose that you have a tracking portfolio of 10 assets in a variable `x0` and you want to ensure that the tracking error of any portfolio on the efficient frontier is no more than 8% relative to this portfolio. To set this constraint:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio('TrackingError', 0.08, 'TrackingPort', x0);
disp(p.NumAssets)
disp(p.TrackingError)
disp(p.TrackingPort)
```

```
10
```

```
0.0800
```

```
0.1200
```

```
0.0900
```

```
0.0800
```

```
0.0700
```

```
0.1000
```

```
0.1000
```

```
0.1500
```

```
0.1100
```

```
0.0800
```

```
0.1000
```

If the `NumAssets` or `TrackingPort` properties are not set before or when the tracking error constraint is set, various rules are applied to assign default values to these properties (see “Setting Up a Tracking Portfolio” on page 4-40).

Setting Tracking Error Constraints Using the `setTrackingError` Function

You can also set properties for portfolio tracking error using the `setTrackingError` function to specify both the upper bound for tracking error and a designated tracking portfolio. Suppose that you

have a tracking portfolio of 10 assets in a variable `x0` and want to ensure that tracking error is no more than 8%. Given a `Portfolio` object `p`, use `setTrackingError` to set the tracking error constraint with and without the initial portfolio being set previously:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio('TrackingPort', x0);
p = setTrackingError(p, 0.08);
```

```
disp(p.NumAssets)
disp(p.TrackingError)
disp(p.TrackingPort)
```

```
10

0.0800

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

or

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio('TrackingPort', x0);
p = setTrackingError(p, 0.08, x0);
```

```
disp(p.NumAssets)
disp(p.TrackingError)
disp(p.TrackingPort)
```

```
10

0.0800

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

If the `NumAssets` or `TrackingPort` properties are not set before or when the tracking error constraint is set, various rules are applied to assign default values to these properties (see “Setting Up a Tracking Portfolio” on page 4-40).

`setTrackingError` implements scalar expansion on the argument for the tracking portfolio. If the `NumAssets` property is already set in the `Portfolio` object, a scalar argument for `TrackingPort` expands to have the same value across all dimensions. In addition, `setTrackingError` lets you specify `NumAssets` as an optional argument. To clear tracking error from your `Portfolio` object, use the `Portfolio` object or `setTrackingError` with empty inputs for the properties to be cleared.

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Validate the Portfolio Problem for Portfolio Object

In this section...

“Validating a Portfolio Set” on page 4-94

“Validating Portfolios” on page 4-95

Sometimes, you may want to validate either your inputs to, or outputs from, a portfolio optimization problem. Although most error checking that occurs during the problem setup phase catches most difficulties with a portfolio optimization problem, the processes to validate portfolio sets and portfolios are time consuming and are best done offline. So, the portfolio optimization tools have specialized functions to validate portfolio sets and portfolios. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Validating a Portfolio Set

Since it is necessary and sufficient that your portfolio set must be a nonempty, closed, and bounded set to have a valid portfolio optimization problem, the `estimateBounds` function lets you examine your portfolio set to determine if it is nonempty and, if nonempty, whether it is bounded. Suppose that you have the following portfolio set which is an empty set because the initial portfolio at 0 is too far from a portfolio that satisfies the budget and turnover constraint:

```
p = Portfolio('NumAssets', 3, 'Budget', 1);
p = setTurnover(p, 0.3, 0);
```

If a portfolio set is empty, `estimateBounds` returns NaN bounds and sets the `isbounded` flag to []:

```
[lb, ub, isbounded] = estimateBounds(p)
```

```
lb =
```

```
NaN
NaN
NaN
```

```
ub =
```

```
NaN
NaN
NaN
```

```
isbounded =
```

```
[]
```

Suppose that you create an unbounded portfolio set as follows:

```
p = Portfolio('AInequality', [1 -1; 1 1], 'bInequality', 0);
[lb, ub, isbounded] = estimateBounds(p)
```

```
lb =
```

```
-Inf
-Inf
```

```

ub =

    1.0e-08 *
    -0.3712
    Inf

isbounded =

    logical

    0

```

In this case, `estimateBounds` returns (possibly infinite) bounds and sets the `isbounded` flag to `false`. The result shows which assets are unbounded so that you can apply bound constraints as necessary.

Finally, suppose that you created a portfolio set that is both nonempty and bounded. `estimateBounds` not only validates the set, but also obtains tighter bounds which are useful if you are concerned with the actual range of portfolio choices for individual assets in your portfolio set:

```

p = Portfolio;
p = setBudget(p, 1,1);
p = setBounds(p, [ -0.1; 0.2; 0.3; 0.2 ], [ 0.5; 0.3; 0.9; 0.8 ]);

[lb, ub, isbounded] = estimateBounds(p)

lb =

    -0.1000
     0.2000
     0.3000
     0.2000

ub =

     0.3000
     0.3000
     0.7000
     0.6000

isbounded =

    logical

     1

```

In this example, all but the second asset has tighter upper bounds than the input upper bound implies.

Validating Portfolios

Given a portfolio set specified in a `Portfolio` object, you often want to check if specific portfolios are feasible with respect to the portfolio set. This can occur with, for example, initial portfolios and with portfolios obtained from other procedures. The `checkFeasibility` function determines

whether a collection of portfolios is feasible. Suppose that you perform the following portfolio optimization and want to determine if the resultant efficient portfolios are feasible relative to a modified problem.

First, set up a problem in the `Portfolio` object `p`, estimate efficient portfolios in `pwgt`, and then confirm that these portfolios are feasible relative to the initial problem:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontier(p);
```

```
checkFeasibility(p, pwgt)
```

```
ans =
```

```
1      1      1      1      1      1      1      1      1      1
```

Next, set up a different portfolio problem that starts with the initial problem with an additional a turnover constraint and an equally weighted initial portfolio:

```
q = setTurnover(p, 0.3, 0.25);
checkFeasibility(q, pwgt)
```

```
ans =
```

```
0      0      0      1      1      0      0      0      0      0
```

In this case, only two of the 10 efficient portfolios from the initial problem are feasible relative to the new problem in `Portfolio` object `q`. Solving the second problem using `checkFeasibility` demonstrates that the efficient portfolio for `Portfolio` object `q` is feasible relative to the initial problem:

```
qwgt = estimateFrontier(q);
checkFeasibility(p, qwgt)
```

```
ans =
```

```
1      1      1      1      1      1      1      1      1      1
```

See Also

`Portfolio` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122

- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object

There are two ways to look at a portfolio optimization problem and this depends on what is your goal:

- To estimate efficient portfolios, see “Obtaining Portfolios Along the Entire Efficient Frontier” on page 4-99.
- To estimate efficient frontiers, see “Estimate Efficient Frontiers for Portfolio Object” on page 4-122.

For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Obtaining Portfolios Along the Entire Efficient Frontier

The most basic way to obtain optimal portfolios is to obtain points over the entire range of the efficient frontier.

Given a portfolio optimization problem in a `Portfolio` object, the `estimateFrontier` function computes efficient portfolios spaced evenly according to the return proxy from the minimum to maximum return efficient portfolios. The number of portfolios estimated is controlled by the hidden property `defaultNumPorts` which is set to 10. A different value for the number of portfolios estimated is specified as an input argument to `estimateFrontier`. This example shows the default number of efficient portfolios over the entire range of the efficient frontier.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontier(p);

disp(pwgt)
```

0.8891	0.7215	0.5540	0.3865	0.2190	0.0515	0	0	0	1.0000
0.0369	0.1289	0.2209	0.3129	0.4049	0.4969	0.4049	0.2314	0.0579	0.0000
0.0404	0.0567	0.0730	0.0893	0.1056	0.1219	0.1320	0.1394	0.1468	0.0000
0.0336	0.0929	0.1521	0.2113	0.2705	0.3297	0.4630	0.6292	0.7953	0.0000

If you want only four portfolios, you can use `estimateFrontier` with `NumPorts` specified as 4.

```
pwgt = estimateFrontier(p, 4);
disp(pwgt)
```

0.8891	0.3865	0	0
0.0369	0.3129	0.4049	0
0.0404	0.0893	0.1320	0
0.0336	0.2113	0.4630	1.0000

Starting from the initial portfolio, `estimateFrontier` also returns purchases and sales to get from your initial portfolio to each efficient portfolio on the efficient frontier. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontier(p);

display(pwgt)
```

`pwgt = 4×10`

0.8891	0.7215	0.5540	0.3865	0.2190	0.0515	0	0	0	1.0000
0.0369	0.1289	0.2209	0.3129	0.4049	0.4969	0.4049	0.2314	0.0579	0.0000
0.0404	0.0567	0.0730	0.0893	0.1056	0.1219	0.1320	0.1394	0.1468	0.0000
0.0336	0.0929	0.1521	0.2113	0.2705	0.3297	0.4630	0.6292	0.7953	0.0000

```
display(pbuy)
```

```
pbuy = 4×10
```

```

    0.5891    0.4215    0.2540    0.0865         0         0         0         0
         0         0         0    0.0129    0.1049    0.1969    0.1049         0
         0         0         0         0         0         0         0         0
         0         0    0.0521    0.1113    0.1705    0.2297    0.3630    0.5292    0.6953    0.9

```

```
display(psell)
```

```
psell = 4×10
```

```

         0         0         0         0    0.0810    0.2485    0.3000    0.3000    0.3000    0.3
    0.2631    0.1711    0.0791         0         0         0         0    0.0686    0.2421    0.3
    0.1596    0.1433    0.1270    0.1107    0.0944    0.0781    0.0680    0.0606    0.0532    0.2
    0.0664    0.0071         0         0         0         0         0         0         0

```

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

`Portfolio` | `estimateFrontier` | `estimateFrontierLimits` | `estimatePortMoments` | `estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` | `estimatePortRisk` | `estimateFrontierByRisk` | `estimateMaxSharpeRatio` | `setSolver`

Related Examples

- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- [Getting Started with Portfolio Optimization \(4 min 13 sec\)](#)
- [MATLAB for Advanced Portfolio Construction and Stock Selection Models \(30 min 28 sec\)](#)

Obtaining Endpoints of the Efficient Frontier

Often when using a `Portfolio` object, you might be interested in the endpoint portfolios for the efficient frontier. Suppose that you want to determine the range of returns from minimum to maximum to refine a search for a portfolio with a specific target return. Use the `estimateFrontierLimits` function to obtain the endpoint portfolios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierLimits(p);
```

```
disp(pwgt)
```

```
    0.8891         0
    0.0369         0
    0.0404         0
    0.0336    1.0000
```

The `estimatePortMoments` function shows the range of risks and returns for efficient portfolios:

```
[prsk, pret] = estimatePortMoments(p, pwgt);
disp([prsk, pret])
```

```
    0.0769    0.0590
    0.3500    0.1800
```

Starting from an initial portfolio, `estimateFrontierLimits` also returns purchases and sales to get from the initial portfolio to the endpoint portfolios on the efficient frontier. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
```

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontierLimits(p);
```

```
display(pwgt)
```

```
pwgt = 4×2
```

```
    0.8891         0
```

```

0.0369      0
0.0404      0
0.0336      1.0000

```

```
display(pbuy)
```

```
pbuy = 4×2
```

```

0.5891      0
      0      0
      0      0
      0      0.9000

```

```
display(psell)
```

```
psell = 4×2
```

```

      0      0.3000
0.2631      0.3000
0.1596      0.2000
0.0664      0

```

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

`Portfolio` | `estimateFrontier` | `estimateFrontierLimits` | `estimatePortMoments` | `estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` | `estimatePortRisk` | `estimateFrontierByRisk` | `estimateMaxSharpeRatio` | `setSolver`

Related Examples

- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20

- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Obtaining Efficient Portfolios for Target Returns

This example shows how to obtain efficient portfolios that have targeted portfolio returns using the `estimateFrontierByReturn` function.

The `estimateFrontierByReturn` function accepts one or more target portfolio returns and obtains efficient portfolios with the specified returns. For example, assume that you have a universe of four assets where you want to obtain efficient portfolios with target portfolio returns of 6%, 9%, and 12%:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierByReturn(p, [0.06, 0.09, 0.12]);

display(pwgt)

pwgt = 4×3

    0.8772    0.5032    0.1293
    0.0434    0.2488    0.4541
    0.0416    0.0780    0.1143
    0.0378    0.1700    0.3022
```

Sometimes, you can request a return for which no efficient portfolio exists. Based on the previous code, suppose that you want a portfolio with a 5% return (which is the return of the first asset). A portfolio that is fully invested in the first asset, however, is inefficient. `estimateFrontierByReturn` warns if your target returns are outside the range of efficient portfolio returns and replaces it with the endpoint portfolio of the efficient frontier closest to your target return. The best way to avoid this situation is to bracket your target portfolio returns with `estimateFrontierLimits` and `estimatePortReturn` (see “Obtaining Endpoints of the Efficient Frontier” on page 4-102 and “Obtaining Portfolio Risks and Returns” on page 4-122).

```
pret = estimatePortReturn(p, p.estimateFrontierLimits);
display(pret)

pret = 2×1

    0.0590
    0.1800
```

This result indicates that efficient portfolios have returns that range between 5.9% and 18%.

If you have an initial portfolio, `estimateFrontierByReturn` also returns purchases and sales to get from your initial portfolio to the target portfolios on the efficient frontier. For example, given an initial portfolio in `pwgt0`, to obtain purchases and sales with target returns of 6%, 9%, and 12%:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontierByReturn(p, [0.06, 0.09, 0.12]);
```

```
display(pwgt)
```

```
pwgt = 4×3
```

0.8772	0.5032	0.1293
0.0434	0.2488	0.4541
0.0416	0.0780	0.1143
0.0378	0.1700	0.3022

```
display(pbuy)
```

```
pbuy = 4×3
```

0.5772	0.2032	0
0	0	0.1541
0	0	0
0	0.0700	0.2022

```
display(psell)
```

```
psell = 4×3
```

0	0	0.1707
0.2566	0.0512	0
0.1584	0.1220	0.0857
0.0622	0	0

If you do not have an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

[Portfolio](#) | [estimateFrontier](#) | [estimateFrontierLimits](#) | [estimatePortMoments](#) | [estimateFrontierByReturn](#) | [estimatePortReturn](#) | [estimateFrontierByRisk](#) | [estimatePortRisk](#) | [estimateFrontierByRisk](#) | [estimateMaxSharpeRatio](#) | [setSolver](#)

Related Examples

- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222

- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Obtaining Efficient Portfolios for Target Risks

This example shows how to obtain efficient portfolios that have targeted portfolio risks using the `estimateFrontierByRisk` function.

The `estimateFrontierByRisk` function accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks. Suppose that you have a universe of four assets where you want to obtain efficient portfolios with target portfolio risks of 12%, 14%, and 16%.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierByRisk(p, [0.12, 0.14, 0.16]);

display(pwgt)

pwgt = 4×3

    0.3984    0.2659    0.1416
    0.3064    0.3791    0.4474
    0.0882    0.1010    0.1131
    0.2071    0.2540    0.2979
```

Sometimes, you can request a risk for which no efficient portfolio exists. Based on the previous code, suppose that you want a portfolio with 7% risk (individual assets in this universe have risks ranging from 8% to 35%). It turns out that a portfolio with 7% risk cannot be formed with these four assets. `estimateFrontierByRisk` warns if your target risks are outside the range of efficient portfolio risks and replaces it with the endpoint of the efficient frontier closest to your target risk. The best way to avoid this situation is to bracket your target portfolio risks with `estimateFrontierLimits` and `estimatePortRisk` (see “Obtaining Endpoints of the Efficient Frontier” on page 4-102 and “Obtaining Portfolio Risks and Returns” on page 4-122).

```
prsk = estimatePortRisk(p, p.estimateFrontierLimits);
display(prsk)

prsk = 2×1

    0.0769
    0.3500
```

This result indicates that efficient portfolios have risks that range from 7.7% to 35%.

Starting with an initial portfolio, `estimateFrontierByRisk` also returns purchases and sales to get from your initial portfolio to the target portfolios on the efficient frontier. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales from the example with target risks of 12%, 14%, and 16%:

```

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontierByRisk(p, [0.12, 0.14, 0.16]);

```

```
display(pwgt)
```

```
pwgt = 4×3
```

```

    0.3984    0.2659    0.1416
    0.3064    0.3791    0.4474
    0.0882    0.1010    0.1131
    0.2071    0.2540    0.2979

```

```
display(pbuy)
```

```
pbuy = 4×3
```

```

    0.0984         0         0
    0.0064    0.0791    0.1474
         0         0         0
    0.1071    0.1540    0.1979

```

```
display(psell)
```

```
psell = 4×3
```

```

         0    0.0341    0.1584
         0         0         0
    0.1118    0.0990    0.0869
         0         0         0

```

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

Portfolio | estimateFrontier | estimateFrontierLimits | estimatePortMoments | estimateFrontierByReturn | estimatePortReturn | estimatePortRisk | estimateFrontierByRisk | estimateMaxSharpeRatio | setSolver

Related Examples

- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222

- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Efficient Portfolio That Maximizes Sharpe Ratio

Portfolios that maximize the Sharpe ratio are portfolios on the efficient frontier that satisfy several theoretical conditions in finance. Such portfolios are called tangency portfolios since the tangent line from the risk-free rate to the efficient frontier taps the efficient frontier at portfolios that maximize the Sharpe ratio.

The Sharpe ratio is defined as the ratio

$$\frac{\mu(x) - r_0}{\sqrt{\Sigma(x)}}$$

where $x \in R^n$ and r_0 is the risk-free rate (μ and Σ proxies for the portfolio return and risk). For more information, see “Portfolio Optimization Theory” on page 4-4.

To obtain efficient portfolios that maximizes the Sharpe ratio, the `estimateMaxSharpeRatio` function accepts a `Portfolio` object and obtains efficient portfolios that maximize the Sharpe Ratio. Suppose that you have a universe with four risky assets and a riskless asset and you want to obtain a portfolio that maximizes the Sharpe ratio, where, in this example, r_0 is the return for the riskless asset.

```
r0 = 0.03;
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio('RiskFreeRate', r0);
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateMaxSharpeRatio(p);

display(pwgt)

pwgt = 4×1

    0.4251
    0.2917
    0.0856
    0.1977
```

If you start with an initial portfolio, `estimateMaxSharpeRatio` also returns purchases and sales to get from your initial portfolio to the portfolio that maximizes the Sharpe ratio. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateMaxSharpeRatio(p);

display(pwgt)
```

```
pwgt = 4×1

    0.4251
    0.2917
    0.0856
    0.1977

display(pbuy)

pbuy = 4×1

    0.1251
         0
         0
    0.0977

display(psell)

psell = 4×1

         0
    0.0083
    0.1144
         0
```

If you do not specify an initial portfolio, the purchase and sale weights assume that you initial portfolio is 0.

See Also

`Portfolio` | `estimateFrontier` | `estimateFrontierLimits` | `estimatePortMoments` | `estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` | `estimatePortRisk` | `estimateFrontierByRisk` | `estimateMaxSharpeRatio` | `setSolver`

Related Examples

- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization

The default solver for mean-variance portfolio optimization is `lcprog`, which implements a linear complementarity programming (LCP) algorithm. Although `lcprog` works for most problems, you can adjust arguments to control the algorithm. Alternatively, the mean-variance portfolio optimization tools let you use any of the variations of `quadprog` from Optimization Toolbox™ software. Like Optimization Toolbox, which uses the `interior-point-convex` algorithm as the default algorithm for `quadprog`, the portfolio optimization tools also use the `interior-point-convex` algorithm as the default. For details about `quadprog` and quadratic programming algorithms and options, see “Quadratic Programming Algorithms”.

Using 'lcprog' and 'quadprog'

To modify either `lcprog` or to specify `quadprog` as your solver, use the `setSolver` function to set the hidden properties `solverType` and `solverOptions` that specify and control the solver. Because the solver properties are hidden, you cannot set these options using the `Portfolio` object. The default solver is `lcprog` so you do not need to use `setSolver` to specify this solver. To use `quadprog`, you can set the default `interior-point-convex` algorithm of `quadprog` using this code:

```
p = Portfolio;
p = setSolver(p, 'quadprog');
display(p.solverType)
display(p.solverOptions)

quadprog options:

Options used by current Algorithm ('interior-point-convex'):
(Other available algorithms: 'active-set', 'trust-region-reflective')

Set properties:
    Algorithm: 'interior-point-convex'
    Display: 'off'
OptimalityTolerance: 1.0000e-12

Default properties:
ConstraintTolerance: 1.0000e-08
    LinearSolver: 'auto'
    MaxIterations: 200
    StepTolerance: 1.0000e-12
```

You can switch back to `lcprog` with this code:

```
p = setSolver(p, 'lcprog');
display(p.solverType);
display(p.solverOptions)

lcprog
    MaxIter: []
    TieBreak: []
    TolPiv: 5.0000e-08
```

In both cases, `setSolver` sets up default options associated with either solver. If you want to specify additional options associated with a given solver, `setSolver` accepts these options with name-value

arguments in the function call. For example, if you intend to use `quadprog` and want to use the 'trust-region-reflective' algorithm, call `setSolver` with this code:

```
p = Portfolio;
p = setSolver(p, 'quadprog', 'Algorithm', 'trust-region-reflective');
display(p.solverOptions)

quadprog options:

Options used by current Algorithm ('trust-region-reflective'):
(Other available algorithms: 'active-set', 'interior-point-convex')

Set properties:
    Algorithm: 'trust-region-reflective'

Default properties:
    Display: 'final'
    FunctionTolerance: 'default dependent on problem'
    HessianMultiplyFcn: []
    MaxIterations: 'default dependent on problem'
    OptimalityTolerance: 'default dependent on problem'
    StepTolerance: 2.2204e-14
    SubproblemAlgorithm: 'cg'
    TypicalX: 'ones(numberOfVariables,1)'
```

In addition, if you want to specify any of the options for `quadprog` that you typically set through `optimoptions` from Optimization Toolbox, `setSolver` accepts an `optimoptions` object as the second argument. For example, you can start with the default options for `quadprog` set by `setSolver` and then change the algorithm to 'trust-region-reflective' with no displayed output:

```
p = Portfolio;
options = optimoptions('quadprog', 'Algorithm', 'trust-region-reflective', 'Display', 'off');
p = setSolver(p, 'quadprog', options);
display(p.solverOptions.Algorithm)
display(p.solverOptions.Display)

trust-region-reflective
off
```

Using the Mixed Integer Nonlinear Programming (MINLP) Solver

The mixed integer nonlinear programming (MINLP) solver, configured using `setSolverMINLP`, enables you to specify associated solver options for portfolio optimization for a `Portfolio` object. The MINLP solver is used when any one, or any combination of 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active. In this case, you formulate the portfolio problem by adding `NumAssets` binary variables, where 0 indicates not invested, and 1 is invested. For more information on using 'Conditional' `BoundType`, see `setBounds`. For more information on specifying `MinNumAssets` and `MaxNumAssets`, see `setMinMaxNumAssets`.

When using the estimate functions with a `Portfolio` object where 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active, the mixed integer nonlinear programming (MINLP) solver is automatically used.

Solver Guidelines for Portfolio Objects

The following table provides guidelines for using `setSolver` and `setSolverMINLP`.

Portfolio Problem	Portfolio Function	Type of Optimization Problem	Main Solver	Helper Solver
Portfolio without tracking error constraints	<code>estimateFrontierByRisk</code>	Optimizing a portfolio for a certain risk level introduces a nonlinear constraint. Therefore, this problem has a linear objective with linear and nonlinear constraints.	'fmincon' using <code>setSolver</code>	For 'min': quadratic objective, 'quadprog' or 'lcprog' using <code>setSolver</code> For 'max': linear objective, 'linprog' or 'lcprog' using <code>setSolver</code>
Portfolio without tracking error constraints	<code>estimateFrontierByReturn</code>	Quadratic objective with linear constraints	'quadprog' or 'lcprog' using <code>setSolver</code>	For 'min': quadratic objective, 'quadprog' or 'lcprog' using <code>setSolver</code> For 'max': linear objective, 'linprog' or 'lcprog' using <code>setSolver</code>
Portfolio without tracking error constraints	<code>estimateFrontierLimits</code>	Quadratic or linear objective with linear constraints	For 'min': quadratic objective, 'quadprog' or 'lcprog' using <code>setSolver</code> For 'max': linear objective, 'linprog' or 'lcprog' using <code>setSolver</code>	Not applicable
Portfolio without tracking error constraints	<code>estimateMaxSharpeRatio</code>	Quadratic objective with linear constraints	'quadprog' using <code>setSolver</code>	Because <code>estimateMaxSharpeRatio</code> internally calls <code>estimateFrontierLimits</code> , all solvers needed by <code>estimateFrontierLimits</code> are the helper solvers

Portfolio Problem	Portfolio Function	Type of Optimization Problem	Main Solver	Helper Solver
Portfolio with tracking error constraints	<code>estimateFrontierByRisk</code>	Linear objective with linear and nonlinear constraints	'fmincon' using <code>setSolver</code>	Not applicable
Portfolio with tracking error constraints	<code>estimateFrontierByReturn</code>	Linear objective with linear and nonlinear constraints	'fmincon' using <code>setSolver</code>	Not applicable
Portfolio with tracking error constraints	<code>estimateFrontierLimits</code>	Quadratic (minimum risk problem) or linear (maximum return problem) objective with linear and nonlinear constraints	'fmincon' using <code>setSolver</code>	Not applicable
Portfolio with tracking error constraints	<code>estimateMaxSharpeRatio</code>	Quadratic objective with linear and nonlinear constraints	'fmincon' using <code>setSolver</code>	Not applicable
Portfolio with active 'Conditional' <code>BoundType</code> , <code>MinNumAssets</code> , <code>MaxNumAssets</code> , or conditional budget constraints	<code>estimateFrontierByRisk</code>	The problem is formulated by introducing <code>NumAssets</code> binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see <code>setSolverMINLP</code> .	Mixed integer nonlinear programming solver (MINLP) using <code>setSolverMINLP</code>	'quadprog' or 'fmincon' are used when the <code>estimate</code> functions reduce the problem into NLP. These two solvers can be configured through <code>setSolver</code> .

Portfolio Problem	Portfolio Function	Type of Optimization Problem	Main Solver	Helper Solver
Portfolio with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierByReturn	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'quadprog' or 'fmincon' are used when the estimate functions reduce the problem into NLP. These two solvers can be configured through setSolver.
Portfolio with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierLimits	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'quadprog' or 'fmincon' are used when the estimate functions reduce the problem into NLP. These two solvers can be configured through setSolver.

Portfolio Problem	Portfolio Function	Type of Optimization Problem	Main Solver	Helper Solver
Portfolio with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateMaxSharpeRatio	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'quadprog' or 'fmincon' are used when the estimate functions reduce the problem into NLP. These two solvers can be configured through setSolver.

Solver Guidelines for Custom Objective Problems Using Portfolio Objects

The following table provides guidelines for using setSolver and setSolverMINLP when using estimateCustomObjectivePortfolio.

Portfolio Problem	Portfolio Function	Type of Optimization Problem	Main Solver	Helper Solver
Portfolio with custom linear objective without tracking error constraints	estimateCustomObjectivePortfolio	Linear objective with linear constraints	'linprog' using setSolver	Not applicable
Portfolio with custom quadratic objective without tracking error constraints	estimateCustomObjectivePortfolio	Quadratic objective with linear constraints	'quadprog' using setSolver	Not applicable
Portfolio with custom nonlinear objective without tracking error constraints	estimateCustomObjectivePortfolio	Nonlinear objective with linear constraints	'fmincon' using setSolver	Not applicable
Portfolio with custom linear objective with tracking error constraints	estimateCustomObjectivePortfolio	Linear objective with linear and nonlinear constraints	'fmincon' using setSolver	Not applicable

Portfolio Problem	Portfolio Function	Type of Optimization Problem	Main Solver	Helper Solver
Portfolio with custom quadratic objective with tracking error constraints	<code>estimateCustomObjectivePortfolio</code>	Quadratic objective with linear and nonlinear constraints	'fmincon' using <code>setSolver</code>	Not applicable
Portfolio with custom nonlinear objective with tracking error constraints	<code>estimateCustomObjectivePortfolio</code>	Nonlinear objective with linear and nonlinear constraints	'fmincon' using <code>setSolver</code>	Not applicable
Portfolio with active 'Conditional' <code>BoundType</code> , <code>MinNumAssets</code> , <code>MaxNumAssets</code> , or conditional budget constraints	<code>estimateCustomObjectivePortfolio</code>	The problem is formulated by introducing <code>NumAssets</code> binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see <code>setSolverMINLP</code> .	Mixed integer nonlinear programming solver (MINLP) using <code>setSolverMINLP</code>	'intlinprog' is used when the problem reduces to a mixed-integer linear problem. This solver can be configured using <code>setSolverMINLP</code> and its name-value argument <code>IntMainSolverOptions</code> .

See Also

`Portfolio` | `estimatePortReturn` | `estimatePortMoments` | `plotFrontier` | `setConditionalBudget`

Related Examples

- “Plotting the Efficient Frontier for a Portfolio Object” on page 4-125
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231

- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Mixed-Integer Mean-Variance Portfolio Optimization Problem” on page 4-402
- “Adding Constraints to Satisfy UCITS Directive” on page 4-457

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Choose MINLP Solvers for Portfolio Problems” on page 4-140
- “Portfolio Object Workflow” on page 4-18
- “Role of Convexity in Portfolio Problems” on page 4-157

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Estimate Efficient Frontiers for Portfolio Object

Whereas “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98 focused on estimation of efficient portfolios, this section focuses on the estimation of efficient frontiers. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Obtaining Portfolio Risks and Returns

This example shows how to calculate the risk and returns for a portfolio. Given any portfolio and, in particular, efficient portfolios, the functions `estimatePortReturn`, `estimatePortRisk`, and `estimatePortMoments` provide estimates for the return (or return proxy), risk (or the risk proxy), and, in the case of mean-variance portfolio optimization, the moments of expected portfolio returns. Each function has the same input syntax but with different combinations of outputs. Suppose that you have this following portfolio optimization problem that gave you a collection of portfolios along the efficient frontier in `pwgt`:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = Portfolio('AssetMean', m, 'AssetCovar', C, 'InitPort', pwgt0);
p = setDefaultConstraints(p);
pwgt = estimateFrontier(p);
```

Given `pwgt0` and `pwgt`, use the portfolio risk and return estimation functions to obtain risks and returns for your initial portfolio and the portfolios on the efficient frontier.

```
[prsk0, pret0] = estimatePortMoments(p, pwgt0);
[prsk, pret] = estimatePortMoments(p, pwgt);
display(prsk0)
```

```
prsk0 =
0.1103
```

```
display(pret0)
```

```
pret0 =
0.0870
```

```
display(prsk)
```

```
prsk = 10×1
```

```
0.0769
0.0831
0.0994
0.1217
0.1474
0.1750
0.2068
0.2487
0.2968
```

```

0.3500

display(pret)

pret = 10×1

0.0590
0.0725
0.0859
0.0994
0.1128
0.1262
0.1397
0.1531
0.1666
0.1800

```

The returns and risks are at the periodicity of the moments of asset returns so that, if you have values for `AssetMean` and `AssetCovar` in terms of monthly returns, the estimates for portfolio risk and return are in terms of monthly returns as well. In addition, the estimate for portfolio risk in the mean-variance case is the standard deviation of portfolio returns, not the variance of portfolio returns.

See Also

`Portfolio` | `estimatePortReturn` | `estimatePortMoments` | `plotFrontier`

Related Examples

- “Plotting the Efficient Frontier for a Portfolio Object” on page 4-125
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- [Getting Started with Portfolio Optimization \(4 min 13 sec\)](#)
- [MATLAB for Advanced Portfolio Construction and Stock Selection Models \(30 min 28 sec\)](#)

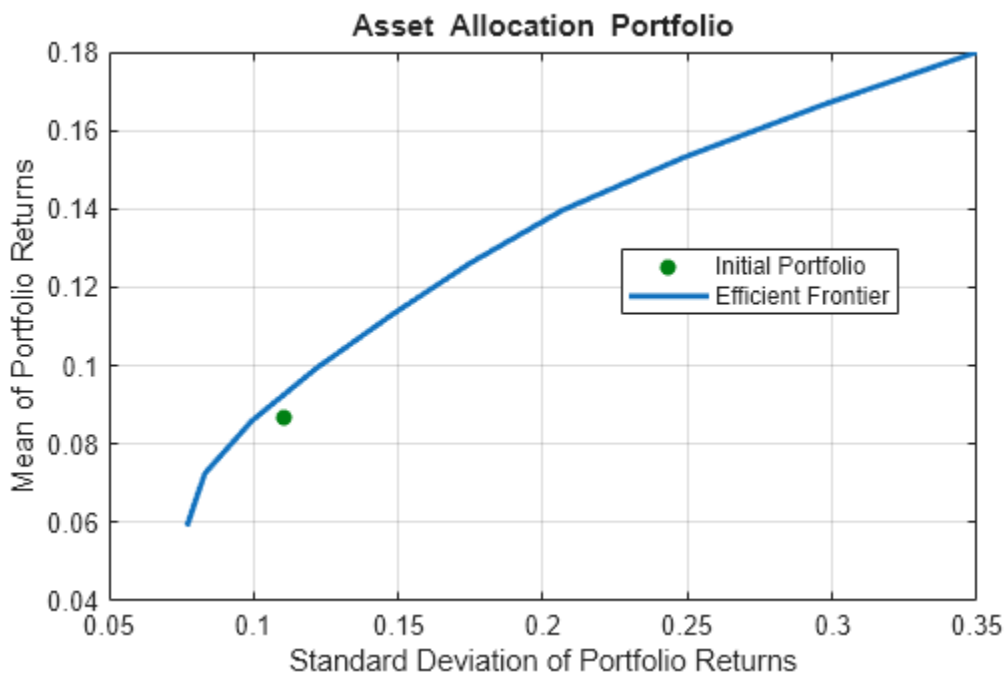
Plotting the Efficient Frontier for a Portfolio Object

This example shows how to use the `plotFrontier` function to create a plot of the efficient frontier for a given portfolio optimization problem. The `plotFrontier` function accepts several types of inputs and generates a plot with an optional possibility to output the estimates for portfolio risks and returns along the efficient frontier.

`plotFrontier` has four different ways that it can be used. In addition to a plot of the efficient frontier, if you have an initial portfolio in the `InitPort` property, `plotFrontier` also displays the return versus risk of the initial portfolio on the same plot. If you have a well-posed portfolio optimization problem set up in a `Portfolio` object and you use `plotFrontier`, you get a plot of the efficient frontier with the default number of portfolios on the frontier (the default number is 10 and is maintained in the hidden property `defaultNumPorts`). The following code illustrates a typical use of `plotFrontier` to create a new plot.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

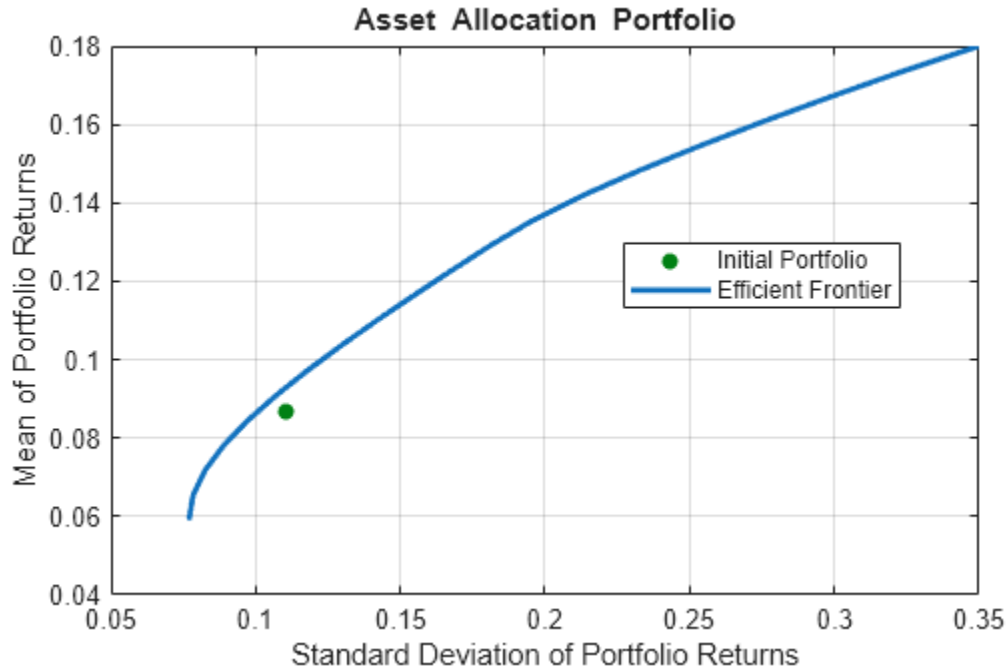
p = Portfolio('Name', 'Asset Allocation Portfolio', 'InitPort', pwgt0);
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
plotFrontier(p)
```



The `Name` property appears as the title of the efficient frontier plot if you set it in the `Portfolio` object. Without an explicit name, the title on the plot would be "Efficient Frontier." If you want to obtain a specific number of portfolios along the efficient frontier, use `plotFrontier` with the

number of portfolios that you want. Suppose that you have the `Portfolio` object from the previous code and you want to plot 20 portfolios along the efficient frontier and to obtain 20 risk and return values for each portfolio.

```
[prsk, pret] = plotFrontier(p, 20);
```



```
display([pret, prsk])
```

0.0590	0.0769
0.0654	0.0784
0.0718	0.0825
0.0781	0.0890
0.0845	0.0973
0.0909	0.1071
0.0972	0.1179
0.1036	0.1296
0.1100	0.1418
0.1163	0.1545
0.1227	0.1676
0.1291	0.1810
0.1354	0.1955
0.1418	0.2128
0.1482	0.2323
0.1545	0.2535
0.1609	0.2760
0.1673	0.2995
0.1736	0.3239
0.1800	0.3500

Plotting Existing Efficient Portfolios

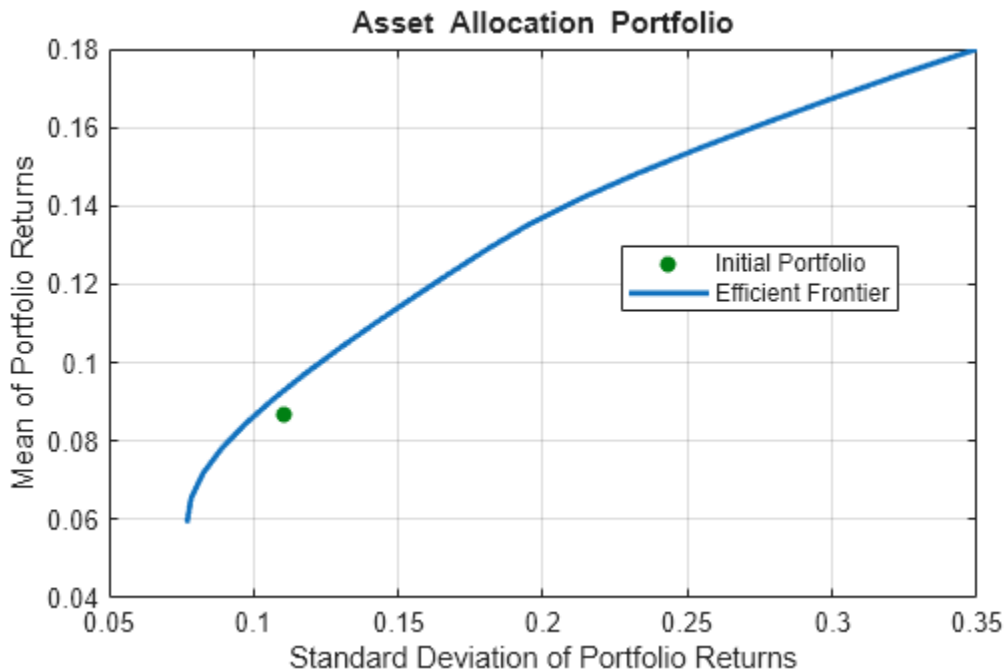
If you already have efficient portfolios from any of the "estimateFrontier" functions (see "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98), pass them into `plotFrontier` directly to plot the efficient frontier.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = Portfolio('Name', 'Asset Allocation Portfolio', 'InitPort', pwgt0);
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontier(p, 20);
plotFrontier(p, pwgt)

```



Plotting Existing Efficient Portfolio Risks and Returns

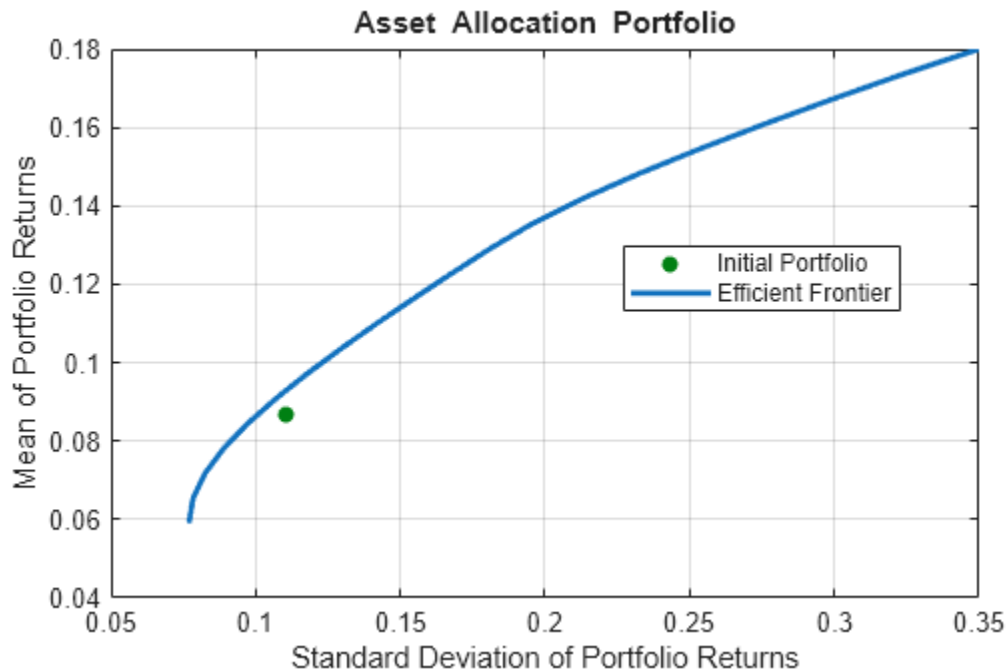
If you already have efficient portfolio risks and returns, you can use the interface to `plotFrontier` to pass them into `plotFrontier` to obtain a plot of the efficient frontier.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = Portfolio('Name', 'Asset Allocation Portfolio', 'InitPort', pwgt0);
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
[prsk, pret] = estimatePortMoments(p, p.estimateFrontier(20));
plotFrontier(p, prsk, pret)

```



See Also

Portfolio | estimatePortReturn | estimatePortMoments | plotFrontier

Related Examples

- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- [Getting Started with Portfolio Optimization \(4 min 13 sec\)](#)
- [MATLAB for Advanced Portfolio Construction and Stock Selection Models \(30 min 28 sec\)](#)

Postprocessing Results to Set Up Tradable Portfolios

This example shows how to use your results for efficient portfolios or estimates for expected portfolio risks and returns to set up trades to move toward an efficient portfolio. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Suppose that you set up a portfolio optimization problem and obtained portfolios on the efficient frontier. Use the `dataset` object to form a blotter that lists your portfolios with the names for each asset. For example, suppose that you want to obtain five portfolios along the efficient frontier. You can set up a blotter with weights multiplied by 100 to view the allocations for each portfolio:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = Portfolio('InitPort', pwgt0);
p = setAssetList(p, 'Bonds', 'Large-Cap Equities', 'Small-Cap Equities', 'Emerging Equities');
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end

Blotter = dataset([100*pwgt], pnames, 'obsnames', p.AssetList);
display(Blotter)
```

Blotter =

	Port1	Port2	Port3	Port4	Port5
Bonds	88.906	51.216	13.525	0	0
Large-Cap Equities	3.6875	24.387	45.086	27.479	0
Small-Cap Equities	4.0425	7.7088	11.375	13.759	0
Emerging Equities	3.364	16.689	30.014	58.762	100

This result indicates that you would invest primarily in bonds at the minimum-risk/minimum-return end of the efficient frontier (Port1), and that you would invest completely in emerging equity at the maximum-risk/maximum-return end of the efficient frontier (Port5). You can also select a particular efficient portfolio, for example, suppose that you want a portfolio with 15% risk and you add purchase and sale weights outputs obtained from the “estimateFrontier” functions to set up a trade blotter:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = Portfolio('InitPort', pwgt0);
p = setAssetList(p, 'Bonds', 'Large-Cap Equities', 'Small-Cap Equities', 'Emerging Equities');
p = setAssetMoments(p, m, C);
```

```

p = setDefaultConstraints(p);
[pwgt, pbuy, psell] = estimateFrontierByRisk(p, 0.15);

Blotter = dataset([100*[pwgt0, pwgt, pbuy, psell]], ...
    {'Initial','Weight', 'Purchases','Sales'}, 'obsnames', p.AssetList);

display(Blotter)

Blotter =

```

	Initial	Weight	Purchases	Sales
Bonds	30	20.299	0	9.7007
Large-Cap Equities	30	41.366	11.366	0
Small-Cap Equities	20	10.716	0	9.2838
Emerging Equities	10	27.619	17.619	0

If you have prices for each asset (in this example, they can be ETFs), add them to your blotter and then use the tools of the `dataset` object to obtain shares and shares to be traded. For an example, see “Asset Allocation Case Study” on page 4-180.

See Also

Portfolio | `estimateAssetMoments` | `checkFeasibility`

Related Examples

- “Troubleshooting Portfolio Optimization Results” on page 4-145
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

When to Use Portfolio Objects Over Optimization Toolbox

While you can use Optimization Toolbox to solve portfolio optimization problems, Financial Toolbox has the `Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` objects that you can use as well. Which tool you use depends on the problem case:

- **Always** use a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object when the problem can most easily be written and implemented using one of these objects. This case includes problems that can be solved only with the internal solvers of the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. The Optimization Toolbox solvers cannot directly handle these problems. For details, see “Always Use `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` Object” on page 4-134.
- **Prefer** to use a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object when the problem can be modeled and implemented using both the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object and the Optimization Toolbox problem-based framework. The advantage of using a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object instead of the Optimization Toolbox is that the internal tools of these objects simplify the analysis. For details, see “Preferred Use of `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` Object” on page 4-135.
- **Opt** to use Optimization Toolbox for problems that cannot be solved with the internal solvers of the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. Some of the problems that the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object cannot solve can be addressed using the Optimization Toolbox problem-based framework. Problems that cannot be directly solved with either framework require either some restructuring of the model or an implementation of a specialized solver. For details, see “Use Optimization Toolbox” on page 4-136.

The following table summarizes the objective functions, constraints, and variables that apply in each case for solving a portfolio problem.

Case for Solving Portfolio Problem	Objective Function	Constraints	Integer (Binary) Variables
"Always" case with Financial Toolbox	- Return — Gross portfolio returns or net portfolio returns - Risk — Variance, CVaR, or MAD - Sharpe ratio (only for mean-variance problems using <code>Portfolio</code> object) - Continuous convex functions (only using the <code>Portfolio</code> object)	- Return — Gross portfolio returns or net portfolio returns - Risk — Variance, CVaR, or MAD (risk constraints are only supported when the objective is the return) - Linear equalities - Linear inequalities - Tracking error (only for mean-variance problems using <code>Portfolio</code> object). Tracking error is supported with custom objectives using <code>estimateCustomObjectivePortfolio</code>. Tracking error is only supported when the objective is the return, variance, or Sharpe ratio. - Turnover	- Bounds on the number of assets - Conditional budget constraint (an upper bound on the sum of all assets that go above a specified threshold) - Conditional (semicontinuous) bounds (for example, if asset i is selected, then $x_i \geq lb_i$, otherwise $x_i = 0$)

Case for Solving Portfolio Problem	Objective Function	Constraints	Integer (Binary) Variables
"Preferred" case with Financial Toolbox	- Return — Gross portfolio returns or net portfolio returns - Risk — Variance, CVaR, or MAD - Sharpe ratio (only for mean-variance problems using Portfolio object) - Continuous functions (only using the Portfolio object)	- Return — Gross portfolio returns or net portfolio returns - Risk — Variance, CVaR, or MAD (risk constraints are only supported when the objective is the return) - Linear equalities - Linear inequalities - Tracking error (only for mean-variance problems using Portfolio object). Tracking error is supported with custom objectives using <code>estimateCustomObjectivePortfolio</code>. Tracking error is only supported when the objective is the return, variance, or Sharpe ratio.	None
Optimization Toolbox	Any other nonlinear function not mentioned in "Always Use Portfolio, PortfolioCVaR, or PortfolioMAD Object" on page 4-134 and "Preferred Use of Portfolio, PortfolioCVaR, or PortfolioMAD Object" on page 4-135	Any other nonlinear function not mentioned in "Always Use Portfolio, PortfolioCVaR, or PortfolioMAD Object" on page 4-134 and "Preferred Use of Portfolio, PortfolioCVaR, or PortfolioMAD Object" on page 4-135	None

Always Use Portfolio, PortfolioCVaR, or PortfolioMAD Object

The two general cases for always using the Portfolio, PortfolioCVaR, or PortfolioMAD object are:

- Problems with *both* supported nonlinear constraints *and* conditional bounds or bounds in the number of assets.

These problems include:

- Minimum risk problems subject to constraints for return, linear equality, linear inequality, turnover, and tracking error where the supported risk measures are variance, conditional value-at-risk (CVaR), and mean-absolute-deviation (MAD)
- Maximum return problems subject to constraints for linear equality, linear inequality, turnover, risk, and tracking error where the supported risk measures are variance, CVaR, and MAD

Tracking error is supported only for mean-variance problems using the `Portfolio` object. For more information on the supported constraints for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object, see Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9.

- Custom objective minimization or maximization. The custom objective function (using `estimateCustomObjectivePortfolio`) must be continuous and convex subject to constraints for return, risk, linear equality, linear inequality, tracking error, and turnover.

Tracking error is supported using the `Portfolio` object only when the objective is the return, variance or Sharpe ratio. Sharpe ratio is supported only using the `Portfolio` object.

For more information on the supported constraints and nonlinear risk functions for `Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` objects, see Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9 and “Portfolio Optimization Theory” on page 4-4. The integer (binary) variables can come from either of the following sources: bounds on the number of assets that can be selected in the portfolio, setting conditional (semicontinuous) bounds on the assets, or adding a conditional budget constraint. For example, if asset i is selected, then $x_i \geq lb_i$, otherwise $x_i = 0$. These problems cannot be solved using the Optimization Toolbox solvers. However, you can implement your own mixed-integer solver. For more information, see “Mixed-Integer Quadratic Programming Portfolio Optimization: Problem-Based”.

- Problems with turnover constraints and sell or buy costs

Although the continuous version of these problems can be solved by the Optimization Toolbox solvers, the variable space must be manipulated to rewrite the nonsmooth constraints into equivalent smooth constraints. Given that rewriting the problem requires optimization knowledge, it is recommended to use the `Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` objects instead.

Preferred Use of Portfolio, PortfolioCVaR, or PortfolioMAD Object

The general case for preferred use of the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object is:

- Continuous problems with minimum risk, maximum return, and maximum Sharpe ratio that are subject to linear equality, linear inequality, turnover, and tracking error constraints.

Sharpe ratio is supported only for mean-variance problems using the `Portfolio` object. For more information on the supported constraints for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object, see Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9.

- Custom objective minimization or maximization. The custom objective function (using `estimateCustomObjectivePortfolio`) must be continuous and convex subject to constraints for return, risk, linear equality, linear inequality, tracking error, and turnover.

Tracking error is supported using the `Portfolio` object only when the objective is the return, variance or Sharpe ratio. Sharpe ratio is supported only using the `Portfolio` object.

The supported risk measures are variance, CVaR, and MAD. For more information on the supported constraints for these risk measures, see Supported Constraints for Portfolio Optimization Using

Portfolio Objects on page 4-9, Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8, and Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8. For all other risk measures and constraints and if tracking error is in the objective, use the Optimization Toolbox.

The advantage of the `Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` object framework over the problem-based framework for the type of problems in the "preferred" case is that common portfolio optimization workflows are leveraged. For example, the `Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` object framework supports the following workflows:

- Estimating and plotting the efficient frontier
- Exchanging the return and risk proxies from the objective function to a constraint
- Solving the maximum Sharpe ratio problem
- Adding bounds on the number of assets selected
- Adding semicontinuous bounds
- Simplifying the use of turnover constraints and sell or buy costs

Use Optimization Toolbox

The two general cases to use Optimization Toolbox is:

- Problems that have nonlinear constraints other than the constraints for risk or tracking error

See Also

`Portfolio` | `PortfolioCVaR` | `PortfolioMAD`

Related Examples

- "Creating the Portfolio Object" on page 4-25
- "Working with Portfolio Constraints Using Defaults" on page 4-58
- "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
- "Estimate Efficient Frontiers for Portfolio Object" on page 4-122
- "Asset Allocation Case Study" on page 4-180
- "Portfolio Optimization Examples Using Financial Toolbox" on page 4-161
- "Portfolio Optimization with Semicontinuous and Cardinality Constraints" on page 4-190
- "Black-Litterman Portfolio Optimization Using Financial Toolbox" on page 4-222
- "Portfolio Optimization Using Factor Models" on page 4-231
- "Portfolio Optimization Using Social Performance Measure" on page 4-264
- "Diversify Portfolios Using Custom Objective" on page 4-331

More About

- "Portfolio Object" on page 4-20
- "Portfolio Optimization Theory" on page 4-4
- "Choose MINLP Solvers for Portfolio Problems" on page 4-140

- “Portfolio Object Workflow” on page 4-18
- “Problem-Based Optimization Workflow”

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Comparison of Methods for Covariance Estimation

Mean-variance portfolio optimization is a common technique in portfolio allocation. It usually requires an estimate of the covariance matrix to compute the portfolio weights. Many methods can be used for covariance estimation. The traditional covariance estimate is computed using `cov`. However, in practice the standard covariance estimate is noisy. Since the optimal solution of the portfolio weights is very sensitive to the mean and covariance estimates, noise in the covariance matrix increases the portfolio weights estimation error leading to large turnover and transaction costs. Here are some methods that mitigate the estimation error:

- `robustcov` computes a covariance estimate from a data sample. This function reduces the noise in the estimator by identifying the outliers in the sample and removing them from the final covariance estimate.
- `covarianceShrinkage` computes a covariance estimate from a data sample. This function reduces the noise in the estimator generated from using a finite sample. Shrinkage estimators are used to reduce the mean squared error (MSE) of an estimate. MSE consists of two parts, the bias and the noise. Shrinkage methods reduce the noise in the estimator by pulling the standard covariance estimate towards a target matrix. Thus, the shrinkage estimator results in a biased estimate, but with a lower variance than that of the standard covariance estimate. The method implemented in `covarianceShrinkage` considers a multiple of the identity as the target matrix.

In covariance shrinkage, the intensity parameter determines how much the standard covariance estimate is pulled towards the target matrix. Because computing the optimal intensity parameter depends on the sample size, `covarianceDenoising` runs slower when the sample size significantly increases.

- `covarianceDenoising` computes a covariance estimate from a data sample or from an initial covariance estimate (usually the standard covariance estimate from `cov`). The `covarianceDenoising` function reduces the noise in the estimator generated from using a finite sample. Unlike `covarianceShrinkage`, the `covarianceDenoising` function differentiates between the noise and the signal in the data to pull only the eigenvalues associated with noise towards a target value. This technique results in an estimate that decreases noise while increasing the signal.

The computation of the `covarianceDenoising` estimate depends on the number of assets (variables). If the number of assets is too small, there is not enough information to identify between the noisy and meaningful factors. On the other hand, if the number of assets is too large, the underlying optimization problem used to identify the noisy eigenvalues grows, and it takes longer to compute the estimate.

The following table summarizes the information needed to compute the different covariance estimates.

Function	Number of Assets	Sample Size	Sample
<code>cov</code> (MATLAB)	Any	Any	Required
<code>robustcov</code> (Statistics and Machine Learning Toolbox™)	Any	$\geq 2 \times \text{numAssets}$	Required
<code>covarianceShrinkage</code> (Financial Toolbox)	Any (≥ 3 recommended)	Any ($\leq 25,000$ recommended)	Required

Function	Number of Assets	Sample Size	Sample
<code>covarianceDenoising</code> (Financial Toolbox)	Any (≥ 10 recommended)	Any	Not required

Another useful tool for covariance estimation is `nearcorr`. Use `nearcorr` to compute the nearest correlation matrix by minimizing the Frobenius distance to an initial estimate. This is particularly useful to get a positive definite covariance matrix when the number of observations is less than the number of assets.

See Also

`Portfolio` | `covarianceShrinkage` | `covarianceDenoising` | `nearcorr`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Compare Performance of Covariance Denoising with Factor Modeling Using Backtesting” on page 4-394
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

Choose MINLP Solvers for Portfolio Problems

The following table lists the type of Mixed Integer Nonlinear Programming (MINLP) solvers that you can select to find the solution to different portfolio problems using a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. You can set the solvers using `setSolverMINLP`, where the first input argument, `solverTypeMINLP`, is one of the solvers listed in the table that follows.

		Available Solvers (Use <code>setSolverMINLP</code> to specify the solver)		
Portfolio Problem	Use Function:	Portfolio Object	PortfolioCVaR Object	PortfolioMAD Object
Estimate efficient frontier	<code>estimateFrontier(p)</code> (See <code>estimateFrontier</code>)	- ExtendedCP - OuterApproximation (default) Supports extended formulation using the name-value argument "ExtendedFormulation" on page 15-0 . - TrustRegionCP	- ExtendedCP - OuterApproximation - TrustRegionCP (default)	- ExtendedCP - OuterApproximation - TrustRegionCP (default)
Minimize risk without return constraint	<code>estimateFrontierLimits(p, 'min')</code> (See <code>estimateFrontierLimits</code>)	- ExtendedCP - OuterApproximation (default) Supports extended formulation using the name-value argument "ExtendedFormulation" on page 15-0 . - TrustRegionCP	- ExtendedCP - OuterApproximation - TrustRegionCP (default)	- ExtendedCP - OuterApproximation - TrustRegionCP (default)

		Available Solvers (Use setSolverMINLP to specify the solver)		
Portfolio Problem	Use Function:	Portfolio Object	PortfolioCVaR Object	PortfolioMAD Object
Maximize return without risk constraint or tracking error constraint	estimateFrontierLimits(p, 'max') (See estimateFrontierLimits)	intlinprog Use IntMainSolverOptions name-value argument to control intlinprog options.	intlinprog Use IntMainSolverOptions name-value argument to control intlinprog options.	intlinprog Use IntMainSolverOptions name-value argument to control intlinprog options.
Maximize return with risk constraint and/or tracking error constraint	estimateFrontierLimits(p, 'max') (See estimateFrontierLimits)	- ExtendedCP - OuterApproximation (default) Supports extended formulation using the name-value argument "ExtendedFormulation" on page 15-0 . - TrustRegionCP	- ExtendedCP - OuterApproximation - TrustRegionCP (default)	- ExtendedCP - OuterApproximation - TrustRegionCP (default)
Minimize risk subject to a return constraint	estimateFrontierByReturn(p, returnTarget) (See estimateFrontierByReturn)	- ExtendedCP - OuterApproximation (default) Supports extended formulation using the name-value argument "ExtendedFormulation" on page 15-0 . - TrustRegionCP	- ExtendedCP - OuterApproximation - TrustRegionCP (default)	- ExtendedCP - OuterApproximation - TrustRegionCP (default)

		Available Solvers (Use setSolverMINLP to specify the solver)		
Portfolio Problem	Use Function:	Portfolio Object	PortfolioCVaR Object	PortfolioMAD Object
Maximize return subject to a risk constraint	estimateFrontierByRisk(p, riskTarget) (See estimateFrontierByRisk)	- ExtendedCP - OuterApproximation (default) Supports extended formulation using the name-value argument "ExtendedFormulation" on page 15-0 . - TrustRegionCP	- ExtendedCP - OuterApproximation - TrustRegionCP (default)	- ExtendedCP - OuterApproximation - TrustRegionCP (default)
Maximize Sharpe ratio	estimateMaxSharpeRatio(p) (See estimateMaxSharpeRatio)	- ExtendedCP - OuterApproximation (default) Supports extended formulation using the name-value argument "ExtendedFormulation" on page 15-0 . - TrustRegionCP	- ExtendedCP - OuterApproximation - TrustRegionCP (default)	- ExtendedCP - OuterApproximation - TrustRegionCP (default)
Optimize custom linear objective without risk or tracking error constraint	estimateCustomObjectivePortfolio(p, linFun) (See estimateCustomObjectivePortfolio)	intlinprog Use IntMainSolverOptions name-value argument to control intlinprog options.	Not supported	Not supported

		Available Solvers (Use <code>setSolverMINLP</code> to specify the solver)		
Portfolio Problem	Use Function:	Portfolio Object	PortfolioCVaR Object	PortfolioMAD Object
Optimize custom linear objective with risk or/or tracking error constraint	<code>estimateCustomObjectivePortfolio(p,linFun)</code> (See <code>estimateCustomObjectivePortfolio</code>)	- ExtendedCP - OuterApproximation (default) Supports extended formulation using the name-value argument "ExtendedFormulation" on page 15-0 . - TrustRegionCP	Not supported	Not supported
Optimize custom quadratic objective	<code>estimateCustomObjectivePortfolio(p,quadFun)</code> (See <code>estimateCustomObjectivePortfolio</code>)	- ExtendedCP - OuterApproximation (default) Supports extended formulation using the name-value argument "ExtendedFormulation" on page 15-0 . - TrustRegionCP	Not supported	Not supported
Optimize custom nonlinear nonquadratic objective	<code>estimateCustomObjectivePortfolio(p,NLFun)</code> (See <code>estimateCustomObjectivePortfolio</code>)	- ExtendedCP - OuterApproximation (default) - TrustRegionCP	Not supported	Not supported

See Also

Portfolio | PortfolioCVaR | PortfolioMAD

Related Examples

- “Mixed-Integer CVaR Portfolio Optimization Problem” on page 5-141
- “Mixed-Integer MAD Portfolio Optimization Problem” on page 6-89
- “Mixed-Integer Mean-Variance Portfolio Optimization Problem” on page 4-402

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9
- Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8
- Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8
- “When to Use Portfolio Objects Over Optimization Toolbox” on page 4-132

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

Troubleshooting Portfolio Optimization Results

Portfolio Object Destroyed When Modifying

If a Portfolio object is destroyed when modifying, remember to pass an existing object into the Portfolio object if you want to modify it, otherwise it creates a new object. See "Creating the Portfolio Object" on page 4-25 for details.

Optimization Fails with "Bad Pivot" Message

If the optimization fails with a "bad pivot" message from `lcprog`, try a larger value for `tolpiv` which is a tolerance for pivot selection in the `lcprog` algorithm (try `1.0e-7`, for example) or try the `interior-point-convex` version of `quadprog`. For details, see "Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization" on page 4-114, the help header for `lcprog`, and `quadprog`.

Speed of Optimization

Although it is difficult to characterize when one algorithm is faster than the other, the default solver, `lcprog` is faster for smaller problems and the `quadprog` solver is faster for larger problems. If one solver seems to take too much time, try the other solver. To change solvers, use `setSolver`.

Matrix Incompatibility and "Non-Conformable" Errors

If you get matrix incompatibility or "non-conformable" errors, the representation of data in the tools follows a specific set of basic rules described in "Conventions for Representation of Data" on page 4-23.

Missing Data Estimation Fails

If asset return data has missing or NaN values, the `estimateAssetMoments` function with the 'missingdata' flag set to true may fail with either too many iterations or a singular covariance. To correct this problem, consider this:

- If you have asset return data with no missing or NaN values, you can compute a covariance matrix that may be singular without difficulties. If you have missing or NaN values in your data, the supported missing data feature requires that your covariance matrix must be positive-definite, that is, nonsingular.
- `estimateAssetMoments` uses default settings for the missing data estimation procedure that might not be appropriate for all problems.

In either case, you might want to estimate the moments of asset returns separately with either the ECM estimation functions such as `ecmmle` or with your own functions.

mv_optim_transform Errors

If you obtain optimization errors such as:

```
Error using mv_optim_transform (line 233)
Portfolio set appears to be either empty or unbounded. Check constraints.
```

```
Error in Portfolio/estimateFrontier (line 63)
  [A, b, f0, f, H, g, lb] = mv_optim_transform(obj);
```

or

```
Error using mv_optim_transform (line 238)
Cannot obtain finite lower bounds for specified portfolio set.
```

```
Error in Portfolio/estimateFrontier (line 63)
  [A, b, f0, f, H, g, lb] = mv_optim_transform(obj);
```

Since the portfolio optimization tools require a bounded portfolio set, these errors (and similar errors) can occur if your portfolio set is either empty and, if nonempty, unbounded. Specifically, the portfolio optimization algorithm requires that your portfolio set have at least a finite lower bound. The best way to deal with these problems is to use the validation functions in “Validate the Portfolio Problem for Portfolio Object” on page 4-94. Specifically, use `estimateBounds` to examine your portfolio set, and use `checkFeasibility` to ensure that your initial portfolio is either feasible and, if infeasible, that you have sufficient turnover to get from your initial portfolio to the portfolio set.

Tip To correct this problem, try solving your problem with larger values for turnover or tracking-error and gradually reduce to the value that you want.

solveContinuousCustomObjProb or solveMICustomObjProb Errors

These errors can occur if the portfolio set is empty or unbounded. If the portfolio set is empty, the error states that the problem is infeasible. The best way to deal with these problems is to use the validation functions in “Validate the Portfolio Problem for Portfolio Object” on page 4-94. Specifically, use `estimateBounds` to examine your portfolio set, and use `checkFeasibility` to ensure that your initial portfolio is either feasible or, if infeasible, that you have sufficient turnover to get from your initial portfolio to the portfolio set.

Efficient Portfolios Do Not Make Sense

If you obtain efficient portfolios that do not seem to make sense, this can happen if you forget to set specific constraints or you set incorrect constraints. For example, if you allow portfolio weights to fall between 0 and 1 and do not set a budget constraint, you can get portfolios that are 100% invested in every asset. Although it may be hard to detect, the best thing to do is to review the constraints you have set with `display` of the object. If you get portfolios with 100% invested in each asset, you can review the display of your object and quickly see that no budget constraint is set. Also, you can use `estimateBounds` and `checkFeasibility` to determine if the bounds for your portfolio set make sense and to determine if the portfolios you obtained are feasible relative to an independent formulation of your portfolio set.

Efficient Frontiers Do Not Make Sense

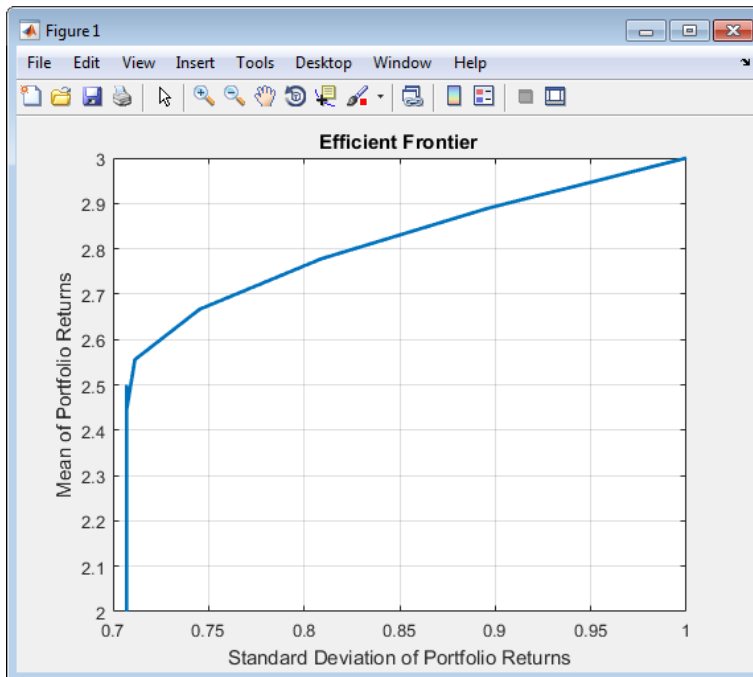
If you obtain efficient frontiers that do not seem to make sense, this can happen for some cases of mean and covariance of asset returns. It is possible for some mean-variance portfolio optimization problems to have difficulties at the endpoints of the efficient frontier. It is rare for standard portfolio problems, but this can occur. For example, this can occur when using unusual combinations of turnover constraints and transaction costs. Usually, the workaround of setting the hidden property `enforcePareto` produces a single portfolio for the entire efficient frontier, where any other solutions are not Pareto optimal (which is what efficient portfolios must be).

An example of a portfolio optimization problem that has difficulties at the endpoints of the efficient frontier is this standard mean-variance portfolio problem (long-only with a budget constraint) with the following mean and covariance of asset returns:

```
m = [ 1; 2; 3 ];
C = [ 1 1 0; 1 1 0; 0 0 1 ];

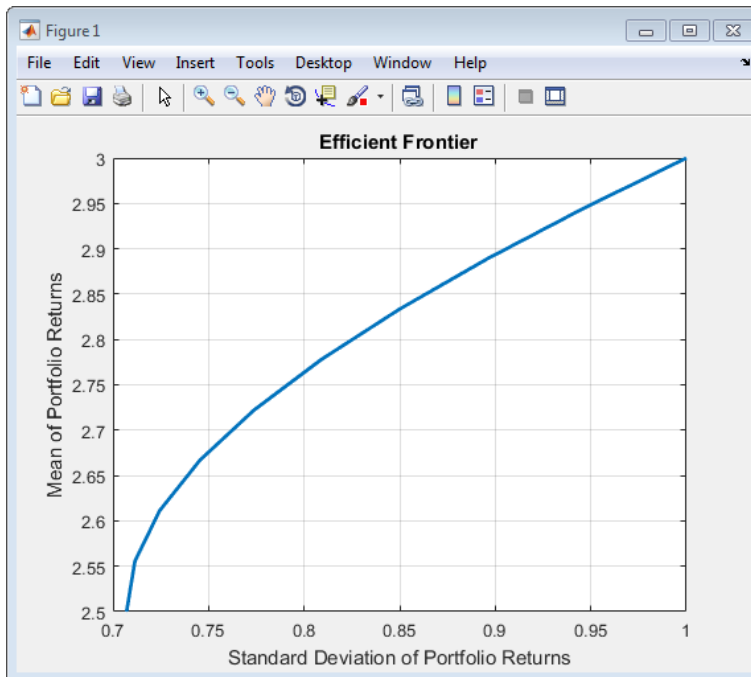
p = Portfolio;
p = Portfolio(p, 'assetmean', m, 'assetcovar', C);
p = Portfolio(p, 'lowerbudget', 1, 'upperbudget', 1);
p = Portfolio(p, 'lowerbound', 0);

plotFrontier(p)
```



To work around this problem, set the hidden Portfolio object property for `enforcePareto`. This property instructs the optimizer to perform extra steps to ensure a Pareto-optimal solution. This slows down the solver, but guarantees a Pareto-optimal solution.

```
p.enforcePareto = true;
plotFrontier(p)
```



Troubleshooting estimateCustomObjectivePortfolio

When using `estimateCustomObjectivePortfolio`, nonconvex functions are not supported for problems with cardinality constraints or conditional bounds. Specifically, the following error displays when using `estimateCustomObjectivePortfolio` with a `Portfolio` object that includes 'Conditional' `BoundType` (semicontinuous) constraints using `setBounds` or `MinNumAssets` and `MaxNumAssets` (cardinality) constraints using `setMinMaxNumAssets`.

```
Error using solveMICustomObjProb
Objective function must be convex in problems with cardinality constraints and/or conditional bounds.

Error in Portfolio/estimateCustomObjectivePortfolio (line 88)
    [pwgt,exitflag] = solveMICustomObjProb(obj,prob,fun,flags);
```

Note This error applies only to quadratic functions. This error is not detected in nonlinear functions. Therefore, if you are using a nonlinear function, you must validate your input.

For more information, see “Role of Convexity in Portfolio Problems” on page 4-157.

Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints

When configuring a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to include 'Conditional' `BoundType` (semicontinuous) constraints using `setBounds` or `MinNumAssets` and `MaxNumAssets` (cardinality) constraints using `setMinMaxNumAssets`, the values of the inputs that you supply can result in warning messages.

Conditional Bounds with LowerBound Defined as Empty or Zero

When using `setBounds` with the `BoundType` set to 'Conditional' and the `LowerBound` input argument is empty (`[]`) or `0`, the Conditional bound is not effective and is equivalent to a Simple bound.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
p = setBounds(p, 0, 0.5, 'BoundType', 'Conditional');
p = setMinNumAssets(p, 3, 3);
estimateFrontier(p, 10)

Warning: Conditional bounds with 'LowerBound' as zero are equivalent to simple bounds.
Consider either using strictly positive 'LowerBound' or 'simple' as the 'BoundType'
instead.
> In internal.finance.PortfolioMixedInteger/checkBoundType (line 46)
   In Portfolio/checkarguments (line 204)
   In Portfolio/setBounds (line 80)
Warning: The solution may have less than 'MinNumAssets' assets with nonzero weight. To
enforce 'MinNumAssets' requirement, set strictly positive lower conditional bounds.
> In internal.finance.PortfolioMixedInteger/hasIntegerConstraints (line 44)
   In Portfolio/estimateFrontier (line 51)

ans =

Columns 1 through 8

    0.5000    0.3555    0.3011    0.3299    0.3585    0.3873    0.4160    0.4448
    0.5000    0.5000    0.4653    0.3987    0.3322    0.2655    0.1989    0.1323
    0.0000    0.1445    0.2335    0.2714    0.3093    0.3472    0.3850    0.4229

Columns 9 through 10

    0.4735    0.5000
    0.0657         0
    0.4608    0.5000
```

In all the 10 optimal allocations, there are allocations (the first and last ones) that only have two assets, which is in conflict with the `MinNumAssets` constraint that three assets should be allocated. Also there are two warnings, which actually explain what happens. In this case, the 'Conditional' bound constraints are defined as $x_i = 0$ or $0 \leq x_i \leq 0.5$, which are internally modeled as $0 \leq x_i \leq 0.5 \cdot v_i$, where v_i is 0 or 1, where 0 indicates not allocated, and 1 indicates allocated. Here, $v_i=1$, which still allows the asset to have a weight of 0. In other words, setting `LowerBound` as 0 or empty, doesn't clearly define the minimum allocation for an allocated asset. Therefore, a 0 weighted asset is also considered as an allocated asset. To fix this warning, follow the instructions in the warning message, and set a `LowerBound` value that is strictly positive.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
p = setBounds(p, 0.3, 0.5, 'BoundType', 'Conditional');
p = setMinNumAssets(p, 3, 3);
estimateFrontier(p, 10)

ans =

Columns 1 through 8

    0.3000    0.3180    0.3353    0.3489    0.3580    0.3638    0.3694    0.3576
    0.4000    0.3820    0.3642    0.3479    0.3333    0.3199    0.3067    0.3001
    0.3000    0.3000    0.3005    0.3032    0.3088    0.3163    0.3240    0.3423

Columns 9 through 10

    0.3289    0.3000
    0.3000    0.3000
    0.3711    0.4000
```

Length of 'BoundType' Must Be Conformable with NumAssets

The `setBounds` optional name-value argument for 'BoundType' must be defined for all assets in a `Portfolio`, `PortfolioCvAr`, or `PortfolioMAD` object. By default, the 'BoundType' is 'Simple' and applies to all assets. Using `setBounds`, you can choose to define a 'BoundType' for each asset. In this case, the number of 'BoundType' specifications must match the number of assets (`NumAssets`) in the `Portfolio`, `PortfolioCvAr`, or `PortfolioMAD` object. The following example demonstrates the error when the number of 'BoundType' specifications do not match the number of assets in the `Portfolio` object.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
p = setBounds(p, 0.1, 0.5, 'BoundType', ["simple"; "conditional"]);

Cannot create bound constraints.

Caused by:
  Error using internal.finance.PortfolioMixedInteger/checkBoundType (line 28)
    Length of 'BoundType' must be conformable with 'NumAssets'=3.
```

To correct this, modify the `BoundType` to include three specifications because the `Portfolio` object has three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
p = setBounds(p, 0.1, 0.5, 'BoundType', ["simple"; "conditional"; "conditional"]);
p.BoundType

p =

Portfolio with properties:
    BuyCost: []
    SellCost: []
    RiskFreeRate: []
    AssetMean: [3x1 double]
    AssetCovar: [3x3 double]
    TrackingError: []
    TrackingPort: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
    Name: []
    NumAssets: 3
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [3x1 double]
    UpperBound: [3x1 double]
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    BoundType: [3x1 categorical]
    MinNumAssets: []
    MaxNumAssets: []

ans =

3x1 categorical array
```

```
simple
conditional
conditional
```

Redundant Constraints from 'BoundType', 'MinNumAssets', 'MaxNumAssets' Constraints

When none of the constraints from 'BoundType', 'MinNumAssets', or 'MaxNumAssets' are active, the redundant constraints from 'BoundType', 'MinNumAssets', 'MaxNumAssets' warning occurs. This happens when you explicitly use `setBounds` and `setMinMaxNumAssets` but with values that are inactive. That is, the 'Conditional' BoundType has a LowerBound = [] or 0, 'MinNumAssets' is 0, or 'MaxNumAssets' is the same value as NumAssets. In other words, if any of these three are active, the warning will not show up when using the estimate functions or `plotFrontier`. The following two examples show the rationale.

The first example is when the BoundType is explicitly set as 'Conditional' but the LowerBound is 0, and no 'MinNumAssets' and 'MaxNumAssets' constraints are defined using `setMinMaxNumAssets`.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
p = setBounds(p, 0, 0.5, 'BoundType', 'Conditional');
estimateFrontier(p, 10)
```

```
Warning: Redundant constraints from 'BoundType', 'MinNumAssets', 'MaxNumAssets'.
> In internal.finance.PortfolioMixedInteger/hasIntegerConstraints (line 24)
   In Portfolio/estimateFrontier (line 51)
```

```
ans =
```

```
Columns 1 through 8
```

```
0.5000    0.3555    0.3011    0.3299    0.3586    0.3873    0.4160    0.4448
0.5000    0.5000    0.4653    0.3987    0.3321    0.2655    0.1989    0.1323
0         0.1445    0.2335    0.2714    0.3093    0.3471    0.3850    0.4229
```

```
Columns 9 through 10
```

```
0.4735    0.5000
0.0657     0
0.4608    0.5000
```

The second example is when you explicitly set the three constraints, but all with inactive values. In this example, the BoundType is 'Conditional' and the LowerBound is 0, thus specifying ineffective 'Conditional' BoundType constraints, and the 'MinNumAssets' and 'MaxNumAssets' values are 0 and 3, respectively. The `setMinMaxNumAssets` function specifies ineffective 'MinNumAssets' and 'MaxNumAssets' constraints.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
p = setBounds(p, 0, 0.5, 'BoundType', 'Conditional');
p = setMinMaxNumAssets(p, 0, 3);
estimateFrontier(p, 10)
```

```
Warning: Redundant constraints from 'BoundType', 'MinNumAssets', 'MaxNumAssets'.
> In internal.finance.PortfolioMixedInteger/hasIntegerConstraints (line 24)
   In Portfolio/estimateFrontier (line 51)
```

```
ans =
```

```
Columns 1 through 8
```

```
0.5000    0.3555    0.3011    0.3299    0.3586    0.3873    0.4160    0.4448
0.5000    0.5000    0.4653    0.3987    0.3321    0.2655    0.1989    0.1323
0         0.1445    0.2335    0.2714    0.3093    0.3471    0.3850    0.4229
```

```
Columns 9 through 10
```

```

0.4735    0.5000
0.0657    0
0.4608    0.5000

```

Infeasible Problem with Conditional (Semi-Continuous) Bounds, Cardinality Constraints, or Conditional Budget Constraints

The `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` object performs validations of all the constraints before solving any specific optimization problems. The `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` object first considers all constraints other than conditional bounds, cardinality constraints, and conditional budget constraints and issues an error message if they are not compatible. Then the `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` object adds the remaining constraints to check if they are compatible with the already checked constraints. This separation is natural because conditional bounds, cardinality constraints, and conditional budget constraints require additional binary variables in the mathematical formulation that leads to a MINLP, while other constraints only need continuous variables. You can follow the error messages to check when the infeasible problem occurs and take actions to fix the constraints.

One possible scenario is when the `BoundType` is 'Conditional' and Groups are defined for the `Portfolio` object. In this case, the Group definitions are themselves in conflict. Consequently, the 'Conditional' bound constraint cannot be applied when running `estimateFrontierLimits`.

```

AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
p = setBounds(p, 0.1, 0.5, 'BoundType', 'Conditional');
p = setGroups(p, [1,1,0], 0.3, 0.5);
p = addGroups(p, [0,1,0], 0.6, 0.7);
pwgt = estimateFrontierLimits(p)

Error using Portfolio/buildMixedIntegerProblem (line 31)
Infeasible problem prior to considering constraints involving integer variables.
Verify compatibility of continuous constraints.

Error in Portfolio/estimateFrontierLimits>int_frontierLimits (line 93)
ProbStruct = buildMixedIntegerProblem(obj);

Error in Portfolio/estimateFrontierLimits (line 73)
pwgt = int_frontierLimits(obj, minsolution, maxsolution);

```

To correct this error, change the `LowerGroup` in the `addGroups` function to also be 0.3 to match the `GroupMatrix` input from `setGroups`.

```

AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
p = setBounds(p, 0.1, 0.5, 'BoundType', 'Conditional');
p = setGroups(p, [1,1,0], 0.3, 0.5);
p = addGroups(p, [0,1,0], 0.3, 0.7);
pwgt = estimateFrontierLimits(p)

pwgt =

    0    0.2000
 0.5000 0.3000
 0.5000 0.5000

```

A second possible scenario is when the `BoundType` is 'Conditional' and the `setEquality` function is used with the `bEquality` parameter set to 0.04. This sets an equality constraint to have $x_1 + x_3 = 0.04$. At the same time, `setBounds` also set the semicontinuous constraints to have $x_i = 0$ or $0.1 \leq x_i \leq 2.5$, which lead to $x_1 + x_3 = 0$ or $0.1 \leq x_1 + x_3 \leq 5$. The semicontinuous

constraints are not compatible with the equality constraint because there is no way to get $x_1 + x_3$ to equal 0.04. Therefore, the error message is displayed.

```
AssetMean = [ 0.05; 0.1; 0.12; 0.18 ];
AssetCovar = [ 0.0064  0.00408  0.00192  0;
               0.00408  0.0289  0.0204  0.0119;
               0.00192  0.0204  0.0576  0.0336;
               0        0.0119  0.0336  0.1225 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
A = [ 1 0 1 0 ];
b = 0.04;
p = setEquality(p, A, b);
p = setBounds(p, 0.1, 2.5, 'BoundType', 'Conditional');
p = setMinMaxNumAssets(p, 2, 2);
pwgt = estimateFrontierLimits(p)

Error using Portfolio/buildMixedIntegerProblem (line 109)
Infeasible problem when considering constraints involving integer variables.
Verify constraints compatibility or decrease 'AbsoluteGapTolerance' or 'RelativeGapTolerance'.

Error in Portfolio/estimateFrontierLimits>int_frontierLimits (line 93)
ProbStruct = buildMixedIntegerProblem(obj);

Error in Portfolio/estimateFrontierLimits (line 73)
    pwgt = int_frontierLimits(obj, minsolution, maxsolution);
```

To correct this error, change the `bEquality` parameter from 0.04 to .4.

```
AssetMean = [ 0.05; 0.1; 0.12; 0.18 ];
AssetCovar = [ 0.0064  0.00408  0.00192  0;
               0.00408  0.0289  0.0204  0.0119;
               0.00192  0.0204  0.0576  0.0336;
               0        0.0119  0.0336  0.1225 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, 'Budget', 1);
A = [ 1 0 1 0 ];
b = 0.4;
p = setEquality(p, A, b);
p = setBounds(p, 0.1, 2.5, 'BoundType', 'Conditional');
p = setMinMaxNumAssets(p, 2, 2);
pwgt = estimateFrontierLimits(p)

pwgt =

    0.4000         0
    0.6000         0
         0    0.4000
         0    0.6000
```

Redundant Conditional Budget Constraint

This redundant conditional budget constraint warning occurs when you are using `setConditionalBudget` with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object and the conditional threshold of all assets is greater than or equal to their respective upper bounds. This happens when the $\text{UpperBound} \leq \text{ConditionalBudgetThreshold}$. In this case, assets are not allowed to invest more than the conditional threshold, which makes the conditional budget constraint unnecessary. The following example shows the case where this happens:

```
assetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
assetCovar = [ 0.00324625  0.00022983  0.00420395;
               0.00022983  0.00049937  0.00019247;
               0.00420395  0.00019247  0.00764097 ];
p = Portfolio(AssetMean=assetMean, AssetCovar=assetCovar);
p = setBounds(p, 0, 0.5);
p = setConditionalBudget(p, 0.5, 0.75);
estimateFrontier(p)

Warning: Redundant conditional budget constraint.
> In internal.finance.PortfolioMixedInteger/hasIntegerConstraints (line 59)
In Portfolio/estimateFrontier (line 54)

ans =

         0    0.0454    0.0909    0.1825    0.3186    0.4547    0.5000    0.5000    0.5000    0.5000
```

```

0 0.2008 0.4015 0.5000 0.5000 0.5000 0.5000 0.5000 0.5000 0.5000
0 0.0169 0.0338 0.0491 0.0629 0.0766 0.1574 0.2716 0.3858 0.5000

```

To correct this warning, use `setBounds` to specify an `UpperBound` value that is `>` than the `ConditionalBudgetThreshold` that you specify using `setConditionalBudget`.

```

assetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
assetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio(AssetMean=assetMean,AssetCovar=assetCovar);
p = setBounds(p,0,0.55);
p = setConditionalBudget(p,0.5,0.75);
estimateFrontier(p)

ans =

0.0000 0.0477 0.0953 0.1799 0.3226 0.4653 0.5000 0.5000 0.5000 0.5000
0.0001 0.2107 0.4212 0.5500 0.5500 0.5500 0.5500 0.5500 0.5500 0.5000
0.0000 0.0177 0.0355 0.0519 0.0663 0.0808 0.1749 0.2947 0.4144 0.5500

```

Unbounded Portfolio Problem

This error occurs when you are using a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object and there is no `UpperBound` defined in `setBounds` and you are using `setMinNumAssets`. In this case, this is formulated as a mixed integer programming problem and an `UpperBound` is required to enforce `MinNumAssets` and `MaxNumAssets` constraints.

The optimizer first attempts to estimate the upper bound of each asset, based on all the specified constraints. If the `UpperBound` cannot be found, an error message occurs which instructs you to set an explicit `UpperBound`. In most cases, as long as you set some upper bounds to the problem using any `set` function, the optimizer can successfully find a good estimation.

```

AssetMean = [ 0.05; 0.1; 0.12; 0.18 ];
AssetCovar = [ 0.0064 0.00408 0.00192 0;
               0.00408 0.0289 0.0204 0.0119;
               0.00192 0.0204 0.0576 0.0336;
               0 0.0119 0.0336 0.1225 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setBounds(p, 0.1, 'BoundType','Conditional');
p = setGroups(p, [1,1,1,0], 0.3, 0.5);
p = setMinNumAssets(p, 3, 3);
pwgt = estimateFrontierLimits(p)

Error using Portfolio/buildMixedIntegerProblem (line 42)
Unbounded portfolio problem. Upper bounds cannot be inferred from the existing
constraints. Set finite upper bounds using 'setBounds'.

Error in Portfolio/estimateFrontierLimits>int_frontierLimits (line 93)
ProbStruct = buildMixedIntegerProblem(obj);

Error in Portfolio/estimateFrontierLimits (line 73)
pwgt = int_frontierLimits(obj, minsolution, maxsolution);

```

To correct this error, specify an `UpperBound` value for `setBounds`.

```

AssetMean = [ 0.05; 0.1; 0.12; 0.18 ];
AssetCovar = [ 0.0064 0.00408 0.00192 0;
               0.00408 0.0289 0.0204 0.0119;
               0.00192 0.0204 0.0576 0.0336;
               0 0.0119 0.0336 0.1225 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setBounds(p, 0.1, .9, 'BoundType','Conditional');
p = setGroups(p, [1,1,1,0], 0.3, 0.5);
p = setMinNumAssets(p, 3, 3);
pwgt = estimateFrontierLimits(p)

pwgt =

0.1000 0
0.1000 0.1000
0.1000 0.4000
0 0.9000

```

Total Number of Portfolio Weights with a Value > 0 Are Greater Than MaxNumAssets Specified

When using a Portfolio, PortfolioCVaR, or PortfolioMAD object, the optimal allocation w may contain some very small values that leads to $\text{sum}(w>0)$ larger than MaxNumAssets, even though the MaxNumAssets constraint is specified using setMinMaxNumAssets. For example, in the following code when setMinMaxNumAssets is used to set MaxNumAssets to 15, the $\text{sum}(w>0)$ indicates that there are 19 assets. A close examination of the weights shows that the weights are extremely small and are actually 0.

```
T = readtable('dowPortfolio.xlsx');
symbol = T.Properties.VariableNames(3:end);
assetReturn = tick2ret(T(:,3:end));
p = Portfolio('AssetList', symbol, 'budget', 1);
p = setMinMaxNumAssets(p, 10, 15);
p = estimateAssetMoments(p,assetReturn);
p = setBounds(p,0.01,0.5,'BoundType','Conditional','NumAssets',30);
p = setTrackingError(p,0.05,ones(1, p.NumAssets)/p.NumAssets);
w = estimateFrontierLimits(p,'min'); % minimum risk portfolio

sum(w>0) % Number of assets that are allocated in the optimal portfolio
w(w<eps) % Check the weights of the very small weighted assets

ans =

    19

ans =

    1.0e-20 *
    -0.0000
         0
         0
    0.0293
         0
    0.3626
    0.2494
         0
    0.0926
   -0.0000
         0
    0.0020
         0
         0
         0
         0
         0
```

This situation only happens when the OuterApproximation algorithm is used with setSolverMINLP to solve a MINLP portfolio optimization problem. The OuterApproximation internally fixes the latest solved integer variables and runs an NLP with quadprog or fmincon, which introduces numerical issues and leads to weights that are very close to 0.

If you do not want to deal with very small values, you can use setSolverMINLP to select a different algorithm. In this example, the 'TrustRegionCP' algorithm is specified.

```
T = readtable('dowPortfolio.xlsx');
symbol = T.Properties.VariableNames(3:end);
assetReturn = tick2ret(T(:,3:end));
p = Portfolio('AssetList', symbol, 'budget', 1);
p = setMinMaxNumAssets(p, 10, 15);
p = estimateAssetMoments(p,assetReturn);
p = setBounds(p,0.01,0.5,'BoundType','Conditional','NumAssets',30);
p = setTrackingError(p,0.05,ones(1, p.NumAssets)/p.NumAssets);
p = setSolverMINLP(p,'TrustRegionCP');
w = estimateFrontierLimits(p,'min'); % minimum risk portfolio

sum(w>0) % Number of assets that are allocated in the optimal portfolio
w(w<eps) % The weights of the very small weighted assets are strictly zeros

ans =
```

14

ans =

0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0

See Also

`Portfolio` | `estimateAssetMoments` | `checkFeasibility` | `setBounds` | `setMinMaxNumAssets`

Related Examples

- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121
- “Troubleshooting MAD Portfolio Optimization Results” on page 6-118

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

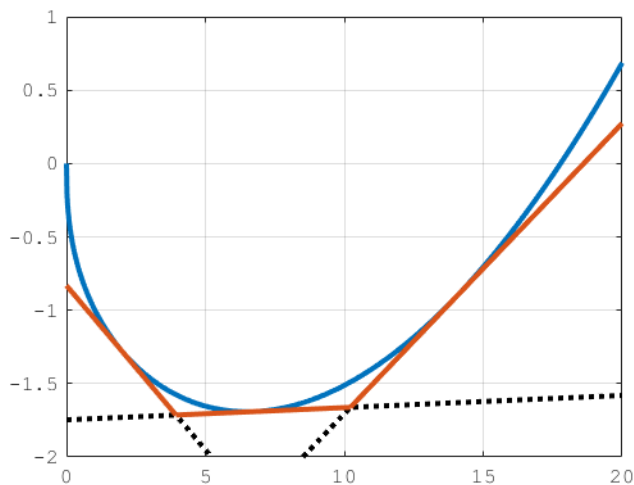
External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

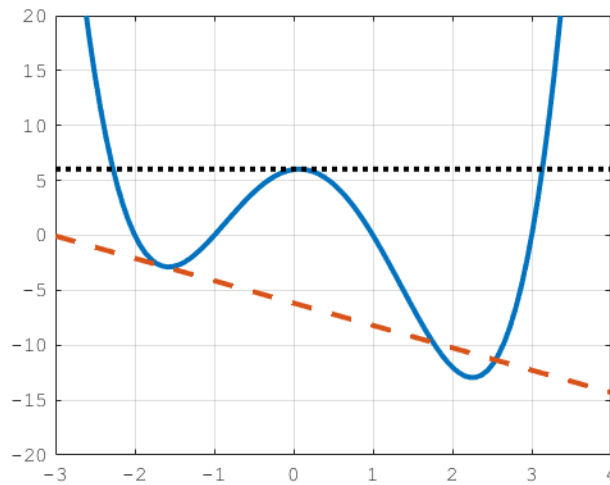
Role of Convexity in Portfolio Problems

When using a `Portfolio` object and the `estimateCustomObjectivePortfolio` function with the Financial Toolbox solvers, the solvers return a local minimum, which is not guaranteed to be a global minimum. A local minimum of a function is a point where the function value is smaller than the points around it, but the value is larger than a distant point. A global minimum is the point where the function value is smaller than at all other points. Convex functions play an important role in optimization because they guarantee that all local minima are global minima. In other words, any solution found by the solver is guaranteed to be a global minimum.

Another important feature of convex functions is that all tangents of the function lie underneath the function. This feature is particularly useful in problems with cardinality constraints or conditional bounds. For more information on cardinality constraints and conditional bounds, see “Working with ‘Conditional’ `BoundType`, `MinNumAssets`, and `MaxNumAssets` Constraints Using Portfolio Objects” on page 4-81 and `setMinMaxNumAssets`. The Financial Toolbox internal solvers for portfolio problems with binary variables use an iterative method that approximates the nonlinear functions (for example, variance) with a collection of linear functions. The linear functions used to approximate the original function are the tangents of the function at different points. Because the tangents for convex functions always lie underneath the function, the Financial Toolbox solvers guarantee that the approximation does not remove any minimum from the problem.



Using Financial Toolbox solvers for portfolio problems with nonconvex functions and cardinality constraints or conditional bounds might result in suboptimal solutions. The reason is that the tangent lines used to approximate the nonconvex functions might cut off different minima as demonstrated in the following figure:



Even though the red dashed tangent line cuts off only a small piece of the graph, that small piece already contains an important point, the global minima. In the worst case scenario, the Financial Toolbox solvers compute the tangent line at a point that removes all minima of the function, like the black dotted line does. Therefore, it is important to be aware of the type of function passed to a Financial Toolbox solver.

All the above applies to minimization problems. Conversely, maximization problems with cardinality constraints or conditional bounds can be solved by the Financial Toolbox solvers only if the functions are concave.

Examples of Convex Functions

Here are some examples of common convex functions used in finance. You can use these convex functions in minimization problems when using a `Portfolio` object with the `estimateCustomObjectivePortfolio` function:

- Variance:

$$x^T \Sigma x$$

- Tracking error:

$$(x - x_0)^T \Sigma (x - x_0)$$

- Herfindahl-Hirschman index:

$$x^T x$$

- Any linear function
- Any positive combination of convex functions:

$$x^T \Sigma x + \lambda (x^T \Sigma (x - x_0)), \lambda \geq 0$$

$$x^T \Sigma x + \lambda_1 x^T x + \lambda_2 (x^T \Sigma (x - x_0)), \lambda_1, \lambda_2 \geq 0$$

Note If the portfolio problem has cardinality constraints or conditional bounds specified using `setMinMaxNumAssets` or `setBounds`, the objective function must also be convex.

Examples of Concave Functions

Here are some examples of common concave functions used in finance. You can use concave functions in maximization problems when using a `Portfolio` object with the `estimateCustomObjectivePortfolio` function.

- Risk penalized return:

$$\mu^T x - \lambda x^T \Sigma x, \lambda \geq 0$$

- Any linear function
- Any positive combination of concave functions

Examples of Nonconvex Functions

Here are some examples of common nonconvex functions used in finance. You can use nonconvex functions in minimization or maximization problems when using a `Portfolio` object *without cardinality constraints or conditional bounds* with the `estimateCustomObjectivePortfolio` function.

- Sharpe ratio:

$$\frac{\mu^T x - r_f}{\sqrt{x^T \Sigma x}}$$

- Information ratio:

$$\frac{\mu^T (x - x_0)}{\sqrt{(x - x_0)^T \Sigma (x - x_0)}}$$

- Diversification ratio:

$$\frac{\sigma^T x}{\sqrt{x^T \Sigma x}}$$

- Risk budgeting deviation:

$$\sum_i \left(\frac{x_i (\Sigma x)_i}{x^T \Sigma x} - b_i \right)^2$$

- The following function is a special case:

$$x^T \Sigma x - \lambda \sum_i \ln x_i$$

This function is commonly used to construct risk parity portfolios. Although this function is convex, problems with cardinality constraints or conditional bounds make this function ill-defined. This happens because $\ln x_i$ is not defined when $x_i = 0$. Since adding cardinality constraints or conditional bounds imply that some assets have a weight of 0, the function becomes ill-defined at the points of interest.

Note If the portfolio problem has cardinality constraints or conditional bounds specified using `setMinMaxNumAssets` or `setBounds`, the objective function cannot be nonconvex.

See Also

Portfolio | `estimateCustomObjectivePortfolio`

Related Examples

- “Risk Parity or Budgeting with Constraints” on page 4-357
- “Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty” on page 4-351
- “Solve Problem for Minimum Tracking Error with Net Return Constraint” on page 4-349
- “Solve Tracking Error Portfolio Problems” on page 4-343
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Solver Guidelines for Custom Objective Problems Using Portfolio Objects” on page 4-119

Portfolio Optimization Examples Using Financial Toolbox

Follow a sequence of examples that highlight features of the `Portfolio` object. Specifically, the examples use the `Portfolio` object to show how to set up mean-variance portfolio optimization problems that focus on the two-fund theorem, the impact of transaction costs and turnover constraints, how to obtain portfolios that maximize the Sharpe ratio, and how to set up two popular hedge-fund strategies — dollar-neutral and 130-30 portfolios.

Set up the Data

Every example works with moments for monthly total returns of a universe of 30 "blue-chip" stocks. Although derived from real data, these data are for illustrative purposes and are not meant to be representative of specific assets or of market performance. The data are contained in the file `BlueChipStockMoments.mat` with a list of asset identifiers in the variable `AssetList`, a mean and covariance of asset returns in the variables `AssetMean` and `AssetCovar`, and the mean and variance of cash and market returns in the variables `CashMean`, `CashVar`, `MarketMean`, and `MarketVar`. Since most of the analysis requires the use of the standard deviation of asset returns as the proxy for risk, cash, and market variances are converted into standard deviations.

```
load BlueChipStockMoments
```

```
mret = MarketMean;
mrsk = sqrt(MarketVar);
cret = CashMean;
crsk = sqrt(CashVar);
```

Create a Portfolio Object

First create a "standard" `Portfolio` object with `Portfolio` to incorporate the list of assets, the risk-free rate, and the moments of asset returns into the object.

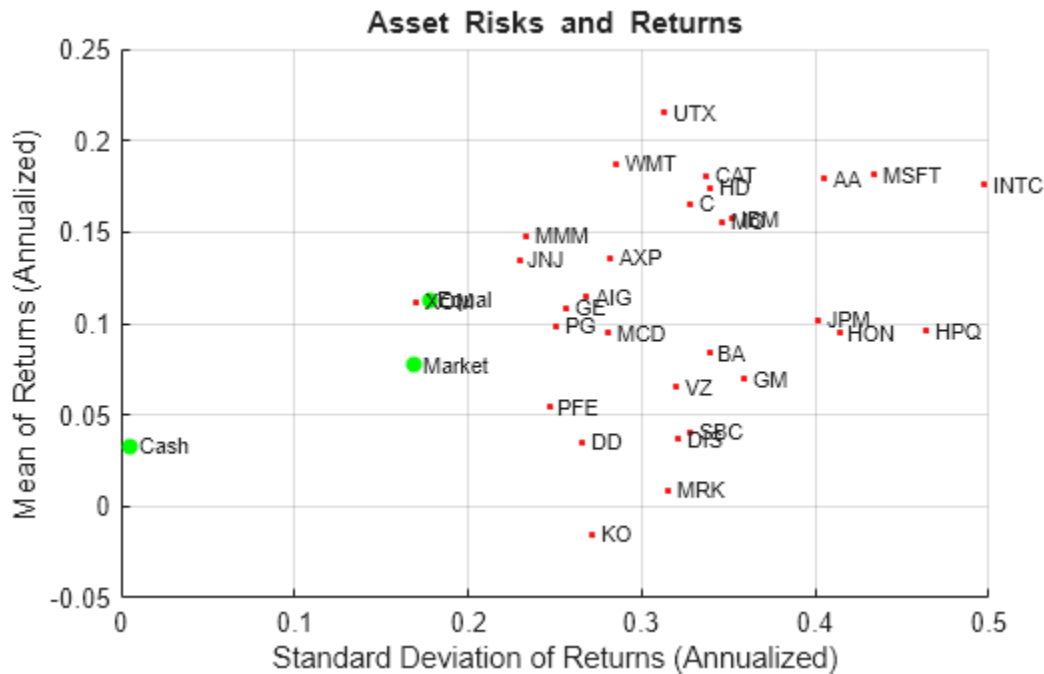
```
p = Portfolio('AssetList',AssetList,'RiskFreeRate',CashMean);
p = setAssetMoments(p,AssetMean,AssetCovar);
```

To provide a basis for comparison, set up an equal-weight portfolio and make it the initial portfolio in the `Portfolio` object. Keep in mind that the hedged portfolios to be constructed later will require a different initial portfolio. Once the initial portfolio is created, the `estimatePortMoments` function estimates the mean and standard deviation of equal-weight portfolio returns.

```
p = setInitPort(p,1/p.NumAssets);
[ersk,eret] = estimatePortMoments(p,p.InitPort);
```

A specialized "helper" function `portfolioexamples_plot` makes it possible to plot all results to be developed here. The first plot shows the distribution of individual assets according to their means and standard deviations of returns. In addition, the equal-weight, market, and cash portfolios are plotted on the same plot. Note that the `portfolioexamples_plot` function converts monthly total returns into annualized total returns.

```
clf;
portfolioexamples_plot('Asset Risks and Returns', ...
    {'scatter', mrsk, mret, {'Market'}}, ...
    {'scatter', crsk, cret, {'Cash'}}, ...
    {'scatter', ersk, eret, {'Equal'}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, '.r'});
```



Set up a Portfolio Optimization Problem

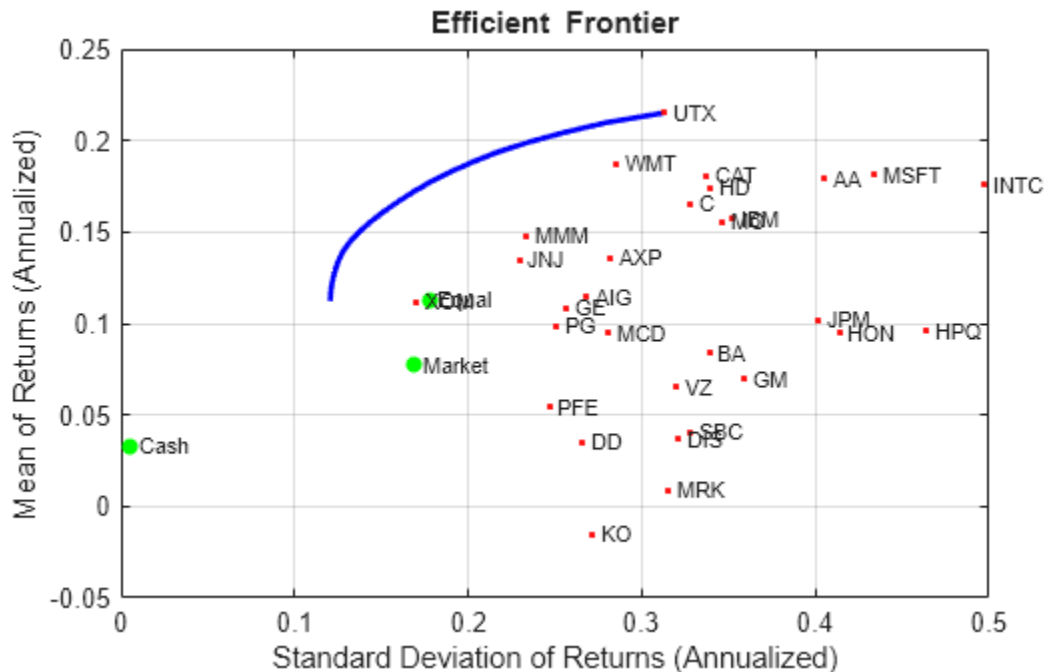
Set up a "standard" or default mean-variance portfolio optimization problem with the `setDefaultConstraints` function that requires fully-invested long-only portfolios (non-negative weights that must sum to 1). Given this initial problem, estimate the efficient frontier with the functions `estimateFrontier` and `estimatePortMoments`, where `estimateFrontier` estimates efficient portfolios and `estimatePortMoments` estimates risks and returns for portfolios. The next figure overlays the efficient frontier on the previous plot.

```
p = setDefaultConstraints(p);

pwgt = estimateFrontier(p,20);
[prsk,pret] = estimatePortMoments(p,pwgt);

% Plot the efficient frontier.

clf;
portfolioexamples_plot('Efficient Frontier', ...
    {'line', prsk, pret}, ...
    {'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, '.r'});
```



Illustrate the Tangent Line to the Efficient Frontier

Tobin's mutual fund theorem (Tobin 1958 on page 4-177) says that the portfolio allocation problem is viewed as a decision to allocate between a riskless asset and a risky portfolio. In the mean-variance framework, cash serves as a proxy for a riskless asset and an efficient portfolio on the efficient frontier serves as the risky portfolio such that any allocation between cash and this portfolio dominates all other portfolios on the efficient frontier. This portfolio is called a *tangency portfolio* because it is located at the point on the efficient frontier where a tangent line that originates at the riskless asset touches the efficient frontier.

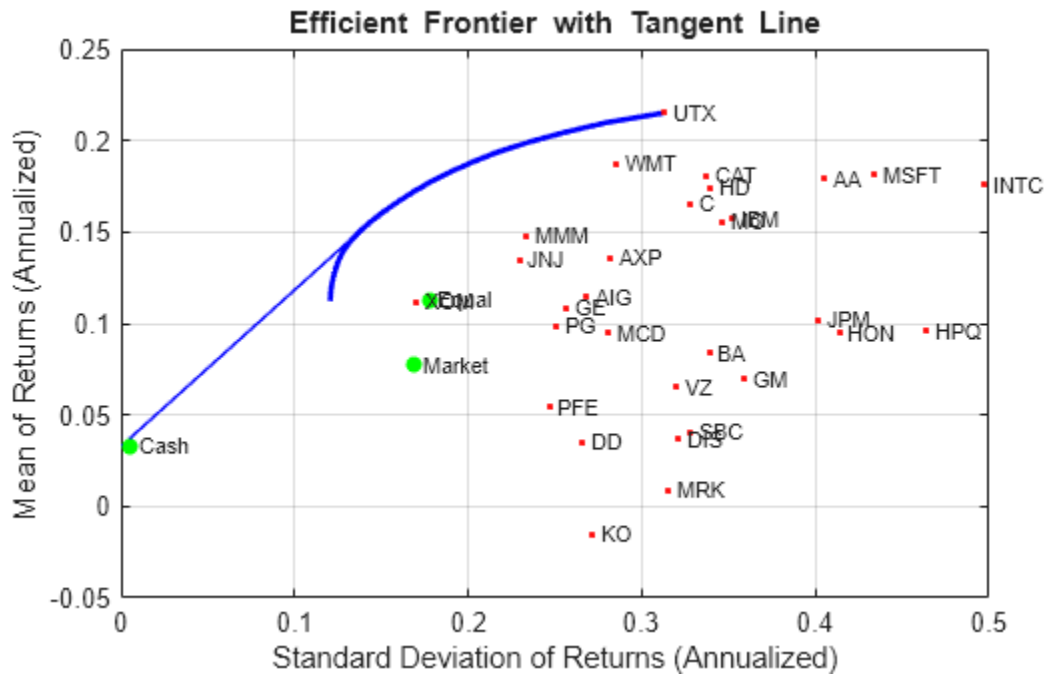
Given that the `Portfolio` object already has the risk-free rate, obtain the tangent line by creating a copy of the `Portfolio` object with a budget constraint that permits allocation between 0% and 100% in cash. Since the `Portfolio` object is a value object, it is easy to create a copy by assigning the output of either `Portfolio` or the "set" functions to a new instance of the `Portfolio` object. The plot shows the efficient frontier with Tobin's allocations that form the tangent line to the efficient frontier.

```
q = setBudget(p, 0, 1);

qwgts = estimateFrontier(q,20);
[qrsk,qret] = estimatePortMoments(q,qwgts);

% Plot efficient frontier with tangent line (0 to 1 cash).

clf;
portfolioexamples_plot('Efficient Frontier with Tangent Line', ...
    {'line', prsk, pret}, ...
    {'line', qrsk, qret, [], [], 1}, ...
    {'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, '.r'});
```



Note that cash actually has a small risk so that the tangent line does not pass through the cash asset.

Obtain Range of Risks and Returns

To obtain efficient portfolios with target values of either risk or return, it is necessary to obtain the range of risks and returns among all portfolios on the efficient frontier. This is accomplished with the `estimateFrontierLimits` function.

```
[rsk,ret] = estimatePortMoments(p,estimateFrontierLimits(p));

display(rsk)

rsk = 2×1

    0.0348
    0.0903

display(ret)

ret = 2×1

    0.0094
    0.0179
```

The range of monthly portfolio returns is between 0.9% and 1.8% and the range for portfolio risks is between 3.5% and 9.0%. In annualized terms, the range of portfolio returns is 11.2% to 21.5% and the range of portfolio risks is 12.1% to 31.3%.

Find a Portfolio with a Targeted Return and Targeted Risk

Given the range of risks and returns, it is possible to locate specific portfolios on the efficient frontier that have target values for return and risk using the functions `estimateFrontierByReturn` and `estimateFrontierByRisk`.

```
TargetReturn = 0.20;           % Input target annualized return and risk here.
TargetRisk = 0.15;

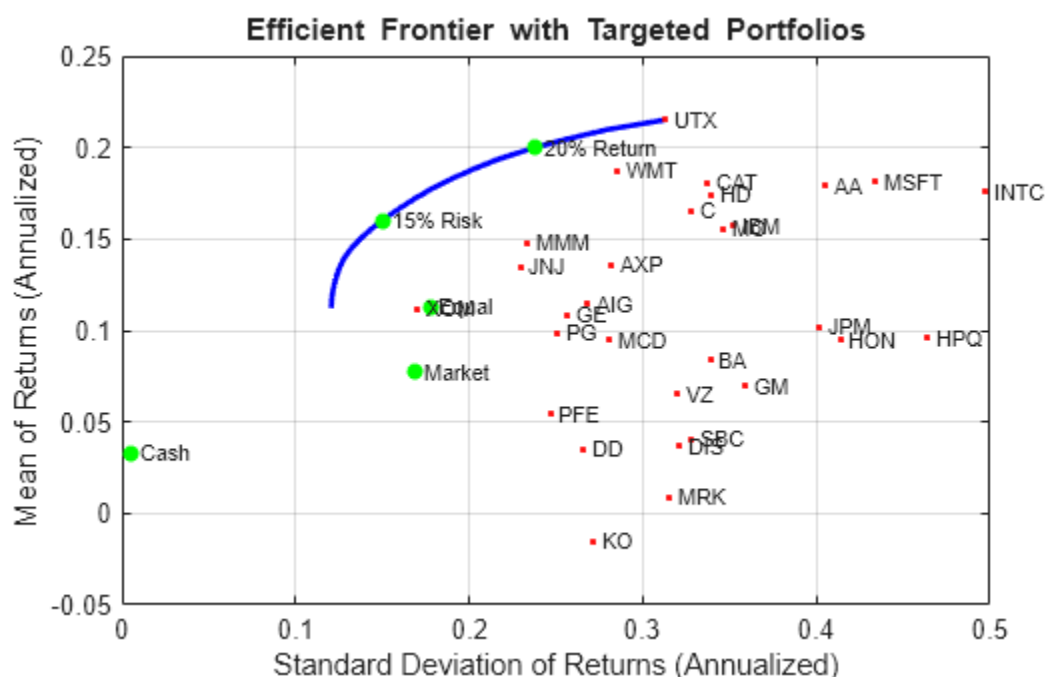
% Obtain portfolios with targeted return and risk.

awgt = estimateFrontierByReturn(p,TargetReturn/12);
[arsk,aret] = estimatePortMoments(p,awgt);

bwgt = estimateFrontierByRisk(p,TargetRisk/sqrt(12));
[brsk,bret] = estimatePortMoments(p,bwgt);

% Plot efficient frontier with targeted portfolios.

clf;
portfolioexamples_plot('Efficient Frontier with Targeted Portfolios', ...
    {'line', prsk, pret}, ...
    {'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
    {'scatter', arsk, aret, {sprintf('%g%% Return',100*TargetReturn)}}, ...
    {'scatter', brsk, bret, {sprintf('%g%% Risk',100*TargetRisk)}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, 'r'});
```



To see what these targeted portfolios look like, use the `dataset` object to set up "blotters" that contain the portfolio weights and asset names (which are obtained from the `Portfolio` object).

```
aBlotter = dataset({100*awgt(awgt > 0)}, 'Weight', 'obsnames', p.AssetList(awgt > 0));
displayPortfolio(sprintf('Portfolio with %g%% Target Return', 100*TargetReturn), aBlotter, false)
```

Portfolio with 20% Target Return

	Weight
CAT	1.1445
INTC	0.17452
MO	9.6521
MSFT	0.85862
UTX	56.918
WMT	31.253

```
bBlotter = dataset({100*bwgt(bwgt > 0)}, 'Weight', 'obsnames', p.AssetList(bwgt > 0));
```

```
displayPortfolio(sprintf('Portfolio with %g%% Target Risk', 100*TargetRisk), bBlotter, false);
```

Portfolio with 15% Target Risk

	Weight
AA	1.4503e-21
AIG	1.4891e-21
AXP	2.2204e-14
BA	2.2204e-14
C	2.2204e-14
CAT	2.2861e-21
DD	2.2204e-14
DIS	2.2204e-14
GE	1.7122e-21
GM	2.1146e-22
HD	2.2204e-14
HON	2.4868e-21
HPQ	3.3135e-21
IBM	5.4874e-21
INTC	2.2585
JNJ	9.2162
JPM	2.2204e-14
KO	2.2204e-14
MCD	2.2204e-14
MMM	16.603
MO	15.388
MRK	2.2204e-14
MSFT	4.4467
PFE	2.2204e-14
PG	4.086
SBC	2.4069e-22
UTX	10.281
VZ	2.2204e-14
WMT	25.031
XOM	12.69

Transactions Costs

The `Portfolio` object makes it possible to account for transaction costs as part of the optimization problem. Although individual costs can be set for each asset, use the scalar expansion features of the `Portfolio` object's functions to set up uniform transaction costs across all assets and compare efficient frontiers with gross versus net portfolio returns.

```
BuyCost = 0.0020;
SellCost = 0.0020;

q = setCosts(p, BuyCost, SellCost);
```

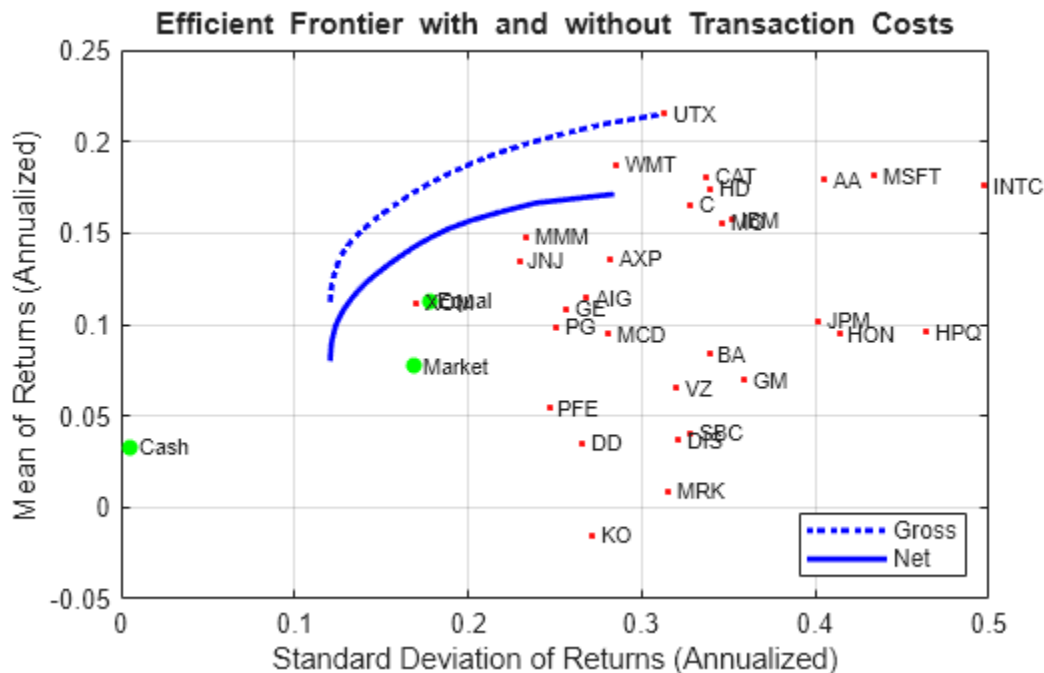
```

qwggt = estimateFrontier(q,20);
[qrsk,qret] = estimatePortMoments(q,qwggt);

% Plot efficient frontiers with gross and net returns.

clf;
portfolioexamples_plot('Efficient Frontier with and without Transaction Costs', ...
    {'line', prsk, pret, {'Gross'}, ':b'}, ...
    {'line', qrsk, qret, {'Net'}}, ...
    {'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, '.r'});

```



Turnover Constraint

In addition to transaction costs, the `Portfolio` object can handle turnover constraints. The following example demonstrates that a turnover constraint produces an efficient frontier in the neighborhood of an initial portfolio that may restrict trading. Moreover, the introduction of a turnover constraint often implies that multiple trades may be necessary to shift from an initial portfolio to an unconstrained efficient frontier. Consequently, the turnover constraint introduces a form of time diversification that can spread trades out over multiple time periods. In this example, note that the sum of purchases and sales from the `estimateFrontier` function confirms that the turnover constraint is satisfied.

```

BuyCost = 0.0020;
SellCost = 0.0020;
Turnover = 0.2;

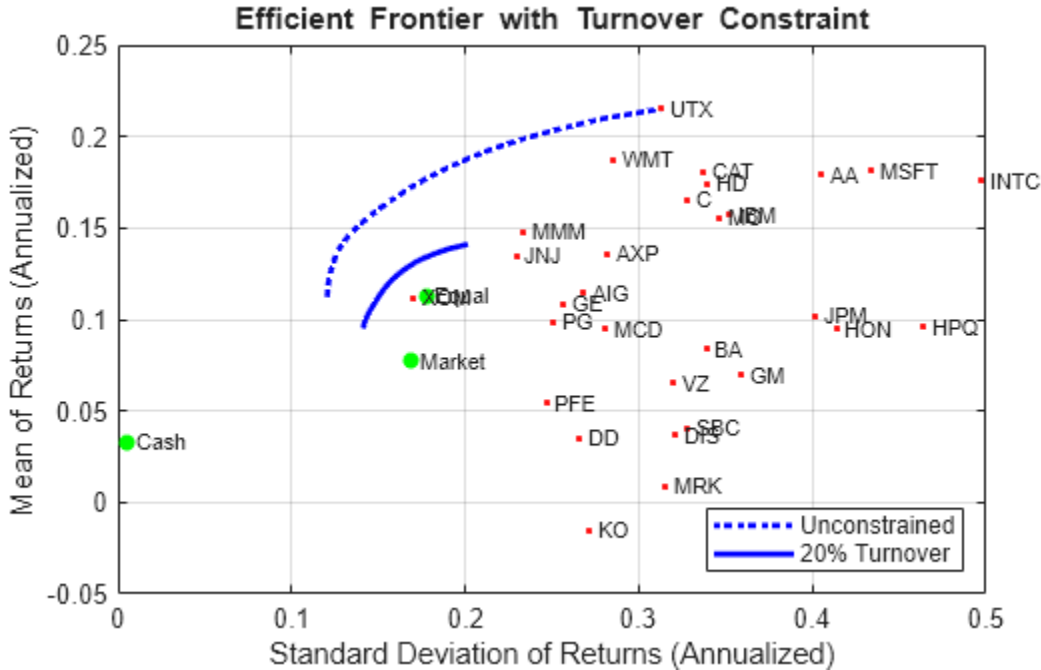
q = setCosts(p, BuyCost, SellCost);
q = setTurnover(q, Turnover);

[qwggt, qbuy, qsell] = estimateFrontier(q, 20);
[qrsk, qret] = estimatePortMoments(q, qwggt);

```

```
% Plot efficient frontier with turnover constraint.
```

```
clf;
portfolioexamples_plot('Efficient Frontier with Turnover Constraint', ...
    {'line', prsk, pret, {'Unconstrained'}, 'b'}, ...
    {'line', qrsk, qret, {sprintf('%g%% Turnover', 100*Turnover)}}, ...
    {'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, 'r'});
```



```
displaySumOfTransactions(Turnover, qbuy, qsell)
```

Sum of Purchases by Portfolio along Efficient Frontier (Max. Turnover 20%)

[illegible]

Sum of Sales by Portfolio along Efficient Frontier (Max. Turnover 20%)

[illegible]

Tracking-Error Constraint

The `Portfolio` object can handle tracking-error constraints, where tracking-error is the relative risk of a portfolio compared with a tracking portfolio. In this example, a sub-collection of nine assets forms an equally-weighted tracking portfolio. The goal is to find efficient portfolios with tracking errors that are within 5% of this tracking portfolio.

```
ii = [15, 16, 20, 21, 23, 25, 27, 29, 30]; % Indexes of assets to include in the tracking portfolio
```

```
TrackingError = 0.05/sqrt(12);
```

```
TrackingPort = zeros(30, 1);
```

```
TrackingPort(ii) = 1;
```

```
TrackingPort = (1/sum(TrackingPort))*TrackingPort;
```

```
q = setTrackingError(p,TrackingError,TrackingPort);
```

```
qwgt = estimateFrontier(q,20);
```

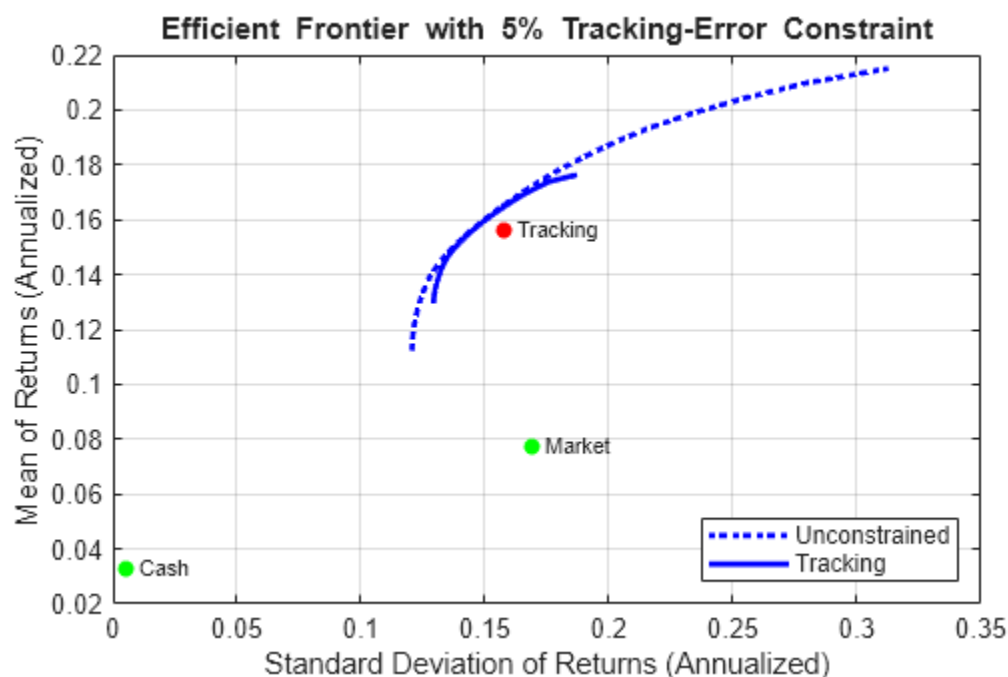


```
[qrsk,qret] = estimatePortMoments(q,qwgt);

[trsk,tret] = estimatePortMoments(q,TrackingPort);

% Plot the efficient frontier with tracking-error constraint.

clf;
portfolioexamples_plot('Efficient Frontier with 5% Tracking-Error Constraint', ...
    {'line', prsk, pret, {'Unconstrained'}, ':b'}, ...
    {'line', qrsk, qret, {'Tracking'}}, ...
    {'scatter', [mrsk, crsk], [mret, cret], {'Market', 'Cash'}}, ...
    {'scatter', trsk, tret, {'Tracking'}, 'r'});
```



Combined Turnover and Tracking-Error Constraints

This example illustrates the interactions that can occur with combined constraints. In this case, both a turnover constraint relative to an initial equal-weight portfolio and a tracking-error constraint relative to a tracking portfolio must be satisfied. The turnover constraint has a maximum of 30% turnover and the tracking-error constraint has a maximum of 5% tracking error. Note that the turnover to get from the initial portfolio to the tracking portfolio is 70% so that an upper bound of 30% turnover means that the efficient frontier will lie somewhere between the initial portfolio and the tracking portfolio.

```
Turnover = 0.3;
InitPort = (1/q.NumAssets)*ones(q.NumAssets, 1);

ii = [15, 16, 20, 21, 23, 25, 27, 29, 30]; % Indexes of assets to include in tracking portfolio.

TrackingError = 0.05/sqrt(12);
TrackingPort = zeros(30, 1);
TrackingPort(ii) = 1;
TrackingPort = (1/sum(TrackingPort))*TrackingPort;
```

```

q = setTurnover(q,Turnover,InitPort);

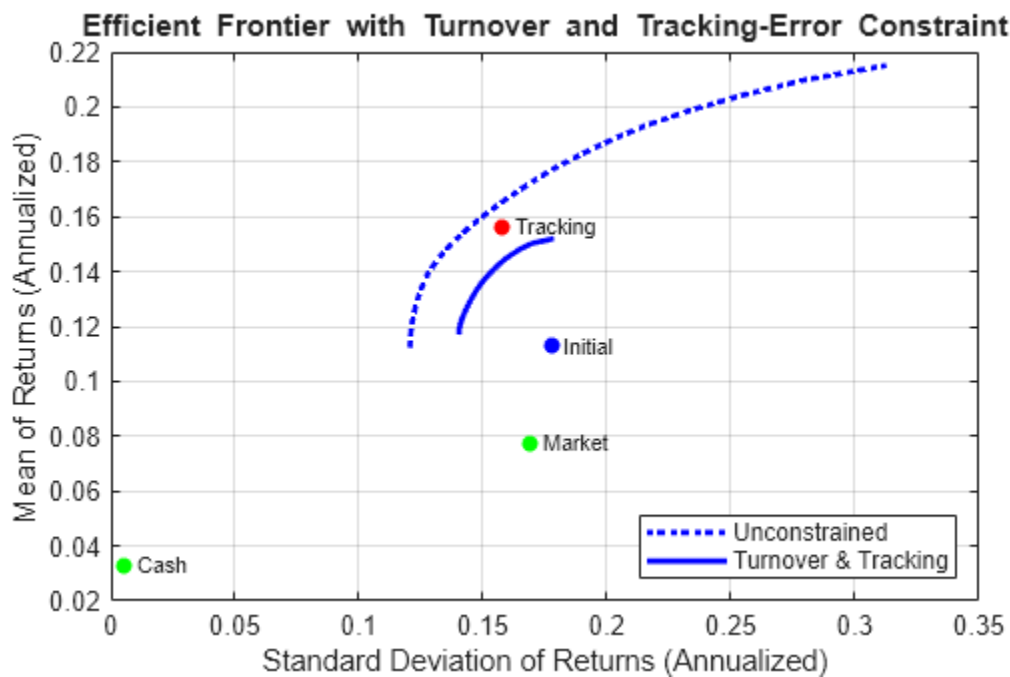
qwgt = estimateFrontier(q,20);
[qrsk,qret] = estimatePortMoments(q,qwgt);

[trsk,tret] = estimatePortMoments(q,TrackingPort);
[ersk,eret] = estimatePortMoments(q,InitPort);

% Plot the efficient frontier with combined turnover and tracking-error constraint.

clf;
portfolioexamples_plot('Efficient Frontier with Turnover and Tracking-Error Constraint', ...
    {'line', prsk, pret, {'Unconstrained'}, ':b'}, ...
    {'line', qrsk, qret, {'Turnover & Tracking'}}, ...
    {'scatter', [mrsk, crsk], [mret, cret], {'Market', 'Cash'}}, ...
    {'scatter', trsk, tret, {'Tracking'}, 'r'}, ...
    {'scatter', ersk, eret, {'Initial'}, 'b'});

```



Maximize the Sharpe Ratio

The Sharpe ratio (Sharpe 1966 on page 4-177) is a measure of return-to-risk that plays an important role in portfolio analysis. Specifically, a portfolio that maximizes the Sharpe ratio is also the tangency portfolio on the efficient frontier from the mutual fund theorem. The maximum Sharpe ratio portfolio is located on the efficient frontier with the function `estimateMaxSharpeRatio` and the `dataset` object is used to list the assets in this portfolio.

```

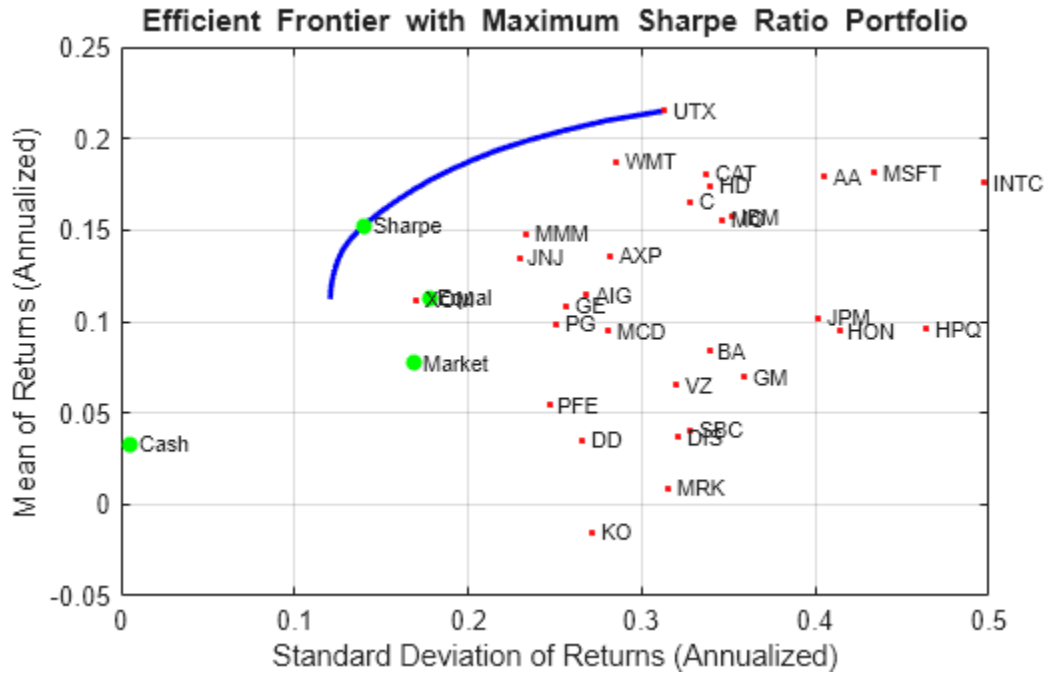
p = setInitPort(p, 0);

swgt = estimateMaxSharpeRatio(p);
[srsk,sret] = estimatePortMoments(p,swgt);

% Plot the efficient frontier with portfolio that attains maximum Sharpe ratio.

```

```
clf;
portfolioexamples_plot('Efficient Frontier with Maximum Sharpe Ratio Portfolio', ...
    {'line', prsk, pret}, ...
    {'scatter', srsk, sret, {'Sharpe'}}, ...
    {'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, '.r'});
```



```
% Set up a dataset object that contains the portfolio that maximizes the Sharpe ratio.
```

```
Blotter = dataset({100*swgt(swgt > 0)}, 'Weight', 'obsnames', AssetList(swgt > 0));
```

```
displayPortfolio('Portfolio with Maximum Sharpe Ratio', Blotter, false);
```

```
Portfolio with Maximum Sharpe Ratio
```

	Weight
AA	6.3534e-13
AIG	6.1632e-13
AXP	3.4228e-13
BA	1.8189e-13
C	6.4939e-13
CAT	9.6475e-13
DD	1.0192e-13
DIS	1.9315e-13
GE	1.8145e-12
GM	2.3744e-13
HD	5.669e-09
HON	1.0744e-13
HPQ	6.5591e-13
IBM	2.8773e-12
INTC	2.6638
JNJ	9.0044
JPM	1.7502e-13
KO	8.0048e-14
MCD	2.8352e-13

MMM	15.502
MO	13.996
MRK	1.0499e-13
MSFT	4.4777
PFE	2.3857e-13
PG	7.4588
SBC	1.2458e-13
UTX	6.0056
VZ	1.2724e-13
WMT	22.051
XOM	18.841

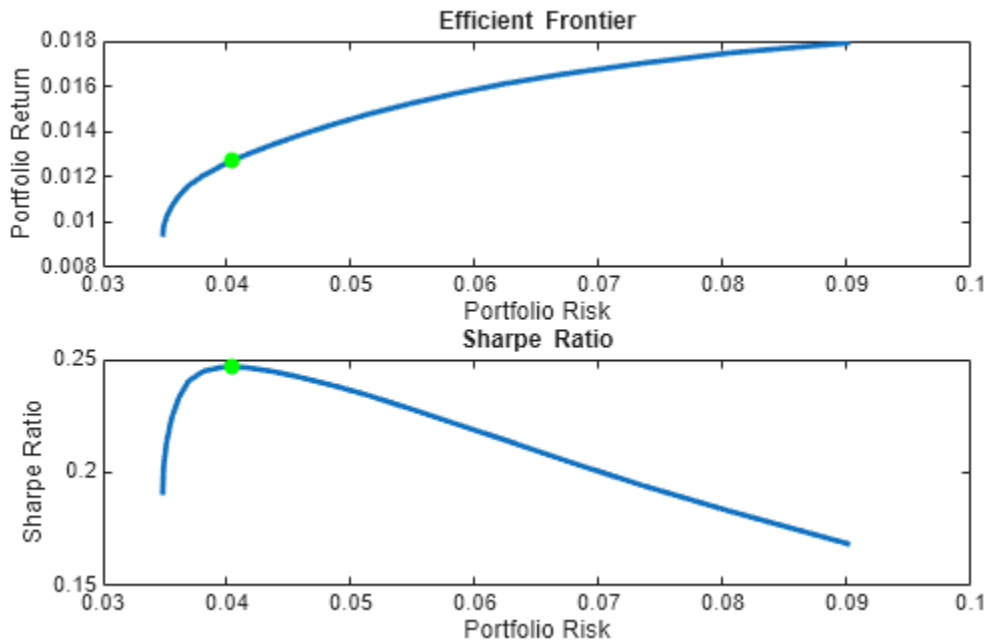
Confirm that Maximum Sharpe Ratio is a Maximum

The following plot demonstrates that this portfolio (which is located at the dot on the plots) indeed maximizes the Sharpe ratio among all portfolios on the efficient frontier.

```
psratio = (pret - p.RiskFreeRate) ./ prsk;
ssratio = (sret - p.RiskFreeRate) / srsk;

clf;
subplot(2,1,1);
plot(prsk, pret, 'LineWidth', 2);
hold on
scatter(srsk, sret, 'g', 'filled');
title('\bfEfficient Frontier');
xlabel('Portfolio Risk');
ylabel('Portfolio Return');
hold off

subplot(2,1,2);
plot(prsk, ppsratio, 'LineWidth', 2);
hold on
scatter(srsk, ssratio, 'g', 'filled');
title('\bfSharpe Ratio');
xlabel('Portfolio Risk');
ylabel('Sharpe Ratio');
hold off
```



Illustrate that Sharpe is the Tangent Portfolio

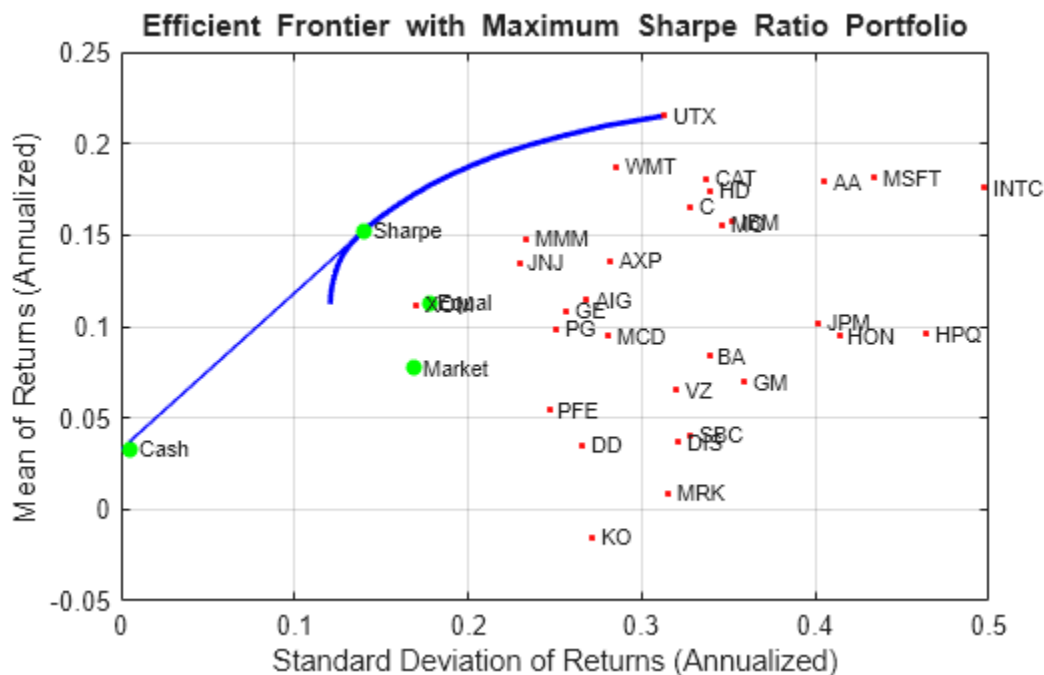
The next plot demonstrates that the portfolio that maximizes the Sharpe ratio is also a tangency portfolio (in this case, the budget constraint is opened up to permit between 0% and 100% in cash).

```
q = setBudget(p, 0, 1);

qwgt = estimateFrontier(q,20);
[qrsk,qret] = estimatePortMoments(q,qwgt);

% Plot showing that the Sharpe ratio portfolio is the tangency portfolio.

clf;
portfolioexamples_plot('Efficient Frontier with Maximum Sharpe Ratio Portfolio', ...
    {'line', prsk, pret}, ...
    {'line', qrsk, qret, [], [], 1}, ...
    {'scatter', srsk, sret, {'Sharpe'}}, ...
    {'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, '.r'});
```



Dollar-Neutral Hedge-Fund Structure

To illustrate how to use the portfolio optimization tools in hedge fund management, two popular strategies with dollar-neutral and 130-30 portfolios are examined. The dollar-neutral strategy invests equally in long and short positions such that the net portfolio position is 0. Such a portfolio is said to be "dollar-neutral."

To set up a dollar-neutral portfolio, start with the "standard" portfolio problem and set the maximum exposure in long and short positions in the variable `Exposure`. The bounds for individual asset weights are plus or minus `Exposure`. Since the net position must be dollar-neutral, the budget constraint is 0 and the initial portfolio must be 0. Finally, the one-way turnover constraints provide the necessary long and short restrictions to prevent "double-counting" of long and short positions. The blotter shows the portfolio weights for the dollar-neutral portfolio that maximizes the Sharpe ratio. The long and short positions are obtained from the buy and sell trades relative to the initial portfolio.

`Exposure = 1;`

```
q = setBounds(p, -Exposure, Exposure);
q = setBudget(q, 0, 0);
q = setOneWayTurnover(q, Exposure, Exposure, 0);
```

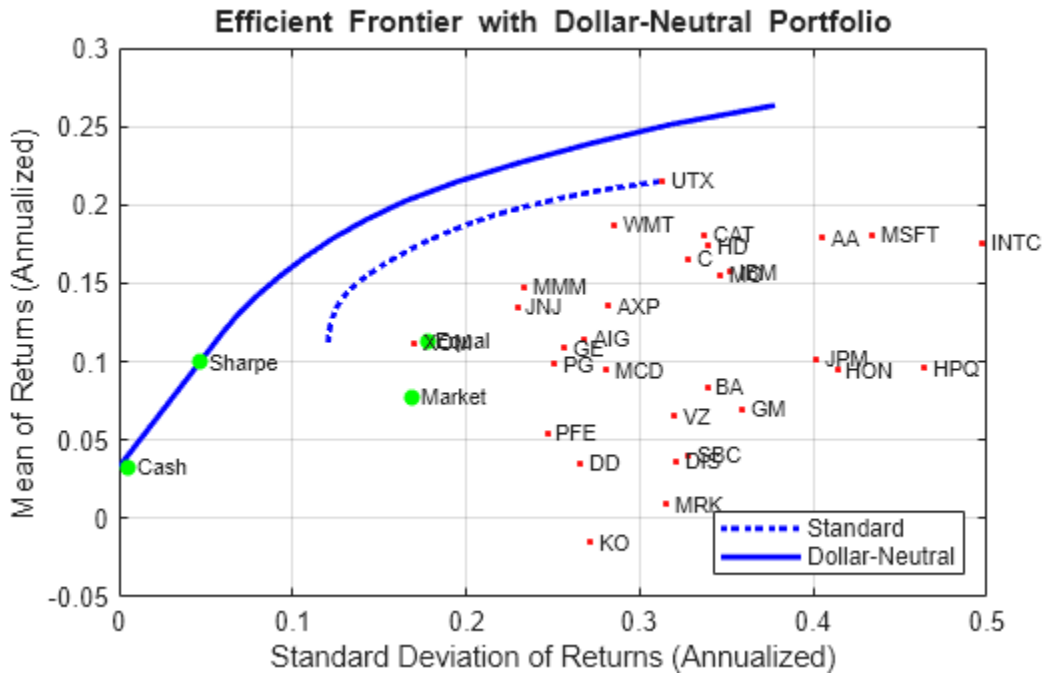
```
[qwgt,qlong,qshort] = estimateFrontier(q,20);
[qrsk,qret] = estimatePortMoments(q,qwgt);
```

```
[qswgt,qslong,qsshort] = estimateMaxSharpeRatio(q);
[qsrsk,qsrret] = estimatePortMoments(q,qswgt);
```

`% Plot the efficient frontier for a dollar-neutral fund structure with tangency portfolio.`

```
clf;
portfolioexamples_plot('Efficient Frontier with Dollar-Neutral Portfolio', ...
```

```
{'line', prsk, pret, {'Standard'}, 'b:'}, ...
{'line', qrsk, qret, {'Dollar-Neutral'}, 'b'}, ...
{'scatter', qsrsk, qsret, {'Sharpe'}}, ...
{'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
{'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, 'r'});
```



```
% Set up a dataset object that contains the portfolio that maximizes the Sharpe ratio.
```

```
Blotter = dataset({100*qswgt(abs(qswgt) > 1.0e-4), 'Weight'}, ...
    {100*qslong(abs(qswgt) > 1.0e-4), 'Long'}, ...
    {100*qsshort(abs(qswgt) > 1.0e-4), 'Short'}, ...
    'obsnames', AssetList(abs(qswgt) > 1.0e-4));
```

```
displayPortfolio('Dollar-Neutral Portfolio with Maximum Sharpe Ratio', Blotter, true, 'Dollar-Neu
```

```
Dollar-Neutral Portfolio with Maximum Sharpe Ratio
```

	Weight	Long	Short
AA	0.5088	0.5088	0
AIG	3.0394	3.0394	0
AXP	0.92797	0.92797	0
BA	-3.4952	0	3.4952
C	14.003	14.003	0
CAT	3.7261	3.7261	0
DD	-18.063	0	18.063
DIS	-4.8236	0	4.8236
GE	-3.6178	0	3.6178
GM	-3.7211	0	3.7211
HD	1.101	1.101	0
HON	-1.4349	0	1.4349
HPQ	0.09909	0.09909	0
IBM	-8.0585	0	8.0585
INTC	1.7693	1.7693	0
JNJ	1.3696	1.3696	0
JPM	-2.5271	0	2.5271

KO	-14.205	0	14.205
MCD	3.91	3.91	0
MMM	7.5995	7.5995	0
MO	4.0856	4.0856	0
MRK	3.747	3.747	0
MSFT	4.0769	4.0769	0
PFE	-9.096	0	9.096
PG	1.6493	1.6493	0
SBC	-5.2547	0	5.2547
UTX	5.7454	5.7454	0
VZ	-2.438	0	2.438
WMT	0.84844	0.84844	0
XOM	18.529	18.529	0

Confirm Dollar-Neutral Portfolio
(Net, Long, Short)
0.0000 76.7350 76.7350

130/30 Fund Structure

Finally, the turnover constraints are used to set up a 130-30 portfolio structure, which is a structure with a net long position but permits leverage with long and short positions up to a maximum amount of leverage. In the case of a 130-30 portfolio, the leverage is 30%.

To set up a 130-30 portfolio, start with the "standard" portfolio problem and set the maximum value for leverage in the variable `Leverage`. The bounds for individual asset weights range between `-Leverage` and `1 + Leverage`. Since the net position must be long, the budget constraint is 1 and, once again, the initial portfolio is 0. Finally, the one-way turnover constraints provide the necessary long and short restrictions to prevent "double-counting" of long and short positions. The blotter shows the portfolio weights for the 130-30 portfolio that maximizes the Sharpe ratio. The long and short positions are obtained from the buy and sell trades relative to the initial portfolio.

`Leverage = 0.3;`

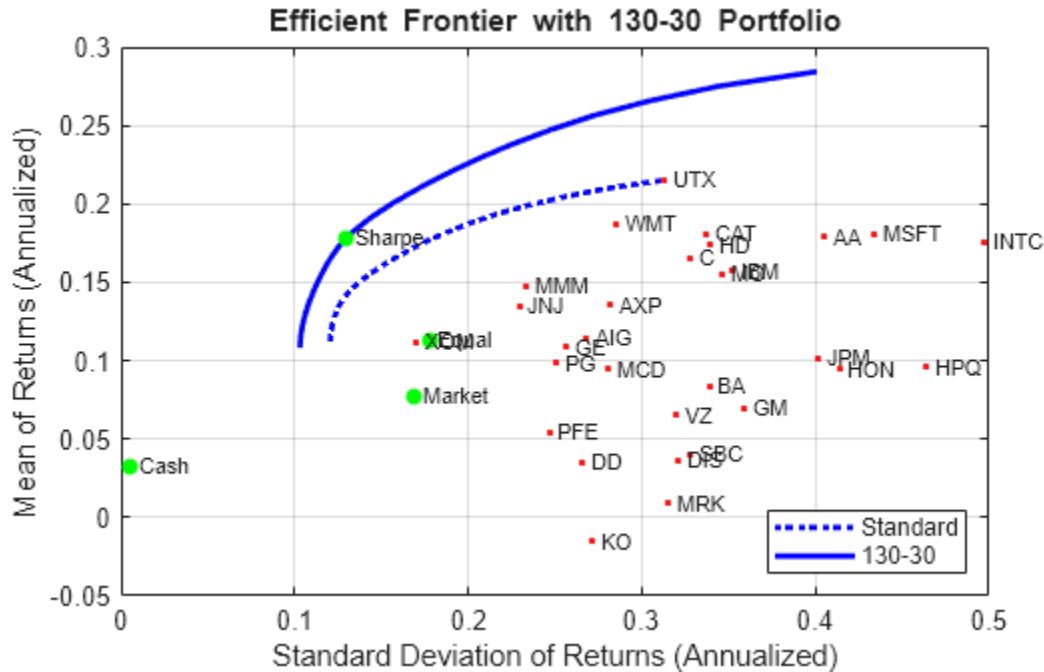
```
q = setBounds(p, -Leverage, 1 + Leverage);
q = setBudget(q, 1, 1);
q = setOneWayTurnover(q, 1 + Leverage, Leverage);
```

```
[qwgt,qbuy,qsell] = estimateFrontier(q,20);
[qrsk,qret] = estimatePortMoments(q,qwgt);
```

```
[qswgt,qslong,qsshort] = estimateMaxSharpeRatio(q);
[qsrsk,qsret] = estimatePortMoments(q,qswgt);
```

`% Plot the efficient frontier for a 130-30 fund structure with tangency portfolio.`

```
clf;
portfolioexamples_plot(sprintf('Efficient Frontier with %g-%g Portfolio', ...
    100*(1 + Leverage),100*Leverage), ...
    {'line', prsk, pret, {'Standard'}, 'b:'}, ...
    {'line', qrsk, qret, {'130-30'}, 'b'}, ...
    {'scatter', qsrsk, qsret, {'Sharpe'}}, ...
    {'scatter', [mrsk, crsk, ersk], [mret, cret, eret], {'Market', 'Cash', 'Equal'}}, ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, '.r'});
```

% Set up a dataset object that contains the portfolio that maximizes the Sharpe ratio.

```
Blotter = dataset({100*qswgt(abs(qswgt) > 1.0e-4), 'Weight'}, ...
    {100*qslong(abs(qswgt) > 1.0e-4), 'Long'}, ...
    {100*qsshort(abs(qswgt) > 1.0e-4), 'Short'}, ...
    'obsnames', AssetList(abs(qswgt) > 1.0e-4));
```

```
displayPortfolio(sprintf('%g-%g Portfolio with Maximum Sharpe Ratio', 100*(1 + Leverage), 100*Leverage), Blotter)
```

130-30 Portfolio with Maximum Sharpe Ratio

	Weight	Long	Short
DD	-9.5565	0	9.5565
HON	-6.0244	0	6.0244
INTC	4.0335	4.0335	0
JNJ	7.1234	7.1234	0
JPM	-0.44583	0	0.44583
KO	-13.646	0	13.646
MMM	20.908	20.908	0
MO	14.433	14.433	0
MSFT	4.5592	4.5592	0
PG	17.243	17.243	0
SBC	-0.32712	0	0.32712
UTX	5.3584	5.3584	0
WMT	21.018	21.018	0
XOM	35.323	35.323	0

```
Confirm 130-30 Portfolio
(Net, Long, Short)
100.0000 130.0000 30.0000
```

References

- 1 R. C. Grinold and R. N. Kahn. *Active Portfolio Management*. 2nd ed., 2000.

- 2 H. M. Markowitz. "Portfolio Selection." *Journal of Finance*. Vol. 1, No. 1, pp. 77-91, 1952.
- 3 J. Lintner. "The Valuation of Risk Assets and the Selection of Risky Investments in Stock Portfolios and Capital Budgets." *Review of Economics and Statistics*. Vol. 47, No. 1, pp. 13-37, 1965.
- 4 H. M. Markowitz. *Portfolio Selection: Efficient Diversification of Investments*. John Wiley & Sons, Inc., 1959.
- 5 W. F. Sharpe. "Mutual Fund Performance." *Journal of Business*. Vol. 39, No. 1, Part 2, pp. 119-138, 1966.
- 6 J. Tobin. "Liquidity Preference as Behavior Towards Risk." *Review of Economic Studies*. Vol. 25, No.1, pp. 65-86, 1958.
- 7 J. L. Treynor and F. Black. "How to Use Security Analysis to Improve Portfolio Selection." *Journal of Business*. Vol. 46, No. 1, pp. 68-86, 1973.

Local Functions

```
function displaySumOfTransactions(Turnover, qbuy, qsell)
fprintf('Sum of Purchases by Portfolio along Efficient Frontier (Max. Turnover %g%%)\n', ...
    100*Turnover);
fprintf('%0.4f ', 100*sum(qbuy)), sprintf('\n\n');
fprintf('\n')
fprintf('Sum of Sales by Portfolio along Efficient Frontier (Max. Turnover %g%%)\n', ...
    100*Turnover);
fprintf('%0.4f ', 100*sum(qsell));
end

function displayPortfolio(Description, Blotter, LongShortFlag, portfolioType)
fprintf('%s\n', Description);
disp(Blotter);
if (LongShortFlag)
    fprintf('Confirm %s Portfolio\n', portfolioType);
    fprintf(' (Net, Long, Short)\n');
    fprintf('%0.4f ', [ sum(Blotter.Weight), sum(Blotter.Long), sum(Blotter.Short) ]);
end
end
```

See Also

Portfolio | setBounds | addGroups | setAssetMoments | estimateAssetMoments |
estimateBounds | plotFrontier | estimateFrontierLimits | estimateFrontierByRisk |
estimatePortRisk

Related Examples

- "Creating the Portfolio Object" on page 4-25
- "Working with Portfolio Constraints Using Defaults" on page 4-58
- "Validate the Portfolio Problem for Portfolio Object" on page 4-94
- "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
- "Estimate Efficient Frontiers for Portfolio Object" on page 4-122
- "Postprocessing Results to Set Up Tradable Portfolios" on page 4-130
- "Portfolio Optimization with Semicontinuous and Cardinality Constraints" on page 4-190
- "Black-Litterman Portfolio Optimization Using Financial Toolbox" on page 4-222

- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Analysis with Turnover Constraints” on page 4-211
- “Leverage in Portfolio Optimization with a Risk-Free Asset” on page 4-217
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Asset Allocation Case Study

This example shows how to set up a basic asset allocation problem that uses mean-variance portfolio optimization with a `Portfolio` object to estimate efficient portfolios.

Step 1. Defining the portfolio problem.

Suppose that you want to manage an asset allocation fund with four asset classes: bonds, large-cap equities, small-cap equities, and emerging equities. The fund is long-only with no borrowing or leverage, should have no more than 85% of the portfolio in equities, and no more than 35% of the portfolio in emerging equities. The cost to trade the first three assets is 10 basis points annualized and the cost to trade emerging equities is four times higher. Finally, you want to ensure that average turnover is no more than 15%. To solve this problem, you will set up a basic mean-variance portfolio optimization problem and then slowly introduce the various constraints on the problem to get to a solution.

To set up the portfolio optimization problem, start with basic definitions of known quantities associated with the structure of this problem. Each asset class is assumed to have a tradeable asset with a real-time price. Such assets can be, for example, exchange-traded funds (ETFs). The initial portfolio with holdings in each asset that has a total of \$7.5 million along with an additional cash position of \$60,000. These basic quantities and the costs to trade are set up in the following variables with asset names in the cell array `Asset`, current prices in the vector `Price`, current portfolio holdings in the vector `Holding`, and transaction costs in the vector `UnitCost`.

To analyze this portfolio, you can set up a blotter in a `table` object to help track prices, holdings, weights, and so forth. In particular, you can compute the initial portfolio weights and maintain them in a new blotter field called `InitPort`.

```
Asset = { 'Bonds', 'Large-Cap Equities', 'Small-Cap Equities', 'Emerging Equities' };
Price = [ 52.4; 122.7; 35.2; 46.9 ];
Holding = [ 42938; 24449; 42612; 15991 ];
UnitCost = [ 0.001; 0.001; 0.001; 0.004 ];
```

```
Blotter = table('RowNames', Asset);
Blotter.Price = Price;
Blotter.InitHolding = Holding;
Wealth = sum(Blotter.Price .* Blotter.InitHolding);
Blotter.InitPort = (1/Wealth)*(Blotter.Price .* Blotter.InitHolding);
Blotter.UnitCost = UnitCost;
Blotter
```

Blotter=4×4 table

	Price	InitHolding	InitPort	UnitCost
Bonds	52.4	42938	0.3	0.001
Large-Cap Equities	122.7	24449	0.4	0.001
Small-Cap Equities	35.2	42612	0.2	0.001
Emerging Equities	46.9	15991	0.1	0.004

Step 2. Simulating asset prices.

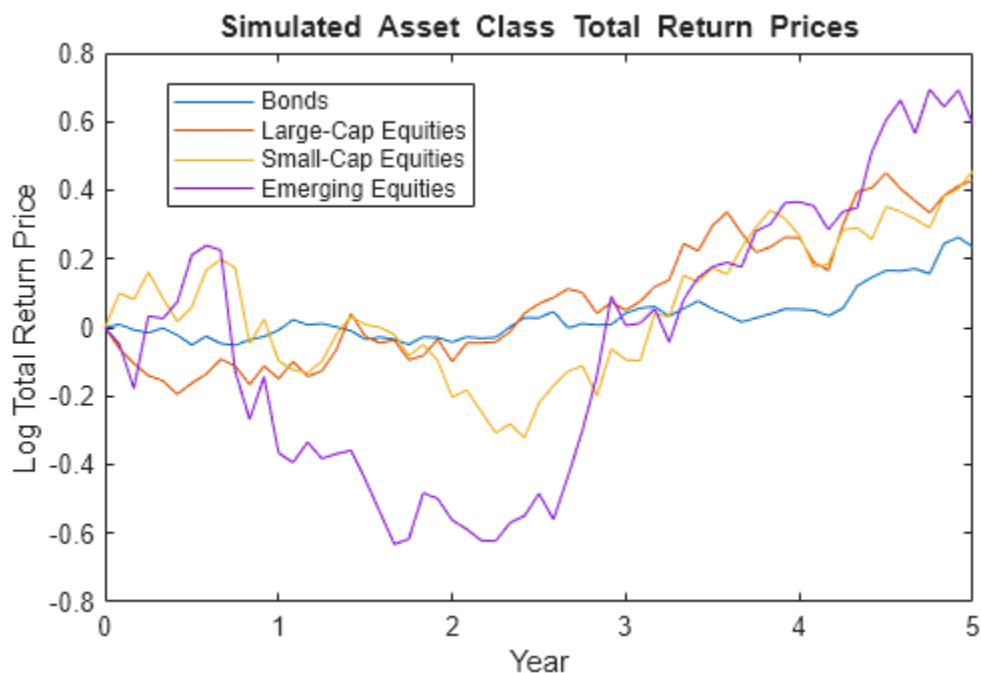
Since this is a hypothetical example, to simulate asset prices from a given mean and covariance of annual asset total returns for the asset classes, the `portsim` function is used to create asset returns

with the desired mean and covariance. Specifically, `portsim` is used to simulate five years of monthly total returns and then plotted to show the log of the simulated total return prices.

The mean and covariance of annual asset total returns are maintained in the variables `AssetMean` and `AssetCovar`. The simulated asset total return prices (which are compounded total returns) are maintained in the variable `Y`. All initial asset total return prices are normalized to 1 in this example.

```
AssetMean = [ 0.05; 0.1; 0.12; 0.18 ];
AssetCovar = [ 0.0064 0.00408 0.00192 0;
               0.00408 0.0289 0.0204 0.0119;
               0.00192 0.0204 0.0576 0.0336;
               0 0.0119 0.0336 0.1225 ];

X = portsim(AssetMean'/12, AssetCovar/12, 60); % Monthly total returns for 5 years (60 months)
[Y, T] = ret2tick(X, [], 1/12); % form total return prices.
plot(T, log(Y));
title('\bfSimulated Asset Class Total Return Prices');
xlabel('Year');
ylabel('Log Total Return Price');
legend(Asset, 'Location', 'best');
```



Step 3. Setting up the Portfolio object.

To explore portfolios on the efficient frontier, set up a `Portfolio` object using these specifications:

- Portfolio weights are nonnegative and sum to 1.
- Equity allocation is no more than 85% of the portfolio.
- Emerging equity is no more than 35% of the portfolio.

These specifications are incorporated into the `Portfolio` object `p` in the following sequence of using functions that starts with using the `Portfolio` object.

- 1 The specification of the initial portfolio from `Blotter` gives the number of assets in your universe so you do not need to specify the `NumAssets` property directly. Next, set up default constraints (long-only with a budget constraint). In addition, set up the group constraint that imposes an upper bound on equities in the portfolio (equities are identified in the group matrix with 1's) and the upper bound constraint on emerging equities. Although you could have set the upper bound on emerging equities using the `setBounds` function, notice how the `addGroups` function is used to set up this constraint.
- 2 To have a fully specified mean-variance portfolio optimization problem, you must specify the mean and covariance of asset returns. Since starting with these moments in the variables `AssetMean` and `AssetCovar`, you can use the `setAssetMoments` function to enter these variables into your `Portfolio` object (remember that you are assuming that your raw data are monthly returns which is why you divide your annual input moments by 12 to get monthly returns).
- 3 Use the total return prices with the `estimateAssetMoments` function with a specification that your data in `Y` are prices, and not returns, to estimate asset return moments for your `Portfolio` object.
- 4 Although the returns in your `Portfolio` object are in units of monthly returns, and since subsequent costs are annualized, it is convenient to specify them as annualized total returns with this direct transformation of the `AssetMean` and `AssetCovar` properties of your `Portfolio` object `p`.
- 5 Display the `Portfolio` object `p`.

```
p = Portfolio('Name', 'Asset Allocation Portfolio', ...
'AssetList', Asset, 'InitPort', Blotter.InitPort);
```

```
p = setDefaultConstraints(p);
p = setGroups(p, [ 0, 1, 1, 1 ], [], 0.85);
p = addGroups(p, [ 0, 0, 0, 1 ], [], 0.35);
```

```
p = setAssetMoments(p, AssetMean/12, AssetCovar/12);
p = estimateAssetMoments(p, Y, 'DataFormat', 'Prices');
```

```
p.AssetMean = 12*p.AssetMean;
p.AssetCovar = 12*p.AssetCovar;
```

```
display(p)
```

```
p =
Portfolio with properties:

    BuyCost: []
    SellCost: []
RiskFreeRate: []
    AssetMean: [4x1 double]
    AssetCovar: [4x4 double]
TrackingError: []
TrackingPort: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
        Name: 'Asset Allocation Portfolio'
    NumAssets: 4
    AssetList: {'Bonds' 'Large-Cap Equities' 'Small-Cap Equities' 'Emerging E
    InitPort: [4x1 double]
    AInequality: []
```

```

bInequality: []
AEquality: []
bEquality: []
LowerBound: [4×1 double]
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: [2×4 double]
LowerGroup: []
UpperGroup: [2×1 double]
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: [4×1 categorical]

```

Step 4. Validate the portfolio problem.

An important step in portfolio optimization is to validate that the portfolio problem is feasible and the main test is to ensure that the set of portfolios is nonempty and bounded. Use the `estimateBounds` function to determine the bounds for the portfolio set. In this case, since both `lb` and `ub` are finite, the set is bounded.

```

[lb, ub] = estimateBounds(p);
display([lb, ub])

```

```

0.1500    1.0000
         0    0.8500
         0    0.8500
         0    0.3500

```

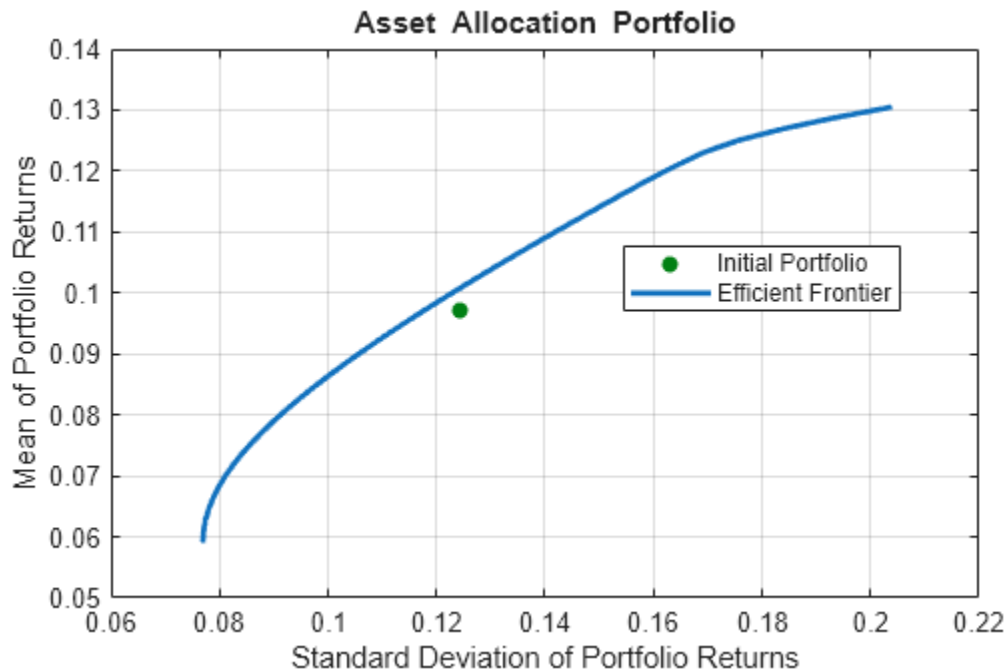
Step 5. Plotting the efficient frontier.

Given the constructed `Portfolio` object, use the `plotFrontier` function to view the efficient frontier. Instead of using the default of 10 portfolios along the frontier, you can display the frontier with 40 portfolios. Notice gross efficient portfolio returns fall between approximately 6% and 16% per years.

```

plotFrontier(p, 40)

```



Step 6. Evaluating gross vs. net portfolio returns.

The Portfolio object `p` does not include transaction costs so that the portfolio optimization problem specified in `p` uses gross portfolio return as the return proxy. To handle net returns, create a second Portfolio object `q` that includes transaction costs.

```
q = setCosts(p, UnitCost, UnitCost);
display(q)
```

```
q =
Portfolio with properties:

    BuyCost: [4×1 double]
    SellCost: [4×1 double]
    RiskFreeRate: []
    AssetMean: [4×1 double]
    AssetCovar: [4×4 double]
    TrackingError: []
    TrackingPort: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
    Name: 'Asset Allocation Portfolio'
    NumAssets: 4
    AssetList: {'Bonds' 'Large-Cap Equities' 'Small-Cap Equities' 'Emerging I
    InitPort: [4×1 double]
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
```



```

UpperBudget: 1
GroupMatrix: [2×4 double]
LowerGroup: []
UpperGroup: [2×1 double]
    GroupA: []
    GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: [4×1 categorical]

```

Step 7. Analyzing descriptive properties of the Portfolio structures.

To be more concrete about the ranges of efficient portfolio returns and risks, use the `estimateFrontierLimits` function to obtain portfolios at the endpoints of the efficient frontier. Given these portfolios, compute their moments using the `estimatePortMoments` function. The following code generates a table that lists the risk and return of the initial portfolio as well as the gross and net moments of portfolio returns for the portfolios at the endpoints of the efficient frontier:

```

[prsk0, pret0] = estimatePortMoments(p, p.InitPort);

pret = estimatePortReturn(p, p.estimateFrontierLimits);
qret = estimatePortReturn(q, q.estimateFrontierLimits);

displayReturns(pret0, pret, qret)

Annualized Portfolio Returns ...

```

	Gross	Net
Initial Portfolio Return	9.70 %	9.70 %
Minimum Efficient Portfolio Return	5.90 %	5.77 %
Maximum Efficient Portfolio Return	13.05 %	12.86 %

The results show that the cost to trade ranges from 14 to 19 basis points to get from the current portfolio to the efficient portfolios at the endpoints of the efficient frontier (these costs are the difference between gross and net portfolio returns.) In addition, notice that the maximum efficient portfolio return (13%) is less than the maximum asset return (18%) due to the constraints on equity allocations.

Step 8. Obtaining a Portfolio at the specified return level on the efficient frontier.

A common approach to select efficient portfolios is to pick a portfolio that has a desired fraction of the range of expected portfolio returns. To obtain the portfolio that is 30% of the range from the minimum to maximum return on the efficient frontier, obtain the range of net returns in `qret` using the `Portfolio` object `q` and interpolate to obtain a 30% level with the `interp1` function to obtain a portfolio `qwgt`.

```

Level = 0.3;

qret = estimatePortReturn(q, q.estimateFrontierLimits);
qwgt = estimateFrontierByReturn(q, interp1([0, 1], qret, Level));
[qrsk, qret] = estimatePortMoments(q, qwgt);

displayReturnLevel(Level, qret, qrsk);

```

```
Portfolio at 30% return level on efficient frontier ...
      Return      Risk
      7.90       9.09
```

```
display(qwgt)
```

```
qwgt = 4×1
```

```
0.6252
0.1856
0.0695
0.1198
```

The target portfolio that is 30% of the range from minimum to maximum net returns has a return of 7.9% and a risk of 9.1%.

Step 9. Obtaining a Portfolio at the specified risk levels on the efficient frontier.

Although you could accept this result, suppose that you want to target values for portfolio risk. Specifically, suppose that you have a conservative target risk of 10%, a moderate target risk of 15%, and an aggressive target risk of 20% and you want to obtain portfolios that satisfy each risk target. Use the `estimateFrontierByRisk` function to obtain targeted risks specified in the variable `TargetRisk`. The resultant three efficient portfolios are obtained in `qwgt`.

```
TargetRisk = [ 0.10; 0.15; 0.20 ];
qwgt = estimateFrontierByRisk(q, TargetRisk);
display(qwgt)
```

```
qwgt = 4×3
```

```
0.5407    0.2020    0.1500
0.2332    0.4000    0.0318
0.0788    0.1280    0.4682
0.1474    0.2700    0.3500
```

Use the `estimatePortRisk` function to compute the portfolio risks for the three portfolios to confirm that the target risks have been attained:

```
display(estimatePortRisk(q, qwgt))
```

```
0.1000
0.1500
0.2000
```

Suppose that you want to shift from the current portfolio to the moderate portfolio. You can estimate the purchases and sales to get to this portfolio:

```
[qwgt, qbuy, qsell] = estimateFrontierByRisk(q, 0.15);
```

If you average the purchases and sales for this portfolio, you can see that the average turnover is 17%, which is greater than the target of 15%:

```
disp(sum(qbuy + qsell)/2)
```

```
0.1700
```

Since you also want to ensure that average turnover is no more than 15%, you can add the average turnover constraint to the `Portfolio` object using `setTurnover`:

```
q = setTurnover(q, 0.15);
[qwgt, qbuy, qsell] = estimateFrontierByRisk(q, 0.15);
```

You can enter the estimated efficient portfolio with purchases and sales into the `Blotter`:

```
qbuy(abs(qbuy) < 1.0e-5) = 0;
qsell(abs(qsell) < 1.0e-5) = 0; % Zero out near 0 trade weights.
```

```
Blotter.Port = qwgt;
Blotter.Buy = qbuy;
Blotter.Sell = qsell;
```

```
display(Blotter)
```

Blotter=4×7 table

	Price	InitHolding	InitPort	UnitCost	Port	Buy	Sell
Bonds	52.4	42938	0.3	0.001	0.18787	0	0.18787
Large-Cap Equities	122.7	24449	0.4	0.001	0.4	0	0.4
Small-Cap Equities	35.2	42612	0.2	0.001	0.16213	0	0.16213
Emerging Equities	46.9	15991	0.1	0.004	0.25	0.15	0.1

The `Buy` and `Sell` elements of the `Blotter` are changes in portfolio weights that must be converted into changes in portfolio holdings to determine the trades. Since you are working with net portfolio returns, you must first compute the cost to trade from your initial portfolio to the new portfolio. This is accomplished as follows:

```
TotalCost = Wealth * sum(Blotter.UnitCost .* (Blotter.Buy + Blotter.Sell))
```

```
TotalCost =
5.6248e+03
```

The cost to trade is \$5,625, so that, in general, you would have to adjust your initial wealth accordingly before setting up your new portfolio weights. However, to keep the analysis simple, note that you have sufficient cash (\$60,000) set aside to pay the trading costs and that you will not touch the cash position to build up any positions in your portfolio. Thus, you can populate your blotter with the new portfolio holdings and the trades to get to the new portfolio without making any changes in your total invested wealth. First, compute the portfolio holding:

```
Blotter.Holding = Wealth * (Blotter.Port ./ Blotter.Price);
```

Compute number of shares to `Buy` and `Sell` in your `Blotter`:

```
Blotter.BuyShare = Wealth * (Blotter.Buy ./ Blotter.Price);
Blotter.SellShare = Wealth * (Blotter.Sell ./ Blotter.Price);
```

Notice how you used an *ad hoc* truncation rule to obtain unit numbers of shares to buy and sell. Clean up the `Blotter` by removing the unit costs and the buy and sell portfolio weights:

```
Blotter.Buy = [];
Blotter.Sell = [];
Blotter.UnitCost = [];
```

Step 10. Displaying the final results.

The final result is a blotter that contains proposed trades to get from your current portfolio to a moderate-risk portfolio. To make the trade, you would need to sell 16,049 shares of your bond asset and 8,069 shares of your small-cap equity asset and would need to purchase 23,986 shares of your emerging equities asset.

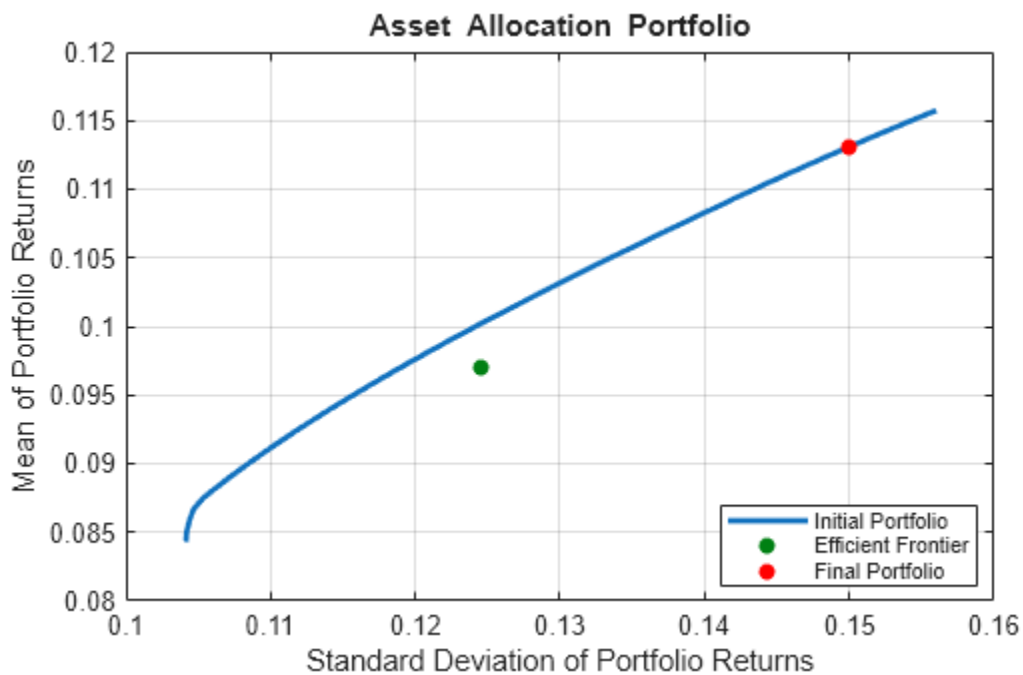
```
display(Blotter)
```

Blotter=4×7 table

	Price	InitHolding	InitPort	Port	Holding	BuyShare
Bonds	52.4	42938	0.3	0.18787	26889	0
Large-Cap Equities	122.7	24449	0.4	0.4	24449	0
Small-Cap Equities	35.2	42612	0.2	0.16213	34543	0
Emerging Equities	46.9	15991	0.1	0.25	39977	23986

The final plot uses the `plotFrontier` function to display the efficient frontier and the initial portfolio for the fully specified portfolio optimization problem. It also adds the location of the moderate-risk or final portfolio on the efficient frontier.

```
plotFrontier(q, 40);
hold on
scatter(estimatePortRisk(q, qwgt), estimatePortReturn(q, qwgt), 'filled', 'r');
h = legend('Initial Portfolio', 'Efficient Frontier', 'Final Portfolio', 'location', 'best');
set(h, 'FontSize', 8);
hold off
```

**Local Functions**

```
function displayReturns(pret0, pret, qret)
fprintf('Annualized Portfolio Returns ...\\n');
```

```

fprintf('                                %6s    %6s\n','Gross','Net');
fprintf('Initial Portfolio Return          %6.2f %%    %6.2f %%\n',100*pret0,100*pret0);
fprintf('Minimum Efficient Portfolio Return %6.2f %%    %6.2f %%\n',100*pret(1),100*qret(1));
fprintf('Maximum Efficient Portfolio Return %6.2f %%    %6.2f %%\n',100*pret(2),100*qret(2));
end

function displayReturnLevel(Level, qret, qrsk)
fprintf('Portfolio at %g%% return level on efficient frontier ...\n',100*Level);
fprintf('%10s %10s\n','Return','Risk');
fprintf('%10.2f %10.2f\n',100*qret,100*qrsk);
end

```

See Also

Portfolio | setBounds | addGroups | setAssetMoments | estimateAssetMoments |
 estimateBounds | plotFrontier | estimateFrontierLimits | estimateFrontierByRisk |
 estimatePortRisk

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Analysis with Turnover Constraints” on page 4-211
- “Leverage in Portfolio Optimization with a Risk-Free Asset” on page 4-217
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Portfolio Optimization with Semicontinuous and Cardinality Constraints

This example shows how to use a `Portfolio` object to directly handle semicontinuous and cardinality constraints when performing portfolio optimization. Portfolio optimization finds the asset allocation that maximizes the return or minimizes the risk, subject to a set of investment constraints. The `Portfolio` class in Financial Toolbox™ is designed and implemented based on the Markowitz Mean-Variance Optimization framework. The Mean-Variance Optimization framework handles problems where the return is the expected portfolio return, and the risk is the variance of portfolio returns. Using the `Portfolio` class, you can minimize the risk on the efficient frontier (EF), maximize the return on the EF, maximize the return for a given risk, and minimize the risk for a given return. You can also use `PortfolioCVaR` or `PortfolioMAD` classes in Financial Toolbox™ to specify semicontinuous and cardinality constraints. Such optimization problems integrate with constraints such as group, linear inequality, turnover, and tracking error constraints. These constraints are formulated as nonlinear programming (NLP) problems with continuous variables represented as the asset weights x_i .

Semicontinuous and cardinality constraints are two other common categories of portfolio constraints that are formulated mathematically by adding the binary variables v_i .

- A *semicontinuous constraint* confines the allocation of an asset. For example, you can use this constraint to confine the allocated weight of an allocated asset to between 5% and 50%. By using this constraint, you can avoid very small or large positions to minimize the churns and operational costs. To mathematically formulate this type of constraint, a binary variable v_i is needed, where v_i is 0 or 1. The value 0 indicates that asset i is not allocated and the value 1 indicates that asset i is allocated. The mathematical form is $lb * v_i \leq x_i \leq ub * v_i$, where v_i is 0 or 1. Specify this type of constraint as a 'Conditional' `BoundType` in the `Portfolio` class using the `setBounds` function.
- A *cardinality constraint* limits the number of assets in the optimal allocation. For example, for a portfolio with a universe of 100 assets, you can specify an optimal portfolio allocation between 20 and 40 assets. This capability helps limit the number of positions, and thus reduce operational costs. To mathematically formulate this type of constraint, binary variables represented as v_i are needed, where v_i is 0 or 1. The value 0 indicates that asset i is not allocated and the value 1 indicates that asset i is allocated. The mathematical form is $MinNumAssets \leq \sum_1^{NumAssets} v_i \leq MaxNumAssets$, where v_i is 0 or 1. Specify this type of constraint by setting the 'MinNumAssets' and 'MaxNumAssets' constraints in the `Portfolio` class using the `setMinMaxNumAssets` function.

For more information on semicontinuous and cardinality constraints, see “Algorithms” on page 15-1531.

When semicontinuous and cardinality constraints are used for portfolio optimization, this leads to mixed integer nonlinear programming problems (MINLP). The `Portfolio` class allows you to configure these two constraints, specifically, semicontinuous constraints using `setBounds` with 'Conditional' `BoundType`, and cardinality constraints using `setMinMaxNumAssets`. The `Portfolio` class automatically formulates the mathematical problems and validates the specified constraints. The `Portfolio` class also provides built-in MINLP solvers and flexible solver options for you to tune the solver performance using the `setSolverMINLP` function.

This example demonstrates a `Portfolio` object with semicontinuous and cardinality constraints and uses the `BlueChipStockMoments` dataset, which has a universe of 30 assets.

```
load BlueChipStockMoments
numAssets = numel(AssetList)

numAssets = 30
```

Limit the Minimum Weight for Each Allocated Asset

Create a fully invested portfolio with only long positions: $x_i \geq 0$ and $\sum(x_i) = 1$. These are configured with `setDefaultConstraints`.

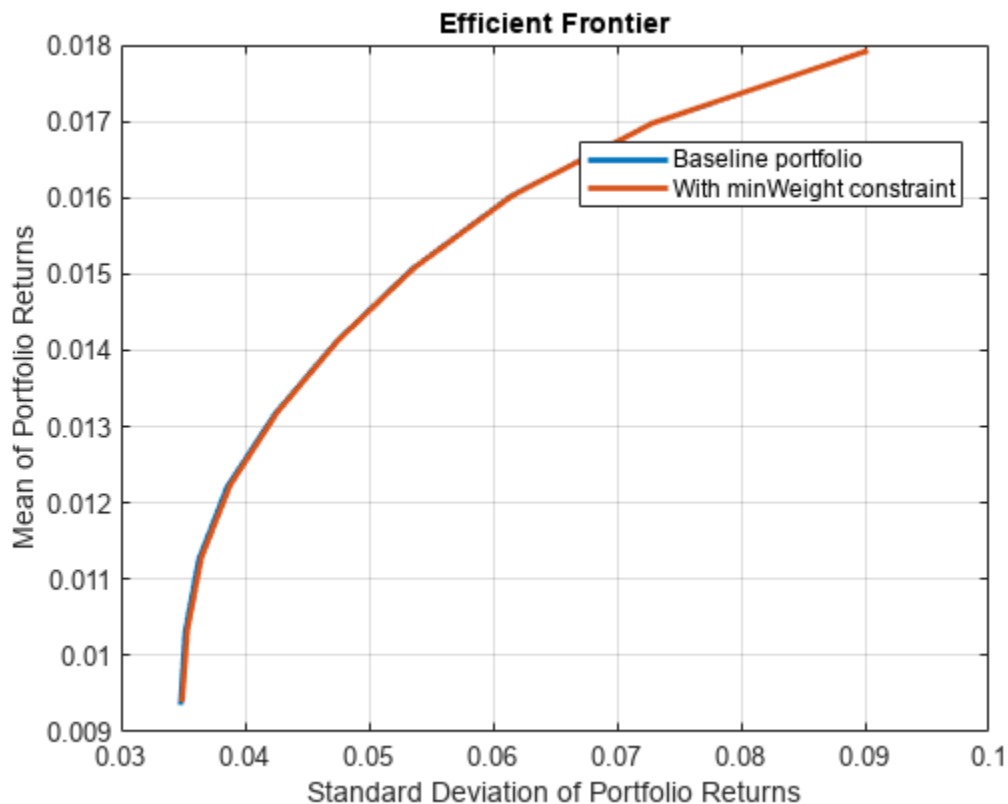
```
p = Portfolio('AssetList', AssetList, 'AssetCovar', AssetCovar, 'AssetMean', AssetMean);
p = setDefaultConstraints(p);
```

Suppose that you want to avoid very small positions to minimize the churn and operational costs. Add another constraint to confine the allocated positions to be no less than 5%, by setting the constraints $x_i = 0$ or $x_i \geq 0.05$ using `setBounds` with a 'Conditional' `BoundType`.

```
pWithMinWeight = setBounds(p, 0.05, 'BoundType', 'Conditional');
```

Plot the efficient frontiers for both `Portfolio` objects.

```
wgt = estimateFrontier(p);
wgtWithMinWeight = estimateFrontier(pWithMinWeight);
figure(1);
plotFrontier(p, wgt); hold on;
plotFrontier(pWithMinWeight, wgtWithMinWeight); hold off;
legend('Baseline portfolio', 'With minWeight constraint', 'location', 'best');
```



The figure shows that the two `Portfolio` objects have almost identical efficient frontiers. However, the one with the minimum weight requirement is more practical, since it prevents the close-to-zero positions.

Check the optimal weights for the portfolio with default constraints to see how many assets are below the 5% limit for each optimal allocation.

```
toler = eps;
sum(wgt>toler & wgt<0.05)
```

```
ans = 1×10
```

```
5    7    5    4    2    3    4    2    0    0
```

Use `estimateFrontierByReturn` to investigate the portfolio compositions for a target return on the frontier for both cases.

```
targetRetn = 0.011;
pwgt = estimateFrontierByReturn(p, targetRetn);
pwgtWithMinWeight = estimateFrontierByReturn(pWithMinWeight, targetRetn);
```

Plot the composition of the two `Portfolio` objects for the universe of 30 assets.

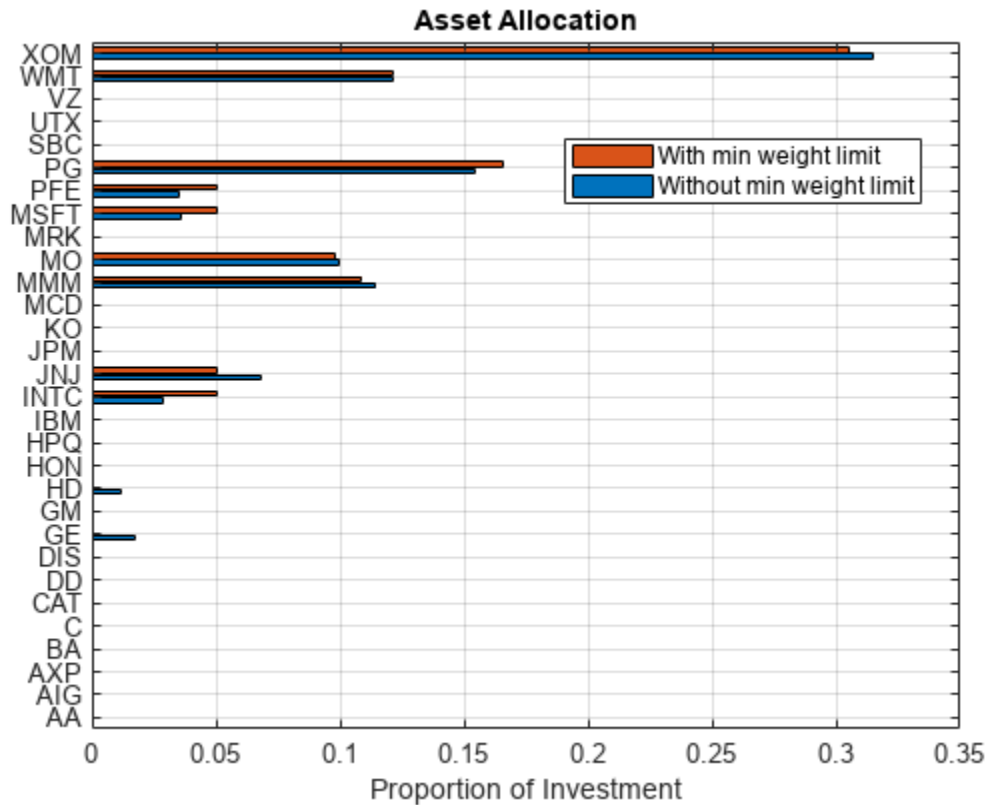
```
figure(2);
barh([pwgt, pwgtWithMinWeight]);
grid on
xlabel('Proportion of Investment')
```



```

yticks(1:p.NumAssets);
yticklabels(p.AssetList);
title('Asset Allocation');
legend('Without min weight limit', 'With min weight limit', 'location', 'best');

```



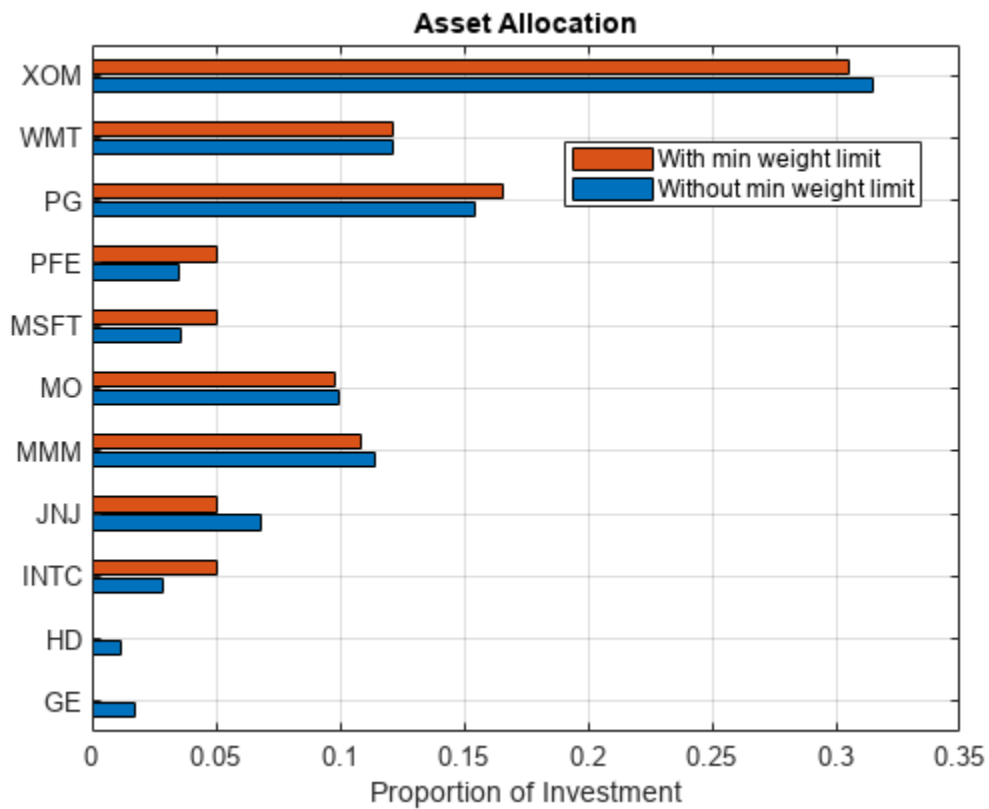
Show only the allocated assets.

```

idx = (pwgt>toler) | (pwgtWithMinWeight>toler);

barh([pwgt(idx), pwgtWithMinWeight(idx)]);
grid on
xlabel('Proportion of Investment')
yticks(1:sum(idx));
yticklabels(p.AssetList(idx));
title('Asset Allocation');
legend('Without min weight limit', 'With min weight limit', 'location', 'best');

```



Limit the Maximum Number of Assets to Allocate

Use `setMinMaxNumAssets` to set the maximum number of allocated assets for the `Portfolio` object. Suppose that you want no more than eight assets invested in the optimal portfolio. To do this with a `Portfolio` object, use `setMinMaxNumAssets`.

```
pWithMaxNumAssets = setMinMaxNumAssets(p, [], 8);
```

```
wgt = estimateFrontier(p);
```

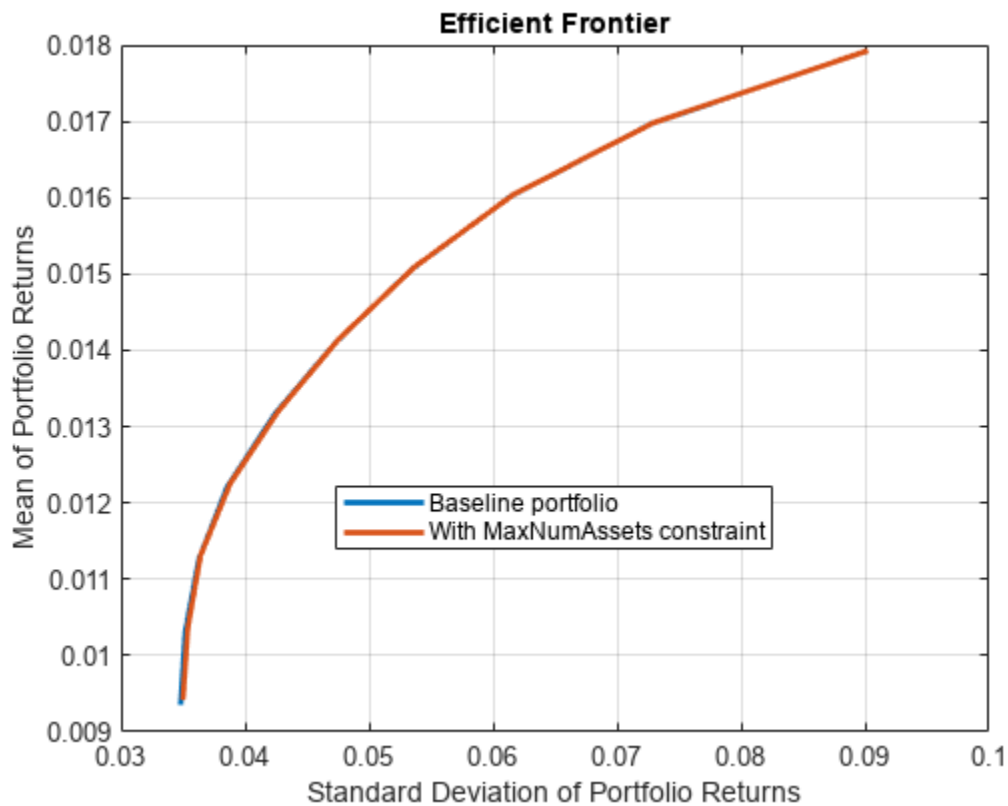
```
wgtWithMaxNumAssets = estimateFrontier(pWithMaxNumAssets);
```

Warning: Risk minimization problem failed to converge. Number of iterations exceeded 'MaxIterations'.

```
plotFrontier(p, wgt); hold on;
```

```
plotFrontier(pWithMaxNumAssets, wgtWithMaxNumAssets); hold off;
```

```
legend('Baseline portfolio', 'With MaxNumAssets constraint', 'location', 'best');
```



Use `estimateFrontierByReturn` to find the allocation that minimizes the risk on the frontier for the given target return.

```
pwgtWithMaxNum = estimateFrontierByReturn(pWithMaxNumAssets, targetRetn);
```

Warning: Risk minimization problem failed to converge. Number of iterations exceeded 'MaxIterations'.

Plot the composition of the two Portfolio objects for the universe of 30 assets.

```
idx = (pwgt>toler) | (pwgtWithMaxNum>toler);
```

```
barh([pwgt(idx), pwgtWithMaxNum(idx)]);
```

```
grid on
```

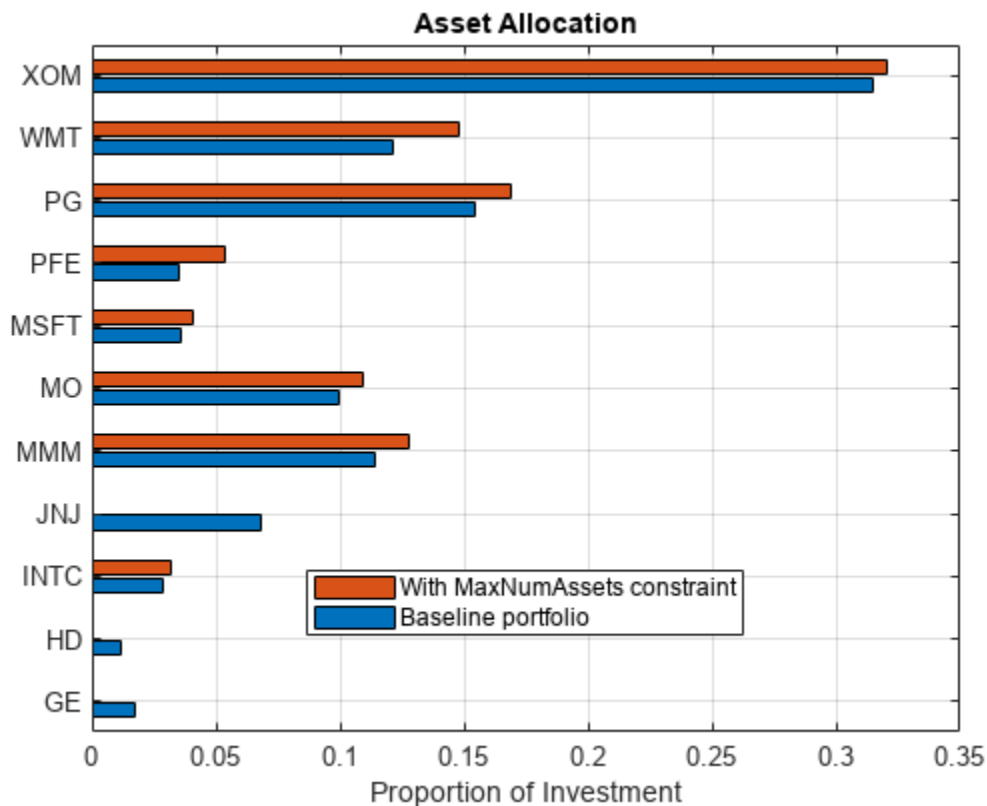
```
xlabel('Proportion of Investment')
```

```
yticks(1:sum(idx));
```

```
yticklabels(p.AssetList(idx));
```

```
title('Asset Allocation');
```

```
legend('Baseline portfolio', 'With MaxNumAssets constraint', 'location', 'best');
```



```
sum(abs(pwgt)>toler)
```

```
ans = 11
```

Count the total number of allocated assets to verify that only eight assets at most are allocated.

```
sum(abs(pwgtWithMaxNum)>toler)
```

```
ans = 8
```

Limit the Minimum and Maximum Number of Assets to Allocate

Suppose that you want to set both the lower and upper bounds for the number of assets to allocate in a portfolio, given the universe of assets. Use `setBounds` to specify the allowed number of assets to allocate as from 5 through 10, and the allocated weight as no less than 5%.

```
p1 = setMinMaxNumAssets(p, 5, 10);
p1 = setBounds(p1, 0.05, 'BoundType', 'conditional');
```

If an asset is allocated, it is necessary to clearly define the minimum weight requirement for that asset. This is done using `setBounds` with a 'Conditional' `BoundType`. Otherwise, the optimizer cannot evaluate which assets are allocated and cannot formulate the `MinNumAssets` constraint. For more details, see “Conditional Bounds with LowerBound Defined as Empty or Zero” on page 4-149.

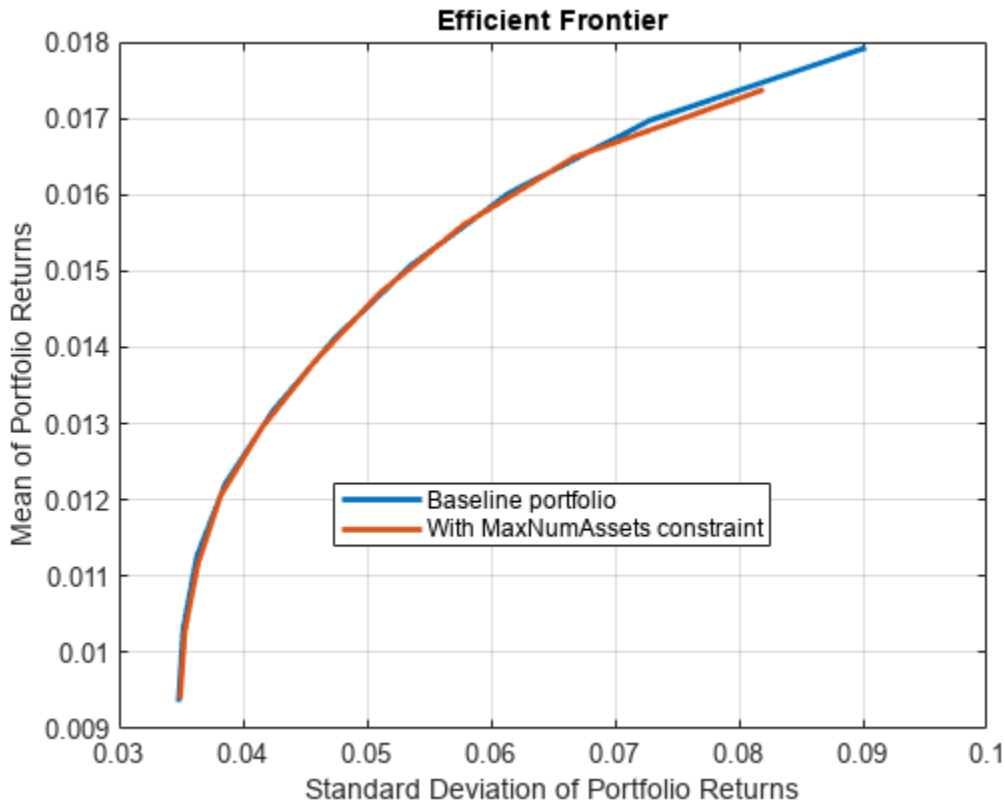
Plot the efficient frontier to compare this portfolio to the baseline portfolio, which has only default constraints.

```
wgt = estimateFrontier(p);
wgt1 = estimateFrontier(p1);
```

```

plotFrontier(p, wgt); hold on;
plotFrontier(pl, wgt1); hold off;
legend('Baseline portfolio', 'With MaxNumAssets constraint', 'location', 'best');

```



Asset Allocation for an Equal-Weighted Portfolio

Create an equal-weighted portfolio using both `setBounds` and `setMinMaxNumAssets` functions.

```

numAssetsAllocated = 8;
weight= 1/numAssetsAllocated;
p2 = setBounds(p, weight, weight, 'BoundType', 'conditional');
p2 = setMinMaxNumAssets(p2, numAssetsAllocated, numAssetsAllocated);

```

When any one, or any combination of 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` are active, the optimization problem is formulated as a mixed integer nonlinear programming (MINLP) problem. The `Portfolio` class automatically constructs the MINLP problem based on the specified constraints.

When working with a `Portfolio` object, you can select one of three solvers using the `setSolverMINLP` function. In this example, instead of using default MINLP solver options, customize the solver options to help with a convergence issue. Use a large number (50) for 'MaxIterationsInactiveCut' with `setSolverMINLP`, instead of the default value of 30 for 'MaxIterationsInactiveCut'. The value 50 works well in finding the efficient frontier of optimal asset allocation.

```

p2 = setSolverMINLP(p2, 'OuterApproximation', 'MaxIterationsInactiveCut', 50);

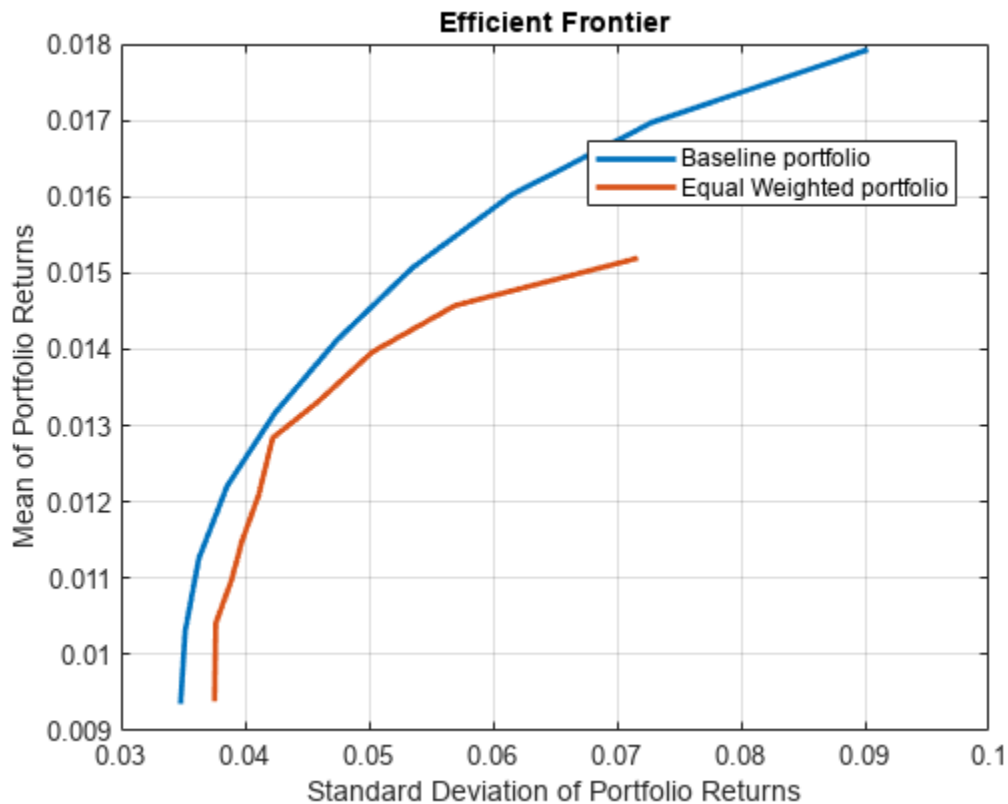
```

Plot the efficient frontiers for the baseline and equal-weighted portfolios.

```

wgt = estimateFrontier(p);
wgt2 = estimateFrontier(p2);
plotFrontier(p, wgt); hold on;
plotFrontier(p2, wgt2); hold off;
legend('Baseline portfolio', 'Equal Weighted portfolio', 'location', 'best');

```



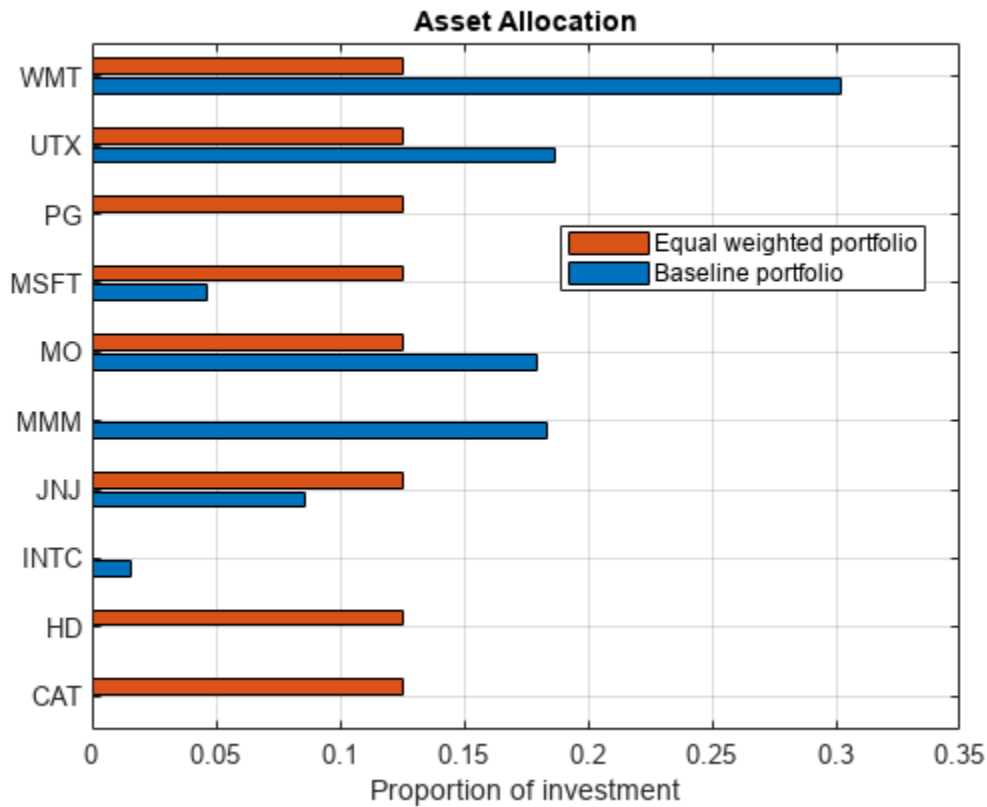
Use `estimateFrontierByRisk` to optimize for a specific risk level, in this case .05, to determine what allocation maximizes the portfolio return.

```

targetRisk = 0.05;
pwgt = estimateFrontierByRisk(p, targetRisk);
pwgt2 = estimateFrontierByRisk(p2, targetRisk);

idx = (pwgt>toler) | (pwgt2>toler);
barh([pwgt(idx), pwgt2(idx)]);
grid on
xlabel('Proportion of investment')
yticks(1:sum(idx));
yticklabels(p.AssetList(idx));
title('Asset Allocation');
legend('Baseline portfolio', 'Equal weighted portfolio', 'location', 'best');

```



Use 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints with Other Constraints

You can define other constraints for a `Portfolio` object using the `set` functions. These other constraints for a `Portfolio` object, such as group, linear inequality, turnover, and tracking error can be used together with the 'Conditional' `BoundType`, 'MinNumAssets', and 'MaxNumAssets' constraints. For example, specify a tracking error constraint using `setTrackingError`.

```
ii = [15, 16, 20, 21, 23, 25, 27, 29, 30]; % indexes of assets to include in tracking portfolio
trackingPort(ii) = 1/numel(ii);
q = setTrackingError(p, 0.5, trackingPort);
```

Then use `setMinMaxNumAssets` to add a constraint to limit maximum number of assets to invest.

```
q = setMinMaxNumAssets(q, [], 8);
```

On top of these previously specified constraints, use `setBounds` to add a constraint to limit the weight for the allocated assets. You can use constraints with mixed `BoundType` values, where 'Simple' means $lb \leq x_i \leq ub$ and 'Conditional' means $x_i = 0$ or $lb \leq x_i \leq ub$.

Allow the assets in `trackingPort` to have the `BoundType` value 'Conditional' in the optimum allocation.

```
lb = zeros(q.NumAssets, 1);
ub = zeros(q.NumAssets, 1)*0.5;
lb(ii) = 0.1;
ub(ii) = 0.3;
```

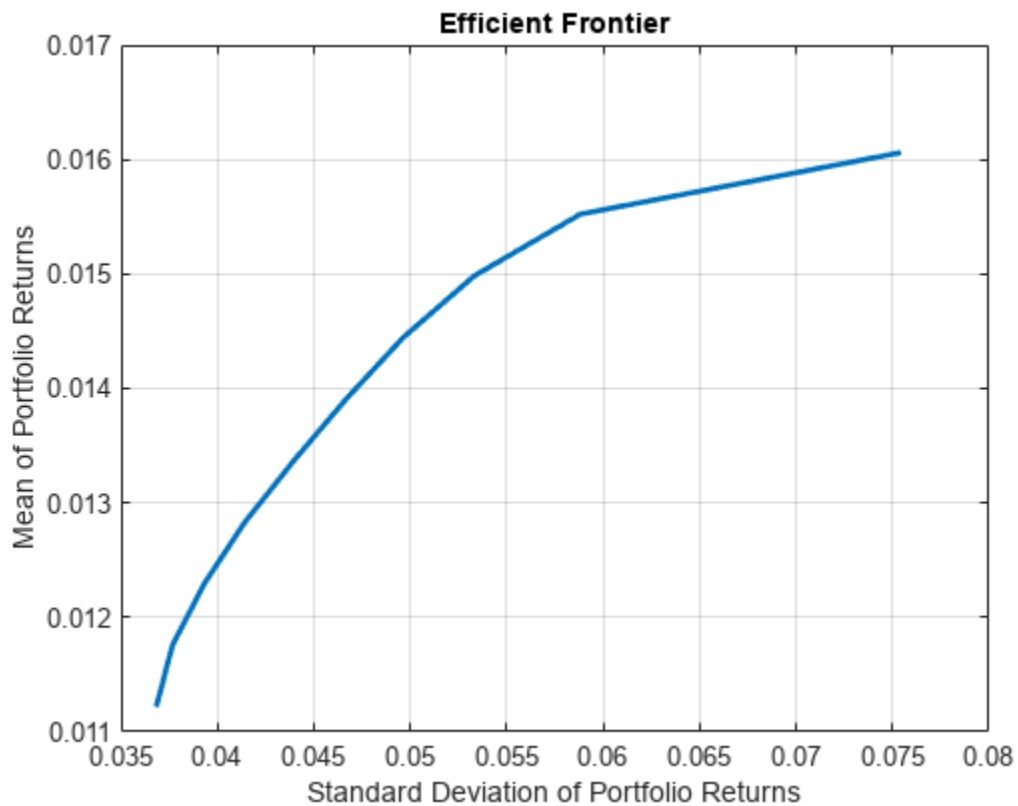
```

boundType = repmat("simple",q.NumAssets,1);
boundType(ii) = "conditional";
q = setBounds(q, lb, ub, 'BoundType',boundType);

```

Plot the efficient frontier:

```
plotFrontier(q);
```



Use `estimateFrontierByReturn` to find the allocation that minimizes the risk for a given return at 0.125.

```

targetRetn = 0.0125;
pwgt = estimateFrontierByReturn(q, targetRetn);

```

Show the allocation of assets by weight.

```

idx = abs(pwgt)>eps;
assetnames = q.AssetNameList';
Asset = assetnames(idx);
Weight = pwgt(idx);
resultAlloc = table(Asset, Weight)

```

```

resultAlloc=7x2 table
    Asset    Weight
    _____
    {'JNJ' }    0.1
    {'MMM' }    0.19503

```



```
{ 'MO' }      0.1485
{ 'MSFT' }    0.1
{ 'PG' }      0.1
{ 'WMT' }     0.2212
{ 'XOM' }     0.13527
```

See Also

`Portfolio` | `setBounds` | `setMinNumAssets` | `setSolverMINLP`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81
- “Portfolio Object Workflow” on page 4-18

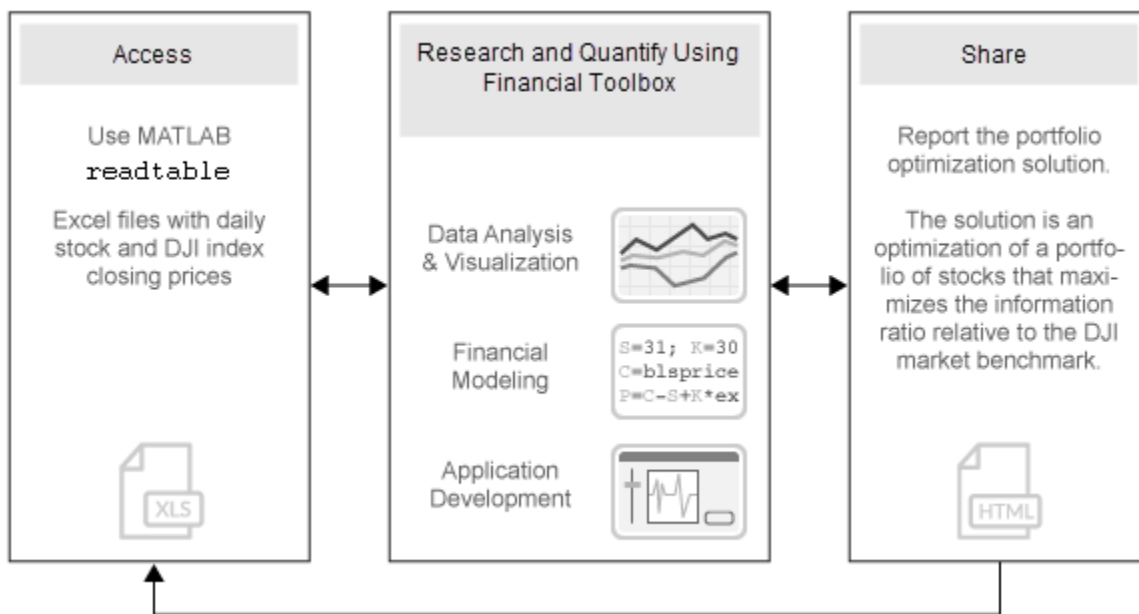
External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Portfolio Optimization Against a Benchmark

This example shows how to perform portfolio optimization using the `Portfolio` object in Financial Toolbox™.

This example, in particular, demonstrates optimizing a portfolio to maximize the information ratio relative to a market benchmark. Specifically, financial data contained in a `table` is read into MATLAB® and visualizations (at both daily and annual levels) are performed. A `Portfolio` object is created with the market data using an active daily return for each asset. Using functions supporting a `Portfolio` object, the efficient frontier is calculated directly and a customized optimization problem is solved to find the asset allocation with the maximized information ratio.



Import Historical Data Using MATLAB®

Import historical prices for the asset universe and the Dow Jones Industrial Average (DJI) market benchmark. The data is imported into a `table` from a Microsoft® Excel® spreadsheet using the MATLAB® `readtable` function.

```
data = readtable('dowPortfolio.xlsx');
head(data, 10)
```

Dates	DJI	AA	AIG	AXP	BA	C	CAT	DD	DIS
03-Jan-2006	10847	28.72	68.41	51.53	68.63	45.26	55.86	40.68	24.18
04-Jan-2006	10880	28.89	68.51	51.03	69.34	44.42	57.29	40.46	23.77
05-Jan-2006	10882	29.12	68.6	51.57	68.53	44.65	57.29	40.38	24.19
06-Jan-2006	10959	29.02	68.89	51.75	67.57	44.65	58.43	40.55	24.52
09-Jan-2006	11012	29.37	68.57	53.04	67.01	44.43	59.49	40.32	24.78
10-Jan-2006	11012	28.44	69.18	52.88	67.33	44.57	59.25	40.2	25.09
11-Jan-2006	11043	28.05	69.6	52.59	68.3	44.98	59.28	38.87	25.33
12-Jan-2006	10962	27.68	69.04	52.6	67.9	45.02	60.13	38.02	25.41

13-Jan-2006	10960	27.81	68.84	52.5	67.7	44.92	60.24	37.86	25.47
17-Jan-2006	10896	27.97	67.84	52.03	66.93	44.47	60.85	37.75	25.15

Separate the asset names, asset prices, and DJI benchmark prices from the table. The visualization shows the evolution of all the asset prices normalized to start at unity, that is accumulative returns.

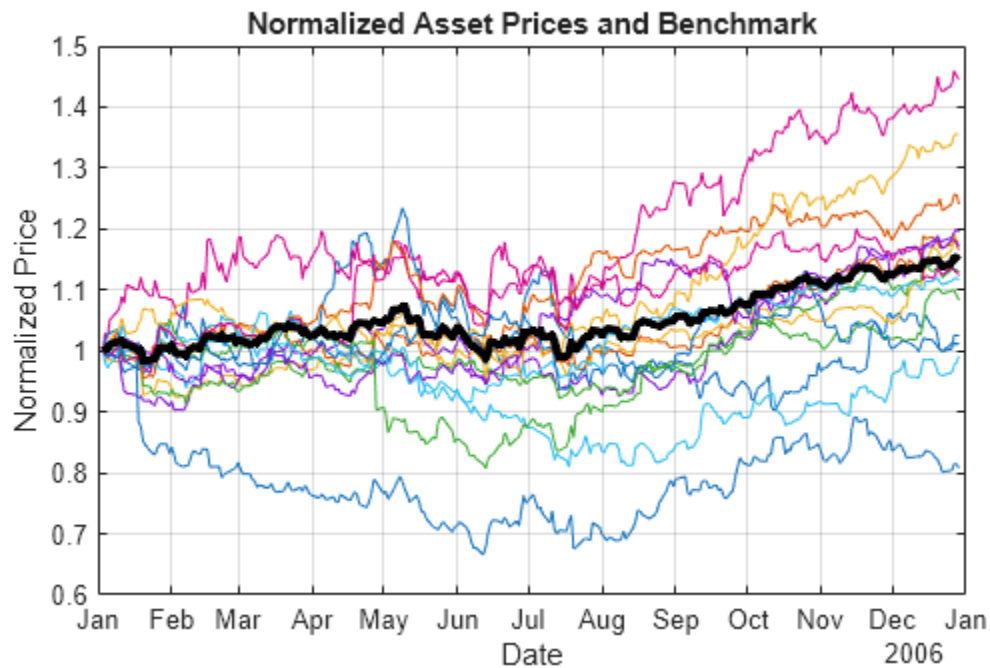
```

benchPrice = data.DJI;
assetNames = data.Properties.VariableNames(3:2:end); % Use half of the assets for display.
assetPrice = data(:,assetNames).Variables;

assetP = assetPrice./assetPrice(1, :);
benchmarkP = benchPrice / benchPrice(1);

figure;
plot(data.Dates,assetP);
hold on;
plot(data.Dates,benchmarkP,'LineWidth',3,'Color','k');
hold off;
xlabel('Date');
ylabel('Normalized Price');
title('Normalized Asset Prices and Benchmark');
grid on;

```



The bold line indicates the DJIA market benchmark.

Compute Returns and Risk-Adjusted Returns

Calculate the return series from the price series and compute the asset moments (historical returns and standard deviations). The visualization shows a scatter plot of the risk-return characteristics of all the assets and the DJI market benchmark.

```

benchReturn = tick2ret(benchPrice);
assetReturn = tick2ret(assetPrice);

```

```

benchRetn = mean(benchReturn);
benchRisk = std(benchReturn);
assetRetn = mean(assetReturn);
assetRisk = std(assetReturn);

```

Calculate historical statistics and plot the annual risk-return. Note that the plot is at the annual level, therefore scaling is performed on the daily returns.

```
scale = 252;
```

```

assetRiskR = sqrt(scale) * assetRisk;
benchRiskR = sqrt(scale) * benchRisk;
assetReturnR = scale * assetRetn;
benchReturnR = scale * benchRetn;

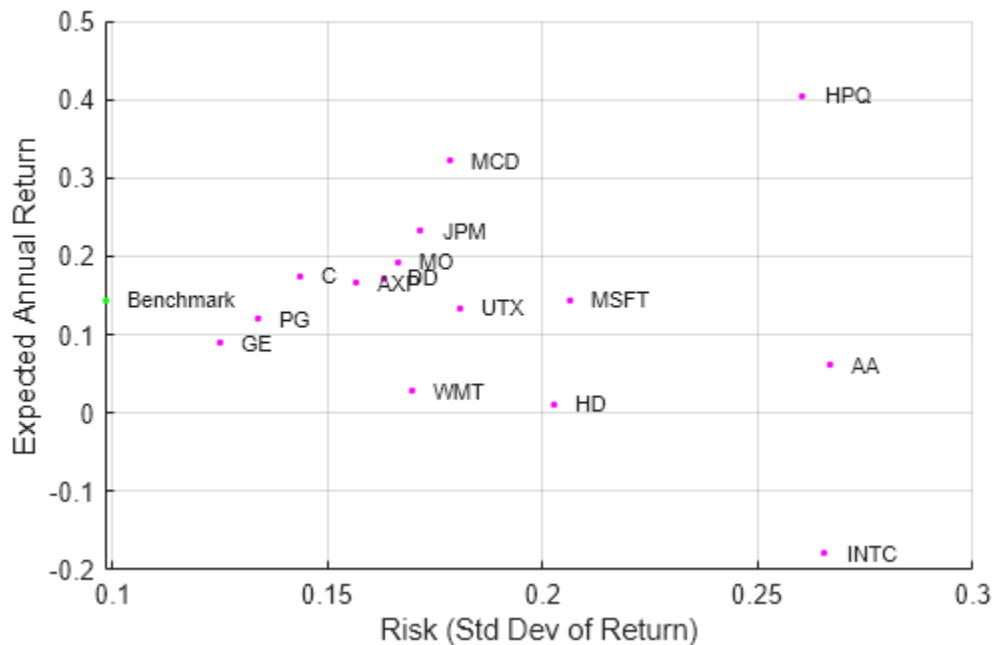
```

```

figure;
scatter(assetRiskR, assetReturnR, 6, 'm', 'Filled');
hold on
scatter(benchRiskR, benchReturnR, 6, 'g', 'Filled');
for k = 1:length(assetNames)
    text(assetRiskR(k) + 0.005, assetReturnR(k), assetNames{k}, 'FontSize', 8);
end
text(benchRiskR + 0.005, benchReturnR, 'Benchmark', 'FontSize', 8);
hold off;

xlabel('Risk (Std Dev of Return)');
ylabel('Expected Annual Return');
grid on;

```



Set Up a Portfolio Optimization

Set up a portfolio optimization problem by populating the object using `Portfolio`. Because the goal is to optimize portfolio allocation against a benchmark, the active return of each asset is computed

and used in the `Portfolio` object. In this example, the expected returns and covariances of the assets in the portfolio are set to their historical values.

```
p = Portfolio('AssetList',assetNames);
```

Set up default portfolio constraints (all weights sum to 1, no shorting, and 100% investment in risky assets).

```
p = setDefaultConstraints(p);
```

Add asset returns and covariance to the `Portfolio` object.

```
activReturn = assetReturn - benchReturn;
pAct = estimateAssetMoments(p,activReturn,'missingdata',false)
```

```
pAct =
```

```
Portfolio with properties:
```

```

    BuyCost: []
    SellCost: []
    RiskFreeRate: []
    AssetMean: [15x1 double]
    AssetCovar: [15x15 double]
    TrackingError: []
    TrackingPort: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
    Name: []
    NumAssets: 15
    AssetList: {'AA' 'AXP' 'C' 'DD' 'GE' 'HD' 'HPQ' 'INTC' 'JPM' 'MCD'}
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [15x1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
    ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
    BoundType: [15x1 categorical]
```

Compute the Efficient Frontier Using the Portfolio Object

Compute the mean-variance efficient frontier of 20 optimal portfolios. Visualize the frontier over the risk-return characteristics of the individual assets. Furthermore, calculate and visualize the information ratio for each portfolio along the frontier.

```
pwgtAct = estimateFrontier(pAct, 20); % Estimate the weights.
[portRiskAct, portRetnAct] = estimatePortMoments(pAct, pwgtAct); % Get the risk and return.

% Extract the asset moments and names.
[assetActRetnDaily, assetActCovarDaily] = getAssetMoments(pAct);
assetActRiskDaily = sqrt(diag(assetActCovarDaily));
assetNames = pAct.AssetNameList;

% Rescale.
assetActRiskAnnual = sqrt(scale) * assetActRiskDaily;
portRiskAnnual = sqrt(scale) * portRiskAct;
assetActRetnAnnual = scale * assetActRetnDaily;
portRetnAnnual = scale * portRetnAct;

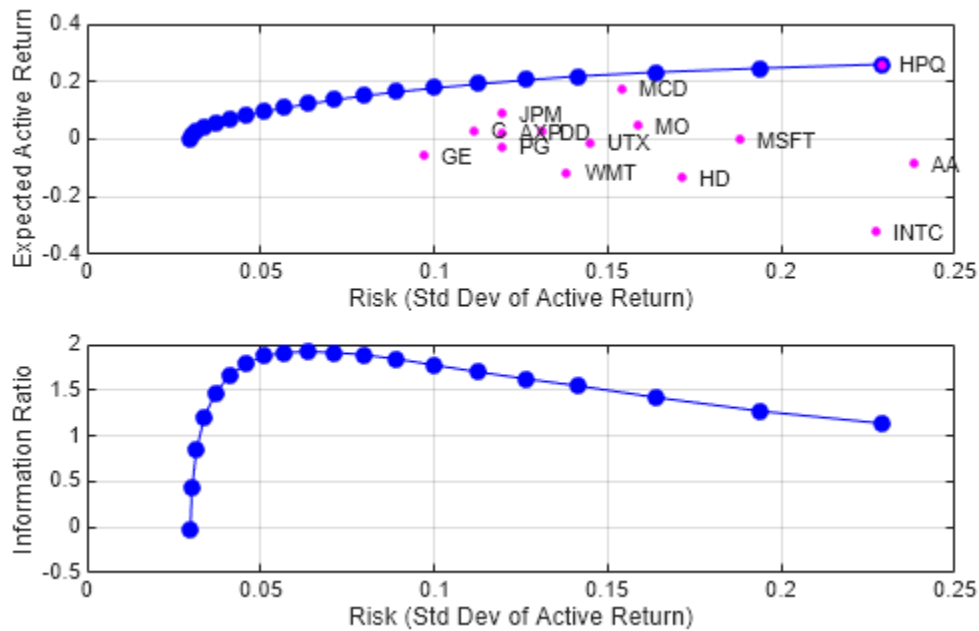
figure;
subplot(2,1,1);
plot(portRiskAnnual, portRetnAnnual, 'bo-', 'MarkerFaceColor', 'b');
hold on;

scatter(assetActRiskAnnual, assetActRetnAnnual, 12, 'm', 'Filled');
hold on;
for k = 1:length(assetNames)
    text(assetActRiskAnnual(k) + 0.005, assetActRetnAnnual(k), assetNames{k}, 'FontSize', 8);
end

hold off;

xlabel('Risk (Std Dev of Active Return)');
ylabel('Expected Active Return');
grid on;

subplot(2,1,2);
plot(portRiskAnnual, portRetnAnnual./portRiskAnnual, 'bo-', 'MarkerFaceColor', 'b');
xlabel('Risk (Std Dev of Active Return)');
ylabel('Information Ratio');
grid on;
```



Perform Information Ratio Maximization

Find the portfolio along the frontier with the maximum information ratio. The information ratio is the ratio of relative return to relative risk (also known as the "tracking error"). Whereas the Sharpe ratio looks at returns relative to a riskless asset, the information ratio is based on returns relative to a risky benchmark, in this case the DJI benchmark. You can compute the information ratio using `estimateCustomObjectivePortfolio`.

```
infoRatio = @(x) (pAct.AssetMean'*x)/sqrt(x'*pAct.AssetCovar*x);
optWts = estimateCustomObjectivePortfolio(pAct,infoRatio,...
    ObjectiveSense="maximize");
```

Get the information ratio, risk, and return for the optimal portfolio.

```
optInfoRatio = infoRatio(optWts);
[optPortRisk,optPortRetn] = estimatePortMoments(pAct,optWts)
```

```
optPortRisk =
0.0040
```

```
optPortRetn =
4.8166e-04
```

Plot the Optimal Portfolio

Verify that the portfolio found is indeed the maximum information-ratio portfolio.

```
% Rescale.
optPortRiskAnnual = sqrt(scale) * optPortRisk;
optPortReturnAnnual = scale * optPortRetn;
```

```
figure;
subplot(2,1,1);
```

```

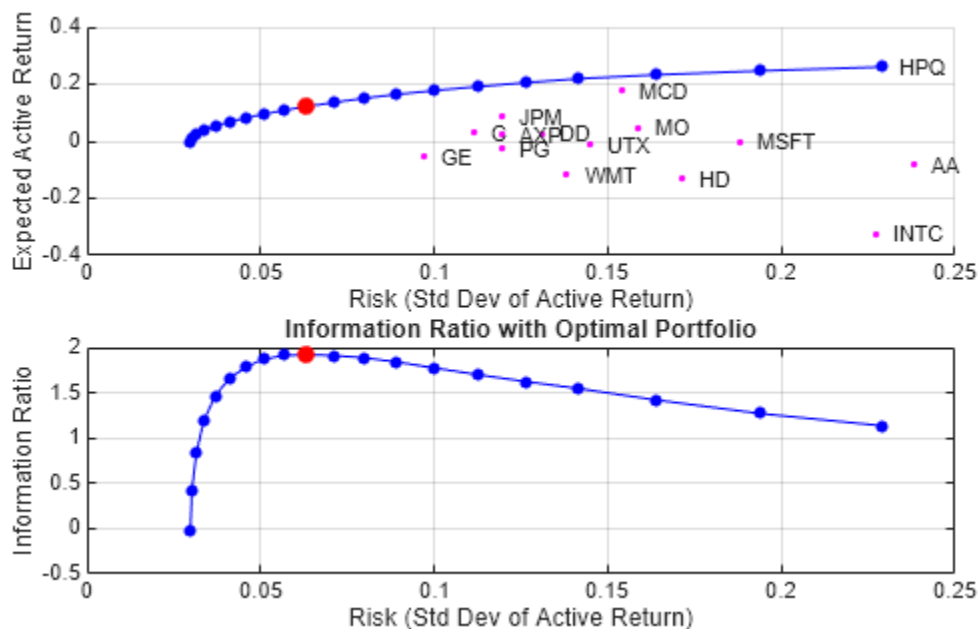
scatter(assetActRiskAnnual, assetActRetnAnnual, 6, 'm', 'Filled');
hold on
for k = 1:length(assetNames)
    text(assetActRiskAnnual(k) + 0.005,assetActRetnAnnual(k),assetNames{k},'FontSize',8);
end
plot(portRiskAnnual,portRetnAnnual,'bo-','MarkerSize',4,'MarkerFaceColor','b');
plot(optPortRiskAnnual,optPortReturnAnnual,'ro-','MarkerFaceColor','r');
hold off;

xlabel('Risk (Std Dev of Active Return)');
ylabel('Expected Active Return');
grid on;

subplot(2,1,2);
plot(portRiskAnnual,portRetnAnnual./portRiskAnnual,'bo-','MarkerSize',4,'MarkerFaceColor','b');
hold on
plot(optPortRiskAnnual,optPortReturnAnnual./optPortRiskAnnual,'ro-','MarkerFaceColor','r');
hold off;

xlabel('Risk (Std Dev of Active Return)');
ylabel('Information Ratio');
title('Information Ratio with Optimal Portfolio');
grid on;

```



Display the Portfolio Optimization Solution

Display the portfolio optimization solution.

```

assetIdx = optWts > .001;
results = table(assetNames(assetIdx)', optWts(assetIdx)*100, 'VariableNames',{'Asset', 'Weight'});
disp('Maximum Information Ratio Portfolio:')

```

Maximum Information Ratio Portfolio:


```
disp(results)
```

Asset	Weight
{ 'AA' }	1.5389
{ 'AXP' }	0.35545
{ 'C' }	9.6533
{ 'DD' }	4.0684
{ 'HPQ' }	17.699
{ 'JPM' }	21.565
{ 'MCD' }	26.736
{ 'MO' }	13.648
{ 'MSFT' }	2.6858
{ 'UTX' }	2.0509

```
fprintf('Active return for Max. Info Ratio portfolio is %.2f%%\n', optPortRetn*25200);
```

```
Active return for Max. Info Ratio portfolio is 12.14%
```

```
fprintf('Tracking error for Max. Info Ratio portfolio is %.2f%%\n', optPortRisk*sqrt(252)*100);
```

```
Tracking error for Max. Info Ratio portfolio is 6.32%
```

See Also

Portfolio | inforatio | fminbnd

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Performance Metrics Overview” on page 7-2
- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- [Getting Started with Portfolio Optimization \(4 min 13 sec\)](#)
- [MATLAB for Advanced Portfolio Construction and Stock Selection Models \(30 min 28 sec\)](#)

Portfolio Analysis with Turnover Constraints

This example shows how to analyze the characteristics of a portfolio of equities, and then compare them with the efficient frontier. This example seeks to answer the question of how much closer can you get to the efficient frontier by only risking a certain percentage of a portfolio to avoid transaction costs.

Import Data for the Portfolio Holdings

Load information on the current portfolio holdings from a Microsoft® Excel® spreadsheet into a table using the MATLAB® `readtable` function.

```
AssetHoldingData = readtable('portfolio.xls');
% Create a normalized current holdings vector that shows the respective
% investments as a percentage of total capital:
W = AssetHoldingData.Value/sum(AssetHoldingData.Value);
```

Import Market Data for Share Prices

Import the market data from a data source supported by Datafeed Toolbox™ that constitutes three years of closing prices for the stocks listed in the portfolio.

```
load SharePrices
```

Create a Portfolio Object

The `Portfolio` class enables you to use the imported data to create a `Portfolio` object. The `estimateAssetMoments` function for the `Portfolio` object enables you to set up a portfolio given only a historical price or returns series. The `estimateAssetMoments` function estimates mean and covariance of asset returns from data even if there is missing data.

```
P = Portfolio('Name', 'Sample Turnover Constraint Portfolio');
P = estimateAssetMoments(P,data,'DataFormat','Prices');

% You can assign text names to each asset in the portfolio.
P = setAssetList(P,AssetHoldingData.Symbol);

% Provide the current holdings.
P = setInitPort(P,W);
```

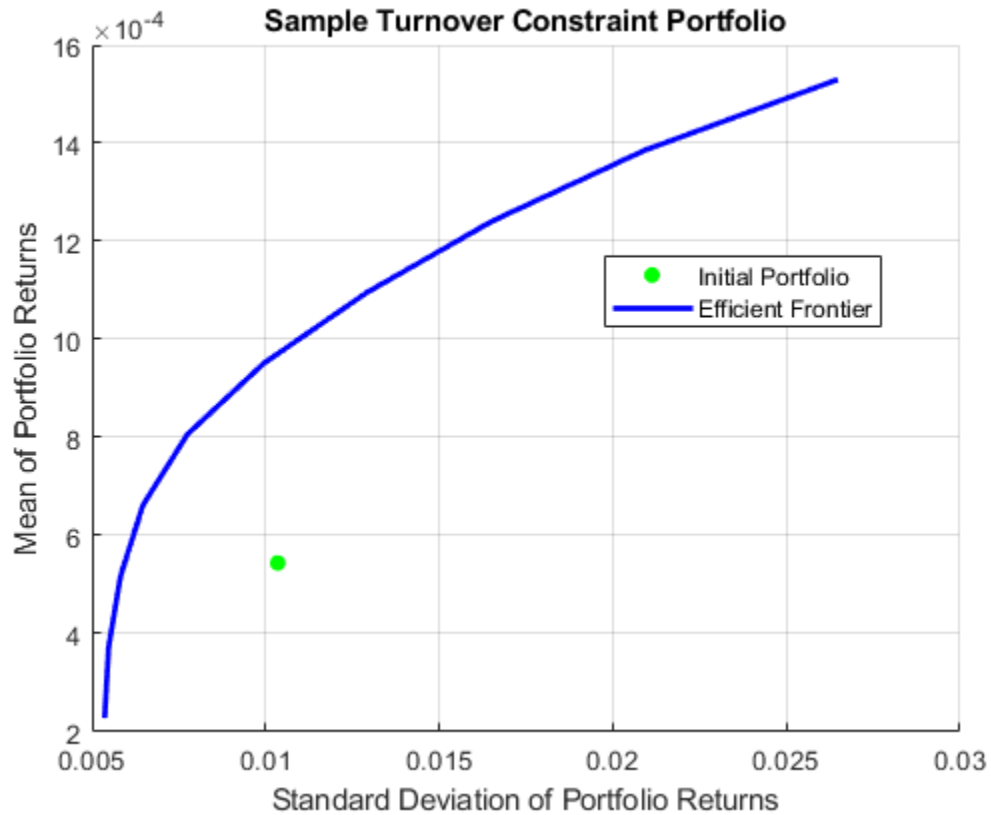
Perform Portfolio Optimization with No Turnover Constraint

The `Portfolio` object can optimize the holdings given any number of constraints. This example demonstrates using a simple, default constraint, that is, long positions only and 100% invested in assets.

```
P = setDefaultConstraints(P);
```

Visualize this efficient frontier with the `plotFrontier` function.

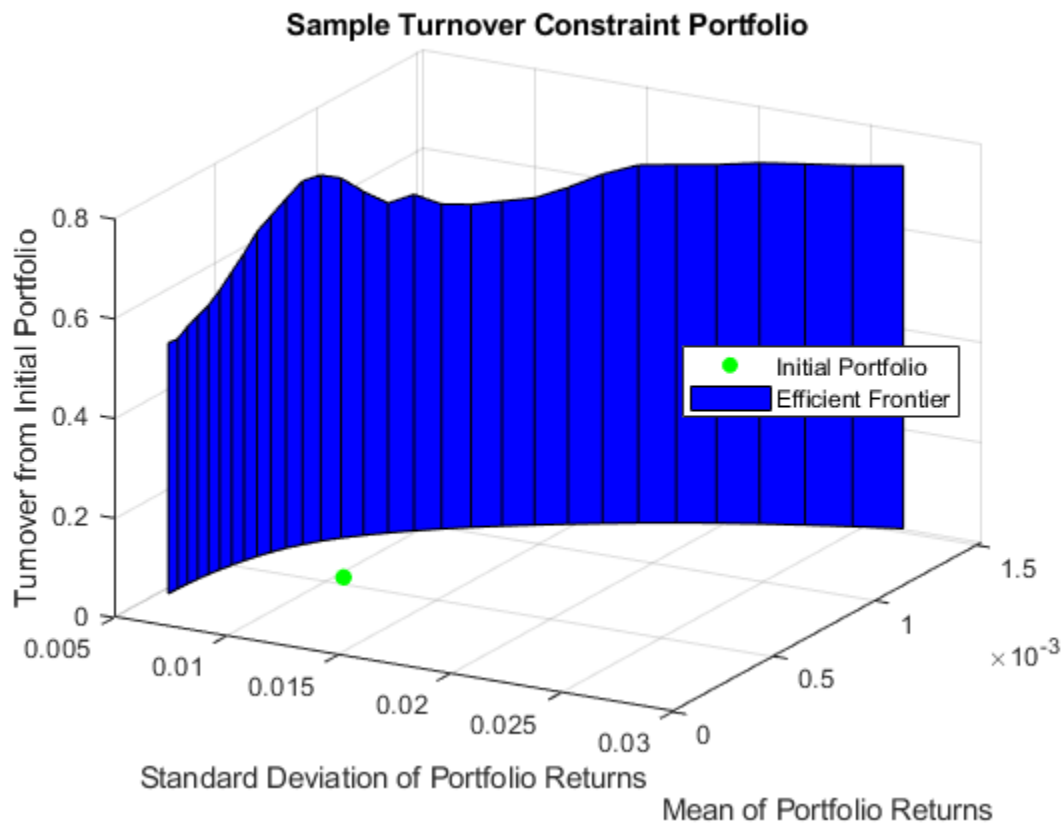
```
plotFrontier(P)
```



Visualize the Transaction Costs and Turnover

Due to transaction costs, it can be expensive to shift holdings from the current portfolio to a portfolio along this efficient frontier. The following custom plot shows that you must turn over between 50% and 75% of the holdings to get to this frontier.

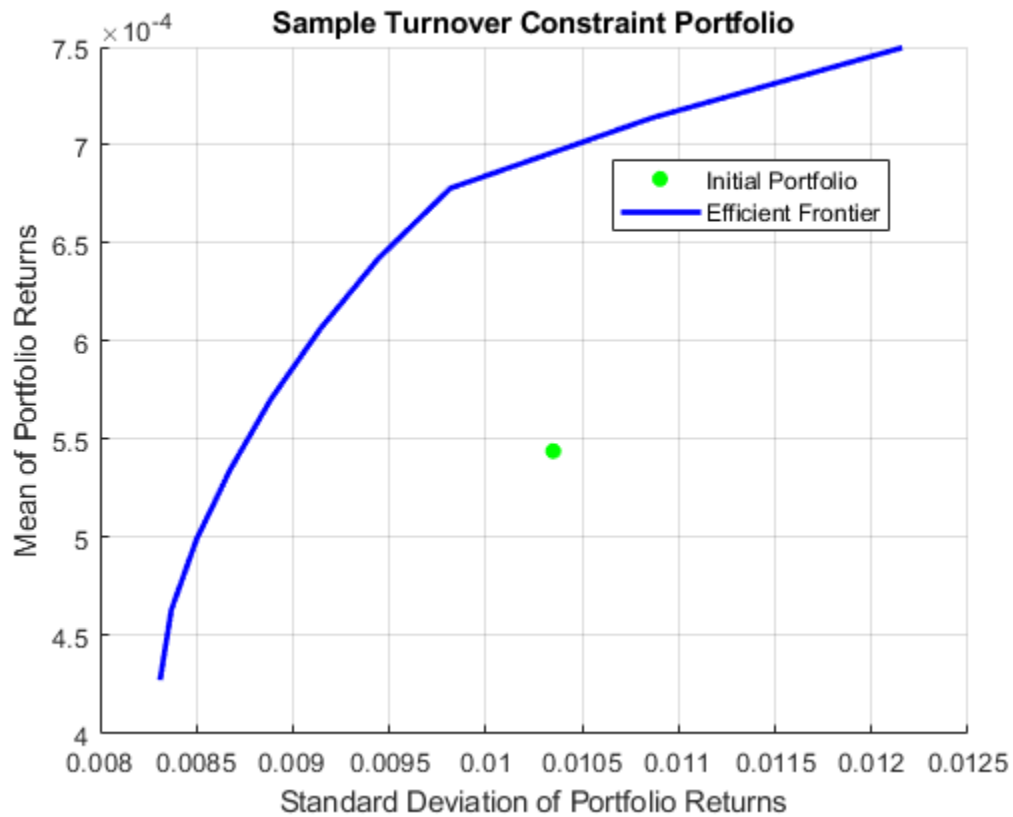
TurnoverPlot(P)



Perform Portfolio Optimization with a Turnover Constraint

How close can you get to this efficient frontier by only trading some of the portfolio? Assume that you want to trade only a certain percentage of the portfolio to avoid too much turnover in your holdings. This requirement imposes some nonlinear constraints on the problem and gives a problem with multiple local minima. Even so, the `Portfolio` object solves the problem, and you specify the turnover constraint using the `setTurnover` function.

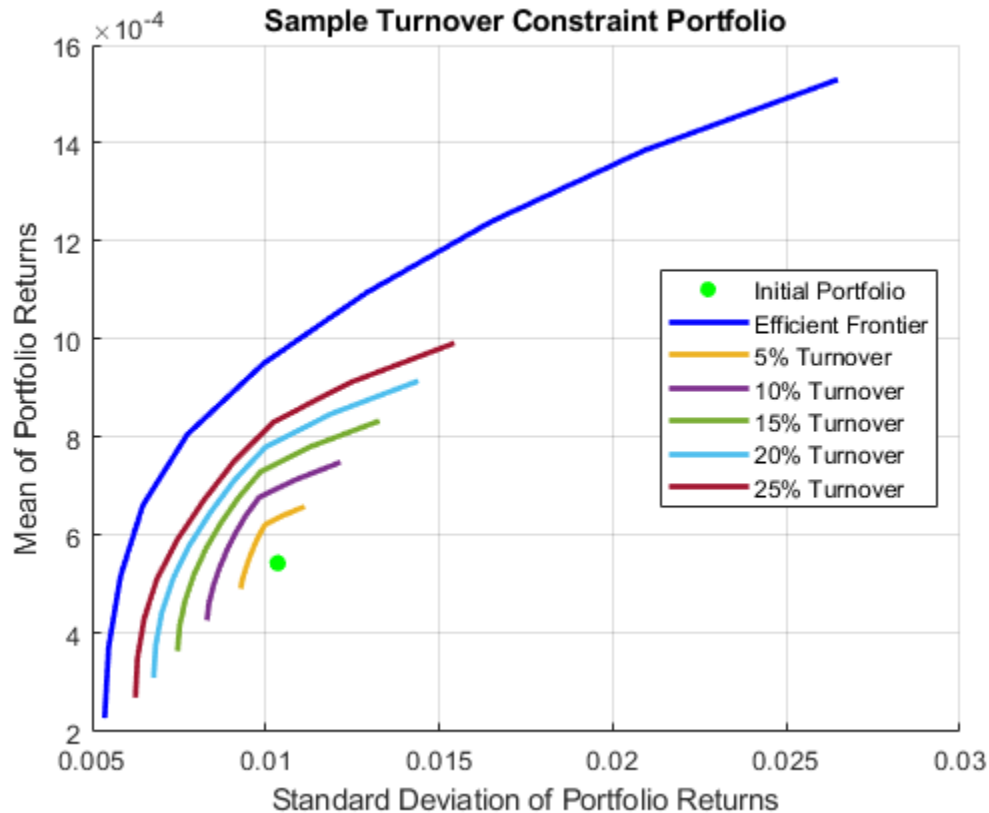
```
P10 = setTurnover(P,0.10);
plotFrontier(P10)
```



Visualize the Efficient Frontier at Different Turnover Thresholds

This efficient frontier is much closer to the initial portfolio than the starting efficient frontier without turnover constraints. To visualize this difference, use the custom function `TurnoverConstraintPlot` to visualize multiple constrained efficient frontiers at different turnover thresholds.

```
turnovers = 0.05:0.05:0.25;
TurnoverConstraintPlot(P,turnovers)
```



The `Portfolio` object is a powerful and efficient tool for performing various portfolio analysis tasks. In addition to turnover constraints, you can also optimize a `Portfolio` object for transaction costs for buying and selling portfolio assets using the `setCosts` function.

See Also

`Portfolio` | `setBounds` | `addGroups` | `setAssetMoments` | `estimateAssetMoments` | `estimateBounds` | `plotFrontier` | `estimateFrontierLimits` | `estimateFrontierByRisk` | `estimatePortRisk`

Related Examples

- "Creating the Portfolio Object" on page 4-25
- "Working with Portfolio Constraints Using Defaults" on page 4-58
- "Validate the Portfolio Problem for Portfolio Object" on page 4-94
- "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
- "Estimate Efficient Frontiers for Portfolio Object" on page 4-122
- "Postprocessing Results to Set Up Tradable Portfolios" on page 4-130
- "Portfolio Optimization with Semicontinuous and Cardinality Constraints" on page 4-190
- "Black-Litterman Portfolio Optimization Using Financial Toolbox" on page 4-222
- "Portfolio Optimization Against a Benchmark" on page 4-202
- "Portfolio Optimization Examples Using Financial Toolbox" on page 4-161

- “Leverage in Portfolio Optimization with a Risk-Free Asset” on page 4-217
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Leverage in Portfolio Optimization with a Risk-Free Asset

This example shows how to use the `setBudget` function for the `Portfolio` class to define the limits on the `sum(AssetWeight_i)` in risky assets.

If the `sum(AssetWeight_i)` is less than 1, the extra cash is invested in a risk-free asset. If the `sum(AssetWeight_i)` is larger than 1, meaning that total risky asset investment is larger than initial available cash, the risk-free asset is shorted (borrowed) to fund the extra investment in a risky asset. The cost associated with borrowing a risk-free asset is automatically captured in the mean-variance optimization model for the `Portfolio` class. Therefore, you can use the `setBudget` function directly to control the level of leverage of cash for the portfolio.

Portfolio Without Leverage

Consider the following example that does not leverage a risk-free asset.

```
assetsMean = [ 0.05; 0.1; 0.12; 0.18; ];
assetCovar = [ 0.0064 0.00408 0.00192 0;
               0.00408 0.0289 0.0204 0.0119;
               0.00192 0.0204 0.0576 0.0336;
               0 0.0119 0.0336 0.1225];
riskFreeRate = 0.03;
```

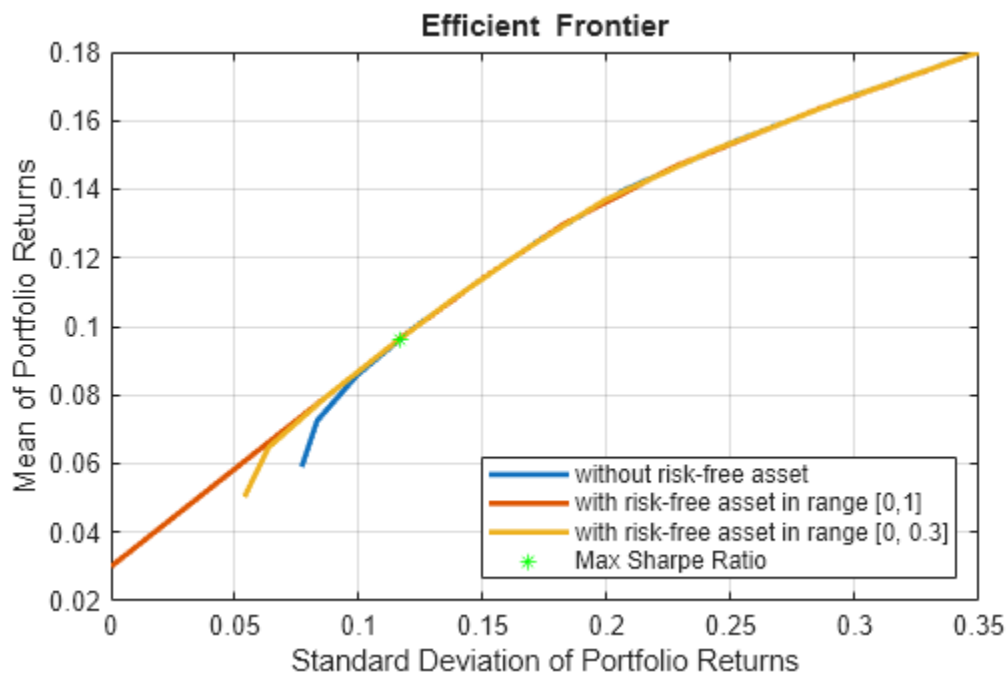
```
% create a portfolio and define risk-free rate.
```

```
p = Portfolio('RiskFreeRate', riskFreeRate, 'assetmean', assetsMean, 'assetcovar', assetCovar, 'location', 'southeast');
```

Create multiple portfolios with different budgets on risky assets to control the limits for investing in a risk-free asset.

```
p = setBudget(p, 1, 1); % allow 0% risk free asset allocation, meaning fully invested in risky assets
p1 = setBudget(p, 0, 1); % allow 0% to 100% risk free asset allocation
p2 = setBudget(p, 0.7, 1); % allow 0% to 30% risk free asset allocation
[risk, retn] = estimatePortMoments(p, estimateMaxSharpeRatio(p));

figure;
plotFrontier(p); hold on;
plotFrontier(p1); hold on;
plotFrontier(p2); hold on;
plot(risk, retn, 'g*'); hold off;
legend('without risk-free asset', ...
       'with risk-free asset in range [0,1]', ...
       'with risk-free asset in range [0, 0.3]', ...
       'Max Sharpe Ratio', 'location','southeast');
```



In the efficient frontiers in the above figure, the lower-left part of the red efficient frontier line for the portfolio with a risk-free asset is in range $[0, 1]$ and is actually the capital allocation line (CAL). The slope of this line is the maximum Sharpe ratio of the portfolio, which demonstrates how return is best awarded by taking extra risk. The upper right of the red efficient frontier line is the same as a fully invested portfolio (blue line). Once the portfolio crosses the Sharpe ratio point, the portfolio is fully invested and there is no more cash available to allow high risk-awarded returns following the straight CAL. However, if borrowing of a risk-free asset is allowed, you can effectively use the funds from the borrowing of a risk-free asset to invest in more risky assets, as demonstrated in the "Portfolio with Leverage" section.

Portfolio with Leverage

To fund investments in risky assets, consider using leverage by borrowing a risk-free asset. The `Portfolio` class enables you to use leverage in asset allocation when a risk-free asset is available in the portfolio.

First, check if the `RiskFreeRate` property for the `Portfolio` object is nonzero.

```
p
```

```
p =
```

```
Portfolio with properties:
```

```

    BuyCost: []
    SellCost: []
    RiskFreeRate: 0.0300
    AssetMean: [4x1 double]
    AssetCovar: [4x4 double]
    TrackingError: []
    TrackingPort: []
    Turnover: []
    BuyTurnover: []

```

```

SellTurnover: []
    Name: []
    NumAssets: 4
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
    ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
    BoundType: []

```

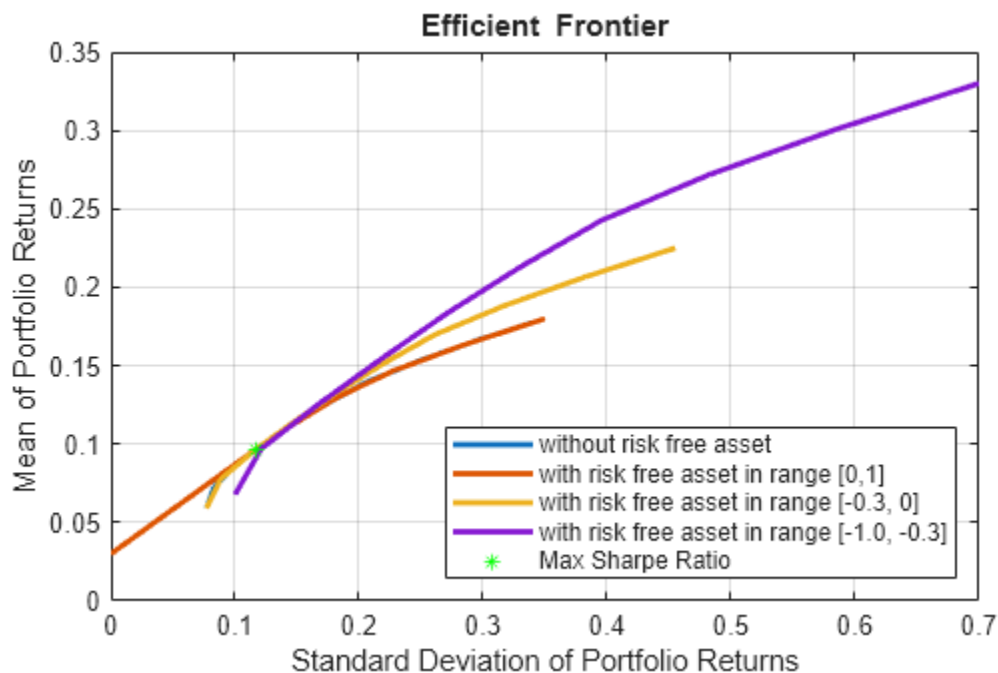
In this Portfolio object, the lower and upper budgets are both 1. Limits must be set on the total investment in risky assets. Borrowing a risk-free asset funds the extra investment in risky assets. Use `setBudget` to set the lower and upper bounds to set the limits of the borrowed risk-free assets.

```

p = setBudget(p, 1, 1);      % allow 0% risk free asset allocation, meaning fully invested in ri
p3 = setBudget(p, 1, 1.3);  % allow 0% risk free asset allocation, and allow borrowing of risk
p4 = setBudget(p, 1.3, 2);  % allow 0% risk free asset allocation, and allow borrowing of risk
[risk, retn] = estimatePortMoments(p, estimateMaxSharpeRatio(p));

figure
plotFrontier(p); hold on;
plotFrontier(p1);hold on;
plotFrontier(p3); hold on;
plotFrontier(p4); hold on;
plot(risk, retn, 'g*'); hold off;
legend('without risk free asset', ...
    'with risk free asset in range [0,1]', ...
    'with risk free asset in range [-0.3, 0]', ...
    'with risk free asset in range [-1.0, -0.3]',...
    'Max Sharpe Ratio', 'location','southeast');

```



In this figure, the upper-right parts of both the orange and purple efficient frontiers extend from the CAL (lower-left red line), because of the leverage of a risk-free asset. The same levels of risk-awarded returns are obtained. Once the portfolio exhausts the maximum allowed leverage, the efficient frontier starts to fall below the CAL again, resulting in portfolios with lower Sharpe ratios.

See Also

Portfolio | setBounds | addGroups | setAssetMoments | estimateAssetMoments | estimateBounds | plotFrontier | estimateFrontierLimits | estimateFrontierByRisk | estimatePortRisk

Related Examples

- "Creating the Portfolio Object" on page 4-25
- "Working with Portfolio Constraints Using Defaults" on page 4-58
- "Validate the Portfolio Problem for Portfolio Object" on page 4-94
- "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
- "Estimate Efficient Frontiers for Portfolio Object" on page 4-122
- "Postprocessing Results to Set Up Tradable Portfolios" on page 4-130
- "Portfolio Optimization with Semicontinuous and Cardinality Constraints" on page 4-190
- "Black-Litterman Portfolio Optimization Using Financial Toolbox" on page 4-222
- "Portfolio Optimization Against a Benchmark" on page 4-202
- "Portfolio Optimization Examples Using Financial Toolbox" on page 4-161
- "Black-Litterman Portfolio Optimization Using Financial Toolbox" on page 4-222
- "Portfolio Optimization Using Factor Models" on page 4-231
- "Portfolio Optimization Using Social Performance Measure" on page 4-264

- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Black-Litterman Portfolio Optimization Using Financial Toolbox

This example shows the workflow to implement the Black-Litterman model with the `Portfolio` class in Financial Toolbox™. The Black-Litterman model is an asset allocation approach that allows investment analysts to incorporate subjective views (based on investment analyst estimates) into market equilibrium returns. By blending analyst views and equilibrium returns instead of relying only on historical asset returns, the Black-Litterman model provides a systematic way to estimate the mean and covariance of asset returns.

	Mean-Variance Optimization	Black-Litterman Approach
Asset Mean	Mean of historical asset returns	Blended asset returns estimated from analyst views and equilibrium returns
Asset Covariance	Covariance of historical asset returns	Covariance of historical asset returns + Estimation uncertainty of the blended asset returns

In the Black-Litterman model, the blended expected return is $\bar{\mu} = [P^T \Omega^{-1} P + C^{-1}]^{-1} [P^T \Omega^{-1} q + C^{-1} \pi]$ and the estimation uncertainty is $\text{cov}(\mu) = [P^T \Omega^{-1} P + C^{-1}]^{-1}$. To use the Black-Litterman model, you must prepare the inputs: P , q , Ω , π , and C . The inputs for P , q , and Ω are view-related and defined by the investment analyst. π is the equilibrium return and C is the uncertainty in prior belief. This example guides you to define these inputs and use the resulting blended returns in a portfolio optimization. For more information on the concept and derivation of the Black-Litterman model, see the Appendix section Black-Litterman Model under a Bayesian Framework on page 4-227.

Define the Universe of Assets

The `dowPortfolio.xlsx` data set includes 30 assets and one benchmark. Seven assets from this data set comprise the investment universe in this example. The risk-free rate is assumed to be zero.

```
T = readtable('dowPortfolio.xlsx');
```

Define the asset universe and extract the asset returns from the price data.

```
assetNames = ["AA", "AIG", "WMT", "MSFT", "BA", "GE", "IBM"];
benchmarkName = "DJI";
head(T(:, ["Dates" benchmarkName assetNames]))
```

Dates	DJI	AA	AIG	WMT	MSFT	BA	GE	IBM
03-Jan-2006	10847	28.72	68.41	44.9	26.19	68.63	33.6	80.13
04-Jan-2006	10880	28.89	68.51	44.99	26.32	69.34	33.56	80.03
05-Jan-2006	10882	29.12	68.6	44.38	26.34	68.53	33.47	80.56
06-Jan-2006	10959	29.02	68.89	44.56	26.26	67.57	33.7	82.96
09-Jan-2006	11012	29.37	68.57	44.4	26.21	67.01	33.61	81.76
10-Jan-2006	11012	28.44	69.18	44.54	26.35	67.33	33.43	82.1
11-Jan-2006	11043	28.05	69.6	45.23	26.63	68.3	33.66	82.19
12-Jan-2006	10962	27.68	69.04	44.43	26.48	67.9	33.25	81.61

```
retnsT = tick2ret(T(:, 2:end));
assetRetns = retnsT(:, assetNames);
```

```
benchRetn = retnsT(:, "DJI");
numAssets = size(assetRetns, 2);
```

Specify Views of the Market

The views represent the subjective views of the investment analyst regarding future market changes, expressed as $\mathbf{q} = \mathbf{P}^* \boldsymbol{\mu} + \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim N(0, \boldsymbol{\Omega})$, $\boldsymbol{\Omega} = \text{diag}(\omega_1, \omega_2, \dots, \omega_v)$, where v is total number of views. For more information, see the Appendix section Assumptions and Views on page 4-227. With v views and k assets, $\mathbf{P}$ is a v -by- k matrix, $\mathbf{q}$ is a v -by-1 vector, and $\boldsymbol{\Omega}$ is a v -by- v diagonal matrix (representing the independent uncertainty in the views). The views do not necessarily need to be independent among themselves and the structure of $\boldsymbol{\Omega}$ can be chosen to account for investment analyst uncertainties in the views [4 on page 4-229]. The smaller the ω_i in $\boldsymbol{\Omega}$, the smaller the variance in the distribution of the i th view, and the stronger or more certain the investor's i th view. This example assumes three independent views.

- 1 AIG is going to have 5% annual return with uncertainty 1e-3. This is a weak absolute view due to its high uncertainty.
- 2 WMT is going to have 3% annual return with uncertainty 1e-3. This is a weak absolute view due to its high uncertainty.
- 3 MSFT is going to outperform IBM by 5% annual return with uncertainty 1e-5. This is a strong relative view due to its low uncertainty.

```
v = 3; % total 3 views
P = zeros(v, numAssets);
q = zeros(v, 1);
Omega = zeros(v);
```

```
% View 1
P(1, assetNames=="AIG") = 1;
q(1) = 0.05;
Omega(1, 1) = 1e-3;
```

```
% View 2
P(2, assetNames=="WMT") = 1;
q(2) = 0.03;
Omega(2, 2) = 1e-3;
```

```
% View 3
P(3, assetNames=="MSFT") = 1;
P(3, assetNames=="IBM") = -1;
q(3) = 0.05;
Omega(3, 3) = 1e-5;
```

Visualize the three views in table form.

```
viewTable = array2table([P q diag(Omega)], 'VariableNames', [assetNames "View_Return" "View_Uncertainty"])
```

```
viewTable=3x9 table
```

AA	AIG	WMT	MSFT	BA	GE	IBM	View_Return	View_Uncertainty
0	1	0	0	0	0	0	0.05	0.001
0	0	1	0	0	0	0	0.03	0.001
0	0	0	1	0	0	-1	0.05	1e-05

Because the returns from `dowPortfolio.xlsx` data set are daily returns and the views are on the annual returns, you must convert views to be on daily returns.

```
bizyear2bizday = 1/252;
q = q*bizyear2bizday;
Omega = Omega*bizyear2bizday;
```

Estimate the Covariance from the Historical Asset Returns

Σ is the covariance of the historical asset returns.

```
Sigma = cov(assetRetns.Variables);
```

Define the Uncertainty C

The Black-Litterman model makes the assumption that the structure of C is proportional to the covariance Σ . Therefore, $C = \tau\Sigma$, where τ is a small constant. A smaller τ indicates a higher confidence in the prior belief of μ . The work of He and Litterman uses a value of 0.025. Other authors suggest using $1/n$ where n is the number of data points used to generate the covariance matrix [3 on page 4-229]. This example uses $1/n$.

```
tau = 1/size(assetRetns.Variables, 1);
C = tau*Sigma;
```

Market Implied Equilibrium Return

In the absence of any views, the equilibrium returns are likely equal to the implied returns from the equilibrium portfolio holding. In practice, the applicable equilibrium portfolio holding can be any optimal portfolio that the investment analyst would use in the absence of additional views on the market, such as the portfolio benchmark, an index, or even the current portfolio [2 on page 4-229]. In this example, you use linear regression to find a market portfolio that tracks the returns of the DJI benchmark. Then, you use the market portfolio as the equilibrium portfolio and the equilibrium returns are implied from the market portfolio. The `findMarketPortfolioAndImpliedReturn` function, defined in Local Functions on page 4-226, implements the equilibrium returns. This function takes historical asset returns and benchmark returns as inputs and outputs the market portfolio and the corresponding implied returns.

```
[wtsMarket, PI] = findMarketPortfolioAndImpliedReturn(assetRetns.Variables, benchRetn.Variables)
```

Compute the Estimated Mean Return and Covariance

Use the P , q , Ω , π , and C inputs to compute the blended asset return and variance using the Black-Litterman model.

You can compute $\bar{\mu}$ and $\text{cov}(\mu)$ directly by using this matrix operation:

$$\bar{\mu} = [P^T\Omega^{-1}P + C^{-1}]^{-1}[P^T\Omega^{-1}q + C^{-1}\pi], \text{cov}(\mu) = [P^T\Omega^{-1}P + C^{-1}]^{-1}$$

```
mu_bl = (P'*(Omega\P) + inv(C)) \ ( C\PI + P'*(Omega\q));
cov_mu = inv(P'*(Omega\P) + inv(C));
```

Comparing the blended expected return from Black-Litterman model to the prior belief of expected return π , you find that the expected return from Black-Litterman model is indeed a mixture of both prior belief and investor views. For example, as shown in the table below, the prior belief assumes similar returns for MSFT and IBM, but in the blended expected return, MSFT has a higher return

than IBM by more than 4%. This difference is due to the imposed strong view that MSFT outperforms IBM by 5%.

```
table(assetNames', PI*252, mu_bl*252, 'VariableNames', ["Asset_Name", ...
    "Prior_Belief_of_Expected_Return", "Black_Litterman_Blended_Expected_Return"])
```

```
ans=7x3 table
    Asset_Name    Prior_Belief_of_Expected_Return    Black_Litterman_Blended_Expected_Return
    _____    _____    _____
    "AA"          0.19143          0.19012
    "AIG"         0.14432          0.13303
    "WMT"         0.15754          0.1408
    "MSFT"        0.14071          0.17557
    "BA"          0.21108          0.2017
    "GE"          0.13323          0.12525
    "IBM"         0.14816          0.12877
```

Portfolio Optimization and Results

The Portfolio object in Financial Toolbox™ implements the Markowitz mean variance portfolio optimization framework. Using a Portfolio object, you can find the efficient portfolio for a given risk or return level, and you can also maximize the Sharpe ratio.

Use estimateMaxSharpeRatio with the Portfolio object to find allocations with the maximum Sharpe ratio for the following portfolios:

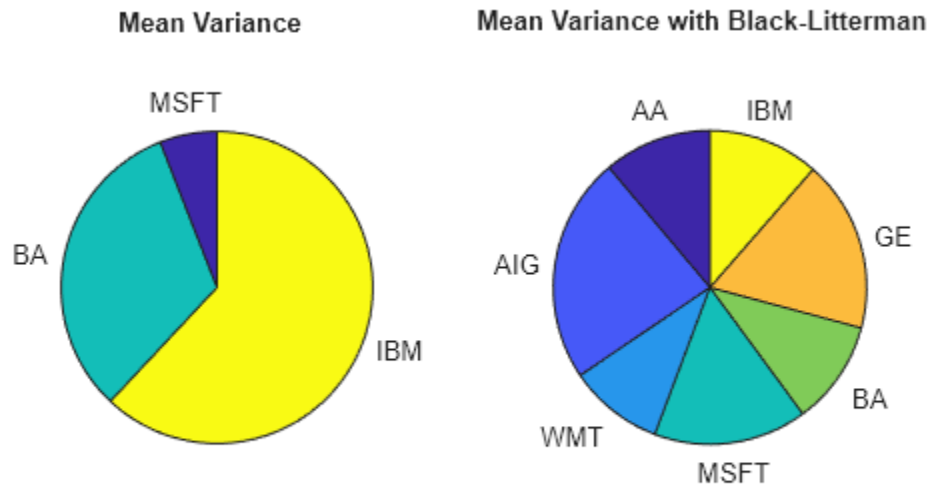
- Portfolio with asset mean and covariance from historical asset returns
- Portfolio with blended asset return and covariance from the Black-Litterman model

```
port = Portfolio('NumAssets', numAssets, 'lb', 0, 'budget', 1, 'Name', 'Mean Variance');
port = setAssetMoments(port, mean(assetRetns.Variables), Sigma);
wts = estimateMaxSharpeRatio(port);
```

```
portBL = Portfolio('NumAssets', numAssets, 'lb', 0, 'budget', 1, 'Name', 'Mean Variance with Bl
portBL = setAssetMoments(portBL, mu_bl, Sigma + cov_mu);
wtsBL = estimateMaxSharpeRatio(portBL);
```

```
ax1 = subplot(1,2,1);
idx = wts>0.001;
pie(ax1, wts(idx), assetNames(idx));
title(ax1, port.Name, 'Position', [-0.05, 1.6, 0]);
```

```
ax2 = subplot(1,2,2);
idx_BL = wtsBL>0.001;
pie(ax2, wtsBL(idx_BL), assetNames(idx_BL));
title(ax2, portBL.Name, 'Position', [-0.05, 1.6, 0]);
```



```
table(assetNames', wts, wtsBL, 'VariableNames', ["AssetName", "Mean_Variance", ...
    "Mean_Variance_with_Black_Litterman"])
```

```
ans=7x3 table
  AssetName  Mean_Variance  Mean_Variance_with_Black_Litterman
  _____  _____  _____
  "AA"        5.9935e-12        0.1115
  "AIG"       5.2227e-13        0.23314
  "WMT"       1.6889e-13       0.098048
  "MSFT"       0.059393        0.15824
  "BA"         0.32068         0.10748
  "GE"        6.8239e-11        0.1772
  "IBM"        0.61993         0.11439
```

When you use the values for the blended asset return and the covariance from the Black-Litterman model in a mean-variance optimization, the optimal allocations reflect the views of the investment analyst directly. The allocation from the Black-Litterman model is more diversified, as the pie chart shows. Also, the weights among the assets in the Black-Litterman model agree with the investment analyst views. For example, when you compare the Black-Litterman result with the plain mean-variance optimization result, you can see that the Black-Litterman result is more heavily invested in MSFT than in IBM. This is because the investment analyst has a strong view that MSFT will outperform IBM.

Local Functions

```
function [wtsMarket, PI] = findMarketPortfolioAndImpliedReturn(assetRetn, benchRetn)
% Find the market portfolio that tracks the benchmark and its corresponding implied expected return
```

The implied return is calculated by reverse optimization. The risk-free rate is assumed to be zero. The general formulation of a portfolio optimization is given by the Markowitz optimization problem:

$\arg\max_{\omega} \omega^T \mu - \frac{\delta}{2} \omega^T \Sigma \omega$. Here ω is an N -element vector of asset weights, μ is an N -element vector of expected asset returns, Σ is the N -by- N covariance matrix of asset returns, and δ is a positive risk aversion parameter. Given δ , in the absence of constraints, a closed form solution to this problem is $\omega = \frac{1}{\delta} \Sigma^{-1} \mu$. Therefore, with a market portfolio, the implied expected return is $\pi = \delta \Sigma \omega_{mkt}$.

To compute an implied expected return, you need Σ , ω_{mkt} , δ .

1) Find Σ .

Σ is calculated from historical asset returns.

```
Sigma = cov(assetRetn);
```

2) Find the market portfolio.

To find the market portfolio, regress against the DJI. The imposed constraints are fully invested and

long only: $\sum_{i=1}^n \omega_i = 1, 0 \leq \omega_i, \forall i \in \{1, \dots, n\}$

```
numAssets = size(assetRetn,2);
LB = zeros(1,numAssets);
Aeq = ones(1,numAssets);
Beq = 1;
opts = optimoptions('lsqlin','Algorithm','interior-point','Display','off');
wtsMarket = lsqlin(assetRetn, benchRetn, [], [], Aeq, Beq, LB, [], [], opts);
```

3) Find δ .

Multiply both sides of $\pi = \delta \Sigma \omega_{mkt}$ with ω_{mkt}^T to output $\delta = \frac{\text{SharpeRatio}}{\sigma_m}$. Here, the Benchmark is assumed to be maximizing the Sharpe ratio and the corresponding value is used as market Sharpe ratio. Alternatively, you can calibrate an annualized Sharpe ratio to be 0.5, which leads to $\text{shpr}=0.5/\text{sqrt}(252)$ [1 on page 4-229]. σ_m is the standard deviation of the market portfolio.

```
shpr = mean(benchRetn)/std(benchRetn);
delta = shpr/sqrt(wtsMarket'*Sigma*wtsMarket);
```

4) Compute the implied expected return.

Assuming that the market portfolio maximizes the Sharpe ratio, the implied return, without the effects from constraints, is computed directly as $\pi = \delta \Sigma \omega$.

```
PI = delta*Sigma*wtsMarket;
end
```

Appendix: Black-Litterman Model Under a Bayesian Framework

Assumptions and Views

Assume that the investment universe is composed of k assets and the vector of asset returns $\mathbf{r}$ is modeled as a random variable, following a multivariate normal distribution $\mathbf{r} \sim N(\mu, \Sigma)$. Σ is the covariance from historical asset returns. The unknown model parameter is the expected return μ . From the perspective of Bayesian statistics, the Black-Litterman model attempts to estimate μ by

combining the investment analyst views (or "observations of the future") and some prior knowledge about μ .

In addition, assume the prior knowledge that μ is a normally distributed random variable $\mu \sim N(\pi, C)$ [1, 2 on page 4-229]. In the absence of any views (observations), the prior mean π is likely to be the equilibrium returns, implied from the equilibrium portfolio holding. In practice, the applicable equilibrium portfolio holding is not necessarily the equilibrium portfolio, but rather a target optimal portfolio that the investment analyst would use in the absence of additional views on the market, such as the portfolio benchmark, an index, or even the current portfolio. C represents the uncertainty in the prior and the Black-Litterman model makes the assumption that the structure of C is $\tau\Sigma$. τ is a small constant, and many authors use different values. A detailed discussion about τ can be found in [3 on page 4-229].

Observations are necessary to perform a statistical inference on μ . In the Black-Litterman model, the observations are views about future asset returns expressed at the portfolio level. A view is the expected return of a portfolio composed of the universe of k assets. Usually, the portfolio return has uncertainty, so an error term is added to catch the departure. Assume that there is a total of v views. For a view i , $\mathbf{p}_i$ is a row vector with dimension $1 \times k$, and q_i is a scalar [2 on page 4-229].

$$q_i = E[\mathbf{p}_i^* \mathbf{r} \mid \mu] + \varepsilon_i, \quad i = 1, 2, \dots, v$$

You can stack the v views vertically, and Ω is the covariance of the uncertainties from all views. Assume that the uncertainties are independent.

$$\mathbf{q} = E[\mathbf{P}^* \mathbf{r} \mid \mu] + \varepsilon, \quad \varepsilon \sim N(0, \Omega), \quad \Omega = \text{diag}(\omega_1, \omega_2, \dots, \omega_v).$$

Note that Ω does not necessarily need to be a diagonal matrix. The investment analyst can choose the structure of Ω to account for their uncertainties in the views [4 on page 4-229].

Under the previous assumption $\mathbf{r} \sim N(\mu, \Sigma)$, it follows that

$$\mathbf{q} = \mathbf{P}^* \mu + \varepsilon, \quad \varepsilon \sim N(0, \Omega), \quad \Omega = \text{diag}(\omega_1, \omega_2, \dots, \omega_v).$$

The Bayesian Definition of the Black-Litterman Model

Based on Bayesian statistics, it is known that: posterior $\propto$ likelihood * prior.

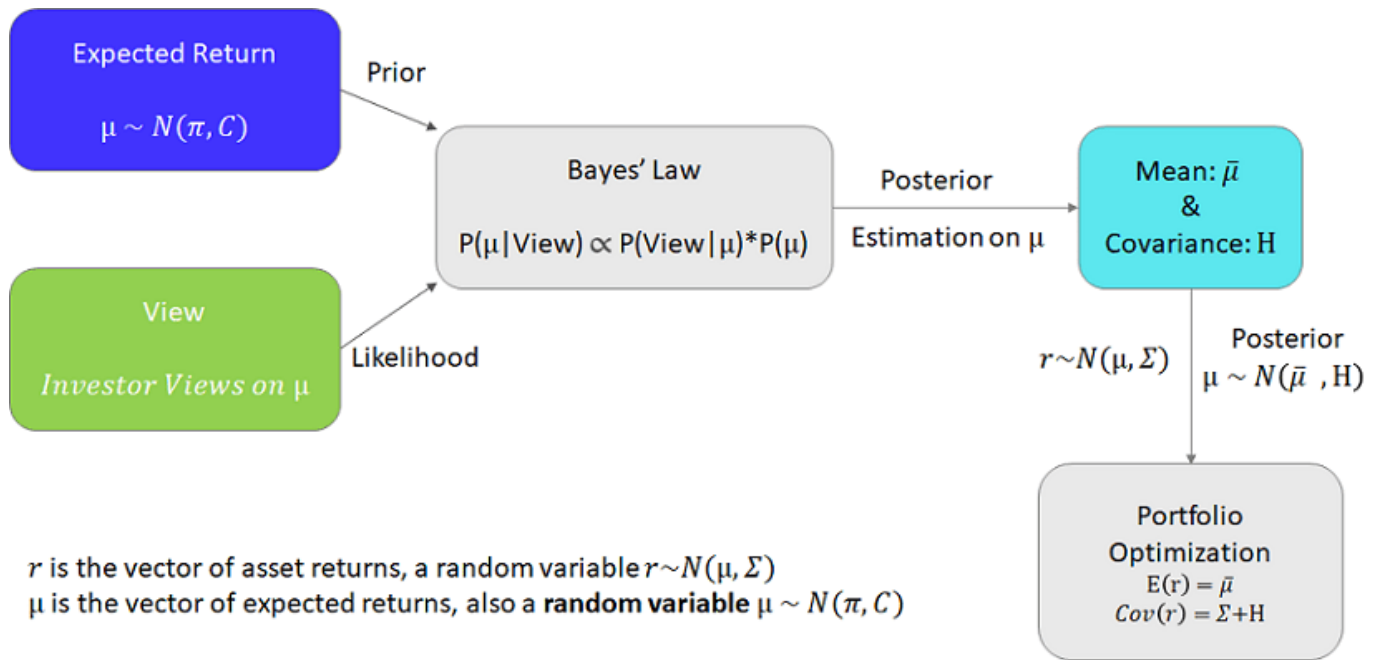


Figure 1. Black-Litterman Model

In the context of the Black-Litterman model, posterior $\propto$ likelihood * prior is expressed as $f(\mu | q) \propto f(q | \mu) * f(\mu)$, where each Bayesian term is defined as follows [2 on page 4-229]:

- The *likelihood* is how likely it is for the views to happen given μ and is expressed as $f(q | \mu) \propto \exp\left[-\frac{1}{2}(P\mu - q)' \Omega^{-1}(P\mu - q)\right]$.
- The *prior* assumes the prior knowledge that $\mu \sim N(\pi, C)$ and is expressed as $f(\mu) \propto \exp\left[-\frac{1}{2}(\mu - \pi)' C^{-1}(\mu - \pi)\right]$.
- The *posterior* is the distribution of μ given views and is expressed as $f(\mu | q) \propto \exp\left[-\frac{1}{2}(P\mu - q)' \Omega^{-1}(P\mu - q) - \frac{1}{2}(\mu - \pi)' C^{-1}(\mu - \pi)\right]$.

As previously stated, the posterior distribution of μ is also a normal distribution. By completing the squares, you can derive the posterior mean and covariance as

$$\bar{\mu} = [P^T \Omega^{-1} P + C^{-1}]^{-1} [P^T \Omega^{-1} q + C^{-1} \pi], \text{ cov}(\mu) = [P^T \Omega^{-1} P + C^{-1}]^{-1}.$$

Finally, by combining the Bayesian posterior distribution of μ and the model of asset returns $r \sim N(\mu, \Sigma)$, you then have the posterior prediction of asset returns as $r \sim N(\bar{\mu}, \Sigma + \text{cov}(\mu))$.

References

- 1 Walters, J. "The Black-Litterman Model in Detail." 2014. Available at SSRN: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=1314585.
- 2 Kolm, P. N., and Ritter, G. "On the Bayesian Interpretation of Black-Litterman." *European Journal of Operational Research*. Vol. 258, Number 2, 2017, pp. 564-572.

- 3 Attilio, M. "Beyond Black-Litterman in Practice: A Five-Step Recipe to Input Views on Non-Normal Markets." 2006. Available at SSRN: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=872577.
- 4 Ulf, H. "Computing implied returns in a meaningful way." *Journal of Asset Management*. Vol 6, Number 1, 2005, pp. 53-64.

See Also

Portfolio | setBounds | addGroups | setAssetMoments | estimateAssetMoments | estimateBounds | plotFrontier | estimateFrontierLimits | estimateFrontierByRisk | estimatePortRisk

Related Examples

- "Creating the Portfolio Object" on page 4-25
- "Working with Portfolio Constraints Using Defaults" on page 4-58
- "Validate the Portfolio Problem for Portfolio Object" on page 4-94
- "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
- "Estimate Efficient Frontiers for Portfolio Object" on page 4-122
- "Postprocessing Results to Set Up Tradable Portfolios" on page 4-130
- "Portfolio Optimization with Semicontinuous and Cardinality Constraints" on page 4-190
- "Portfolio Optimization Against a Benchmark" on page 4-202
- "Portfolio Optimization Examples Using Financial Toolbox" on page 4-161
- "Portfolio Optimization Using Factor Models" on page 4-231
- "Portfolio Optimization Using Social Performance Measure" on page 4-264
- "Diversify Portfolios Using Custom Objective" on page 4-331

More About

- "Portfolio Object" on page 4-20
- "Portfolio Optimization Theory" on page 4-4
- "Portfolio Object Workflow" on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Portfolio Optimization Using Factor Models

This example shows two approaches for using a factor model to optimize asset allocation under a mean-variance framework. Multifactor models are often used in risk modeling, portfolio management, and portfolio performance attribution. A multifactor model reduces the dimension of the investment universe and is responsible for describing most of the randomness of the market [1 on page 4-236]. The factors can be statistical, macroeconomic, and fundamental. In the first approach in this example, you build statistical factors from asset returns and optimize the allocation directly against the factors. In the second approach you use the given factor information to compute the covariance matrix of the asset returns and then use the `Portfolio` class to optimize the asset allocation.

Load Data

Load a simulated data set, which includes asset returns total $p = 100$ assets and 2000 daily observations.

```
load('asset_return_100_simulated.mat');
[nObservation, p] = size(stockReturns)
```

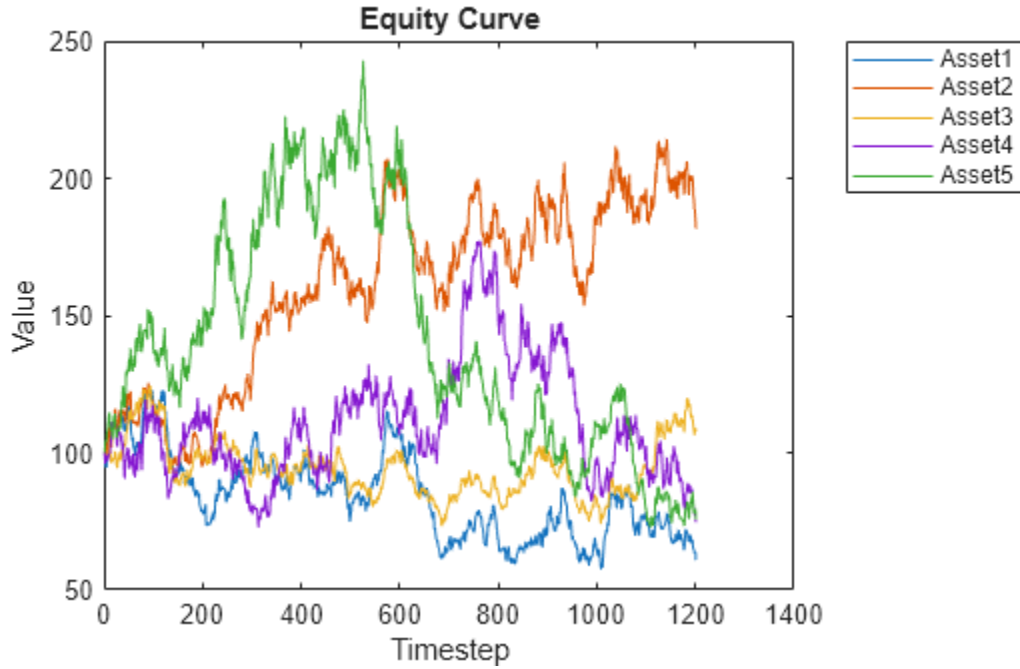
```
nObservation =
2000
```

```
p =
100
```

```
splitPoint = ceil(nObservation*0.6);
training = 1:splitPoint;
test = splitPoint+1:nObservation;
trainingNum = numel(training);
```

Visualize the equity curve for each stock. For this example, plot the first five stocks.

```
plot(ret2tick(stockReturns{training,1:5}, 'method', 'simple')*100); hold off;
xlabel('Timestep');
ylabel('Value');
title('Equity Curve');
legend(stockReturns.Properties.VariableNames(1:5), 'Location', 'bestoutside', 'Interpreter', 'none')
```



Optimize the Asset Allocation Directly Against Factors with a Problem-Based Definition Framework

For the factors, you can use statistical factors extracted from the asset return series. In this example, you use principal component analysis (PCA) to extract statistical factors [1 on page 4-236]. You can then use this factor model to solve the portfolio optimization problem.

With a factor model, p asset returns can be expressed as a linear combination of k factor returns, $r_a = \mu_a + F r_f + \varepsilon_a$, where $k \ll p$. In the mean-variance framework, portfolio risk is

$$\text{Var}(R_p) = \text{Var}(r_a^T w_a) = \text{Var}((\mu_a + F r_f + \varepsilon_a)^T w_a) = w_a^T (F \Sigma_f F^T + D) w_a = w_f^T \Sigma_f w_f + w_a^T D w_a, \text{ with } w_f = F^T w_a,$$

where:

R_p is the portfolio return (a scalar).

r_a is the asset return.

μ_a is the mean of asset return.

F is the factor loading, with the dimensions p -by- k .

r_f is the factor return.

ε_a is the idiosyncratic return related to each asset.

w_a is the asset weight.

w_f is the factor weight.

Σ_f is the covariance of factor returns.

D is the variance of idiosyncratic returns.

The parameters r_a , w_a , μ_a , ε_a are p -by-1 column vectors, r_f and w_f are k -by-1 column vectors, Σ_a is a p -by- p matrix, Σ_k is a k -by- k matrix, and D is a p -by- p diagonal matrix.

Therefore, the mean-variance optimization problem is formulated as

$$\max \mu_a^T w_a, \text{ s.t. } F^T w_a = w_f, w_f^T \Sigma_f w_f + w_a^T D w_a \leq t_{\text{risk}}, 0 \leq w \leq 1, e^T w_a = 1.$$

In the p dimensional space formed by p asset returns, PCA finds the most important k directions that capture the most important variations in the given returns of p assets. Usually, k is less than p . Therefore, by using PCA you can decompose the p asset returns into the k factors, which greatly reduces the dimension of the problem. The k principal directions are interpreted as factor loadings and the scores from the decomposition are interpreted as factor returns. For more information, see `pca` (Statistics and Machine Learning Toolbox™). In this example, use $k = 10$ as the number of principal components. Alternatively, you can also find the k value by defining a threshold for the total variance represented as the top k principal components. Usually 95% is an acceptable threshold.

```
k = 10;
[factorLoading, factorRetn, latent, tsq, explained, mu] = pca(stockReturns{training,:}, 'NumComponents', k);
disp(size(factorLoading))

    100     10

disp(size(factorRetn))

    1200     10
```

In the output p -by- k `factorLoading`, each column is a principal component. The asset return vector at each timestep is decomposed to these k dimensional spaces, where $k \ll p$. The output `factorRetn` is a *trainingNum*-by- k dimension.

Alternatively, you can also use the `covarianceDenoising` function to estimate a covariance matrix (`denoisedCov`) using denoising to reduce the noise and enhance the signal of the empirical covariance matrix. [1 on page 4-236] The goal of denoising is to separate the eigenvalues associated with signal from the ones associated with noise and shrink the eigenvalues associated with noise to enhance the signal. Thus, `covarianceDenoising` is a reasonable tool to identify the number of factors.

```
[denoisedCov, numFactorsDenoising] = covarianceDenoising(stockReturns{training,:});
```

The number of factors (`numFactorsDenoising`) is 6.

```
disp(numFactorsDenoising)

    6
```

Estimate the factor covariance matrix with factor returns (`factorRetn`) obtained using the `pca` function.

```
covarFactor = cov(factorRetn);
```

You can reconstruct the p asset returns for each observation using each k factor returns by following $r_a = \mu_a + F r_f + \varepsilon_a$.

Reconstruct the total 1200 observations for the training set.

```
reconReturn = factorRetn*factorLoading' + mu;
unexplainedRetn = stockReturns{training,:} - reconReturn;
```

There are unexplained asset returns ε_a because the remaining $(p - k)$ principal components are dropped. You can attribute the unexplained asset returns to the asset-specific risks represented as D .

```
unexplainedCovar = diag(cov(unexplainedRetn));
D = diag(unexplainedCovar);
```

You can use a problem-based definition framework from Optimization Toolbox™ to construct the

variables, objective, and constraints for the problem: $\max \mu_a^T w_a, s.t. F^T w_a = w_f,$

$w_f^T \Sigma_f w_f + w_a^T D w_a \leq t_{risk}, e^T w_a = 1, 0 \leq w \leq 1$. The problem-based definition framework enables you to define variables and express objective and constraints symbolically. You can add other constraints or use a different objective based on your specific problem. For more information, see “First Choose Problem-Based or Solver-Based Approach”.

```
targetRisk = 0.007; % Standard deviation of portfolio return
tRisk = targetRisk*targetRisk; % Variance of portfolio return
meanStockRetn = mean(stockReturns{training,:});
```

```
optimProb = optimproblem('Description','Portfolio with factor covariance matrix','ObjectiveSense',
wgtAsset = optimvar('asset_weight', p, 1, 'Type', 'continuous', 'LowerBound', 0, 'UpperBound', 1,
wgtFactor = optimvar('factor_weight', k, 1, 'Type', 'continuous');
```

```
optimProb.Objective = sum(meanStockRetn'.*wgtAsset);
```

```
optimProb.Constraints.asset_factor_weight = factorLoading'*wgtAsset - wgtFactor == 0;
optimProb.Constraints.risk = wgtFactor'*covarFactor*wgtFactor + wgtAsset'*D*wgtAsset <= tRisk;
optimProb.Constraints.budget = sum(wgtAsset) == 1;
```

```
x0.asset_weight = ones(p, 1)/p;
x0.factor_weight = zeros(k, 1);
opt = optimoptions("fmincon", "Algorithm", "sqp", "Display", "off", ...
'ConstraintTolerance', 1.0e-8, 'OptimalityTolerance', 1.0e-8, 'StepTolerance', 1.0e-8);
x = solve(optimProb,x0, "Options",opt);
assetWgt1 = x.asset_weight;
```

In this example, you are maximizing the portfolio return for a target risk. This is a nonlinear programming problem with a quadratic constraint and you use `fmincon` to solve this problem.

Check for asset allocations that are over 5% to determine which assets have large investment weights.

```
percentage = 0.05;
AssetName = stockReturns.Properties.VariableNames(assetWgt1>=percentage)';
Weight = assetWgt1(assetWgt1>=percentage);
T1 = table(AssetName, Weight)
```

```
T1=7x2 table
    AssetName    Weight
    _____    _____
    {'Asset9' }    0.080054
    {'Asset32'}    0.22355
    {'Asset47'}    0.11369
```

```
{'Asset57'}    0.088321
{'Asset61'}    0.068845
{'Asset75'}    0.063648
{'Asset94'}    0.22163
```

Optimize Asset Allocation Using Portfolio Class with Factor Information

If you already have the factor loading and factor covariance matrix from some other analysis or third-party provider, you can use this information to compute the asset covariance matrix and then directly run a mean-variance optimization using the `Portfolio` class. Recall that portfolio risk is

$\text{Var}(R_p) = \text{Var}((\mu_a + F r_f + \varepsilon_a)^T w_a) = w_a^T (F \Sigma_f F^T + D) w_a$, so you can obtain the covariance of the asset returns by $\Sigma_a = F \Sigma_f F^T + D$.

Use `estimateFrontierByRisk` in the `Portfolio` class to solve the optimization problem:

$\max \mu_a^T w_a$, s.t. $w_a^T \Sigma_a w_a \leq t_{\text{risk}}$, $0 \leq w_a \leq 1$, $e^T w_a = 1$. The `Portfolio` class supports has a variety of built-in constraints that you can use to describe your portfolio problem and estimate the risk and returns on the efficient frontier. For more information, see “Supported Constraints for Portfolio Optimization Using Portfolio Objects” on page 4-9.

```
covarAsset = factorLoading*covarFactor*factorLoading'+D;
port = Portfolio("AssetMean", meanStockRetn, 'AssetCovar', covarAsset, 'LowerBound', 0, 'UpperBound', 1, 'Budget', 1);
assetWgt2 = estimateFrontierByRisk(port, targetRisk);

AssetName = stockReturns.Properties.VariableNames(assetWgt2>=percentage)';
Weight = assetWgt2(assetWgt2>=percentage);
T2 = table(AssetName, Weight)
```

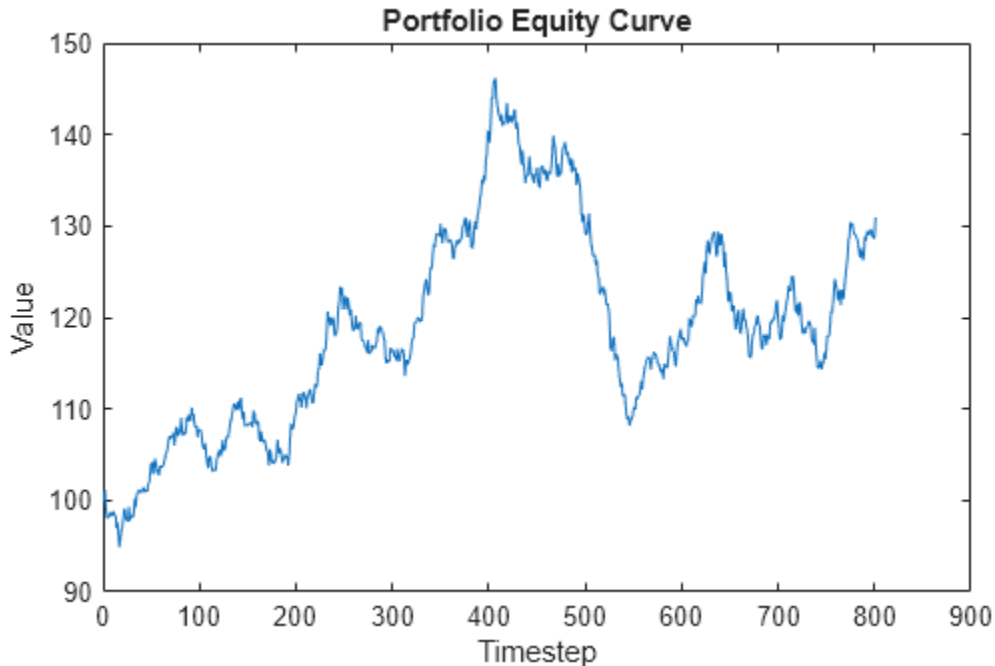
```
T2=7x2 table
    AssetName    Weight
    _____    _____
    {'Asset9' }    0.080061
    {'Asset32'}    0.22355
    {'Asset47'}    0.11369
    {'Asset57'}    0.088314
    {'Asset61'}    0.068847
    {'Asset75'}    0.063643
    {'Asset94'}    0.22163
```

Portfolio Optimization Results

Tables T1 and T2 show an identical allocation for the asset allocations that are over 5%. Therefore, in this example, both approaches to portfolio optimization with a factor model obtain asset weights that are identical.

Visualize the performance of the optimized allocation over the testing period.

```
retn = stockReturns{test, :}*assetWgt1;
plot(ret2tick(retn, 'method', 'simple')*100); hold off;
xlabel('Timestep');
ylabel('Value');
title('Portfolio Equity Curve');
```



This example demonstrates how to derive statistical factors from asset returns using PCA and then use these factors to perform a factor-based portfolio optimization. This example also shows how to use these statistical factors with the `Portfolio` class. In practice, you can adapt this example to incorporate some measurable market factors, such as industrial factors or ETF returns from various sectors, to describe the randomness in the market [1 on page 4-236]. You can define custom constraints on the weights of factors or assets with high flexibility using the problem-based definition framework from Optimization Toolbox™. Alternatively, you can work directly with the `Portfolio` class to run a portfolio optimization with various built-in constraints.

Reference

- 1 Lòpez de Prado, M. *Machine Learning for Asset Managers*. Cambridge University Press, 2020.
- 2 Meucci, A. "Modeling the Market." *Risk and Asset Allocation*. Berlin:Springer, 2009.

See Also

`Portfolio` | `setBounds` | `addGroups` | `setAssetMoments` | `estimateAssetMoments` | `estimateBounds` | `plotFrontier` | `estimateFrontierLimits` | `estimateFrontierByRisk` | `estimatePortRisk`

Related Examples

- "Creating the Portfolio Object" on page 4-25
- "Working with Portfolio Constraints Using Defaults" on page 4-58
- "Validate the Portfolio Problem for Portfolio Object" on page 4-94
- "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
- "Estimate Efficient Frontiers for Portfolio Object" on page 4-122
- "Postprocessing Results to Set Up Tradable Portfolios" on page 4-130

- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Backtest Investment Strategies Using Financial Toolbox

Perform backtesting of portfolio strategies using a backtesting framework. Backtesting is a useful tool to compare how investment strategies perform over historical or simulated market data. This example develops five different investment strategies and then compares their performance after running over a one-year period of historical stock data. The backtesting framework is implemented in two Financial Toolbox™ classes: `backtestStrategy` and `backtestEngine`.

Load Data

Load one year of adjusted price data for 30 stocks. The backtesting frameworks require adjusted asset prices, meaning prices adjusted for dividends, splits, or other events. The prices must be stored in a MATLAB® `timetable` with each column holding a time series of asset prices for an investable asset.

For this example, use one year of asset price data from the component stocks of the Dow Jones Industrial Average.

```
% Read a table of daily adjusted close prices for 2006 DJIA stocks.
T = readtable('dowPortfolio.xlsx');
```

```
% For readability, use only 15 of the 30 DJI component stocks.
assetSymbols = ["AA", "CAT", "DIS", "GM", "HPQ", "JNJ", "MCD", "MMM", "MO", "MRK", "MSFT", "PFE", "PG", "T", "X"];
```

```
% Prune the table to hold only the dates and selected stocks.
timeColumn = "Dates";
T = T(:,[timeColumn assetSymbols]);
```

```
% Convert to the table to a timetable.
pricesTT = table2timetable(T, 'RowTimes', 'Dates');
```

```
% View the structure of the prices timetable.
head(pricesTT)
```

Dates	AA	CAT	DIS	GM	HPQ	JNJ	MCD	MMM	MO
03-Jan-2006	28.72	55.86	24.18	17.82	28.35	59.08	32.72	75.93	52.27
04-Jan-2006	28.89	57.29	23.77	18.3	29.18	59.99	33.01	75.54	52.65
05-Jan-2006	29.12	57.29	24.19	19.34	28.97	59.74	33.05	74.85	52.52
06-Jan-2006	29.02	58.43	24.52	19.61	29.8	60.01	33.25	75.47	52.95
09-Jan-2006	29.37	59.49	24.78	21.12	30.17	60.38	33.88	75.84	53.11
10-Jan-2006	28.44	59.25	25.09	20.79	30.33	60.49	33.91	75.37	53.04
11-Jan-2006	28.05	59.28	25.33	20.61	30.88	59.91	34.5	75.22	53.31
12-Jan-2006	27.68	60.13	25.41	19.76	30.57	59.63	33.96	74.57	53.23

```
% View the size of the asset price data set.
numSample = size(pricesTT.Variables, 1);
numAssets = size(pricesTT.Variables, 2);
table(numSample, numAssets)
```

```
ans=1x2 table
    numSample    numAssets
    _____    _____
```

Define the Strategies

Investment strategies capture the logic used to make asset allocation decisions while a backtest is running. As the backtest runs, each strategy is periodically given the opportunity to update its portfolio allocation based on the trailing market conditions, which it does by setting a vector of asset weights. The asset weights represent the percentage of available capital invested into each asset, with each element in the weights vector corresponding to the respective column in the asset `pricesTT` timetable. If the sum of the weights vector is 1, then the portfolio is fully invested.

In this example, there are five backtest strategies. The backtest strategies assign asset weights using the following criteria:

- Equal-weighted

$$\omega_{EW} = (\omega_1, \omega_2, \dots, \omega_N), \omega_i = \frac{1}{N}$$

- Maximization of Sharpe ratio

$\omega_{SR} = \operatorname{argmax}_{\omega} \left\{ \frac{r'\omega}{\sqrt{\omega'Q\omega}} \mid \omega \geq 0, \sum_1^N \omega_i = 1, 0 \leq \omega \leq 0.1 \right\}$, where r is a vector of expected returns and Q is the covariance matrix of asset returns.

- Inverse variance

$\omega_{IV} = (\omega_1, \omega_2, \dots, \omega_N), \omega_i = \frac{(\sigma_{ii}^{-1})}{\sum_{i=1}^N \sigma_{ii}^{-1}}$, where σ_{ii} are diagonal elements of the asset return covariance matrix.

- Markowitz portfolio optimization (maximizing return and minimizing risk with fixed risk-aversion coefficient)

$R_{Mkwtz} = \max_{\omega} \left\{ r'\omega - \lambda \omega'Q\omega \mid \omega \geq 0, \sum_1^N \omega_i = 1, 0 \leq \omega \leq 0.1 \right\}$, where λ is the risk-aversion coefficient.

- Robust optimization with uncertainty in expected returns
- The robust portfolio optimization strategy, in contrast to the deterministic Markowitz formulation, takes into consideration the uncertainty expected returns of the assets and their variances and covariances. Instead of modeling unknown values (for example, expected returns) as one point, typically represented by the mean value calculated from the past, unknowns are specified as a set of values that contain the most likely possible realizations, $r = \{r \mid r \in S(r_0)\}$.

In this case, the expected return is defined not by the deterministic vector r_0 but by the region $S(r_0)$ around the vector r_0 .

Taking this into consideration, there are several ways to reformulate the portfolio optimization problem. One of the most frequently used methods is to formulate the problem as a problem of finding the maximum and minimum:

$$R_{\text{robust}} = \max_{\omega} \min_{r \in S(r_0)} \left\{ r'\omega - \lambda \omega'Q\omega \mid \omega \geq 0, \sum_1^N \omega_i = 1, 0 \leq \omega \leq 0.1 \right\}$$

In this example, the region of uncertainty $S(r_0)$ is specified as an ellipsoid:

$$S(r_0) = \{r | (r - r_0)' \Sigma_r^{-1} (r - r_0) \leq \kappa^2\}$$

Here, κ - is the uncertainty aversion coefficient that defines how wide the uncertainty region is, and Σ_r is the matrix of estimation errors in expected returns r .

With the addition of the ellipsoid uncertainty to the Markowitz model, the robust optimization problem is reformulated as:

$$R_{\text{robust}} = \max_{\omega} \{r' \omega - \lambda \omega' Q \omega - \kappa z \mid \omega \geq 0, z \geq 0, \omega' \Sigma_r \omega - z^2 \leq 0, \sum_1^N \omega_i = 1, 0 \leq \omega \leq 0.1\}$$

Implement the Strategy Rebalance Functions

The core logic of each strategy is implemented in a rebalance function. A rebalance function is a user-defined MATLAB® function that specifies how a strategy allocates capital in a portfolio. The rebalance function is an input argument to `backtestStrategy`. The rebalance function must implement the following fixed signature:

```
function new_weights = allocationFunctionName(current_weights,
pricesTimetable)
```

This fixed signature is the API that the backtest framework uses when rebalancing a portfolio. As the backtest runs, the backtesting engine calls the rebalance function of each strategy, passing in these inputs:

- `current_weights` — Current portfolio weights before rebalancing
- `pricesTimetable` — MATLAB® timetable object containing the rolling window of asset prices.

The `backtestStrategy` rebalance function uses this information to compute the desired new portfolio weights, which are returned to the backtesting engine in the function output `new_weights`. See the Local Functions on page 4-247 sections for the rebalance function for each of the five strategies.

Compute Initial Strategy Weights

Use the strategy rebalance functions to compute the initial weights for each strategy. Setting the initial weights is important because otherwise the strategies begin the backtest with 100% in cash, earning the risk-free rate, until the first rebalance date.

This example uses the first 40 days of the data set (about 2 months) to initialize the strategies. The backtest is then run over the remaining data (about 10 months).

```
warmupPeriod = 40;
```

The initial weights are calculated by calling the `backtestStrategy` rebalance function in the same way that the backtesting engine will call it. To do so, pass in a vector of current weights (all zeros, that is 100% cash) as well as a window of price data that the strategies will use to set the desired weights (the warm-up data partition). Using the rebalance functions to compute the initial weights in this way is not required. The initial weights are a vector of the initial portfolio weights and can be set to any appropriate value. The rebalance functions in this example approximate the state the strategies would be in had they been already running at the start of the backtest.

```
% No current weights (100% cash position).
current_weights = zeros(1,numAssets);
```

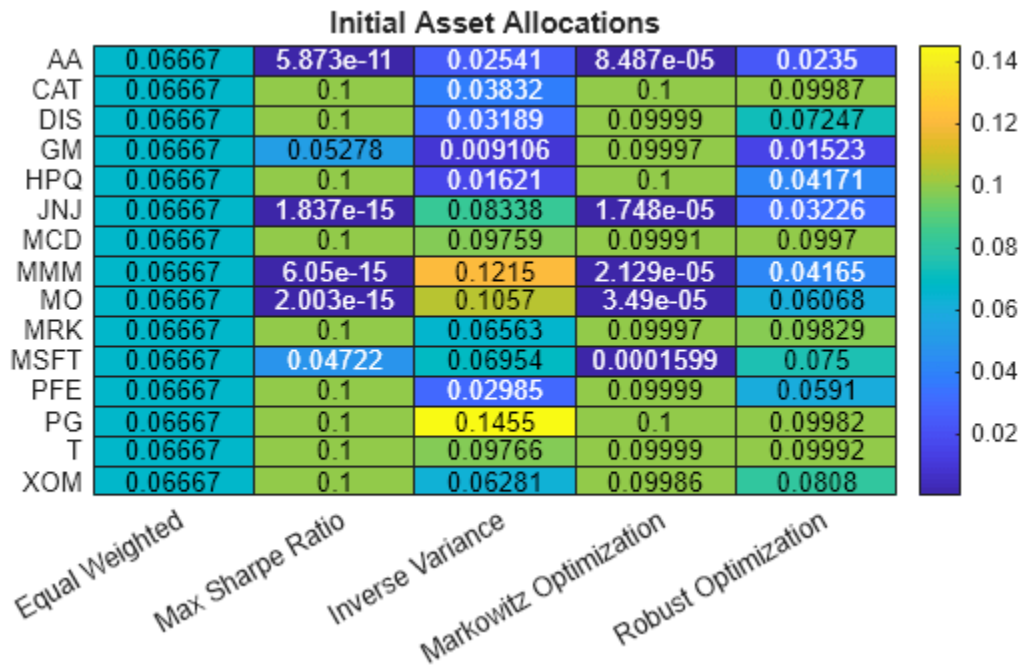


```
% Warm-up partition of data set timetable.
warmupTT = pricesTT(1:warmupPeriod,:);
```

```
% Compute the initial portfolio weights for each strategy.
equalWeight_initial = equalWeightFcn(current_weights,warmupTT);
maxSharpeRatio_initial = maxSharpeRatioFcn(current_weights,warmupTT);
inverseVariance_initial = inverseVarianceFcn(current_weights,warmupTT);
markowitz_initial = markowitzFcn(current_weights,warmupTT);
robustOptim_initial = robustOptimFcn(current_weights,warmupTT);
```

Visualize the initial weight allocations from the strategies.

```
strategyNames = {'Equal Weighted', 'Max Sharpe Ratio', 'Inverse Variance', 'Markowitz Optimization', 'Robust Optimization'};
assetSymbols = pricesTT.Properties.VariableNames;
initialWeights = [equalWeight_initial(:), maxSharpeRatio_initial(:), inverseVariance_initial(:), markowitz_initial(:), robustOptim_initial(:)];
heatmap(strategyNames, assetSymbols, initialWeights, 'title','Initial Asset Allocations','Colormap',cmap)
```



Create Backtest Strategies

To use the strategies in the backtesting framework, you must build `backtestStrategy` objects, one for each strategy. The `backtestStrategy` function takes as input the strategy name and rebalancing function for each strategy. Additionally, the `backtestStrategy` can take a variety of name-value pair arguments to specify various options. For more information on creating backtest strategies, see `backtestStrategy`.

Set the rebalance frequency and lookback window size are set in terms of number of time steps (that is, rows of the `pricesTT` timetable). Since the data is daily price data, specify the rebalance frequency and rolling lookback window in days.

```
% Rebalance approximately every 1 month (252 / 12 = 21).
rebalFreq = 21;
```

```
% Set the rolling lookback window to be at least 40 days and at most 126
% days (about 6 months).
lookback = [40 126];

% Use a fixed transaction cost (buy and sell costs are both 0.5% of amount
% traded).
transactionsFixed = 0.005;

% Customize the transaction costs using a function. See the
% variableTransactionCosts function below for an example.
transactionsVariable = @variableTransactionCosts;

% The first two strategies use fixed transaction costs. The equal-weighted
% strategy does not require a lookback window of trailing data, as its
% allocation is fixed.
strat1 = backtestStrategy('Equal Weighted', @equalWeightFcn, ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', 0, ...
    'TransactionCosts', transactionsFixed, ...
    'InitialWeights', equalWeight_initial);

strat2 = backtestStrategy('Max Sharpe Ratio', @maxSharpeRatioFcn, ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', transactionsFixed, ...
    'InitialWeights', maxSharpeRatio_initial);

% Use variable transaction costs for the remaining strategies.
strat3 = backtestStrategy('Inverse Variance', @inverseVarianceFcn, ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', @variableTransactionCosts, ...
    'InitialWeights', inverseVariance_initial);

strat4 = backtestStrategy('Markowitz Optimization', @markowitzFcn, ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', transactionsFixed, ...
    'InitialWeights', markowitz_initial);

strat5 = backtestStrategy('Robust Optimization', @robustOptimFcn, ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', transactionsFixed, ...
    'InitialWeights', robustOptim_initial);

% Aggregate the strategy objects into an array.
strategies = [strat1, strat2, strat3, strat4, strat5];
```

Backtest the Strategies

Use the following the workflow to backtest the strategies with a `backtestEngine`.

Define Backtesting Engine

The `backtestEngine` function takes as input an array of `backtestStrategy` objects. Additionally, when using `backtestEngine`, you can set several options, such as the risk-free rate and the initial portfolio value. When the risk-free rate is specified in annualized terms, the `backtestEngine` uses

Basis property to set the day count convention. For more information on creating backtesting engines, see `backtestEngine`.

```
% Risk-free rate is 1% annualized
annualRiskFreeRate = 0.01;
```

```
% Create the backtesting engine object
backtester = backtestEngine(strategies, 'RiskFreeRate', annualRiskFreeRate)
```

```
backtester =
    backtestEngine with properties:

        Strategies: [1x5 backtestStrategy]
        RiskFreeRate: 0.0100
        CashBorrowRate: 0
        RatesConvention: "Annualized"
        Basis: 0
        InitialPortfolioValue: 10000
        DateAdjustment: "Previous"
        PayExpensesFromCash: 0
        NumAssets: []
        Returns: []
        Positions: []
        Turnover: []
        BuyCost: []
        SellCost: []
        TransactionCosts: []
        Fees: []
```

Run Backtest

Use `runBacktest` to run the backtest using the test data partition. Use the `runBacktest` name-value pair argument `'Start'` to avoid look-ahead bias (that is, "seeing the future"). Begin the backtest at the end of the "warm-up" period. Running the backtest populates the empty fields of the `backtestEngine` object with the day-by-day backtest results. For more information, see "runBacktest Processing Steps" on page 15-291.

```
backtester = runBacktest(backtester, pricesTT, 'Start', warmupPeriod)
```

```
backtester =
    backtestEngine with properties:

        Strategies: [1x5 backtestStrategy]
        RiskFreeRate: 0.0100
        CashBorrowRate: 0
        RatesConvention: "Annualized"
        Basis: 0
        InitialPortfolioValue: 10000
        DateAdjustment: "Previous"
        PayExpensesFromCash: 0
        NumAssets: 15
        Returns: [211x5 timetable]
        Positions: [1x1 struct]
        Turnover: [211x5 timetable]
        BuyCost: [211x5 timetable]
        SellCost: [211x5 timetable]
        TransactionCosts: [1x1 struct]
```

```
Fees: [1x1 struct]
```

Examine Backtest Results

Use the `summary` function to generate a table of strategy performance results for the backtest.

```
summaryByStrategies = summary(backtester)
```

```
summaryByStrategies=9x5 table
```

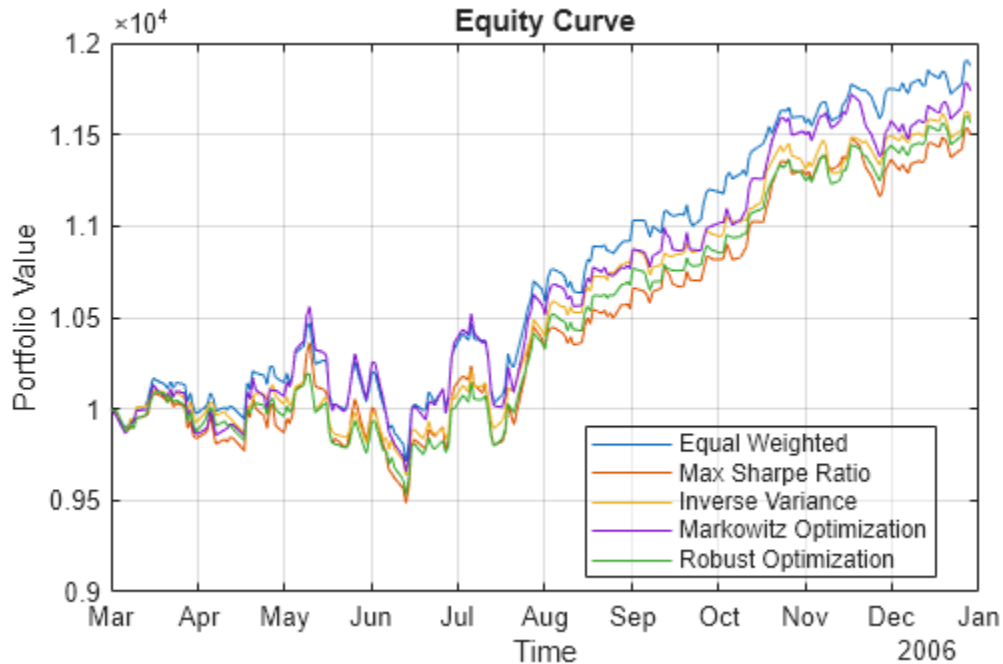
	Equal_Weighted	Max_Sharpe_Ratio	Inverse_Variance	Markowitz_Optim
TotalReturn	0.18745	0.14991	0.15906	0.17404
SharpeRatio	0.12559	0.092456	0.12179	0.10339
Volatility	0.0063474	0.0070186	0.0055626	0.0072468
AverageTurnover	0.00087623	0.0065762	0.0028666	0.0058268
MaxTurnover	0.031251	0.239	0.09114	0.21873
AverageReturn	0.00083462	0.00068672	0.0007152	0.0007868
MaxDrawdown	0.072392	0.084768	0.054344	0.08554
AverageBuyCost	0.047298	0.3449	0.15228	0.315
AverageSellCost	0.047298	0.3449	0.22842	0.315

The detailed backtest results, including the daily returns, asset positions, turnover, and fees are stored in properties of the `backtestEngine` object.

When analyzing these results, remember that the backtest engine computes the weights using *past* data. At the moment the weights are computed (with the past data), the portfolio weights obtained from the `Max_Sharpe_Ratio` strategy are the ones that achieve the maximum Sharpe ratio. After those weights are computed, they are invested in *future unseen* data. Consequently, in this summary, the `Max_Sharpe_Ratio` results are the lowest `SharpeRatio` with lowest returns. This example demonstrates that past performance is no guarantee of future results and a strategy that has performed well in the past may not perform well in the future.

Use `equityCurve` to plot the equity curve for the five different investment strategies.

```
equityCurve(backtester)
```



Transposing the summary table to make plots of certain metrics can be useful.

```
% Transpose the summary table to plot the metrics.
```

```
summaryByMetrics = rows2vars(summaryByStrategies);
```

```
summaryByMetrics.Properties.VariableNames{1} = 'Strategy'
```

```
summaryByMetrics=5x10 table
```

Strategy	TotalReturn	SharpeRatio	Volatility	AverageTurnover
{'Equal_Weighted' }	0.18745	0.12559	0.0063474	0.00087623
{'Max_Sharpe_Ratio' }	0.14991	0.092456	0.0070186	0.0065762
{'Inverse_Variance' }	0.15906	0.12179	0.0055626	0.0028666
{'Markowitz_Optimization' }	0.17404	0.10339	0.0072466	0.0058268
{'Robust_Optimization' }	0.15655	0.11442	0.0058447	0.0025172

```
% Compare the strategy turnover.
```

```
names = [backtester.Strategies.Name];
```

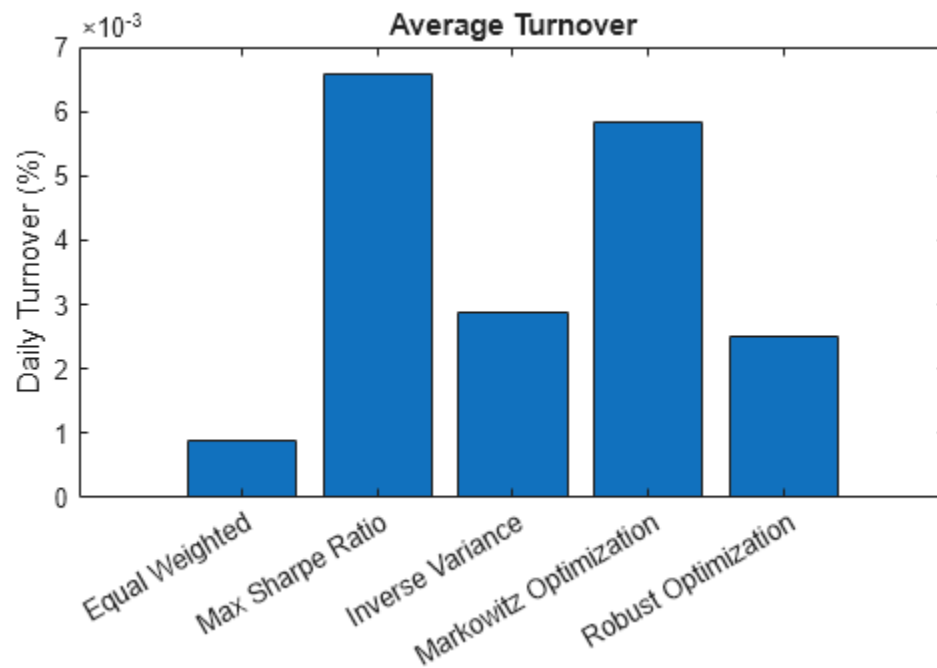
```
nameLabels = strep(names, '_', ' ');
```

```
bar(summaryByMetrics.AverageTurnover)
```

```
title('Average Turnover')
```

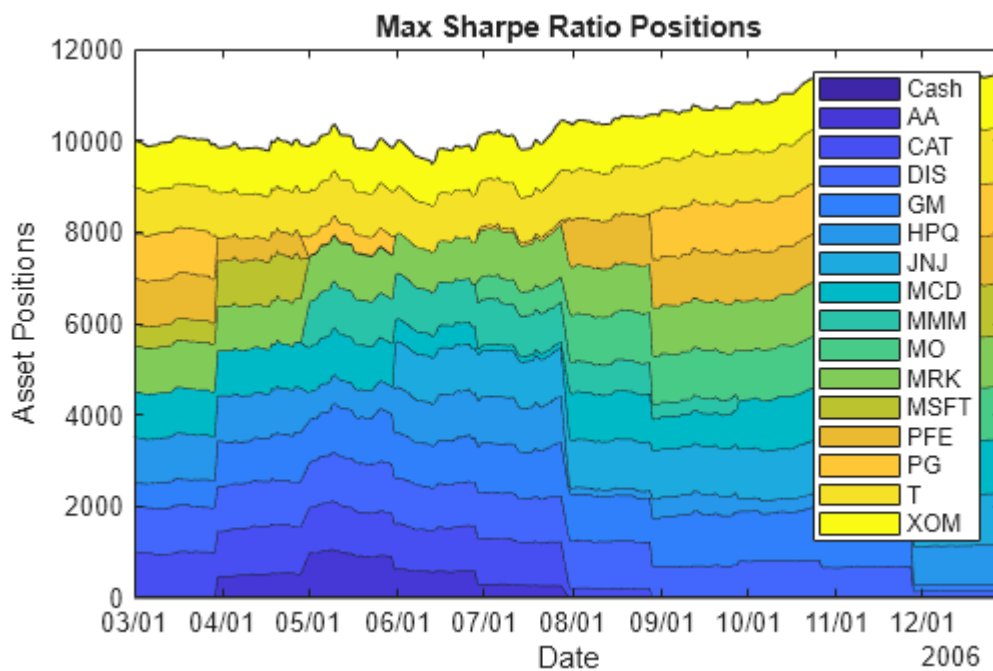
```
ylabel('Daily Turnover (%)')
```

```
set(gca, 'xticklabel', nameLabels)
```



You can visualize the change in the strategy allocations over time using an area chart of the daily asset positions. For information on the `assetAreaPlot` function, see the Local Functions on page 4-249 section.

```
strategyName =  ;
assetAreaPlot(backtester, strategyName)
```



Local Functions

The strategy rebalancing functions as well as the variable transaction cost function follow.

```
function new_weights = equalWeightFcn(current_weights, pricesTT)
% Equal-weighted portfolio allocation

nAssets = size(pricesTT, 2);
new_weights = ones(1,nAssets);
new_weights = new_weights / sum(new_weights);

end

function new_weights = maxSharpeRatioFcn(current_weights, pricesTT)
% Mean-variance portfolio allocation

nAssets = size(pricesTT, 2);
assetReturns = tick2ret(pricesTT);
% Max 25% into a single asset (including cash)
p = Portfolio('NumAssets',nAssets,...
    'LowerBound',0,'UpperBound',0.1,...
    'LowerBudget',1,'UpperBudget',1);
p = estimateAssetMoments(p, assetReturns{:,:});
new_weights = estimateMaxSharpeRatio(p);

end

function new_weights = inverseVarianceFcn(current_weights, pricesTT)
% Inverse-variance portfolio allocation

assetReturns = tick2ret(pricesTT);
assetCov = cov(assetReturns{:,:});
new_weights = 1 ./ diag(assetCov);
new_weights = new_weights / sum(new_weights);

end

function new_weights = robustOptimFcn(current_weights, pricesTT)
% Robust portfolio allocation

nAssets = size(pricesTT, 2);
assetReturns = tick2ret(pricesTT);

Q = cov(table2array(assetReturns));
SIGMAx = diag(diag(Q));

% Robust aversion coefficient
k = 1.1;

% Robust aversion coefficient
lambda = 0.05;

rPortfolio = mean(table2array(assetReturns))';

% Create the optimization problem
pRobust = optimproblem('Description','Robust Portfolio');

% Define the variables
% xRobust - x allocation vector
```

```
xRobust = optimvar('x',nAssets,1,'Type','continuous','LowerBound',0.0,'UpperBound',0.1);
zRobust = optimvar('z','LowerBound',0);

% Define the budget constraint
pRobust.Constraints.budget = sum(xRobust) == 1;

% Define the robust constraint
pRobust.Constraints.robust = xRobust'*SIGMAx*xRobust - zRobust*zRobust <=0;
pRobust.Objective = -rPortfolio'*xRobust + k*zRobust + lambda*xRobust'*Q*xRobust;
x0.x = zeros(nAssets,1);
x0.z = 0;
opt = optimoptions('fmincon','Display','off');
[solRobust,~,~] = solve(pRobust,x0,'Options',opt);
new_weights = solRobust.x;

end

function new_weights = markowitzFcn(current_weights, pricesTT)
% Robust portfolio allocation

nAssets = size(pricesTT, 2);
assetReturns = tick2ret(pricesTT);

Q = cov(table2array(assetReturns));

% Risk aversion coefficient
lambda = 0.05;

rPortfolio = mean(table2array(assetReturns))';

% Create the optimization problem
pMrkwtz = optimproblem('Description','Markowitz Mean Variance Portfolio ');

% Define the variables
% xRobust - x allocation vector
xMrkwtz = optimvar('x',nAssets,1,'Type','continuous','LowerBound',0.0,'UpperBound',0.1);

% Define the budget constraint
pMrkwtz.Constraints.budget = sum(xMrkwtz) == 1;

% Define the Markowitz objective
pMrkwtz.Objective = -rPortfolio'*xMrkwtz + lambda*xMrkwtz'*Q*xMrkwtz;
x0.x = zeros(nAssets,1);

opt = optimoptions('quadprog','Display','off');
[solMrkwtz,~,~] = solve(pMrkwtz,x0,'Options',opt);
new_weights = solMrkwtz.x;

end

function [buy, sell] = variableTransactionCosts(deltaPositions)
% Variable transaction cost function
%
% This function is an example of how to compute variable transaction costs.
%
% Compute scaled transaction costs based on the change in market value of
% each asset after a rebalance. Costs are computed at the following rates:
%
```



```

% Buys:
%   $0-$10,000 : 0.5%
%   $10,000+   : 0.35%
% Sells:
%   $0-$1,000  : 0.75%
%   $1,000+    : 0.5%

buy = zeros(1,numel(deltaPositions));
sell = zeros(1,numel(deltaPositions));

% Buys
idx = 0 < deltaPositions & deltaPositions < 1e4;
buy(idx) = 0.005 * deltaPositions(idx); % 50 basis points
idx = 1e4 <= deltaPositions;
buy(idx) = 0.0035 * deltaPositions(idx); % 35 basis points
buy = sum(buy);

% Sells
idx = -1e3 < deltaPositions & deltaPositions < 0;
sell(idx) = 0.0075 * -deltaPositions(idx); % 75 basis points
idx = deltaPositions <= -1e3;
sell(idx) = 0.005 * -deltaPositions(idx); % 50 basis points
sell = sum(sell);

end

function assetAreaPlot(backtester,strategyName)
% Plot the asset allocation as an area plot.

t = backtester.Positions.(strategyName).Time;
positions = backtester.Positions.(strategyName).Variables;
h = area(t,positions);
title(sprintf('%s Positions',strrep(strategyName,'_',' ')));
xlabel('Date');
ylabel('Asset Positions');
xtickformat('MM/dd');
xlim([t(1) t(end)])
oldylim = ylim;
ylim([0 oldylim(2)]);
cm = parula(numel(h));
for i = 1:numel(h)
    set(h(i),'FaceColor',cm(i,:));
end
legend(backtester.Positions.(strategyName).Properties.VariableNames)

end

```

See Also

[backtestStrategy](#) | [backtestEngine](#) | [runBacktest](#) | [summary](#)

Related Examples

- “Backtest Investment Strategies with Trading Signals” on page 4-251
- “Backtest Strategies Using Deep Learning” on page 4-302

- “Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data” on page 2-78
- “Backtest with Brinson Attribution to Evaluate Portfolio Performance” on page 4-315

Backtest Investment Strategies with Trading Signals

This example shows how to perform backtesting of portfolio strategies that incorporate investment signals in their trading strategy. The term *signals* includes any information that a strategy author needs to make with respect to trading decisions outside of the price history of the assets. Such information can include technical indicators, the outputs of machine learning models, sentiment data, macroeconomic data, and so on. This example uses three simple investment strategies based on derivative signal data:

- Moving average crossovers
- Moving average convergence/divergence
- Relative strength index

In this example you can run a backtest using these strategies over one year of stock data. You then analyze the results to compare the performance of each strategy.

Even though technical indicators are not typically used as standalone trading strategies, this example uses these strategies to demonstrate how to build investment strategies based on signal data when you use the `backtestEngine` object in MATLAB®.

Load Data

Load the adjusted price data for 15 stocks for the year 2006. This example uses a small set of investable assets for readability.

Read a table of daily adjusted close prices for 2006 DJIA stocks.

```
T = readtable('dowPortfolio.xlsx');
```

For readability, use only 15 of the 30 DJI component stocks.

```
symbols = ["AA", "CAT", "DIS", "GM", "HPQ", "JNJ", "MCD", "MMM", "MO", "MRK", "MSFT", "PFE", "PG", "T", "XOM"]
```

Prune the table to hold only the dates and selected stocks.

```
timeColumn = "Dates";
T = T(:,[timeColumn symbols]);
```

Convert the data to a timetable.

```
pricesTT = table2timetable(T, 'RowTimes', 'Dates');
```

View the structure of the prices timetable.

```
head(pricesTT)
```

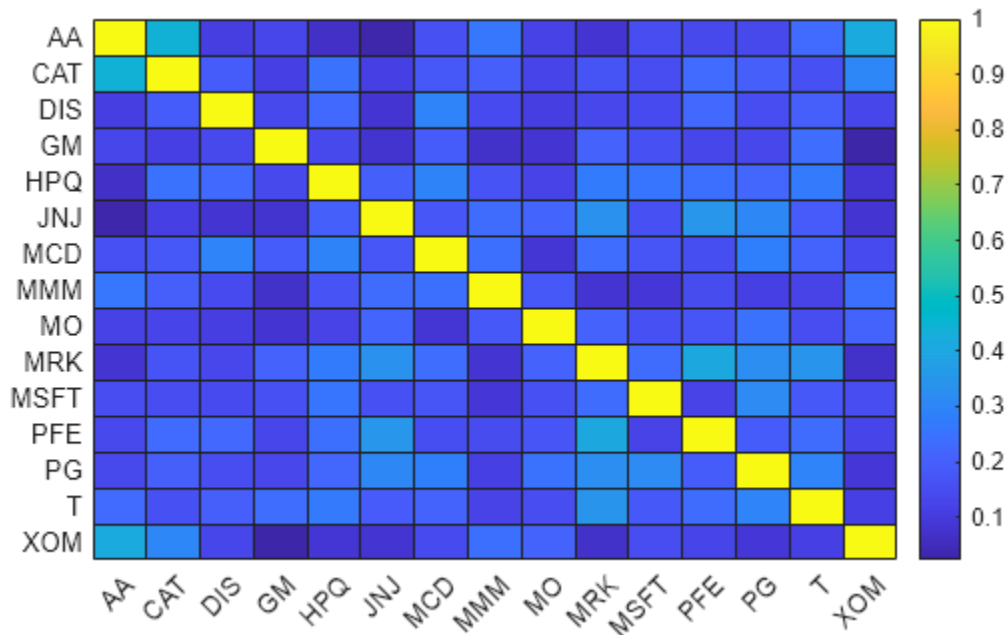
Dates	AA	CAT	DIS	GM	HPQ	JNJ	MCD	MMM	MO
03-Jan-2006	28.72	55.86	24.18	17.82	28.35	59.08	32.72	75.93	52.27
04-Jan-2006	28.89	57.29	23.77	18.3	29.18	59.99	33.01	75.54	52.65
05-Jan-2006	29.12	57.29	24.19	19.34	28.97	59.74	33.05	74.85	52.52
06-Jan-2006	29.02	58.43	24.52	19.61	29.8	60.01	33.25	75.47	52.95
09-Jan-2006	29.37	59.49	24.78	21.12	30.17	60.38	33.88	75.84	53.11
10-Jan-2006	28.44	59.25	25.09	20.79	30.33	60.49	33.91	75.37	53.04

11-Jan-2006	28.05	59.28	25.33	20.61	30.88	59.91	34.5	75.22	53.31
12-Jan-2006	27.68	60.13	25.41	19.76	30.57	59.63	33.96	74.57	53.23

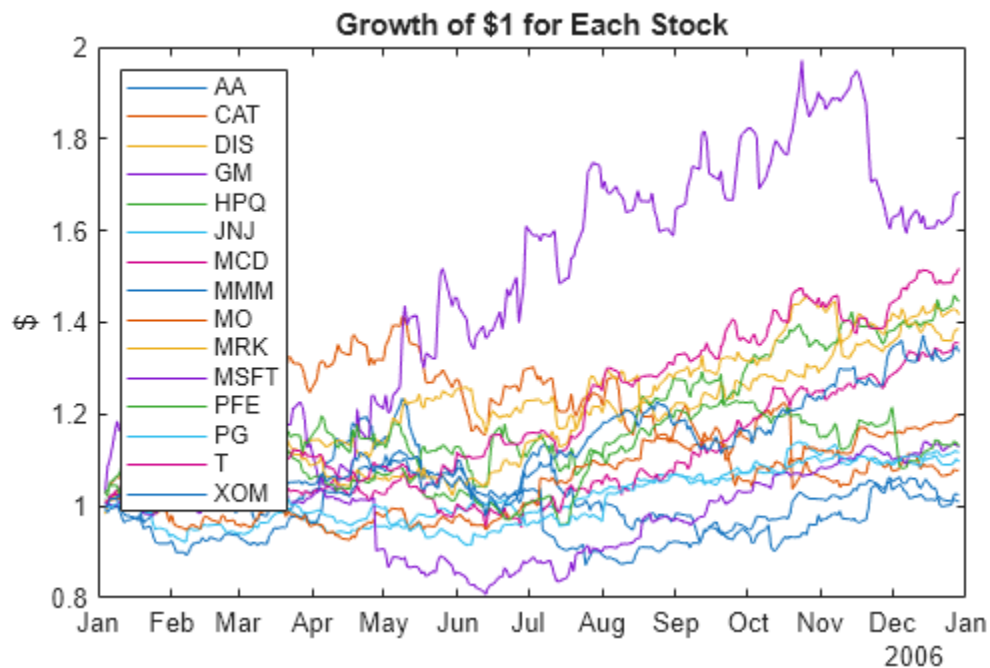
Inspect Data Set

Visualize the correlation and total return of each stock in the data set.

```
% Visualize the correlation between the 15 stocks.
returns = tick2ret(pricesTT);
stockCorr = corr(returns.Variables);
heatmap(symbols,symbols,stockCorr,'Colormap',parula);
```



```
% Visualize the performance of each stock over the range of price data.
totalRet = ret2tick(returns);
plot(totalRet.Dates,totalRet.Variables);
legend(symbols,'Location','NW');
title('Growth of $1 for Each Stock')
ylabel('$')
```



```
% Get the total return of each stock for the duration of the data set.
totalRet(end,:)
```

```
ans=1x15 timetable
      Dates      AA      CAT      DIS      GM      HPQ      JNJ      MCD      MMM
      _____  _____  _____  _____  _____  _____  _____  _____
29-Dec-2006    1.0254    1.0781    1.4173    1.6852    1.4451    1.0965    1.3548    1.0087
```

Build Signal Table

In addition to the historical adjusted asset prices, the backtesting framework allows you to optionally specify *signal* data when running a backtest. Specify the signal data in a similar way as the prices by using a MATLAB® *timetable*. The "time" dimension of the *signal* timetable must match that of the *prices* timetable — that is, the rows of each table must have matching datetime values for the Time column.

This example builds a signal timetable to support each of the three investment strategies:

- Simple moving average crossover (SMA) strategy
- Moving Average Convergence / Divergence (MACD) strategy
- Relative Strength Index (RSI) strategy

Each strategy has a timetable of signals that are precomputed. Before you run the backtest, you merge the three separate signal timetables into a single aggregate signal timetable to use for the backtest.

SMA: Simple Moving Average Crossover

The SMA indicator uses 5-day and 20-day simple moving averages to make buy and sell decisions. When the 5-day SMA crosses the 20-day SMA (moving upwards), then the stock is bought. When the 5-day SMA crosses below the 20-day SMA, the stock is sold.

```
% Create SMA timetables using the movavg function.
sma5 = movavg(pricesTT,'simple',5);
sma20 = movavg(pricesTT,'simple',20);

Create the SMA indicator signal timetable.

smaSignalNameEnding = '_SMA5over20';

smaSignal = timetable;
for i = 1:numel(symbols)
    symi = symbols(i);
    % Build a timetable for each symbol, then aggregate them together.
    smaSignal_i = timetable(pricesTT.Dates,...
        double(sma5.(symi) > sma20.(symi)),...
        'VariableNames',{sprintf('%s%s',symi,smaSignalNameEnding)});
    % Use the synchronize function to merge the timetables together.
    smaSignal = synchronize(smaSignal,smaSignal_i);
end
```

The SMA signal timetable contains an indicator with a value of 1 when the 5-day moving average is above the 20-day moving average for each asset, and a 0 otherwise. The column names for each stock indicator are [stock symbol]SMA5over20. The `backtestStrategy` object makes trading decisions based on these crossover events.

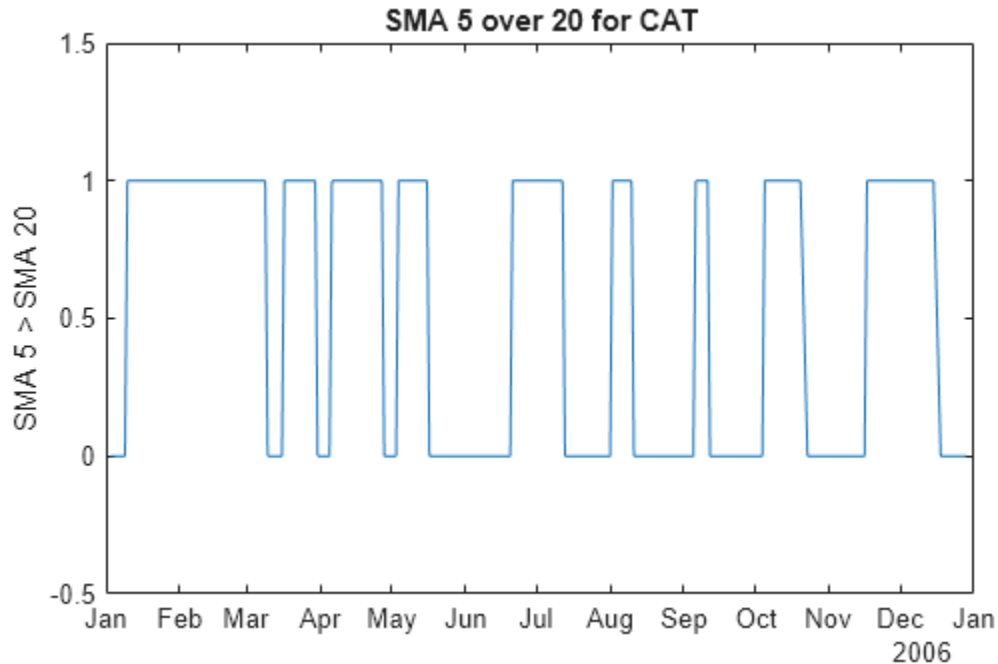
View the structure of the SMA signal timetable.

```
head(smaSignal)
```

Time	AA_SMA5over20	CAT_SMA5over20	DIS_SMA5over20	GM_SMA5over20	HPQ_SMA5over20
03-Jan-2006	0	0	0	0	0
04-Jan-2006	0	0	0	0	0
05-Jan-2006	0	0	0	0	0
06-Jan-2006	0	0	0	0	0
09-Jan-2006	1	0	0	0	1
10-Jan-2006	1	1	1	1	1
11-Jan-2006	0	1	1	1	1
12-Jan-2006	0	1	1	1	1

Plot the signal for a single asset to preview the trading frequency.

```
plot(smaSignal.Time,smaSignal.CAT_SMA5over20);
ylim([-0.5, 1.5]);
ylabel('SMA 5 > SMA 20');
title(sprintf('SMA 5 over 20 for CAT'));
```



MACD: Moving Average Convergence/Divergence

You can use the MACD metric in a variety of ways. Often, MACD is compared to its own exponential moving average, but for this example, MACD serves as a trigger for a buy signal when the MACD rises above 0. A position is sold when the MACD falls back below 0.

```
% Create a timetable of the MACD metric using the MACD function.
macdTT = macd(pricesTT);
```

Create the MACD indicator signal timetable.

```
macdSignalNameEnding = '_MACD';

macdSignal = timetable;
for i = 1:numel(symbols)
    symi = symbols(i);
    % Build a timetable for each symbol, then aggregate the symbols together.
    macdSignal_i = timetable(pricesTT.Dates,...
        double(macdTT.(symi) > 0),...
        'VariableNames',{sprintf('%s%s',symi,macdSignalNameEnding)});
    macdSignal = synchronize(macdSignal,macdSignal_i);
end
```

The MACD signal table contains a column for each asset with the name `[stock symbol]_MACD`. Each signal has a value of 1 when the MACD of the stock is above 0. The signal has a value of 0 when the MACD of the stock falls below 0.

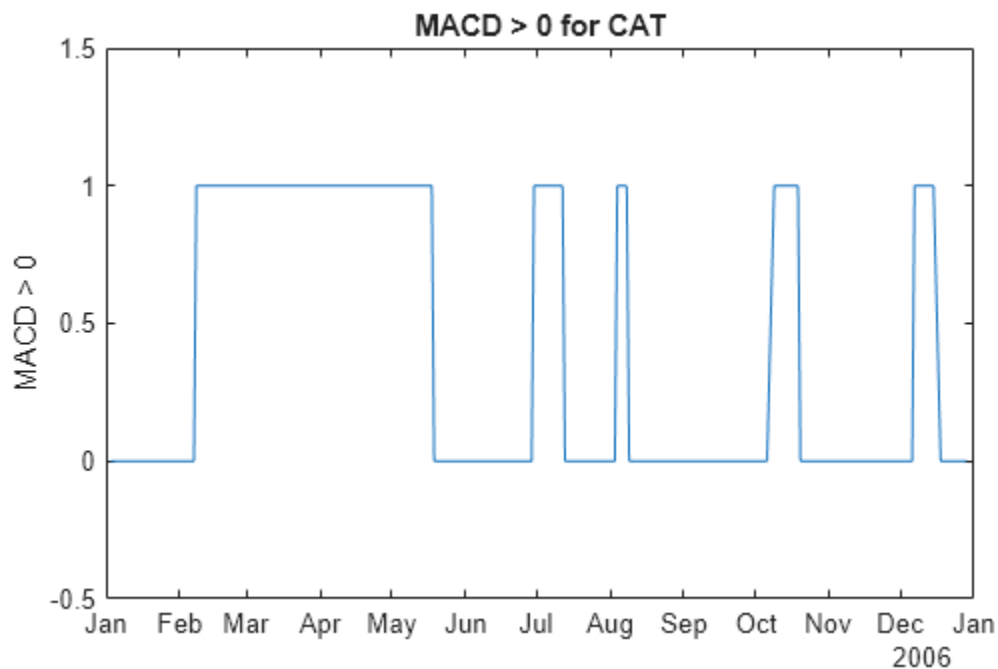
```
head(macdSignal)
```

Time	AA_MACD	CAT_MACD	DIS_MACD	GM_MACD	HPQ_MACD	JNJ_MACD	MCD_MACD
03-Jan-2006	0	0	0	0	0	0	0

04-Jan-2006	0	0	0	0	0	0	0
05-Jan-2006	0	0	0	0	0	0	0
06-Jan-2006	0	0	0	0	0	0	0
09-Jan-2006	0	0	0	0	0	0	0
10-Jan-2006	0	0	0	0	0	0	0
11-Jan-2006	0	0	0	0	0	0	0
12-Jan-2006	0	0	0	0	0	0	0

Similar to the SMA, plot the signal for a single asset to preview the trading frequency.

```
plot(macdSignal.Time,macdSignal.CAT_MACD)
ylim([-0.5, 1.5]);
ylabel('MACD > 0');
title(sprintf('MACD > 0 for CAT'));
```



RSI: Relative Strength Index

The RSI is a metric to capture momentum. A common heuristic is to buy when the RSI falls below 30 and to sell when the RSI rises above 70.

```
rsiSignalNameEnding = '_RSI';

rsiSignal = timetable;
for i = 1:numel(symbols)
    symi = symbols(i);
    rsiValues = rsindex(pricesTT.(symi));
    rsiBuySell = zeros(size(rsiValues));
    rsiBuySell(rsiValues < 30) = 1;
    rsiBuySell(rsiValues > 70) = -1;
    % Build a timetable for each symbol, then aggregate the symbols together.
    rsiSignal_i = timetable(pricesTT.Dates,...
        rsiBuySell,...
        'VariableNames',{sprintf('%s%s',symi,rsiSignalNameEnding)});
```



```

    rsiSignal = synchronize(rsiSignal,rsiSignalI);
end

```

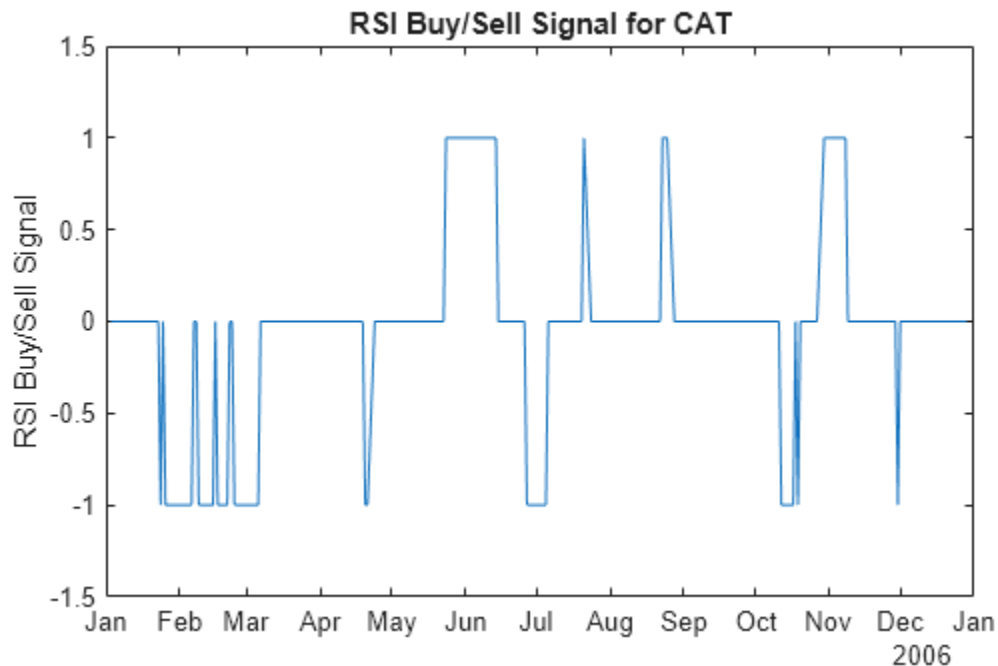
The RSI signal takes a value of 1 (indicating a buy signal) when the RSI value for the stock falls below 30. The signal takes a value of -1 (indicating a sell signal) when the RSI for the stock rises above 70. Otherwise, the signal takes a value of 0, indicating no action.

Plot the signal for a single asset to preview the trading frequency.

```

plot(rsiSignal.Time,rsiSignal.CAT_RSI)
ylim([-1.5, 1.5]);
ylabel('RSI Buy/Sell Signal');
title(sprintf('RSI Buy/Sell Signal for CAT'));

```



Build the Strategies

Build the strategies for the `backtestStrategy` object using the rebalance functions defined in the Local Functions on page 4-261 section. Each strategy uses the rebalance function to make trading decisions based on the appropriate signals.

The signals require sufficient trailing data to compute the trading signals (for example, computing the SMA20 for day X requires prices from the 20 days prior to day X). All of the trailing data is captured in the precomputed trading signals. So the actual strategies need only a 2-day lookback window to make trading decisions to evaluate when the signals cross trading thresholds.

All strategies pay 25 basis points transaction costs on buys and sells.

The initial weights are computed based on the signal values after 20 trading days. The backtest begins after this 20 day initialization period.

```
tradingCosts = 0.0025;
```

```
% Use the crossoverRebalanceFunction for both the SMA
```

```

% strategy as well as the MACD strategy. This is because they both trade
% on their respective signals in the same way (buy when signal goes from
% 0->1, sell when signal goes from 1->0). Build an anonymous
% function for the rebalance functions of the strategies that calls the
% shared crossoverRebalanceFcn() with the appropriate signal name string
% for each strategy.

% Each anonymous function takes the current weights (w), prices (p),
% and signal (s) data from the backtest engine and passes it to the
% crossoverRebalanceFcn function with the signal name string.
smaInitWeights = computeInitialWeights(smaSignal(20,:));
smaRebalanceFcn = @(w,p,s) crossoverRebalanceFcn(w,p,s,smaSignalNameEnding);
smaStrategy = backtestStrategy('SMA',smaRebalanceFcn,...
    'TransactionCosts',tradingCosts,...
    'LookbackWindow',2,...
    'InitialWeights',smaInitWeights);

macdInitWeights = computeInitialWeights(macdSignal(20,:));
macdRebalanceFcn = @(w,p,s) crossoverRebalanceFcn(w,p,s,macdSignalNameEnding);
macdStrategy = backtestStrategy('MACD',macdRebalanceFcn,...
    'TransactionCosts',tradingCosts,...
    'LookbackWindow',2,...
    'InitialWeights',macdInitWeights);

% The RSI strategy uses its signal differently, buying on a 0->1
% transition and selling on a 0->-1 transition. This logic is captured in
% the rsiRebalanceFcn function defined in the Local Functions section.
rsiInitWeights = computeInitialWeights(rsiSignal(20,:));
rsiStrategy = backtestStrategy('RSI',@rsiRebalanceFcn,...
    'TransactionCosts',tradingCosts,...
    'LookbackWindow',2,...
    'InitialWeights',rsiInitWeights);

```

Set Up Backtest

As a benchmark, this example also runs a simple equal-weighted strategy to determine if the trading signals are providing valuable insights into future returns of the assets. The benchmark strategy is rebalanced every four weeks.

```

% The equal weight strategy requires no history, so set LookbackWindow to 0.
benchmarkStrategy = backtestStrategy('Benchmark',@equalWeightFcn,...
    'TransactionCosts',tradingCosts,...
    'RebalanceFrequency',20,...
    'LookbackWindow',0);

```

Aggregate each of the individual signal timetables into a single backtest signal timetable.

```

% Combine the three signal timetables.
signalTT = timetable;
signalTT = synchronize(signalTT, smaSignal);
signalTT = synchronize(signalTT, macdSignal);
signalTT = synchronize(signalTT, rsiSignal);

```

Use `backtestEngine` to create the backtesting engine and then use `runBacktest` to run the backtest. The risk-free rate earned on uninvested cash is 1% annualized.

```

% Put the benchmark strategy and three signal strategies into an array.
strategies = [benchmarkStrategy smaStrategy macdStrategy rsiStrategy];

```

```

% Create the backtesting engine.
bt = backtestEngine(strategies, 'RiskFreeRate', 0.01)

bt =
    backtestEngine with properties:

        Strategies: [1x4 backtestStrategy]
        RiskFreeRate: 0.0100
        CashBorrowRate: 0
        RatesConvention: "Annualized"
        Basis: 0
        InitialPortfolioValue: 10000
        DateAdjustment: "Previous"
        PayExpensesFromCash: 0
        NumAssets: []
        Returns: []
        Positions: []
        Turnover: []
        BuyCost: []
        SellCost: []
        TransactionCosts: []
        Fees: []

```

Backtest Strategies

```

% Start with the end of the initial weights calculation warm-up period.
startIdx = 20;

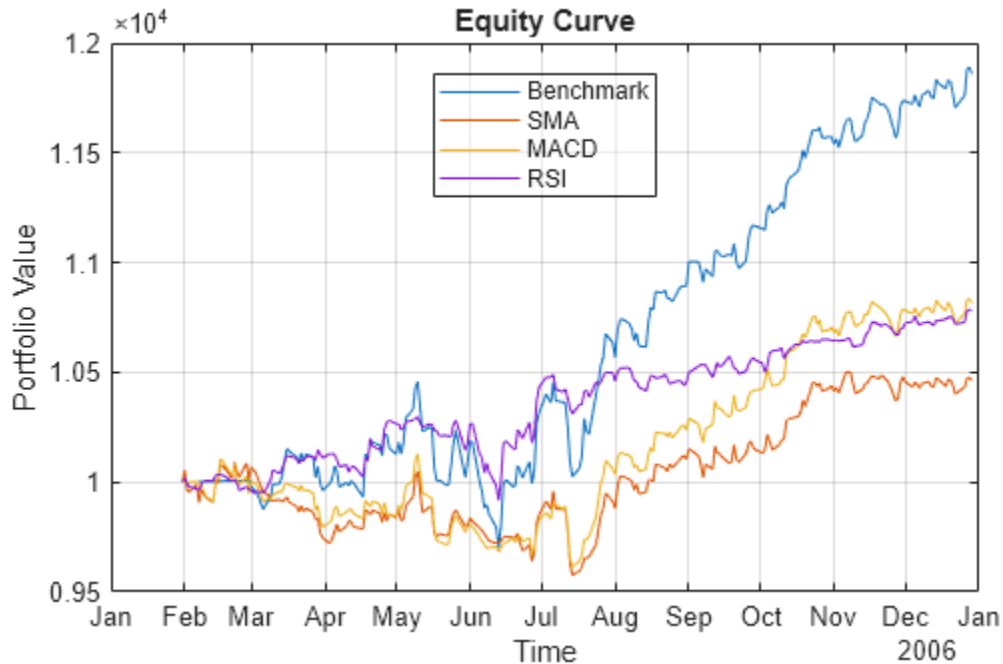
% Run the backtest.
bt = runBacktest(bt, pricesTT, signalTT, 'Start', startIdx);

```

Examine Backtest Results

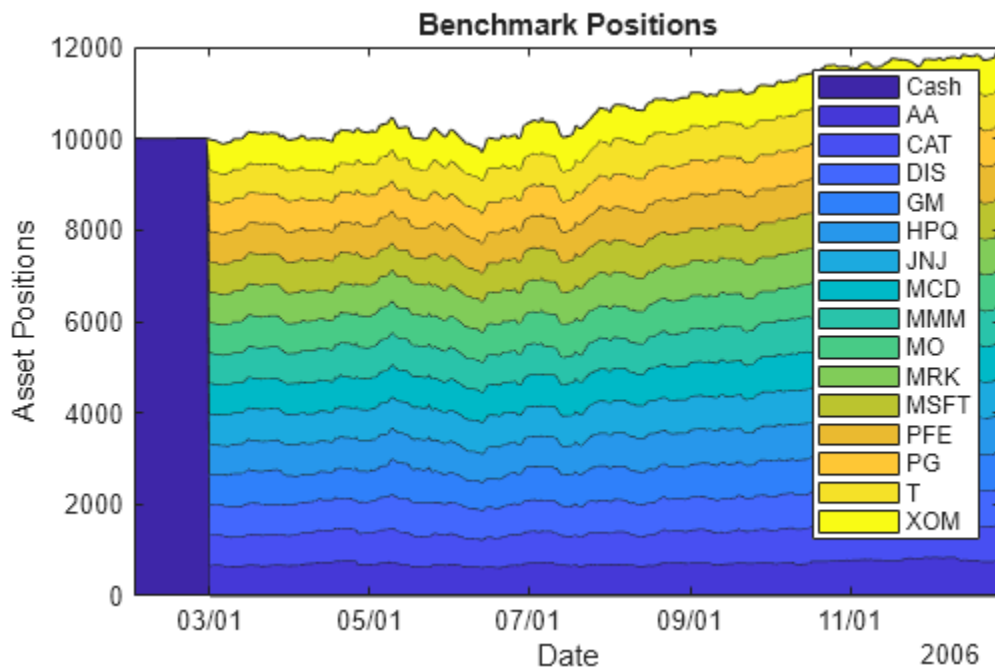
Use `equityCurve` to plot the strategy equity curves to visualize their performance over the backtest.

```
equityCurve(bt)
```



As mentioned previously, these strategies are not typically used as standalone trading signals. In fact, these three strategies perform worse than the simple benchmark strategy for the 2006 timeframe. You can visualize how the strategy allocations change over time using an area chart of the daily asset positions. To do so, use the `assetAreaPlot` helper function, defined in the Local Functions on page 4-261 section.

```
strategyName =  ;  
assetAreaPlot(bt, strategyName)
```



Conclusion

The broad equity market had a very bullish 6 months in the second half of 2006 and all three of these strategies failed to fully capture that growth by leaving too much capital in cash. While none of these strategies performed well on their own, this example demonstrates how you can build signal-based trading strategies and backtest them to assess their performance.

Local Functions

The initial weight calculation function as well as the strategy rebalancing functions follow.

```
function initial_weights = computeInitialWeights(signals)
% Compute initial weights based on most recent signal.

nAssets = size(signals,2);
final_signal = signals{end,:};
buys = final_signal == 1;
initial_weights = zeros(1,nAssets);
initial_weights(buys) = 1 / nAssets;

end

function new_weights = crossoverRebalanceFcn(current_weights, pricesTT, signalTT, signalNameEnding)
% Signal crossover rebalance function.

% Build cell array of signal names that correspond to the crossover signals.
symbols = pricesTT.Properties.VariableNames;
signalNames = cellfun(@(s) sprintf('%s%s',s,signalNameEnding), symbols, 'UniformOutput', false);

% Pull out the relevant signal data for the strategy.
crossoverSignals = signalTT(:,signalNames);

% Start with our current weights.
```

```
new_weights = current_weights;

% Sell any existing long position where the signal has turned to 0.
idx = crossoverSignals{end,:} == 0;
new_weights(idx) = 0;

% Find the new crossovers (signal changed from 0 to 1).
idx = crossoverSignals{end,:} == 1 & crossoverSignals{end-1,:} == 0;

% Bet sizing, split available capital across all remaining assets, and then
% invest only in the new positive crossover assets. This leaves some
% proportional amount of capital uninvested for future investments into the
% zero-weight assets.
availableCapital = 1 - sum(new_weights);
uninvestedAssets = sum(new_weights == 0);
new_weights(idx) = availableCapital / uninvestedAssets;

end

function new_weights = rsiRebalanceFcn(current_weights, pricesTT, signalTT)
% Buy and sell on 1 and -1 rebalance function.

signalNameEnding = '_RSI';

% Build cell array of signal names that correspond to the crossover signals.
symbols = pricesTT.Properties.VariableNames;
signalNames = cellfun(@(s) sprintf('%s%s',s,signalNameEnding), symbols, 'UniformOutput', false);

% Pull out the relevant signal data for the strategy.
buySellSignals = signalTT(:,signalNames);

% Start with the current weights.
new_weights = current_weights;

% Sell any existing long position where the signal has turned to -1.
idx = buySellSignals{end,:} == -1;
new_weights(idx) = 0;

% Find the new buys (signal is 1 and weights are currently 0).
idx = new_weights == 0 & buySellSignals{end,:} == 1;

% Bet sizing, split available capital across all remaining assets, and then
% invest only in the new positive crossover assets. This leaves some
% proportional amount of capital uninvested for future investments into the
% zero-weight assets.
availableCapital = 1 - sum(new_weights);
uninvestedAssets = sum(new_weights == 0);
new_weights(idx) = availableCapital / uninvestedAssets;

end

function new_weights = equalWeightFcn(current_weights,~)
% Equal-weighted portfolio allocation.

nAssets = numel(current_weights);
new_weights = ones(1,nAssets);
```

```

new_weights = new_weights / sum(new_weights);

end

function assetAreaPlot(backtester, strategyName)
% Plot the asset allocation as an area plot.

t = backtester.Positions.(strategyName).Time;
positions = backtester.Positions.(strategyName).Variables;
h = area(t, positions);
title(sprintf('%s Positions', strrep(strategyName, '_', ' ')));
xlabel('Date');
ylabel('Asset Positions');
xtickformat('MM/dd');
xlim([t(1) t(end)])
oldylim = ylim;
ylim([0 oldylim(2)]);
cm = parula(numel(h));
for i = 1: numel(h)
    set(h(i), 'FaceColor', cm(i, :));
end
legend(backtester.Positions.(strategyName).Properties.VariableNames)

end

```

See Also

[backtestStrategy](#) | [backtestEngine](#) | [runBacktest](#) | [summary](#)

Related Examples

- “Backtest Investment Strategies Using Financial Toolbox” on page 4-238
- “Backtest Strategies Using Deep Learning” on page 4-302
- “Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data” on page 2-78
- “Backtest with Brinson Attribution to Evaluate Portfolio Performance” on page 4-315

Portfolio Optimization Using Social Performance Measure

Use a `Portfolio` object to minimize the variance, maximize return, and maximize the average percentage of women on a company's board. The same workflow can be applied with other Environmental, Social and Governance (ESG) criteria, such as an ESG score, a climate, or a temperature score.

The goal of this example is to find portfolios that are efficient in the sense that they minimize the variance, maximize return, and maximize the average percentage of women on the board of directors. To find the average percentage of women on a company's board (WoB) for a given portfolio, this example uses a weighted sum of the percentages of WoB for each individual asset, where the weights are given by the amount invested in each asset for the portfolio. By defining the average percentage of WoB this way, the WoB function is linear with respect to the weights.

Load Portfolio Data

```
load CAPMuniverse
% Assume that the percentage of women on the board of directors per
% company are as follows:
WoB = [0.2857; 0.5; 0.3; 0.2857; 0.3077; 0.2727; ...
       0.4167; 0.2143; 0.3; 0.4167; 0.3077];
table(WoB, 'VariableNames', {'WoB'}, 'RowNames', Assets(1:11))
```

ans=11x1 table

	WoB
AAPL	0.2857
AMZN	0.5
CSCO	0.3
DELL	0.2857
EBAY	0.3077
GOOG	0.2727
HPQ	0.4167
IBM	0.2143
INTC	0.3
MSFT	0.4167
ORCL	0.3077

Create Portfolio Object

Create a standard `Portfolio` object and incorporate the list of assets and estimate the moments of the assets' returns from the data. Use `setDefaultConstraints` to set the default mean-variance portfolio constraints. These constraints require fully invested, long-only portfolios where the nonnegative weights must sum to 1.

```
% Create portfolio with default constraints.
p = Portfolio('AssetList', Assets(1:11));
p = estimateAssetMoments(p, Data(:, 1:11));
p = setDefaultConstraints(p);
```


Set Group Constraints

Use `getGroups` to include group constraints. The first group constraint ensures that the weights invested in mixed retail (Amazon and eBay) are at least 15%. The second group constraint ensures that the weights invested in computer companies (Apple, Dell and HP) are between 25% and 50%.

```
% Group constraints
G = [0 1 0 0 1 0 0 0 0 0 0;
     1 0 0 1 0 0 1 0 0 0 0];
LowG = [0.15; 0.25];
UpG = [Inf; 0.5];
p = setGroups(p, G, LowG, UpG);
```

Find the minimum and maximum percentage of WoB that a portfolio can attain given these extra group constraints. This is done using the `estimateCustomObjectivePortfolio` function with the average percentage of women on a company's board as the objective function.

```
% Set the average WoB as the objective.
objFun = @(x) WoB'*x;
```

Find the portfolio with the minimum average percentage of WoB with the group constraints using `estimateCustomObjectivePortfolio` with the function handle `objFun`.

```
% Minimum percentage of women on the board
wgt_minWoB = estimateCustomObjectivePortfolio(p,objFun);
minWoB = objFun(wgt_minWoB)

minWoB =
0.2462
```

Find the portfolio with the maximum average percentage of WoB with the group constraints using `estimateCustomObjectivePortfolio` with the name-value argument `ObjectiveSense` set to `maximize`.

```
% Maximum percentage of women on the board
wgt_maxWoB = estimateCustomObjectivePortfolio(p,objFun,...
    ObjectiveSense="maximize");
maxWoB = objFun(wgt_maxWoB)

maxWoB =
0.4792
```

Compute and Plot the Efficient Surface

Define a grid of WoB percentages such that $\text{minWoB} = \text{targetWoB}(1) \leq \dots \leq \text{targetWoB}(N) = \text{maxWoB}$.

```
N = 20; % Size of the grid.
targetWoB = linspace(minWoB,maxWoB,N);
```

Use `setInequality` to set the percentage of WoB as a constraint. The coefficients of the linear constraint should be the WoB percentages associated to each asset, and the right-hand side should be the target portfolio WoB. The convention of the inequality is $\leq$. Since the goal is to maximize portfolio WoB, then the target WoB should be a lower bound for the portfolio WoB. Therefore, the signs of the coefficients and the right-hand side of the added inequality should be flipped.

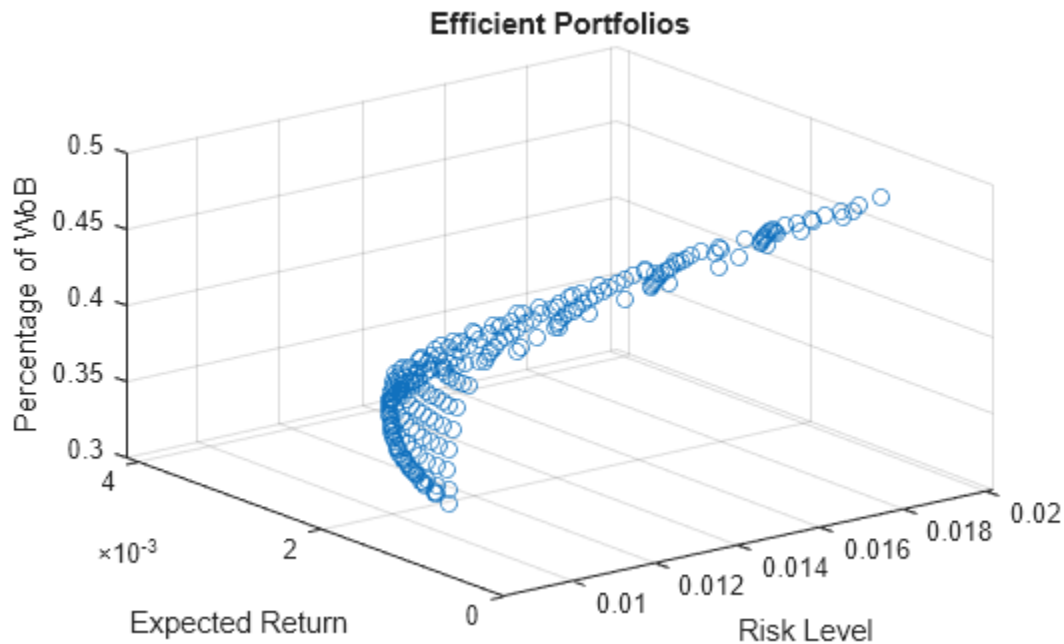
```
Ain = -WoB';
bin = -minWoB; % Start with the smallest WoB.
p = setInequality(p,Ain,bin);
```

For each target WoB, `targetWoB(i)`, find the efficient mean-variance frontier using `estimateFrontier`. At each iteration, the right-hand side of the WoB portfolio constraint should be changed to ensure that the returned portfolios achieve at least the target WoB. This method returns the weights of the portfolios on the mean-variance efficient frontier that have a WoB of at least `targetWoB(i)`. Using the weights obtained for each target WoB, compute the portfolios' expected return, risk, and percentage of WoB.

```
% Get the efficient surface values.
prsk = cell(N,1);
pret = cell(N,1);
pWoB = cell(N,1);
for i = 1:N
    p.bInequality = -targetWoB(i);
    pwgt = estimateFrontier(p,N);
    [prsk{i},pret{i}] = estimatePortMoments(p,pwgt);
    pWoB{i} = pwgt'*WoB;
end
```

Plot the efficient portfolios.

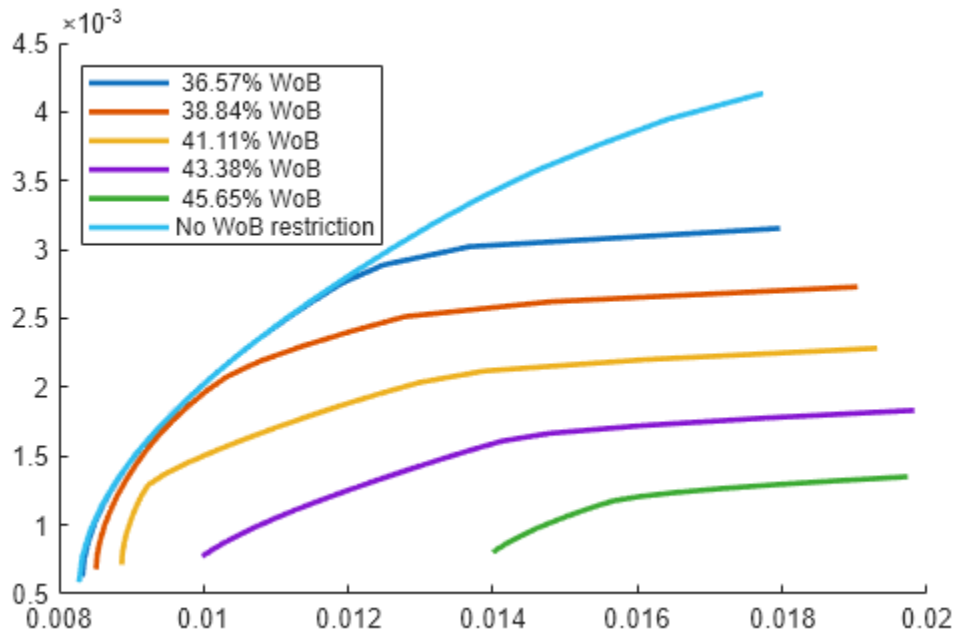
```
% Plot the efficient surface.
scatter3(cell2mat(prsk),cell2mat(pret),cell2mat(pWoB))
title('Efficient Portfolios')
xlabel('Risk Level')
ylabel('Expected Return')
zlabel('Percentage of WoB')
```



To visualize the tradeoff between a portfolio's average percentage of WoB and the traditional mean-variance efficient frontier, a set of contour plots are computed for some target WoB percentages using the `plotContours` function in Local Functions on page 4-269.

```
nC = 5; % Number of contour plots
minContour = max(pWoB{1}); % WoB values lower than this return
                             % overlapped contours.

% Plot the contours.
plotContours(p,minContour,maxWoB,nC,N)
```



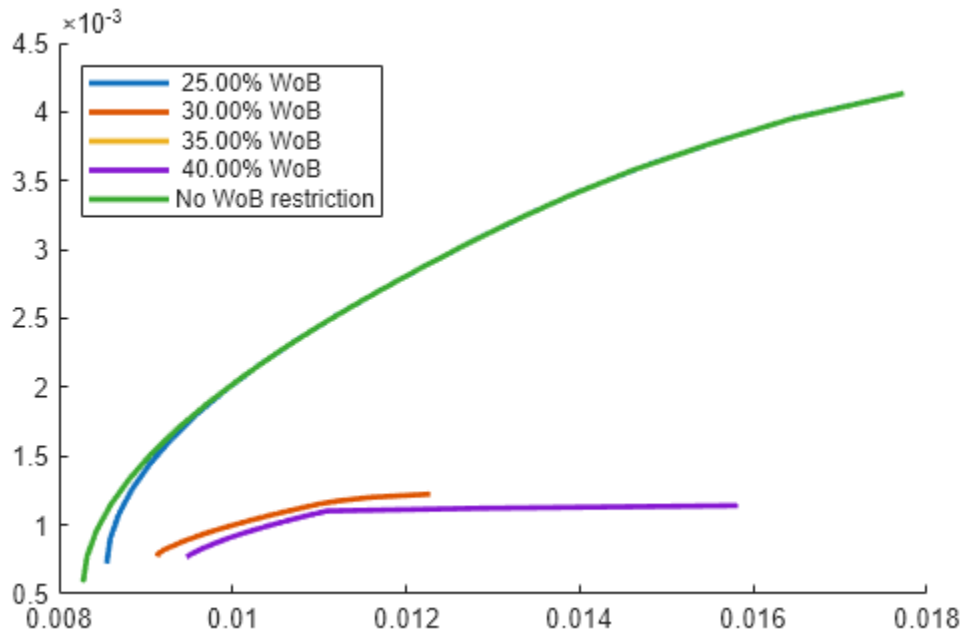
Exclusion Examples

Instead of requiring a specific level for the portfolio's average percentage of WoB, the goal is to find the traditional mean-variance efficient frontiers while excluding assets that have a percentage of WoB lower than a given threshold. You can plot the exclusion using the `plotExclusionExample` function in Local Functions on page 4-269.

```
% Remove the average percentage of WoB constraint.
p.AInequality = []; p.bInequality = [];

% Set of thresholds for excluding assets.
thresholdWoB = 0.25:0.05:0.40;

% Plot the exclusion example.
plotExclusionExample(p,WoB,thresholdWoB,N)
```



The differences between this approach and the one presented in the previous section are quite evident. Requiring all the assets to have a WoB percentage of at least 35% gives an efficient frontier that can achieve a return of at most around $1.2(10)^{-3}$. On the other hand, requiring only that the portfolio's average percentage of WoB is 36.57% gives the possibility to reach a return of around $3.2(10)^{-3}$, almost 2.5 times the return obtained when excluding assets. To better show the differences between these two approaches, compute the maximum return achieved for a given standard deviation for the two ways of including the percentage of WoB requirements to the portfolio.

Approach 1

In the first approach, exclude all assets with a WoB percentage lower than 33% and find the portfolio of maximum return that has a standard deviation of at most 0.012.

```
% Select the assets to exclude.
ub = zeros(p.NumAssets,1);
ub(WoB >= 0.33) = 1;
p.UpperBound = ub;
% Estimate the return for a risk level of 0.012.
pwgt_exclude = estimateFrontierByRisk(p,0.012);
ret_exclude = estimatePortReturn(p,pwgt_exclude)

ret_exclude =
0.0011

% Return the constraints to the original portfolio.
p.UpperBound = [];
```

Approach 2

For the second approach, ensure that the average WoB percentage is of at least 33% and find the portfolio of maximum return that has a standard deviation of at most 0.012.

```
% Include the WoB constraint into the portfolio.
p = addInequality(p,-WoB',-0.33);
% Estimate the return for a risk level of 0.012.
pwgt_avgWoB = estimateFrontierByRisk(p,0.012);
ret_avgWoB = estimatePortReturn(p,pwgt_avgWoB)
```

```
ret_avgWoB =
0.0028
```

```
% Return the constraints to the original portfolio.
p.AInequality = []; p.bInequality = [];
```

Compute the increase in return between these two approaches.

```
ret_increase = (ret_avgWoB-ret_exclude)/ret_exclude

ret_increase =
1.5202
```

This `ret_increase` value shows that the return from the approach that only bounds the portfolio's average WoB percentage instead of excluding certain assets has a return 152% higher (for the same risk level). Hence, when tackling problems with more than two objectives, excluding assets that do not meet a certain criteria might not be the best option. Instead, a weighted sum of the criteria of interest might show better results.

Local Functions

```
function [] = plotContours(p,minWoB,maxWoB,nContour,nPort)
```

```
% Set of WoB levels for contour plot.
contourWoB = linspace(minWoB,maxWoB,nContour+1);
```

```
% Compute and plot efficient frontier for each value in contourWoB.
```

```
figure;
hold on
labels = strings(nContour+1,1);
for i = 1:nContour
    p.bInequality = -contourWoB(i);
    plotFrontier(p,nPort);
    labels(i) = sprintf("%6.2f%% WoB",contourWoB(i)*100);
end
```

```
% Plot the "original" mean-variance frontier, i.e., the frontier
% without WoB requirements.
```

```
p.AInequality = []; p.bInequality = [];
plotFrontier(p,nPort);
labels(i+1) = "No WoB restriction";
legend(labels,'Location','northwest')
hold off
```

```
end
```

```
function [] = plotExclusionExample(p,WoB,thresholdWoB, ...
    nPort)
```

```
% Compute and plot efficient frontier excluding assets that are below
% the WoB threshold.
```

```
nT = length(thresholdWoB);
figure;
```

```
hold on
labels = strings(nT+1,1);
for i=1:nT
    ub = zeros(p.NumAssets,1);
    % Only select the assets above WoB threshold.
    ub(WoB >= thresholdWoB(i)) = 1;
    p.UpperBound = ub;
    plotFrontier(p,nPort);
    labels(i) = sprintf("%6.2f%% WoB",thresholdWoB(i)*100);
end
% Plot the "original" mean-variance frontier, i.e., the frontier
% without the WoB threshold.
p.UpperBound = [];
plotFrontier(p,nPort);
labels(i+1) = "No WoB restriction";
legend(labels,'Location','northwest')
hold off

end
```

See Also

[Portfolio](#) | [setBounds](#) | [addGroups](#) | [setAssetMoments](#) | [estimateAssetMoments](#) | [estimateBounds](#) | [plotFrontier](#) | [estimateFrontierLimits](#) | [estimateFrontierByRisk](#) | [estimatePortRisk](#)

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- [Getting Started with Portfolio Optimization \(4 min 13 sec\)](#)
- [MATLAB for Advanced Portfolio Construction and Stock Selection Models \(30 min 28 sec\)](#)

Diversify ESG Portfolios

This example shows how to include qualitative factors for environmental, social, and corporate governance (ESG) in the portfolio selection process. The example extends the traditional mean-variance portfolio using a `Portfolio` object to include the ESG metric. First, the `estimateFrontier` function computes the mean-variance efficient frontier for different ESG levels. Then, the example illustrates how to combine the ESG performance measure with portfolio diversification techniques. Specifically, it introduces hybrid models that use the Herfindahl-Hirshman (HH) index and the most diversified portfolio (MDP) approach using the `estimateCustomObjectivePortfolio` function. Finally, the `backtestEngine` framework compares the returns and behavior of the different ESG strategies.

Define Mean-Variance Portfolio

Load the table with the asset returns and a vector with ESG scores for the assets. Both the asset returns and their ESG scores are simulated values and do not represent the performance of any real securities. However, you can apply the code and workflow in this example to any data set with prices and returns and ESG information.

```
% Load data
load('asset_return_100_simulated.mat') % Returns table
load('ESG_s26.mat')                  % ESG scores
```

Transform the returns table into a prices timetable.

```
% Transform the returns to prices.
assetPrices = ret2tick(stockReturns);
% Transform the prices table to a timetable.
nRows = size(stockReturns,1);
day = datetime("today");
Time = (day-nRows):day;
assetPrices = table2timetable(assetPrices,"RowTimes",Time);
```

Define a `Portfolio` object with default constraints where the weights must be nonnegative and sum to 1.

```
% Create a mean-variance Portfolio object with default constraints.
p = Portfolio;
p = estimateAssetMoments(p,stockReturns);
p = setDefaultConstraints(p);
```

Compute the Mean-Variance Efficient Frontier for Different ESG Levels

Obtain contour plots of the ESG-mean-variance efficient surface. You obtain the efficient surface from the Pareto optima of the multiobjective problem that includes all the performance metrics: average ESG score, average return, and return variance.

First, obtain feasible values of the ESG metric by finding the minimum and maximum ESG levels. To do this step, use the `estimateCustomObjectivePortfolio` function assigning the average ESG score as the objective function. The average ESG score of a portfolio is the weighted sum of the individual asset ESG scores, where the weights are given by the amount invested in each asset.

```
% Define the objective function: average ESG score.
ESGscore = @(x) ESGnumeric'*x;
```

```
% Find the minimum ESG score.
solMin = estimateCustomObjectivePortfolio(p,ESGscore);
minESG = ESGscore(solMin)

minESG =
0.0187

% Find the maximum ESG score.
solMax = estimateCustomObjectivePortfolio(p,ESGscore, ...
    ObjectiveSense="maximize");
maxESG = ESGscore(solMax)

maxESG =
0.9735
```

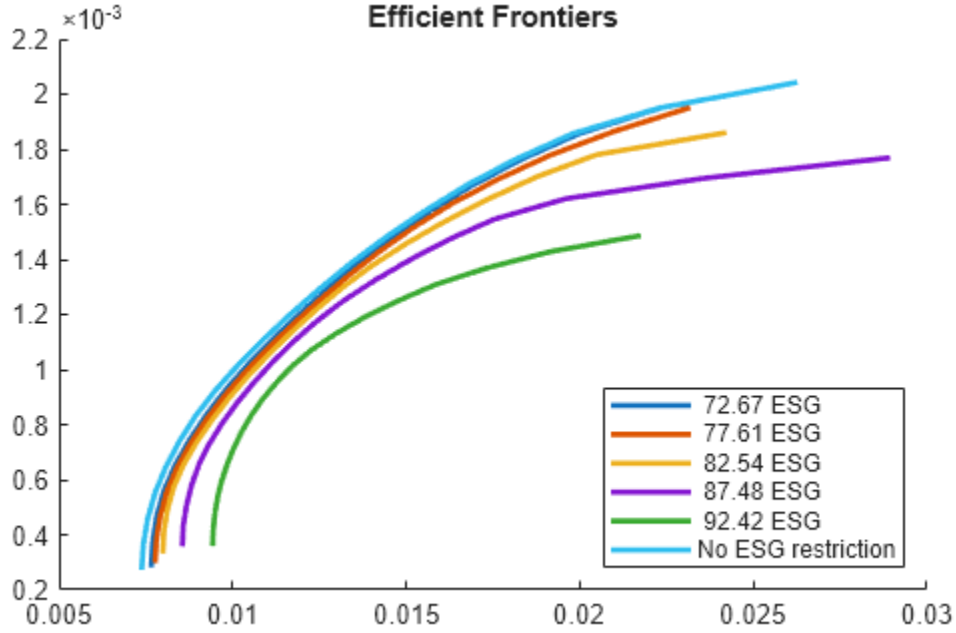
To compute the contours of the efficient surface, you add the average ESG score as a constraint using the `setInequality` function. The coefficients of the linear constraint are the ESG scores associated with each asset and the right-hand side is the target ESG score for the desired contour. Notice that the convention of the inequalities in the `Portfolio` object is $\leq$. Because the goal is to maximize the average ESG score, the target ESG value for the contour should be a lower bound. Therefore, you need to flip the signs of the coefficients and the right-hand side of the added inequality. The function that computes and plots the contours is in the Local Functions on page 4-282 section.

```
N = 20; % Number of efficient portfolios per ESG value

% Add ESG score as a constraint to the Portfolio object.
Ain = -ESGnumeric';
bin = -minESG; % Start with the smallest ESG score
p = setInequality(p,Ain,bin);

% Estimate the lower value for contour plots.
pwgt = estimateFrontier(p,N);
pESG = pwgt'*ESGnumeric;
minContour = max(pESG); % All ESG scores lower than this have
                        % overlapped contours.

% Plot the contours.
nC = 5; % Number of contour plots
plotESGContours(p,ESGnumeric,minContour,maxESG,nC,N);
```

Diversification Techniques

In this example, the two diversification measures are the equally weighted (EW) portfolio and the most diversified portfolio (MDP).

You obtain the EW portfolio using the Herfindahl-Hirschman (HH) index as the diversification measure

$$HH(x) = \sum_{i=1}^n x_i^2$$

An equally weighted portfolio minimizes this index. Therefore, the portfolios that you obtain from using this index as a penalty have weights that satisfy the portfolio constraints and are more evenly weighted.

The diversification measure for the most diversified portfolio (MDP) is

$$MDP(x) = - \sum_{i=1}^n \sigma_i x_i$$

where σ_i represents the standard deviation of asset i . Using this measure as a penalty function maximizes the diversification ratio [1 on page 4-281].

$$\varphi(x) = \frac{x^T \sigma}{\sqrt{x^T \Sigma x}}$$

If the portfolio is fully invested in one asset, or if all assets are perfectly correlated, the diversification ratio $\varphi(x)$ is equal to 1. For all other cases, $\varphi(x) > 1$. Therefore, if $\varphi(x) \approx 1$, there is no diversification, so the goal is to maximize $\varphi(x)$. Unlike the HH index, the goal of MDP is not to obtain a portfolio

whose weights are evenly distributed among all assets, but to obtain a portfolio whose selected (nonzero) assets have the same correlation to the portfolio as a whole.

Diversification for Fixed ESG Level

You can extend the traditional minimum variance portfolio problem subject to an expected return to include the ESG metric by setting an ESG constraint for the problem. The purpose of the ESG constraint is to force the portfolio to achieve an ESG score greater than a certain target. Then, add a penalty term to the objective function to guide the problem toward more or less diversified portfolios. How diversified a portfolio is depends on your choice of the penalty parameter.

Choose an ESG level of 0.85 and an expected return of 0.001.

```
% Minimum ESG and return levels
ESG0 = 0.85;
ret0 = 1e-3;
```

Currently, the portfolio problem assumes that the weights must be nonnegative and sum to 1. Add the requirement that the return of the portfolio is at least `ret0` and the ESG score is at least `ESG0`. The feasible set is represented as X , which is

$$X = \left\{ x \mid x \geq 0, \sum_{i=1}^n x_i = 1, \mu^T x \geq \text{ret}_0, \text{ESG}(x) \geq \text{ESG}_0 \right\}$$

Add the ESG constraint using the `setInequality` function.

```
% Add the ESG constraint.
Ain = -ESGnumeric';
bin = -ESG0; % Set the target ESG score.
p = setInequality(p,Ain,bin);
```

To add the return constraint, pass the name-value argument `TargetReturn=ret0` to the `estimateCustomObjectivePortfolio` function for each of the different custom objective portfolios of interest.

The portfolio that minimizes the variance with the HH penalty is

$$\min_{x \in X} x^T \Sigma x + \lambda_{HH} x^T x$$

```
% HH penalty parameter
```

```
lambdaHH = 0.001  ;
% Variance + Herfindahl-Hirschman (HH) index
var_HH = @(x) x'*p.AssetCovar*x + lambdaHH*(x'*x);
% Solution that accounts for risk and HH diversification
wHHmix = estimateCustomObjectivePortfolio(p,var_HH,TargetReturn=ret0);
```

The portfolio that minimizes the variance with the MDP penalty is

$$\min_{x \in X} x^T \Sigma x - \lambda_{MDP} \sigma^T x$$

```
% MDP penalty parameter
```

```
lambdaMDP = 0.01  ;
% Variance + Most Diversified Portfolio (MDP)
```

```

sigma = sqrt(diag(p.AssetCovar));
var_MDP = @(x) x'*p.AssetCovar*x - lambdaMDP*(sigma'*x);
% Solution that accounts for risk and MDP diversification
wMDPMix = estimateCustomObjectivePortfolio(p,var_MDP,TargetReturn=ret0);

```

Plot the asset allocation from the penalized strategies.

```

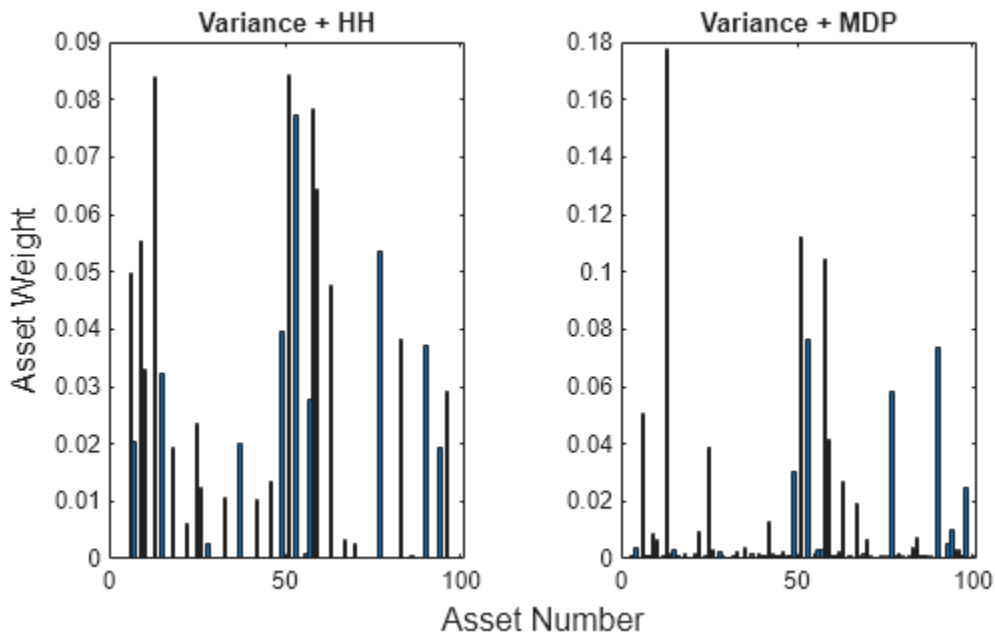
% Plot the asset allocation.
figure
t = tiledlayout(1,2);

% HH penalized method
nexttile
bar(wHHMix');
title('Variance + HH')

% MDP penalized method
nexttile
bar(wMDPMix')
title('Variance + MDP')

% General labels
ylabel(t,'Asset Weight')
xlabel(t,'Asset Number')

```



The strategies that include the penalty function in the objective give weights that are between the minimum variance portfolio weights and the weights from the respective maximum diversification technique. In fact, for the problem with the HH penalty, choosing $\lambda_{HH} = 0$ returns the minimum variance solution, and as $\lambda_{HH} \rightarrow \infty$, the solution approaches the one that maximizes HH diversification. For the problem with the MDP penalty, choosing $\lambda_{MDP} = 0$ also returns the minimum variance solution, and there exists a value $\hat{\lambda}_{MDP}$ such that the MDP problem and the penalized

version are equivalent. Consequently, values of $\lambda_{\text{MDP}} \in [0, \hat{\lambda}_{\text{MDP}}]$ return asset weights that range from the minimum variance behavior to the MDP behavior.

Diversification by "Tilting"

The strategies in Diversification for Fixed ESG Level on page 4-274 explicitly set a target ESG average score. However, a different set of strategies controls the ESG score in a less direct way. The method in these strategies uses ESG *tilting*. With tilting, you discretize the ESG score into 'high' and 'low' levels and the objective function penalizes each level differently. In other words, you use the diversification measure to tilt the portfolio toward higher or lower ESG values. Therefore, instead of explicitly requiring that the portfolios maintain a target ESG average score, you select assets, with respect to their ESG score, implicitly through the choice of penalty parameters.

Start by labeling the assets with an ESG score less than or equal to 0.5 as 'low' and assets with an ESG score greater than 0.5 as 'high', and then remove the ESG constraint.

```
% Label the ESG data.
ESGlabel = discretize(ESGnumeric,[0 0.5 1], ...
    "categorical',{'low','high'});
% Create a table with ESG scores and labels.
ESG = table(ESGnumeric,ESGlabel);
% Remove the ESG constraint.
p = setInequality(p,[],[]);
```

The tilted version of the portfolio with the HH index is

$$\begin{aligned} \min \quad & x^T \Sigma x + \lambda_{\text{HH}}^{\text{high}} \sum_{i \in H} x_i^2 + \lambda_{\text{HH}}^{\text{low}} \sum_{i \in L} x_i^2 \\ \text{s.t.} \quad & \sum_{i=1}^n x_i = 1, \mu^T x \geq \text{ret}_0, x \geq 0 \end{aligned}$$

The feasible region does not bound the average ESG level of the portfolio. Instead, you implicitly control the ESG score by applying different penalization terms to assets with a 'high' ESG score ($i \in H$) and assets with a 'low' ESG score ($i \in L$). To achieve a portfolio with a high average ESG score, the penalty terms must satisfy that $0 \leq \lambda_{\text{HH}}^{\text{high}} \leq \lambda_{\text{HH}}^{\text{low}}$.

```
% HH penalty parameters
% Penalty parameter for assets with 'low' ESG score
lambdaLowHH = 0.01 ;
% Penalty parameter for assets with 'high' ESG score
lambdaHighHH = 0.001 ;
% Lambda for HH 'tilted' penalty
lambdaTiltHH = (ESG.ESGlabel=='low').*lambdaLowHH + (ESG.ESGlabel=='high').*lambdaHighHH;

% Variance + Herfindahl-Hirschman (HH) index
tilt_HH = @(x) x'*p.AssetCovar*x + lambdaTiltHH*(x.^2);
% Solution that minimizes variance + HH term
wTiltHH = estimateCustomObjectivePortfolio(p,tilt_HH,TargetReturn=ret0);
```

Similarly, the tilted version of the portfolio with the MDP penalty term is given by

$$\begin{aligned} \min \quad & x^T \Sigma x - \lambda_{\text{MDP}}^{\text{high}} \sum_{i \in H} \sigma_i x_i - \lambda_{\text{MDP}}^{\text{low}} \sum_{i \in L} \sigma_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n x_i = 1, \mu^T x \geq \text{ret}_0, x \geq 0 \end{aligned}$$

In this case, to achieve a portfolio with a high average ESG score, the penalty terms must satisfy that $0 \leq \lambda_{\text{MDP}}^{\text{low}} \leq \lambda_{\text{MDP}}^{\text{high}}$.

```
% MDP penalty parameters
% Penalty parameter for assets with 'low' ESG score
lambdaLowMDP = 0.001  ;
% Penalty parameter for assets with 'high' ESG score
lambdaHighMDP = 0.01  ;
% Lambda for MDP 'tilted' penalty
lambdaTiltMDP = (ESG.ESGlabel=='low').*lambdaLowMDP + (ESG.ESGlabel=='high').*lambdaHighMDP;

% Variance + MDP index
tilt_MDP = @(x) x'*p.AssetCovar*x - lambdaTiltMDP'*(sigma.*x);
% Solution that minimizes variance + HH term
wTiltMDP = estimateCustomObjectivePortfolio(p,tilt_MDP,TargetReturn=ret0);
```

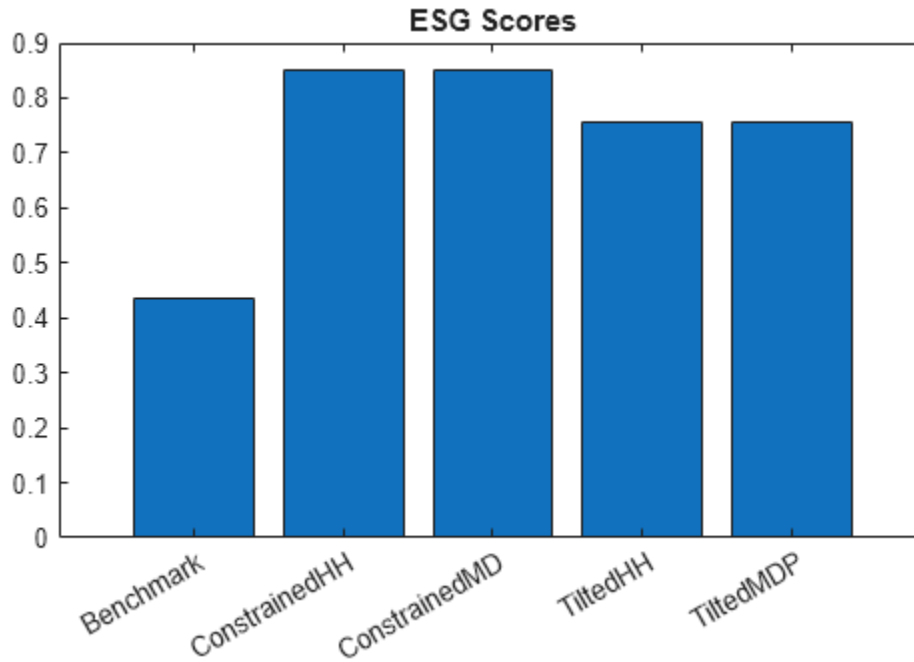
Compare ESG Scores

Compare the ESG scores using tilting and a target ESG constraint. For the comparison to be more meaningful, compute the minimum variance portfolio without ESG constraints or penalty terms, and use the ESG score of the minimum variance portfolio as benchmark.

```
% Compute the minimum variance portfolio without ESG constraints.
p = setInequality(p,[],[]);
wBmk = estimateFrontierByReturn(p,ret0);

% Compute the ESG scores for the different strategies.
Benchmark = ESGnumeric'*wBmk;
ConstrainedHH = ESGnumeric'*wHHMix;
ConstrainedMD = ESGnumeric'*wMDPMix;
TiltedHH = ESGnumeric'*wTiltHH;
TiltedMDP = ESGnumeric'*wTiltMDP;

% Create table
strategiesESG = table(Benchmark,ConstrainedHH,ConstrainedMD,TiltedHH,TiltedMDP);
figure;
bar(categorical(strategiesESG.Properties.VariableNames),strategiesESG.Variables)
title('ESG Scores')
```



As expected, the ESG scores of the penalized strategies are better than the ESG scores of the minimum variance portfolio without constraint and penalty terms. However, the tilted strategies achieve lower ESG scores than the ones that include the target ESG score as a constraint. This comparison shows the flexibility of the tilted strategy. If the ESG target score is not an essential requirement, then you can consider a tilting strategy.

Backtest Using Strategies

To show the performance through time for the two strategies (Diversification for Fixed ESG Level on page 4-274 and Diversification by "Tilting" on page 4-276), use the `backtestEngine` framework. Use `backtestStrategy` to compare the strategies with a prespecified target ESG score with the strategies that use ESG tilting.

```
% Store info to pass to ESG constrained strategies
Ain = -ESGnumeric';
bin = -ESG0; % Set the target ESG score.
conStruct.p = setInequality(p, -ESGnumeric', -ESG0);
conStruct.ret0 = ret0;

conStruct.lambdaHH = 0.01 ;
conStruct.lambdaMDP = 0.05 ;

% Store info to pass to ESG tilting strategies
tiltStruct.p = setInequality(p, [], []);
tiltStruct.ret0 = ret0;
% HH tilting penalty parameters
% Penalty parameter for assets with 'low' ESG score
HHlambdaLow = 0.1 ;
% Penalty parameter for assets with 'high' ESG score
HHlambdaHigh = 0.01 ;
```

```

% Combined penalty terms for HH
tiltStruct.lambdaHH = (ESG.ESGlabel=='low').*HHlambdaLow + ...
    (ESG.ESGlabel=='high').*HHlambdaHigh;
% MDP tilting penalty parameters
% Penalty parameter for assets with 'low' ESG score
MDPlambdaLow = 0.005  ;
% Penalty parameter for assets with 'high' ESG score
MDPlambdaHigh = 0.05  ;
% Combined penalty terms for MDP
tiltStruct.lambdaMDP = (ESG.ESGlabel=='low').*MDPlambdaLow + ...
    (ESG.ESGlabel=='high').*MDPlambdaHigh;

```

Define the investment strategies that you want to use to make the asset allocation decisions at each investment period. For this example, four investment strategies are defined as input to `backtestStrategy`. The first two strategies require a minimum ESG score and the last two use the ESG tilting method.

```

% Define backtesting parameters
warmupPeriod = 84; % Warmup period
rebalFreq = 42; % Rebalance frequency
lookback = [42 126]; % Lookback window
transactionCost = 0.001; % Transaction cost for trade
% Constrained variance + HH strategy
strat1 = backtestStrategy('MixedHH', @(w,P) MixHH(w,P,conStruct), ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', transactionCost, ...
    'InitialWeights', wHHMix);
% Constrained variance + MDP
strat2 = backtestStrategy('MixedMDP', @(w,P) MixMDP(w,P,conStruct), ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', transactionCost, ...
    'InitialWeights', wMDPMix);
% HH tilted strategy
strat3 = backtestStrategy('TiltedHH', @(w,P) tiltedHH(w,P,tiltStruct), ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', transactionCost, ...
    'InitialWeights', wTiltHH);
% MDP tilted strategy
strat4 = backtestStrategy('TiltedMDP', @(w,P) tiltedMDP(w,P,tiltStruct), ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', transactionCost, ...
    'InitialWeights', wTiltMDP);
% All strategies
strategies = [strat1,strat2,strat3,strat4];

```

Run the backtest using `runBacktest` and generate a summary for each strategy's performance results.

```

% Create the backtesting engine object.
backtester = backtestEngine(strategies);
% Run backtest
backtester = runBacktest(backtester,assetPrices, ...

```

```
'Start',warmupPeriod);
```

```
% Summary  
summary(backtester)
```

```
ans=9x4 table
```

	MixedHH	MixedMDP	TiltedHH	TiltedMDP
TotalReturn	0.7077	1.1927	0.79527	1.3295
SharpeRatio	0.029317	0.03188	0.031911	0.034608
Volatility	0.011965	0.017845	0.011719	0.016851
AverageTurnover	0.0054528	0.0065605	0.0053991	0.0070898
MaxTurnover	0.80059	0.72624	0.80099	0.76676
AverageReturn	0.00035068	0.00056875	0.00037387	0.00058304
MaxDrawdown	0.23083	0.34334	0.22526	0.34688
AverageBuyCost	0.068778	0.097277	0.070048	0.12019
AverageSellCost	0.068778	0.097277	0.070048	0.12019

To visualize their performance over the entire investment period, plot the daily results of the strategies using `equityCurve`.

```
% Plot the daily stocks and ESG strategy behavior.  
figure
```

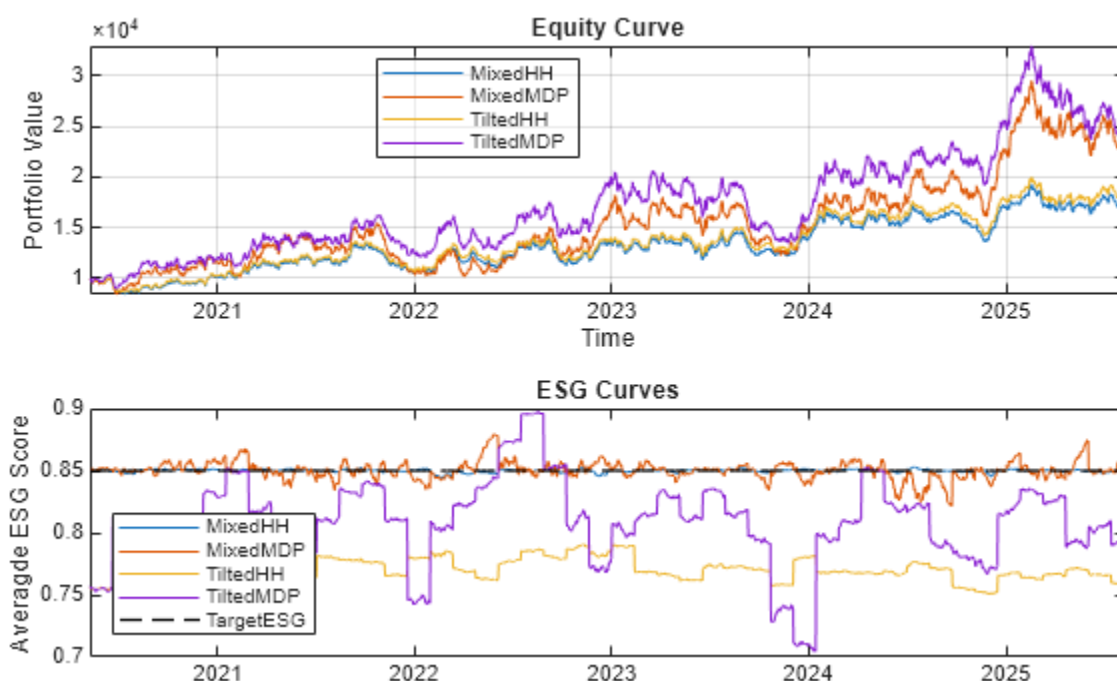
```
t = tiledlayout(2,1,'Padding','none');
```

```
% Equity curves
```

```
nexttile(t)  
equityCurve(backtester);
```

```
% Table and plot of the average ESG score for the mixed and tilted  
% strategies throughout the investment period.
```

```
nexttile(t)  
TAvgESG = averageESGTimetable(backtester,ESGnumeric,ESG0);
```

In the Equity Curve plot, the tilted MDP strategy is the one that performs the best by the end of the investment period, followed by the mixed MDP strategy with the ESG constraint. The performance of the mixed and tilted strategies depend on the choice of the penalty parameters. Both ESG methods, the constrained and the tilted methods, require defining two parameters for each strategy. The ESG constrained method requires you to provide a target ESG score and a penalty parameter for the diversification term. The ESG tilting requires one penalty parameter value for the assets with 'high' ESG scores and a different one for 'low' ESG scores. In addition, the ESG tilting requires a third parameter to determine the cutoff point between 'high' and 'low' ESG assets. Given the dependency of the penalized strategies on the value of their parameters, the performance of those strategies vary widely. However, this example shows that it is possible to find values of the parameters so that the resulting strategies obtain good returns while improving on the average ESG score.

The ESG Curves plot shows the average ESG evolution computed throughout the investment period for the penalized investment strategies, both for the ESG constrained method and the ESG tilting method. You can see that, unlike the choice of the ESG target for the ESG constraint, the selection of the tilting penalty parameters has an effect on the ESG score of the optimal portfolios that is less explicit. Therefore, the average ESG score varies more with the tilting strategies than with the constrained strategies, as expected.

Although not shown in this example, the traditional minimum variance portfolio strategy, subject to the same expected return and ESG levels, results in higher average turnover and transaction costs than any of the strategies covered in the example. Another advantage of adding a diversification measure to the formulation of the problem is to reduce the average turnover and transaction costs.

References

[1] Richard, J. C., and T. Roncalli. *Smart Beta: Managing Diversification of Minimum Variance Portfolios. Risk-Based and Factor Investing*. Elsevier, pp. 31-63, 2015.

Local Functions

```
function [] = plotESGContours(p,ESGscores,minESG,maxESG,nCont, ...
    nPort)
% Plot the mean-variance frontier for different ESG levels.

% Add ESG constraint
p.AInequality = -ESGscores';

% Compute the mean-variance risks and returns for different ESG levels.
contourESG = linspace(minESG,maxESG,nCont+1);
figure
hold on
labels = strings(nCont+1,1);
for i = 1:nCont
    p.bInequality = -contourESG(i); % Change target ESG score
    plotFrontier(p,nPort);
    labels(i) = sprintf("%6.2f ESG",contourESG(i)*100);
end

% Plot the original mean-variance frontier.
p.AInequality = []; p.bInequality = [];
plotFrontier(p,nPort);
labels(i+1) = "No ESG restriction";
title('Efficient Frontiers')
legend(labels,'Location','southeast')
hold off

end

function [ESGTimetable] = averageESGTimetable(backtester, ...
    ESGscores,targetESG)
% Create a table of the average ESG score for the mix and tilted
% strategies throughout the investment period and plot it.

% Normalize weights
wMixedHH = backtester.Positions.MixedHH{:,2:end}./...
    sum(backtester.Positions.MixedHH.Variables,2);
wMixedMDP = backtester.Positions.MixedMDP{:,2:end}./...
    sum(backtester.Positions.MixedMDP.Variables,2);
wTiltedHH = backtester.Positions.TiltedHH{:,2:end}./...
    sum(backtester.Positions.TiltedHH.Variables,2);
wTiltedMDP = backtester.Positions.TiltedMDP{:,2:end}./...
    sum(backtester.Positions.TiltedMDP.Variables,2);

% Compute the ESG scores for the different strategies.
consHH_ESG = wMixedHH*ESGscores;
consMDP_ESG = wMixedMDP*ESGscores;
tiltedHH_ESG = wTiltedHH*ESGscores;
tiltedMDP_ESG = wTiltedMDP*ESGscores;

% Create timetable
ESGTimetable = timetable(backtester.Positions.TiltedHH.Time, ...
    consHH_ESG,consMDP_ESG,tiltedHH_ESG,tiltedMDP_ESG);

% Plot ESG curves
plot(ESGTimetable.Time, ESGTimetable.Variables);
hold on
```

```

plot(ESGTimetable.Time, targetESG*ones(size(ESGTimetable.Time)), ...
     'k--','LineWidth',1); % Plots target ESG scores
title('ESG Curves');
ylabel('Average ESG Score');
legend('MixedHH','MixedMDP','TiltedHH','TiltedMDP','TargetESG', ...
       'Location','southwest');

end

function new_weights = MixHH(~, assetPrices, struct)
% Min variance + max HH diversification strategy

% Retrieve the portfolio information.
p = struct.p;
ret0 = struct.ret0;

% Define the returns and covariance matrix.
assetReturns = tick2ret(assetPrices);
p = estimateAssetMoments(p,assetReturns{:,:});

% Objective function: Variance + Herfindahl-Hirschman
% diversification term
% min x'*Sigma*x + lambda*x'*x
objFun = @(x) x'*p.AssetCovar*x + struct.lambdaHH*(x'*x);

% Solve problem
% Solution that minimizes the variance + HH index
new_weights = estimateCustomObjectivePortfolio(p,objFun, ...
       TargetReturn=ret0);

end

function new_weights = MixMDP(~, assetPrices, struct)
% Min variance + MDP diversification strategy

% Retrieve the portfolio information.
p = struct.p;
ret0 = struct.ret0;

% Define the returns and covariance matrix.
assetReturns = tick2ret(assetPrices);
p = estimateAssetMoments(p,assetReturns{:,:});
sigma = sqrt(diag(p.AssetCovar));

% Objective function: Variance + MDP diversification term
% min x'*Sigma*x - lambda*sigma'*x
objFun = @(x) x'*p.AssetCovar*x - struct.lambdaMDP*(sigma'*x);

% Solve problem
% Solution that minimizes variance + MDP term
new_weights = estimateCustomObjectivePortfolio(p,objFun, ...
       TargetReturn=ret0);

end

function new_weights = tiltedHH(~,assetPrices,struct)
% Tilted HH approach

```

```
% Retrieve the portfolio information.
p = struct.p;
ret0 = struct.ret0;
lambda = struct.lambdaHH;

% Define the returns and covariance matrix.
assetReturns = tick2ret(assetPrices);
p = estimateAssetMoments(p,assetReturns{:,:});

% Objective function: Variance + Herfindahl-Hirschman
% diversification term
% min x'*Sigma*x + lambda*x'*x
objFun = @(x) x'*p.AssetCovar*x + lambda'*(x.^2);

% Solve problem
% Solution that minimizes variance + HH term
new_weights = estimateCustomObjectivePortfolio(p,objFun, ...
    TargetReturn=ret0);

end

function new_weights = tiltedMDP(~,assetPrices,struct)
% Tilted MDP approach

% Retrieve the portfolio information.
p = struct.p;
ret0 = struct.ret0;
lambda = struct.lambdaMDP;

% Define the returns and covariance matrix.
assetReturns = tick2ret(assetPrices);
p = estimateAssetMoments(p,assetReturns{:,:});
sigma = sqrt(diag(p.AssetCovar));

% Objective function: Variance + MDP
% min x'*Sigma*x - lambda*sigma'*x
objFun = @(x) x'*p.AssetCovar*x - lambda'*(sigma.*x);

% Solve problem
% Solution that minimizes variance + MDP term
new_weights = estimateCustomObjectivePortfolio(p,objFun, ...
    TargetReturn=ret0);

end

% The API of the rebalance functions (MixHH, MixMDP, tiltedHH, and
% tiltedMDP) require a first input with the current weights. They
% are redundant for these strategies and can be ignored.
```

See Also

Portfolio | setBounds | addGroups | setAssetMoments | estimateAssetMoments |
estimateBounds | plotFrontier | estimateFrontierLimits | estimateFrontierByRisk |
estimatePortRisk

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Postprocessing Results to Set Up Tradable Portfolios” on page 4-130
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Risk Budgeting Portfolio

This example shows how to use `riskBudgetingPortfolio` to create a risk budgeting portfolio and `portfolioRiskContribution` to compute the risk contribution of the assets in the portfolio.

Risk budgeting is a portfolio allocation strategy that focuses on the allocation of risk to define the weights of a portfolio. The risk budgeting portfolio is constructed by ensuring that the risk contribution of each asset to the portfolio overall risk matches a target risk budget.

A common risk budgeting portfolio is the risk parity or equal risk contribution portfolio. For risk parity portfolios, the goal is to ensure that all assets contribute equally to the portfolio risk, that is, the target risk budget is the same for all assets. Key advantages of the risk budgeting portfolio are stability and diversification of weights:

- **Stability of the allocations:** The weights of the risk parity portfolio are more robust to changes in the returns covariance than the weights of the mean-variance portfolio.
- **Diversification of the portfolio weights:** The risk parity portfolio keeps the portfolio from concentrating in just a few number of assets.

This example follows the example in Section 3.1.1 of Bruder and Roncalli.[1 on page 4-290]

Define the Asset Returns Covariance

Define the covariance matrix of the asset returns (`Sigma`). `Sigma` must be a positive semidefinite matrix.

```
% Define the returns covariance matrix.
sigma = [0.2; 0.21; 0.1];           % Assets risk
rho = 0.8;                          % Assets correlation
C = @(rho) [ 1 rho rho;
             rho 1 rho;
             rho rho 1 ];           % Uniform correlation
Sigma = corr2cov(sigma,C(rho)); % Covariance
% Check that Sigma is positive semidefinite. All
% eigenvalues should be nonnegative (>= 0).
eig(Sigma)

ans = 3×1

    0.0026
    0.0084
    0.0831
```

Create Risk Budgeting Portfolio

Use `riskBudgetingPortfolio` with the covariance matrix (`Sigma`) and a vector of risk budgets (`budget`) to compute the long-only fully invested risk budgeting portfolio. When you specify the target risk budget, the function computes the risk parity portfolio.

```
% Define a risk budget.
budget = [0.5; 0.25; 0.25];
wRB = riskBudgetingPortfolio(Sigma,budget)

wRB = 3×1
```

```
0.3912
0.1964
0.4124
```

Compute Risk Contribution for Each Asset

Use `portfolioRiskContribution` to check that the risk contribution of the portfolio obtained in `wRB` matches the target risk budget. By default, `portfolioRiskContribution` computes the relative risk contribution of a given portfolio (`wRB`) with respect to the provided covariance matrix (`Sigma`).

```
prc = portfolioRiskContribution(wRB,Sigma)

prc = 3×1

0.5000
0.2500
0.2500
```

As expected, the risk contribution of each asset matches the target risk budget (`budget`). You can also use `portfolioRiskContribution` to compute the absolute risk contribution of each asset.

```
mrc = portfolioRiskContribution(wRB,Sigma,RiskContributionType="absolute")

mrc = 3×1

0.0751
0.0376
0.0376
```

Compare Risk Budgeting Portfolios with Mean-Variance

While the allocation strategy of the risk budgeting portfolio focuses only on the risk, the allocation strategy of the mean-variance portfolio also makes use of the assets' return information.

```
% Define the returns mean.
mu = [0.08; 0.08; 0.05];
```

Initialize a `Portfolio` object with default constraints using `Sigma` and `mu`.

```
% Define the mean-variance portfolio.
p = Portfolio(AssetMean=mu,AssetCovar=Sigma);
p = setDefaultConstraints(p);
```

Compute the portfolio on the efficient frontier with a target volatility equal to the risk attained by the risk budgeting portfolio previously computed as `wRB`.

```
% Define the target volatility.
targetRisk = sqrt(wRB'*Sigma*wRB);
wMV = estimateFrontierByRisk(p,targetRisk)

wMV = 3×1

0.3846
0.2030
```

```
0.4124
```

Assume that the correlation between the assets is 0.7 instead of 0.8. Use the new covariance matrix (Sigma7) to compute the same mean-variance and risk budgeting portfolios.

```
% Update the covariance matrix.
Sigma7 = corr2cov(sigma,C(0.7));
% Compute the mean-variance portfolio.
p = Portfolio(AssetMean=mu,AssetCovar=Sigma7);
p = setDefaultConstraints(p);
wMV_rho7 = estimateFrontierByRisk(p,targetRisk)

wMV_rho7 = 3x1

    0.3841
    0.2600
    0.3559
```

```
% Compute the risk budgeting portfolio.
wRB_rho7 = riskBudgetingPortfolio(Sigma7,budget)

wRB_rho7 = 3x1

    0.3844
    0.1986
    0.4170
```

What if the correlation is 0.9 instead of 0.8 (Sigma9) and the risk of the second asset is 0.18 instead of 0.21?

```
% Update the covariance matrix.
sigma2 = sigma;
sigma2(2) = 0.18;
Sigma9 = corr2cov(sigma2,C(0.9));
% Compute the mean-variance portfolio.
p = Portfolio(AssetMean=mu,AssetCovar=Sigma9);
p = setDefaultConstraints(p);
wMV_rho9 = estimateFrontierByRisk(p,targetRisk)

wMV_rho9 = 3x1

    0.0000
    0.6612
    0.3388
```

```
% Compute the risk budgeting portfolio.
wRB_rho9 = riskBudgetingPortfolio(Sigma9,budget)

wRB_rho9 = 3x1

    0.3852
    0.2196
    0.3952
```


What if the original covariance (Sigma) is correct, but the mean of the first asset is 0.09 instead of 0.08?

```
% Update the returns vector.
mu1 = mu;
mu1(1) = 0.09;
% Compute the mean-variance portfolio.
p = Portfolio(AssetMean=mu1,AssetCovar=Sigma);
p = setDefaultConstraints(p);
wMV_mu1 = estimateFrontierByRisk(p,targetRisk)

wMV_mu1 = 3x1

    0.5664
    0.0000
    0.4336

% Compute the risk budgeting portfolio.
wRB_mu1 = riskBudgetingPortfolio(Sigma,budget)

wRB_mu1 = 3x1

    0.3912
    0.1964
    0.4124
```

The weights of the risk budgeting portfolio are more stable than those of the mean-variance portfolio. When the change is in the expected returns, the risk budgeting portfolio remains the same as in the original problem. The risk budgeting portfolio accounts only for changes to the covariance matrix and a change in the vector of expected returns does not affect the weights allocation.

```
% Display a table of weights for the risk budgeting portfolios.
RBtable = table(wRB,wRB_rho7,wRB_rho9,wRB_mu1)
```

```
RBtable=3x4 table
      wRB      wRB_rho7      wRB_rho9      wRB_mu1
      _____      _____      _____      _____
    0.39123    0.38444    0.38521    0.39123
    0.19638    0.19857    0.21957    0.19638
    0.41239    0.41699    0.39522    0.41239
```

```
% Display a table of weights for the mean-variance portfolios.
MVtable = table(wMV,wMV_rho7,wMV_rho9,wMV_mu1)
```

```
MVtable=3x4 table
      wMV      wMV_rho7      wMV_rho9      wMV_mu1
      _____      _____      _____      _____
    0.38462    0.38411    5.2827e-17    0.56638
    0.20301    0.26003    0.6612    2.2204e-16
    0.41237    0.35586    0.3388    0.43362
```

The results on these tables show that the risk budgeting portfolio is more robust to changes in the parameters. The weights in the third and fourth column of the mean-variance portfolio table change

so drastically that the portfolios stop investing in some assets altogether when the parameters change. The mean-variance portfolio tends to concentrate weights in a few number of assets, which decreases portfolio diversification.

References

[1] Bruder, B. and T. Roncalli. *Managing Risk Exposures Using the Risk Budgeting Approach*. January 2012. Available at SSRN: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2009778.

See Also

`portfolioRiskContribution` | `riskBudgetingPortfolio` | `Portfolio` | `setDefaultConstraints` | `plotFrontier` | `estimateFrontierLimits` | `estimateFrontierByRisk` | `estimatePortRisk`

Related Examples

- “Backtest Using Risk-Based Equity Indexation” on page 4-291
- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Backtest Using Risk-Based Equity Indexation

This example shows how to use backtesting with a risk parity or equal risk contribution strategy rebalanced approximately every month as a risk-based indexation. In this example, you use the backtesting engine (`backtestEngine`) to create the risk parity strategy, that all assets in the portfolio contribute equally to the risk of the portfolio at each rebalancing period.

To highlight advantages, the example compares the proposed risk-based indexation against a capitalization-weighted indexation. In contrast to a risk parity strategy, the assets of capitalization-weighted portfolios are weighted with respect to the market portfolio. One disadvantage of capitalization-weighted indexation is that it is a trend-following strategy that leads to bubble-risk exposure as the best performers represent a larger proportion of the market. Also, capitalization-weighted indexation can sometimes lead to weights concentrating in a sector. A risk-based indexation is an alternative investment strategy to avoid these issues of capitalization-weighted indexation.

Define Assets

```
% Read the table of daily adjusted close prices for 2006 DJIA stocks.
T = readtable('dowPortfolio.xlsx');
% Convert the table to a timetable.
pricesTT = table2timetable(T);
% Define the number of assets (not counting the market index).
nAssets = size(pricesTT,2)-1;
```

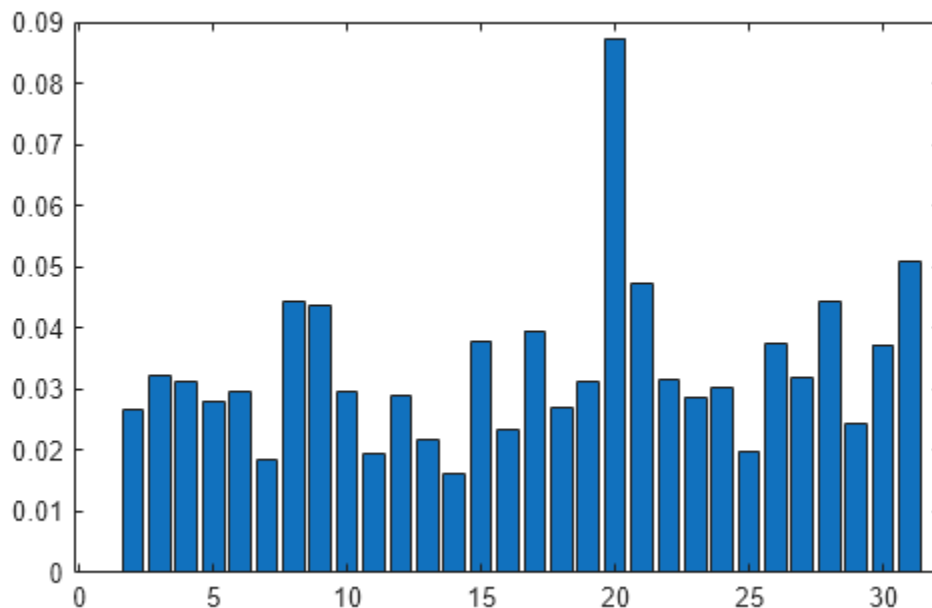
Define Risk Parity Strategy

To obtain the long-only fully invested risk parity portfolio, use the `riskBudgetingPortfolio` function. When you pass a returns covariance matrix to `riskBudgetingPortfolio`, the function computes the associated risk parity portfolio. To compute the initial weights for the risk parity strategy, `riskBudgetingPortfolio` uses the covariance matrix estimated with information from the first two months.

```
% Set backtesting warm-up period to two 21-day months.
warmupPeriod = 21*2;
% Set warm-up partition of timetable.
% Invest only in assets. First column of pricesTT is the market index.
warmupTT = pricesTT(1:warmupPeriod,2:end);
% Estimate the covariance matrix.
assetReturns = tick2ret(warmupTT);
assetCov = cov(assetReturns{:,:});
% Compute the initial risk parity portfolio.
initialRiskParity = [0; riskBudgetingPortfolio(assetCov)];
```

Visualize initial risk parity portfolio weights.

```
% Plot a bar chart.
bar(initialRiskParity)
```



Create the risk parity backtesting strategy using a `backtestStrategy` object. Set the risk parity strategy to rebalance every month.

```
% Rebalance frequency
rebalFreq = 21; % Every month
```

To gather enough data, set the lookback window to at least 2 months. To remove old data from the covariance estimation, set the lookback window to no more than 6 months.

```
% Lookback window
lookback = [42 126];
```

Use a fixed transaction cost equal to 0.5% of the amount traded.

```
% Fixed transaction cost
transactionCost = 0.005;
```

Define the risk parity allocation strategy using the `riskParityFcn` rebalancing function that is defined in Local Functions on page 4-294. Notice that the `riskBudgetingPortfolio` function is at the core of `riskParityFcn`.

```
% Risk parity allocation strategy
stratRP = backtestStrategy('Risk Parity', @riskParityFcn, ...
    RebalanceFrequency=rebalFreq, ...
    LookbackWindow=lookback, ...
    TransactionCosts=transactionCost, ...
    InitialWeights=initialRiskParity);
```

Define Benchmark Strategy

Next, compare the risk parity strategy against the portfolio that follows the Dow Jones Industrial Average (DJI) market index. To obtain the market-following portfolio, invest all assets to the first entry of the weights vector because the first column of the prices timetable (`pricesTT`) represents the market index.

```
% Compute the initial market portfolio.
initialMkt = [1;zeros(nAssets,1)];
```

Define the benchmark strategy using the `mktFcn` rebalancing function that is defined in Local Functions on page 4-294.

```
% Market index allocation strategy
stratBmk = backtestStrategy('Market Index', @mktFcn, ...
    RebalanceFrequency=rebalFreq, ...
    LookbackWindow=lookback, ...
    TransactionCosts=transactionCost, ...
    InitialWeights=initialMkt);
```

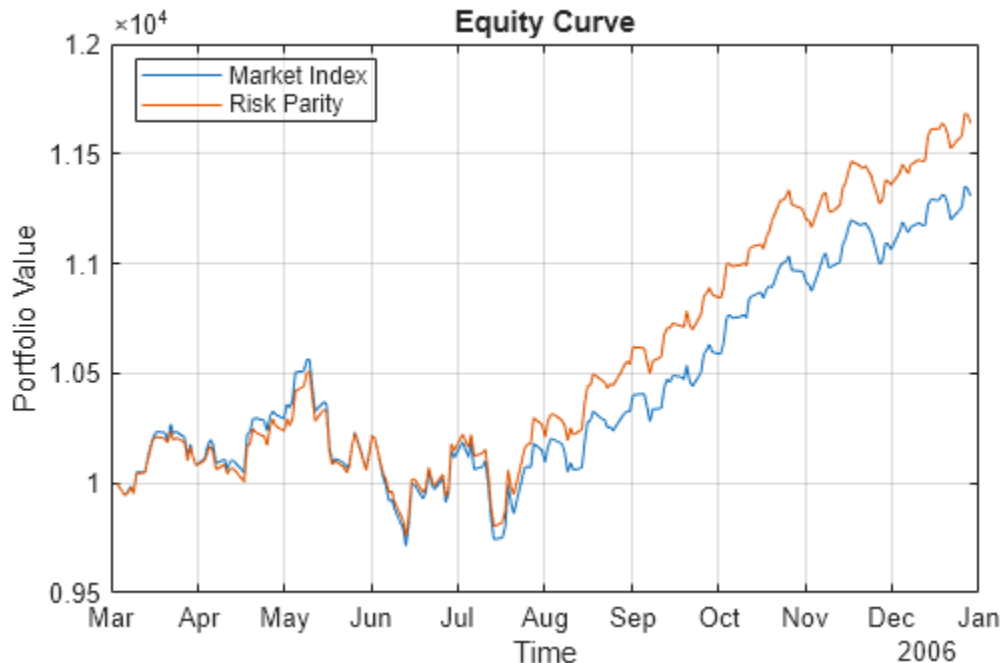
Run Backtest

Run the backtest using a `backtestEngine` object and `runBacktest`.

```
% Define strategies for backtest engine.
strategies = [stratBmk stratRP];
% Define a backtest engine.
backtester = backtestEngine(strategies);
% Run the backtest.
backtester = runBacktest(backtester,pricesTT,Start=warmupPeriod);
```

Use `equityCurve` to plot the equity curve.

```
% Plot equity curve.
equityCurve(backtester)
```



The risk parity strategy outperforms the market index by the end of the investment period. To understand why, you can look at which strategy performs better in the event of meaningful losses.

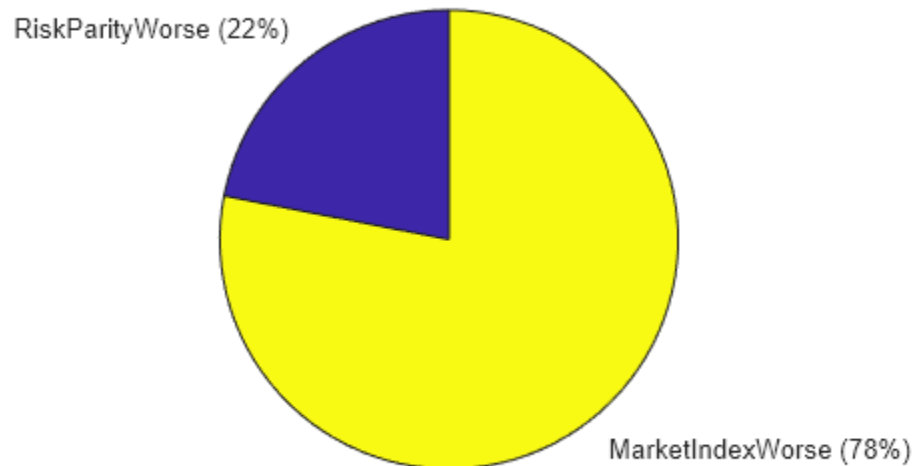
```
% Get returns timetable.
returnsTT = backtester>Returns;
```

```

% Determine levels of losses to consider.
cutoff = 0.004  ;

% Obtain scenarios with losses below cutoff.
idxLoss = returnsTT.Market_Index <= -cutoff | ...
    returnsTT.Risk_Parity <= -cutoff;
% Indices when risk parity outperforms the market
idxLossMarketWorse = ...
    returnsTT.Market_Index(idxLoss) <= returnsTT.Risk_Parity(idxLoss);
catLoss = categorical(idxLossMarketWorse, ...
    [0 1], {'RiskParityWorse', 'MarketIndexWorse'});
% Plot pie chart.
pie(catLoss)

```



The plot shows that in 78% of the scenarios with *meaningful losses*, the market index loses more than the risk parity strategy. In this example, the reason the risk parity strategy outperforms the market index is because the risk parity strategy is more resilient to larger losses.

Local Functions

```

function new_weights = riskParityFcn(~,pricesTT)
% Risk parity rebalance function

% Invest only in assets. First column of pricesTT is the market index.
assetReturns = tick2ret(pricesTT(:,2:end));
assetCov = cov(assetReturns{:,:});

% Do not invest in the market index.
new_weights = [0; riskBudgetingPortfolio(assetCov)];

end

```

```
function new_weights = mktFcn(~,pricesTT)
% Market index rebalance function

% Invest only in the market index.
new_weights = zeros(size(pricesTT,2),1);
new_weights(1) = 1;

end

% The API of the rebalance functions (riskParityFcn and mktFcn) require
% a first input with the current weights. They are
% redundant for these two strategies and can be ignored.
```

See Also

[backtestStrategy](#) | [backtestEngine](#) | [portfolioRiskContribution](#) | [riskBudgetingPortfolio](#)

Related Examples

- “Risk Budgeting Portfolio” on page 4-286
- “Create Hierarchical Risk Parity Portfolio” on page 4-296
- “Backtest Investment Strategies Using Financial Toolbox” on page 4-238
- “Backtest Investment Strategies with Trading Signals” on page 4-251
- “Backtest with Brinson Attribution to Evaluate Portfolio Performance” on page 4-315

Create Hierarchical Risk Parity Portfolio

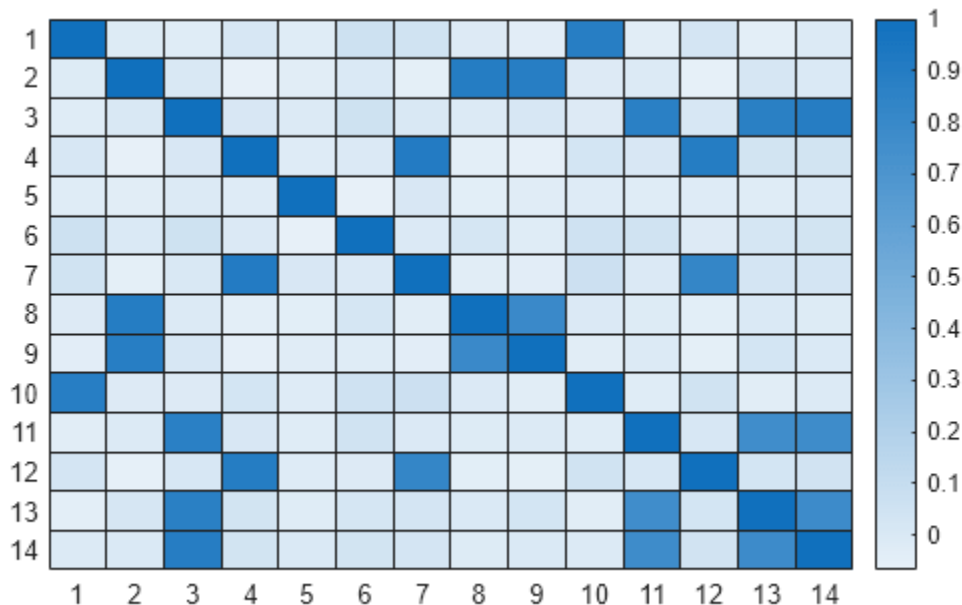
This example shows how to compute a hierarchical risk parity (HRP) portfolio. You can use HRP as a technique for portfolio diversification where the assets are divided and weighted according to a hierarchical tree structure. The weights of the assets within a cluster and between clusters can be assigned in many ways. A few ideas of the ways to allocate the weights are:

- Compute an inverse variance portfolio within each cluster. Then, allocate weights to each cluster using a value proportional to the inverse of the variance of the cluster's portfolio.
- Compute a risk parity portfolio within each cluster. Then, use a risk parity allocation strategy to assign each cluster's weights. The risk parity between clusters uses the covariance matrix between the cluster's portfolios. This example focuses on this allocation strategy.
- Use a bisection approach like the one described in Lopez de Prado [1 on page 4-300]. For more information, see the example Asset Allocation - Hierarchical Risk Parity.

Begin by loading the data and looking at the correlation between the assets returns.

```
% Load data
assetRetn = readmatrix("./retns_assets.txt");
[nSample,nAssets] = size(assetRetn);

% Compute covariance and correlation matrices
Sigma = cov(assetRetn);
C = corrcov(Sigma);
heatmap(C);
```



Hierarchical Clustering

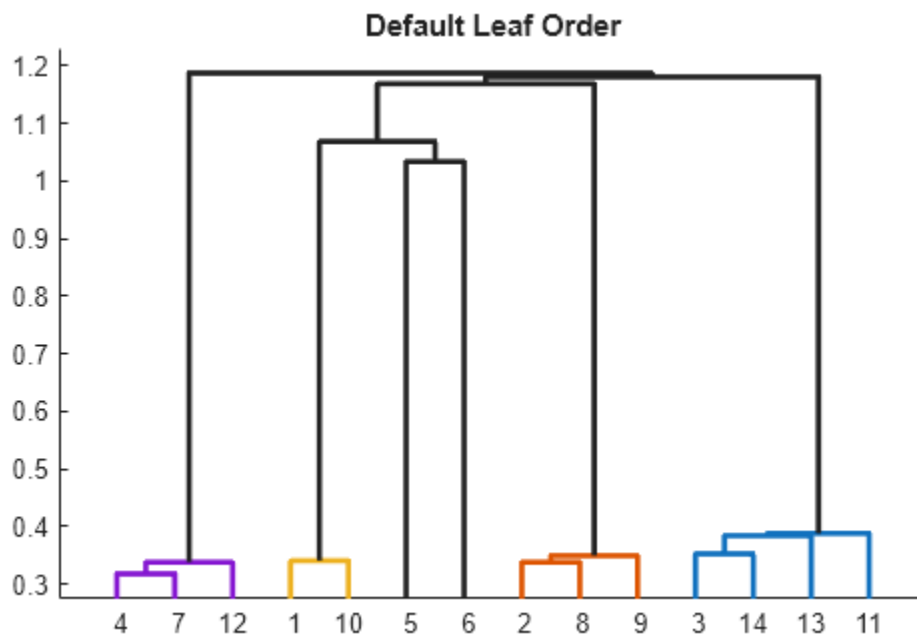
Hierarchical clustering is a common clustering technique in machine learning. In the context of asset allocation, a hierarchical clustering algorithm is applied to find the distance or similarity between each pair of assets and group them into a multilevel binary hierarchical tree.

Begin by defining a measure of likeness or distance between the assets. The more correlated two assets are, the closer they should be.

```
% Compute the correlation distance matrix.
distCorr = ((1-C)/2).^0.5;
```

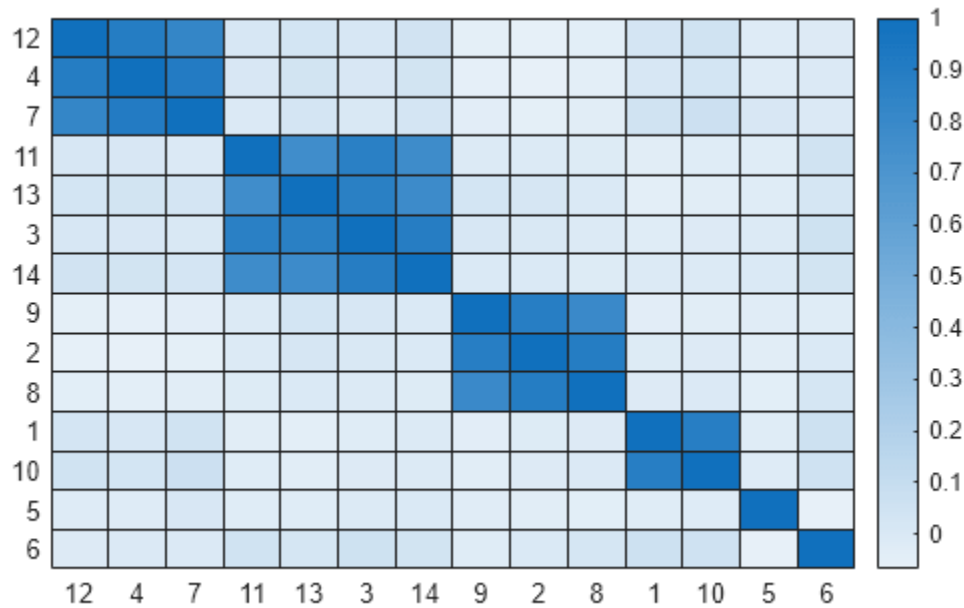
Use the linkage function to compute the matrix that encodes the hierarchical tree of the assets in the universe. Then use the dendrogram function to visualize the hierarchical structure.

```
% Compute the linkage.
link = linkage(distCorr);
figure;
h = dendrogram(link, ColorThreshold='default');
set(h, LineWidth=2);
title('Default Leaf Order');
```



The covariance or correlation matrix can be rearranged to be very close to a block diagonal matrix using the information obtained from the hierarchical tree. Each block in the diagonal shows the assets that are closely related.

```
% Sort the assets for quasi-diagonalization.
nLeafNodes = size(link,1) + 1;
rootNodeId = 2*nLeafNodes - 1;
sortedIdx = getLeafNodesInGroup(rootNodeId,link);
heatmap(C(sortedIdx,sortedIdx), XData=sortedIdx, YData=sortedIdx);
```



The plot shows that there are 6 blocks of assets that are closer together. You can divide the assets into 6 clusters using the `cluster` function.

```
% Get the clusters.
T = cluster(link, MaxClust=6);
```

Hierarchical Risk Parity Algorithm

Given a clustering of the assets, the HRP algorithm presented in this example follows these steps:

- 1 Build a risk parity portfolio within each cluster. The `hrpPortfolio` function in Local Functions on page 4-300 computes the HRP portfolio by receiving a vector with the cluster assignment and a covariance matrix Σ . Then, a risk parity portfolio is computed within each cluster by using `riskBudgetingPortfolio`. The `riskBudgetingPortfolio` function receives a reduced covariance matrix that only includes the information of the assets within the cluster and it returns the weights of the assets in the cluster

$$w^j = \text{riskBudgetingPortfolio}(\Sigma^j),$$

where Σ^j is a matrix whose entries include the covariance information only for the assets in the j th cluster.

2. Compute each cluster's weight using the covariance between each cluster's portfolio. Now, the `riskBudgetingPortfolio` function receives a matrix (Γ) that represents the covariance between the cluster's portfolios (w^j).

$$\Gamma_{ij} = W^T \Sigma W,$$

where $W_{ij} = w_i^j$ if asset i is in cluster j , otherwise $W_{ij} = 0$. In other words $W = (w^1 | \dots | w^K)$, for K clusters and $\gamma = \text{riskBudgetingPortfolio}(\Gamma)$.

3. The final asset allocation is given by the cluster's risk parity portfolio (w^j) multiplied by each cluster's weight (γ_j)

$$w = W\gamma.$$

```
% Compute HRP portfolio
wHRP = hrpPortfolio(T,Sigma)
```

```
wHRP = 14×1
```

```
    0.0854
    0.0584
    0.0423
    0.0567
    0.1867
    0.1648
    0.0521
    0.0540
    0.0539
    0.0755
    0.0397
    0.0513
    0.0394
    0.0399
    :
```

Compare HRP and Mean-Variance Portfolios

Define a long-only, fully-invested mean-variance Portfolio object. Then, compute the associated minimum variance portfolio.

```
% Define the Portfolio object.
p = Portfolio(AssetMean=mean(assetRetn), AssetCovar=Sigma);
p = setDefaultConstraints(p); % Long-only, fully-invested portfolio
% Min variance portfolio
wMV = estimateFrontierLimits(p,'min');
```

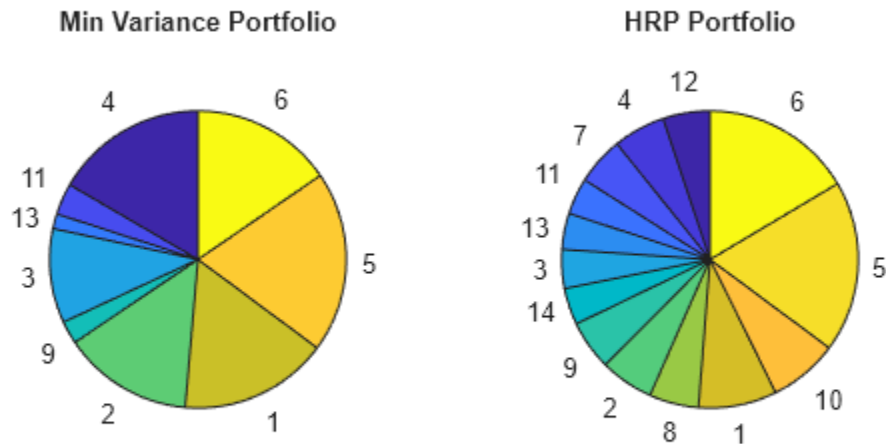
Visualize the resulting allocations from these two strategies.

```
% Create the pie chart labels to improve plot reading.
labels = 1:nAssets;
labels = string(labels);

% Sort the assets following quasi-diagonalization order.
labels = labels(sortedIdx);
wMV = wMV(sortedIdx);
wHRP = wHRP(sortedIdx);

% Plot the pie charts.
tiledlayout(1,2);
% Min variance portfolio
nexttile
pie(wMV(wMV>=1e-8),labels(wMV>=1e-8))
title('Min Variance Portfolio',Position=[0,1.5]);
% HRP portfolio
nexttile
```

```
pie(wHRP,labels)
title('HRP Portfolio',Position=[0,1.5]);
```



You can see that the HRP portfolio is a much more diversified portfolio as compared to the portfolio obtained using the traditional mean-variance approach. In addition, you can see the following:

- The assets that were not correlated with others, assets 5 and 6, represent a much larger area of the pie. In fact, the sum of the areas of assets that are in the same cluster (for example, assets 1 and 10 or assets 11, 13, 3 and 14) are close to the individual areas of assets 5 and 6. This shows that the weights are divided somewhat evenly between clusters and that each cluster's weight is divided somewhat evenly among the assets within the cluster. This is consistent with the HRP theory.
- The risk parity portfolio of fully correlated assets with the same variance is an equally weighted portfolio. The same happens for assets that are completely uncorrelated. Since all the assets in the universe have a similar variance, and assets between clusters are almost uncorrelated, the weights allocated to each cluster are almost even. And, since the assets within a cluster are meant to be highly correlated, the weights of the assets within a cluster are evenly distributed.

References

- 1 Lopez de Prado, M. "Building Diversified Portfolios That Outperform Out of Sample." *The Journal of Portfolio Management*. 42(4), 59-69, 2016.

Local Functions

```
function pwgt = hrpPortfolio(T,Sigma)
% Function that computes a hierarchical risk parity portfolio. The
% algorithm first computes a risk parity portfolio for each cluster. Then,
% each cluster is assigned a weight based on a risk parity allocation of
% the covariance between the cluster's portfolios.
```

```

% Get the problem information.
nAssets = size(Sigma,1);
nClusters = max(T);

% Compute the risk parity portfolio within each cluster.
W = zeros(nAssets,nClusters);
for i = 1:nClusters
    % Identify assets in cluster i and the sub-covariance matrix.
    idx = T == i;
    tempSigma = Sigma(idx,idx);
    % Compute the risk parity portfolio of cluster i.
    W(idx,i) = riskBudgetingPortfolio(tempSigma);
end

% Compute the covariance between the risk parity portfolios of each
% cluster.
covCluster = W'*Sigma*W;

% Compute the weights of each cluster.
wBetween = riskBudgetingPortfolio(covCluster);

% Multiply the weight assigned to each cluster with its portfolio and
% assign to the corresponding assets.
pwgt = W*wBetween;

end

function idxInGroup = getLeafNodesInGroup(nodeId, link)
% getLeafNodesInGroup finds all leaf nodes for a given node id
% in a linkage matrix.

nLeaves= size(link, 1)+1;
if nodeId > nLeaves
    tempNodeIds = link(nodeId-nLeaves,1:2);
    idxInGroup = [getLeafNodesInGroup(tempNodeIds(1), link), ...
        getLeafNodesInGroup(tempNodeIds(2), link)];
else
    idxInGroup = nodeId;
end
end

```

See Also

portfolioRiskContribution | riskBudgetingPortfolio

Related Examples

- “Risk Budgeting Portfolio” on page 4-286
- “Backtest Using Risk-Based Equity Indexation” on page 4-291

External Websites

- Asset Allocation - Hierarchical Risk Parity (2 min 42 sec)

Backtest Strategies Using Deep Learning

Construct trading strategies using a deep learning model and then backtest the strategies using the Financial Toolbox™ backtesting framework. The example uses Deep Learning Toolbox™ to train a predictive model from a set of time series and demonstrates the steps necessary to convert the model output into trading signals. It builds a variety of trading strategies that backtest the signal data over a 5-year period.

This example illustrates the following workflow:

- 1 Load price data for a set of energy commodities on page 4-302.
- 2 Clean and trim data on page 4-303.
- 3 Use the historical data to train a long short-term memory (LSTM) network to predict the change in energy prices over the next trading day on page 4-304.
- 4 Use the LSTM network to build a timetable of trading signal data for the backtest engine on page 4-309.
- 5 Construct trading strategies that allocate capital based on the trading signals on page 4-310.
- 6 Backtest the strategies using the backtesting framework on page 4-310.
- 7 Examine the backtest results on page 4-311.

The focus of this example is on the workflow from data, to a trained model, to trading strategies, and finally to a backtest of the strategies. The deep learning model, its output, the subsequent trading signals, and the strategies are fictional. The intent is only to show the steps for developing and deploying this type of model.

Load Data

Load the historical price data. This data set contains daily spot prices for 12 different energy products ranging from 1986 to 2021 and consists of the following time series:

- WTI — West Texas Intermediate light crude oil
- Brent — Brent light crude oil
- NaturalGas — Henry Hub natural gas
- Propane — Mon Belvieu propane
- Kerosene — US Gulf Coast kerosene-type jet fuel
- HeatingOil — New York Harbor no. 2 heating oil
- GulfRegular — US Gulf Coast conventional gasoline
- LARegular — Los Angeles reformulated RBOB regular gasoline
- NYRegular — New York Harbor conventional gasoline
- GulfDiesel — US Gulf Coast ultra-low sulfur no. 2 diesel
- LADiesel — Los Angeles ultra-low sulfur CARB diesel
- NYDiesel — New York Harbor ultra-low sulfur no. 2 diesel

The source of this data is the US Energy Information Administration (Nov 2021).

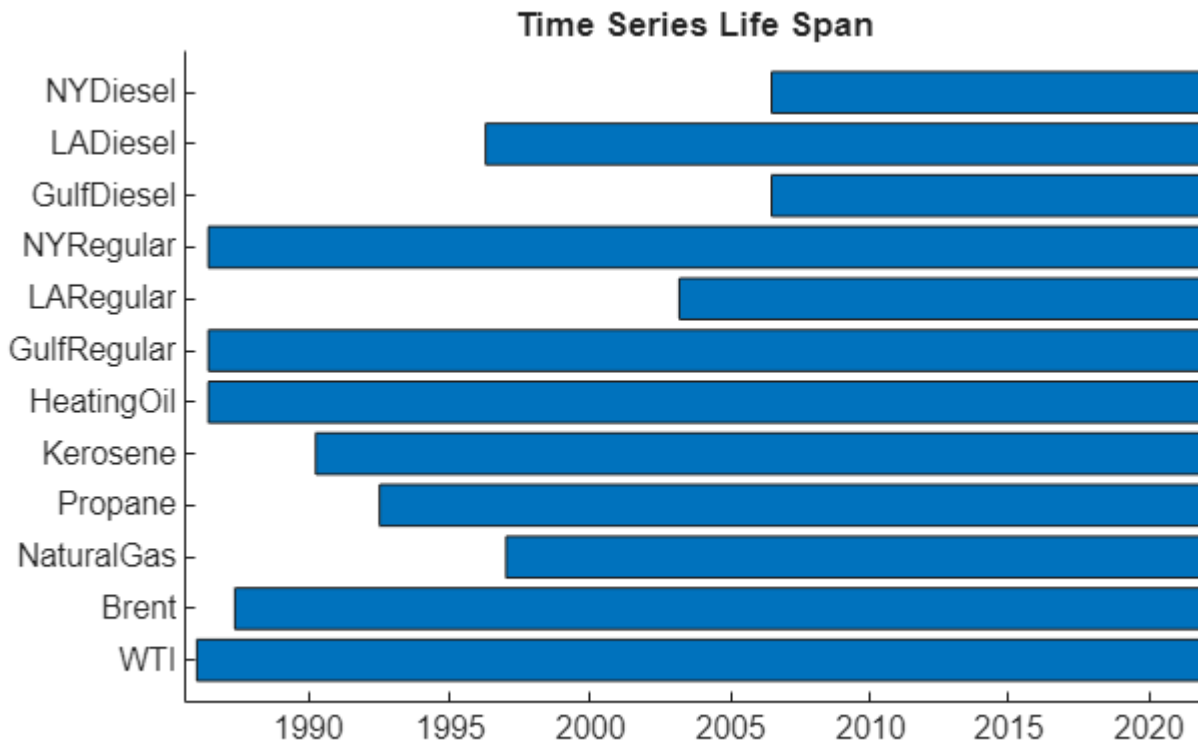
```
priceData = load('energyPrices.mat','energyPrices');
priceData = priceData.energyPrices;
tail(priceData)
```

Time	WTI	Brent	NaturalGas	Propane	Kerosene	HeatingOil	GulfReg
22-Oct-2021	84.53	85.43	5.1	1.485	2.312	2.414	2.481
25-Oct-2021	84.64	84.85	5.72	1.378	2.326	2.429	2.506
26-Oct-2021	85.64	85.11	5.59	1.398	2.339	2.436	2.552
27-Oct-2021	82.66	84.12	5.91	1.365	2.271	2.368	2.469
28-Oct-2021	82.78	83.4	5.68	1.36	2.278	2.363	2.471
29-Oct-2021	83.5	83.1	5.49	1.383	2.285	2.342	2.485
01-Nov-2021	84.08	84.51	5.22	1.385	2.301	2.364	2.457
02-Nov-2021	83.91	84.42	5.33	1.388	2.3	2.405	2.466

Clean and Trim Data

The price datasets do not all start at the same time. Some datasets start later than others and have fewer data points. The following plot shows the time span for each price series.

```
seriesLifespanPlot(priceData)
```



To avoid large spans of missing data, remove the series with shorter histories.

```
priceData = removevars(priceData,["NYDiesel","GulfDiesel","LARegular"]);
```

The remaining table variables contain sporadic missing elements (NaNs) due to holidays or other reasons. Missing data is handled in a variety of ways depending on the dataset. In some cases, it may

be appropriate to interpolate or use the `fillmissing` function. In this example, you can remove the remaining NaN prices.

```
priceData = rmmissing(priceData);
```

Then, convert the price data to a return series using the `tick2ret` function. The final dataset consists of nine price series with daily data from 1997 through 2021.

```
returnData = tick2ret(priceData)
```

```
returnData=6167x9 timetable
    Time          WTI          Brent    NaturalGas    Propane    Kerosene    HeatingOil
    _____  _____  _____  _____  _____  _____  _____
    08-Jan-1997    0.011429    0.00080775    -0.0052356         0    0.012931    0.010142
    09-Jan-1997   -0.0094162    0.0020178         -0.05    -0.036969   -0.0085106   -0.0013
    10-Jan-1997   -0.0057034   -0.024567    0.085873    0.0095969   -0.010014   -0.011
    13-Jan-1997   -0.036329   -0.033443    0.020408   -0.024715   -0.034682   -0.03
    15-Jan-1997    0.029762   -0.0042717         0.085   -0.048733    0.023952    0.02
    16-Jan-1997   -0.019268         0    0.085253   -0.028689   -0.019006   -0.02
    17-Jan-1997   -0.0019646   -0.018876   -0.16985   -0.016878   -0.020864   -
    20-Jan-1997   -0.011811   -0.00043725   -0.16624   -0.027897   -0.022831   -0.02
    21-Jan-1997   -0.011952    0.0052493   -0.082822   -0.004415   -0.014019   -0.02
    22-Jan-1997   -0.016129   -0.0021758    0.020067   -0.0044346    0.031596    0.01
    23-Jan-1997   -0.022541         0   -0.029508   -0.0022272   -0.0061256   -0.0
    24-Jan-1997         0   -0.0056694   -0.11486   -0.075893    0.010786    0.006
    27-Jan-1997         0   -0.010526     0.1374     0.016908    0.012195    0.00
    28-Jan-1997    0.0020964    0.0026596     0.02349   -0.0047506    0.0090361   -0.008
    29-Jan-1997    0.025105    0.017241   -0.045902   -0.042959    0.059701    0.03
    30-Jan-1997    0.012245    0.018253   -0.017182         0    0.016901    0.02
    :
```

Prepare Data for Training LSTM Model

Prepare and partition the dataset in order to train the LSTM model. The model uses a 30-day rolling window of trailing feature data and predicts the next day price changes for four of the assets: Brent crude oil, natural gas, propane, and kerosene.

```
% Model is trained using a 30-day rolling window to predict 1 day in the
% future.
historySize = 30;
futureSize = 1;

% Model predicts returns for oil, natural gas, propane, and kerosene.
outputVarName = ["Brent" "NaturalGas", "Propane" "Kerosene"];
numOutputs = numel(outputVarName);

% start_idx and end_idx are the index positions in the returnData
% timetable corresponding to the first and last date for making a prediction.
start_idx = historySize + 1;
end_idx = height(returnData) - futureSize + 1;
numSamples = end_idx - start_idx + 1;

% The date_vector variable stores the dates for making predictions.
date_vector = returnData.Time(start_idx-1:end_idx-1);
```


Convert the `returnData` timetable to a `numSamples-by-1` cell array. Each cell contains a `numFeatures-by-seqLength` matrix. The response variable is a `numSamples-by-numResponses` matrix.

```
network_features = cell(numSamples,1);
network_responses = zeros(numSamples,numOutputs);

for j = 1:numSamples
    network_features{j} = (returnData(j:j+historySize-1,:).Variables)';
    network_responses(j,:) = ...
        (returnData(j+historySize:j+historySize+futureSize-1,outputVarName).Variables)';
end
```

Split the `network_features` and the `network_responses` into three parts: training, validation, and backtesting. Select the backtesting set as a set of sequential data points. The remainder of the data is randomly split into a training and a validation set. Use the validation set to prevent overfitting while training the model. The backtesting set is not used in the training process, but it is reserved for the final strategy backtest.

```
% Specify rows to use in the backtest (31-Dec-2015 to 2-Nov-2021).
backtest_start_idx = find(date_vector < datetime(2016,1,1),1,'last');
backtest_indices = backtest_start_idx:size(network_responses,1);

% Specify data reserved for the backtest.
Xbacktest = network_features(backtest_indices);
Tbacktest = network_responses(backtest_indices,:);

% Remove the backtest data.
network_features = network_features(1:backtest_indices(1)-1);
network_responses = network_responses(1:backtest_indices(1)-1,:);

% Partition the remaining data into training and validation sets.
rng('default');
cv_partition = cvpartition(size(network_features,1),'HoldOut',0.2);

% Training set
Xtraining = network_features(~cv_partition.test,:);
Ttraining = network_responses(~cv_partition.test,:);

% Validation set
Xvalidation = network_features(cv_partition.test,:);
Tvalidation = network_responses(cv_partition.test,:);
```

Define LSTM Network Architecture

Specify the network architecture as a series of layers. For more information on LSTM networks, see “Long Short-Term Memory Neural Networks” (Deep Learning Toolbox). The Deep Network Designer (Deep Learning Toolbox) is a powerful tool for designing deep learning models.

```
numFeatures = width(returnData);
numHiddenUnits_LSTM = 10;

layers_LSTM = [ ...
    sequenceInputLayer(numFeatures)
    lstmLayer(numHiddenUnits_LSTM)
    layerNormalizationLayer
    lstmLayer(numHiddenUnits_LSTM)
```

```
layerNormalizationLayer
lstmLayer(numHiddenUnits_LSTM,'OutputMode','last')
layerNormalizationLayer
fullyConnectedLayer(numOutputs)];
```

Specify Training Options for LSTM Model

Next, you specify training options using the `trainingOptions` (Deep Learning Toolbox) function. Many training options are available and their use varies depending on your use case. Use the Experiment Manager (Deep Learning Toolbox) to explore different network architectures and sets of network hyperparameters.

```
max_epochs = 500;
mini_batch_size = 128;
learning_rate = 1e-4;

options_LSTM = trainingOptions('adam', ...
    'InputDataFormats','CTB',...
    'Plots','training-progress', ...
    'Verbose',0, ...
    'MaxEpochs',max_epochs, ...
    'MiniBatchSize',mini_batch_size, ...
    'Shuffle','every-epoch', ...
    'ValidationData',{Xvalidation,Tvalidation}, ...
    'ValidationFrequency',50, ...
    'ValidationPatience',10, ...
    'InitialLearnRate',learning_rate, ...
    'GradientThreshold',1);
```

Train LSTM Model

Train the LSTM network. Use the `trainNetwork` (Deep Learning Toolbox) function to train the network until the network meets a stopping criteria. This process can take several minutes depending on the computer running the example. For more information on increasing the network training performance, see “Scale Up Deep Learning in Parallel, on GPUs, and in the Cloud” (Deep Learning Toolbox).

To avoid waiting for the network training, load the pretrained network by setting the `doTrain` flag to `false`. To train the network using `trainNetwork` (Deep Learning Toolbox), set the `doTrain` flag to `true`.

```
doTrain = false;

if doTrain
    % Train the LSTM network.
    net_LSTM = trainnet(Xtraining,Ttraining,layers_LSTM,"mse",options_LSTM);
else
    % Load the pretrained network.
    load lstmBacktestNetwork
end
```

Visualize Training Results

Visualize the results of the trained model by comparing the predicted values against the actual values from the validation set.

```

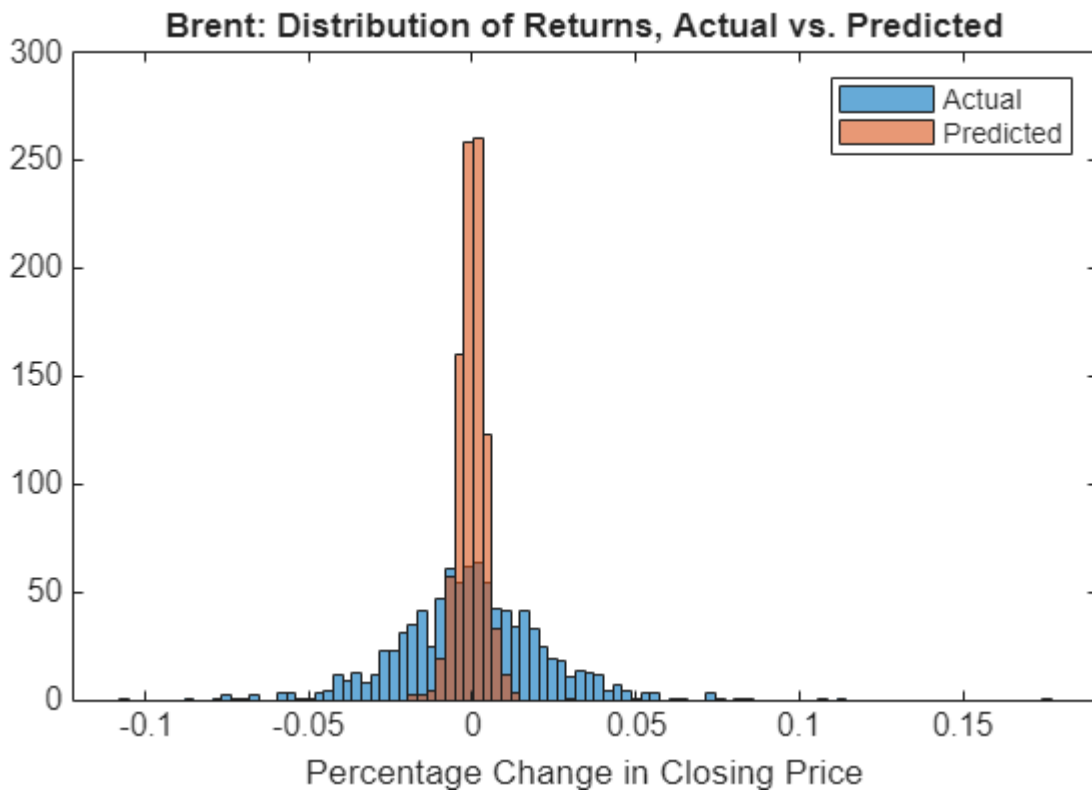
% Compare the actual returns to model predicted returns.
actual = Tvalidation;
predicted = minibatchpredict(net_LSTM,Xvalidation,InputDataFormats="CTB");

% Overlay histogram of actual vs. predicted returns for the validation set.

output_idx = ;
figure;

[~,edges] = histcounts(actual(:,output_idx),100);
histogram(actual(:,output_idx),edges);
hold on
histogram(predicted(:,output_idx),edges)
hold off
xlabel('Percentage Change in Closing Price')
legend('Actual','Predicted')
title(sprintf('%s: Distribution of Returns, Actual vs. Predicted', outputVarName(output_idx)))

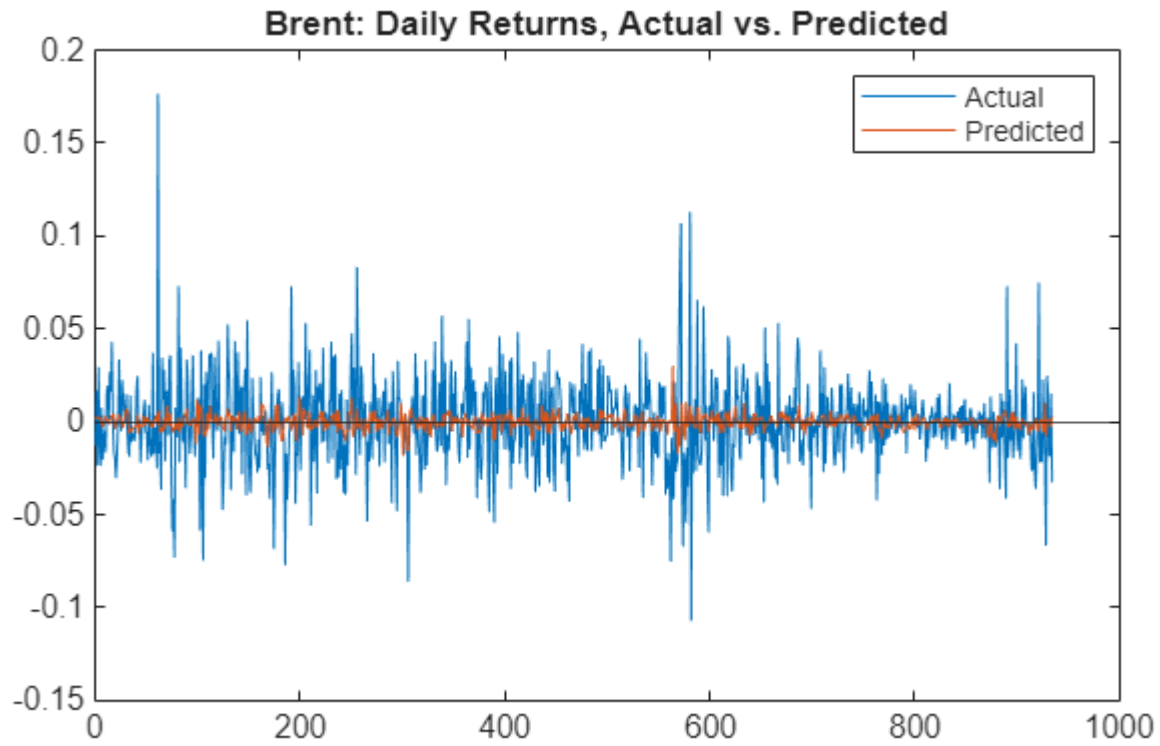
```



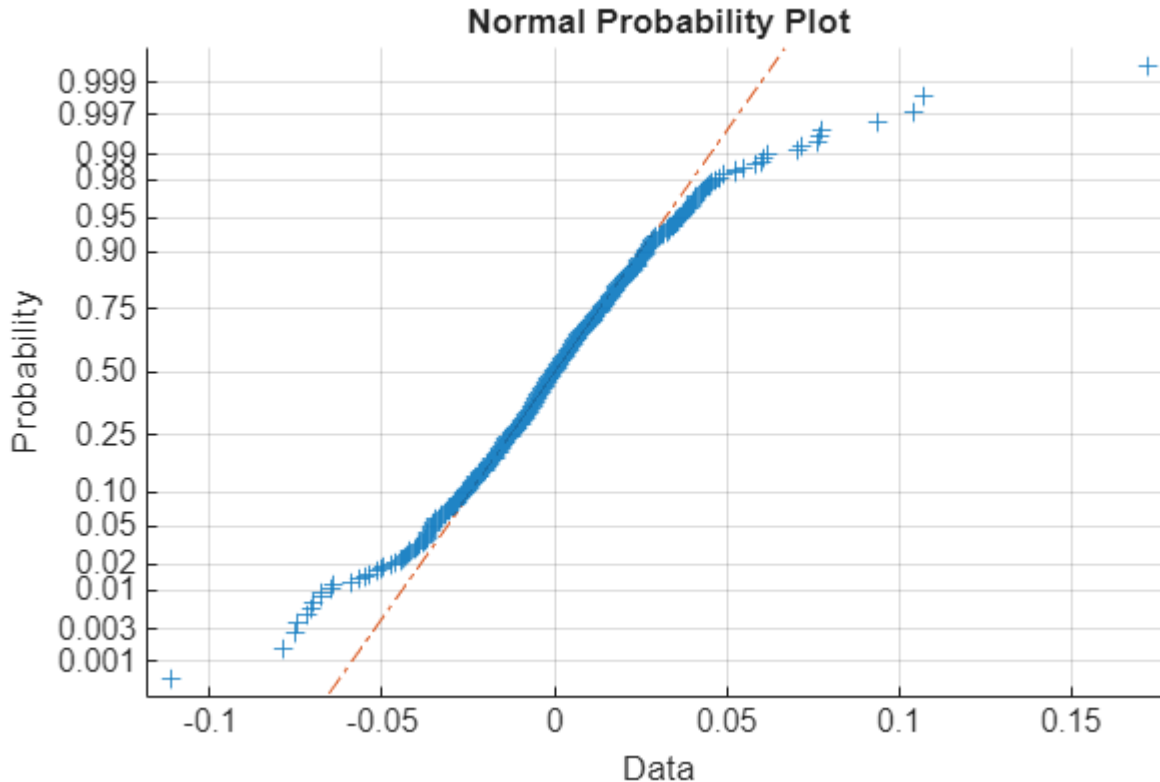
```

% Display the predicted vs. actual daily returns for the validation set.
figure
plot(actual(:,output_idx))
hold on
plot(predicted(:,output_idx))
yline(0)
legend({'Actual','Predicted'})
title(sprintf('%s: Daily Returns, Actual vs. Predicted', outputVarName(output_idx)))

```



```
% Examine the residuals.  
residuals = actual(:,output_idx) - predicted(:,output_idx);  
figure;  
normplot(residuals);
```



The actual data has fatter tails than the trained model predictions. The model predictions are not accurate, but the goal of this example is to show the workflow from loading data, to model development, to backtesting. A more sophisticated model with a larger and more varied set of training data is likely to have more predictive power.

Prepare Backtest Data

Use the predictions from the LSTM model to build the backtest strategies. You can post-process the model output in a number of ways to create trading signals. However, for this example, take the model regression output and convert it to a timetable.

Use `predict` (Deep Learning Toolbox) with the trained network to generate model predictions over the backtest period.

```
backtestPred_LSTM = minibatchpredict(net_LSTM,Xbacktest,InputDataFormats="CTB");
```

Convert the predictions to a trading signal timetable.

```
backtestSignalTT = timetable(date_vector(backtest_indices),backtestPred_LSTM);
```

Construct the prices timetable corresponding to the backtest time span. The backtest trades in and out of the four energy commodities. The prices timetable has the closing price for the day on which the prediction is made.

```
backtestPriceTT = priceData(date_vector(backtest_indices),outputVarName);
```

Set the risk-free rate to be 1% annualized. The backtest engine also supports setting the risk-free rate to a timetable containing the historical daily rates.

```
risk_free_rate = 0.01;
```

Create Backtest Strategies

Use `backtestStrategy` to create four trading strategies based on the signal indicators. The following trading strategies are intended as examples to show how to convert the trading signals into actionable asset allocation strategies that you can then backtest:

- Long Only — Invest all capital across the assets with positive predicted return, proportional to their signal strength (predicted return).
- Long Short — Invest capital across the assets, both long and short positions, proportional to their signal strength.
- Best Bet — Invest all capital into the single asset with the highest predicted return.
- Equal Weight — Rebalance each day to equal-weighted allocation.

```
% Specify 10 basis points as the trading cost.
```

```
tradingCosts = 0.001;
```

```
% Invest in long positions proportionally to their predicted return.
```

```
LongStrategy = backtestStrategy('LongOnly',@LongOnlyRebalanceFcn, ...
    'TransactionCosts',tradingCosts, ...
    'LookbackWindow',1);
```

```
% Invest in both long and short positions proportionally to their predicted returns.
```

```
LongShortStrategy = backtestStrategy('LongShort',@LongShortRebalanceFcn, ...
    'TransactionCosts',tradingCosts, ...
    'LookbackWindow',1);
```

```
% Invest 100% of capital into single asset with highest predicted returns.
```

```
BestBetStrategy = backtestStrategy('BestBet',@BestBetRebalanceFcn, ...
    'TransactionCosts',tradingCosts, ...
    'LookbackWindow',1);
```

```
% For comparison, invest in an equal-weighted (buy low and sell high) strategy.
```

```
equalWeightFcn = @(current_weights,prices,signal) ones(size(current_weights)) / numel(current_weights);
EqualWeightStrategy = backtestStrategy('EqualWeight',equalWeightFcn, ...
    'TransactionCosts',tradingCosts, ...
    'LookbackWindow',0);
```

Put the strategies into an array and then use `backtestEngine` to create the backtesting engine.

```
strategies = [LongStrategy LongShortStrategy BestBetStrategy EqualWeightStrategy];
```

```
bt = backtestEngine(strategies,'RiskFreeRate',risk_free_rate);
```

Run Backtest

Use `runBacktest` to backtest the strategies over the backtest range.

```
bt = runBacktest(bt,backtestPriceTT,backtestSignalTT)
```

```
bt =
    backtestEngine with properties:
```

```
    Strategies: [1x4 backtestStrategy]
    RiskFreeRate: 0.0100
    CashBorrowRate: 0
```

```

    RatesConvention: "Annualized"
    Basis: 0
    InitialPortfolioValue: 10000
    DateAdjustment: "Previous"
    PayExpensesFromCash: 0
    NumAssets: 4
    Returns: [1462×4 timetable]
    Positions: [1×1 struct]
    Turnover: [1462×4 timetable]
    BuyCost: [1462×4 timetable]
    SellCost: [1462×4 timetable]
    TransactionCosts: [1×1 struct]
    Fees: [1×1 struct]

```

Examine Backtest Results

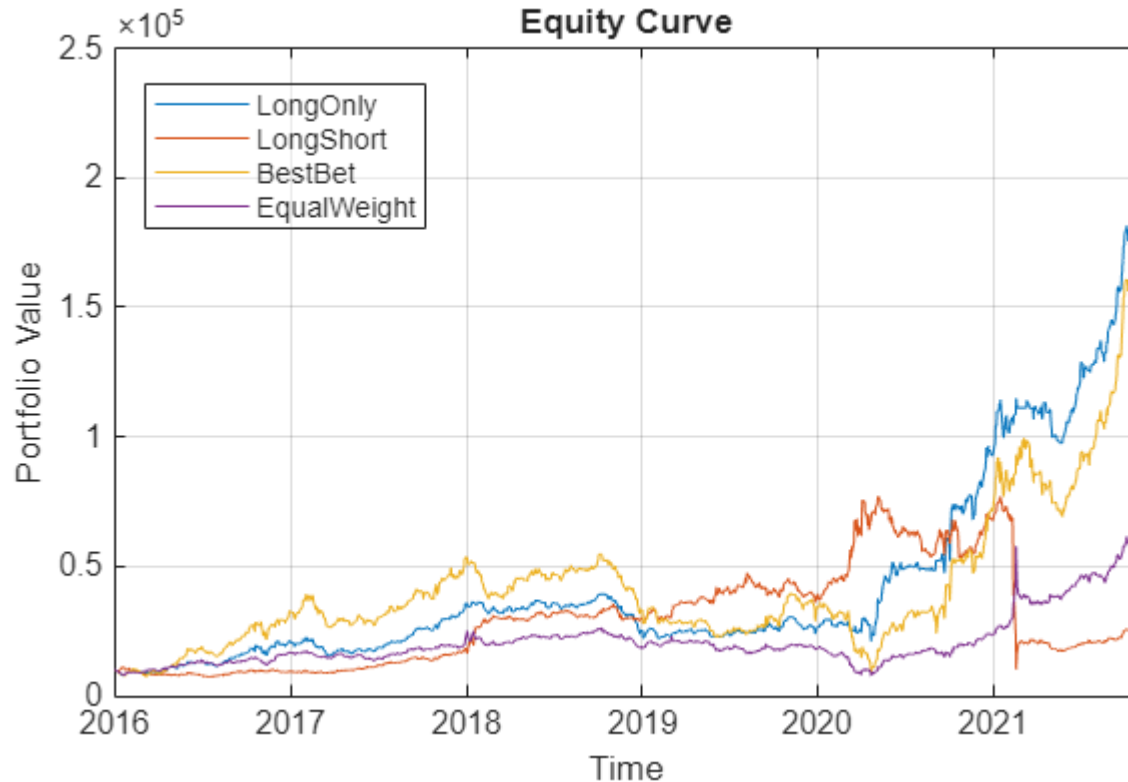
Use the `summary` and `equityCurve` functions to summarize and plot the backtest results. This model and its derivative trading strategies are not expected to be profitable in a realistic trading scenario. However, this example illustrates a workflow that should be useful for practitioners with more comprehensive data sets and more sophisticated models and strategies.

```
summary(bt)
```

```
ans=9×4 table
```

	LongOnly	LongShort	BestBet	EqualWeight
TotalReturn	18.652	1.8281	17.652	4.8347
SharpeRatio	0.085807	0.039827	0.070637	0.056775
Volatility	0.027652	0.033492	0.037706	0.026712
AverageTurnover	0.2372	0.28884	0.25958	0.0095931
MaxTurnover	1	1.3867	1	0.5
AverageReturn	0.0024119	0.0013733	0.0027024	0.001556
MaxDrawdown	0.46656	0.86605	0.82556	0.70509
AverageBuyCost	10.315	8.373	11.393	0.20262
AverageSellCost	10.445	8.4198	11.386	0.19578

```
figure;
equityCurve(bt)
```



Local Functions

```
function new_weights = LongOnlyRebalanceFcn(current_weights,pricesTT,signalTT)
% Long only strategy, in proportion to the signal.
```

```
signal = signalTT.backtestPred_LSTM(end,:);
```

```
if any(0 < signal)
    signal(signal < 0) = 0;
    new_weights = signal / sum(signal);
else
    new_weights = zeros(size(current_weights));
end
end
```

```
function new_weights = LongShortRebalanceFcn(current_weights,pricesTT,signalTT) %#ok<INUSD>
% Long/Short strategy, in proportion to the signal
```

```
signal = signalTT.backtestPred_LSTM(end,:);
abssum = sum(abs(signal));

if 0 < abssum
    new_weights = signal / abssum;
else
    new_weights = zeros(size(current_weights));
end
```



```
end
```

```
function new_weights = BestBetRebalanceFcn(current_weights,pricesTT,signalTT) %#ok<INUSD>
% Best bet strategy, invest in the asset with the most upside.
```

```
signal = signalTT.backtestPred_LSTM(end,:);
new_weights = zeros(size(current_weights));
new_weights(signal == max(signal)) = 1;
```

```
end
```

```
function seriesLifespanPlot(priceData)
% Plot the lifespan of each time series.
```

```
% Specify all time series end on same day.
d2 = numel(priceData.Time);
```

```
% Plot the lifespan patch for each series.
numSeries = size(priceData,2);
for i = 1:numSeries
    % Find start date index.
    d1 = find(~isnan(priceData{:,i}),1,'first');
    % Plot patch.
    x = [d1 d1 d2 d2];
    y = i + [-0.4 0.4 0.4 -0.4];
    patch(x,y,[0 0.4470 0.7410])
```

```
    hold on
```

```
end
```

```
hold off
```

```
% Set the plot properties.
```

```
xlim([-100 d2]);
ylim([0.2 numSeries + 0.8]);
```

```
yticks(1:numSeries);
yticklabels(priceData.Properties.VariableNames');
flipud(gca);
```

```
years = 1990:5:2021;
```

```
xtick_idx = zeros(size(years));
```

```
for yidx = 1:numel(years)
```

```
    xtick_idx(yidx) = find(years(yidx) == year(priceData.Time),1,'first');
```

```
end
```

```
xticks(xtick_idx);
xticklabels(string(years));
```

```
title('Time Series Life Span');
```

```
end
```

See Also

Deep Network Designer | trainNetwork | trainingOptions | backtestStrategy |
backtestEngine | runBacktest | equityCurve | summary

Related Examples

- “Backtest Investment Strategies Using Financial Toolbox” on page 4-238
- “Backtest Investment Strategies with Trading Signals” on page 4-251
- “Backtest Using Risk-Based Equity Indexation” on page 4-291
- “Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data” on page 2-78
- “Backtest with Brinson Attribution to Evaluate Portfolio Performance” on page 4-315
- “Backtest with Brinson Attribution to Evaluate Portfolio Performance” on page 4-315

Backtest with Brinson Attribution to Evaluate Portfolio Performance

This example shows how to compute Brinson attribution using the output of the MATLAB® backtest framework. The backtest framework allows you to build custom trading strategies and then backtest them against historical or simulated market data. The example uses the Brinson attribution function (`brinsonAttribution`) to explain a portfolio's performance versus a benchmark.

Load Data

Brinson attribution requires that a category is assigned to each asset. Load the `dowPortfolio.xlsx` data set, then assign each of the 30 stocks to a category.

```
% Read a table of daily adjusted close prices for 2006 DJIA stocks.
pricesTT = readtimetable('dowPortfolio.xlsx');

% Remove the index from the dataset.
pricesTT = removevars(pricesTT, 'DJI');
num_assets = width(pricesTT);

% Set the sectors for each stock.
asset_categories = categorical(["Materials"; "Financials"; "Financials"; ...
    "Industrials"; "Financials"; "Industrials"; "Materials"; ...
    "Communication Services"; "Industrials"; "Consumer Discretionary"; ...
    "Consumer Discretionary"; "Industrials"; "Information Technology"; ...
    "Information Technology"; "Information Technology"; "Health Care"; ...
    "Financials"; "Consumer Staples"; "Consumer Discretionary"; ...
    "Industrials"; "Consumer Staples"; "Health Care"; ...
    "Information Technology"; "Health Care"; "Consumer Staples"; ...
    "Communication Services"; "Industrials"; "Communication Services"; ...
    "Consumer Staples"; "Energy"]);

% Display the stock categories.
Symbol = pricesTT.Properties.VariableNames(:);
table(Symbol, asset_categories)

ans=30x2 table
    Symbol    asset_categories
    _____    _____
    {'AA' }    Materials
    {'AIG' }    Financials
    {'AXP' }    Financials
    {'BA' }    Industrials
    {'C' }    Financials
    {'CAT' }    Industrials
    {'DD' }    Materials
    {'DIS' }    Communication Services
    {'GE' }    Industrials
    {'GM' }    Consumer Discretionary
    {'HD' }    Consumer Discretionary
    {'HON' }    Industrials
    {'HPQ' }    Information Technology
    {'IBM' }    Information Technology
    {'INTC' }    Information Technology
```

```
{'JNJ' }    Health Care
:
```

Define and Create Backtest Strategies

Multiperiod Brinson attribution expects fixed, regular time periods. In this case, the backtest strategy rebalance schedules will match the Brinson periods. You can set the rebalance dates to be monthly starting from January first.

```
% Rebalance on the first trading day of each month.
rebalance_schedule = datetime(2006,1,1):calmonths(1):datetime(2006,12,1);
for idx = 1:numel(rebalance_schedule)
    priceIdx = find(rebalance_schedule(idx) <= pricesTT.Dates,1,'first');
    rebalance_schedule(idx) = pricesTT.Dates(priceIdx);
end
```

The risk budgeting strategy needs historical data in order to set initial weights. You can use the first month of price data as a warm-up period to set the initial weights. The backtest begins at the end of the warm-up period.

```
% Use first month as warm-up period for the risk budgeting strategy.
start_date = rebalance_schedule(2);

% Get the first month of prices to initialize the risk budgeting strategy.
initial_prices = pricesTT(pricesTT.Dates <= start_date,:);
```

Brinson attribution measures portfolio manager performance against some benchmark. For this example, use a simple equal-weighted strategy as the benchmark. This example examines the performance of the following two strategies relative to this benchmark:

- Equal Category — This strategy allocates resources equally *per category*, and then allocates equal weights for stocks within each category.
- Risk Budgeting — This strategy sets weights for stocks with the goal of minimizing each asset's risk contribution.

Create a `backtestStrategy` object for the benchmark and the two candidate strategies. Define each of these strategies to use the same rebalance schedule and assign the appropriate initial weights.

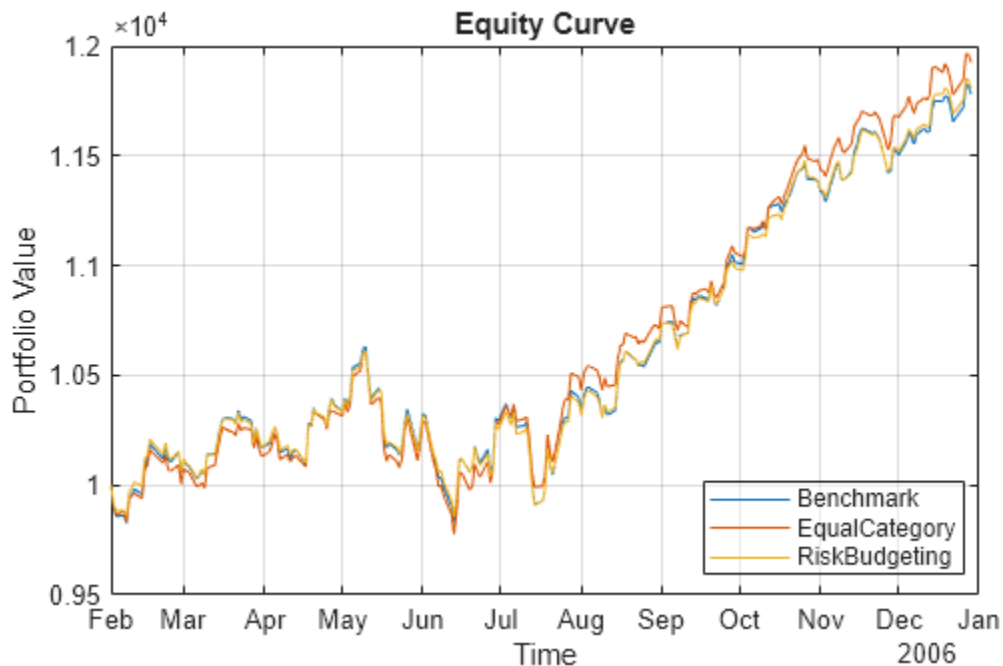
```
% Define the Benchmark strategy.
benchmark = backtestStrategy('Benchmark',@equalWeightRebalanceFcn, ...
    'RebalanceFrequency',rebalance_schedule, ...
    'InitialWeights',computeEqualWeights(num_assets), ...
    'UserData',struct());

% Define the equal category strategy.
equalCategory = backtestStrategy('EqualCategory',@equalCategoryRebalanceFcn, ...
    'RebalanceFrequency',rebalance_schedule, ...
    'InitialWeights',computeEqualCategory(asset_categories), ...
    'UserData',struct('Categories',asset_categories));

% Define the risk budgeting strategy.
riskBudgeting = backtestStrategy('RiskBudgeting',@riskBudgetingRebalanceFcn, ...
    'RebalanceFrequency',rebalance_schedule, ...
    'InitialWeights',computeRiskBudgeting(initial_prices));
```

Use `backtestEngine` to create a `backtestEngine` object for the strategies and then use `runBacktest` to run the backtest. Display the equity curve using `equityCurve` and then use `summary` to display the backtest summary table. Both the equal category and risk budgeting strategies outperform the benchmark.

```
strategies = [benchmark, equalCategory, riskBudgeting];
bt = backtestEngine(strategies);
bt = runBacktest(bt, pricesTT, 'Start', start_date);
equityCurve(bt)
```



```
summary(bt)
```

```
ans=9×3 table
```

	Benchmark	EqualCategory	RiskBudgeting
TotalReturn	0.17811	0.19249	0.18065
SharpeRatio	0.11671	0.12645	0.12393
Volatility	0.0062905	0.0062211	0.0059849
AverageTurnover	0.00073556	0.00076049	0.0015965
MaxTurnover	0.025681	0.028611	0.080538
AverageReturn	0.00073259	0.00078493	0.00074008
MaxDrawdown	0.07502	0.078323	0.071364
AverageBuyCost	0	0	0
AverageSellCost	0	0	0

Prepare Backtest Results for Brinson Attribution

Set the strategy name of the benchmark.

```
benchmark_name = 'Benchmark';
```

Define Brinson periods by the rebalance dates and the backtest end date.

```
period_dates = [rebalance_schedule(2:end), pricesTT.Dates(end)];
```

Compute Brinson Attribution for Backtested Investment Strategy

Select whether to analyze the equal category or risk budgeting strategy. Then, use the `backtest2brinson` function in Local Functions on page 4-320 to compute `asset_table` that is formatted for use with the `brinsonAttribution` function. For more information on the format of the `asset_table`, see the description for the `AssetTable` input argument in the `brinsonAttribution` function.

```
% Select strategy to analyze
```

```
strategy_name =  ;
```

```
asset_table = backtest2brinson(bt,strategy_name,benchmark_name,pricesTT,asset_categories,period_
head(asset_table))
```

Period	Name	Return	Category	PortfolioWeight	BenchmarkWeight
1	"AA"	-0.050283	Materials	0.055556	0.033333
1	"AIG"	0.0015356	Financials	0.027778	0.033333
1	"AXP"	0.028439	Financials	0.027778	0.033333
1	"BA"	0.022929	Industrials	0.018519	0.033333
1	"C"	0.015045	Financials	0.027778	0.033333
1	"CAT"	0.074416	Industrials	0.018519	0.033333
1	"DD"	0.037791	Materials	0.055556	0.033333
1	"DIS"	0.11182	Communication Services	0.037037	0.033333

```
% Compute Brinson attribution
```

```
brinson = brinsonAttribution(asset_table);
summary(brinson)
```

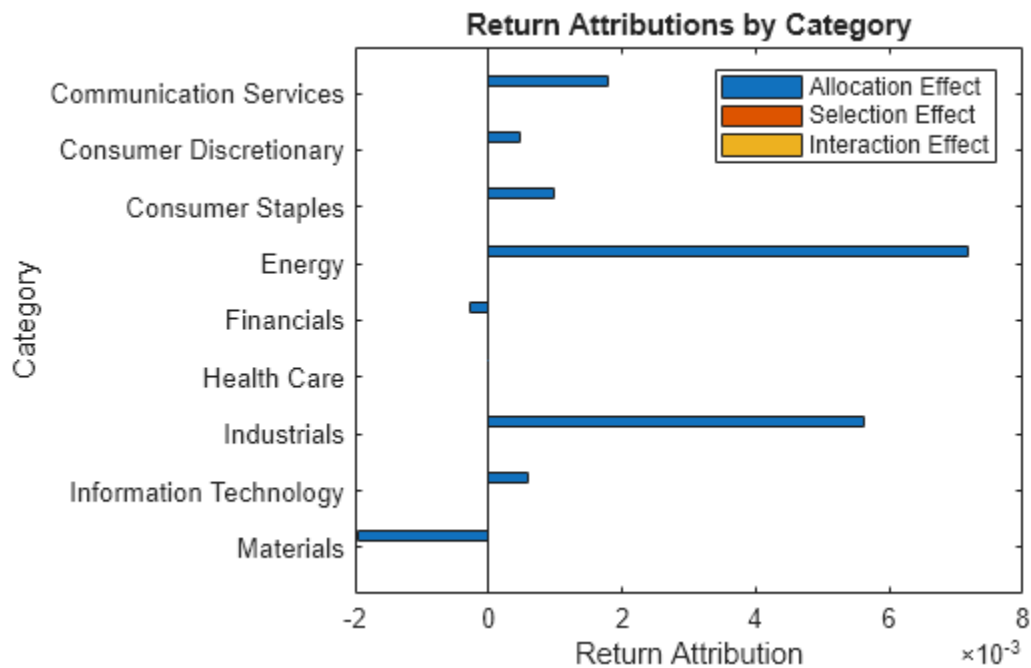
```
ans=11x1 table
```

Brinson Attribution Summary

Total Number of Assets	30
Number of Assets in Portfolio	30
Number of Assets in Benchmark	30
Number of Periods	11
Number of Categories	9
Portfolio Return	0.19249
Benchmark Return	0.17811
Active Return	0.014376
Allocation Effect	0.014376
Selection Effect	8.5567e-18
Interaction Effect	-1.1701e-19

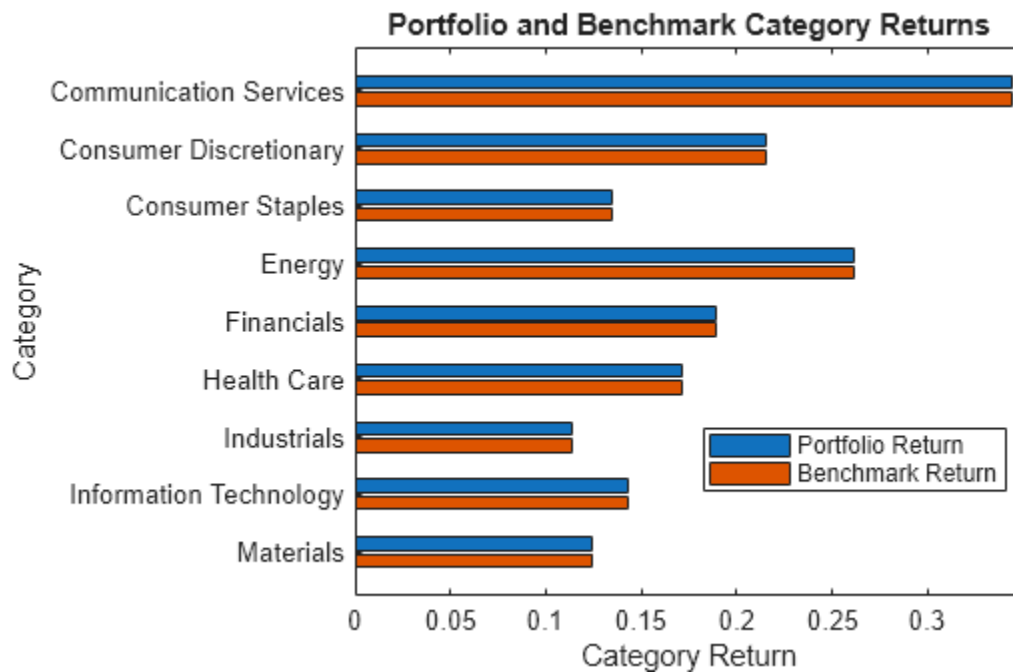
Generate the attribution chart using `attributionsChart` with the `brinson` object. The `attributionsChart` creates a horizontal bar chart of portfolio performance attributions by category, aggregated over all time periods.

```
attributionsChart(brinson)
```



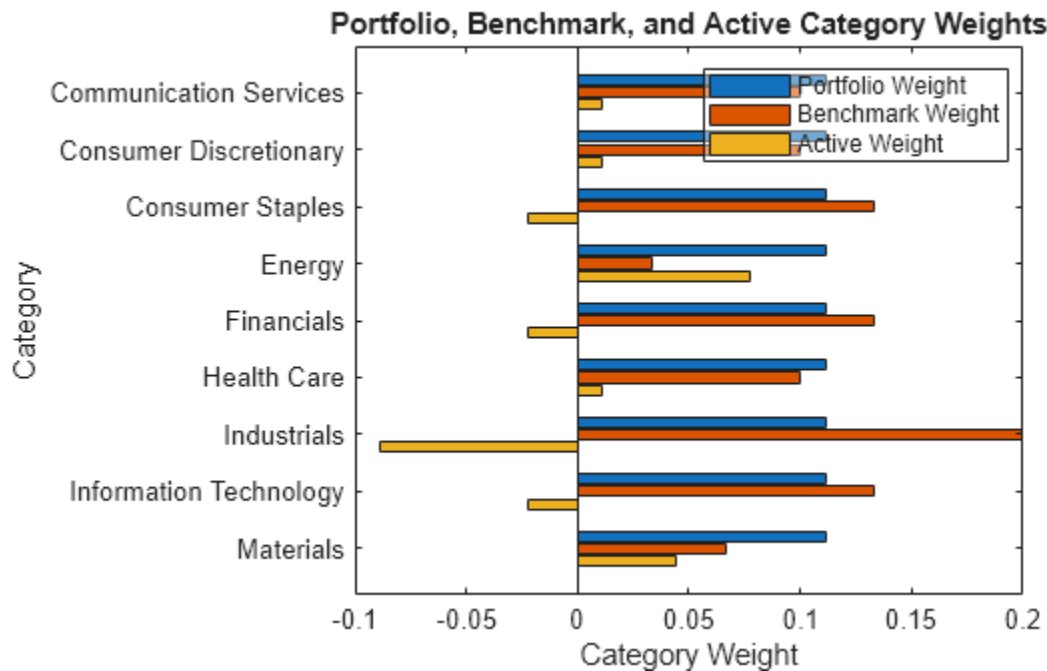
Generate a category returns chart using `categoryReturnsChart` with the `brinson` object. The category returns chart creates a horizontal bar chart of portfolio and benchmark category returns, aggregated over all time periods.

```
categoryReturnsChart(brinson)
```



Generate a category weights chart using `categoryWeightsChart` with the `brinson` object. The category weights chart creates a horizontal bar chart of portfolio, benchmark, and active weights by category, averaged over all time periods.

```
categoryWeightsChart(brinson)
```



The equal category strategy, despite having different category weights from the benchmark, has the same equal-weighted asset selection process *within each category*. Therefore, the equal category strategy has no selection or interaction effect (relative to the benchmark). Meanwhile, the risk budgeting strategy disregards the asset categories when setting portfolio weights, so it has selection and interaction effects.

Local Functions

Both the equal asset weight and equal category weight rebalance functions compute a fixed weight that is assigned at each rebalance date. In order to avoid calculating the weights each time, the weights are calculated once and then the result is saved in the `UserData` struct for the `backtestStrategy` object. For more information on the `UserData` struct, see the “rebalanceFcn” on page 15-0 input argument for `backtestStrategy`.

```
function [new_weights,user_data] = equalWeightRebalanceFcn(~,pricesTT,user_data)
% Equal asset weight rebalance function

if ~isfield(user_data,'FixedAllocation')
    % If this is the first call to the rebalance function, calculate the
    % desired fixed allocation and save it.
    user_data.FixedAllocation = computeEqualWeights(width(pricesTT));
end
new_weights = user_data.FixedAllocation;

end
```



```

function weights = computeEqualWeights(num_assets)
% Equal asset weight portfolio allocation

weights = ones(1,num_assets) / num_assets;

end

function [new_weights,user_data] = equalCategoryRebalanceFcn(~, ~, user_data)
% Equal category weight rebalance function

if ~isfield(user_data,'FixedAllocation')
    % If this is the first call to the rebalance function, calculate the
    % desired fixed allocation and save it.
    user_data.FixedAllocation = computeEqualCategory(user_data.Categories);
end
new_weights = user_data.FixedAllocation;

end

function weights = computeEqualCategory(asset_categories)
% Equal category weight portfolio allocation

weights = zeros(1,numel(asset_categories));
unique_categories = unique(asset_categories);
category_weight = 1 / numel(unique_categories);

for i = 1:numel(unique_categories)
    category_mask = asset_categories == unique_categories(i);
    weights(category_mask) = category_weight / sum(category_mask);
end

end

function new_weights = riskBudgetingRebalanceFcn(~,pricesTT)
% Risk budgeting rebalance function

new_weights = computeRiskBudgeting(pricesTT);

end

function new_weights = computeRiskBudgeting(pricesTT)
% Risk budgeting portfolio allocation

asset_returns = tick2ret(pricesTT);
asset_cov = cov(asset_returns{:,:});
new_weights = riskBudgetingPortfolio(asset_cov);

end

function asset_table = backtest2brinson(bt,strategy_name,benchmark_name,pricesTT,asset_categories)
% Build Brinson attribution input asset table based on the results of the
% completed backtest.

```

```
% Compute asset returns per period
asset_returns = tick2ret(pricesTT(period_dates,:));
num_periods = height(asset_returns);

% Brinson Return input
Return = asset_returns.Variables';

% Brinson Category input
Category = repmat(asset_categories,1,num_periods);

% Brinson Period input
num_assets = width(pricesTT);
Period = (1:num_periods) .* ones(num_assets,1);

% Brinson Name input
Name = repmat(string(pricesTT.Properties.VariableNames(:)),1,num_periods);

% Benchmark weights
benchmark_positions = bt.Positions.(benchmark_name){period_dates(1:end-1),2:end}';
BenchmarkWeight = benchmark_positions ./ sum(benchmark_positions);

% Strategy weights
portfolio_positions = bt.Positions.(strategy_name){period_dates(1:end-1),2:end}';
PortfolioWeight = portfolio_positions ./ sum(portfolio_positions);

% Aggregate the inputs into the Brinson asset table
asset_table = table(Period(:),Name(:),Return(:),Category(:), ...
    PortfolioWeight(:),BenchmarkWeight(:),...
    VariableNames=["Period","Name","Return","Category", ...
        "PortfolioWeight","BenchmarkWeight"]);
end
```

See Also

[brinsonAttribution](#) | [categoryAttribution](#) | [categoryReturns](#) | [categoryWeights](#) | [totalAttribution](#) | [summary](#)

Related Examples

- “Backtest Investment Strategies Using Financial Toolbox” on page 4-238
- “Backtest Investment Strategies with Trading Signals” on page 4-251
- “Backtest Using Risk-Based Equity Indexation” on page 4-291

Analyze Performance Attribution Using Brinson Model

This example shows how to prepare data, create a `brinsonAttribution` object, and then analyze the performance attribution with respect to category (sector) weights and returns. In this example, you use the `categoryAttribution`, `categoryReturns`, `categoryWeights`, `totalAttribution`, and `summary` functions for the analysis. Also, you can generate plots for the results, using `categoryReturnsChart`, `categoryWeightsChart`, and `attributionsChart`.

Prepare Data

Load the stock price data into a table.

```
T = readtable('dowPortfolio.xlsx');
MonthIdx = [1;20;39;62;81;103;125;145;168;188;210;231;251];
MonthlyPrices = T(MonthIdx,3:end);
```

Compute the monthly returns using `tick2ret`.

```
MonthlyReturns = tick2ret(MonthlyPrices.Variables)';
[NumAssets,NumPeriods] = size(MonthlyReturns);
```

Define the asset categories (sectors) using `categorical`.

```
Category = categorical(["Materials";"Financials";"Financials"; ...
    "Industrials";"Financials";"Industrials";"Materials"; ...
    "Communication Services";"Industrials";"Consumer Discretionary"; ...
    "Consumer Discretionary";"Industrials";"Information Technology"; ...
    "Information Technology";"Information Technology";"Health Care"; ...
    "Financials";"Consumer Staples";"Consumer Discretionary"; ...
    "Industrials";"Consumer Staples";"Health Care"; ...
    "Information Technology";"Health Care";"Consumer Staples"; ...
    "Communication Services";"Industrials";"Communication Services"; ...
    "Consumer Staples";"Energy"]);
Category = repmat(Category,1,NumPeriods);
```

Define Benchmark and Portfolio Weights

Define the benchmark weight and the portfolio weights, by quarter, for each of the stock assets.

```
BenchmarkWeight = 1./length(MonthlyReturns).*ones(NumAssets, NumPeriods);
```

```
PortfolioWeightQ1 = [0;0;0.022;0.033;0;0.044;0.022;0.011;0.065;0.033; ...
    0.055;0.072;0;0.04;0;0.05;0.08;0.042;0.03;0.043;0.055;0.036;0.111; ...
    0.036;0;0.03;0;0.05;0;0.04]*ones(1,3);
```

```
PortfolioWeightQ2 = [0;0;0.022;0.033;0;0.044;0.022;0.011;0.049;0.033; ...
    0.055;0.074;0;0.04;0;0.05;0.08;0.042;0.03;0.02;0.055;0.036;0.148; ...
    0.036;0;0.03;0;0.05;0;0.04]*ones(1,3);
```

```
PortfolioWeightQ3 = [0;0;0.022;0.033;0;0.042;0.022;0.01;0.049;0.033; ...
    0.05;0.07;0;0.04;0;0.05;0.08;0.042;0.03;0.02;0.055;0.036;0.16; ...
    0.036;0;0.03;0;0.05;0;0.04]*ones(1,3);
```

```
PortfolioWeightQ4 = [0;0;0.022;0.033;0;0.042;0.02;0.01;0.039;0.033; ...
    0.05;0.07;0;0.04;0;0.05;0.08;0.042;0.03;0.02;0.055;0.036;0.198; ...
    0.036;0;0.03;0;0.03;0;0.034]*ones(1,3);
```

```
PortfolioWeight = [PortfolioWeightQ1 PortfolioWeightQ2 ...
    PortfolioWeightQ3 PortfolioWeightQ4];

Period = (1:NumPeriods).*ones(NumAssets,1);

Name = repmat(string(MonthlyPrices.Properties.VariableNames(:)),1,NumPeriods);
```

Create AssetTable Input

Use table to create the AssetTable input to use when creating a brinsonAttribution object.

```
AssetTable = table(Period(:), Name(:), ...
    MonthlyReturns(:), Category(:), ...
    PortfolioWeight(:), BenchmarkWeight(:), ...
    VariableNames=["Period", "Name", "Return", "Category", ...
    "PortfolioWeight", "BenchmarkWeight"])
```

AssetTable=360×6 table

Period	Name	Return	Category	PortfolioWeight	BenchmarkWeight
1	"AA"	0.053621	Materials	0	0.033333
1	"AIG"	-0.05964	Financials	0	0.033333
1	"AXP"	-0.00019406	Financials	0.022	0.033333
1	"BA"	-0.030162	Industrials	0.033	0.033333
1	"C"	-0.055015	Financials	0	0.033333
1	"CAT"	0.17956	Industrials	0.044	0.033333
1	"DD"	-0.090708	Materials	0.022	0.033333
1	"DIS"	0.037221	Communication Services	0.011	0.033333
1	"GE"	-0.07381	Industrials	0.065	0.033333
1	"GM"	0.27273	Consumer Discretionary	0.033	0.033333
1	"HD"	-0.016838	Consumer Discretionary	0.055	0.033333
1	"HON"	0.025457	Industrials	0.072	0.033333
1	"HPQ"	0.083598	Information Technology	0	0.033333
1	"IBM"	-0.009235	Information Technology	0.04	0.033333
1	"INTC"	-0.1685	Information Technology	0	0.033333
1	"JNJ"	-0.066351	Health Care	0.05	0.033333
:					

Create brinsonAttribution Object

Use brinsonAttribution to create a brinsonAttribution object.

```
BrinsonPAobj = brinsonAttribution(AssetTable)
```

```
BrinsonPAobj =
    brinsonAttribution with properties:
```

```
    NumAssets: 30
  NumPortfolioAssets: 22
  NumBenchmarkAssets: 30
    NumPeriods: 12
   NumCategories: 9
    AssetName: [30×1 string]
   AssetReturn: [30×12 double]
  AssetCategory: [30×12 categorical]
PortfolioAssetWeight: [30×12 double]
```

```

BenchmarkAssetWeight: [30×12 double]
PortfolioCategoryReturn: [9×12 double]
BenchmarkCategoryReturn: [9×12 double]
PortfolioCategoryWeight: [9×12 double]
BenchmarkCategoryWeight: [9×12 double]
PortfolioReturn: 0.2234
BenchmarkReturn: 0.2046
ActiveReturn: 0.0188

```

Compute Category Weights

Use the `brinsonAttribution` object and `categoryWeights` to compute the average and periodic category weights for the portfolio and the benchmark, as well as, the corresponding active weights.

```
[AverageCategoryWeights, PeriodicCategoryWeights] = categoryWeights(BrinsonPAobj)
```

AverageCategoryWeights=9×4 table

Category	AveragePortfolioWeight	AverageBenchmarkWeight	AverageActiveWeight
Communication Services	0.0855	0.1	-0.0145
Consumer Discretionary	0.1155	0.1	0.0155
Consumer Staples	0.097	0.13333	-0.036333
Energy	0.0385	0.033333	0.0051667
Financials	0.102	0.13333	-0.031333
Health Care	0.122	0.1	0.022
Industrials	0.22375	0.2	0.02375
Information Technology	0.19425	0.13333	0.060917
Materials	0.0215	0.066667	-0.045167

PeriodicCategoryWeights=108×5 table

Period	Category	PortfolioWeight	BenchmarkWeight	ActiveWeight
1	Communication Services	0.091	0.1	-0.009
1	Consumer Discretionary	0.118	0.1	0.018
1	Consumer Staples	0.097	0.13333	-0.036333
1	Energy	0.04	0.033333	0.0066667
1	Financials	0.102	0.13333	-0.031333
1	Health Care	0.122	0.1	0.022
1	Industrials	0.257	0.2	0.057
1	Information Technology	0.151	0.13333	0.017667
1	Materials	0.022	0.066667	-0.044667
2	Communication Services	0.091	0.1	-0.009
2	Consumer Discretionary	0.118	0.1	0.018
2	Consumer Staples	0.097	0.13333	-0.036333
2	Energy	0.04	0.033333	0.0066667
2	Financials	0.102	0.13333	-0.031333
2	Health Care	0.122	0.1	0.022
2	Industrials	0.257	0.2	0.057
:				

Compute Category Returns

Use the `brinsonAttribution` object and `categoryReturns` to compute the aggregate and periodic category (sector) returns for the portfolio and the benchmark.

```
[AggregateCategoryReturns,PeriodicCategoryReturns] = categoryReturns(BrinsonPAobj)
```

AggregateCategoryReturns=9×3 *table*

Category	AggregatePortfolioReturn	AggregateBenchmarkReturn
Communication Services	0.41756	0.42797
Consumer Discretionary	0.27772	0.34718
Consumer Staples	0.20572	0.14112
Energy	0.33598	0.33598
Financials	0.22678	0.15838
Health Care	0.19451	0.20679
Industrials	0.14132	0.14253
Information Technology	0.14339	0.13485
Materials	0.17109	0.10402

PeriodicCategoryReturns=108×4 *table*

Period	Category	PortfolioReturn	BenchmarkReturn
1	Communication Services	0.056262	0.052385
1	Consumer Discretionary	0.079767	0.10017
1	Consumer Staples	-0.01485	-0.0033588
1	Energy	0.073093	0.073093
1	Financials	-0.0021179	-0.029374
1	Health Care	0.012124	0.022297
1	Industrials	0.0018686	0.0088589
1	Information Technology	0.033481	-0.011315
1	Materials	-0.090708	-0.018543
2	Communication Services	0.069253	0.078033
2	Consumer Discretionary	-0.023091	-0.036416
2	Consumer Staples	0.0027749	0.00096619
2	Energy	-0.0487	-0.0487
2	Financials	0.033379	0.02054
2	Health Care	0.014975	0.015929
2	Industrials	0.047732	0.041906
:			

Compute Category Attribution

Use the `brinsonAttribution` object and `categoryAttribution` to compute the performance attribution of the portfolio for each category (sector).

```
[AggregateCategoryAttribution,PeriodicCategoryAttribution] = categoryAttribution(BrinsonPAobj)
```

AggregateCategoryAttribution=9×5 *table*

Category	Allocation	Selection	Interaction	ActiveReturn
Communication Services	-0.0025318	-0.00071062	0.0002061	-0.0030363
Consumer Discretionary	0.0024505	-0.00678	-0.0011609	-0.0054903
Consumer Staples	0.0023692	0.0093618	-0.0025511	0.00918
Energy	0.00038824	1.3872e-19	2.7745e-20	0.00038824
Financials	0.0013464	0.0097268	-0.0022858	0.0087874
Health Care	0.00011567	-0.0013031	-0.00028667	-0.0014741
Industrials	0.0016701	-0.00035051	-5.7623e-05	0.001262
Information Technology	0.0015686	0.0017812	0.00016898	0.0035187

Materials 0.0040725 0.0050687 -0.0034678 0.0056734

PeriodicCategoryAttribution=108×7 table

Period	LinkingCoefficient	Category	Allocation	Selection	Interaction
1	1.1901	Communication Services	-0.00034021	0.00038766	-3.4e-05
1	1.1901	Consumer Discretionary	0.0015405	-0.0020403	-0.0004998
1	1.1901	Consumer Staples	0.00065192	-0.0015322	0.00088028
1	1.1901	Energy	0.00039006	0	0.00039006
1	1.1901	Financials	0.0013774	0.0036342	-0.0022568
1	1.1901	Health Care	0.00016968	-0.0010173	-0.00084762
1	1.1901	Industrials	-0.00032634	-0.0013981	-0.00172444
1	1.1901	Information Technology	-0.00045756	0.0059728	0.00551524
1	1.1901	Materials	0.0014797	-0.004811	-0.0033313
2	1.1935	Communication Services	-0.00058016	-0.000878	-7.9e-05
2	1.1935	Consumer Discretionary	-0.00089975	0.0013325	0.00043275
2	1.1935	Consumer Staples	0.00045794	0.00024116	-6.5e-05
2	1.1935	Energy	-0.00041514	0	-0.00041514
2	1.1935	Financials	-0.00021838	0.0017119	-0.00149352
2	1.1935	Health Care	5.19e-05	-9.5418e-05	-2.09e-05
2	1.1935	Industrials	0.0016152	0.0011651	0.0004501
⋮					

Compute Total Attribution

Use the `brinsonAttribution` object and `totalAttribution` to compute the total performance attribution of the portfolio summed up for all categories (sectors).

[AggregateTotalAttribution,PeriodicTotalAttribution] = `totalAttribution(BrinsonPAobj)`

AggregateTotalAttribution=1×4 table

Allocation	Selection	Interaction	ActiveReturn
0.011449	0.016794	-0.0094348	0.018809

PeriodicTotalAttribution=12×6 table

Period	LinkingCoefficient	Allocation	Selection	Interaction	ActiveReturn
1	1.1901	0.0044852	-0.00080413	0.0025538	0.0062349
2	1.1935	0.00087285	0.0032617	-0.0025973	0.0015372
3	1.1904	-6.9025e-05	0.0052394	0.00069386	0.0058643
4	1.208	-0.004707	-0.0095748	-0.0046622	-0.018944
5	1.1995	-0.00030704	-0.006091	-0.00061525	-0.0070132
6	1.1935	0.0012025	-0.00026724	0.00058932	0.0015246
7	1.1813	0.0032236	0.017541	-0.0021062	0.018658
8	1.196	0.004629	-0.004929	-0.0018038	-0.0021038
9	1.1927	0.0013046	0.00036681	0.00099195	0.0026634
10	1.1923	0.0032172	0.0023077	-0.0023663	0.0031586
11	1.194	-0.001881	0.00044958	0.0022018	0.00077039
12	1.1918	-0.0022819	0.0069324	-0.0007375	0.003913

Generate Summary for Performance Attribution

Use the `brinsonAttribution` object and `summary` to generate a table that summarizes the final results (aggregated over all time periods and categories) of the performance attribution using the Brinson model.

```
SummaryTable = summary(BrinsonPAobj)
```

SummaryTable=11x1 table

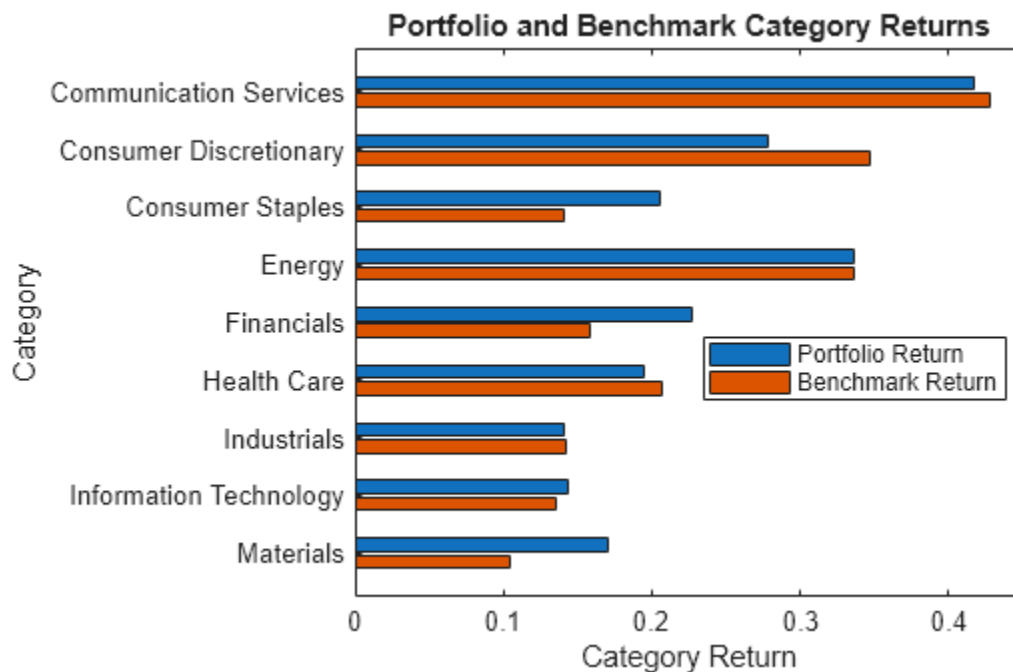
Brinson Attribution Summary

Total Number of Assets	30
Number of Assets in Portfolio	22
Number of Assets in Benchmark	30
Number of Periods	12
Number of Categories	9
Portfolio Return	0.22345
Benchmark Return	0.20464
Active Return	0.018809
Allocation Effect	0.011449
Selection Effect	0.016794
Interaction Effect	-0.0094348

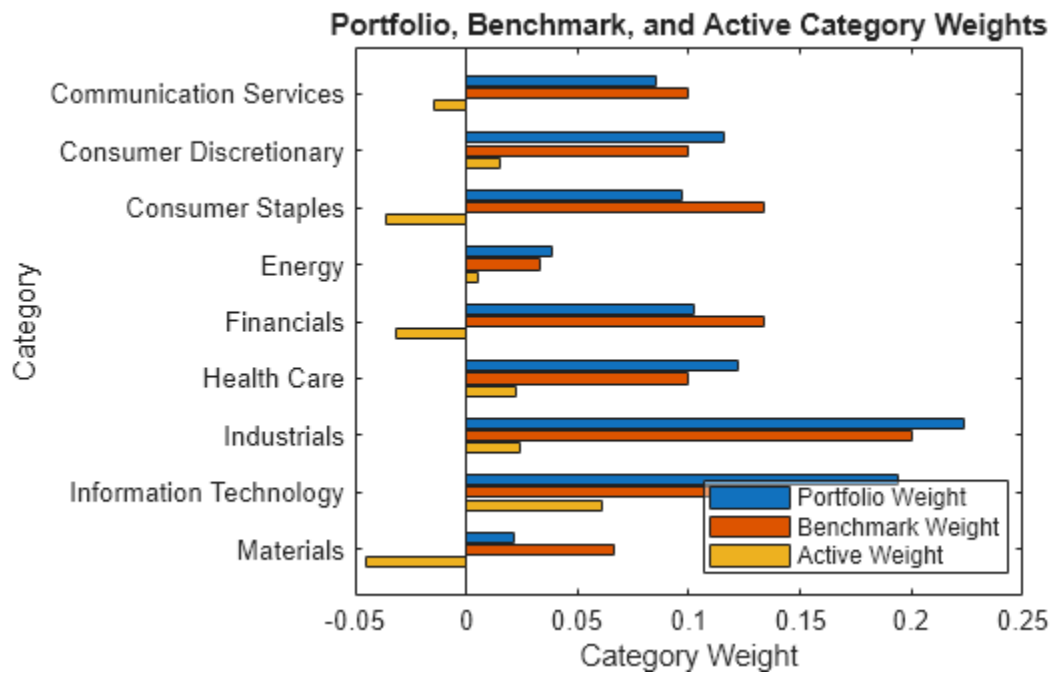
Generate Horizontal Bar Charts for Returns, Weights, and Performance Attribution

Use the `brinsonAttribution` object and `categoryReturnsChart`, `categoryWeightsChart`, and `attributionsChart` to generate horizontal bar charts.

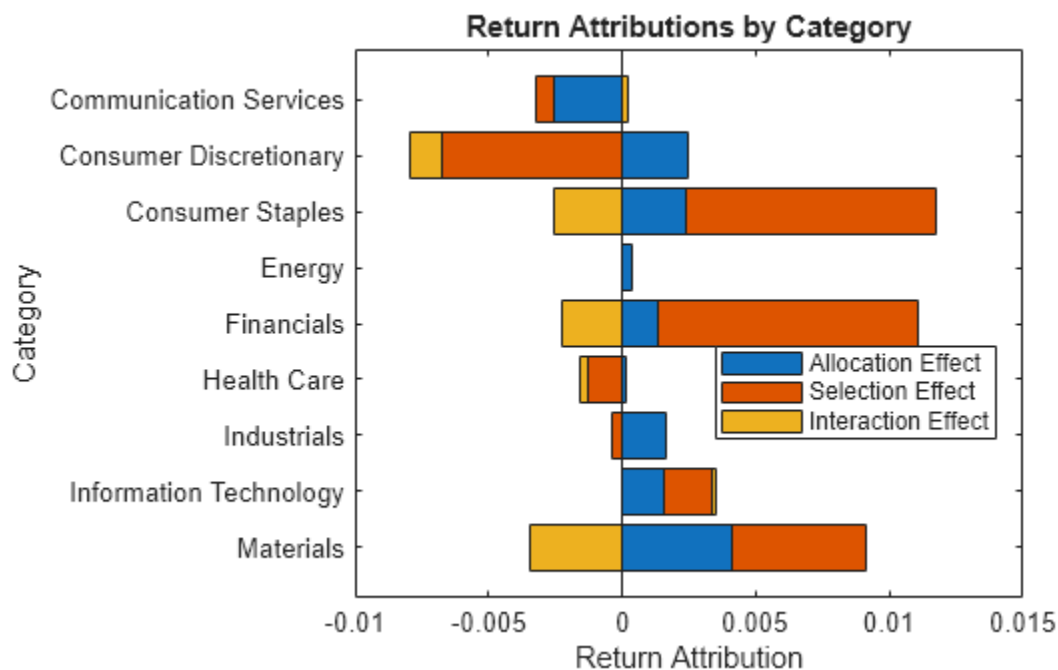
```
categoryReturnsChart(BrinsonPAobj)
```



```
categoryWeightsChart(BrinsonPAobj)
```

```
attributionsChart(BrinsonPAobj, Style="stacked")
```



See Also

`brinsonAttribution` | `categoryAttribution` | `categoryReturns` | `categoryWeights` | `totalAttribution` | `summary`

Related Examples

- “Backtest with Brinson Attribution to Evaluate Portfolio Performance” on page 4-315

Diversify Portfolios Using Custom Objective

This example shows three techniques of asset diversification in a portfolio using the `estimateCustomObjectivePortfolio` function with a `Portfolio` object. The purpose of asset diversification is to balance the exposure of the portfolio to any given asset to reduce volatility over a period of time. Given the sensitivity of the minimum variance portfolio to the estimation of the covariance matrix, some practitioners have added diversification techniques to the portfolio selection with the hope of minimizing risk measures other than the variance measures such as turnover, maximum drawdown, and so on.

This example applies these common diversification techniques:

- Equally weighted (EW) portfolio on page 4-333
- Equal risk contribution (ERC) on page 4-336
- Most diversified portfolio (MDP) on page 4-334

Additionally, this example demonstrates penalty methods that you can use to achieve different degrees of diversification. In those methods, you add a penalty term to the `estimateCustomObjectivePortfolio` function to balance the level of variance reduction and the diversification of the portfolio.

Retrieve Market Data and Define Mean-Variance Portfolio

Begin by loading and computing the expected returns and their covariance matrix.

```
% Load data
load('port5.mat');
% Store the returns and covariance.
mu = mean_return;
Sigma = Correlation .* (stdDev_return * stdDev_return');
```

Define a mean-variance portfolio using a `Portfolio` object with default constraints to create a fully invested, long-only portfolio.

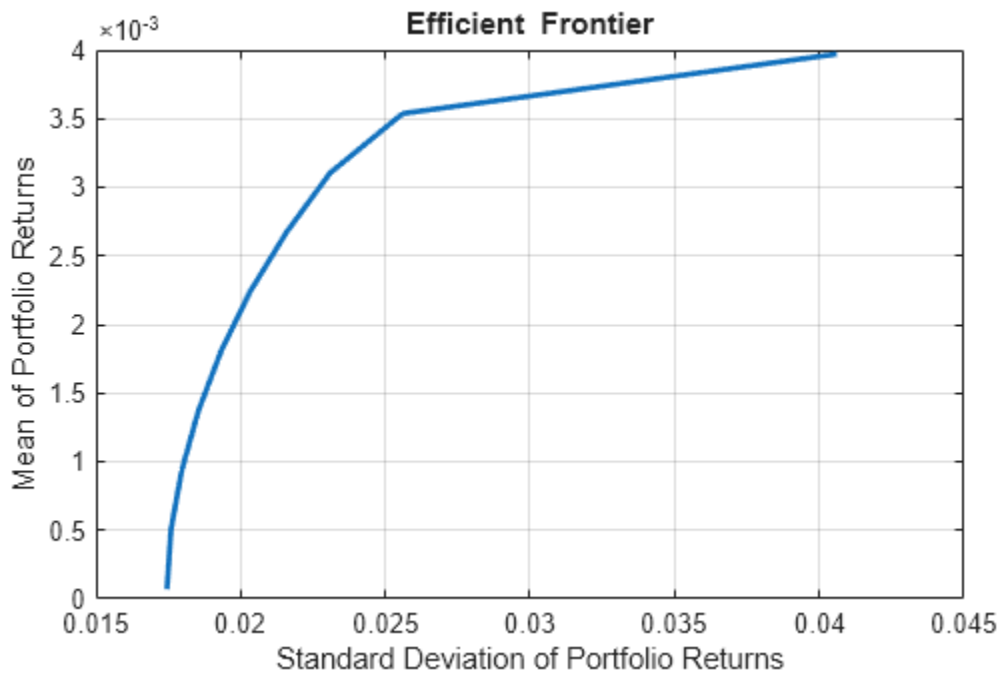
```
% Create a mean-variance Portfolio object with default constraints.
p = Portfolio('AssetMean',mu,'AssetCovar',Sigma);
p = setDefaultConstraints(p);
```

One of the many features of the `Portfolio` object is that it can obtain the efficient frontier of the portfolio problem. The efficient frontier is computed by solving a series of optimization problems for which the return level of the portfolio, μ_0 , is modified to obtain different points on the efficient frontier. These problems are defined as

$$\begin{aligned} \min \quad & x^T \Sigma x \\ \text{st.} \quad & \sum_{i=1}^n x_i = 1 \\ & x^T \mu \geq \mu_0 \\ & x \geq 0 \end{aligned}$$

The advantage of using the `Portfolio` object to compute the efficient frontier is that you can obtain without having to manually formulate and solve the multiple optimization problems shown above.

```
plotFrontier(p);
```



The `Portfolio` object can also compute the weights associated with the minimum variance portfolio, which is defined by the following problem.

$$\begin{aligned} \min \quad & x^T \Sigma x \\ \text{st.} \quad & \sum_{i=1}^n x_i = 1 \\ & x \geq 0 \end{aligned}$$

The minimum variance weights is the benchmark against which all the weights of the diversification strategies are compared.

```
wMinVar = estimateFrontierLimits(p, 'min');
```

To learn more about the problems that you can solve with the `Portfolio` object, see “When to Use Portfolio Objects Over Optimization Toolbox” on page 4-132.

Specify Diversification Techniques

This section presents the three diversification methods. Each of the three diversification methods is associated with a diversification measure and that diversification measure defines a penalty term to achieve different diversification levels. The diversification, obtained by adding the penalty term to the objective function, ranges from the behavior achieved by the minimum variance portfolio to the behavior of the EW, ECR, and MDP, respectively.

The default portfolio has only one equality constraint and a lower bound for the assets weights. The weights must be nonnegative and they must sum to 1. The feasible set is represented as X :

$$X = \left\{ x \mid x \geq 0, \sum_{i=1}^n x_i = 1 \right\}$$

Equally Weighted Portfolio

One of the diversification measures is the Herfindahl-Hirschman (HH) index defined as:

$$HH(x) = \sum_{i=1}^n x_i^2$$

This index is minimized when the portfolio is equally weighted. The portfolios obtained from using this index as a penalty have weights that satisfy the portfolio constraints and that are more evenly weighted.

The portfolio that minimizes the HH index is $\min_{x \in X} x^T x$. Since the constraints in X are the default constraints, the solution of this problem is the equally weighted (EW) portfolio. If X had extra constraints, the solution would be the portfolio that satisfies all the constraints and, at the same time, keeps the weights as equal as possible. Use the function handle `HHObjFun` for the Herfindahl-Hirschman (HH) index with the `estimateCustomObjectivePortfolio` function.

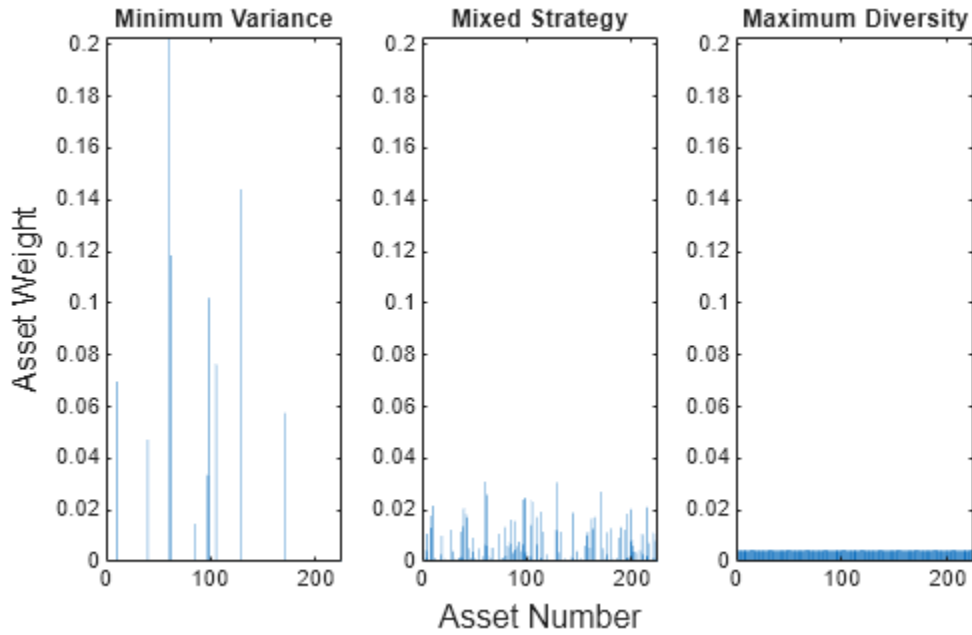
```
% Maximize the HH diversification (by minimizing the HH index).
HHObjFun = @(x) x'*x;
% Solution that minimizes the HH index
wHH = estimateCustomObjectivePortfolio(p,HHObjFun);
```

The portfolio that minimizes the variance with the HH penalty is $\min_{x \in X} x^T \Sigma x + \lambda_{HH} x^T x$. Use the function handle `HHMixObjFun` for the HH penalty with the `estimateCustomObjectivePortfolio` function.

```
% HH penalty parameter
lambdaHH = 1e-2;
% Variance + Herfindahl-Hirschman (HH) index
HHMixObjFun = @(x) x'*p.AssetCovar*x + lambdaHH*(x'*x);
% Solution that accounts for risk and HH diversification
wHHMix = estimateCustomObjectivePortfolio(p,HHMixObjFun);
```

Plot the weights distribution for the minimum variance portfolio, the equal weight portfolio, and the penalized strategy.

```
% Plot the different strategies.
plotAssetAllocationChanges(wMinVar,wHHMix,wHH)
```



This plot shows how the penalized strategy returns weights that are between the minimum variance portfolio and the EW portfolio. In fact, choosing $\lambda_{HH} = 0$ returns the minimum variance solution, and as $\lambda_{HH} \rightarrow \infty$, the solution approaches the EW portfolio.

Most Diversified Portfolio

The diversification index associated to the most diversified portfolio (MDP) is defined as

$$\text{MDP}(x) = - \sum_{i=1}^n \sigma_i x_i$$

where σ_i represents the standard deviation of asset i .

The MDP is the portfolio that maximizes the diversification ratio:

$$\varphi(x) = \frac{x^T \sigma}{\sqrt{x^T \Sigma x}}$$

The diversification ratio $\varphi(x)$ is equal to 1 if the portfolio is fully invested in one asset or if all assets are perfectly correlated. For all other cases, $\varphi(x) > 1$. If $\varphi(x) \approx 1$, there is no diversification, so the goal is to find the portfolio that maximizes $\varphi(x)$:

$$\max_{x \in X} \frac{\sigma^T x}{\sqrt{x^T \Sigma x}}$$

Unlike the HH index, the MDP goal is not to obtain a portfolio whose weights are evenly distributed among all assets, but to obtain a portfolio whose selected (nonzero) assets have the same correlation to the portfolio as a whole. Use the function handle `MDPObjFun` for the most diversified portfolio (MDP) with the `estimateCustomObjectivePortfolio` function.

```

% Maximize the diversification ratio.
sigma = sqrt(diag(p.AssetCovar));
MDPObjFun = @(x) (sigma'*x)/sqrt(x'*p.AssetCovar*x);
% Solution of MDP
wMDP = estimateCustomObjectivePortfolio(p,MDPObjFun, ...
    ObjectiveSense="maximize");

```

The following code shows that there exists a value $\hat{\lambda}_{\text{MDP}} > 0$ such that the MDP problem and the problem with its penalized version are equivalent. The portfolio that minimizes the variance with the MDP penalty is $\min_{x \in X} x^T \Sigma x - \lambda_{\text{MDP}} \sigma^T x$.

Define an MDP penalty parameter and solve for MDP using the function handle `MDPMixObjFun` for the MDP penalty with the `estimateCustomObjectivePortfolio` function.

```

% MDP penalty parameter
lambdaMDP = 1e-2;
% Variance + Most Diversified Portfolio (MDP)
MDPMixObjFun = @(x) x'*p.AssetCovar*x - lambdaMDP*(sigma'*x);
% Solution that accounts for risk and MDP diversification
wMDPMix = estimateCustomObjectivePortfolio(p,MDPMixObjFun);

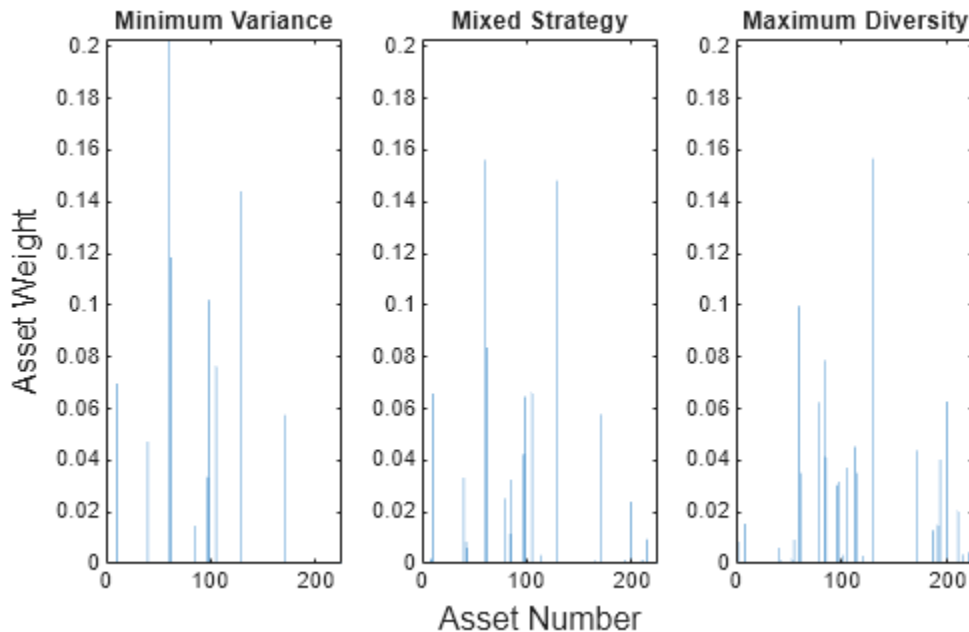
```

Plot the weights distribution for the minimum variance portfolio, the MDP, and the penalized strategy.

```

% Plot the different strategies.
plotAssetAllocationChanges(wMinVar,wMDPMix,wMDP)

```



In this plot, the penalized strategy weights are between the minimum variance portfolio and the MDP. This result is the same as with the HH penalty, where choosing $\lambda_{\text{MDP}} = 0$ returns the minimum variance solution and values of $\lambda_{\text{MDP}} \in [0, \hat{\lambda}_{\text{MDP}}]$ return asset weights that range from the minimum variance behavior to the MDP behavior.

Equal Risk Contribution Portfolio

The diversification index associated with the equal risk contribution (ERC) portfolio is defined as

$$\text{ERC}(x) = - \sum_{i=1}^n \ln(x_i)$$

This index is related to a convex reformulation shown by Maillard [1 on page 4-340] that computes the ERC portfolio. The authors show that you can obtain the ERC portfolio by solving the following optimization problem

$$\begin{aligned} \min_{y \geq 0} \quad & y^T \Sigma y \\ \text{st.} \quad & \sum_{i=1}^n \ln(y_i) \geq c \end{aligned}$$

and by defining x , the ERC portfolio with default constraints, as $x = \frac{y}{\sum_i y_i}$, where $c > 0$ can be any constant. You implement this procedure in the `riskBudgetingPortfolio` on page 4-340 function.

The purpose of the ERC portfolio is to select the assets weights in such a way that the risk contribution of each asset to the portfolio volatility is the same for all assets.

```
% ERC portfolio
wERC = riskBudgetingPortfolio(p.AssetCovar);
```

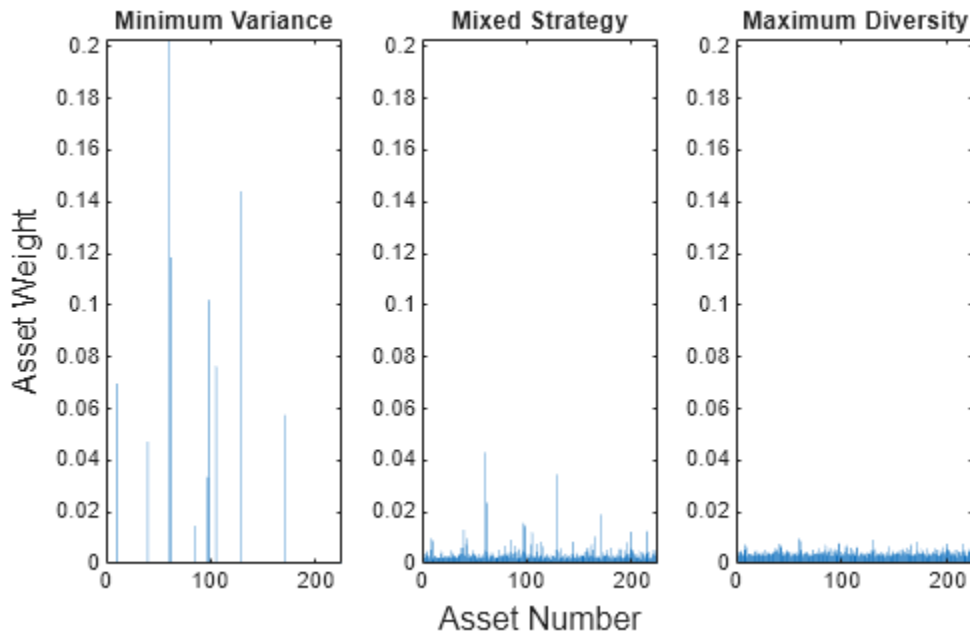
The portfolio that minimizes the variance with the ERC penalty is $\min_{x \in X} x^T \Sigma x - \lambda_{\text{ERC}} \sum_{i=1}^n \ln(x_i)$.

Similar to the case for the MDP penalized formulation, there exists a $\hat{\lambda}_{\text{ERC}}$ such that the ERC problem and its penalized version are equivalent. Use the function handle `ERCMixObjFun` for the ERC penalty with the `estimateCustomObjectivePortfolio` function.

```
% ERC penalty parameter
lambdaERC = 3e-6; % lambdaERC is so small because the log of a number
                  % close to zero (the portfolio weights) is large
% Variance + Equal Risk Contribution (ERC)
ERCMixObjFun = @(x) x'*p.AssetCovar*x - lambdaERC*sum(log(x));
% Solution that accounts for risk and ERC diversification
wERCMix = estimateCustomObjectivePortfolio(p,ERCMixObjFun);
```

Plot the weights distribution for the minimum variance portfolio, the ERC, and the penalized strategy.

```
% Plot the different strategies.
plotAssetAllocationChanges(wMinVar,wERCMix,wERC)
```

Comparable to the two diversification measures above, here the penalized strategy weights are between the minimum variance portfolio and the ERC portfolio. Choosing $\lambda_{\text{ERC}} = 0$ returns the minimum variance solution and the values of $\lambda_{\text{ERC}} \in [0, \hat{\lambda}_{\text{ERC}}]$ return asset weights that range from the minimum variance behavior to the ERC portfolio behavior.

Compare Diversification Strategies

Compute the number of assets that are selected in each portfolio. Assume that an asset is selected if the weight associated to that asset is above a certain threshold.

```
% Build a weights table.
varNames = {'MinVariance', 'MixedHH', 'HH', 'MixedMDP', 'MDP', ...
            'MixedERC', 'ERC'};
weightsTable = table(wMinVar, wHHMix, wHH, wMDPMix, wMDP, ...
                    wERCMix, wERC, 'VariableNames', varNames);
% Number of assets with nonzero weights
cutOff = 1e-3; % Weights below cut-off point are considered zero.
[reweightedTable, TnonZero] = tableWithNonZeroWeights(weightsTable, ...
    cutOff, varNames);
display(TnonZero)
```

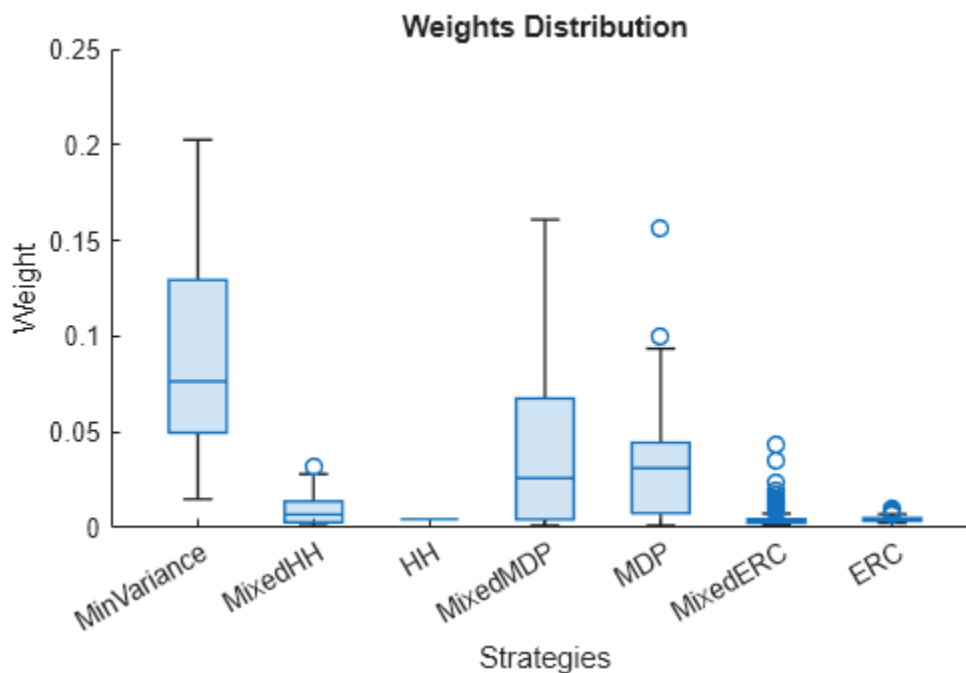
TnonZero=1×7 table

	MinVariance	MixedHH	HH	MixedMDP	MDP	MixedERC	ERC
Nonzero weights	11	104	225	23	28	225	225

As discussed above, the HH penalty goal is to obtain more evenly weighted portfolios. The portfolio that maximizes the HH diversity (and corresponds to the EW portfolio when only the default constraints are selected) has the largest number of assets selected and the weights of these assets are closer together. You can see this latter quality in the following boxchart. Also, the strategy that

adds the HH index as a penalty function to the objective has a larger number of assets than the minimum variance portfolio but less than the portfolio that maximizes HH diversity. The ERC portfolio also selects all the assets because all weights need to be nonzero to have some risk contribution.

```
% Boxchart of portfolio weights
figure;
matBoxPlot = reweightedTable.Variables;
matBoxPlot(matBoxPlot == 0) = NaN;
boxchart(matBoxPlot)
xticklabels(varNames)
title('Weights Distribution')
xlabel('Strategies')
ylabel('Weight')
```

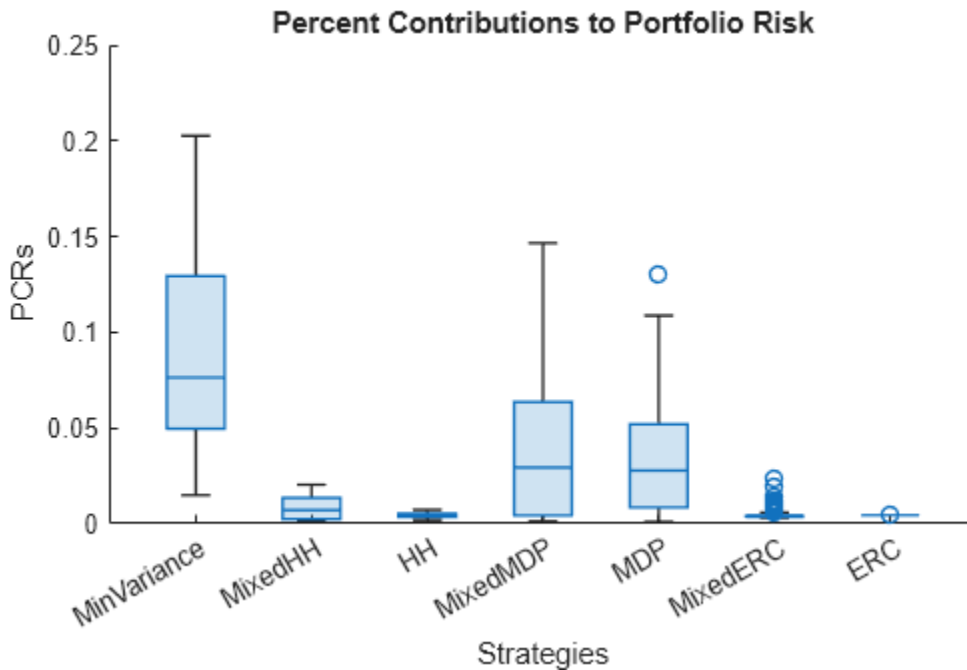


This boxchart shows the spread of the assets' positive weights for the different portfolios. As previously discussed, the weights of the portfolio that maximize the HH diversity are all the same. If the portfolio had other types of constraints, the weights would not all be the same, but they would have the lowest variance. The ERC portfolio weights also have a small variance. In fact, you can observe as the number of assets increases as the variance of the ERC portfolio weights becomes smaller.

The weights variability of the MDP is smaller than the variability of the minimum variance weights. However, it is not necessarily true that the MDP's weights will have less variability than the minimum variance weights because the goal of the MDP is not to obtain equally weighted assets, but to distribute the correlation of each asset with its portfolio evenly.

```
% Compute and plot the risk contribution of each individual
% asset to the portfolio.
riskContribution = portfolioRiskContribution(p.AssetCovar, ...
    weightsTable.Variables);
% Remove small contributions
riskContribution(riskContribution < 1e-3) = NaN;
```

```
% Compare percent contribution to portofolio risk
boxchart(riskContribution)
xticklabels(varNames)
title('Percent Contributions to Portfolio Risk')
xlabel('Strategies')
ylabel('PCRs')
```

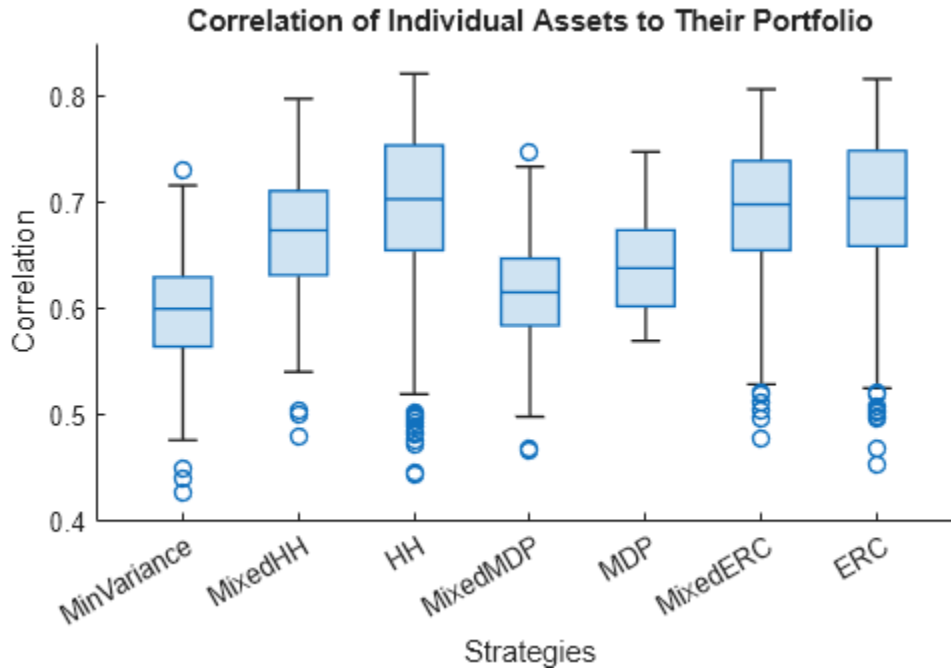


This boxchart shows the percent risk contribution of each asset to the total portfolio risk. The percent risk contribution is computed as

$$(\text{PRC})_i = \frac{x_i(\Sigma x)_i}{x^T \Sigma x}$$

As expected, all the ERC portfolio assets have the same risk contribution to the portfolio. As discussed after the weights distribution plot, if the problem had other types of constraints, the risk contribution of the ERC portfolio would not be the same for all assets, but they would have the lowest variance. Also, the behavior shown in this picture is similar to the behavior shown by the weights distribution.

```
% Compute and plot the correlation of each individual asset to its
% portfolio.
corrAsset2Port = correlationInfo(p.AssetCovar, ...
    weightsTable.Variables);
% Boxchart of assets to portfolio correlations
figure
boxchart(corrAsset2Port)
xticklabels(varNames)
title('Correlation of Individual Assets to Their Portfolio')
xlabel('Strategies')
ylabel('Correlation')
```



This boxchart shows the distribution of the correlations of each asset with its respective portfolio. The correlation of asset i to its portfolio is computed with the following formula:

$$\rho_{iP} = \frac{(\Sigma x)_i}{\sigma_i \sqrt{x^T \Sigma x}}$$

The MDP is the portfolio whose correlations are closer together and this is followed by the strategy that uses the MDP penalty term. In fact, if the portfolio problem allowed negative weights, then all the assets of the MDP would have the same correlation to its portfolio. Also, both the HH (EW) and ERC portfolios have almost the same correlation variability.

References

- 1 Maillard, S., Roncalli, T., & Teïletche, J. "The Properties of Equally Weighted Risk Contribution Portfolios." *The Journal of Portfolio Management*, 36(4), 2010, pp. 60–70.
- 2 Richard, J. C., & Roncalli, T. "Smart Beta: Managing Diversification of Minimum Variance Portfolios." *Risk-Based and Factor Investing*. Elsevier, 2015, pp. 31–63.
- 3 Tütüncü, R., Peña, J., Cornuéjols, G. *Optimization Methods in Finance*. United Kingdom: Cambridge University Press, 2018.

Local Functions

```
function [] = plotAssetAllocationChanges(wMinVar,wMix,wMaxDiv)
% Plots the weights allocation from the strategies

figure
t = tiledlayout(1,3);
nexttile
bar(wMinVar')
```

```

axis([0 225 0 0.203])
title('Minimum Variance')
nexttile
bar(wMix')
axis([0 225 0 0.203])
title('Mixed Strategy')
nexttile
bar(wMaxDiv')
axis([0 225 0 0.203])
title('Maximum Diversity')
ylabel(t,'Asset Weight')
xlabel(t,'Asset Number')

end

function [weightsTable,TnonZero] = ...
    tableWithNonZeroWeights(weightsTable,cutOff,varNames)
% Creates a table with the number of nonzero weights for each strategy

% Select only meaningful weights
funSelect = @(x) (x >= cutOff).*x./sum(x(x >= cutOff));
weightsTable = varfun(funSelect,weightsTable);

% Number of assets with positive weights
funSum = @(x) sum(x > 0);
TnonZero = varfun(funSum,weightsTable);
TnonZero.Properties.VariableNames = varNames;
TnonZero.Properties.RowNames = {'Nonzero weights'};

end

function [corrAsset2Port] = correlationInfo(Sigma,portWeights)
% Returns a matrix with the correlation of each individual asset to its
% portfolio.

nX = size(portWeights,1); % Number of assets
nP = size(portWeights,2); % Number of portfolios

auxM = eye(nX);
corrAsset2Port = zeros(nX,nP);
for j = 1:nP
    % Portfolio's standard deviation
    sigmaPortfolio = sqrt(portWeights(:,j)'*Sigma*portWeights(:,j));
    for i = 1:nX
        % Assets's standard deviation
        sigmaAsset = sqrt(Sigma(i,i));
        % Asset to portfolio correlation
        corrAsset2Port(i,j) = (auxM(:,i)'*Sigma*portWeights(:,j))/ ...
            (sigmaAsset*sigmaPortfolio);
    end
end

end

function [riskContribution] = portfolioRiskContribution(Sigma,...
    portWeights)
% Returns a matrix with the risk contribution of each asset to
% the underlying portfolio.

```

```
nX = size(portWeights,1); % Number of assets
nP = size(portWeights,2); % Number of portfolios

riskContribution = zeros(nX,nP);
for i = 1:nP
    weights = portWeights(:,i);
    % Portfolio variance
    portVar = weights'*Sigma*weights;
    % Marginal contribution to portfolio risk (MCR)
    margRiskCont = weights.*(Sigma*weights)/sqrt(portVar);
    % Percent contribution to portfolio risk
    riskContribution(:,i) = margRiskCont/sqrt(portVar);
end
end
```

See Also

[estimatePortSharpeRatio](#) | [estimateFrontier](#) | [estimateFrontierByReturn](#) | [estimateFrontierByRisk](#)

Related Examples

- “Diversify ESG Portfolios” on page 4-271
- “Diversify Portfolios Using Optimization Toolbox”

More About

- “Portfolio Optimization Theory” on page 4-4

Solve Tracking Error Portfolio Problems

This example shows how to compute a portfolio that minimizes the tracking error subject to a benchmark portfolio. This example uses `estimateCustomObjectivePortfolio` to solve a problem for a minimum variance `Portfolio` with a tracking error penalty. The example also shows how to add the tracking error as a "soft constraint" to any portfolio optimization problem. The tracking error measures the standard deviation of the divergence between a portfolio's return and that of a benchmark.

Define the mean and covariance of the assets returns.

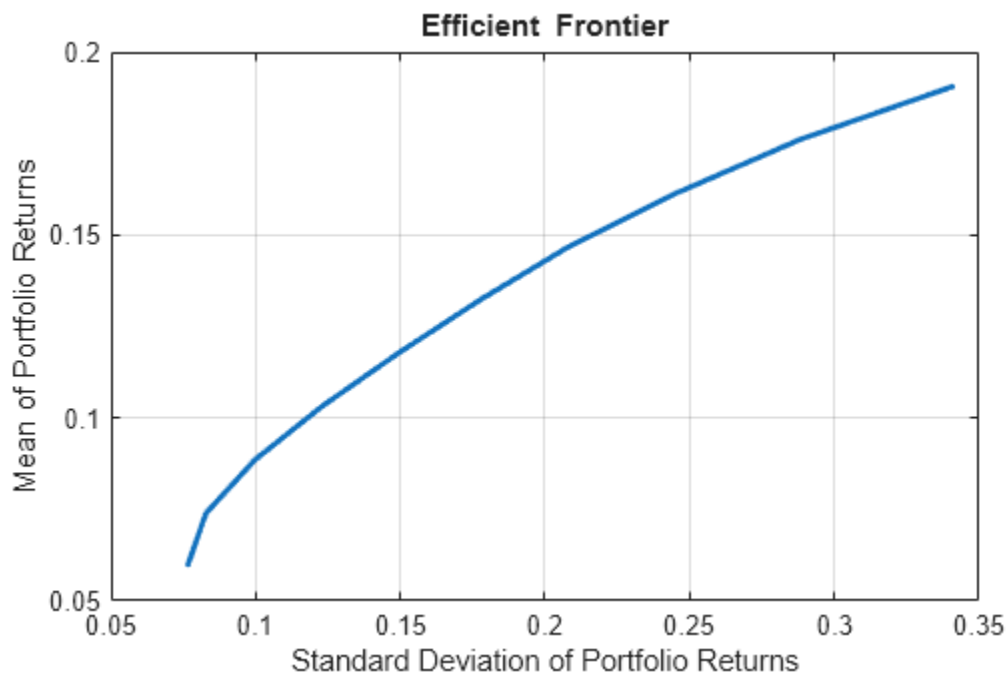
```
% Define the assets mean.
mu = [ 0.05; 0.1; 0.12; 0.18 ];
% Define the assets covariance.
Sigma = [ 0.0064 0.00408 0.00192 0;
          0.00408 0.0289 0.0204 0.0119;
          0.00192 0.0204 0.0576 0.0336;
          0 0.0119 0.0336 0.1225 ];
```

Simulate a timeseries of assets returns with mean `mu` and covariance `Sigma`. From this point onward, assume that the true mean and covariance are not known and they can only be estimated from the timetable data.

```
% Find all business days of the last two years.
tEnd = datetime('today');
bdates = busdays(tEnd-calyears(2),tEnd);
% Create a simulated timetable.
rng('default')
nScen = size(bdates,1);
assetsTT = array2timetable(mvnrnd(mu,Sigma,nScen),RowTimes=bdates);
```

Define a long-only, fully invested portfolio problem using `Portfolio`.

```
% Define a Portfolio object.
p = Portfolio;
p = estimateAssetMoments(p,assetsTT);
% Specify constraints for a long-only, fully invested portfolio.
p = setDefaultConstraints(p);
plotFrontier(p);
```



Assume that there is a market index that follows the maximum Sharpe ratio portfolio. Use `estimateMaxSharpeRatio` to create the benchmark portfolio.

```
% Create the benchmark portfolio.
bmkPort = estimateMaxSharpeRatio(p);
% Create the benchmark returns timetable.
marketTT = timetable(assetsTT.Time, assetsTT.Variables*bmkPort, ...
    VariableNames={'Benchmark'});
```

Minimize Tracking Error

You can use two methods to obtain the tracking error. The first method uses the variance of the excess return of the portfolio with respect to the benchmark. This method assumes that the benchmark prices or returns are given, but does not require knowledge of the weights of the assets associated to the benchmark. The second method requires you to decompose the benchmark into the different weights for each asset.

Tracking Error Using Benchmark Returns

To compute the tracking error, you first need to compute the excess return of each asset with respect to the benchmark.

```
% Compute the excess return.
excessReturn = assetsTT.Variables - marketTT.Variables;
```

Then, you can compute the excess returns covariance matrix $\Sigma_{\hat{R}}$.

```
excessRetSigma = cov(excessReturn);
```

Compute the tracking error as the variance of the excess return of the portfolio

$$x^T \Sigma_{\hat{R}} x.$$

For this example, you want to find the portfolio that minimizes the tracking error to the benchmark with, at most, half of the assets in the universe. Furthermore, if an asset is selected in the portfolio, at least 10% must be invested in that asset. This problem is as follows:

$$\begin{aligned} \min_x & x^T \hat{\Sigma}_R x \\ \text{s.t.} & \sum_i x_i = 1, \\ & \sum_i \#(x_i \neq 0) \leq 2, \\ & x = 0 \text{ or } x \geq 0.1 \end{aligned}$$

To solve this problem, first add the constraints to the `Portfolio` object using the `setBudget`, `setMinMaxNumAssets`, and `setBounds` functions.

```
% Specify that the budget sums to one constraint.
p = setBudget(p,1,1);
% Specify the maximum number of assets as 15.
p = setMinMaxNumAssets(p,[],2);
% Specify the conditional bounds.
p = setBounds(p,0.1,[],BoundType="conditional");
```

Define a function handle for the objective function $x^T \hat{\Sigma}_R x$.

```
% Define the objective function.
TEsquared1 = @(x) x'*excessRetSigma*x;
```

Use `estimateCustomObjectivePortfolio` to compute the solution to the problem.

```
% Solve the portfolio problem.
wMinTE1 = estimateCustomObjectivePortfolio(p,TEsquared1)
```

```
wMinTE1 = 4×1
```

```
0.8317
0
0
0.1683
```

Tracking Error Using Benchmark Weights

You can also define the tracking error as

$$(x - x_0)^T \Sigma (x - x_0).$$

In this formulation, Σ is the covariance matrix of the assets returns (not the excess returns) and this example assumes that the weight vector of the benchmark portfolio is known. Using the same constraints as the Tracking Error Using the Benchmark Returns on page 4-344 method, the problem is as follows:

$$\begin{aligned}
& \min_x (x - x_0)^T \Sigma (x - x_0) \\
& s.t. \sum_i x_i = 1, \\
& \sum_i \#(x_i \neq 0) \leq 2, \\
& x = 0 \text{ or } x \geq 0.01
\end{aligned}$$

Since the constraints are the same, you need only to define a new objective function.

```
% Define the objective function.
TESquared2 = @(x) (x-bmkPort)'*p.AssetCovar*(x-bmkPort);
```

Use `estimateCustomObjectivePortfolio` to compute the solution to the problem.

```
% Solve the portfolio problem.
wMinTE2 = estimateCustomObjectivePortfolio(p, TESquared2)
```

```
wMinTE2 = 4x1
```

```
0.8317
0
0
0.1683
```

The difference in the allocation for the different methods is negligible within a numerical accuracy. This means that the solution to both problems is the same.

```
norm(wMinTE1-wMinTE2, "inf")
```

```
ans =
5.5511e-17
```

Add Tracking Error as Soft Constraint

When you want to find a portfolio that focuses on two different objectives, you can define one of those objectives as a soft constraint and adding that objective as a penalty to the objective function. In this example, the tracking error becomes a soft constraint and the strength of the constraint is controlled with the penalty parameter $\lambda \geq 0$. The larger λ is, the stronger the tracking error constraint becomes, and conversely, the smaller λ is, the weaker the tracking error constraint becomes.

Assume that you are interested in obtaining the portfolio closest to the equally weighted portfolio (EWP) while controlling its tracking error. Minimizing the Herfindahl-Hirschman (HH) index, given by $x^T x$, returns the portfolio closest to the EWP that satisfies the necessary constraints. Assuming that the portfolio must satisfy the same constraints as in Minimizing Tracking Error on page 4-344, the problem is as follows

$$\begin{aligned}
& \min_x x^T x + \lambda (x - x_0)^T \Sigma (x - x_0) \\
& s.t. \sum_i x_i = 1, \\
& \sum_i \#(x_i \neq 0) \leq 2, \\
& x = 0 \text{ or } x \geq 0.01
\end{aligned}$$

Define the tracking error penalized objective function using the max Sharpe ratio portfolio as the benchmark.

`% Define the objective function.`

```
lambda = 100  ;
penalizedObjFun = @(x) x'*x + ...
    lambda*(x-bmkPort)'*p.AssetCovar*(x-bmkPort);
```

Use `estimateCustomObjectivePortfolio` to solve problem.

`% Solve the penalized problem.`

```
wEWPwithTE = estimateCustomObjectivePortfolio(p,penalizedObjFun)
```

```
wEWPwithTE = 4×1
```

```
0.7856
0
0
0.2144
```

Check the size of the tracking error. If the tracking error is too big, make λ larger.

`% Check size of tracking error.`

```
sqrt(TESquared2(wEWPwithTE))
```

```
ans =
0.0388
```

Compare the different portfolio allocations.

`% Compare the weights of the different portfolios.`

```
pwgt = table(wEWPwithTE,wMinTE1,bmkPort, ...
    VariableNames={'EWPwithTE','MinTE','Benchmark'})
```

```
pwgt=4×3 table
```

EWPwithTE	MinTE	Benchmark
0.78558	0.83165	0.64102
0	0	0.15599
0	0	0.067687
0.21442	0.16835	0.13531

When you compare the weights of the portfolio that minimizes the HH index with the tracking error penalty against the weights of the portfolio that minimizes the tracking error, you can see:

- As $\lambda \rightarrow 0$, the weights of the penalized problem become exactly 0.5 in two assets. This happens because a portfolio with 0.5 weights in two assets is the one that is closest to the EW portfolio and satisfies the constraints.
- As $\lambda \rightarrow \infty$, the weights of the penalized problem come closer to the weights of the minimum tracking error problem. This result happens because the penalized term in the objective overcomes the HH index.

Selecting different values of $\lambda \geq 0$ modifies the strength of the tracking error constraint.

See Also

`estimatePortSharpeRatio` | `estimateFrontier` | `estimateFrontierByReturn` |
`estimateFrontierByRisk` | `estimateCustomObjectivePortfolio`

Related Examples

- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Risk Parity or Budgeting with Constraints” on page 4-357
- “Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty” on page 4-351
- “Solve Problem for Minimum Tracking Error with Net Return Constraint” on page 4-349

More About

- “Solver Guidelines for Custom Objective Problems Using Portfolio Objects” on page 4-119

Solve Problem for Minimum Tracking Error with Net Return Constraint

This example shows how to use `estimateCustomObjectivePortfolio` to solve a portfolio problem for minimum tracking error with a net return constraint using a custom objective.

Create Portfolio Object

Create a Portfolio object.

```
% Create a Portfolio object
load('SixStocks.mat')
p = Portfolio(AssetMean=AssetMean,AssetCovar=AssetCovar);
```

Define Problem

The portfolio problem for a minimum tracking error problem with a net return constraint is defined as

$$\begin{aligned} \min_x & (x - x_0)^T \Sigma (x - x_0) \\ \text{s.t.} & \mu^T x - c_B^T \max(0, x - x_0) - c_S^T \max(0, x_0 - x) \geq \mu_0 \\ & \sum_i x_i = 1 \\ & x \geq 0 \end{aligned}$$

Define Problem Parameters

Define the problem parameters for the Portfolio object.

```
% Initial portfolio
initPort = 1/p.NumAssets*ones(p.NumAssets,1);
% Buy cost
buyCost = 0.001*ones(p.NumAssets,1);
% Sell cost
sellCost = 0.002*ones(p.NumAssets,1);
% Net return target
ret0 = 0.03;
```

Solve Portfolio Problem

Use `estimateCustomObjectivePortfolio` to solve this portfolio problem for a minimum tracking error with a net return constraint. When using the `estimateCustomObjectivePortfolio` function with a Portfolio object, you add return constraints by using the `estimateCustomObjectivePortfolio` name-value argument `TargetReturn` with a return target value.

```
% Long-only, fully invested portfolio
p = setDefaultConstraints(p);
% Set the buy and sell costs
p = setCosts(p,buyCost,sellCost,initPort);

% Set the objective
fun = @(x) (x-initPort)'*p.AssetCovar*(x-initPort);
```

```
% Solve the portfolio problem
wFinTbx = estimateCustomObjectivePortfolio(p,fun,TargetReturn=ret0)

wFinTbx = 6×1

    0.1633
    0.0648
    0.1950
    0.2618
    0.0625
    0.2525
```

See Also

[estimatePortSharpeRatio](#) | [estimateFrontier](#) | [estimateFrontierByReturn](#) | [estimateFrontierByRisk](#) | [estimateCustomObjectivePortfolio](#)

Related Examples

- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Solve Tracking Error Portfolio Problems” on page 4-343
- “Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty” on page 4-351
- “Risk Parity or Budgeting with Constraints” on page 4-357

More About

- “Solver Guidelines for Custom Objective Problems Using Portfolio Objects” on page 4-119

Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty

This example shows a robust formulation of portfolio optimization with uncertainty in the assets returns. It uses `estimateCustomObjectivePortfolio` to solve a robust Portfolio problem for a maximum return problem with an ellipsoidal uncertainty. Robust portfolio optimization, in contrast to the deterministic Markowitz mean-variance formulation, considers the uncertainty in the parameters of the problem: the assets' expected returns and their covariance matrix. In the traditional mean-variance model, the mean and covariance are assumed to take the point-values of the historical mean and covariance. However, in robust optimization, the mean and covariance are in a set that contains the most likely realizations, and the problem is optimized with respect to the worst possible outcome in that set.

Define Problem

If the vector of expected returns r has no uncertainty, the portfolio optimization problem that maximizes the returns is

$$R_{\text{port}} = \max_{x \in X} r^T x,$$

where

- $x \in \mathbb{R}^n$ is the vector of portfolio weights.
- X is the set of feasible allocations represented by a set of constraints.

Robust optimization assumes that the estimates of the expected returns are unreliable, but live in a set that contains the most likely realizations. The set of possible realizations of the expected returns is the *uncertainty set* and is given by

$$\{r \mid r \in S(r_0)\},$$

where

- $S(r_0)$ is a region around the vector r_0 .

A common robust formulation of the portfolio problem is the one that tries to maximize the expected return of the portfolio in the worst case scenario of the uncertainty set. This is represented with the following "max-min" problem

$$R_{\text{port}} = \max_{x \in X} \min_{r \in S(r_0)} r^T x.$$

In this example, assume that the uncertainty set follows the common ellipsoidal uncertainty given by

$$S(r_0) = \{r \mid (r - r_0)^T \Sigma_r^{-1} (r - r_0) \leq \kappa^2\},$$

where

- κ is the uncertainty aversion parameter that defines the width of the uncertainty.
- Σ_r is the covariance matrix of estimation errors in the expected returns r .

Goldfarb and Iyengar [1 on page 4-355] show that the robust maximization return problem with ellipsoidal uncertainty in the return is formulated as

$$\max_{x \in X} r_0^T x - \kappa \sqrt{x^T \Sigma_r x}.$$

Build Matrix of Estimated Errors in Expected Returns

Assume that the covariance matrix of estimation errors Σ_r is a diagonal matrix whose entries are proportional to the assets variance.

Load the data.

```
% Read the table of daily adjusted close prices for 2006 DJI stocks.
T = readtable('dowPortfolio.xlsx');
% Convert the table to a timetable.
pricesTT = table2timetable(T);
% Remove the DJI stock from the table.
pricesTT = pricesTT(:,2:end);
```

Build the covariance matrix of estimation errors in the expected returns.

```
% Compute the returns from the prices.
returnsTT = tick2ret(pricesTT);
% Compute the assets covariance matrix.
Sigma = cov(returnsTT.Variables);
% Compute the covariance matrix of estimation errors in the expected returns.
SigmaR = diag(diag(Sigma));
```

Solve Robust Problem

Define a Portfolio object.

```
% Define a traditional mean-variance portfolio.
p = Portfolio;
p = estimateAssetMoments(p, returnsTT, MissingData=true, GetAssetList=true);
```

Add constraints to the Portfolio object. The portfolio is a fully invested, long-short portfolio with bounds $[-0.2, 0.2]$.

```
% Specify a fully invested constraint.
p = setBudget(p, 1, 1);
% Specify the long-short bounds.
p = setBounds(p, -0.2, 0.2);
```

Define the objective function with $\kappa = 0.5$ as the ellipsoidal uncertainty parameter.

```
% Objective function handle
kappa = 0.5;
robustObjective = @(x) p.AssetMean'*x - kappa*sqrt(x'*SigmaR*x);
```

Solve the robust maximum return problem using estimateCustomObjectivePortfolio.

```
wRobustMaxRet = estimateCustomObjectivePortfolio(p, robustObjective, ObjectiveSense="maximize");
```

Compare Mean-Variance and Robust Portfolio Strategies

Compare the performance of the traditional maximum return portfolio against its robust counterpart using backtesting.

Define the warmup period to compute the initial portfolios for the different strategies using the data in that period.

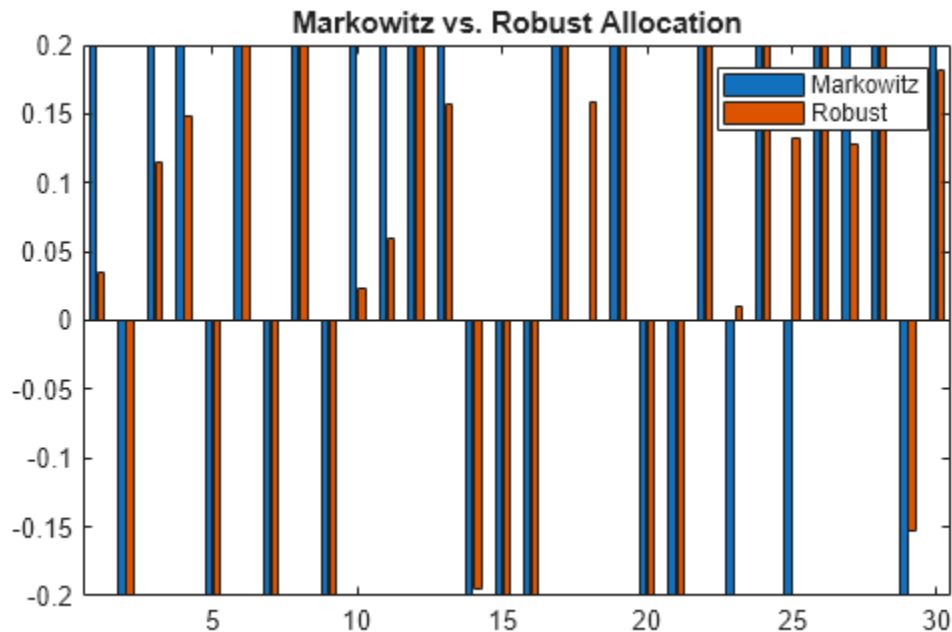
```
% Set backtesting warmup period to two 21-day months.
warmupPeriod = 21*2;
% Set a warmup partition of the timetable.
warmupTT = pricesTT(1:warmupPeriod,:);
```

Compute the initial portfolios using the rebalancing functions (markowitzFcn and robustFcn) that are defined in Local Functions on page 4-356.

```
% Define the initial Markowitz portfolio.
initialMarkowitz = markowitzFcn(zeros(p.NumAssets,1),warmupTT,p);
% Define the initial robust portfolio.
initialRP = robustFcn(zeros(p.NumAssets,1),warmupTT,p,kappa);
```

Plot both types of portfolios to compare the initial behavior.

```
% Plot the initial portfolios.
bar([initialMarkowitz initialRP])
legend('Markowitz','Robust')
title('Markowitz vs. Robust Allocation')
```



Out of 30 assets in the Markowitz allocation, 29 are at the extremes of the feasible bounds. On the other hand, only 17 assets in the robust allocation are at their extremes. The number of assets at the extremes of the feasible bounds shows the sensitivity of the traditional maximum return allocation strategy to the parameters of the problem. The Markowitz strategy invests everything possible in the assets with the highest returns. This result means that the Markowitz strategy shorts the assets with the lowest returns to be able to allocate more to the assets with the largest returns. On the other hand, the robust strategy does not have the same extreme behavior.

Define Backtesting Parameters

Set the backtesting strategies to rebalance every month.

```
% Define the monthly rebalance frequency.
```

```
rebalFreq = 21;
```

To gather enough data, set the `backtestStrategy` lookback window for the backtesting to at least 2 months. To remove old data from the parameter estimation, set the lookback window to no more than 6 months.

```
% Define the lookback window.
```

```
lookback = [42 126];
```

Use a fixed transaction cost equal to 0.5% of the amount traded.

```
% Define a fixed transaction cost.
```

```
transactionCost = 0.005;
```

Define the allocation strategies using `backtestStrategy`.

```
% Define a traditional mean-variance strategy.
```

```
stratMarkowitz = backtestStrategy('Markowitz', @(w,TT) markowitzFcn(w,TT,p), ...  
    RebalanceFrequency=rebalFreq, ...  
    LookbackWindow=lookback, ...  
    TransactionCosts=transactionCost, ...  
    InitialWeights=initialMarkowitz);
```

```
% Define a robust strategy.
```

```
stratRobust = backtestStrategy('Robust', @(w,TT) robustFcn(w,TT,p,kappa), ...  
    RebalanceFrequency=rebalFreq, ...  
    LookbackWindow=lookback, ...  
    TransactionCosts=transactionCost, ...  
    InitialWeights=initialRP);
```

Run Backtest

Create a `backtestEngine` object and then run the backtest using `runBacktest`.

```
% Define strategies for backtest engine
```

```
strategies = [stratMarkowitz stratRobust];
```

```
% Define a backtest engine
```

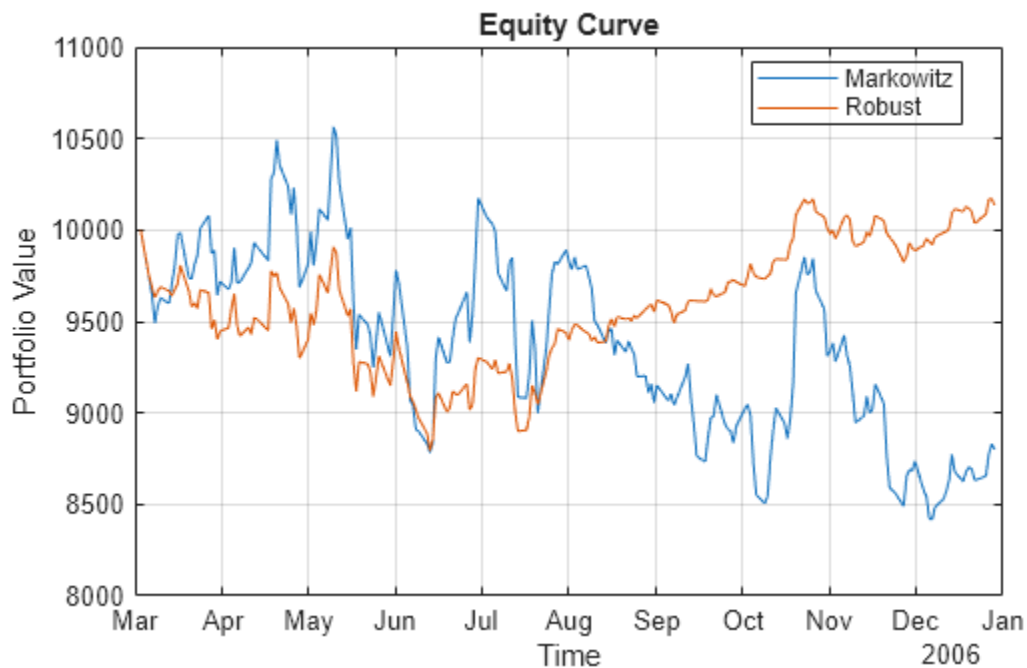
```
backtester = backtestEngine(strategies);
```

```
% Run the backtest
```

```
backtester = runBacktest(backtester,pricesTT,Start=warmupPeriod);
```

Use `equityCurve` to plot the equity curve.

```
equityCurve(backtester)
```



Use summary to generate a table of performance results for both strategies.

```
summary(backtester)
```

```
ans=9x2 table
```

	Markowitz	Robust
TotalReturn	-0.12004	0.013641
SharpeRatio	-0.02926	0.011774
Volatility	0.016383	0.0088013
AverageTurnover	0.047756	0.021642
MaxTurnover	1.8849	1.0392
AverageReturn	-0.00047822	0.00010338
MaxDrawdown	0.20332	0.12068
AverageBuyCost	2.276	1.036
AverageSellCost	2.276	1.036

As expected, the robust strategy has lower volatility than the traditional mean-variance strategy. This characteristic is observed in all the performance indicators that measure variability: volatility, turnover, and maximum drawdown. This result is the expected outcome of robust strategies. Although it is not always the case, in this particular example, the robust strategy also outperforms the traditional mean-variance allocation.

References

[1] Goldfarb, D. and G. Iyengar. "Robust Portfolio Selection Problems." *Mathematics of Operations Research*. 28(1), pp. 1-38, 2003.

Local Functions

```
function new_weights = markowitzFcn(~,pricesTT,portObj)
% Traditional mean-variance portfolio maximum return allocation

% Estimate the portfolio's mean and variance.
p = estimateAssetMoments(portObj,pricesTT,DataFormat='Prices', ...
    MissingData=true);

% Compute the max return portfolio.
new_weights = estimateFrontierLimits(p,'max');

end

function new_weights = robustFcn(~,pricesTT,portObj, ...
    kappa)
% Robust portfolio maximum return allocation

% Estimate the portfolio's mean and variance.
p = estimateAssetMoments(portObj,pricesTT,DataFormat='Prices', ...
    MissingData=true);

% Compute covariance matrix of estimation errors in the expected returns.
SigmaR = diag(diag(p.AssetCovar));

% Define the objective function handle.
robustObjective = @(x) p.AssetMean'*x - kappa*sqrt(x'*SigmaR*x);

% Compute the maximum return portfolio.
new_weights = estimateCustomObjectivePortfolio(p,robustObjective, ...
    ObjectiveSense="maximize");

end
```

See Also

[estimatePortSharpeRatio](#) | [estimateFrontier](#) | [estimateFrontierByReturn](#) | [estimateFrontierByRisk](#) | [estimateCustomObjectivePortfolio](#)

Related Examples

- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Solve Tracking Error Portfolio Problems” on page 4-343
- “Solve Problem for Minimum Tracking Error with Net Return Constraint” on page 4-349
- “Risk Parity or Budgeting with Constraints” on page 4-357

More About

- “Solver Guidelines for Custom Objective Problems Using Portfolio Objects” on page 4-119

Risk Parity or Budgeting with Constraints

This example shows how to solve risk parity or budgeting problems with constraints using `estimateCustomObjectivePortfolio`.

Risk parity is a portfolio allocation strategy that focuses on the allocation of risk to define the weights of a portfolio. You construct the risk parity portfolio, or equal risk contribution portfolio, by ensuring that all assets in a portfolio have the same risk contribution to the overall portfolio risk. The generalization of this problem is the risk budgeting portfolio in which you set the risk contribution of each asset to the overall portfolio risk. In other words, instead of all assets contributing equally to the risk of the portfolio, the assets' risk contribution must match a target risk budget.

In “Risk Budgeting Portfolio” on page 4-286, the example describes some advantages of risk parity or budgeting portfolios. It shows how to use `riskBudgetingPortfolio` to solve long-only, fully invested risk parity or budgeting problems. When the only constraints of the risk budgeting problem enforce nonnegative weights that sum to 1, the risk budgeting problem is a feasibility problem. In other words, the solution to the problem is the one that exactly matches the target risk budget and it is unique. Therefore, there is no need for an objective function. The `riskBudgetingPortfolio` function is optimized to solve this specific type of problems.

When you add more constraints to the risk parity problem, you must find the weight allocation that satisfies the extra constraints while trying to match the target risk budget as much as possible. The next section explains how.

Define Problem

Start by defining the problem data.

```
% Assets variance
vol = [0.1;0.15;0.2;0.3];
% Assets correlation
rho = [1.00 0.50 0.50 0.50;
       0.50 1.00 0.50 0.50;
       0.50 0.50 1.00 0.75;
       0.50 0.50 0.75 1.00];
% Assets covariance matrix
Sigma = corr2cov(vol,rho);
% Risk budget
budget = [0.3;0.3;0.195;0.205];
```

Obtain the long-only, fully invested risk budgeting portfolio to use as benchmark.

```
% Long-only fully invested risk budgeting portfolio
w_simple = riskBudgetingPortfolio(Sigma,budget);
% Compute the risk contribution of the weight allocation
RC_simple = portfolioRiskContribution(w_simple,Sigma);
% Table with weights and risk contributions
T_simple = table(w_simple,RC_simple,budget, ...
    'VariableNames',{'RB Portfolio','RB Contribution','Budget'})
```

```
T_simple=4×3 table
    RB Portfolio    RB Contribution    Budget
    _____    _____    _____
    0.45053         0.3         0.3
```

0.30035	0.3	0.3
0.14668	0.195	0.195
0.10244	0.205	0.205

As desired, the risk contribution of the risk budgeting portfolio matches the budget.

Add Constraints to Risk Parity or Budgeting Portfolio

When you add constraints, other than the long-only, fully invested constraints, to the risk parity formulation, you must find the portfolio allocation that satisfies all the constraints and minimizes the deviation from the target risk budget.

The percent risk contribution of asset i is defined as

$$RC_i(w) = \frac{w_i(\Sigma w)_i}{w^T \Sigma w},$$

and the goal is to find the portfolio that minimizes the deviation of $RC_i(w)$ to b_i , the target risk budget of asset i . For risk parity problems, $b_i = \frac{1}{n}$, where n is the number of assets.

A common way to measure deviation is to use a norm. In this example, use the 2-norm. The risk parity or budgeting problem with constraints results in

$$\begin{aligned} \min_w \quad & \sum_i \left(\frac{w_i(\Sigma w)_i}{w^T \Sigma w} - b_i \right)^2 \\ \text{s.t.} \quad & w \in \Omega \end{aligned}$$

where Ω represents the feasible set that is defined by the desired constraints. The objective function is the sum of squares of the deviation of the risk contributions from the target. Notice that the objective function is nonconvex.

Next, introduce the constraint $x_i \leq 0.3$, which means that no more than 30% of the capital can be invested in each asset.

```
% Define the Portfolio object
p = Portfolio(AssetCovar=Sigma);
% Set bound constraints
p = setBounds(p,0,0.3);
% Set budget constraint
p = setBudget(p,1,1);
```

Create a function handle for the nonconvex objective

$$\sum_i \left(\frac{w_i(\Sigma w)_i}{w^T \Sigma w} - b_i \right)^2.$$

```
% Define objective function
objFun = @(w) sum(((w.*(Sigma*w))/(w'*Sigma*w) - budget).^2);
```

Solve the problem using `estimateCustomObjectivePortfolio` and `riskBudgetingPortfolio`.

```
% Risk budgeting portfolio with extra constraints
w_extra = estimateCustomObjectivePortfolio(p,objFun);
% Risk contribution
```

```
RC_extra = portfolioRiskContribution(w_extra,Sigma);
% Table with weights and risk contributions
T_extra = table(w_extra,RC_extra,budget, ...
    VariableNames={'RB Portfolio','RB Contribution','Budget'})
```

```
T_extra=4x3 table
    RB Portfolio    RB Contribution    Budget
    _____    _____    _____
         0.3         0.15505         0.3
         0.3         0.24998         0.3
    0.2465         0.30862         0.195
    0.1535         0.28635         0.205
```

Now that you have added additional constraints, the risk contribution of the resulting portfolio does not match the budget. Yet, the solution to the problem is minimizing the deviation of the risk contribution to the target risk budget. Because the risk budgeting portfolio without the extra constraints assigns weights larger than 30% to the first and second assets, the solution to the problem that minimizes the deviation assigns as much as possible to the first and second assets and distributes the rest to the other assets.

As explained in “Role of Convexity in Portfolio Problems” on page 4-157, because the objective function is nonconvex, this formulation cannot be solved by the `Portfolio` object solvers if you add cardinality constraints or conditional bounds to the problem.

Compare Risk Parity and Mean-Variance Portfolios

You can examine the difference in the allocation between the traditional minimum variance portfolio and the risk parity portfolio.

First compute the minimum variance portfolio. The traditional mean-variance framework requires the assets mean return to be defined before you compute any of the portfolios on the efficient frontier.

```
% Define assets mean
p.AssetMean = [0.1; 0.1; 0.15; 0.2];
% Minimum variance portfolio
wMinVar = estimateFrontierLimits(p,'min');
```

To compute the risk parity portfolio, make sure all assets contribute equally to the risk. Set $b_i = \frac{1}{n}$ and update the objective function.

```
% Update budget
budget = 1/p.NumAssets*ones(p.NumAssets,1);
% Update objective
objFun = @(w) sum(((w.*(Sigma*w))/(w'*Sigma*w) - budget).^2);
```

Solve the risk parity problem using `estimateCustomObjectivePortfolio`.

```
% Risk parity portfolio with extra constraints
wRP = estimateCustomObjectivePortfolio(p,objFun);
```

Compare the weight allocation concentrations.

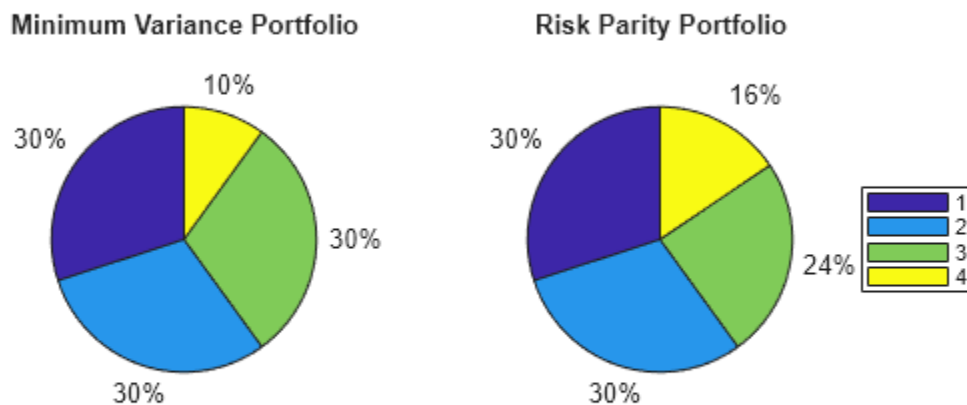
```
% Plot pie charts
tiledlayout(1,2);
% Minimum variance portfolio
```

```

nexttile
pie(wMinVar)
title('Minimum Variance Portfolio',Position=[0,1.5]);
% Risk parity portfolio
nexttile
pie(wRP)
title('Risk Parity Portfolio',Position=[0,1.5]);

% Add legend
lgd = legend({'1','2','3','4'});
lgd.Layout.Tile = 'east';

```



The portfolio allocation of the minimum variance portfolio sets all assets with the smallest risk to their maximum (30%). On the other hand, the risk parity allocation sets only the first two assets to 30% and the last two are more balanced. This result is a simple example of how the risk parity allocation helps to diversify a portfolio.

See Also

`estimatePortSharpeRatio` | `estimateFrontier` | `estimateFrontierByReturn` | `estimateFrontierByRisk` | `estimateCustomObjectivePortfolio`

Related Examples

- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Solve Tracking Error Portfolio Problems” on page 4-343
- “Solve Problem for Minimum Tracking Error with Net Return Constraint” on page 4-349

- “Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty” on page 4-351

More About

- “Solver Guidelines for Custom Objective Problems Using Portfolio Objects” on page 4-119

Single Period Goal-Based Wealth Management

This example shows a method for goal-based wealth management (GBWM). In GBWM, risk is not necessarily measured using the standard deviation, the value-at-risk, or any other common risk measure. Instead, risk is understood as the likelihood of not attaining an investor's goal. You choose a weight allocation that is on the traditional mean-variance efficient frontier and that also maximizes the probability of exceeding a wealth goal at the end of the investment horizon. In other words, you choose the portfolio on the efficient frontier that minimizes the risk of not attaining the investor's goal.

A common issue after computing the mean-variance efficient frontier is how to choose a single portfolio to invest in. This portfolio could be the maximum Sharpe ratio portfolio, which selects the weights that achieve the highest return given the amount of risk taken. However, choosing this portfolio does not take the investor's preferences into account. Another way strategy is to gauge the risk preference of the investor and select a portfolio that achieves a risk no larger than the investor's preferred risk. However, it is difficult to quantify the risk aversion of an investor. In this example, you use the probability of achieving a wealth goal to help choose a portfolio on the efficient frontier.

Load Data

Load the data and then annualize the asset returns and the variance-covariance matrix.

```
% Load data
load BlueChipStockMoments.mat
AssetMean = 12*AssetMean;
AssetCovar = 12*AssetCovar;
```

Plot Efficient Frontier

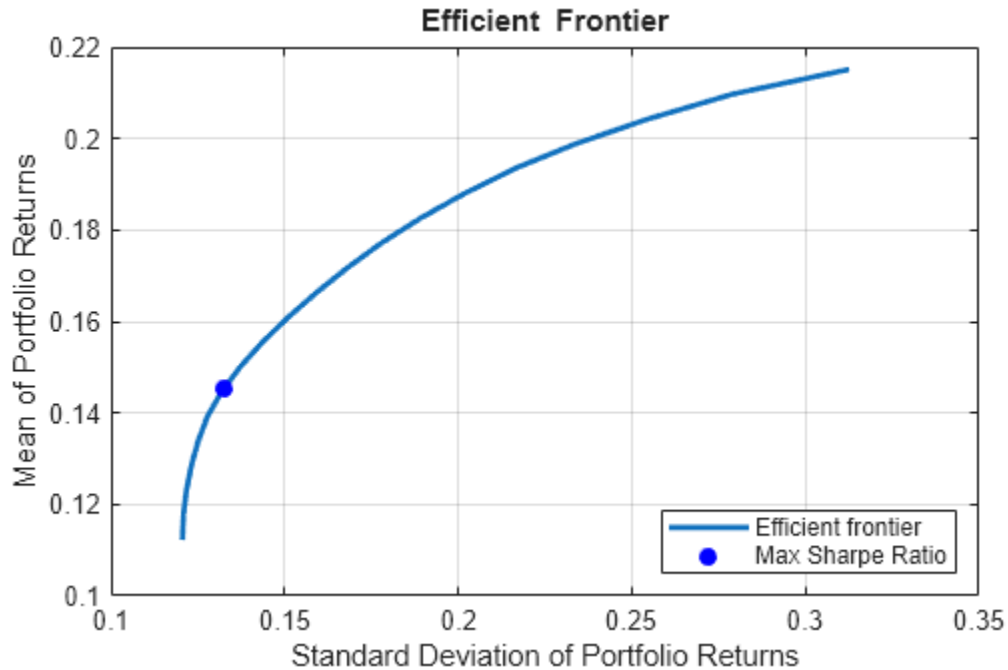
Use Portfolio to create a Portfolio object and estimateMaxSharpeRatio to compute the maximum Sharpe ratio. Then, use plotFrontier to plot the efficient frontier for long-only, fully-invested portfolios and mark the maximum Sharpe ratio portfolio.

```
% Create a Portfolio object.
p = Portfolio(AssetList=AssetList,AssetMean=AssetMean,...
    AssetCovar=AssetCovar);

% Add long-only, fully-invested constraints
p = setDefaultConstraints(p);

% Compute the maximum Sharpe ratio portfolio.
wSR = estimateMaxSharpeRatio(p);

% Plot the efficient frontier.
numPorts = 20;
figure;
[pret,prisk] = plotFrontier(p,20);
hold on
plot(estimatePortRisk(p,wSR),estimatePortReturn(p,wSR),'b.',...
    'MarkerSize',20);
legend('Efficient frontier','Max Sharpe Ratio',...
    'Location','southeast')
hold off
```



Choose Single Portfolio Using GBWM

GBWM provides a way to choose a single portfolio on the efficient frontier that allows you to incorporate the investor's preference in a straightforward way. In this case, the investor's goal is to reach a target wealth G from their initial wealth W_0 at the end of the investment period T . Therefore, you can formulate the problem as choosing the portfolio that maximizes the probability of having a wealth at time T that surpasses the goal G , while making sure that the chosen weights are on the efficient frontier.

$$\begin{aligned}
 & \max_{w, \mu, \sigma} P(W_T \geq G) \\
 & s.t. \quad \mu_0^T w = \mu \\
 & \quad w^T \Sigma_0 w = \sigma^2 \\
 & \quad \sum_{i=1}^n w_i = 1 \\
 & \quad 0 \leq w \leq 1
 \end{aligned}$$

% Define investor's preferences

$W_0 = 100;$

$G = 200;$

$T = 10;$

For this example, assume that the wealth evolves following a Brownian motion [1 on page 4-366]

$$W_T = W_0 \exp \left\{ \left(\mu - \frac{\sigma^2}{2} \right) T + \sigma \sqrt{T} Z \right\},$$

where Z is a standard normal random variable.

The approach in this example also assumes that the returns follow a stationary distribution. In other words, the approach assumes that the returns have the same expectation and covariance throughout the investment period. The term “single-period” refers to the fact that the optimal allocation is computed *only once* at the beginning of the investment horizon assuming that the allocations will remain constant. This is a “set-it-and-forget-it” strategy. Although you can recompute the allocation weights at each reinvestment period by updating the time horizon and initial wealth, a single-period problem does not consider possible allocation changes in the middle of the investment horizon. In contrast, the optimal solution for a “multi-period” approach takes into account possible changes to the weights at intermediate investment periods. Thus, instead of just selecting the initial portfolio weights, multi-period problems compute the optimal strategy that should be followed given the changes in wealth at each of the reinvestment periods. The “Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods” on page 4-367 example shows a multi-period version of the problem in this example.

Define Objective Function and Solve Problem

Find the portfolio that maximizes the probability of achieving the investor's goal given the prior assumptions. Maximizing $P(W_T \geq G)$ is equivalent to minimizing $P(W_T \leq G)$, so the objective function is rewritten as

$$\max_{w, \mu, \sigma} P(W_T \geq G) = \min_{w, \mu, \sigma} P(W_T \leq G) = \min_{w, \mu, \sigma} \Phi \left(\frac{1}{\sigma \sqrt{T}} \left[\ln \left(\frac{G}{W_0} \right) - \left(\mu - \frac{\sigma^2}{2} \right) T \right] \right).$$

Since the relationship of μ and σ to the weights w is represented by equality constraints in the problem, you can substitute these values in the objective function and remove the equality constraints that show the relationship between these three variables.

$$\max_{w, \mu, \sigma} P(W_T \geq G) = \min_w \Phi \left(\frac{1}{\sqrt{T(w^T \Sigma_0 w)}} \left[\ln \left(\frac{G}{W_0} \right) - \left(\mu_0^T w - \frac{w^T \Sigma_0 w}{2} \right) T \right] \right)$$

Define the objective function handle using the definition above and then use `estimateCustomObjectivePortfolio` to solve the problem.

```
% Define objective function
probability = @(w) normcdf(1/sqrt(T*(w'*p.AssetCovar*w)) * ...
    (log(G/W0) - (p.AssetMean'*w - (w'*p.AssetCovar*w)/2)*T));

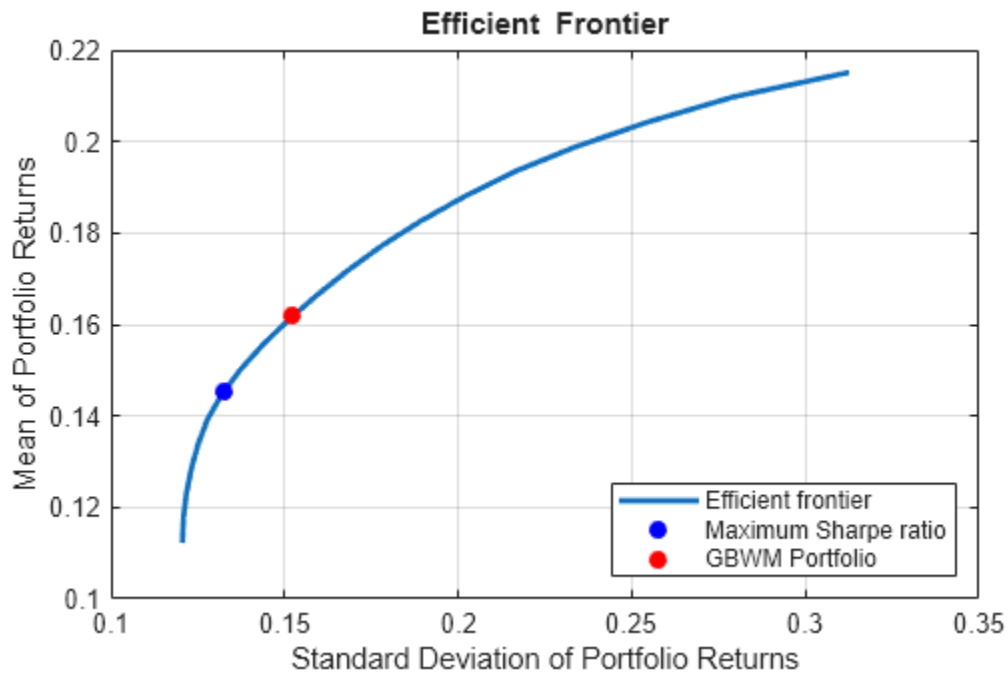
% Solve problem
wGBWM = estimateCustomObjectivePortfolio(p,probability);
```

Plot Efficient Frontier

Use `plotFrontier` to plot the efficient frontier, the maximum Sharpe ratio portfolio, and the portfolio that maximizes the probability of achieving the goal (called the GBWM portfolio).

```
% Plot the efficient frontier.
figure;
plotFrontier(p,pret,prisk);
hold on
plot(estimatePortRisk(p,wSR),estimatePortReturn(p,wSR),'b.', ...
    'MarkerSize',20);
plot(estimatePortRisk(p,wGBWM),estimatePortReturn(p,wGBWM),'r.', ...
    'MarkerSize',20);
legend('Efficient frontier','Maximum Sharpe ratio', ...
```

```
'GBWM Portfolio','Location','southeast')
hold off
```



The GBWM portfolio is riskier than the maximum Sharpe ratio portfolio. This extra risk is necessary to maximize the probability of attaining the investor's wealth goal G by time T .

Compare GBWM Portfolio to Maximum Sharpe Ratio Portfolio

Compare the probabilities of achieving the wealth goal G by the end of the investment period T for the maximum Sharpe ratio portfolio against the GBWM portfolio.

```
Tprobabilities = table(1-probability(wSR),1-probability(wGBWM),...
    RowNames={'P(W_T >= G)'},VariableNames={'SR','GBWM'})
```

```
Tprobabilities=1x2 table
                SR          GBWM
P(W_T >= G)    0.94592    0.95347
```

The probability of successfully meeting the investor's wealth goal does not change much between the maximum Sharpe ratio portfolio and the GBWM portfolio. Using this information, an investor can understand the trade-off between achieving their wealth goal G by time T compared to choosing a less risky portfolio. For example, an investor might choose a minimum variance portfolio that has a much lower probability of achieving their wealth goal G by time T . Find the minimum variance (MV) portfolio, and compare the probability of achieving the wealth goal using this portfolio with the other two approaches.

```
wMV = estimateFrontierLimits(p,'min');
Tprobabilities.MV = 1-probability(wMV)

Tprobabilities=1x3 table
                SR          GBWM          MV
```

P(W_T >= G)	0.94592	0.95347	0.8252

References

[1] Das, Sanriv R., Daniel Ostrov, Anand Radhakrishnan, and Deep Srivastav. "A New Approach to Goals-Based Wealth Management." *Journal of Investment Management*. 16, no. 3 (2018): 1-27.

See Also

Portfolio | estimatePortSharpeRatio | estimateFrontier | estimateFrontierByReturn | estimateFrontierByRisk | estimateCustomObjectivePortfolio

Related Examples

- "Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods" on page 4-367
- "Multiperiod Goal-Based Wealth Management Using Reinforcement Learning" on page 4-379
- "Diversify Portfolios Using Custom Objective" on page 4-331
- "Portfolio Optimization Against a Benchmark" on page 4-202

More About

- "Solver Guidelines for Custom Objective Problems Using Portfolio Objects" on page 4-119

Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods

This example shows a dynamic programming strategy to maximize the probability of obtaining an investor's wealth goal at the end of the investment horizon. This dynamic programming strategy is known in the literature as a goal-based wealth management (GBWM) strategy. In GBWM, risk is not necessarily measured using the standard deviation, the value-at-risk, or any other common risk metric. Instead, risk is understood as the likelihood of not attaining an investor's goal. This alternative concept of risk implies that, sometimes, in order to increase the probability of attaining an investor's goal, the optimal portfolio's traditional risk (that is, standard deviation) must increase if the portfolio is underfunded. In other words, for the investor's view of risk to *decrease*, the traditional view of risk must *increase* if the portfolio's wealth is too low.

The objective of the investment strategy is to determine a dynamic portfolio allocation such that the portfolios chosen at each rebalancing period are on the traditional mean-variance efficient frontier and these portfolios maximize the probability of achieving the wealth goal G at the time horizon T . This objective is represented as

$$\max_{\{A(0), A(1), \dots, A(T-1)\}} \mathbb{P}(W(T) \geq G),$$

where $W(T)$ is the terminal portfolio wealth and $A(t)$ are the possible actions and allocations at time $t = 0, 1, \dots, T-1$.

The dynamic portfolio problem consists of the following parts:

- Time periods — Portfolio rebalancing periods
- Actions — Portfolio weights rebalancing
- States — Wealth levels
- Value function — Probability of achieving the goal at the end of the investment period
- Policy or strategy — Optimal actions at each state and time period

This example follows the GBWM strategy of Das [1 on page 4-378]. For an example of GBWM for a single time period, see the “Single Period Goal-Based Wealth Management” on page 4-362 example.

Inputs for Investment Wealth Goals

Specify the initial wealth.

$W_0 = 100;$

Specify the target wealth at the end of the investment horizon.

$G = 200;$

Define the time horizon.

$T = 10;$

Load the asset information.

load BlueChipStockMoments.mat

Rebalancing Period Actions for Efficient Portfolios

The action at each rebalancing period is to choose the weights of the investment portfolio for the next time period. In this example assume that the possible portfolio weights are associated with the portfolios on the efficient frontier. This example also assumes that the choice of possible portfolios is fixed throughout the full investment period. However, weights are not required to be on the efficient frontier. The only requirement is that the set of actions (portfolio choices) remains fixed throughout the investment period.

To simplify the computations, define a finite subset of the efficient portfolios for possible actions. Start by creating a `Portfolio` object with the asset information from `BlueChipStockMoments.mat`.

```
p = Portfolio(AssetList=AssetList,AssetMean=AssetMean,...  
             AssetCovar=AssetCovar);
```

Annualize the asset returns and the variance-covariance matrix.

```
p.AssetMean = 12*p.AssetMean;  
p.AssetCovar = 12*p.AssetCovar;
```

Use `setBounds` to add constraints to the portfolio problem. Here, assets do not represent more than 10% of the portfolio wealth and shorting is not allowed.

```
p = setBounds(p,0,0.1);
```

Use `setBudget` to specify that the portfolio must be fully invested.

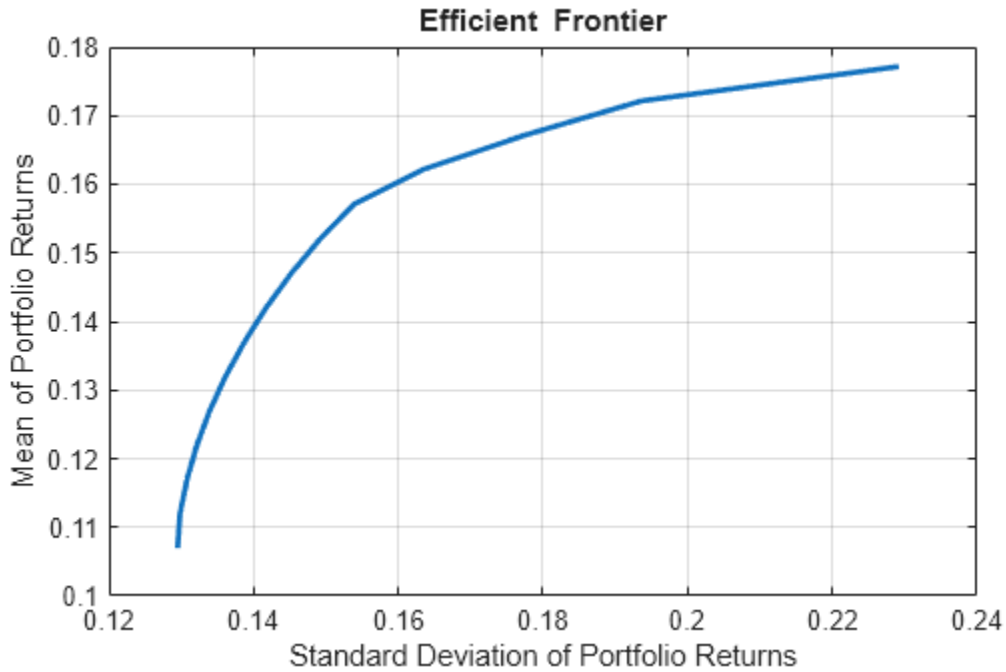
```
p = setBudget(p,1,1);
```

Compute the weights of 15 portfolios on the efficient frontier using `estimateFrontier`. These portfolios represent the possible actions at each rebalancing period.

```
nPortfolios = 15;  
pwgt = estimateFrontier(p,nPortfolios);
```

Plot the efficient frontier using `plotFrontier`.

```
[prsk,pret] = plotFrontier(p,pwgt);
```

States for Wealth Nodes

The states are defined as the wealth values at time t . To simplify the problem, discretize the wealth values to generate a wealth grid, W_i , for $i \in \{1, \dots, i_{\max}\}$. W_1 corresponds to the smallest possible wealth $W_{\min}$ and $W_{i_{\max}}$ corresponds to the largest possible wealth $W_{\max}$. To determine $W_{\min}$ and $W_{\max}$, assume that the evolution of wealth follows a geometric Brownian motion

$$W(t) = W(0)e^{(\mu - \sigma^2/2)t + \sigma\sqrt{t}Z},$$

where Z is a standard normal random variable. GBWM does not require that the wealth evolution follow a Brownian motion. However, the technique in this example requires the evolution model to be Markovian. Therefore, this example assumes that the probability of being at a certain state at time $t + 1$ only depends on the state at time t and the action taken.

This example also assumes that Z realistically takes values between -3 and 3 . When $Z = -3$, $W_{\min}$ corresponds to the smallest value obtained in the trajectory of the Brownian motion setting, μ is the smallest possible portfolio return $\mu_{\min}$, and σ is the largest possible portfolio risk $\sigma_{\max}$. Likewise, when $Z = 3$, $W_{\max}$ corresponds to the largest value in the trajectory, μ is the largest possible portfolio return $\mu_{\max}$, and σ is the largest possible portfolio risk $\sigma_{\max}$. For $W_{\max}$, use $\sigma_{\max}$ because the term in the exponential is increasing for all $\sigma \leq 3/\sqrt{t}$; if $\sigma_{\max} > 3/\sqrt{t}$, use $\sigma = 3/\sqrt{t}$ instead.

Define $\mu_{\min}$, $\mu_{\max}$, $\sigma_{\min}$, and $\sigma_{\max}$.

```
% Define return limits.
mu_min = pret(1);
mu_max = pret(end);

% Define volatility limits.
sigma_min = prsk(1);
sigma_max = prsk(end);
```

The smallest realistic value of W is

$$\widehat{W}_{\min} = \min_{\tau \in \{0, 1, \dots, T\}} \left\{ W(0) e^{\left(\mu_{\min} - \frac{\sigma_{\max}^2}{2} \right) \tau - 3\sigma_{\max} \sqrt{\tau}} \right\}.$$

% Define the minimum possible wealth.

```
timeVec = 0:T;
WMinVec = W0*exp((mu_min-(sigma_max^2)/2)*timeVec- ...
    3*sigma_max*sqrt(timeVec));
WHatMin = min(WMinVec);
```

The largest realistic value of W is

$$\widehat{W}_{\max} = \max_{\tau \in \{0, 1, \dots, T\}} \left\{ W(0) e^{\left(\mu_{\max} - \frac{\sigma_{\max}^2}{2} \right) \tau + 3\sigma_{\max} \sqrt{\tau}} \right\}.$$

% Define the maximum possible wealth.

```
WMaxVec = W0*exp((mu_max-(sigma_max^2)/2)*timeVec+ ...
    3*sigma_max*sqrt(timeVec));
WHatMax = max(WMaxVec);
```

Now, you can fill in the grid points between $\widehat{W}_{\min}$ and $\widehat{W}_{\max}$. When you set $Z = 1$ and $t = 1$ and ignore the drift term $(\mu - \sigma^2/2)t$, note that σ is proportional to the logarithm of the wealth. Thus, you can work in logarithmic space and add a grid point every $\sigma_{\min}/\rho_{\text{grid}}$ units. ρ_{grid} is an exogenous parameter that determines the density of the grid. As ρ_{grid} grows, the number of wealth nodes increases.

% Transform to the logarithmic space.

```
lWHatMin = log(WHatMin);
lWHatMax = log(WHatMax);
% Define the grid in the logarithmic space.
rho = 3;
lWGrid = (lWHatMin:sigma_min/rho:lWHatMax)';
```

To guarantee that there exists a point in the wealth grid that matches $W(0)$, shift the logarithmic grid downwards by the smallest shift such that one of those values matches the logarithm of the initial portfolio value.

% Compute the shift for wealth grid.

```
lW0 = log(W0);
difference = lWGrid-lW0;
minDownShift = min(difference(difference>=0));
% Define the wealth nodes.
WGrid = exp(lWGrid - minDownShift);
nWealthNodes = size(WGrid,1); % Number of wealth nodes
```

Value Function and Optimal Strategy

The problem in this example is a multi-period optimization problem. The goal is to find the action at each rebalancing period that maximizes the probability that the wealth exceeds the target goal at the end of the investment period. To solve this problem, use the Bellman equation, which in this case is given by

$$V(W_i(t)) = \max_{\pi} \left[\sum_j V(W_j(t+1)) p(W_j(t+1) | W_i(t), \pi) \right].$$

The value function $V(W_i(t))$ represents the probability of exceeding the wealth goal G at time T , given that at time t , the wealth level is W_i . Simultaneously, $p(W_j(t+1)|W_i(t), \pi)$ represents the probability of moving to wealth node j at time $t+1$, given that at time t , you are at wealth node i and you chose portfolio π as the rebalancing action.

Compute Transition Probabilities

Since this example assumes that the wealth goal follows a geometric Brownian motion, the transition probabilities of being in wealth node W_j at time $t+1$, given that you are at wealth node W_i at time t , are time-homogeneous. Thus, the transition probability for a given portfolio choice π is defined as

$$p(W_j(t+1)|W_i(t), \pi) = \mathbb{P}(W_j \leq W(t+1) < W_{j+1} | W(t) = W_i, \pi) = \tilde{p}(W_{j+1}|W_i, \pi) - \tilde{p}(W_j|W_i, \pi), \quad \forall t, \in \{0, \dots, T\}$$

where

$$\tilde{p}(W_k|W_i, \pi) = \Phi\left(\frac{1}{\sigma_\pi} \left(\ln\left(\frac{W_k}{W_i}\right) - \left(\mu_\pi - \frac{\sigma_\pi^2}{2}\right) \right)\right),$$

Φ is the cumulative distribution function of the standard normal random variable, and μ_π and σ_π are the return and risk of portfolio π . If $j = 1$,

$$p(W_1(t+1)|W_i(t), \pi) = \tilde{p}(W_2|W_i, \pi), \quad \forall t \in \{0, \dots, T\},$$

and if $j = i_{\max}$,

$$p(W_{i_{\max}}(t+1)|W_i(t), \pi) = 1 - \tilde{p}(W_{i_{\max}}|W_i, \pi), \quad \forall t \in \{0, \dots, T\}.$$

Note that this example computes transition probabilities differently from the transition probabilities as described by Das [1 on page 4-378], to keep a consistent mapping between $W(t)$ and a unique wealth node index.

```
% Compute the transition probabilities for different portfolios:
% pTransition(i,j,k) = p(Wj|Wi,mu_k)
pTransition = zeros(nWealthNodes,nWealthNodes,nPortfolios);
for k = 1:nPortfolios
    % Get the return and variance of portfolio k.
    mu = pret(k);
    sigma = prsk(k);

    % Compute p tilde before the cdf.
    pTilde = 1/sigma*(log(WGrid'./WGrid) - (mu-(sigma^2)/2));
    % Auxiliary matrices for edge cases (j = 1 and j = i_max)
    pTildeAuxUB = [pTilde(:,2:end) inf(nWealthNodes,1)];
    pTildeAuxLB = [-inf(nWealthNodes,1) pTilde(:,2:end)];

    % Compute the transition probabilities for portfolio k.
    pTransition(:, :, k) = normcdf(pTildeAuxUB)-normcdf(pTildeAuxLB);
end
```

Maximize Probability of Achieving Investor's Goal

Since the value function is the probability of achieving the wealth investment goal G at the final time step, you know that the value function at $t = T$ is given by

$$V(W_i(t)) = \begin{cases} 0 & \text{if } W_i(T) < G \\ 1 & \text{if } W_i(T) \geq G \end{cases}$$

```
% Initialize the matrix that stores the value function at different
% wealth nodes and time periods.
V = zeros(nWealthNodes,T+1);
% Define the Value function at t = T.
V(:,end) = (WGrid >= G);
```

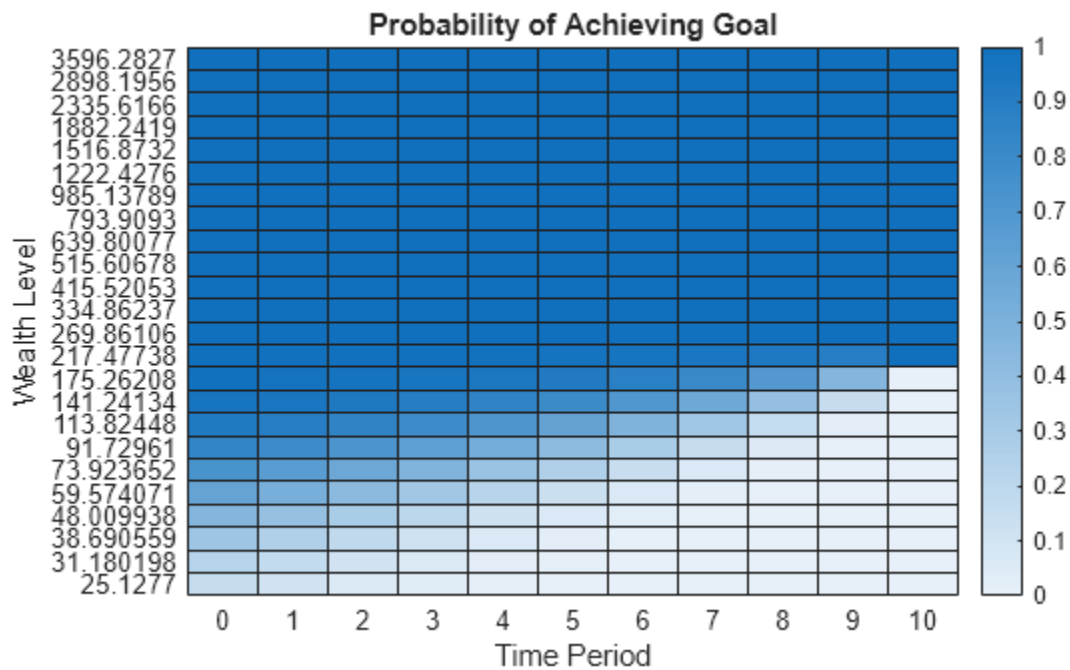
To compute the value of $V(W_i(t))$ for $t < T$, you need to go backwards in time. You start at $t = T - 1$ to determine the portfolio that achieves the maximum in the Bellman equation and $V(W_i(T - 1))$. Then continue with $t = T - 2$, then $t = T - 3$, until you reach $t = 0$. The value of $V(W(0))$ is the optimal probability of attaining the wealth goal G from the initial wealth $W(0)$.

```
% Compute the optimal value function and portfolio at different
% wealth nodes and time periods.
portIdx = zeros(nWealthNodes,T);
for t = T:-1:1
    pTilde = zeros(nWealthNodes,nPortfolios);
    for k = 1:nPortfolios
        % sum_j ( V(Wj(t+1)) * p(Wj|Wi,mu_k) )
        pTilde(:,k) = pTransition(:,k)*V(:,t+1);
    end
    % Choose the portfolios that achieve the best probabilities.
    [V(:,t),portIdx(:,t)] = max(pTilde,[],2);
end
```

Note that $V(i, t)$ is $V(W_i(t - 1))$, which denotes the optimal probability of achieving the wealth goal from wealth node W_i at time period $t - 1$. On the other hand, $\text{portIdx}(i, t)$ defines the optimal portfolio to be chosen at time period $t - 1$ if you are at wealth node W_i . The value of t is shifted down for $V(i, t)$ and $\text{portIdx}(i, t)$, given that array indices start at 1 and t starts at 0.

Use heatmap to plot the value function.

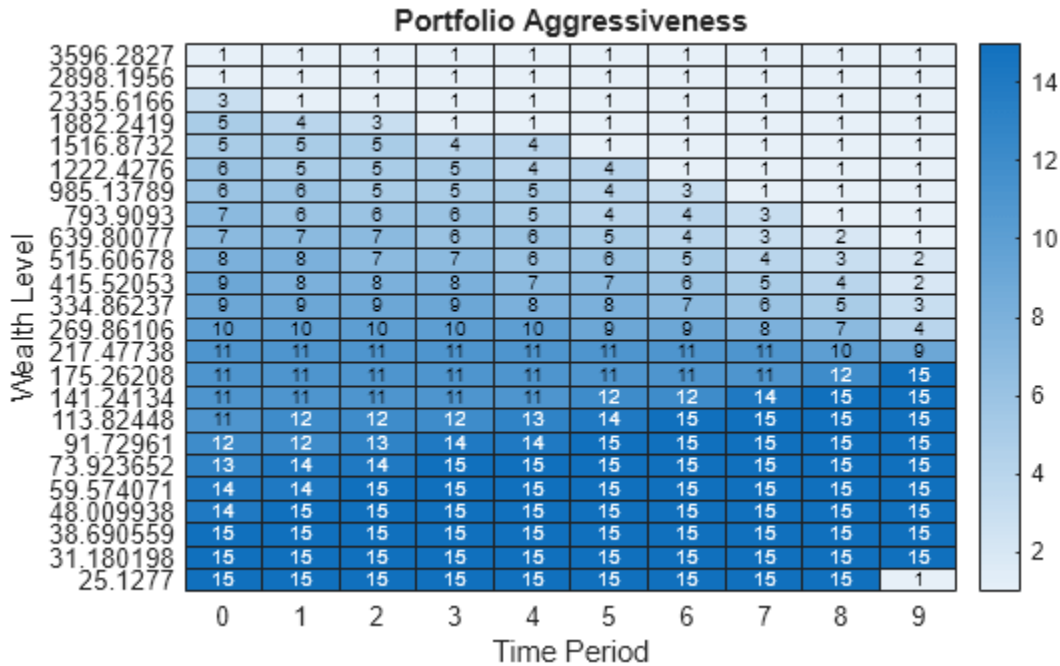
```
% Plot a heatmap of the value function.
figure;
heatmap(0:T,flip(WGrid(1:5:end)),flip(V(1:5:end,:)))
xlabel("Time Period")
ylabel("Wealth Level")
title("Probability of Achieving Goal")
```



The figure shows how the probability of achieving the goal becomes larger as the initial wealth grows. Also, this figure demonstrates that as time passes, the current wealth must be higher than in the previous periods to get the same probability of achieving the goal.

Use heatmap to plot the optimal portfolio.

```
% Plot a heatmap of optimal portfolio.
figure;
heatmap(0:T-1,flip(WGrid(1:5:end)),flip(portIdx(1:5:end,:)))
xlabel("Time Period")
ylabel("Wealth Level")
title("Portfolio Aggressiveness")
```



This figure shows that the higher the wealth, the lower the number of the optimal portfolio. Since the number of the portfolio is directly related to the aggressiveness of the strategy, you can see that the wealthier the portfolio, the less risk is needed to achieve the final wealth goal. When the portfolio has less money, then the optimal portfolio is the one with a larger expected return, even at the cost of increasing the volatility. Also, note that the closer the time period is to the end of the investment horizon, the choice of the optimal portfolio becomes polarized, where the portfolio is either extremely aggressive or extremely conservative.

Implement Optimal Strategy

The following describes the workflow to implement the optimal strategy:

- 1 At $t = 0$, choose the optimal strategy shown in `portIdx(i, j)` such that i is the index associated with the wealth node that satisfies $W_i \leq W(0) < W_{i+1}$ and $j = t + 1$. Also, let $A^*(0) = \text{portIdx}(i, j)$.
- 2 Rebalance the portfolio to match the weights in `pwgt(:, A*(0))`. Then, define $\pi^*(0) = \text{pwgt}(:, A^*(0))$.
- 3 Either using historical data for time period $t = 0$ or simulating one step of a Brownian motion, obtain $W(1)$, given that you started at $W(0)$, and chose portfolio $\pi^*(0)$.
- 4 Repeat steps 1 through 3 for $t = 1, \dots, T - 1$.

```
% Simulate one path following the optimal strategy.
rng('default')
[Z,WPath,portPath,VPath] = DynamicPortSim(W0,WGrid,pret,prsk, ...
    portIdx,V);
```

Present a table with the results of the simulation.

```
simulation = table((0:T)',Z,WPath,portPath,VPath,'VariableNames', ...
    {'t','Z','Wealth','OptPort','ProbOfSuccess'})
```

simulation=11x5 table

t	Z	Wealth	OptPort	ProbOfSuccess
0	0.53767	100	12	0.87953
1	1.8339	126.69	11	0.92018
2	-2.2588	194.27	11	0.98474
3	0.86217	158.68	11	0.9323
4	0.31877	209.52	11	0.98044
5	-1.3077	254.46	10	0.99184
6	-0.43359	241.04	10	0.98366
7	0.34262	260.14	8	0.99024
8	3.5784	311.62	5	0.99927
9	2.7694	566.28	1	1
10	NaN	894.5	NaN	1

Note that as the probability of attaining the wealth goal (the value function) increases, the aggressiveness (number) of the optimal portfolio decreases. Now, you can plot the path of the wealth against the probability of achieving the wealth goal at the end of the investment period.

figure;

% Plot the wealth path.

yyaxis left

plot(0:T,WPath)

hold on

ylines(G,'--k');

ylabel("Wealth Level");

% Plot the value function path.

yyaxis right

plot(0:T,VPath)

ylabel("Probability of Success");

% Specify the title, labels, and legends for the plot.

title("Optimal Strategy Outcome")

xlabel("Time Period");

legend("Wealth path","Wealth goal",Location="nw");

grid on

hold off

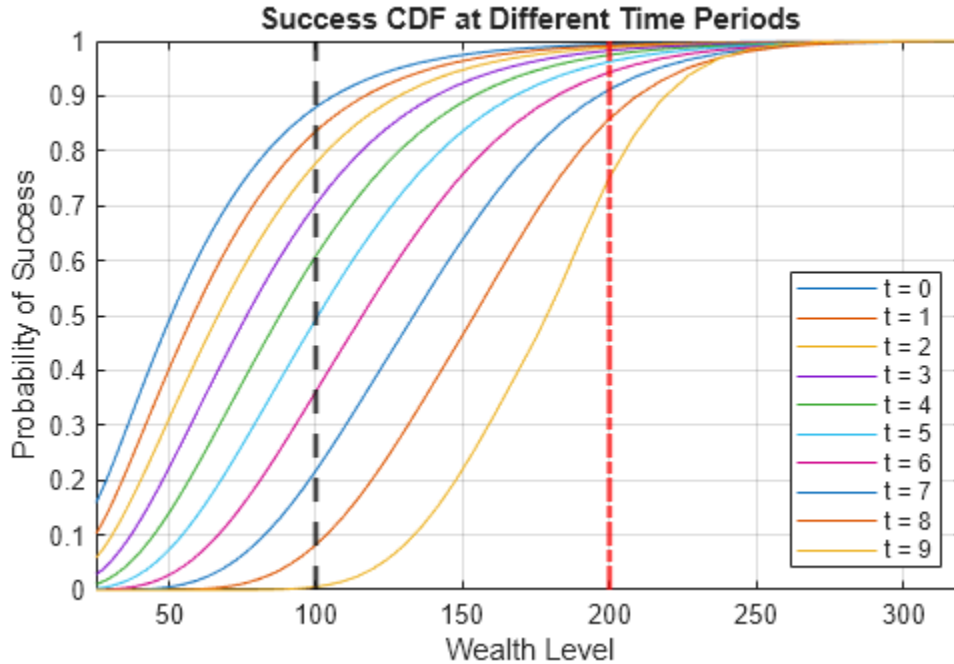


At $t = 4$, the wealth goal has been achieved. However, because of the uncertainty in the problem, the probability of success is not yet 1. Furthermore, trying different paths (random seeds) shows how even a monotonic increase in the wealth does not translate to a monotonic increase in the probability of success. This behavior occurs because the wealth increase at later time periods should be higher than at earlier time periods to have the same increase in the probability of success.

Plot the cumulative distribution function (CDF) of attaining the goal wealth at different time periods.

```
% Plot the cumulative distribution function (CDF) of success at
% different time periods.
figure
nWN = 60;
plot(WGrid(1:nWN),V(1:nWN,1:end-1))
xline(W0,'--k','LineWidth',2); % Initial wealth
xline(G,'-.r','LineWidth',2); % Goal wealth
xlim(WGrid([1,nWN]))

% Specify the title, labels, and legends.
title('Success CDF at Different Time Periods')
xlabel('Wealth Level')
ylabel('Probability of Success')
legend("t = "+num2str((0:T-1)'),Location="se")
grid on
```

This figure shows how as the time period approaches the investment horizon, the CDF shifts to the right and becomes steeper. This shift demonstrates that changes in the wealth level have a higher impact toward the end of the investment horizon. For example, an increase of 100 to 110 wealth points at $t = 0$ only increases the probability of success by 3%, while the same wealth increase at $t = 4$ increases the probability of success by 7%.

Local Functions

```
function [Z,WPath,portPath,VPath] = DynamicPortSim(W0,WGrid,...
    pret,prsk,portIdx,V)
% Define time periods.
T = size(V,2)-1;

% Define standard normal random variables for the Brownian motion.
Z = randn(T,1);
Z = [Z;nan]; % No randomness at the final time period

% Find the wealth node index of the initial wealth.
tol = 1e-8; % Account for numerical inaccuracies
difference = WGrid-W0;
% Assign to wealth node j such that  $W_j \leq W(0) < W_{j+1}$ 
WNodeIdx = find(difference<=tol,1,'last');

% Run the simulation.
WPath = [W0; zeros(T,1)];
portPath = [portIdx(WNodeIdx,1); zeros(T,1)];
VPath = [V(WNodeIdx,1); zeros(T,1)];
for t = 1:T
    % Get the return and variance of optimal portfolio at time t.
    mu = pret(portIdx(WNodeIdx,t));
    sigma = prsk(portIdx(WNodeIdx,t));

    % Compute the wealth at time t+1 following the Brownian motion.
```

```
WPath(t+1) = WPath(t)*exp(mu-(sigma^2)/2+sigma*Z(t));

% Compute the wealth node index corresponding to the current wealth.
difference = WGrid-WPath(t+1);
% Assign to wealth node j such that W_j <= W(t+1) < W_{j+1}
WNodeIdx = find(difference <= tol,1,'last');

% Store the optimum strategy and value function.
if t<T
    portPath(t+1) = portIdx(WNodeIdx,t+1);
else
    portPath(t+1) = nan;
end
VPath(t+1) = V(WNodeIdx,t+1);
end
end
```

References

[1] Das, Sanjiv R., Daniel Ostrov, Anand Radhakrishnan, and Deep Srivastav. “Dynamic Portfolio Allocation in Goals-Based Wealth Management.” *Computational Management Science* 17, no. 4 (December 2020): 613–40. Available at: <https://doi.org/10.1007/s10287-019-00351-7>.

See Also

Portfolio | estimatePortSharpeRatio | estimateFrontier | estimateFrontierByReturn | estimateFrontierByRisk

Related Examples

- “Single Period Goal-Based Wealth Management” on page 4-362
- “Multiperiod Goal-Based Wealth Management Using Reinforcement Learning” on page 4-379

More About

- “Choose MINLP Solvers for Portfolio Problems” on page 4-140

Multiperiod Goal-Based Wealth Management Using Reinforcement Learning

This example shows a reinforcement learning (RL) approach to maximize the probability of obtaining an investor's wealth goal at the end of the investment horizon. This problem is known in the literature as goal-based wealth management (GBWM). In GBWM, risk is not necessarily measured using the standard deviation, the value-at-risk, or any other common risk metric. Instead, risk is understood as the likelihood of not attaining an investor's goal. This alternative concept of risk implies that, sometimes, in order to increase the probability of attaining an investor's goal, the optimal portfolio's traditional risk (that is, standard deviation) must increase if the portfolio is underfunded. In other words, for the investor's view of risk to *decrease*, the traditional view of risk must *increase* if the portfolio's wealth is too low.

The purpose of the investment strategy is to determine a dynamic portfolio allocation that maximizes the probability of achieving a wealth goal G at the time horizon T . The dynamic allocation strategy is the optimal solution of the following multiperiod portfolio optimization problem

$$\max_{\{A(0), A(1), \dots, A(T-1)\}} \mathbb{P}(W(T) \geq G),$$

where $W(T)$ is the terminal portfolio wealth and $A(t)$ are the possible actions and allocations at time $t = 0, 1, \dots, T - 1$.

To solve this problem, this example follows the GBWM strategy of Das and Varma [1 on page 4-391] uses RL to optimize the probability of attaining an investment goal. The goal of RL is to train an agent to complete a task within an unknown environment. The agent receives observations and a reward from the environment and sends actions to the environment. The reward is a measure of how successful an action is with respect to completing the task goal.

The agent contains two components: a policy and a learning algorithm.

- The policy is a mapping that selects actions based on the observations from the environment. Typically, the policy is a function approximator with tunable parameters, such as a deep neural network.
- The learning algorithm continuously updates the policy parameters based on the actions, observations, and reward. The goal of the learning algorithm is to find an optimal policy that maximizes the cumulative reward received during the task.

In other words, reinforcement learning involves an agent learning the optimal behavior through repeated trial-and-error interactions with the environment without human involvement. For more information on reinforcement learning, see "What Is Reinforcement Learning?" (Reinforcement Learning Toolbox).

The main advantage of leveraging reinforcement learning is that you can use an unknown environment. That is, in theory, you do not need to make assumptions about the state transition probabilities, you do not need to define the probability that the investment achieves a certain wealth level in a given time period. However, reinforcement learning assumes that there is an *accurate* way to simulate the transition from one state to the next for a given time period.

Problem Definition

```
rng(0, 'twister');
```

Specify the initial wealth.

```
initialWealth = 100;
```

Specify the target wealth at the end of the investment horizon.

```
targetWealth = 200;
```

Specify a time horizon.

```
finalTimePeriod = 10;
```

Define the mean and covariance of the annual returns.

```
returnMean = [0.05; 0.1; 0.25];  
returnCovariance = [0.0025  0      0      ;  
                    0      0.04   0.02   ;  
                    0      0.02   0.25   ];
```

To solve the problem using RL, you need to define:

- Actions on page 4-380 — Portfolio weights for investment portfolio.
- Observations on page 4-381 — Time period t and wealth at time period t .
- Environment on page 4-382 — Model of the evolution of the problem. The environment simulates the next observation after receiving an action and computes its associated reward.
- Reward on page 4-383 — Part of the environment that measures how well the received action contributes to achieving the task.
- Agent on page 4-384 — Component trained to complete the task within the environment. The agent is responsible for choosing the actions required to complete the task.

Define Actions

The action at each rebalancing period is to choose the weights of the investment portfolio for the next time period. In this example, assume that the possible portfolio weights are those of portfolios on the efficient frontier. Also, you can assume that the choice of possible portfolios is fixed throughout the full investment period and that they represent a finite subset of the efficient portfolios. By avoiding working with a set of continuous actions in this way, you simplify the training and easily account for the constraints on the portfolio weights (for example, that they sum to 1). However, you do have to assume that the distribution of the returns is time-homogeneous; that is, the underlying mean and covariance of the returns is the same at every time period.

Start by creating a `Portfolio` object with the asset information from the Problem Definition on page 4-379.

```
p = Portfolio(AssetMean=returnMean,AssetCovar=returnCovariance);
```

Add constraints to the portfolio problem. Use `setBounds` to bound the portfolio weights. Individual assets cannot represent more than 50% of the portfolio wealth and shorting is not allowed.

```
p = setBounds(p,0,0.5);
```

Use `setBudget` to specify that the portfolio must be fully invested.

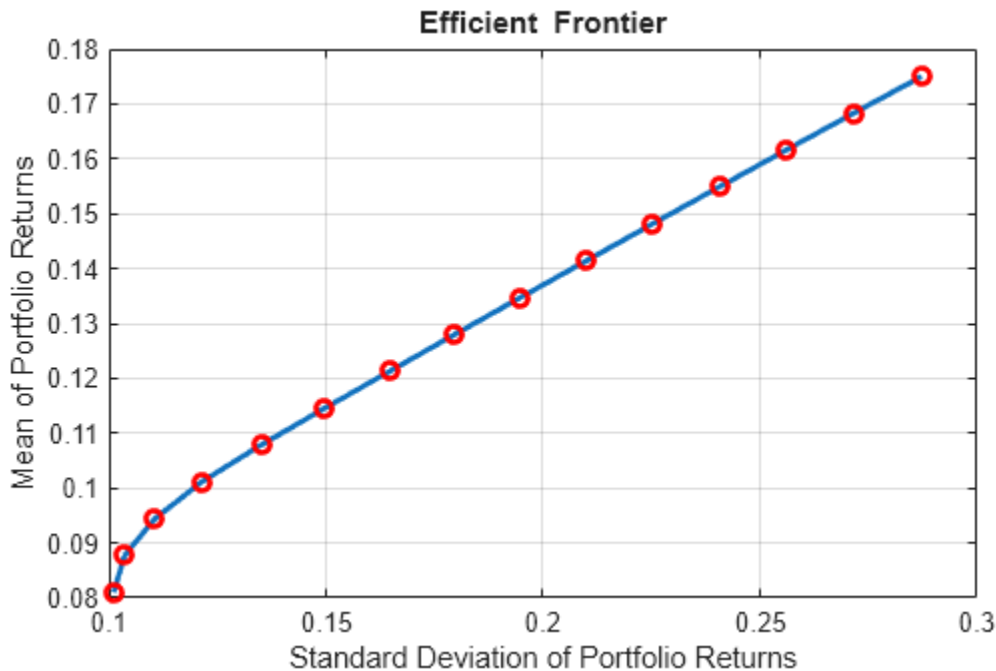
```
p = setBudget(p,1,1);
```

Compute the weights of 15 portfolios on the efficient frontier using `estimateFrontier`. These portfolios represent the possible actions at each rebalancing period.

```
numPortfolios = 15;
pwgt = estimateFrontier(p,numPortfolios);
```

Plot the efficient frontier using `plotFrontier`.

```
figure
[prsk,pret] = plotFrontier(p,pwgt);
hold on
scatter(prsk,pret,"red","o","LineWidth",2)
hold off
```



Use `rlFiniteSetSpec` (Reinforcement Learning Toolbox) to create the discrete action space for the reinforcement learning environment.

```
actionInfo = rlFiniteSetSpec(1:15);
```

Define Observations

The observations are defined as elements of the environment that describe important aspects of the current state. In this example the observations consist of the current time period t and the wealth at that time period W_t . Use `rlNumericSpec` (Reinforcement Learning Toolbox) to create the observations space, a (2×1) vector containing two signals: the wealth level and the time period.

```
% Observation space:
% [wealth; timePeriod]
observationInfo = rlNumericSpec([2 1]);
```

Scale Observations

Signals with widely different ranges can skew the learning process, making it hard for the agent to successfully learn the important features. In this example, we scale the wealth level to the range [0,1]. To do this, we assume that the wealth follows a geometric Brownian motion

$$W(t) = W(0)\exp\left\{\left(\mu - \sigma^2/2\right)t + \sigma\sqrt{t}Z\right\}$$

where Z is a standard normal random variable. The wealth does not need to follow a Brownian motion, but this assumption helps obtain reasonable upper and lower bounds on the wealth evolution.

Assuming that Z realistically takes values between -3 and 3 , we have that the smallest realistic value of W is

$$W_{\min} = \min_{\tau \in \{0, 1, \dots, T\}} \left\{ W(0)\exp\left\{\left(\mu_{\min} - \frac{\sigma_{\max}^2}{2}\right)\tau - 3\sigma_{\max}\sqrt{\tau}\right\} \right\}$$

% Define the minimum possible wealth.

```
timeVec = 0:finalTimePeriod;
mu_min = pret(1);
sigma_max = prsk(end);
WMinVec = initialWealth*exp((mu_min-(sigma_max^2)/2)*timeVec- ...
    3*sigma_max*sqrt(timeVec));
WMin = min(WMinVec);
```

And the largest realistic value of W is

$$W_{\max} = \max_{\tau \in \{0, 1, \dots, T\}} \left\{ W(0)\exp\left\{\left(\mu_{\max} - \frac{\sigma_{\max}^2}{2}\right)\tau + 3\sigma_{\max}\sqrt{\tau}\right\} \right\}$$

% Define the maximum possible wealth.

```
mu_max = pret(end);
WMaxVec = initialWealth*exp((mu_max-(sigma_max^2)/2)*timeVec+ ...
    3*sigma_max*sqrt(timeVec));
WMax = max(WMaxVec);
```

Create Environment Model

You create the environment model using the following steps:

- 1 Define a `myResetFunction` function that describes the initial conditions of the environment at the beginning of a training episode.
- 2 Define a custom `myStepFunction` function that describes the dynamics of the environment. This includes how the state changes from a current state given the agent action and how the earned reward is computed by such an action. For the reward, here you can choose either a *sparse reward* function or a *constant return reward* function. At each training time step, the state of the model is updated using the `myStepFunction` function. This function also determines when a training episode finishes.

For more information see “Create Custom Environment Using Step and Reset Functions” (Reinforcement Learning Toolbox).

Define Reset Function

The `myResetFunction` function (see Local Functions on page 4-391) sets the wealth level to its initial state, `initialWealth`, and the time period to 0.

```
resetFcn = @( ) myResetFunction(initialWealth,WMin,WMax);
```

State Change Model

The custom `myStepFunction` (see Local Functions on page 4-391) simulates the wealth evolution from the current time period to the next given the portfolio weights associated to the action selected by the agent. For simplicity, this example uses a geometric Brownian motion to simulate the wealth evolution. So, given a wealth level at time step t , the wealth level at time $t+1$ is:

$$W_{t+1} = W_t \exp\left\{\left(\mu_i - \frac{\sigma_i^2}{2}\right) + \sigma_i Z\right\},$$

where Z is a standard normal random variable and μ_i and σ_i are the mean and standard deviation of the portfolio associated with the i th action.

Sparse Reward Function

Since the goal of the problem is to achieve the wealth target by the end of the investment period, you can use a reward function awards a value of 1 if the goal is achieved at the end of the investment period and 0 otherwise. See Local Functions on page 4-391 for the `sparseReward` function.

$$R(W, t) = \begin{cases} 0 & \text{if } t < T \text{ or } W < G \\ 1 & \text{if } t = T \text{ and } W \geq G \end{cases}$$

```
% Sparse reward function handle
```

```
sparseRewardFcnHandle = @(loggedSignals) sparseReward(loggedSignals, ...  
    finalTimePeriod, targetWealth, WMin, WMax);
```

Agents defined with a sparse reward are challenging to train because a large amount of states do not return any signal. This issue becomes more apparent as the episodes become long.

Constant Return Reward Function

To reduce the training time, you can also use a reward function that tries to lead the agent to achieve a minimum wealth level at each investment period. The `constantReturnReward` function (see Local Functions on page 4-391) assumes that the returns are the same throughout the entire investment horizon and that the return value satisfies the following inequality:

$$W_0(1+r)^T \geq G.$$

```
% Compute r, the constant return needed to satisfy the goal by the  
% end of the investment horizon.
```

```
constantReturn = ...  
    nthroot(targetWealth/initialWealth, finalTimePeriod) - 1;
```

When the wealth level W_t is greater than $W_0(1+r)^t$, you give the agent a small reward (in this example, select a reward of 0.1). Finally, you still give a reward of 1 if the agent reaches the goal at the end of the investment period and 0 otherwise.

Define the constant return reward function handle.

% Reward Function Handle

```
constantReturnRewardFcnHandle = @(loggedSignals) constantReturnReward(loggedSignals, ...  
    initialWealth,finalTimePeriod,targetWealth,constantReturn,WMin,WMax);
```

Episode Termination

The custom step function flags that the episode has finished when the current time period reaches `finalTimePeriod`.

Define Custom Step Function

Select the reward function, then define the step function handle.

% Select a reward function

```
rewardFcnHandle = ;
```

% Define step function handle

```
stepFcn = @(action,loggedSignals) myStepFunction(action, ...  
    loggedSignals,prsk,pret,finalTimePeriod,rewardFcnHandle,WMin,WMax);
```

Define Environment

Use `rlFunctionEnv` (Reinforcement Learning Toolbox) to construct the custom environment using the defined observations, actions, and reset and step functions.

```
gbwmEnvironment = rlFunctionEnv(observationInfo,actionInfo,stepFcn, ...  
    resetFcn);
```

Define Agent

The purpose of the agent is to select actions that are sent to the environment. The agent then receives new observations from the environment and the reward generated by the submitted actions. The goal of RL is to train the agent to select the best possible actions to maximize the probability of attaining the wealth goal by the end of the investment period.

Reinforcement Learning Toolbox™ provides several built-in agents that you can train in environments with either continuous or discrete observation and action spaces. The table in the section "Built-In Agents" of "Reinforcement Learning Agents" (Reinforcement Learning Toolbox) summarizes the types of agents for the different types of observation and action spaces.

Given that the action space in this example is discrete and the observation space is continuous, this example uses a "Deep Q-Network (DQN) Agent" (Reinforcement Learning Toolbox).

Use `rlAgentInitializationOptions` (Reinforcement Learning Toolbox) to specify the number of neurons in each learnable layer.

```
initializationOptions = ...  
    rlAgentInitializationOptions('NumHiddenUnit',100);
```

Use `rlDQNAgent` (Reinforcement Learning Toolbox) to create the agent using the defined action and observation specifications.

```
DQNAgent = rlDQNAgent(observationInfo,actionInfo, ...  
    initializationOptions);
```

Set the agent options.


```
DQNagent.AgentOptions.DiscountFactor = 1;
DQNagent.AgentOptions.EpsilonGreedyExploration.EpsilonDecay = 1e-4;
DQNagent.AgentOptions.MaxMiniBatchPerEpoch = 10;
```

Train Agent

Once you define the agent and the environment, you can use the `train` (Reinforcement Learning Toolbox) function to train the DQN agent. To configure the options for training, use `rlTrainingOptions` (Reinforcement Learning Toolbox).

```
trainingOptions = rlTrainingOptions;
```

Set the maximum number of episodes to train to 4000. If you use a constant return function, training takes less time, so reduce the number of episodes.

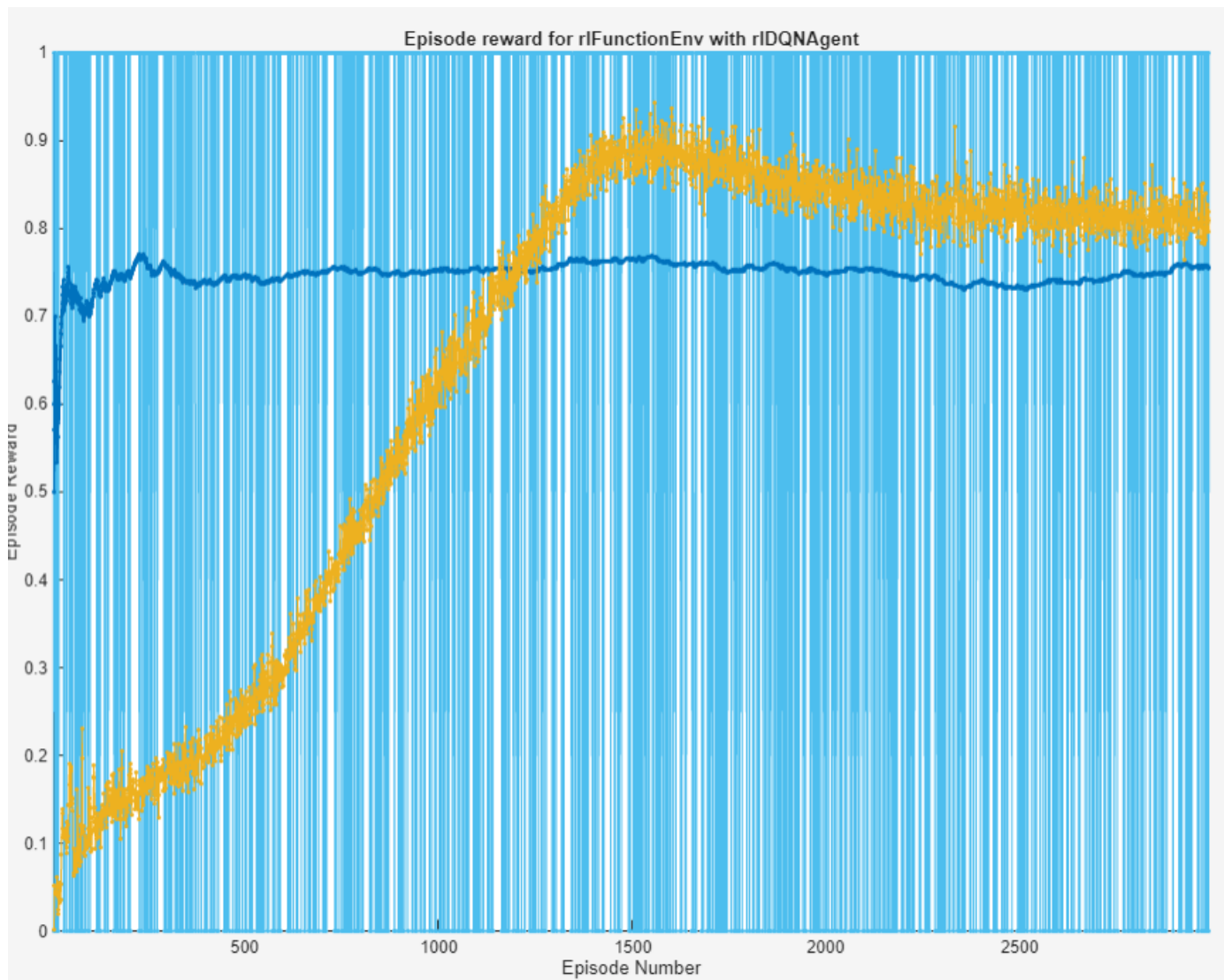
```
trainingOptions.MaxEpisodes = 4000;
```

Set the window length for averaging the rewards to 1,000. `ScoreAveragingWindowLength` is the number of episodes included in the average.

```
trainingOptions.ScoreAveragingWindowLength = 1e3;
```

Train the agent. The Reinforcement Learning Episode Manager is opened. When the `sparseReward` function (see Local Functions on page 4-391) is selected as the reward from the environment, the training time is approximately five minutes. If you set `do_train` to false, a network pretrained using the `sparseReward` function is loaded.

```
do_train = false;
if do_train
    % Train the agent
    trainingStats = train(DQNagent,gbwmEnvironment,trainingOptions);
else
    load('trainedAgent');
end
```



Simulate Using Agent

Simulate 1000 scenarios taking the actions of the training agent `DQNAgent`. Use `rlSimulationOptions` (Reinforcement Learning Toolbox) to set the number of simulations.

```
numSimulations = 1e3;
simulationOptions = rlSimulationOptions(NumSimulations=numSimulations);
```

Simulate the trained agent using `sim` (Reinforcement Learning Toolbox).

```
experience = sim(gbwmEnvironment,DQNAgent,simulationOptions);
```

Obtain the wealth observations per period and the rewards at the end of the episode. These rewards show whether the target wealth is achieved by the end of the investment horizon or not.

```
% Retrieve simulation information
cumulativeReward = 0;
wealthSimulation = zeros(finalTimePeriod+1,numSimulations);
for i = 1:numSimulations
```

```

        cumulativeReward = cumulativeReward+...
            experience(i).Reward.Data(end);
        wealthSimulation(:,i) =...
            squeeze(experience(i).Observation.obs1.Data(1,1,:));
    end

    % Transform wealth levels from the simulation to the original space.
    wealthSimulation = wealthSimulation*(WMax-WMin) + WMin;

```

Calculate an empirical approximation of the success rate of the agent's policy.

```

% Compute the testing success probability.
successProb = cumulativeReward/numSimulations

successProb =
0.7910

```

For this problem, the approximated GBWM optimal probability is 79.1%. This problem can be solved using a dynamic optimization approach as in “Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods” on page 4-367, where the optimal probability is 79.28%. Since both examples use a Brownian motion to simulate the wealth evolution, the solution from the dynamic programming approach is the closest to the theoretical optimum. Given that the probability obtained by the RL agent is 79.1%, it means that the policy obtained by the RL agent is close to the theoretical optimum as well.

% Plot the asset allocation for one simulation

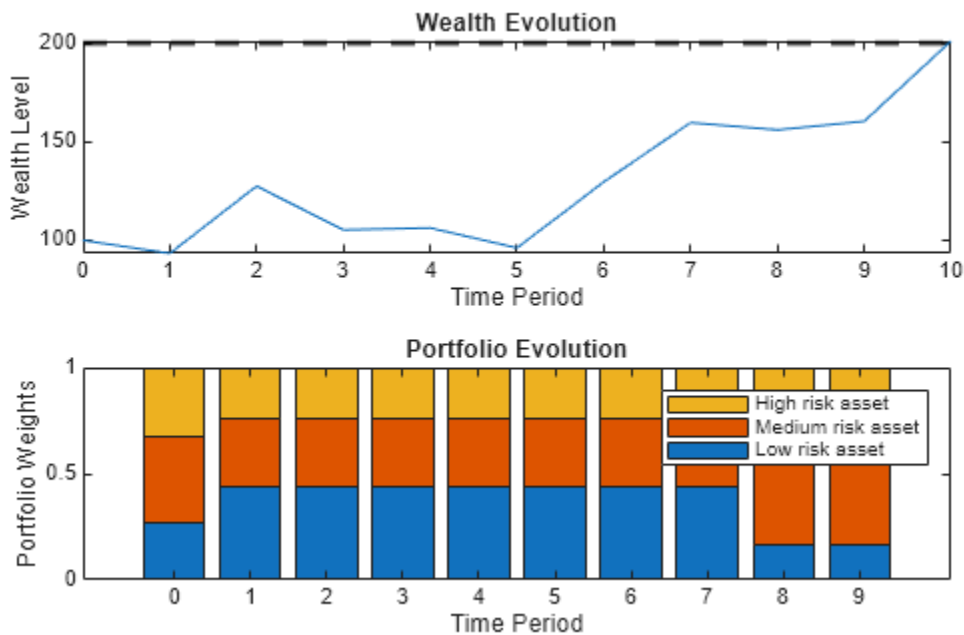
```

simNumber = 650  ;
figure
tiledlayout(2,1)

% Tile 1: Wealth progress
nexttile
plot(0:finalTimePeriod,wealthSimulation(:,simNumber))
hold on
yline(targetWealth,'--k',LineWidth=2)
title('Wealth Evolution')
xlabel('Time Period')
ylabel('Wealth Level')
hold off

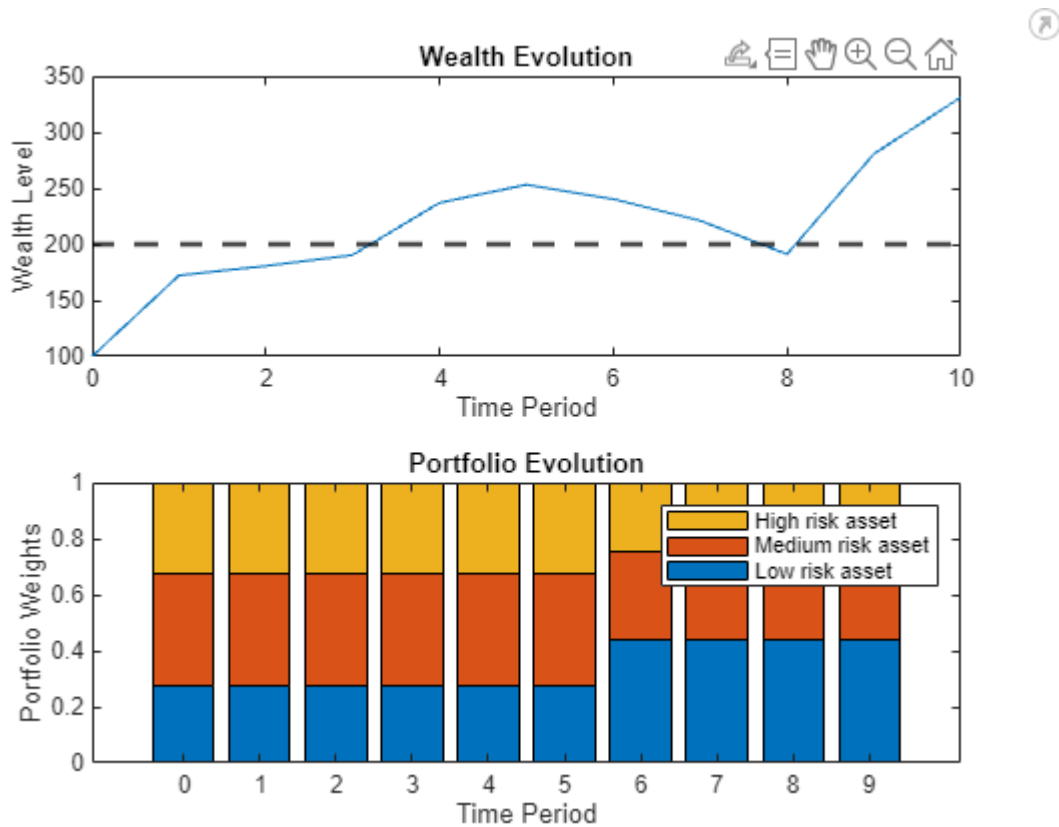
% Tile 2: Optimal action
nexttile
bar(0:finalTimePeriod-1, ...
    pwgt(:,experience(simNumber).Action.act1.Data),'stacked')
title('Portfolio Evolution')
xlabel('Time Period')
ylabel('Portfolio Weights')
legend('Low risk asset','Medium risk asset','High risk asset')

```



Using the sparse reward function, in simulation number 650 at time $t = 1$ the wealth drops below the initial wealth level. This drives the agent to choose a more conservative approach. Since the wealth is not recovering as fast as needed to achieve the goal by the end of the investment horizon, at $t = 8$ the agent chooses the most aggressive portfolio. By choosing an aggressive portfolio, the expected return increases at the cost of increasing the volatility. This extreme choice made by the agent is the result of the objective function not taking into account possible losses. The reward is only obtained if the goal is met, so an all-or-nothing strategy makes sense towards the end of the investment period. If the investor also wants to take their losses into account, they can add those losses into the reward function to penalize cases where the losses become too high. The entire agent's optimal strategy is illustrated in Heatmap of Optimal Policy on page 4-389.

Using the previous code, `simNumber=303` shows the following results. As with simulation 771, these results come from a model trained with the sparse reward function.



From time $t = 0$ to $t = 5$ the wealth evolution is favorable so the action remains the same. At $t = 6$, the wealth drops so the agent chooses a more conservative approach in this time period. Since the wealth level does not recover greatly as time passes, the agent holds the conservative approach to decrease risk all the way until the end of the investment horizon. These results show how the agent incorporates not only the wealth level but also the time period information into its decisions. Notice that even when the wealth level at $t = 9$ goes well above the target goal, the action is still conservative. This is because as time progresses, the agent needs to decrease the risk level to ensure that the goal is attained by the end.

Heatmap of Optimal Policy

Compute a set of possible wealth levels using 25 logarithmically spaced points between the maximum and minimum. Then, create a grid of possible observations using these levels and the time steps.

```
% Logarithmic wealth levels
numWealthPoints = 25;
wealthLevel = logspace(log10(WMin),log10(WMax),numWealthPoints);

% Possible observations
wVar = repmat(wealthLevel',finalTimePeriod,1);
tVar = repelem((0:finalTimePeriod-1)',numWealthPoints);

% Transform wealth to [0,1] space to feed to agent
wVarAgent = (wVar-WMin)/(WMax-WMin);

% Create a grid for agent to evaluate
```

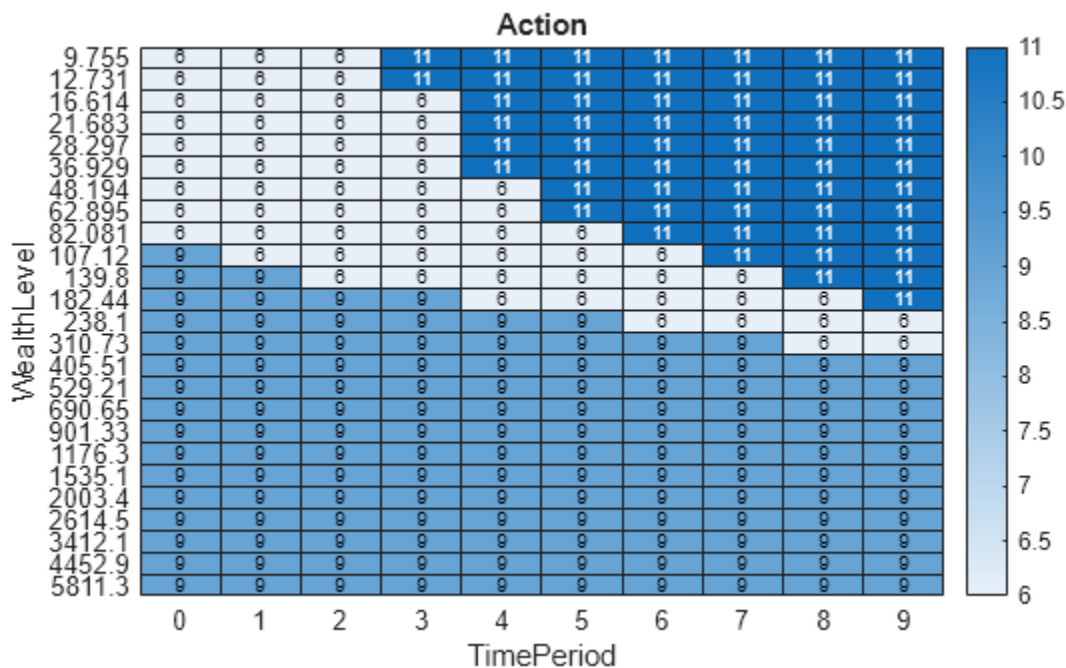
```
obs = zeros(2,1,finalTimePeriod*numWealthPoints);
obs(:,1,:) = [wVarAgent';tVar'];
```

Compute the action generated by the trained agent for each of the observations in the grid.

```
% Action for each observation
actions = getAction(DQNagent,obs);
actions = squeeze(actions{1});
```

Use heatmap to visualize the action taken by the trained agent for each of the observations.

```
% Create heatmap
figure
T = table(wVar,tVar,actions, ...
    VariableNames={'WealthLevel','TimePeriod','Action'});
heatmap(T,'TimePeriod','WealthLevel',ColorVariable='Action', ...
    ColorMethod='none')
```



This figure shows the actions chosen by the agent for different wealth levels and time periods. The numbers inside the grid represent the action taken by the agent at each state. In this example, using the sparse reward function, the trained agent only invests in portfolio #6, #9, or #11, where the higher the portfolio number, the more aggressive the portfolio is. In this policy, the aggressiveness of the portfolio increases as time gets closer to the end of the investment horizon. Notice that only at the beginning of the investment horizon, it is more likely that the agent choose the most conservative portfolio, especially if the wealth is below the wealth that would be achieved by the constant reward assumption (see section Constant Return Reward on page 4-383). However, if the wealth is very low close to the end of the investment horizon, then the agent needs to increase the risk to improve the probability of attaining the desired goal. This choice increases the expected return, even at the cost of increasing the volatility. The extreme choice made by the agent is the result of the objective function not taking into account possible losses. If the investor also wants to take their losses into account, they can add those losses into the objective and reward function to penalize cases where the losses become too high.

References

[1] Das, S. R., and S. Varma. "Dynamic Goals-Based Wealth Management Using Reinforcement Learning." *Journal Of Investment Management*, 18 No.2 (2020): 1-20.

Local Functions

```
function [initialObservation,loggedSignals] =...
    myResetFunction(initialWealth,WMin,WMax)
% Reset function to set wealth and time period to initial state.

% Transform wealth to [0,1] space
initialWealth = (initialWealth-WMin)/(WMax-WMin);

% Store initial state in loggedSignals
loggedSignals.Wealth = initialWealth;
loggedSignals.TimePeriod = 0;

% Return initial observation
initialObservation = [initialWealth; loggedSignals.TimePeriod];
end

function [nextObservation,reward,isDone,loggedSignals] =...
    myStepFunction(action,loggedSignals,prsk,pret,finalTimePeriod, ...
        rewardFunction,WMin,WMax)
% Step function to compute the wealth obtained from investing in
% the portfolio specified by the action. The function also
% computes the reward obtained by the current action and checks if
% episode is done.

% Get current wealth and time period
W = loggedSignals.Wealth;
t = loggedSignals.TimePeriod;

% Transform wealth to original space
W = W*(WMax-WMin) + WMin;

% Get risk and return levels for current action
mu = pret(action,1);
sigma = prsk(action,1);

% Compute next wealth
Z = randn;
W = W*exp(mu-(sigma^2/2)+sigma*Z);

% Return wealth to [0,1] space
W = (W-WMin)/(WMax-WMin);

% Store next state in LoggedSignal
loggedSignals.Wealth = W;
loggedSignals.TimePeriod = t+1;

% Return observation
nextObservation = [loggedSignals.Wealth;loggedSignals.TimePeriod];

% Compute reward
```

```
reward = rewardFunction(loggedSignals);

% Check if episode has ended
if loggedSignals.TimePeriod >= finalTimePeriod
    isDone = true;
else
    isDone = false;
end
end

function reward = sparseReward(loggedSignals,finalTimePeriod, ...
    targetWealth,WMin,WMax)
% Function that computes the reward obtained in the current state
% following the sparse rule:
%   R(W,t) = 1   if t >= T and W >= targetWealth
%           0   if t < T or W < targetWealth

% Transform wealth to original space
W = loggedSignals.Wealth*(WMax-WMin) + WMin;

if loggedSignals.TimePeriod >= finalTimePeriod && W >= targetWealth
    reward = 1;
else
    reward = 0;
end
end

function reward = constantReturnReward(loggedSignals, ...
    initialWealth,finalTimePeriod,targetWealth,constantReturn)
% Function that computes the reward obtained in the current state
% following the constant return rule:
%   R(W,t) = 1   if t >= T and W >= targetWealth
%           0.1  if t < T and W >= W0 * (1+r)^t
%           0    o.w.

% Get current state
t = loggedSignals.TimePeriod;
W = loggedSignals.Wealth;

% Transform wealth to original space
W = W*(WMax-WMin) + WMin;

% Compute reward
if t >= finalTimePeriod && W >= targetWealth
    reward = 1;
elseif W >= initialWealth*(1+constantReturn)^t
    reward = 0.1;
else
    reward = 0;
end
end
```

See Also

[Portfolio](#) | [estimatePortSharpeRatio](#) | [estimateFrontier](#) | [estimateFrontierByReturn](#) | [estimateFrontierByRisk](#)

Related Examples

- “Single Period Goal-Based Wealth Management” on page 4-362
- “Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods” on page 4-367

Compare Performance of Covariance Denoising with Factor Modeling Using Backtesting

This example uses backtesting to compare the performance of two investment strategies that use factor information to compute the portfolio weights. The first investment strategy uses `covarianceDenoising` to estimate both the covariance matrix and the number of factors to use in the second investment strategy. The second investment strategy uses a principal component analysis (PCA) factor model to estimate the covariance matrix with the number of factors obtained with `covarianceDenoising`. The PCA factor model follows the process in “Portfolio Optimization Using Factor Models” on page 4-231.

Load Data

Load a simulated data set that includes asset returns for a total $n = 100$ assets and 2000 daily observations.

```
load('asset_return_100_simulated.mat');
[numObservations,numAssets] = size(stockReturns)

numObservations =
2000

numAssets =
100
```

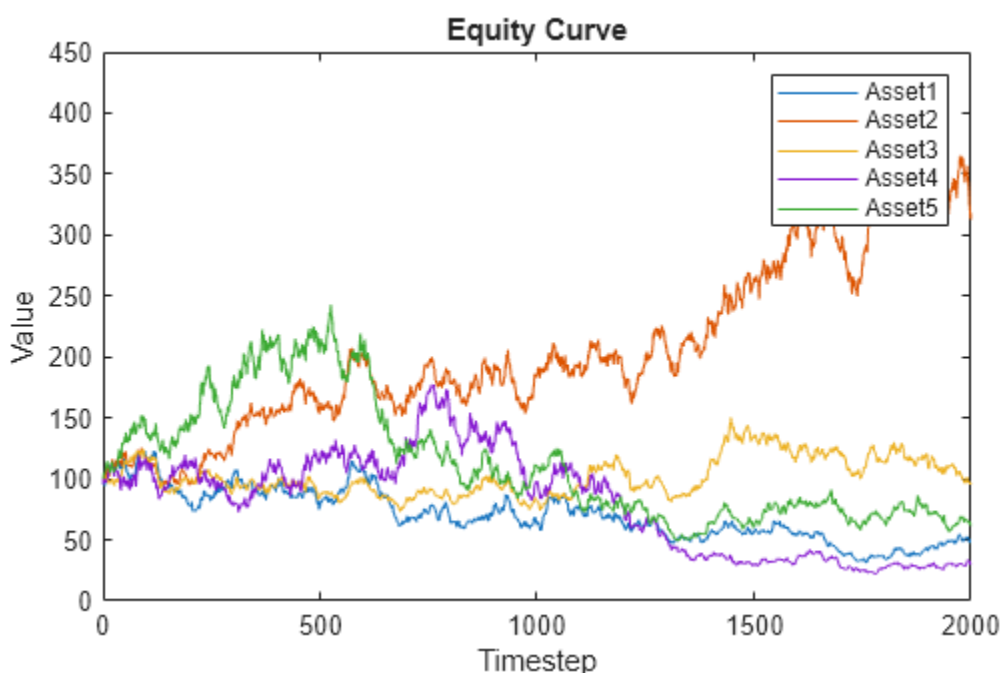
Create a timetable of asset prices from the asset returns.

```
% Convert the returns to prices.
pricesT = ret2tick(stockReturns,'StartPrice',100);

% Create a timetable.
rowTimes = datetime("today"):datetime("today")+numObservations;
pricesTT = table2timetable(pricesT,'RowTimes',rowTimes);
```

Visualize the equity curve for each stock. For this example, plot the first five stocks.

```
figure;
plot(0:2000,pricesTT{:,1:5})
xlabel('Timestep');
ylabel('Value');
title('Equity Curve');
legend(pricesTT.Properties.VariableNames(1:5));
```



Optimize Asset Allocation Using Covariance Denoising

Covariance denoising is a technique that you can use to reduce the noise and enhance the signal in a covariance matrix. First, the eigenvalues that are associated with noise are separated from the eigenvalues associated with signal. Then, the eigenvalues associated with noise are shrunk towards a target value. This technique helps improve the stability of the covariance matrix over time as well as its condition number.

The function `covarianceDenoising` computes the denoised estimate of the covariance matrix and returns as a second output the number of eigenvalues identified with signal. You use this number in the Optimize the Asset Allocation Using Factor Modeling on page 4-396 section to determine the number of factors that the factor model allocation uses.

This example uses the first 42 days (approximately 2 months) of the data set to select the initial portfolio allocations.

```
% Warm-up period
warmupPeriod = 42;

% No current weights (100% cash position)
w0 = zeros(1,numAssets);

% Warm-up partition of prices timetable
warmupTT = pricesTT(1:warmupPeriod,:);
```

Compute the maximum return portfolio subject to a target risk of 0.008 using the denoised covariance estimate.

```
% Compute the weights with denoised strategy.
wDenoised_initial = denoising(w0,warmupTT);
```

Check for asset allocations that are over 5% to identify assets with large investment weights.

```
percentage = 0.05;
AssetName = pricesTT.Properties.VariableNames(...
    wDenoised_initial>=percentage)';
Weight = wDenoised_initial(wDenoised_initial>=percentage);
T1 = table(AssetName,Weight)
```

```
T1=5x2 table
    AssetName    Weight
    _____    _____
    {'Asset6' }    0.066016
    {'Asset47'}    0.10991
    {'Asset50'}    0.24653
    {'Asset75'}    0.11752
    {'Asset94'}    0.31708
```

Optimize Asset Allocation Using Factor Modeling

For factor modeling, you can use statistical factors extracted from the asset return series. In this example, PCA is used to extract these factors [1 on page 4-399]. You can then use this factor model to solve the portfolio optimization problem.

With a factor model, n asset returns can be expressed as a linear combination of k factor returns, $r_a = \mu_a + F r_f + \varepsilon_a$, where $k \ll p$. In the mean-variance framework, portfolio risk is

$$\text{Var}(R_p) = \text{Var}(r_a^T w_a) = \text{Var}((\mu_a + F r_f + \varepsilon_a)^T w_a) = w_a^T (F \Sigma_f F^T + D) w_a,$$

where:

- R_p is the portfolio return (a scalar).
- r_a is the asset returns.
- μ_a is the mean of asset returns.
- F is the factor loading, with dimension $n \times k$.
- r_f is the factor return.
- ε_a is the idiosyncratic return related to each asset.
- w_a is the asset weight.
- Σ_f is the covariance of factor returns.
- D is the variance of idiosyncratic returns.

The parameters r_a , w_a , μ_a and ε_a are $n \times 1$ column vectors, r_f and w_f are $k \times 1$ column vectors, and Σ_k and D are a $k \times k$ and a $n \times n$ matrices, respectively.

Therefore, the mean-variance optimization problem is formulated as

$$\begin{aligned} \max_{w_a} \quad & \mu_a^T w_a \\ \text{s.t.} \quad & w_a^T (F \Sigma_f F^T + D) w_a \leq \tau, \\ & \sum_{a \in A} w_a = 1, \\ & 0 \leq w_a \leq 1. \end{aligned}$$

In the dimensional space formed by n asset returns, PCA finds the k directions that capture the most important variations in the returns. Usually, k is less than n . Therefore, by using PCA, you can decompose the n asset returns into k directions that are interpreted as factor loadings. The scores from the decomposition are interpreted as the factor returns. For more information, see `pca` (Statistics and Machine Learning Toolbox™). In this example, the factor model uses $k = \text{nFactors}$, where `covarianceDenoising` determines the `numFactors`.

Compute the maximum return portfolio subject to a target risk of 0.008 using the factor model covariance estimate. For details on how to obtain the weights allocation using factor modeling, see “Portfolio Optimization Using Factor Models” on page 4-231.

% Compute the weights with denoised strategy.

```
userData.numFactors = [];
[wFactorModel_initial,userData] = factorModeling(w0,warmupTT, ...
    userData);
```

Check for asset allocations that are over 5% to show assets with large investment weights.

```
percentage = 0.05;
AssetName = pricesTT.Properties.VariableNames( ...
    wFactorModel_initial>=percentage);
Weight = wFactorModel_initial(wFactorModel_initial>=percentage);
T2 = table(AssetName,Weight)
```

```
T2=6×2 table
    AssetName    Weight
    _____    _____
    {'Asset6' }    0.075366
    {'Asset35'}    0.069395
    {'Asset47'}    0.10676
    {'Asset50'}    0.21628
    {'Asset75'}    0.12423
    {'Asset94'}    0.3068
```

The assets with large investment weights are almost the same for both investment strategies. Asset35 is the only asset that appears in one table, namely in the factor model strategy, and not the other. Even the weights of the assets are similar.

Backtesting

Use `backtestStrategy` to create strategy objects for the two investment strategies. Compare the denoising strategy (`strat1`) against the factor model strategy (`strat2`) using backtesting.

% Rebalance approximately every month

```
rebalFreq = 21;
```

*% Set the rolling lookback window to be at least 2 months and at
% most 6 months.*

```
lookback = [42 126];
```

*% Use a fixed transaction cost (buy and sell costs are both 0.5%
% of amount traded).*

```
transactionsFixed = 0.005;
```

% Strategies

```

strat1 = backtestStrategy('Factor Modeling', @factorModeling, ...
    UserData=userData, ...
    RebalanceFrequency=rebalFreq, ...
    LookbackWindow=lookback, ...
    TransactionCosts=transactionsFixed, ...
    InitialWeights=wFactorModel_initial);

strat2 = backtestStrategy('Denoising', @denoising, ...
    RebalanceFrequency=rebalFreq, ...
    LookbackWindow=lookback, ...
    TransactionCosts=transactionsFixed, ...
    InitialWeights=wDenoised_initial);

```

```

% Aggregate the strategy objects into an array.
strategies = [strat1, strat2];

```

Create a `backtestEngine` object for the strategies, run the backtest using `runBacktest`, and generate a report using `summary`.

```

% Create the backtesting engine object.
backtester = backtestEngine(strategies);

% Run the backtest.
backtester = runBacktest(backtester, pricesTT, 'Start', warmupPeriod);

% Generate a summary table of strategies performance.
summary(backtester)

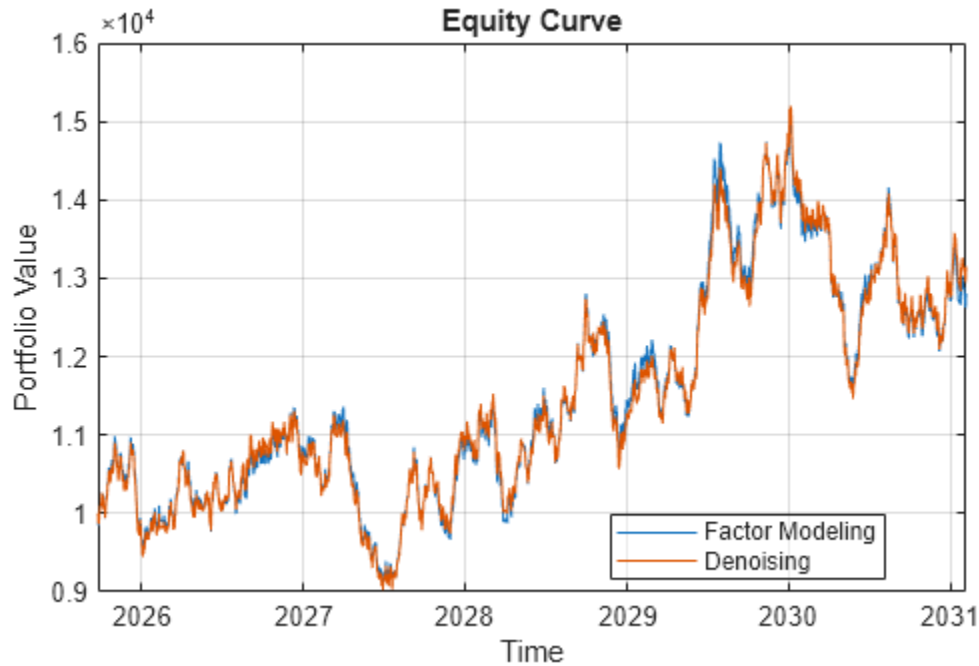
```

ans=9×2 table

	Factor_Modeling	Denoising
TotalReturn	0.28079	0.31501
SharpeRatio	0.018583	0.020462
Volatility	0.0089592	0.0086696
AverageTurnover	0.014646	0.013843
MaxTurnover	0.56184	0.58224
AverageReturn	0.00016644	0.00017736
MaxDrawdown	0.23384	0.24536
AverageBuyCost	0.84447	0.79536
AverageSellCost	0.84447	0.79536

Use `equityCurve` to plot the equity curve to compare the performance of both strategies.

```
equityCurve(backtester)
```



The performance of both strategies is similar, although not identical. This similarity is because the factor model strategy uses the number of factors identified by `covarianceDenoising` to select the number of principal components. Factor model strategies are usually not implemented this way, but rather the number of factors is a fixed parameter that is chosen *a priori*.

In this example, the number of factors that is most frequently identified is 1.

```
% Count the number of different factors
categoricalNumFactors = ...
    categorical(backtester.Strategies(1).UserData.numFactors);
[N,uniqueFactors] = histcounts(categoricalNumFactors);
factorFrequency = table(uniqueFactors',N', ...
    'VariableNames',{'NumFactors','Frequency'})
```

```
factorFrequency=3x2 table
    NumFactors    Frequency
    _____    _____
    {'1' }        81
    {'2' }        12
    {'17'}         1
```

Therefore, you can run the factor modeling strategy using 1 as the number of factors instead of using `covarianceDenoising` to identify the number of factors at each rebalancing period.

Reference

- 1 Meucci, Attilio. "Modeling the Market." In *Risk and Asset Allocation*, by Attilio Meucci, 101-66. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009.

Local Functions

```

function [new_weights,userData] =...
    factorModeling(~,pricesTT,userData)
% Compute minimum variance portfolio using traditional covariance estimate.

% Compute returns from prices timetable.
assetReturns = tick2ret(pricesTT);

% Compute the number of factors identified using covariance denoising.
[~,numFactors] = covarianceDenoising(assetReturns.Variables);
userData.numFactors = [userData.numFactors; numFactors];

% Compute the covariance using the factors model
%   SigmaFactorModel = F*Sigma_f*F' + D
%   r_a = mu_a + F*r_f + epsilon_a
[factorLoading,factorRetn,~,~,~,factorMean] = ...
    pca(assetReturns.Variables,'NumComponents',numFactors);
covFactor = cov(factorRetn);
retnHat = factorRetn*factorLoading' + factorMean;
unexplainedRetn = assetReturns.Variables - retnHat;
unexplainedCovar = diag(cov(unexplainedRetn));
D = diag(unexplainedCovar);

% Define the mean and covariance of the returns.
mu = mean(assetReturns.Variables);
Sigma = factorLoading*covFactor*factorLoading' + D;

% Create the portfolio problem.
p = Portfolio(AssetMean=mu,AssetCovar=Sigma);
% Specify long-only, fully-invested constraints
p = setDefaultConstraints(p);

% Compute the maximum return portfolio subject to the target risk.
targetRisk = 0.008;
new_weights = estimateFrontierByRisk(p,targetRisk);
end

function new_weights = denoising(~, pricesTT)
% Compute minimum variance portfolio using covariance denoising.

% Compute the returns from the prices timetable.
assetReturns = tick2ret(pricesTT);
mu = mean(assetReturns.Variables);
Sigma = covarianceDenoising(assetReturns.Variables);

% Create the portfolio problem.
p = Portfolio(AssetMean=mu,AssetCovar=Sigma);
% Long-only fully invested constraints
p = setDefaultConstraints(p);

% Compute maximum return portfolio subject to the target risk.
targetRisk = 0.008;

```



```
new_weights = estimateFrontierByRisk(p,targetRisk);  
end
```

See Also

Portfolio | covarianceShrinkage | covarianceDenoising

Related Examples

- “Portfolio Optimization Against a Benchmark” on page 4-202

More About

- “Comparison of Methods for Covariance Estimation” on page 4-138

Mixed-Integer Mean-Variance Portfolio Optimization Problem

This example shows how to solve a mean-variance portfolio optimization problem with constraints in the number of selected assets or conditional (semicontinuous) bounds. To solve this problem, you can use a `Portfolio` object along with different mixed integer nonlinear programming (MINLP) solvers.

Mean-Variance Portfolio

Load the returns data in `CAPMuniverse.mat`. Then, create a mean-variance `Portfolio` object with default constraints and a long-only portfolio whose weights sum to 1. For this example, you can define the feasible region of weights X as

$$X = \left\{ x \mid \sum_{i=1}^n x_i = 1, x_i \geq 0 \right\}.$$

```
% Load data
load CAPMuniverse.mat

% Create a mean-variance Portfolio object with default constraints.
p = Portfolio(AssetList=Assets(1:12));
p = estimateAssetMoments(p,Data(:,1:12));
p = setDefaultConstraints(p);
```

Include binary variables for this scenario by setting conditional (semicontinuous) bounds. Conditional bounds are those such that $x_i = 0$ or $x_i \geq \ell_i$. In this example, $\ell_i = 0.1$ for all assets.

```
% Set conditional bounds
condLB = 0.1;
condUB = 0.5;
p = setBounds(p,condLB,condUB,BoundType="conditional");
```

Use `estimateFrontier` to estimate a set of portfolios on the efficient frontier. The efficient frontier is a curve that shows the trade-off between the return and risk achieved by Pareto-optimal portfolios. For a given return level, the portfolio on the efficient frontier is the one that minimizes the risk while maintaining the desired return. Conversely, for a given risk level, the portfolio on the efficient frontier is the one that maximizes return while maintaining the desired risk level.

```
% Compute the efficient frontier.
pwgt = estimateFrontier(p)
```

```
pwgt = 12×10
```

0	0	0.1000	0.1253	0.1745	0.2236	0.2715	0.3327	0.4111	0.5
0	0	0	0	0	0	0	0	0	0.5
0	0	0	0	0	0	0	0	0	0.5
0.1350	0	0	0	0	0	0	0	0	0.5
0	0	0	0	0	0	0	0	0	0.5
0.1000	0.1450	0.1406	0.1910	0.2344	0.2778	0.3200	0.3726	0.4415	0.5
0.1000	0.1609	0.1642	0.2121	0.2415	0.2709	0.3085	0.2947	0.1474	0.5
0.2354	0.1875	0.1290	0	0	0	0	0	0	0.5
0	0	0	0	0	0	0	0	0	0.5
0.4296	0.4066	0.3662	0.3717	0.2496	0.1277	0	0	0	0.5
0	0.1000	0.1000	0.1000	0.1000	0.1000	0.1000	0	0	0.5
0	0	0	0	0	0	0	0	0	0.5

```

:

% Compute risk and returns of the portfolios on the efficient frontier.
[rsk,ret] = estimatePortMoments(p,pwgt)

rsk = 10×1

    0.0076
    0.0080
    0.0085
    0.0094
    0.0105
    0.0117
    0.0132
    0.0147
    0.0168
    0.0193

ret = 10×1

    0.0008
    0.0012
    0.0017
    0.0021
    0.0026
    0.0030
    0.0034
    0.0039
    0.0043
    0.0048

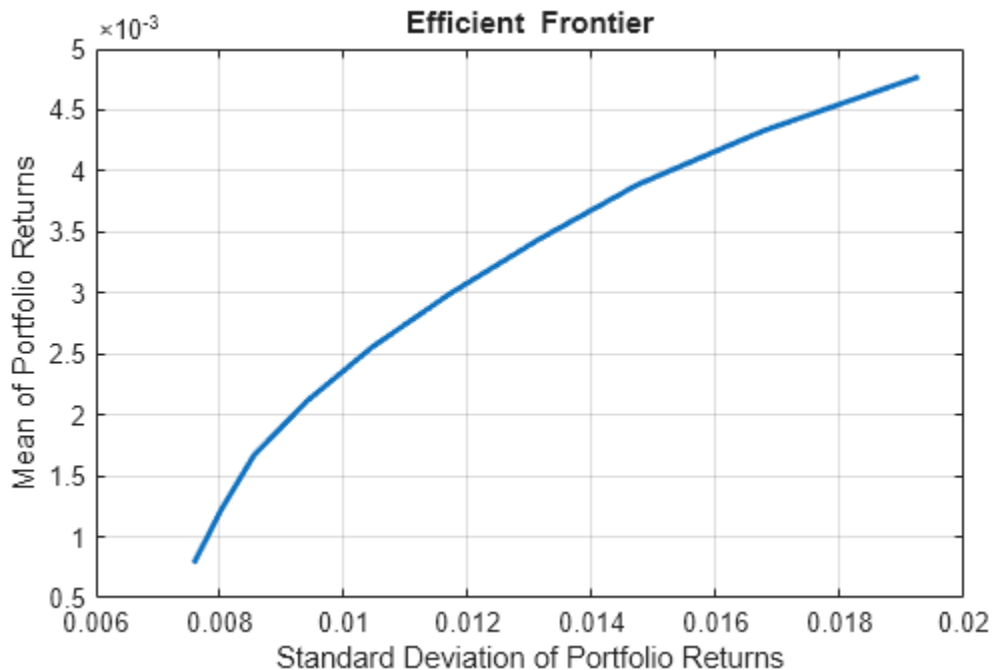
```

Plot the weights from the frontier estimation using `plotFrontier`. The resulting curve is piece-wise concave with vertical jumps (discontinuities) between the concave intervals.

```

% Plot the efficient frontier.
plotFrontier(p,pwgt)

```



Changing MINLP Solvers

In the previous section, you use the default solver for `estimateFrontier`. However, you can solve mixed-integer portfolio problems using any of the three algorithms supported by `setSolverMINLP`: `OuterApproximation`, `ExtendedCP`, and `TrustRegionCP`. Furthermore, the `OuterApproximation` algorithm accepts an additional name-value argument (`ExtendedFormulation`) for `Portfolio` problems, which reformulates problems with quadratic functions to work in an extended space that usually decreases the computation time. All algorithms, including the extended formulation variation of the `OuterApproximation` algorithm, return the same values within numerical accuracy. The available solvers are:

- `OuterApproximation` — The default algorithm, which is robust and usually faster than `ExtendedCP`
- `OuterApproximation` with `ExtendedFormulation` set to `true` — A robust algorithm that is usually faster than other algorithms, but only supported for `Portfolio` object problems
- `ExtendedCP` — The most robust solver, but usually the slowest
- `TrustRegionCP` — The fastest algorithm, but one that is less robust and may provide suboptimal solutions

For more information on solvers for mixed-integer portfolio problems, see “Choose MINLP Solvers for Portfolio Problems” on page 4-140.

To change the MINLP solvers, use `setSolverMINLP`.

```
% Select the extended formulation version of 'OuterApproximation' as the
% solver.
p_EOA = setSolverMINLP(p,'OuterApproximation',...
    ExtendedFormulation=true);
pwgt_EOA = estimateFrontier(p_EOA);
[rskEOA,retEOA] = estimatePortMoments(p_EOA,pwgt_EOA);
```

```
% Select 'TrustRegionCP' as the solver.
p_TR = setSolverMINLP(p, 'TrustRegionCP');
pwgt_TR = estimateFrontier(p_TR);
[rskTR, retTR] = estimatePortMoments(p_TR, pwgt_TR);

% Select 'ExtendedCP' as the solver using 'midway' cuts as 'CutGeneration'.
p_ECP = setSolverMINLP(p, 'ExtendedCP', 'CutGeneration', 'midway');
pwgt_ECP = estimateFrontier(p_ECP);
[rskECP, retECP] = estimatePortMoments(p_ECP, pwgt_ECP);
```

Compare the returns and risks obtained by the portfolios on the efficient frontier from the different solvers. These are same within a numerical accuracy, where the absolute difference is $\leq 10^{-4}$.

```
retTable = table(ret, retEOA, retTR, retECP, ...
    'VariableNames', {'OA', 'EOA', 'TR', 'ECP'})
```

```
retTable=10×4 table
```

OA	EOA	TR	ECP
0.00078336	0.00078336	0.00078334	0.00078348
0.0012267	0.0012267	0.0012267	0.0012268
0.00167	0.00167	0.00167	0.0016701
0.0021133	0.0021133	0.0021133	0.0021134
0.0025566	0.0025566	0.0025566	0.0025567
0.0029999	0.0029999	0.0029999	0.003
0.0034432	0.0034432	0.0034432	0.0034433
0.0038865	0.0038865	0.0038865	0.0038866
0.0043298	0.0043298	0.0043298	0.0043299
0.0047731	0.0047731	0.0047731	0.0047731

```
rskTable = table(rsk, rskEOA, rskTR, rskECP, ...
    'VariableNames', {'OA', 'EOA', 'TR', 'ECP'})
```

```
rskTable=10×4 table
```

OA	EOA	TR	ECP
0.0075778	0.0075778	0.0075778	0.0075778
0.0080234	0.0080234	0.0080235	0.0080236
0.0085488	0.0085488	0.0085488	0.008549
0.0094024	0.0094024	0.009405	0.0094025
0.010456	0.010456	0.010456	0.010456
0.011727	0.011727	0.011727	0.011728
0.013155	0.013155	0.013155	0.013155
0.014729	0.014729	0.014729	0.014729
0.016764	0.016764	0.016764	0.016764
0.019273	0.019273	0.019273	0.019273

```
% Compare the risks from the different OuterApproximation formulations.
norm(rskTable.OA-rskTable.EOA, Inf) <= 1e-4
```

```
ans = logical  
    1
```

See Also

[estimatePortSharpeRatio](#) | [estimateFrontier](#) | [estimateFrontierByReturn](#) | [estimateFrontierByRisk](#)

Related Examples

- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Portfolio Optimization Against a Benchmark” on page 4-202
- “Solve Tracking Error Portfolio Problems” on page 4-343
- “Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty” on page 4-351
- “Risk Parity or Budgeting with Constraints” on page 4-357

More About

- “Choose MINLP Solvers for Portfolio Problems” on page 4-140

Deep Reinforcement Learning for Optimal Trade Execution

This example shows how to use the Reinforcement Learning Toolbox™ and Deep Learning Toolbox™ to design agents for optimal trade execution.

Introduction

Optimal trade execution seeks to minimize trading costs when selling or buying a set number of stock shares over a certain time horizon within the trading day. Optimization is necessary because executing trades too quickly results in suboptimal prices due to market impact, while executing trades too slowly results in exposure to the risk of adverse price changes or the inability to execute all trades within the time horizon. This optimization affects nearly all trading strategies and portfolio management practices, since minimizing trading costs is directly linked to profitability that is related to buying or selling decisions. Also, brokers compete to provide better trade order execution, and many jurisdictions legally require brokers to provide the best execution for their clients. Early studies by Bertsimas and Lo [1 on page 4-440] and Almgren and Chriss [2 on page 4-440] derived analytical solutions for the optimal trade execution problem by assuming a model for the underlying prices. Later, Nevmyvaka et al. [3 on page 4-440] demonstrated that the use of reinforcement learning (RL) for optimal execution did not require making assumptions about the market micro-structure. More recent studies use deep reinforcement learning, which combines reinforcement learning (RL) with deep learning, for optimal trade execution (Ning *et al.* [4 on page 4-440], Lin and Beling [5 on page 4-440, 6 on page 4-440]). This approach overcomes the limitations of Q-learning as noted by Nevmyvaka et al. [3 on page 4-440].

This example uses deep reinforcement learning to design and train two deep Q-network (DQN) agents for optimal trade execution. One DQN agent is for sell trades and the other agent is for buy trades.

Data

This example uses the intraday trading data on Intel Corporation stock (INTC) provided by LOBSTER [7 on page 4-441] and included with Financial Toolbox™. While the reinforcement learning for optimal execution literature typically uses trading data over one year or more to train and test RL agents [3 on page 4-440, 4 on page 4-440, 5 on page 4-440, 6 on page 4-440], in this proof-of-concept example you use a shorter data set, which you can expand for further study. The intraday trading data on the Intel stock (INTC_2012-06-21_34200000_57600000_orderbook_5.csv) contains limit order book data over 6.5 hours, including bid and ask prices as well as corresponding shares for 5 levels. This data is sampled at 5 second intervals to create 390 one minute trading horizons, such that each trading horizon contains 12 steps at five second intervals. Out of these 390 horizons, you use 293 horizons (about 75 percent) to train the agents and the remaining 97 horizons (about 25 percent) to test the agents.

Baseline Trade Execution Algorithm

An agent's performance must be compared with a baseline trade execution algorithm. For this example, the baseline trade execution algorithm is the time-weighted average price (TWAP) policy. Under this policy, the total trading shares are divided equally over the trading horizon, so that the same number of shares are traded at each time step. In contrast, the original paper by Nevmyvaka *et al.* [3 on page 4-440] used a "submit and leave" policy as the baseline, where a limit order is placed at a fixed price for the entire trading shares for the entire horizon, and any remaining unfilled shares are traded with a market order at the last step. However, the TWAP baseline was adopted in later studies [4 on page 4-440, 5 on page 4-440, 6 on page 4-440], since it has become more popular in

recent years with increasing use of algorithmic trading, and it is also a highly effective strategy that is thought to be optimal when the price movement follows a Brownian motion [4 on page 4-440,5 on page 4-440,6 on page 4-440].

Agents

Different types of agents are used in the reinforcement learning for optimal execution studies, including Q-learning [3 on page 4-440], DQN [4 on page 4-440,5 on page 4-440], and PPO [6 on page 4-440] agents. In this example, you design and train DQN agents using the Reinforcement Learning Designer (Reinforcement Learning Toolbox) after creating separate training environments for sell trades and buy trades. These DQN agents only have a critic without an actor, and they use the default setting of `UseDoubleDQN` set to `true`, while the `DiscountFactor` is set to 1 due to the short trading horizon [4 on page 4-440,5 on page 4-440,6 on page 4-440]. In addition, the critic network of these agents include a long short-term memory (LSTM) network because the trading data is sequential in time, and LSTM networks are effective in designing agents for optimal execution [6 on page 4-440]. For more information on setting options for DQN agents, see `rldQNAgentOptions` (Reinforcement Learning Toolbox).

Observation Space

The observation space defines which information (or feature) to provide to the agents at each step. The agents use this observation to decide the next action, based on the policy learned and reinforced by the rewards received after taking the previous actions. Designing a good observation space for the agents is an area of active research in optimal execution studies. You can experiment with different observation spaces, using variables such as the bid-ask spread, the bid or ask volume, price trends, or even the entire limit order book prices and shares [6 on page 4-440]. This example uses the following observation variables:

- Remaining shares to be traded by the agent in the current trading horizon
- Remaining time intervals (steps) left in the current trading horizon
- Cumulative implementation shortfall (IS) of agent until the previous step
- Current limit order book price divergence from arrival price

The first two observation variables (remaining shares to be traded and remaining time in horizon) are called "private" variables by Nevmyvaka *et al.* [3 on page 4-440], as they only apply to the specific situation of the trading agents, and they do not apply to other market participants. As the agent trained by Nevmyvaka *et al.* [3 on page 4-440] using only "private" variables was able to outperform the "submit and leave" baseline policy, the "private" variables are also used in the subsequent studies [4 on page 4-440,5 on page 4-440,6 on page 4-440].

The third variable, cumulative IS, is a measure of trading cost that the agents seek to minimize. As the name "shortfall" indicates, a lower implementation shortfall implies better trade execution. Technically, Implementation Shortfall should include the opportunity cost of failing to execute all shares within the trading horizon. In this example, as with many of the previously mentioned studies, the Implementation Shortfall is computed under the ending inventory constraint:

- For a sell trade, the constraint is to have zero ending inventory, assuming successful liquidation of all shares.
- For a buy trade, the constraint is to have full ending inventory, assuming successful purchase of all shares.

Under this ending inventory constraint, Implementation Shortfall is computed as follows:

- For a sell trade:
Implementation Shortfall = Arrival Price $\times$ Traded Volume – Executed Price $\times$ Traded Volume
- For a buy trade:
Implementation Shortfall = Executed Price $\times$ Traded Volume – Arrival Price $\times$ Traded Volume

The arrival price is the first bid or ask price observed at the beginning of the trade horizon. If Implementation Shortfall is positive, it implies that the average executed price was worse than the arrival price, while a negative Implementation Shortfall implies that the average executed price was better than the arrival price. The cumulative Implementation Shortfall from the beginning of the trade horizon until the previous step reflects the trading cost incurred by the agent in the past, and it serves as the third observation variable.

The last observation variable is the current price divergence from arrival price. This variable reflects the current market condition and it is measured as the difference between the arrival price and the average price of the first two levels of the current limit order book (LOB).

- For a sell trade:
Price Divergence = Average Bid Prices of First 2 Levels of Current LOB – Arrival Price
- For a buy trade:
Price Divergence = Arrival Price – Average Ask Prices of First 2 Levels of Current LOB

A positive price divergence implies more favorable current trading conditions than the arrival price, and a negative divergence less favorable conditions.

Action Space

The action space is defined as the possible number of shares traded at each step. For a TWAP policy, the only possible action at each step is to trade a constant number of shares computed as the total trading shares divided by the number of steps in the horizon. Meanwhile, the agents in this example choose from 39 possible actions that are mostly evenly spaced, ranging from trading 0 shares to trading twice the number of shares as the corresponding TWAP trade.

For both TWAP trades and agent trades, you enforce ending inventory constraints so that if the target ending inventory is not met by the last step of the horizon, the action for the last step is to:

- Sell any remaining shares in the inventory for a Sell trade.
- Buy any missing shares in the inventory for a Buy trade.

Also, you place limits on the agent actions to take advantage of price divergence and to prevent selling more than the available shares in the inventory or buying more than the target number of shares.

Reward

The reward is another important aspect of agent design, as agents learn their policies at each step using the rewards received after taking the previous actions. Research on reward design shows wide variations in optimal execution, and you can experiment with different rewards. Since the agent performance is measured against the baseline using Implementation Shortfall, many studies use Implementation Shortfalls directly in the rewards, while others use related quantities. Some studies use the proceeds from the trade as the reward [3 on page 4-440], while others also impose a penalty for trading many shares too quickly [4 on page 4-440]. Another study uses an elaborately shaped reward algorithm that is designed to use the TWAP policy most of the time, while encouraging the agent to deviate from the TWAP policy only when it is advantageous to do so [5 on page 4-440]. If

there is enough data, the reward may be given sparsely, only at the end of each horizon rather than at each step [6 on page 4-440]. In this example, the data size is small, so you give the reward at each step and design it so that the reward at each step approximates the expected reward at the end of the horizon.

Specifically, you use a reward that compares the Implementation Shortfall of the TWAP baseline against that of the agent at each step, while also taking into account the penalty for the approximate Implementation Shortfall of the remaining unexecuted shares based on the current limit order book. The reward is also scaled based on the current time relative to the end of the horizon.

$$\text{Reward}(t) = \left[(\text{IS}_{\text{TWAP}}(0, t) + \text{IS}_{\text{TWAP, Penalty}}(t, T)) - (\text{IS}_{\text{Agent}}(0, t) + \text{IS}_{\text{Agent, Penalty}}(t, T)) \right] \times \frac{t}{T}$$

$\text{Reward}(t)$ is the reward at time step t

$\text{IS}_{\text{TWAP}}(0, t)$ is the Implementation Shortfall of TWAP from the beginning of the horizon until time step t

$\text{IS}_{\text{TWAP, Penalty}}(t, T)$ is the approximate Implementation Shortfall of TWAP from time step t to the end of the horizon at time step T

$\text{IS}_{\text{Agent}}(0, t)$ is the Implementation Shortfall of the agent from the beginning of horizon until time step t

$\text{IS}_{\text{Agent, Penalty}}(t, T)$ is the approximate Implementation Shortfall of the agent from time step t to the end of the horizon at time step T

Since the agent performance is ultimately measured against the baseline using Implementation Shortfall at the end of the horizon at time step T , and the approximate reward becomes more accurate as t approaches T , $\text{Reward}(t)$ has a scaling factor $\frac{t}{T}$ so that its magnitude increases as t approaches T . At time step T , the reward becomes $\text{Reward}(T)$.

$$\text{Reward}(T) = \text{IS}_{\text{TWAP}}(0, T) - \text{IS}_{\text{Agent}}(0, T)$$

Performance Analysis

You measure agent performance against the TWAP baseline using Implementation Shortfall (IS), by computing the following for all horizons:

- IS total outperformance of agent over TWAP — Sum of $\text{IS}_{\text{TWAP}}(0, T) - \text{IS}_{\text{Agent}}(0, T)$
- IS total-gain-to-loss ratio (TGLR) — Ratio of total positive over total negative $\text{IS}_{\text{TWAP}}(0, T) - \text{IS}_{\text{Agent}}(0, T)$ magnitudes
- IS gain-to-loss ratio (GLR) — Ratio of mean positive over mean negative $\text{IS}_{\text{TWAP}}(0, T) - \text{IS}_{\text{Agent}}(0, T)$ magnitudes

Workflow Organization

The workflow in this example is:

- **Load and Prepare Data** on page 4-411
- - LOBSTER Message Data — Contains time stamps

- - LOBSTER Limit Order Book Data — Contains bid and ask prices and corresponding shares
- **Optimal Execution for Sell Trades** on page 4-412
- - TWAP Baseline
- - Train the DQN Agent
- - Test the DQN Agent
- - Summary of Selling Results
- **Optimal Execution for Buy Trades** on page 4-436
- **Local Functions** on page 4-441
- - executeTWAPTrades — Executes TWAP Baseline trades and computes IS
- - RL_OptimalExecution_LOB_ResetFcn — Reset function for training and testing environments
- - RL_OptimalExecution_LOB_StepFcn — Step function for training and testing environments
- - tradeLOB — Executes trades on current limit order book
- - endingInventoryIS — Computes IS with ending inventory constraints
- - ISTGLR — Computes total-gain-to-loss ratio and gain-to-loss ratio for IS

Load and Prepare Message Data

Extract the LOBSTER data files from the ZIP file included with Financial Toolbox™. The expanded files include two CSV files of data and the text file LOBSTER_SampleFiles_ReadMe.txt.

```
unzip('LOBSTER_SampleFile_INTC_2012-06-21_5.zip');
```

The limit order book file contains the bid and ask prices and corresponding shares for five levels, while the message file contains the description of the limit order book data including time stamps.

```
LOBFileName = 'INTC_2012-06-21_34200000_57600000_orderbook_5.csv'; % Limit Order Book Data file
MSGFileName = 'INTC_2012-06-21_34200000_57600000_message_5.csv'; % Message file (description of o
```

Load the message file and define the time at the start and end of trading hours as seconds after midnight.

```
TradeMessage = readmatrix(MSGFileName);
StartTradingSec = 9.5*60*60; % Starting time 9:30 in seconds after midnight
EndTradingSec = 16*60*60; % Ending time 16:00 in seconds after midnight
```

Define the trading interval as five seconds and pick message index values corresponding to the five-second intervals.

```
TradingIntervalSec = 5; % Trading interval length in seconds
IntervalBoundSec = (StartTradingSec:TradingIntervalSec:EndTradingSec)';
MessageSampleIdx = zeros(size(IntervalBoundSec));
k1 = 1;
for k2 = 1:size(TradeMessage,1)
    if TradeMessage(k2,1) >= IntervalBoundSec(k1)
        MessageSampleIdx(k1) = k2;
        k1 = k1+1;
    end
end
MessageSampleIdx(end) = length(TradeMessage);
```

Load and Prepare Limit Order Book Data

Load the limit order book data and define the number of levels in the limit order book.

```
LOB = readmatrix(LOBFileName);  
NumLevels = 5;
```

Since LOBSTER stores prices in units of 10,000 dollars, convert the prices in the limit order book data into dollars by dividing by 10000.

```
LOB(:,1:2:(4*NumLevels)) = LOB(:,1:2:(4*NumLevels))./10000;
```

Using the message index values obtained previously, sample the limit order book at 5 second intervals.

```
SampledBook = LOB(MessageSampleIdx(2:end),1:NumLevels*4);
```

Define a one minute (60 seconds) trading horizon and compute the total number of intervals over 6.5 hours of trading data.

```
TradingHorizon = 1*60;  
TotalNumIntervals = length(MessageSampleIdx(2:end));  
TotalNumHorizons = 6.5*60*60/TradingHorizon;
```

The number of intervals per horizon is computed by dividing the total number of intervals by the total number of horizons.

```
NumIntervalsPerHorizon = round(TotalNumIntervals/TotalNumHorizons);
```

Allocate 75 percent of data for training and 25 percent for testing.

```
NumTrainingHorizons = round(TotalNumHorizons*0.75);  
NumTestingHorizons = TotalNumHorizons - NumTrainingHorizons;  
NumTrainingSteps = NumTrainingHorizons*NumIntervalsPerHorizon;  
NumTestingSteps = NumTestingHorizons*NumIntervalsPerHorizon;  
TrainingLOB = SampledBook(1:NumTrainingSteps,:);  
TestingLOB = SampledBook(NumTrainingSteps+1:end,:);
```

Optimal Execution for Sell Trades

These sections compose the workflow for designing a DQN agent for the optimal execution of sell trades:

- 1 Define Settings for Sell Trades on page 4-413
- 2 TWAP Baseline Analysis on Training Data on page 4-413
- 3 Set Up RL Training Environment for Sell Trades on page 4-414
- 4 Validate Training Environment Reset and Step Functions for Sell Trades on page 4-415
- 5 Create and Train Agent for Sell Trades on page 4-417
- 6 Compute IS Using Trained Agent and Training Data for Sell Trades on page 4-422
- 7 Plot Training Results for Sell Trades on page 4-423
- 8 Display Summary of Training Results for Sell Trades on page 4-427
- 9 TWAP Baseline Analysis on Testing Data on page 4-428
- 10 Set Up RL Testing Environment for Sell Trades on page 4-429

- 11 Validate Testing Environment Reset and Step Functions for Sell Trades on page 4-430
- 12 Compute IS Using Trained Agent and Testing Data for Sell Trades on page 4-431
- 13 Plot Testing Results for Sell Trades on page 4-432
- 14 Display Summary of Training and Testing Results for Sell Trades on page 4-435

Define Settings for Sell Trades

To indicate sell trades, set the `BuyTrade` logical flag to `false`.

```
BuyTrade = false;
```

Define the total number of shares to sell over the horizon.

```
TotalTradingShares_Sell = 2738;
```

TWAP Baseline Analysis on Training Data

Execute TWAP trades and compute Implementation Shortfall (IS) using the `executeTWAPTrades` function, which uses the `tradeLOB` function to execute the trades and the `endingInventoryIS` function to compute IS. These functions are in the Local Functions on page 4-441 section.

```
[IS_TWAP_Horizon_Train_Sell, IS_TWAP_Step_Train_Sell, ...
    InventoryShares_Step_TWAP_Train_Sell, TradingShares_Step_TWAP_Train_Sell] = ...
    executeTWAPTrades(TrainingLOB, NumLevels, TotalTradingShares_Sell, ...
        NumIntervalsPerHorizon, NumTrainingHorizons, BuyTrade);
```

Plot the implementation shortfall, executed shares, and inventory shares for the TWAP baseline for each step using the training data.

```
figure
subplot(3,1,1)
bar(1:NumTrainingSteps, IS_TWAP_Step_Train_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
ylabel("Implementation Shortfall")
title("Implementation Shortfall (TWAP, Training, Sell)")
xlim([0 300])

subplot(3,1,2)
bar(1:NumTrainingSteps, TradingShares_Step_TWAP_Train_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
ylabel("Executed Shares")
title("Shares Traded at Each Step (TWAP, Training, Sell)")
xlim([0 300])
ylim([0 2000])

subplot(3,1,3)
bar(1:NumTrainingSteps, InventoryShares_Step_TWAP_Train_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
ylabel("Inventory Shares")
title("Inventory Shares (TWAP, Training, Sell)")
xlim([0 300])
```



The TWAP executed shares in the middle panel have a flat profile, since the same number of shares are traded at each step. Also, the inventory shares in the bottom panel have a nearly perfect triangular sawtooth pattern, with the number of inventory shares declining linearly at each horizon as the shares are sold. The IS in the top panel sometimes declines to negative values (implying better executions than arrival price) for some horizons, while it increases to positive values (implying worse executions than arrival price) for other horizons.

Set Up RL Training Environment for Sell Trades

The RL environment requires observation and action information, reset function, and a step function.

Define the observation information for the four observation variables.

```
ObservationDimension = [1 4];
ObservationInfo_Train_Sell = rlNumericSpec(ObservationDimension);
ObservationInfo_Train_Sell.Name = 'Observation Data';
ObservationInfo_Train_Sell.Description = 'RemainingSharesToTrade, RemainingIntervals, IS_Agent, I';
ObservationInfo_Train_Sell.LowerLimit = -inf;
ObservationInfo_Train_Sell.UpperLimit = inf;
```

Define the environment constants for the training data and the reset function. The function definition for `RL_OptimalExecution_LOB_ResetFcn` is in the Local Functions on page 4-441 section.

```
EnvConstants_Train_Sell.SampledBook = TrainingLOB;
EnvConstants_Train_Sell.BuyTrade = BuyTrade;
EnvConstants_Train_Sell.TotalTradingShares = TotalTradingShares_Sell;
EnvConstants_Train_Sell.NumIntervalsPerHorizon = NumIntervalsPerHorizon;
EnvConstants_Train_Sell.NumHorizons = NumTrainingHorizons;
EnvConstants_Train_Sell.NumSteps = NumTrainingSteps;
```

```
EnvConstants_Train_Sell.NumLevels = NumLevels;
EnvConstants_Train_Sell.IS_TWAP = IS_TWAP_Step_Train_Sell;
```

```
ResetFunction_Sell = @( ) RL_OptimalExecution_LOB_ResetFcn(EnvConstants_Train_Sell);
```

Define the action information as the number of shares to be traded at each step, ranging from zero shares to twice the number of shares as in TWAP.

```
NumActions = 39;
LowerActionRange = linspace(0,TradingShares_Step_TWAP_Train_Sell(1), round(NumActions/2));
ActionRange = round([LowerActionRange(1:end-1) ...
    linspace(TradingShares_Step_TWAP_Train_Sell(1), ...
    TradingShares_Step_TWAP_Train_Sell(1)*2,round(NumActions/2))] );
```

```
ActionInfo_Sell = rlFiniteSetSpec(ActionRange);
ActionInfo_Sell.Name = 'Trading Action (Number of Shares)';
ActionInfo_Sell.Description = 'Number of Shares to be Traded at Each Time Step';
```

Define the step function, which computes the rewards and updates the observations at each step. The function definition for `RL_OptimalExecution_LOB_StepFcn` is in the Local Functions on page 4-441 section.

```
StepFunction_Train_Sell = @(Action,LoggedSignals) ...
    RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvConstants_Train_Sell);
```

Create the environment using the custom function handles defined in this section.

```
RL_OptimalExecution_Training_Environment_Sell = rlFunctionEnv(ObservationInfo_Train_Sell,ActionInfo_Sell,StepFunction_Train_Sell,ResetFunction_Sell);
```

```
Reset!
Reset!
```

```
RL_OptimalExecution_Training_Environment_Sell =
    rlFunctionEnv with properties:
```

```
    StepFcn: @(Action,LoggedSignals)RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvConstants_Train_Sell)
    ResetFcn: @( )RL_OptimalExecution_LOB_ResetFcn(EnvConstants_Train_Sell)
    Info: [1x1 struct]
```

Validate Training Environment Reset and Step Functions for Sell Trades

Before using the created environment, the best practice is to validate the reset and step functions for the environment. Call the reset and step functions to see if they produce reasonable results without errors.

Validate the reset function and the initial observation.

```
InitialObservation = reset(RL_OptimalExecution_Training_Environment_Sell);
```

```
Reset!
```

```
InitialObservation
```

```
InitialObservation = 1x4
```

```
    2738    12    0    0
```

Validate the step function by monitoring the logged signals and rewards while calling the step functions multiple times. Here, you can see the time interval index increasing at each step and the current inventory shares changing as the trades are executed at each step.

```
[~,Reward1,~,LoggedSignals1] = step(RL_OptimalExecution_Training_Environment_Sell,TradingShares_9,LoggedSignals1)
```

```
LoggedSignals1 = struct with fields:
    IntervalIdx: 1
    HorizonIdx: 1
    CurrentInventoryShares: 2624
    ArrivalPrice: 27.4600
    ExecutedShares: 114
    ExecutedPrices: 27.4600
    HorizonExecutedShares: [60×1 double]
    HorizonExecutedPrices: [60×1 double]
    IS_Agent: [3516×1 double]
    Reward: [3516×1 double]
    NumIntervalsPerHorizon: 12
    NumHorizons: 293
    NumLevels: 5
```

Reward1

```
Reward1 =
0
```

```
[~,Reward2,~,LoggedSignals2] = step(RL_OptimalExecution_Training_Environment_Sell,10);
LoggedSignals2
```

```
LoggedSignals2 = struct with fields:
    IntervalIdx: 2
    HorizonIdx: 1
    CurrentInventoryShares: 2614
    ArrivalPrice: 27.4600
    ExecutedShares: 10
    ExecutedPrices: 27.4900
    HorizonExecutedShares: [60×1 double]
    HorizonExecutedPrices: [60×1 double]
    IS_Agent: [3516×1 double]
    Reward: [3516×1 double]
    NumIntervalsPerHorizon: 12
    NumHorizons: 293
    NumLevels: 5
```

Reward2

```
Reward2 =
0.6150
```

```
[~,Reward3,~,LoggedSignals3] = step(RL_OptimalExecution_Training_Environment_Sell,35);
LoggedSignals3
```

```
LoggedSignals3 = struct with fields:
    IntervalIdx: 3
    HorizonIdx: 1
    CurrentInventoryShares: 2579
```



```
        ArrivalPrice: 27.4600
        ExecutedShares: 35
        ExecutedPrices: 27.4900
    HorizonExecutedShares: [60×1 double]
    HorizonExecutedPrices: [60×1 double]
        IS_Agent: [3516×1 double]
        Reward: [3516×1 double]
    NumIntervalsPerHorizon: 12
        NumHorizons: 293
        NumLevels: 5
```

Reward3

```
Reward3 =
0.8925
```

Call the reset function again to restore the environment back to the initial state.

```
reset(RL_OptimalExecution_Training_Environment_Sell);
```

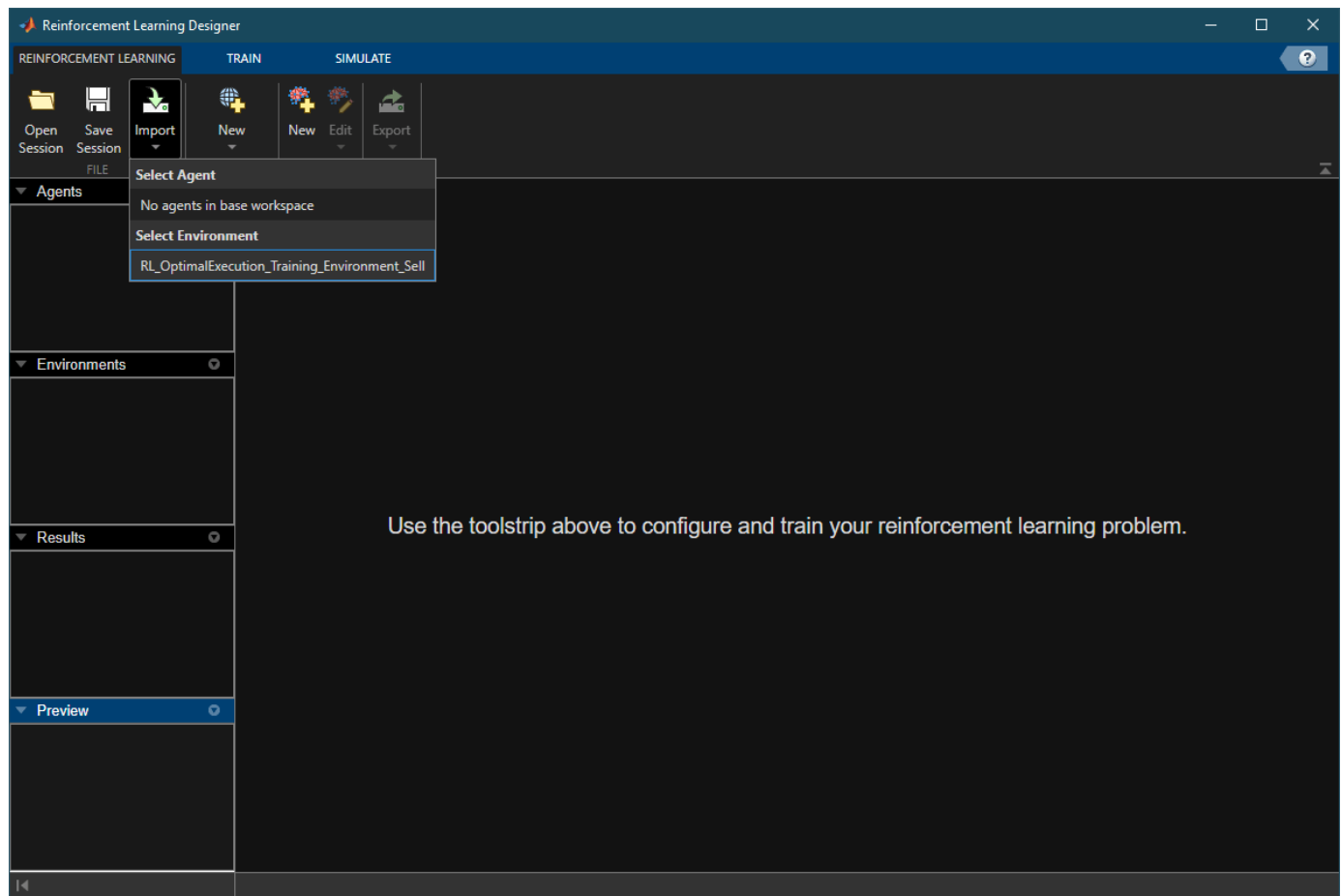
Reset!

Create and Train Agent for Sell Trades

Open Reinforcement Learning Designer (Reinforcement Learning Toolbox).

```
reinforcementLearningDesigner
```

Import the training environment by clicking the **Import** button and selecting the created training environment.



Create a new DQN agent by selecting **New > Agent**, then selecting DQN from the **Compatible algorithm** menu. Enable LSTM by selecting **Use recurrent neural network**.

Create an agent compatible with the specified environment. The agent is initialized with a default neural network structure, which you can modify after creation.

Agent name:

Environment:

Compatible algorithm:

Neural network parameterizations

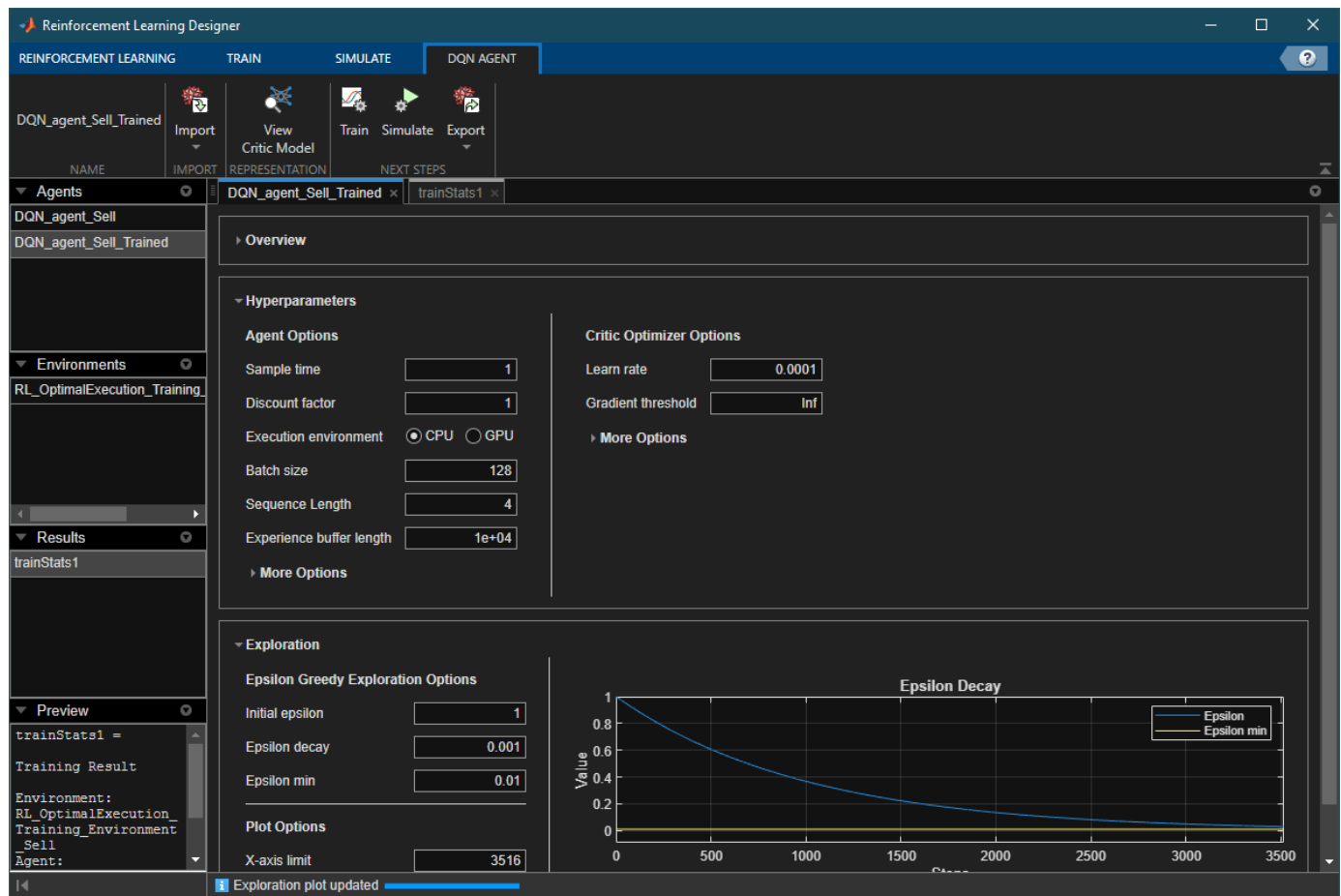
Number of hidden units:

☒ Use recurrent neural network

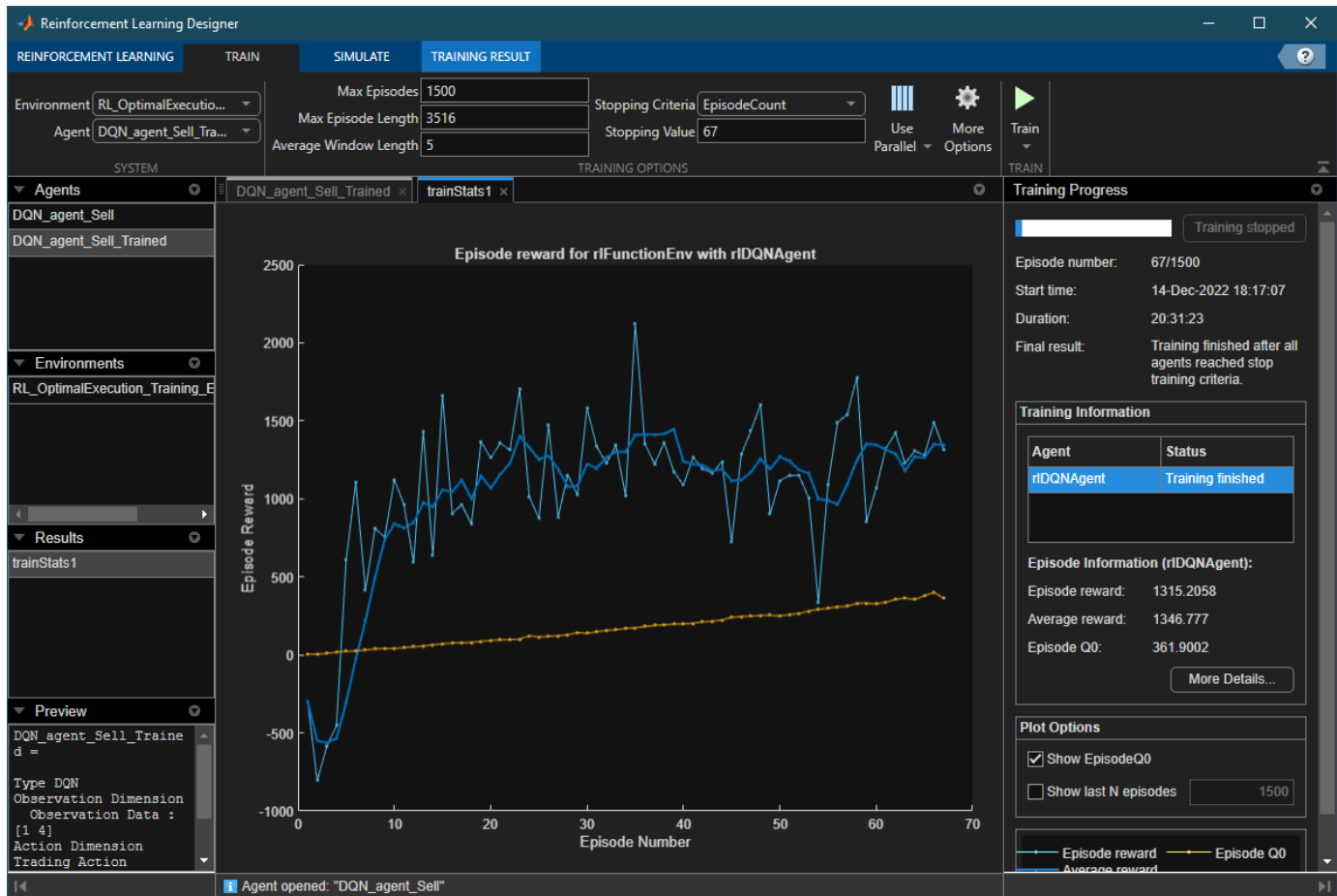
Help OK Cancel

Set the agent hyperparameters and exploration parameters in the **DQN Agent** tab. Use the default options, except for the following:

- **Discount factor** — 1
- **Learn rate** — 0.0001
- **Batch size** — 128
- **Sequence length** — 4
- **Initial epsilon** — 1
- **Epsilon decay** — 0.001



Go to the **Train** tab, and set the **Max Episode Length** to 3516 (NumTrainingSteps). You can experiment with different **Stopping Criteria**, or you can manually stop the training by clicking on the **Stop** button. In this example, you set the **Stopping Criteria** to EpisodeCount and the **Max Episodes** to a value greater than or equal to EpisodeCount. Click **Train** to start the training.



Once the training is complete, export the trained agent to the workspace. Since training can take a long time, you can load a pretrained agent (`DQN_agent_Sell_Trained.mat`) for sell trades.

```
load DQN_agent_Sell_Trained.mat
```

You can view the DQN agent options by examining the `AgentOptions` property.

```
DQN_agent_Sell_Trained.AgentOptions
```

```
ans =
```

```
rlDQNAgentOptions with properties:
```

```

    SampleTime: 1
    DiscountFactor: 1
    EpsilonGreedyExploration: [1x1 rl.option.EpsilonGreedyExploration]
    ExperienceBufferLength: 10000
    MiniBatchSize: 128
    SequenceLength: 4
    CriticOptimizerOptions: [1x1 rl.option.rlOptimizerOptions]
    NumStepsToLookAhead: 1
    NumWarmStartSteps: 128
    NumEpoch: 1
    MaxMiniBatchPerEpoch: 100
    LearningFrequency: -1
    UseDoubleDQN: 1

```

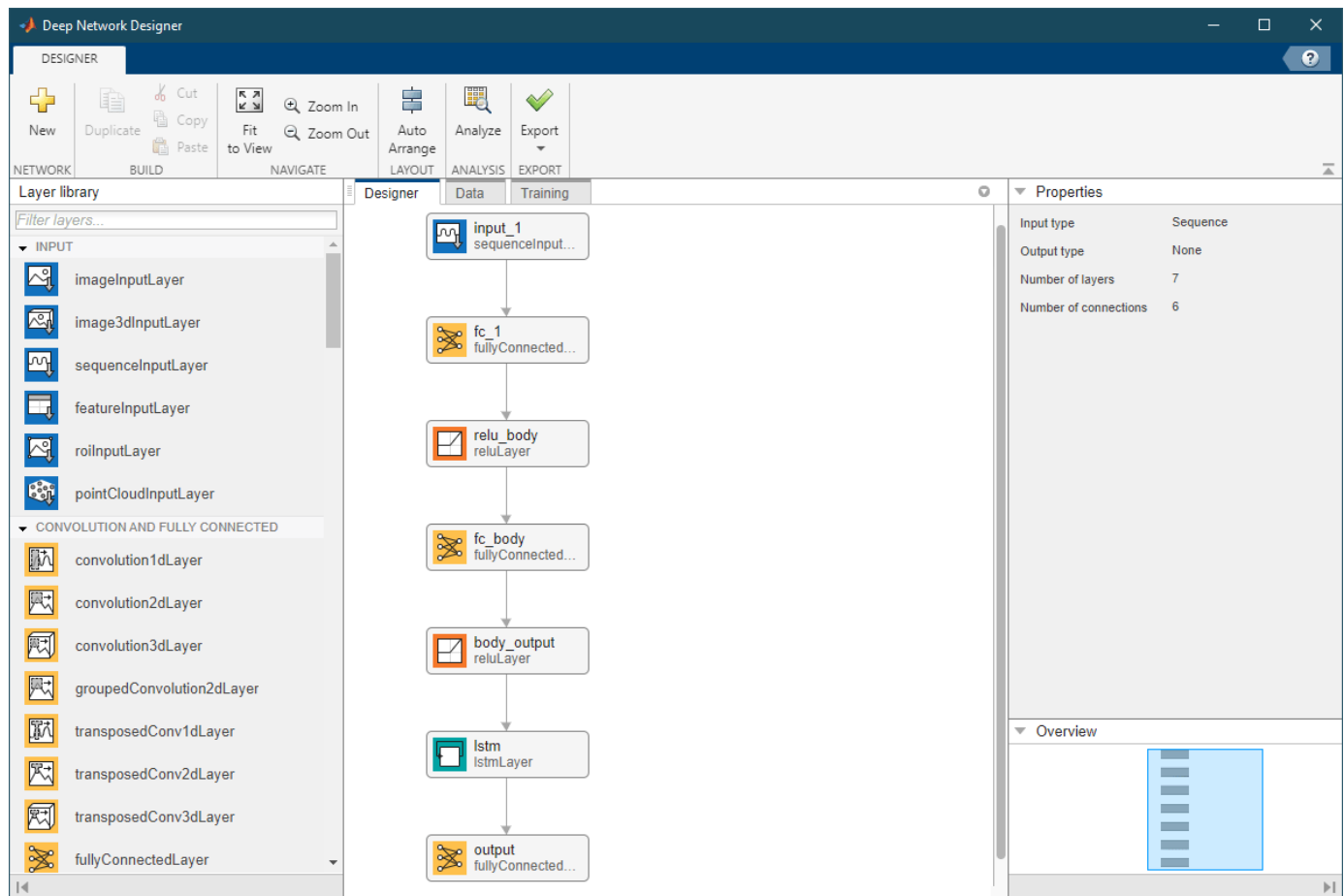
```

        TargetSmoothFactor: 1.0000e-03
        TargetUpdateFrequency: 1
        BatchDataRegularizerOptions: []
        ResetExperienceBufferBeforeTraining: 0
        InfoToSave: [1x1 struct]

```

Also, you can visualize the DQN agent's critic network by using Deep Network Designer (Deep Learning Toolbox).

```
deepNetworkDesigner(layerGraph(getModel(getCritic(DQN_agent_Sell_Trained))))
```



Compute IS Using Trained Agent and Training Data for Sell Trades

Simulate selling actions using the trained agent and training environment.

```

rng('default');
reset(RL_OptimalExecution_Training_Environment_Sell);

Reset!

simOptions = rlSimulationOptions(MaxSteps=NumTrainingSteps)

simOptions =
    rlSimulationOptions with properties:

```

```

        MaxSteps: 3516
        NumSimulations: 1
        StopOnError: "on"
        SimulationStorageType: "memory"
        SaveSimulationDirectory: "savedSims"
        SaveFileVersion: "-v7"
        UseParallel: 0
        ParallelizationOptions: [1x1 rl.option.ParallelSimulation]

Trained_Agent_Sell = DQN_agent_Sell_Trained;
experience_Sell = sim(RL_OptimalExecution_Training_Environment_Sell,Trained_Agent_Sell,simOptions);

Reset!
Last Horizon. Episode is Done!
HorizonIdx:

ans =
293

SimAction_Train_Sell = squeeze(experience_Sell.Action.TradingAction_NumberOfShares_.Data);

Compute the IS using simulated actions on the training data.

NumSimSteps = length(SimAction_Train_Sell);
SimInventory_Train_Sell = nan(NumSimSteps,1);
SimExecutedShares_Train_Sell = SimInventory_Train_Sell;
SimReward_Train_Sell = nan(NumSimSteps,1);

reset(RL_OptimalExecution_Training_Environment_Sell);

Reset!
for k=1:NumSimSteps
    [~,SimReward_Train_Sell(k),~,LoggedSignals] = ...
        step(RL_OptimalExecution_Training_Environment_Sell,SimAction_Train_Sell(k));
    SimInventory_Train_Sell(k) = LoggedSignals.CurrentInventoryShares;
    SimExecutedShares_Train_Sell(k) = sum(LoggedSignals.ExecutedShares);
end

Last Horizon. Episode is Done!
HorizonIdx:

ans =
293

IS_Agent_Step_Train_Sell = LoggedSignals.IS_Agent;
IS_Agent_Horizon_Train_Sell = IS_Agent_Step_Train_Sell( ...
    NumIntervalsPerHorizon:NumIntervalsPerHorizon:NumTrainingSteps);

```

Plot Training Results for Sell Trades

Plot Implementation Shortfall (IS), executed shares, and inventory shares for the agent for each step using training data.

```

figure
subplot(3,1,1)
bar(1:NumTrainingSteps, IS_Agent_Step_Train_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))

```

```

ylabel("Implementation Shortfall")
title("Implementation Shortfall (Agent, Training, Sell)")
xlim([0 300])
ylim([-200 200])

subplot(3,1,2)
bar(1:NumTrainingSteps, SimExecutedShares_Train_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
ylabel("Executed Shares")
title("Shares Traded at Each Step (Agent, Training, Sell)")
xlim([0 300])

subplot(3,1,3)
bar(1:NumTrainingSteps, SimInventory_Train_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
ylabel("Inventory Shares")
title("Inventory Shares (Agent, Training, Sell)")
xlim([0 300])

```



Unlike the previous TWAP baseline plot, where the executed shares in the middle panel have a flat profile, here you can see that the agent actively changes the number of executed shares at each step. Also, the inventory shares in the bottom panel decline in a non-linear manner.

To directly compare the results from the agent with the TWAP baseline, you can plot both results on the same chart. Generate plots to compare implementation shortfall, executed shares, and inventory shares between the TWAP baseline and the agent for each step using the training data.

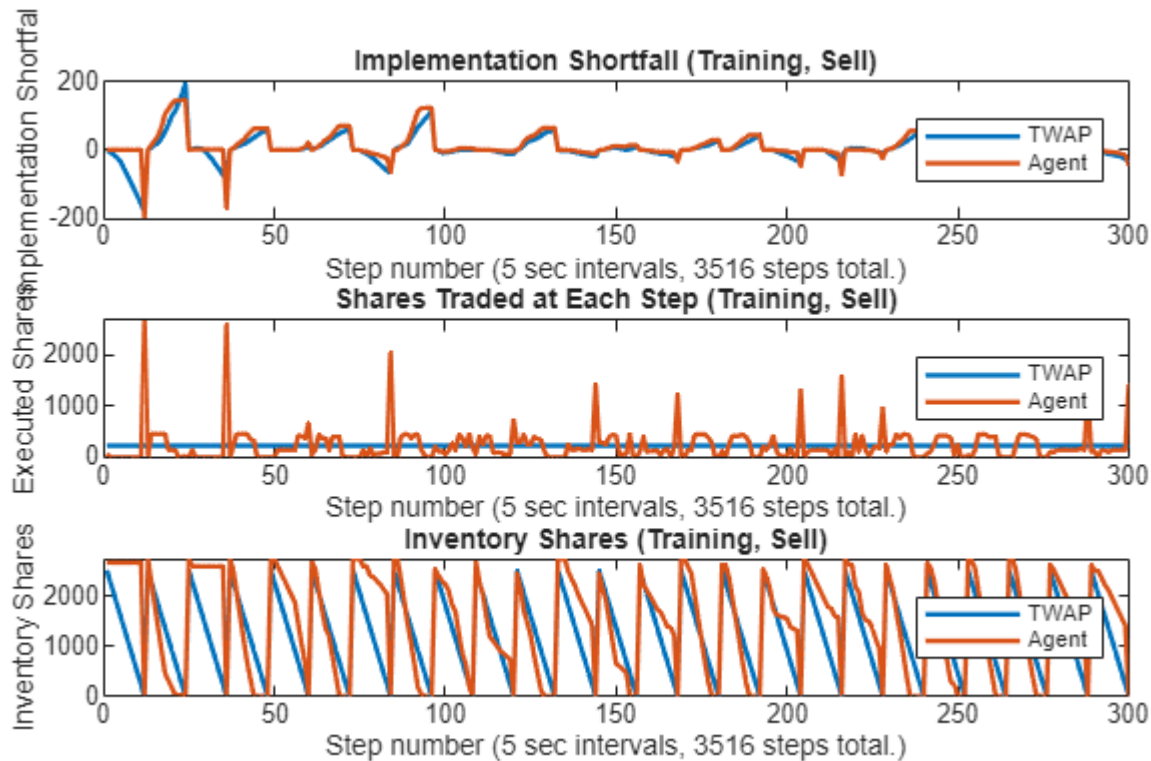

```

figure
subplot(3,1,1)
plot(1:NumTrainingSteps, [IS_TWAP_Step_Train_Sell IS_Agent_Step_Train_Sell], linewidth=1.5)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), "...",
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
ylabel("Implementation Shortfall")
title("Implementation Shortfall (Training, Sell)")
legend(["TWAP" "Agent"], location="east")
xlim([0 300])

subplot(3,1,2)
plot(1:NumTrainingSteps, [TradingShares_Step_TWAP_Train_Sell ...
    SimExecutedShares_Train_Sell], linewidth=1.5)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), "...",
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
ylabel("Executed Shares")
title("Shares Traded at Each Step (Training, Sell)")
legend(["TWAP" "Agent"], location="east")
xlim([0 300])

subplot(3,1,3)
plot(1:NumTrainingSteps, [InventoryShares_Step_TWAP_Train_Sell ...
    SimInventory_Train_Sell], linewidth=1.5)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), "...",
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
ylabel("Inventory Shares")
title("Inventory Shares (Training, Sell)")
legend(["TWAP" "Agent"], location="east")
xlim([0 300])

```



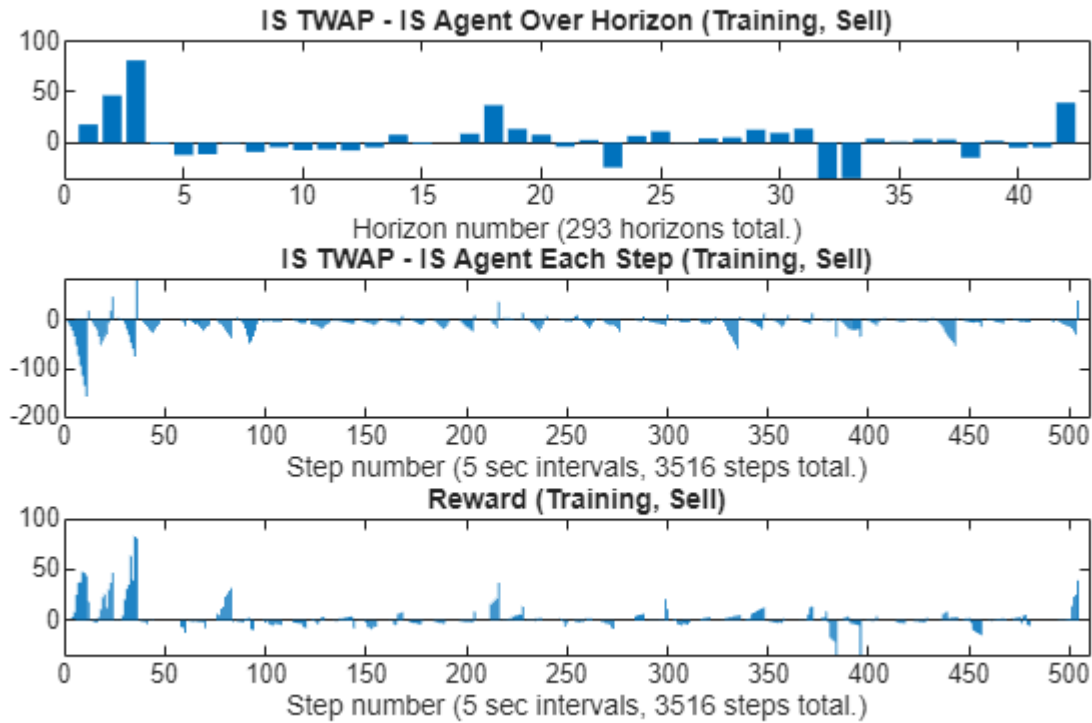
You can see the differences between the TWAP baseline (blue) and the agent (red) in their IS, executed shares, and inventory shares. Also, the agent tends to behave differently when the IS trends negative as compared to positive.

- For example, in the first horizon (step numbers 1 to 12) and the third horizon (step numbers 25 to 36), the IS trends negative (more favorable prices than the arrival price) and the agent initially trades fewer shares than TWAP, until there are large spikes in the executed shares at the ends of the horizons to meet the ending inventory constraint. Also, the inventories initially declines more slowly than TWAP, until the abrupt declines to zero at the end of the horizon. This coincided with negative pulls in the agent's IS at the end of the first and the third horizons that go even lower than the IS of TWAP.
- On the other hand, in the second horizon (step numbers 13 to 24), the IS trends positive (less favorable prices than the arrival price), and the agent initially trades more shares than TWAP to liquidate the inventory faster than TWAP. This has a limiting effect on the agent's IS at the end of the horizon, which is also lower than the IS of TWAP.

You can see similar behaviors in other horizons, although they do not always lead to an outperformance of the agent over TWAP.

Also, you can plot the differences in IS between TWAP and the agent, and then compare them with the rewards.

```
figure
subplot(3,1,1)
bar(1:NumTrainingHorizons, IS_TWAP_Horizon_Train_Sell - IS_Agent_Horizon_Train_Sell)
xlabel(strcat("Horizon number (", num2str(NumTrainingHorizons), " horizons total.)"))
title("IS TWAP - IS Agent Over Horizon (Training, Sell)")
xlim([0 43])
subplot(3,1,2)
bar(1:NumTrainingSteps, IS_TWAP_Step_Train_Sell - IS_Agent_Step_Train_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
title("IS TWAP - IS Agent Each Step (Training, Sell)")
xlim([0 510])
subplot(3,1,3)
bar(1:NumTrainingSteps, SimReward_Train_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTrainingSteps), " steps total.)"))
title("Reward (Training, Sell)")
xlim([0 510])
```



The top panel ($IS_{TWAP}(0, T) - IS_{Agent}(0, T)$) shows the final performance of the agent for the training data as the differences in IS between TWAP and the agent at the ends of the horizons at times T . For example, in the top panel, you can see the outperformance of the agent (positive IS difference) in the first three horizons, while the agent underperforms in some of the other horizons. The middle panel ($IS_{TWAP}(0, t) - IS_{Agent}(0, t)$) shows the differences in IS for all time steps t , including the ones before the ends of the horizons. The middle panel does not appear to track the top panel values very well in some steps (except for the very last steps in the horizons, which are the top panel values when $t = T$), as there are frequent trend reversals between negative and positive values at the last steps of the horizons. Therefore, the simple difference in IS between TWAP and the agent at each step ($IS_{TWAP}(0, t) - IS_{Agent}(0, t)$) shown in the middle panel may not be a good reward function. On the other hand, the bottom panel, which plots the rewards used by the agent in this example at each step, appears to track the top panel values more closely than the middle panel at most steps.

Display Summary of Training Results for Sell Trades

Display the total Implementation Shortfall outperformance of the agent over TWAP for the training data.

```
sum(IS_TWAP_Horizon_Train_Sell - IS_Agent_Horizon_Train_Sell)
```

```
ans =  
164.4700
```

Display total-gain-to-loss ratio (TGLR) for the training data.

```
ISTGLR(IS_TWAP_Horizon_Train_Sell - IS_Agent_Horizon_Train_Sell)
```

```
ans =  
1.2327
```

TWAP Baseline Analysis on Testing Data

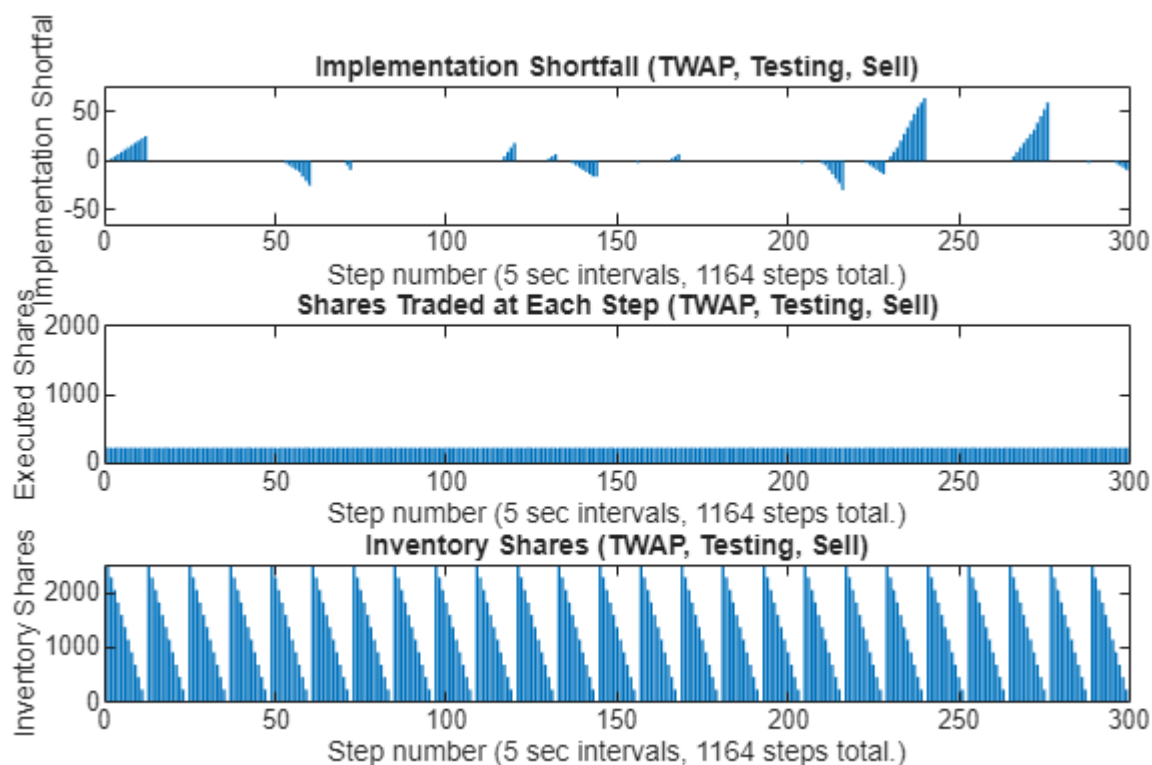
Using the same trained agent as the Compute IS Using Trained Agent and Training Data for Sell Trades on page 4-422 section, you can perform the same analysis on the testing data that you kept separate from the training data.

First, execute the TWAP trades and compute IS for the testing data.

```
[IS_TWAP_Horizon_Test_Sell, IS_TWAP_Step_Test_Sell, ...  
    InventoryShares_Step_TWAP_Test_Sell, TradingShares_Step_TWAP_Test_Sell] = ...  
    executeTWAPTrades(TestingLOB, NumLevels, TotalTradingShares_Sell, ...  
        NumIntervalsPerHorizon, NumTestingHorizons, BuyTrade);
```

Plot the implementation shortfall, executed shares, and inventory shares for the TWAP baseline for each step using testing data.

```
figure  
subplot(3,1,1)  
bar(1:NumTestingSteps, IS_TWAP_Step_Test_Sell)  
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...  
    " sec intervals, ", num2str(NumTestingSteps), " steps total.)"))  
ylabel("Implementation Shortfall")  
title("Implementation Shortfall (TWAP, Testing, Sell)")  
xlim([0 300])  
ylim([-65 75])  
  
subplot(3,1,2)  
bar(1:NumTestingSteps, TradingShares_Step_TWAP_Test_Sell)  
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...  
    " sec intervals, ", num2str(NumTestingSteps), " steps total.)"))  
ylabel("Executed Shares")  
title("Shares Traded at Each Step (TWAP, Testing, Sell)")  
xlim([0 300])  
ylim([0 2000])  
  
subplot(3,1,3)  
bar(1:NumTestingSteps, InventoryShares_Step_TWAP_Test_Sell)  
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...  
    " sec intervals, ", num2str(NumTestingSteps), " steps total.)"))  
ylabel("Inventory Shares")  
title("Inventory Shares (TWAP, Testing, Sell)")  
xlim([0 300])
```



Set Up RL Testing Environment for Sell Trades

Next, define the reset function for the testing environment.

```
EnvConstants_Test_Sell.SampledBook = TestingLOB;
EnvConstants_Test_Sell.BuyTrade = BuyTrade;
EnvConstants_Test_Sell.TotalTradingShares = TotalTradingShares_Sell;
EnvConstants_Test_Sell.NumIntervalsPerHorizon = NumIntervalsPerHorizon;
EnvConstants_Test_Sell.NumHorizons = NumTestingHorizons;
EnvConstants_Test_Sell.NumSteps = NumTestingSteps;
EnvConstants_Test_Sell.NumLevels = NumLevels;
EnvConstants_Test_Sell.IS_TWAP = IS_TWAP_Step_Test_Sell;
```

```
ResetFunction_Test_Sell = @( ) RL_OptimalExecution_LOB_ResetFcn(EnvConstants_Test_Sell);
```

Define the step function for the testing environment.

```
StepFunction_Test_Sell = @(Action,LoggedSignals) ...
    RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvConstants_Test_Sell);
```

Create the testing environment using the reset and step function handles.

```
RL_OptimalExecution_Testing_Environment_Sell = ...
    rlFunctionEnv(ObservationInfo_Train_Sell,ActionInfo_Sell, ...
        StepFunction_Test_Sell,ResetFunction_Test_Sell)
```

```
Reset!
Reset!
```

```
RL_OptimalExecution_Testing_Environment_Sell =
    rlFunctionEnv with properties:
```

```
StepFcn: @(Action,LoggedSignals)RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvCon
ResetFcn: @()RL_OptimalExecution_LOB_ResetFcn(EnvConstants_Test_Sell)
Info: [1x1 struct]
```

Validate Testing Environment Reset and Step Functions for Sell Trades

Now, validate the testing environment using reset and step function handles.

```
InitialObservation = reset(RL_OptimalExecution_Testing_Environment_Sell);
```

Reset!

InitialObservation

```
InitialObservation = 1x4
```

```
      2738      12      0      0
```

```
[~,Reward1,~,LoggedSignals1] = step( ...
    RL_OptimalExecution_Testing_Environment_Sell,TradingShares_Step_TWAP_Train_Sell(1));
LoggedSignals1
```

```
LoggedSignals1 = struct with fields:
    IntervalIdx: 1
    HorizonIdx: 1
    CurrentInventoryShares: 2624
    ArrivalPrice: 26.8000
    ExecutedShares: 114
    ExecutedPrices: 26.8000
    HorizonExecutedShares: [60x1 double]
    HorizonExecutedPrices: [60x1 double]
    IS_Agent: [1164x1 double]
    Reward: [1164x1 double]
    NumIntervalsPerHorizon: 12
    NumHorizons: 97
    NumLevels: 5
```

Reward1

```
Reward1 =
0
```

```
[~,Reward2,~,LoggedSignals2] = step( ...
    RL_OptimalExecution_Testing_Environment_Sell,10);
LoggedSignals2
```

```
LoggedSignals2 = struct with fields:
    IntervalIdx: 2
    HorizonIdx: 1
    CurrentInventoryShares: 2614
    ArrivalPrice: 26.8000
    ExecutedShares: 10
    ExecutedPrices: 26.7900
    HorizonExecutedShares: [60x1 double]
    HorizonExecutedPrices: [60x1 double]
    IS_Agent: [1164x1 double]
```

```

                Reward: [1164×1 double]
NumIntervalsPerHorizon: 12
                NumHorizons: 97
                NumLevels: 5

```

Reward2

```

Reward2 =
-0.2050

```

```

[~,Reward3,~,LoggedSignals3] = step( ...
    RL_OptimalExecution_Testing_Environment_Sell,35);
LoggedSignals3

```

```

LoggedSignals3 = struct with fields:
    IntervalIdx: 3
    HorizonIdx: 1
    CurrentInventoryShares: 2579
    ArrivalPrice: 26.8000
    ExecutedShares: 35
    ExecutedPrices: 26.7900
    HorizonExecutedShares: [60×1 double]
    HorizonExecutedPrices: [60×1 double]
    IS_Agent: [1164×1 double]
    Reward: [1164×1 double]
    NumIntervalsPerHorizon: 12
    NumHorizons: 97
    NumLevels: 5

```

Reward3

```

Reward3 =
-0.2975

```

Call the reset function again to restore the environment back to the initial state.

```

reset(RL_OptimalExecution_Testing_Environment_Sell);

```

Reset!

Compute IS Using Trained Agent and Testing Data for Sell Trades

Simulate actions using the trained agent and testing data for sell trades.

```

simOptions = rlSimulationOptions(MaxSteps=NumTestingSteps);
experience_Sell = sim(RL_OptimalExecution_Testing_Environment_Sell,Trained_Agent_Sell,simOptions);

```

Reset!

```

Last Horizon. Episode is Done!
HorizonIdx:

```

```

ans =
97

```

```

SimAction_Test_Sell = squeeze(experience_Sell.Action.TradingAction_NumberOfShares_.Data);

```

Compute Implementation Shortfall (IS) using simulated actions on testing data.

```

NumSimSteps = length(SimAction_Test_Sell);
SimInventory_Test_Sell = nan(NumSimSteps,1);
SimExecutedShares_Test_Sell = SimInventory_Test_Sell;
SimReward_Test_Sell = nan(NumSimSteps,1);

reset(RL_OptimalExecution_Testing_Environment_Sell);

Reset!

for k=1:NumSimSteps
    [~,SimReward_Test_Sell(k),~,LoggedSignals] = step( ...
        RL_OptimalExecution_Testing_Environment_Sell,SimAction_Test_Sell(k));
    SimInventory_Test_Sell(k) = LoggedSignals.CurrentInventoryShares;
    SimExecutedShares_Test_Sell(k) = sum(LoggedSignals.ExecutedShares);
end

Last Horizon. Episode is Done!
HorizonIdx:

ans =
97

IS_Agent_Step_Test_Sell = LoggedSignals.IS_Agent;
IS_Agent_Horizon_Test_Sell = IS_Agent_Step_Test_Sell( ...
    NumIntervalsPerHorizon:NumIntervalsPerHorizon:NumTestingSteps);

```

Plot Testing Results for Sell Trades

Plot Implementation Shortfall (IS), executed shares, and inventory shares of the agent for each step using testing data.

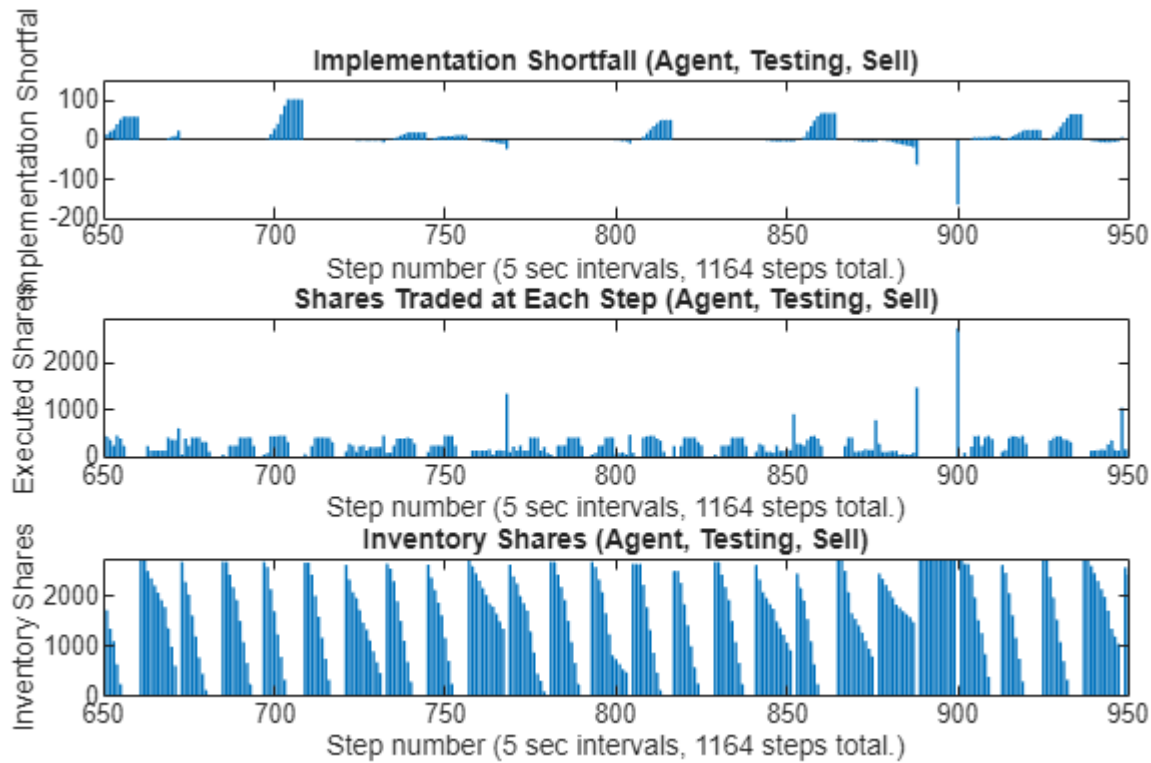
```

figure
subplot(3,1,1)
bar(1:NumTestingSteps, IS_Agent_Step_Test_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTestingSteps), " steps total.)"))
ylabel("Implementation Shortfall")
title("Implementation Shortfall (Agent, Testing, Sell)")
xlim([650 950])
ylim([-200 150])

subplot(3,1,2)
bar(1:NumTestingSteps, SimExecutedShares_Test_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTestingSteps), " steps total.)"))
ylabel("Executed Shares")
title("Shares Traded at Each Step (Agent, Testing, Sell)")
xlim([650 950])
ylim([0 2900])

subplot(3,1,3)
bar(1:NumTestingSteps, SimInventory_Test_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTestingSteps), " steps total.)"))
ylabel("Inventory Shares")
title("Inventory Shares (Agent, Testing, Sell)")
xlim([650 950])

```

Compare implementation shortfall, executed shares, and inventory shares between the TWAP baseline and the agent for each step using testing data.

```
figure
subplot(3,1,1)
plot(1:NumTestingSteps, [IS_TWAP_Step_Test_Sell IS_Agent_Step_Test_Sell], linewidth=1.5)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), " ...
    " sec intervals, ", num2str(NumTestingSteps), " steps total.)"))
ylabel("Implementation Shortfall")
title("Implementation Shortfall (Testing, Sell)")
legend(["TWAP" "Agent"],location='best')
xlim([650 950])
ylim([-200 150])

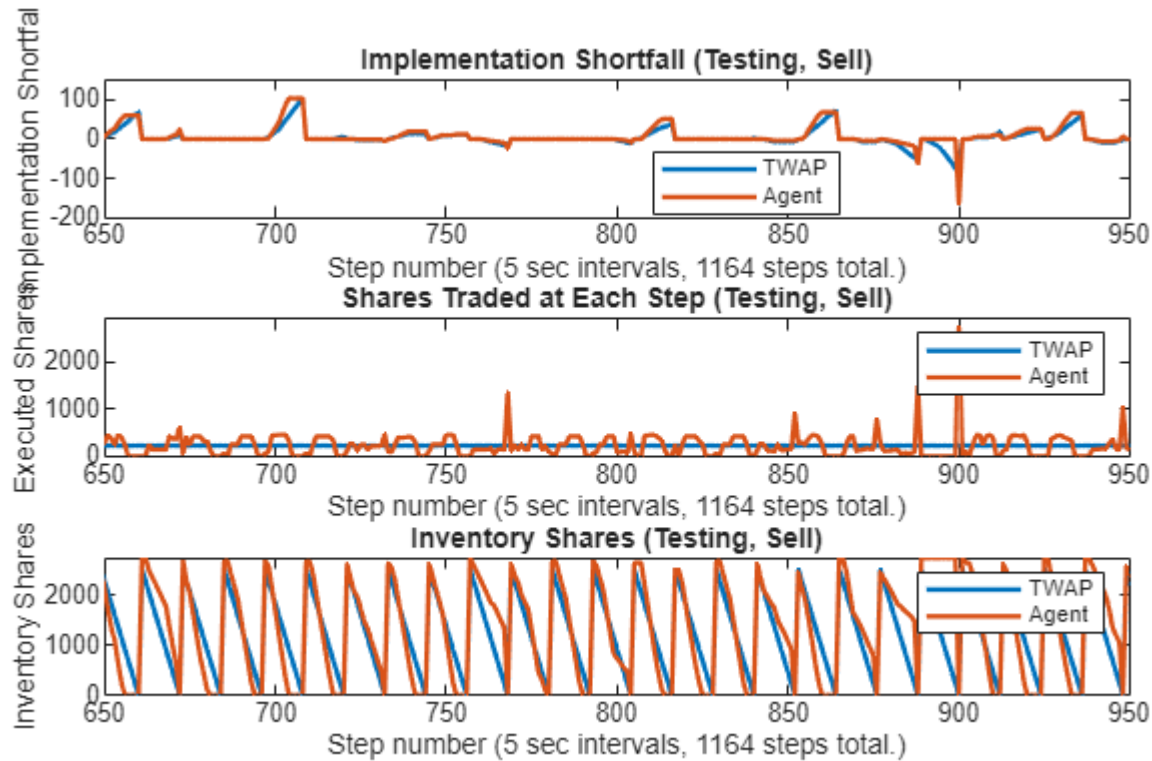
subplot(3,1,2)
plot(1:NumTestingSteps, [TradingShares_Step_TWAP_Test_Sell ...
    SimExecutedShares_Test_Sell], linewidth=1.5)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), " ...
    " sec intervals, ", num2str(NumTestingSteps), " steps total.)"))
ylabel("Executed Shares")
title("Shares Traded at Each Step (Testing, Sell)")
legend(["TWAP" "Agent"])
xlim([650 950])
ylim([0 2900])

subplot(3,1,3)
plot(1:NumTestingSteps, [InventoryShares_Step_TWAP_Test_Sell ...
    SimInventory_Test_Sell], linewidth=1.5)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), " ...
```

```

    " sec intervals, ", num2str(NumTestingSteps), " steps total.))
ylabel("Inventory Shares")
title("Inventory Shares (Testing, Sell)")
legend(["TWAP" "Agent"])
xlim([650 950])

```



Just as in the training data, the agent (red) actively changes the number of executed shares for each step in the testing data, while the TWAP policy (blue) executes the same number of shares at each step.

Plot the differences in IS between TWAP and the agent and then compare them with the rewards.

```

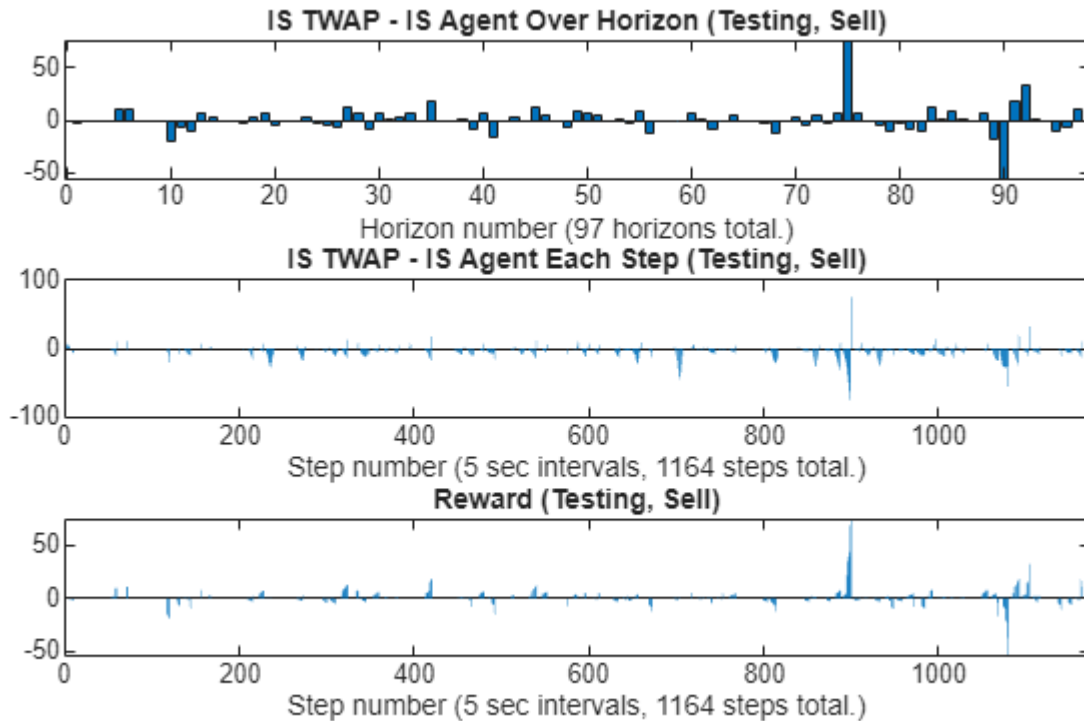
figure
subplot(3,1,1)
bar(1:NumTestingHorizons, IS_TWAP_Horizon_Test_Sell - IS_Agent_Horizon_Test_Sell)
xlabel(strcat("Horizon number (", num2str(NumTestingHorizons), " horizons total.))")
title("IS TWAP - IS Agent Over Horizon (Testing, Sell)")

subplot(3,1,2)
bar(1:NumTestingSteps, IS_TWAP_Step_Test_Sell - IS_Agent_Step_Test_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTestingSteps), " steps total.))")
title("IS TWAP - IS Agent Each Step (Testing, Sell)")
xlim([0 NumTestingSteps+9])

subplot(3,1,3)
bar(1:NumTestingSteps, SimReward_Test_Sell)
xlabel(strcat("Step number (", num2str(TradingIntervalSec), ...
    " sec intervals, ", num2str(NumTestingSteps), " steps total.))")

```

```
title("Reward (Testing, Sell)")
xlim([0 NumTestingSteps+9])
```



Display Summary of Training and Testing Results for Sell Trades

You can summarize the results of the sell trade agent's training and testing.

Display total Implementation Shortfall (IS) outperformance of the agent over TWAP for training data.

```
Total_Outperformance_Train_Sell = sum(IS_TWAP_Horizon_Train_Sell - IS_Agent_Horizon_Train_Sell);
table(Total_Outperformance_Train_Sell)
```

```
ans=table
    Total_Outperformance_Train_Sell
    _____
                164.47
```

Display total-gain-to-loss ratio (TGLR) and gain-to-loss ratio (GLR) for the training data.

```
[TGLR_Train_Sell, GLR_Train_Sell] = ...
    ISTGLR(IS_TWAP_Horizon_Train_Sell - IS_Agent_Horizon_Train_Sell);

table(TGLR_Train_Sell, GLR_Train_Sell)
```

```
ans=1x2 table
    TGLR_Train_Sell    GLR_Train_Sell
    _____    _____
```

1.2327 1.2532

Display the total IS outperformance of the agent over TWAP for the testing data.

```
Total_Outperformance_Test_Sell = sum(IS_TWAP_Horizon_Test_Sell - IS_Agent_Horizon_Test_Sell);
table(Total_Outperformance_Test_Sell)
```

```
ans=table
      Total_Outperformance_Test_Sell
      _____
               104.12
```

Display TGLR and GLR for the testing data.

```
[TGLR_Test_Sell, GLR_Test_Sell] = ...
    ISTGLR(IS_TWAP_Horizon_Test_Sell - IS_Agent_Horizon_Test_Sell);
table(TGLR_Test_Sell, GLR_Test_Sell)
```

```
ans=1x2 table
      TGLR_Test_Sell      GLR_Test_Sell
      _____      _____
               1.389               1.1906
```

Optimal Execution for Buy Trades

You can use a shortened version of the previous procedure (Optimal Execution for Sell Trades on page 4-412) to support the workflow for designing a DQN agent for optimal execution of buy trades. To repeat the same workflow for buy trades, first set the `BuyTrade` logical flag to `true` and define the total number of shares to buy over the horizon.

```
BuyTrade = true;
TotalTradingShares_Buy = 2551;
```

Execute TWAP trades and compute the Implementation Shortfall (IS).

```
[IS_TWAP_Horizon_Train_Buy, IS_TWAP_Step_Train_Buy, ...
    InventoryShares_Step_TWAP_Train_Buy, TradingShares_Step_TWAP_Train_Buy] = ...
    executeTWAPTrades(TrainingLOB, NumLevels, TotalTradingShares_Buy, ...
        NumIntervalsPerHorizon, NumTrainingHorizons, BuyTrade);
```

Set up the RL training environment for buy trades.

```
ObservationDimension = [1 4];
ObservationInfo_Train_Buy = rlNumericSpec(ObservationDimension);
ObservationInfo_Train_Buy.Name = 'Observation Data';
ObservationInfo_Train_Buy.Description = ['RemainingSharesToTrade, ...' ...
    'RemainingIntervals, IS_Agent, PriceDivergence'];
ObservationInfo_Train_Buy.LowerLimit = -inf;
ObservationInfo_Train_Buy.UpperLimit = inf;

EnvConstants_Train_Buy.SampledBook = TrainingLOB;
EnvConstants_Train_Buy.BuyTrade = BuyTrade;
EnvConstants_Train_Buy.TotalTradingShares = TotalTradingShares_Buy;
EnvConstants_Train_Buy.NumIntervalsPerHorizon = NumIntervalsPerHorizon;
```

```

EnvConstants_Train_Buy.NumHorizons = NumTrainingHorizons;
EnvConstants_Train_Buy.NumSteps = NumTrainingSteps;
EnvConstants_Train_Buy.NumLevels = NumLevels;
EnvConstants_Train_Buy.IS_TWAP = IS_TWAP_Step_Train_Buy;
ResetFunction_Buy = @( ) RL_OptimalExecution_LOB_ResetFcn(EnvConstants_Train_Buy);

NumActions = 39;
LowerActionRange = linspace(0,TradingShares_Step_TWAP_Train_Buy(1), round(NumActions/2));
ActionRange = round([LowerActionRange(1:end-1) ...
    linspace(TradingShares_Step_TWAP_Train_Buy(1), ...
    TradingShares_Step_TWAP_Train_Buy(1)*2,round(NumActions/2))]);
NumActions = length(ActionRange);

ActionInfo_Buy = rlFiniteSetSpec(ActionRange);
ActionInfo_Buy.Name = 'Trading Action (Number of Shares)';
ActionInfo_Buy.Description = 'Number of Shares to be Traded at Each Time Step';

StepFunction_Train_Buy = @(Action,LoggedSignals) ...
    RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvConstants_Train_Buy);

RL_OptimalExecution_Training_Environment_Buy = rlFunctionEnv( ...
    ObservationInfo_Train_Buy,ActionInfo_Buy,StepFunction_Train_Buy,ResetFunction_Buy)

Reset!
Reset!

RL_OptimalExecution_Training_Environment_Buy =
    rlFunctionEnv with properties:

    StepFcn: @(Action,LoggedSignals)RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvCon
    ResetFcn: @( )RL_OptimalExecution_LOB_ResetFcn(EnvConstants_Train_Buy)
    Info: [1x1 struct]

```

Use Reinforcement Learning Designer (Reinforcement Learning Toolbox) to create and train the agent for the buy trades. You use the same procedure (Optimal Execution for Sell Trades on page 4-412) as with the sell trades, except that the imported training environment is for buy trades (RL_OptimalExecution_Training_Environment_Buy). Once the training is complete, export the trained agent to the workspace. Since training can take a long time, this example uses a pretrained agent (DQN_agent_Buy_Trained.mat).

```
load DQN_agent_Buy_Trained.mat
```

Compute IS using the trained agent and training data for buy trades.

```

rng('default');
reset(RL_OptimalExecution_Training_Environment_Buy);

Reset!

simOptions = rlSimulationOptions(MaxSteps=NumTrainingSteps);
Trained_Agent_Buy = DQN_agent_Buy_Trained;
experience_Buy = sim(RL_OptimalExecution_Training_Environment_Buy,Trained_Agent_Buy,simOptions);

Reset!
Last Horizon. Episode is Done!
HorizonIdx:

```

```

ans =
293

SimAction_Train_Buy = squeeze(experience_Buy.Action.TradingAction_NumberOfShares_.Data);
NumSimSteps = length(SimAction_Train_Buy);
SimInventory_Train_Buy = nan(NumSimSteps,1);
SimExecutedShares_Train_Buy = SimInventory_Train_Buy;
SimReward_Train_Buy = nan(NumSimSteps,1);
reset(RL_OptimalExecution_Training_Environment_Buy);

```

Reset!

```

for k=1:NumSimSteps
    [~,SimReward_Train_Buy(k),~,LoggedSignals] = ...
        step(RL_OptimalExecution_Training_Environment_Buy,SimAction_Train_Buy(k));
    SimInventory_Train_Buy(k) = LoggedSignals.CurrentInventoryShares;
    SimExecutedShares_Train_Buy(k) = sum(LoggedSignals.ExecutedShares);
end

```

Last Horizon. Episode is Done!
HorizonIdx:

```

ans =
293

```

```

IS_Agent_Step_Train_Buy = LoggedSignals.IS_Agent;
IS_Agent_Horizon_Train_Buy = IS_Agent_Step_Train_Buy( ...
    NumIntervalsPerHorizon:NumIntervalsPerHorizon:NumTrainingSteps);

```

Execute the TWAP trades and compute the IS for the testing data.

```

[IS_TWAP_Horizon_Test_Buy,IS_TWAP_Step_Test_Buy, ...
    InventoryShares_Step_TWAP_Test_Buy,TradingShares_Step_TWAP_Test_Buy] = ...
    executeTWAPTrades(TestingLOB,NumLevels,TotalTradingShares_Buy, ...
        NumIntervalsPerHorizon,NumTestingHorizons,BuyTrade);

```

Set up the RL testing environment for buy trades.

```

EnvConstants_Test_Buy.SampledBook = TestingLOB;
EnvConstants_Test_Buy.BuyTrade = BuyTrade;
EnvConstants_Test_Buy.TotalTradingShares = TotalTradingShares_Buy;
EnvConstants_Test_Buy.NumIntervalsPerHorizon = NumIntervalsPerHorizon;
EnvConstants_Test_Buy.NumHorizons = NumTestingHorizons;
EnvConstants_Test_Buy.NumSteps = NumTestingSteps;
EnvConstants_Test_Buy.NumLevels = NumLevels;
EnvConstants_Test_Buy.IS_TWAP = IS_TWAP_Step_Test_Buy;
ResetFunction_Test_Buy = @( ) RL_OptimalExecution_LOB_ResetFcn(EnvConstants_Test_Buy);
StepFunction_Test_Buy = @(Action,LoggedSignals) ...
    RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvConstants_Test_Buy);
RL_OptimalExecution_Testing_Environment_Buy = ...
    rlFunctionEnv(ObservationInfo_Train_Buy,ActionInfo_Buy, ...
        StepFunction_Test_Buy,ResetFunction_Test_Buy)

```

Reset!
Reset!

```

RL_OptimalExecution_Testing_Environment_Buy =
    rlFunctionEnv with properties:

```

```

    StepFcn: @(Action,LoggedSignals)RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvCon

```

```
ResetFcn: @()RL_OptimalExecution_LOB_ResetFcn(EnvConstants_Test_Buy)
Info: [1x1 struct]
```

Compute IS using the trained agent and testing data for buy trades.

```
simOptions = rlSimulationOptions(MaxSteps=NumTestingSteps);
experience_Buy = sim(RL_OptimalExecution_Testing_Environment_Buy,Trained_Agent_Buy,simOptions);
```

```
Reset!
Last Horizon. Episode is Done!
HorizonIdx:
```

```
ans =
97
```

```
SimAction_Test_Buy = squeeze(experience_Buy.Action.TradingAction_NumberOfShares_.Data);
NumSimSteps = length(SimAction_Test_Buy);
SimInventory_Test_Buy = nan(NumSimSteps,1);
SimExecutedShares_Test_Buy = SimInventory_Test_Buy;
SimReward_Test_Buy = nan(NumSimSteps,1);
reset(RL_OptimalExecution_Testing_Environment_Buy);
```

```
Reset!
```

```
for k=1:NumSimSteps
    [NextObs1,SimReward_Test_Buy(k),IsDone1,LoggedSignals] = step( ...
        RL_OptimalExecution_Testing_Environment_Buy,SimAction_Test_Buy(k));
    SimInventory_Test_Buy(k) = LoggedSignals.CurrentInventoryShares;
    SimExecutedShares_Test_Buy(k) = sum(LoggedSignals.ExecutedShares);
end
```

```
Last Horizon. Episode is Done!
HorizonIdx:
```

```
ans =
97
```

```
IS_Agent_Step_Test_Buy = LoggedSignals.IS_Agent;
IS_Agent_Horizon_Test_Buy = IS_Agent_Step_Test_Buy( ...
    NumIntervalsPerHorizon:NumIntervalsPerHorizon:NumTestingSteps);
```

Display the total Implementation Shortfall (IS) outperformance of the agent over TWAP for training data.

```
Total_Outperformance_Train_Buy = sum(IS_TWAP_Horizon_Train_Buy - IS_Agent_Horizon_Train_Buy);
table(Total_Outperformance_Train_Buy)
```

```
ans=table
    Total_Outperformance_Train_Buy
```

```
326.25
```

Display total-gain-to-loss ratio (TGLR) and gain-to-loss ratio (GLR) for the training data.

```
[TGLR_Train_Buy, GLR_Train_Buy] = ...
    ISTGLR(IS_TWAP_Horizon_Train_Buy - IS_Agent_Horizon_Train_Buy);
table(TGLR_Train_Buy, GLR_Train_Buy)
```

```
ans=1x2 table
    TGLR_Train_Buy    GLR_Train_Buy
    _____    _____
           1.4146           1.2587
```

Display total IS outperformance of the agent over TWAP for the testing data.

```
Total_Outperformance_Test_Buy = sum(IS_TWAP_Horizon_Test_Buy - IS_Agent_Horizon_Test_Buy);
table(Total_Outperformance_Test_Buy)
```

```
ans=table
    Total_Outperformance_Test_Buy
    _____
           196.57
```

Display TGLR and GLR for the testing data.

```
[TGLR_Test_Buy, GLR_Test_Buy] = ...
    ISTGLR(IS_TWAP_Horizon_Test_Buy - IS_Agent_Horizon_Test_Buy);
table(TGLR_Test_Buy, GLR_Test_Buy)
```

```
ans=1x2 table
    TGLR_Test_Buy    GLR_Test_Buy
    _____    _____
           2.018           1.4641
```

References

- [1] Bertsimas, Dimitris, and Andrew W. Lo. "Optimal Control of Execution Costs." *Journal of Financial Markets* 1, no. 1 (1998): 1-50.
- [2] Almgren, Robert, and Neil Chriss. "Optimal Execution of Portfolio Transactions." *Journal of Risk* 3, no. 2 (2000): 5-40.
- [3] Nevmyvaka, Yuriy, Yi Feng, and Michael Kearns. "Reinforcement Learning for Optimized Trade Execution." In *Proceedings of the 23rd International Conference on Machine Learning*, pp. 673-680. 2006.
- [4] Ning B., F. Lin, and S. Jaimungal. "Double Deep Q-Learning for Optimal Execution." Preprint, submitted June 8, 2020. Available at <https://arxiv.org/abs/1812.06600>.
- [5] Lin S. and P. A. Beling. "Optimal Liquidation with Deep Reinforcement Learning." *33rd Conference on Neural Information Processing Systems (NeurIPS 2019) Deep Reinforcement Learning Workshop*. Vancouver, Canada, 2019.

[6] Lin S. and P. A. Beling. "An End-to-End Optimal Trade Execution Framework Based on Proximal Policy Optimization." *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI 2020) Special Track on AI in FinTech*. 4548-4554. 2020.

[7] LOBSTER Limit Order Book Data. Berlin: frischdaten UG (haftungsbeschränkt). Available at <https://lobsterdata.com/>.

Local Functions

```
function [IS_TWAP_Horizon,IS_TWAP_Step,InventorySharesStepTWAP,TradingSharesStepTWAP] = ...
    executeTWAPTrades(InputLOB,NumLevels,TotalTradingShares,NumIntervalsPerHorizon,NumHorizons,B
% executeTWAPTrades Function for Executing Trades for TWAP Baseline
% This function executes buy or sell trades for the TWAP baseline and
% computes the corresponding implementation shortfalls, inventory shares
% and trading shares at each step.

NumSteps = NumHorizons*NumIntervalsPerHorizon;
IS_TWAP_Horizon = nan(NumHorizons,1);
IS_TWAP_Step = nan(NumSteps,1);
TradingSharesStepTWAP = round(TotalTradingShares./NumIntervalsPerHorizon).*ones(NumSteps,1);

if BuyTrade
    InventorySharesStepTWAP = zeros(NumSteps,1);
else
    InventorySharesStepTWAP = zeros(NumSteps,1) + TotalTradingShares;
end

for HorizonIdx = 1:NumHorizons
    HorizonExecutedPrices = zeros(NumLevels*NumIntervalsPerHorizon,1);
    HorizonExecutedShares = HorizonExecutedPrices;
    for IntervalIdx = 1:NumIntervalsPerHorizon
        CurrentBook = InputLOB(IntervalIdx + NumIntervalsPerHorizon*(HorizonIdx-1),:);

        % Execute trade
        if IntervalIdx==NumIntervalsPerHorizon
            if BuyTrade
                % Enforce target ending inventory constraint for buy trade (buy missing shares)
                CurrentTradingShares = max(TotalTradingShares - ...
                    InventorySharesStepTWAP(IntervalIdx + NumIntervalsPerHorizon.*(HorizonIdx-1)
            else
                % Enforce zero ending inventory constraint for sell trade (sell remaining shares)
                CurrentTradingShares = max(InventorySharesStepTWAP(IntervalIdx + ...
                    NumIntervalsPerHorizon.*(HorizonIdx-1)), 0);
            end
        else
            CurrentTradingShares = TradingSharesStepTWAP(IntervalIdx + ...
                NumIntervalsPerHorizon.*(HorizonIdx-1));
        end

        [ExecutedShares,ExecutedPrices,InitialPrice,TradedLevels] = ...
            tradeLOB(CurrentBook,CurrentTradingShares,BuyTrade);
        if IntervalIdx==1
            ArrivalPrice = InitialPrice;
        end
        HorizonExecutedShares((1:TradedLevels) + (IntervalIdx-1)*NumLevels) = ExecutedShares;
        HorizonExecutedPrices((1:TradedLevels) + (IntervalIdx-1)*NumLevels) = ExecutedPrices;
```

```

    if BuyTrade
        InventorySharesStepTWAP((IntervalIdx:NumIntervalsPerHorizon) + ...
            NumIntervalsPerHorizon.*(HorizonIdx-1)) = ...
            InventorySharesStepTWAP(IntervalIdx + ...
                NumIntervalsPerHorizon.*(HorizonIdx-1)) + sum(ExecutedShares);
    else
        InventorySharesStepTWAP((IntervalIdx:NumIntervalsPerHorizon) + ...
            NumIntervalsPerHorizon.*(HorizonIdx-1)) = ...
            InventorySharesStepTWAP(IntervalIdx + ...
                NumIntervalsPerHorizon.*(HorizonIdx-1)) - sum(ExecutedShares);
    end

    % Compute Implementation Shortfall for each Step
    IS_TWAP_Step(IntervalIdx + NumIntervalsPerHorizon.*(HorizonIdx-1)) = ...
        endingInventoryIS(ArrivalPrice,HorizonExecutedPrices,HorizonExecutedShares,BuyTrade)
end

% Compute Implementation Shortfall for each Horizon
IS_TWAP_Horizon(HorizonIdx) = endingInventoryIS(ArrivalPrice, ...
    HorizonExecutedPrices,HorizonExecutedShares,BuyTrade);
end

end

function [InitialObservation,LoggedSignals] = RL_OptimalExecution_LOB_ResetFcn(EnvConstants)
% RL_OptimalExecution_LOB_ResetFcn is Reset Function for RL Agent
% Define Reset Function
%
% Inputs:
%
% EnvConstants: Structure with the following fields:
% - SampledBook: Limit order book sampled at time intervals
% - BuyTrade: Scalar logical indicating the trading direction
%           true: Buy
%           false: Sell
% - TotalTradingShares: Number of shares to trade over the horizon (scalar)
% - NumIntervalsPerHorizon: Number of intervals per horizon (scalar)
% - NumHorizons: Number of horizons in the data (scalar)
% - NumSteps: Number of steps in the data (scalar)
% - NumLevels: Number of levels in the limit order book (scalar)
% - IS_TWAP: Implementation shortfall for TWAP baseline (NumSteps-by-1 vector)
%
% Outputs:
%
% InitialObservation:
% - Remaining shares to be traded by agent
% - Remaining intervals (steps) in the current trading horizon
% - Implementation shortfall of agent
% - Price divergence times 100
%
% LoggedSignals: Structure with following fields:
% - IntervalIdx: Current time interval index (initial value: 0)
% - HorizonIdx: Current time horizon index (initial value: 0)
% - CurrentInventoryShares: Current number of shares in inventory
%   (Initial value: 0 for Buy trade)
%   (Initial value: TotalTradingShares for Sell trade)
% - ArrivalPrice: Arrival price at the beginning of horizon (initial value: 0)

```

```

% - ExecutedShares: Number of executed shares at each traded level (initial value: 0)
% - ExecutedPrices: Prices of executed shares at each traded level (initial value: 0)
% - HorizonExecutedShares: Executed shares at each level over the horizon
%   (Initial value: (NumLevels * NumIntervalsPerHorizon)-by-1 vector of zeros)
% - HorizonExecutedPrices: Prices of Executed shares at each level over the horizon
%   (Initial value: (NumLevels * NumIntervalsPerHorizon)-by-1 vector of zeros)
% - IS_Agent: Implementation shortfall history of agent
%   (Initial value: NumIntervals-by-1 vector of zeros)
% - Reward: Reward history of agent
%   (Initial value: NumIntervals-by-1 vector of zeros)
% - NumIntervalsPerHorizon: Number of intervals per horizon (fixed value: NumIntervalsPerHorizon)
% - NumHorizons: Number of horizons (fixed value: NumHorizon)
% - NumLevels: Number of levels in the limit order book (fixed value: NumLevels)

IntervalIdx = 0;
HorizonIdx = 0;

LoggedSignals.IntervalIdx = IntervalIdx;
LoggedSignals.HorizonIdx = HorizonIdx;

InitialObservation = ...
    [EnvConstants.TotalTradingShares EnvConstants.NumIntervalsPerHorizon 0 0];

if EnvConstants.BuyTrade
    LoggedSignals.CurrentInventoryShares = 0;
else
    LoggedSignals.CurrentInventoryShares = EnvConstants.TotalTradingShares;
end

LoggedSignals.ArrivalPrice = 0;
LoggedSignals.ExecutedShares = 0;
LoggedSignals.ExecutedPrices = 0;
LoggedSignals.HorizonExecutedShares = ...
    zeros(EnvConstants.NumLevels*EnvConstants.NumIntervalsPerHorizon,1);
LoggedSignals.HorizonExecutedPrices = LoggedSignals.HorizonExecutedShares;
LoggedSignals.IS_Agent = zeros(EnvConstants.NumSteps,1);
LoggedSignals.Reward = LoggedSignals.IS_Agent;
LoggedSignals.NumIntervalsPerHorizon = EnvConstants.NumIntervalsPerHorizon;
LoggedSignals.NumHorizons = EnvConstants.NumHorizons;
LoggedSignals.NumLevels = EnvConstants.NumLevels;

disp("Reset!")
end

function [Observation,Reward,IsDone,LoggedSignals] = ...
    RL_OptimalExecution_LOB_StepFcn(Action,LoggedSignals,EnvConstants)
% RL_OptimalExecution_LOB_StepFcn function is Step Function for RL Environment
%   Given current Action and LoggedSignals, update Observation, Reward, and
%   LoggedSignals, and indicate whether the episode is complete.
%
% Inputs:
%
% Action: Number of shares to be traded at the current step (scalar)
%
% LoggedSignals: Structure with following fields
% - IntervalIdx: Current time interval index
% - HorizonIdx: Current time horizon index
% - CurrentInventoryShares: Current number of shares in inventory

```

```

% - ArrivalPrice: Arrival price at the beginning of horizon
% - ExecutedShares: Number of executed shares at each traded level
% - ExecutedPrices: Prices of executed shares at each traded level
% - HorizonExecutedShares: Executed shares at each level over the horizon
%   ((NumLevels * NumIntervalsPerHorizon)-by-1 vector)
% - HorizonExecutedPrices: Prices of Executed shares at each level over the horizon
%   ((NumLevels * NumIntervalsPerHorizon)-by-1 vector)
% - IS_Agent: Implementation shortfall history of agent
%   (NumIntervals-by-1 vector)
% - Reward: Reward history of agent
%   (NumIntervals-by-1 vector)
% - NumIntervalsPerHorizon: Number of intervals per horizon (fixed value: NumIntervalsPerHorizon)
% - NumHorizons: Number of horizons (fixed value: NumHorizon)
% - NumLevels: Number of levels in the limit order book (fixed value: NumLevels)
%
% EnvConstants: Structure with the following fields
% - SampledBook: Limit order book sampled at time intervals
% - BuyTrade: Scalar logical indicating the trading direction
%   true: Buy
%   false: Sell
% - TotalTradingShares: Number of shares to trade over the horizon (scalar)
% - NumIntervalsPerHorizon: Number of intervals per horizon (scalar)
% - NumHorizons: Number of horizons in the data (scalar)
% - NumSteps: Number of steps in the data (scalar)
% - NumLevels: Number of levels in the limit order book (scalar)
% - IS_TWAP: Implementation shortfall for TWAP baseline (NumSteps-by-1 vector)
%
% Outputs:
%
% Observation:
% - Remaining shares to be traded by agent
% - Remaining intervals (steps) in the current trading horizon
% - Implementation shortfall of agent
% - Price divergence times 100
%
% Reward: Reward for current step
%
% IsDone: Logical indicating whether the current episode is complete
%
% LoggedSignals: Updated LoggedSignals

% Update LoggedSignals indices
if (LoggedSignals.IntervalIdx == 0) && (LoggedSignals.HorizonIdx == 0)
    LoggedSignals.IntervalIdx = 1;
    LoggedSignals.HorizonIdx = 1;
elseif (LoggedSignals.IntervalIdx >= LoggedSignals.NumIntervalsPerHorizon)
    LoggedSignals.IntervalIdx = 1;
    LoggedSignals.HorizonIdx = LoggedSignals.HorizonIdx + 1;
    LoggedSignals.HorizonExecutedShares = ...
        zeros(EnvConstants.NumLevels*EnvConstants.NumIntervalsPerHorizon,1);
    LoggedSignals.HorizonExecutedPrices = LoggedSignals.HorizonExecutedShares;
    if EnvConstants.BuyTrade
        LoggedSignals.CurrentInventoryShares = 0;
    else
        LoggedSignals.CurrentInventoryShares = EnvConstants.TotalTradingShares;
    end
else
    LoggedSignals.IntervalIdx = LoggedSignals.IntervalIdx + 1;

```

```

end

StepIdx = LoggedSignals.IntervalIdx + ...
    LoggedSignals.NumIntervalsPerHorizon.*(LoggedSignals.HorizonIdx-1);
RemainingIntervals = LoggedSignals.NumIntervalsPerHorizon - LoggedSignals.IntervalIdx + 1;
IS_TWAP = EnvConstants.IS_TWAP(StepIdx);

if EnvConstants.BuyTrade
    AvgPrice = mean(EnvConstants.SampledBook(StepIdx,[1 5]),2);
    PriceDivergence = LoggedSignals.ArrivalPrice - AvgPrice;
else
    AvgPrice = mean(EnvConstants.SampledBook(StepIdx,[3 7]),2);
    PriceDivergence = AvgPrice - LoggedSignals.ArrivalPrice;
end

% Enforce ending inventory constraint
if LoggedSignals.IntervalIdx >= LoggedSignals.NumIntervalsPerHorizon
    if EnvConstants.BuyTrade
        TradingShares = max(EnvConstants.TotalTradingShares - ...
            LoggedSignals.CurrentInventoryShares, 0);
    else
        TradingShares = LoggedSignals.CurrentInventoryShares;
    end
else
    if EnvConstants.BuyTrade
        if PriceDivergence > 0
            AgentRemainingSharestoTrade = max(EnvConstants.TotalTradingShares - ...
                LoggedSignals.CurrentInventoryShares,0);
            TradingShares = round(AgentRemainingSharestoTrade./RemainingIntervals./2);
            TradingShares = min(TradingShares,Action);
        else
            TradingShares = min(EnvConstants.TotalTradingShares - ...
                LoggedSignals.CurrentInventoryShares, Action);
        end
    else
        if PriceDivergence > 0
            AgentRemainingSharestoTrade = max(LoggedSignals.CurrentInventoryShares, 0);
            TradingShares = round(AgentRemainingSharestoTrade./RemainingIntervals./2);
            TradingShares = min(TradingShares,Action);
        else
            TradingShares = min(LoggedSignals.CurrentInventoryShares, Action);
        end
    end
end

% Update IsDone:
% Episode is complete (IsDone is true) when time step reaches the end of
% the trading data.
if (LoggedSignals.HorizonIdx >= LoggedSignals.NumHorizons) && ...
    (LoggedSignals.IntervalIdx >= LoggedSignals.NumIntervalsPerHorizon)
    IsDone = true;
    disp("Last Horizon. Episode is Done!");
    disp("HorizonIdx:");
    LoggedSignals.HorizonIdx
elseif (LoggedSignals.IntervalIdx >= LoggedSignals.NumIntervalsPerHorizon)
    IsDone = false;
    % disp("HorizonIdx:");

```

```
% LoggedSignals.HorizonIdx
else
    IsDone = false;
end

% Execute trade
CurrentLOB = EnvConstants.SampledBook(LoggedSignals.IntervalIdx + ...
    LoggedSignals.NumIntervalsPerHorizon*(LoggedSignals.HorizonIdx-1),:);

[ExecutedShares,ExecutedPrices,InitialPrice,TradedLevels] = ...
    tradeLOB(CurrentLOB,TradingShares,EnvConstants.BuyTrade);

% Update Reward
if LoggedSignals.IntervalIdx==1
    LoggedSignals.ArrivalPrice = InitialPrice;
end

LoggedSignals.HorizonExecutedShares((1:TradedLevels) + ...
    (LoggedSignals.IntervalIdx-1)*LoggedSignals.NumLevels) = ExecutedShares;
LoggedSignals.HorizonExecutedPrices((1:TradedLevels) + ...
    (LoggedSignals.IntervalIdx-1)*LoggedSignals.NumLevels) = ExecutedPrices;

IS_Agent = endingInventoryIS(LoggedSignals.ArrivalPrice,LoggedSignals.HorizonExecutedPrices, ...
    LoggedSignals.HorizonExecutedShares,EnvConstants.BuyTrade);

if EnvConstants.BuyTrade
    AgentRemainingSharestoTrade = max(EnvConstants.TotalTradingShares - ...
        LoggedSignals.CurrentInventoryShares - sum(ExecutedShares),0);
else
    AgentRemainingSharestoTrade = max(LoggedSignals.CurrentInventoryShares - ...
        sum(ExecutedShares), 0);
end

[PenaltyAgentExecutedShares,PenaltyAgentExecutedPrices] = tradeLOB(CurrentLOB, ...
    round(AgentRemainingSharestoTrade./RemainingIntervals),EnvConstants.BuyTrade);

IS_Agent_Penalty = RemainingIntervals.*endingInventoryIS(LoggedSignals.ArrivalPrice, ...
    PenaltyAgentExecutedPrices,PenaltyAgentExecutedShares,EnvConstants.BuyTrade);

TWAPRemainingSharestoTrade = (EnvConstants.NumIntervalsPerHorizon - LoggedSignals.IntervalIdx)*
    EnvConstants.TotalTradingShares/EnvConstants.NumIntervalsPerHorizon;

[PenaltyTWAPExecutedShares,PenaltyTWAPExecutedPrices] = tradeLOB(CurrentLOB, ...
    round(TWAPRemainingSharestoTrade./RemainingIntervals),EnvConstants.BuyTrade);

IS_TWAP_Penalty = RemainingIntervals.*endingInventoryIS(LoggedSignals.ArrivalPrice, ...
    PenaltyTWAPExecutedPrices,PenaltyTWAPExecutedShares,EnvConstants.BuyTrade);

Reward = ((IS_TWAP + IS_TWAP_Penalty) - (IS_Agent + IS_Agent_Penalty)) .* ...
    LoggedSignals.IntervalIdx./ LoggedSignals.NumIntervalsPerHorizon;

% No reward if trading target already met
if EnvConstants.BuyTrade
    if LoggedSignals.CurrentInventoryShares >= EnvConstants.TotalTradingShares
        Reward = 0;
    end
else
```

```

        if LoggedSignals.CurrentInventoryShares <= 0
            Reward = 0;
        end
    end

    % Update LoggedSignals
    LoggedSignals.ExecutedPrices = ExecutedPrices;
    LoggedSignals.ExecutedShares = ExecutedShares;

    if EnvConstants.BuyTrade
        LoggedSignals.CurrentInventoryShares = ...
            LoggedSignals.CurrentInventoryShares + sum(ExecutedShares);
    else
        LoggedSignals.CurrentInventoryShares = ...
            max(LoggedSignals.CurrentInventoryShares - sum(ExecutedShares), 0);
    end

    % Update LoggedSignals.IS_Agent
    LoggedSignals.IS_Agent(StepIdx) = IS_Agent;

    % Update LoggedSignals.Reward
    LoggedSignals.Reward(StepIdx) = Reward;

    % Update Observation
    if LoggedSignals.IntervalIdx == LoggedSignals.NumIntervalsPerHorizon
        Observation = ...
            [EnvConstants.TotalTradingShares RemainingIntervals-1 IS_Agent PriceDivergence*100];
    else
        Observation = ...
            [AgentRemainingSharestoTrade RemainingIntervals-1 IS_Agent PriceDivergence*100];
    end
end

end

function [ExecutedShares,ExecutedPrices,InitialPrice,TradedLevels] = ...
    tradeLOB(CurrentLOB,TradingShares,BuyTrade)
% tradeLOB function executes market order trade based on current limit order book.
% This function computes the results of executing a market order trade
% for a specified number of shares and trade direction (buy/sell) based on
% the information available in the current snapshot of the limit order book.
%
% Inputs:
%
%     CurrentLOB - 1-by-(NumLevels*4) vector of current limit order book information
%
%         e.g. For NumLevels==5, 1-by-20 vector with the following values:
%         [AskPrice1 AskSize1 BidPrice1 BidSize1 ... AskPrice5 AskSize5 BidPrice5 BidSize5]
%
%         e.g. For NumLevels==10, 1-by-40 vector with the following values:
%         [AskPrice1 AskSize1 BidPrice1 BidSize1 ... AskPrice10 AskSize10 BidPrice10 BidSize10]
%
%     TradingShares - Scalar number of shares to be traded
%
%     BuyTrade - Scalar logical indicating trading direction:
%                 true: Buy
%                 false: Sell
%
%
```

```
% Outputs:
% ExecutedShares - Vector of number of executed shares at each traded level
% ExecutedPrices - Vector of number of executed prices at each traded level
% InitialPrice - Scalar numeric initial price
% TradedLevels - Scalar numeric traded levels

LOBLength = length(CurrentLOB);

% Separate Bid and Ask data
AskPriceIdx = (1:4:LOBLength);
AskSizeIdx = AskPriceIdx + 1;
BidPriceIdx = AskSizeIdx + 1;
BidSizeIdx = BidPriceIdx + 1;

AskPrices = CurrentLOB(AskPriceIdx)';
AskSizes = CurrentLOB(AskSizeIdx)';
BidPrices = CurrentLOB(BidPriceIdx)';
BidSizes = CurrentLOB(BidSizeIdx)';

if BuyTrade
    LOBPrices = AskPrices;
    LOBSizes = AskSizes;
else
    LOBPrices = BidPrices;
    LOBSizes = BidSizes;
end

InitialPrice = LOBPrices(1);

% Execute trade
CumulativeLOBSizes = cumsum(LOBSizes);
TradedLevels = sum(CumulativeLOBSizes <= TradingShares);
if TradedLevels < 1
    TradedLevels = 1;
elseif CumulativeLOBSizes(TradedLevels) < TradingShares
    TradedLevels = TradedLevels + 1;
end

ExecutedPrices = LOBPrices(1:TradedLevels);
ExecutedShares = LOBSizes(1:TradedLevels);
ExecutedShares(end) = ExecutedShares(end) - (sum(ExecutedShares) - TradingShares);
end

function outIS = endingInventoryIS(ArrivalPrice,ExecutedPrices,TradedVolumes,BuyTrade)
% EndingInventoryIS function for Implementation Shortfall given ending inventory constraints
% This function computes the Implementation Shortfall (IS) under the
% assumption of ending inventory constraints (i.e. no opportunity cost).
%
% For a Sell trade, the constraint is to have zero ending inventory
% assuming successful liquidation of all shares.
%
% For a Buy trade, the constraint is to have full ending inventory
% assuming successful purchase of all shares.
%
% Inputs:
%
% ArrivalPrice - Scalar price immediately before trade execution
```



```

%
%   ExecutedPrices - Vector of executed prices
%
%   TradedVolumes - Vector of executed trade volumes
%
%   BuyTrade - Scalar logical indicating trading direction:
%               true: Buy
%               false: Sell
%
%   Outputs:
%       outIS - Implementation shortfall

outIS = ArrivalPrice.*sum(TradedVolumes) - sum(ExecutedPrices.*TradedVolumes);

if BuyTrade
    outIS = -outIS;
end

end

function [TGLR, GLR] = ISTGLR(ISDiffValues)
% ISTGLR function for Total-Gain-to-Loss Ratio for Implementation Shortfalls (IS)
% This function computes the Total-Gain-to-Loss Ratio (TGLR) and the
% Gain-to-Loss Ratio (GLR) for a vector of values (IS_Baseline - IS_Agent)
% where:
%   - IS_Baseline: Implementation shortfalls for baseline (e.g. TWAP)
%   - IS_Agent: Implementation shortfalls for agent
%
%   Input:
%   - ISDiffValues: Vector of IS difference values (IS_Baseline - IS_Agent)
%
%   Outputs:
%   - TGLR: Total-Gain-to-Loss Ratio computed as the ratio of
%           Total Positive over Total Negative magnitudes of IS difference
%   - GLR: Gain-to-Loss Ratio computed as the ratio of
%           Mean Positive over Mean Negative magnitudes of IS difference

ISDiffValues = ISDiffValues(:);

ISGains = ISDiffValues(ISDiffValues>0);
ISLosses = ISDiffValues(ISDiffValues<0);

MeanGains = abs(mean(ISGains));
MeanLosses = abs(mean(ISLosses));

TGLR = sum(ISGains)./abs(sum(ISLosses));
GLR = MeanGains./MeanLosses;

end

```

See Also

Related Examples

- “Hedge Options Using Reinforcement Learning Toolbox” on page 10-40

- “Use Deep Learning to Approximate Barrier Option Prices with Heston Model” (Financial Instruments Toolbox)
- “Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data” on page 2-78

Backtest Investment Strategies Using datetime and calendarDuration

This example shows how to use `datetime` inputs in the backtesting workflow. Backtesting is a useful tool to compare how investment strategies perform over historical or simulated market data. Using `datetime` inputs allows you to accurately specify the exact days that you can then use during the investment period to obtain initial weights from a warm up period, to select a start date to begin the backtest, and to establish a rebalancing frequency.

Load Data

The backtesting framework requires adjusted asset prices, meaning prices adjusted for dividends, splits, or other events. The prices must be stored in a MATLAB® `timetable` with each column holding a time series of asset prices for an investable asset.

In this example, you use the closing prices of the Dow Jones Industrial Average for 2006.

```
% Read a table of daily adjusted close prices for 2006 DJIA stocks.
T = readtable('dowPortfolio.xlsx');
```

```
% For readability, use only 15 of the 30 DJI component stocks.
```

```
assetSymbols = ["AA", "CAT", "DIS", "GM", "HPQ", "JNJ", "MCD", "MMM", "MO", "MRK", "MSFT", "PFE", "PG", "T", "X"];
```

```
% Prune the table to hold only the dates and selected stocks.
```

```
timeColumn = "Dates";
```

```
T = T(:, [timeColumn assetSymbols]);
```

```
% Convert to the table to a timetable.
```

```
pricesTT = table2timetable(T, 'RowTimes', 'Dates');
```

```
% View the structure of the prices timetable.
```

```
head(pricesTT)
```

Dates	AA	CAT	DIS	GM	HPQ	JNJ	MCD	MMM	MO
03-Jan-2006	28.72	55.86	24.18	17.82	28.35	59.08	32.72	75.93	52.27
04-Jan-2006	28.89	57.29	23.77	18.3	29.18	59.99	33.01	75.54	52.65
05-Jan-2006	29.12	57.29	24.19	19.34	28.97	59.74	33.05	74.85	52.52
06-Jan-2006	29.02	58.43	24.52	19.61	29.8	60.01	33.25	75.47	52.95
09-Jan-2006	29.37	59.49	24.78	21.12	30.17	60.38	33.88	75.84	53.11
10-Jan-2006	28.44	59.25	25.09	20.79	30.33	60.49	33.91	75.37	53.04
11-Jan-2006	28.05	59.28	25.33	20.61	30.88	59.91	34.5	75.22	53.31
12-Jan-2006	27.68	60.13	25.41	19.76	30.57	59.63	33.96	74.57	53.23

Define the Strategies

You use `backtestStrategy` to define investment strategies that capture the logic used to make asset allocation decisions while a backtest is running. Each strategy is periodically given the opportunity to update its portfolio allocation. When you use `backtestStrategy`, you specify the allocation frequency with the `RebalanceFrequency` name-value argument. This argument can be an integer, a duration object, a `calendarDuration` object, or a `datetime` object.

This example uses two backtesting strategies:

- Equal-weighted portfolio

$$w_{EW} = (w_1, w_2, \dots, w_N), \quad w_i = \frac{1}{N}$$

- Maximum Sharpe ratio portfolio

$w_{SR} = \operatorname{argmax} \left\{ \frac{\mu^T w}{\sqrt{w^T Q w}} \mid w \geq 0, \sum_{i=1}^N w_i = 1 \right\}$, where μ an $N \times 1$ vector of returns and Q is the $N \times N$ covariance matrix.

Compute Initial Weights

By default, the initial weights allocate all capital to cash. To avoid this, the first two months (January and February) of the data are used to estimate the initial weights for the different strategies.

```
t0 = datetime(2006,1,1);
tEnd = datetime(2006,2,28);
warmupPeriod = t0:tEnd;
```

You calculate the initial weights by calling the `backtestStrategy` rebalance functions with only the dates in January and February.

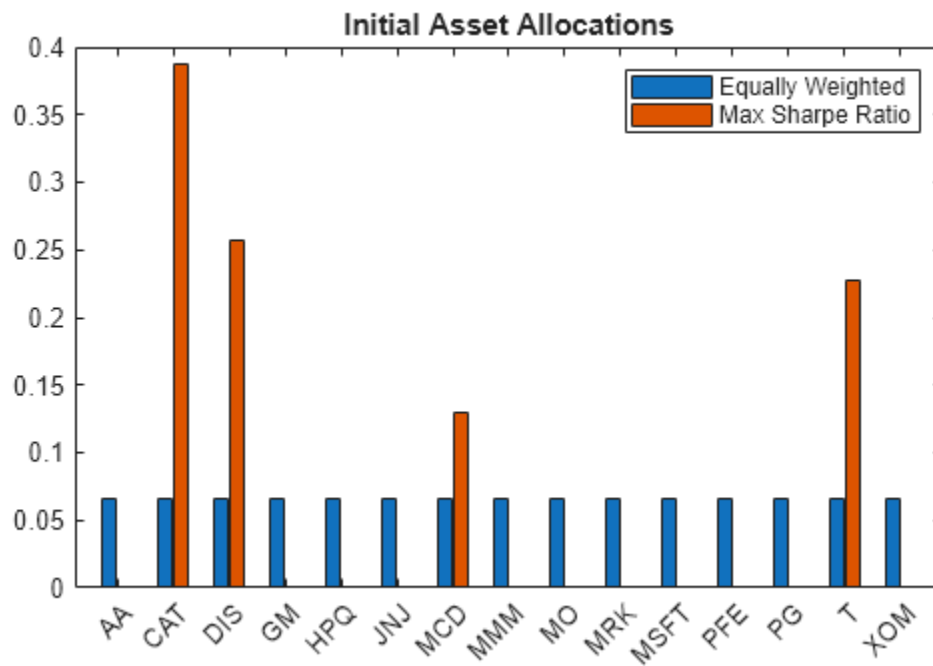
```
% No current weights (100% cash position).
numAssets = size(pricesTT,2);
current_weights = zeros(1,numAssets);

% Warm-up partition of data set timetable.
warmupTT = pricesTT(warmupPeriod,:);

% Compute the initial portfolio weights for each strategy.
equalWeight_initial = equalWeightFcn(current_weights,warmupTT);
maxSharpeRatio_initial = maxSharpeRatioFcn(current_weights,warmupTT);
```

Visualize the initial weight allocations from the strategies.

```
strategyNames = {'Equally Weighted', 'Max Sharpe Ratio'};
assetSymbols = pricesTT.Properties.VariableNames;
initialWeights = [equalWeight_initial(:), maxSharpeRatio_initial(:)];
bar(assetSymbols,initialWeights);
legend(strategyNames);
title('Initial Asset Allocations');
```



Create Backtest Strategies

In this example the rebalancing frequency of the strategies is set to the first available day of each month.

```
% Rebalance at the beginning of the month.
tEnd = datetime(2006,12,31);
rebalFreq = t0:calmonths(1):tEnd;
```

The lookback window defines the minimum and maximum amount of data to consider when rebalancing. In this example, the rebalance occurs if the backtest start time is at least 2 months prior to the rebalancing time, and it will not include prices older than 6 calendar months.

```
% Set the rolling lookback window to be at least 2 calendar months and at
% most 6 calendar months.
minLookback = calmonths(2);
maxLookback = calmonths(6);
lookback = [minLookback maxLookback];
```

```
% Use a fixed transaction cost (buy and sell costs are both 0.5% of amount
% traded).
transactionsFixed = 0.005;
```

```
% Define backtest strategies
strat1 = backtestStrategy('Equal Weighted', @equalWeightFcn, ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
    'TransactionCosts', transactionsFixed, ...
    'InitialWeights', equalWeight_initial);

strat2 = backtestStrategy('Max Sharpe Ratio', @maxSharpeRatioFcn, ...
    'RebalanceFrequency', rebalFreq, ...
    'LookbackWindow', lookback, ...
```

```
'TransactionCosts', transactionsFixed, ...  
'InitialWeights', maxSharpeRatio_initial);  
  
% Aggregate the strategy objects into an array.  
strategies = [strat1, strat2];
```

Run the Backtest

Set an annual risk-free rate of 1%.

```
% 1% annual risk-free rate  
annualRiskFreeRate = 0.01;
```

Create the `backtestEngine` object using the two previously defined strategies. If the rebalancing date is missing in the prices timetable (this usually happens when the day falls on a weekend or a holiday), rebalance on the next available date.

```
% Create the backtesting engine object  
backtester = backtestEngine(strategies, 'RiskFreeRate', annualRiskFreeRate, ...  
    'DateAdjustment', 'Next');
```

Use `runBacktest` to run the backtest. Since this example uses the first two months of data to find the initial weights, use the 'Start' name-value argument to initiate the backtesting after the warm-up period. This is done to avoid the look-ahead bias (that is, "seeing the future").

```
backtester = runBacktest(backtester, pricesTT, 'Start', warmupPeriod(end));
```

Examine Backtest Results

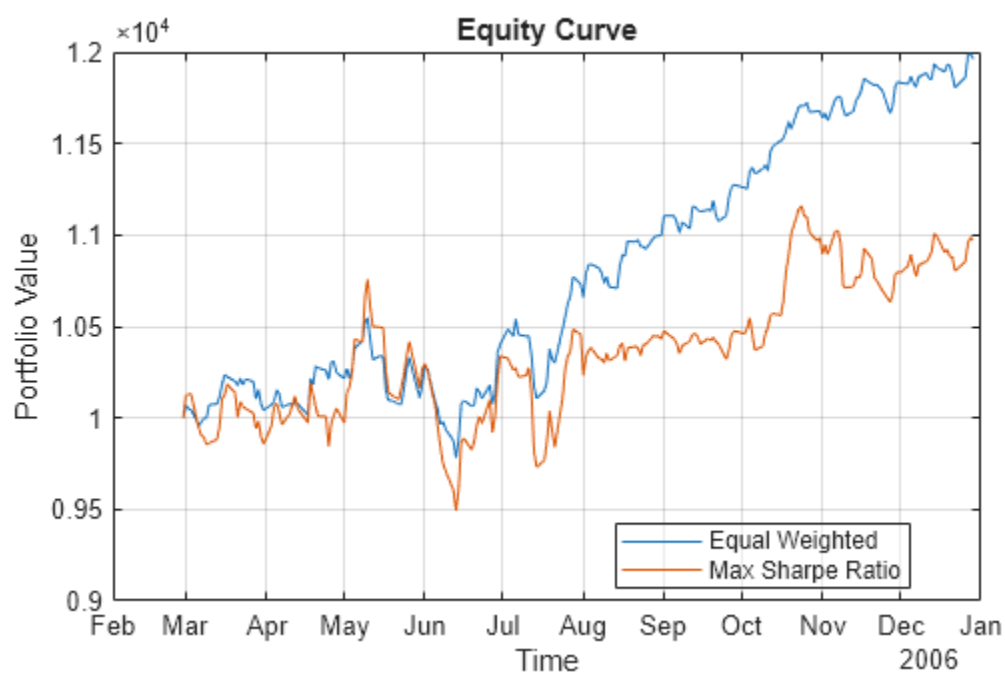
Use the `summary` function to generate a table of strategy performance results for the backtest.

```
summaryByStrategies = summary(backtester)
```

```
summaryByStrategies=9×2 table  
                        Equal_Weighted    Max_Sharpe_Ratio  
-----  
TotalReturn            0.19615           0.097125  
SharpeRatio            0.13059           0.04967  
Volatility             0.0063393        0.0088129  
AverageTurnover        0.00076232       0.012092  
MaxTurnover            0.030952         0.68245  
AverageReturn          0.00086517       0.00047599  
MaxDrawdown            0.072652         0.11764  
AverageBuyCost         0.04131         0.63199  
AverageSellCost        0.04131         0.63199
```

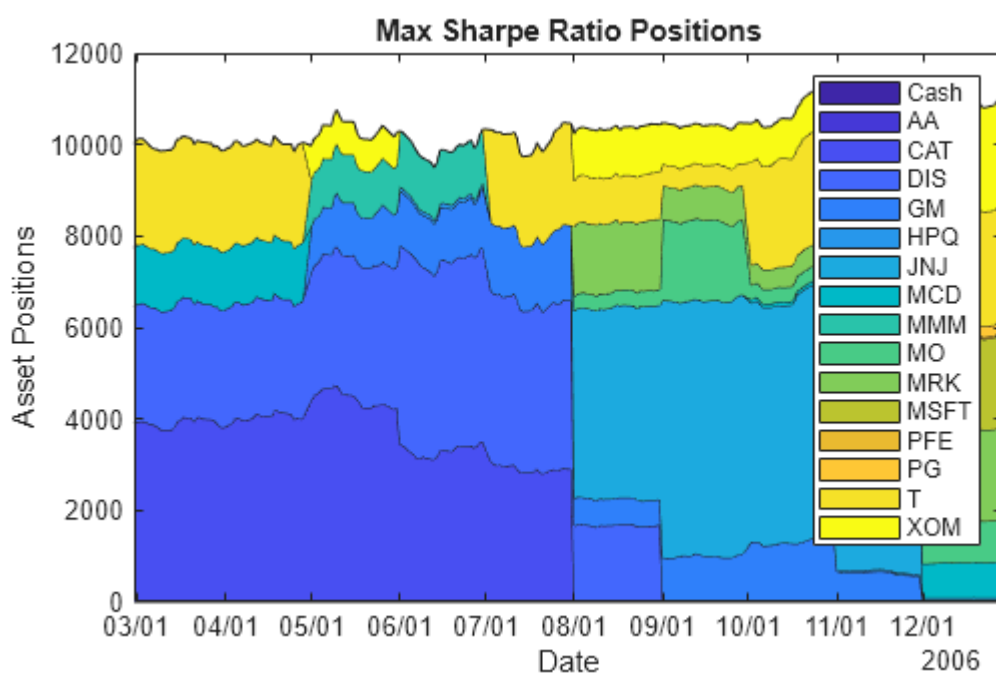
Use `equityCurve` to plot the equity curve for the different investment strategies.

```
equityCurve(backtester)
```



You can visualize the change in the strategy allocations over time using an area chart of the daily asset positions. For information on the `assetAreaPlot` function, see the Local Functions on page 4-456 section.

```
strategyName =  ;
assetAreaPlot(backtester, strategyName)
```



Local Functions

```
function new_weights = equalWeightFcn(~,pricesTT)
% Equal-weighted portfolio allocation

nAssets = size(pricesTT, 2);
new_weights = ones(1,nAssets);
new_weights = new_weights / sum(new_weights);

end

function new_weights = maxSharpeRatioFcn(~,pricesTT)
% Max Sharpe ratio portfolio allocation

nAssets = size(pricesTT, 2);
assetReturns = tick2ret(pricesTT);
% Max 25% into a single asset (including cash)
p = Portfolio('NumAssets',nAssets,...
    'LowerBound',0,'Budget',1);
p = estimateAssetMoments(p,assetReturns{:,:});
new_weights = estimateMaxSharpeRatio(p);

end

function assetAreaPlot(backtester,strategyName)
% Plot the asset allocation as an area plot.

t = backtester.Positions.(strategyName).Time;
positions = backtester.Positions.(strategyName).Variables;
h = area(t,positions);
title(sprintf('%s Positions',strrep(strategyName,'_',' ')));
xlabel('Date');
ylabel('Asset Positions');
xtickformat('MM/dd');
xlim([t(1) t(end)])
oldylim = ylim;
ylim([0 oldylim(2)]);
cm = parula(numel(h));
for i = 1:numel(h)
    set(h(i),'FaceColor',cm(i,:));
end
legend(backtester.Positions.(strategyName).Properties.VariableNames)

end
```


Adding Constraints to Satisfy UCITS Directive

This example shows how to set up and solve a portfolio optimization problem that satisfies the Undertakings for Collective Investment in Transferable Securities (UCITS) Directive. The UCITS directive is a regulatory requirement of European investment funds. As of 2019, the UCITS asset allocation regulation states that funds can only invest only up to 10% in a single issuer, and that investments in excess of 5% must not exceed 40% of the total portfolio, with some exceptions. This rule is known as the 5/10/40 rule.

Create Portfolio Object

To solve a portfolio problem and add this requirement to the problem, start by creating a `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` object, depending on the desired risk measure.

```
% Read table of daily adjusted close prices for 2006 DJI stocks
pricesTT = readtimetable('dowPortfolio.xlsx');
% Compute the returns from the prices
returnsTT = tick2ret(pricesTT);

% Remove the DJI stock from the timetable
assetReturnsTT = returnsTT(:,2:end);
% Compute the asset returns mean and covariance matrix
mu = mean(assetReturnsTT.Variables);
Sigma = cov(assetReturnsTT.Variables);
```

```
% Create Portfolio object
p = Portfolio(AssetMean=mu',AssetCovar=Sigma);
```

To add the 10% upper bound on the maximum that you can invest in any individual asset, use `setBounds`. In this example, any chosen asset must represent at least 1% of the total portfolio.

```
% Weights must be either 0 or between 0.01 and 0.1
p = setBounds(p,0.01,0.1,BoundType="conditional");
```

To set the conditional constraint that limits the aggregate weight that you can invest in assets that exceed 5%, use `setConditionalBudget`.

```
% Assets with weights above 5% must not exceed 40% of the total
% portfolio
p = setConditionalBudget(p,0.05,0.4);
```

Finally, the portfolio weights must sum to one and must have between 15 and 20 active assets.

```
% Sum of weights must be equal to one
p = setBudget(p,1,1);

% 15-20 active assets
p = setMinNumAssets(p,15,20);
```

Compute Minimum Variance Portfolio with Tracking Error Penalty

Given some `trackingPortfolio` weights allocation, find the minimum variance portfolio using `estimateCustomObjectivePortfolio` with a tracking error penalty that satisfies all the constraints in the previous section.

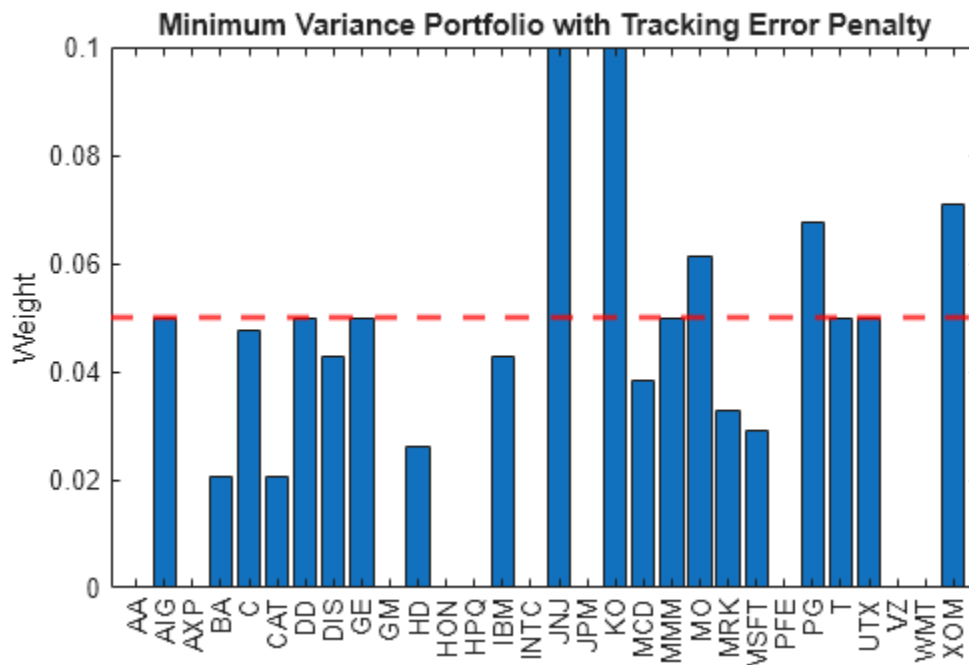
```
% Define tracking portfolio as the portfolio of the DJI
trackingPortfolio = returnsTT(:,2:end)\returnsTT(:,1);
```

```
% Penalized objective function
penalizedVariance = @(w) w'*Sigma*w + ...
    (w-trackingPortfolio)*Sigma*(w-trackingPortfolio);

% Penalized variance portfolio
p = setSolverMINLP(p,"OuterApproximation",ExtendedFormulation=true, ...
    ObjectiveScalingFactor=1e4);
w = estimateCustomObjectivePortfolio(p,penalizedVariance);
```

The following bar chart shows the weights distribution of the portfolio.

```
figure
bar(assetReturnsTT.Properties.VariableNames,w);
hold on
yline(0.05,'r--',LineWidth=2)
title('Minimum Variance Portfolio with Tracking Error Penalty')
ylabel('Weight')
```



Only five assets are above the 5% conditional threshold and the sum of these assets are less than 40%.

```
conditionalBudget = sum(w(w > 0.05))
```

```
conditionalBudget =
0.4000
```

See Also

setConditionalBudget | setBounds

CVaR Portfolio Optimization Tools

- “Portfolio Optimization Theory” on page 5-3
- “Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object” on page 5-8
- “Default Portfolio Problem” on page 5-15
- “PortfolioCVaR Object Workflow” on page 5-16
- “PortfolioCVaR Object” on page 5-17
- “Creating the PortfolioCVaR Object” on page 5-22
- “Common Operations on the PortfolioCVaR Object” on page 5-29
- “Setting Up an Initial or Current Portfolio” on page 5-33
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Working with a Riskless Asset” on page 5-46
- “Working with Transaction Costs” on page 5-47
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Working with ‘Simple’ Bound Constraints Using PortfolioCVaR Object” on page 5-55
- “Working with Budget Constraints Using PortfolioCVaR Object” on page 5-58
- “Working with Conditional Budget Constraints Using PortfolioCVaR Object” on page 5-60
- “Working with Group Constraints Using PortfolioCVaR Object” on page 5-62
- “Working with Group Ratio Constraints Using PortfolioCVaR Object” on page 5-65
- “Working with Linear Equality Constraints Using PortfolioCVaR Object” on page 5-68
- “Working with Linear Inequality Constraints Using PortfolioCVaR Object” on page 5-70
- “Working with ‘Conditional’ BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioCVaR Objects” on page 5-72
- “Working with Average Turnover Constraints Using PortfolioCVaR Object” on page 5-75
- “Working with One-Way Turnover Constraints Using PortfolioCVaR Object” on page 5-78
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Obtaining Portfolios Along the Entire Efficient Frontier” on page 5-86
- “Obtaining Endpoints of the Efficient Frontier” on page 5-89
- “Obtaining Efficient Portfolios for Target Returns” on page 5-92
- “Obtaining Efficient Portfolios for Target Risks” on page 5-95
- “Choosing and Controlling the Solver for PortfolioCVaR Optimizations” on page 5-98
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Plotting the Efficient Frontier for a PortfolioCVaR Object” on page 5-109
- “Postprocessing Results to Set Up Tradable Portfolios” on page 5-115
- “Working with Other Portfolio Objects” on page 5-118
- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121

- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137
- “Mixed-Integer CVaR Portfolio Optimization Problem” on page 5-141
- “Bond Portfolio CVaR Optimization Using Diebold-Li Model” on page 5-146

Portfolio Optimization Theory

In this section...

“Portfolio Optimization Problems” on page 5-3
 “Portfolio Problem Specification” on page 5-3
 “Return Proxy” on page 5-4
 “Risk Proxy” on page 5-5

Portfolio Optimization Problems

Portfolio optimization problems involve identifying portfolios that satisfy three criteria:

- Minimize a proxy for risk.
- Match or exceed a proxy for return.
- Satisfy basic feasibility requirements.

Portfolios are points from a feasible set of assets that constitute an asset universe. A portfolio specifies either holdings or weights in each individual asset in the asset universe. The convention is to specify portfolios in terms of weights, although the portfolio optimization tools work with holdings as well.

The set of feasible portfolios is necessarily a nonempty, closed, and bounded set. The proxy for risk is a function that characterizes either the variability or losses associated with portfolio choices. The proxy for return is a function that characterizes either the gross or net benefits associated with portfolio choices. The terms “risk” and “risk proxy” and “return” and “return proxy” are interchangeable. The fundamental insight of Markowitz (see “Portfolio Optimization” on page A-5) is that the goal of the portfolio choice problem is to seek minimum risk for a given level of return and to seek maximum return for a given level of risk. Portfolios satisfying these criteria are efficient portfolios and the graph of the risks and returns of these portfolios forms a curve called the efficient frontier.

Portfolio Problem Specification

To specify a portfolio optimization problem, you need the following:

- Proxy for portfolio return (μ)
- Proxy for portfolio risk (σ)
- Set of feasible portfolios (X), called a portfolio set

Financial Toolbox has three objects to solve specific types of portfolio optimization problems:

- The `Portfolio` object supports mean-variance portfolio optimization (see Markowitz [46], [47] at “Portfolio Optimization” on page A-5). This object has either gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set.
- The `PortfolioCVaR` object implements what is known as conditional value-at-risk portfolio optimization (see Rockafellar and Uryasev [48], [49] at “Portfolio Optimization” on page A-5), which is generally referred to as CVaR portfolio optimization. CVaR portfolio optimization works

with the same return proxies and portfolio sets as mean-variance portfolio optimization but uses conditional value-at-risk of portfolio returns as the risk proxy.

- The `PortfolioMAD` object implements what is known as mean-absolute deviation portfolio optimization (see Konno and Yamazaki [50] at “Portfolio Optimization” on page A-5), which is referred to as MAD portfolio optimization. MAD portfolio optimization works with the same return proxies and portfolio sets as mean-variance portfolio optimization but uses mean-absolute deviation portfolio returns as the risk proxy.

Return Proxy

The proxy for portfolio return is a function $\mu: X \rightarrow R$ on a portfolio set $X \subset R^n$ that characterizes the rewards associated with portfolio choices. Usually, the proxy for portfolio return has two general forms, gross and net portfolio returns. Both portfolio return forms separate the risk-free rate r_0 so that the portfolio $x \in X$ contains only risky assets.

Regardless of the underlying distribution of asset returns, a collection of S asset returns $y_1, \dots, y_S$ has a mean of asset returns

$$m = \frac{1}{S} \sum_{s=1}^S y_s,$$

and (sample) covariance of asset returns

$$C = \frac{1}{S-1} \sum_{s=1}^S (y_s - m)(y_s - m)^T.$$

These moments (or alternative estimators that characterize these moments) are used directly in mean-variance portfolio optimization to form proxies for portfolio risk and return.

Gross Portfolio Returns

The gross portfolio return for a portfolio $x \in X$ is

$$\mu(x) = r_0 + (m - r_0 \mathbf{1})^T x,$$

where:

r_0 is the risk-free rate (scalar).

m is the mean of asset returns (n vector).

If the portfolio weights sum to 1, the risk-free rate is irrelevant. The properties in the `Portfolio` object to specify gross portfolio returns are:

- `RiskFreeRate` for r_0
- `AssetMean` for m

Net Portfolio Returns

The net portfolio return for a portfolio $x \in X$ is

$$\mu(x) = r_0 + (m - r_0 \mathbf{1})^T x - b^T \max\{0, x - x_0\} - s^T \max\{0, x_0 - x\},$$

where:

r_0 is the risk-free rate (scalar).

m is the mean of asset returns (n vector).

b is the proportional cost to purchase assets (n vector).

s is the proportional cost to sell assets (n vector).

You can incorporate fixed transaction costs in this model also. Though in this case, it is necessary to incorporate prices into such costs. The properties in the `Portfolio` object to specify net portfolio returns are:

- `RiskFreeRate` for r_0
- `AssetMean` for m
- `InitPort` for x_0
- `BuyCost` for b
- `SellCost` for s

Risk Proxy

The proxy for portfolio risk is a function $\sigma: X \rightarrow R$ on a portfolio set $X \subset R^n$ that characterizes the risks associated with portfolio choices.

Variance

The variance of portfolio returns for a portfolio $x \in X$ is

$$\text{Variance}(x) = x^T C x$$

where C is the covariance of asset returns (n -by- n positive-semidefinite matrix). Covariance is a measure of the degree to which returns on two assets move in tandem. A positive covariance means that asset returns move together; a negative covariance means they vary inversely.

The property in the `Portfolio` object to specify the variance of portfolio returns is `AssetCovar` for C .

Although the risk proxy in mean-variance portfolio optimization is the variance of portfolio returns, the square root, which is the standard deviation of portfolio returns, is often reported and displayed. Moreover, this quantity is often called the “risk” of the portfolio. For details, see Markowitz (“Portfolio Optimization” on page A-5).

Conditional Value-at-Risk

The conditional value-at-risk for a portfolio $x \in X$, which is also known as expected shortfall, is defined as

$$CVaR_\alpha(x) = \frac{1}{1-\alpha} \int_{f(x,y) \geq VaR_\alpha(x)} f(x,y)p(y)dy,$$

where:

α is the probability level such that $0 < \alpha < 1$.

$f(x, y)$ is the loss function for a portfolio x and asset return y .

$p(y)$ is the probability density function for asset return y .

VaR_α is the value-at-risk of portfolio x at probability level α .

The value-at-risk is defined as

$$VaR_\alpha(x) = \min\{y: \Pr[f(x, Y) \leq y] \geq \alpha\}.$$

An alternative formulation for CVaR has the form:

$$CVaR_\alpha(x) = VaR_\alpha(x) + \frac{1}{1-\alpha} \int_{R^n} \max\{0, (f(x, y) - VaR_\alpha(x))\} p(y) dy$$

The choice for the probability level α is typically 0.9 or 0.95. Choosing α implies that the value-at-risk $VaR_\alpha(x)$ for portfolio x is the portfolio return such that the probability of portfolio returns falling below this level is $(1 - \alpha)$. Given $VaR_\alpha(x)$ for a portfolio x , the conditional value-at-risk of the portfolio is the expected loss of portfolio returns above the value-at-risk return.

Note Value-at-risk is a positive value for losses so that the probability level α indicates the probability that portfolio returns are below the negative of the value-at-risk.

To describe the probability distribution of returns, the `PortfolioCVaR` object takes a finite sample of return scenarios y_s , with $s = 1, \dots, S$. Each y_s is an n vector that contains the returns for each of the n assets under the scenario s . This sample of S scenarios is stored as a scenario matrix of size S -by- n . Then, the risk proxy for CVaR portfolio optimization, for a given portfolio $x \in X$ and $\alpha \in (0, 1)$, is computed as

$$CVaR_\alpha(x) = VaR_\alpha(x) + \frac{1}{(1-\alpha)S} \sum_{s=1}^S \max\{0, -y_s^T x - VaR_\alpha(x)\}$$

The value-at-risk, $VaR_\alpha(x)$, is estimated whenever the CVaR is estimated. The loss function is $f(x, y_s) = -y_s^T x$, which is the portfolio loss under scenario s .

Under this definition, VaR and CVaR are sample estimators for VaR and CVaR based on the given scenarios. Better scenario samples yield more reliable estimates of VaR and CVaR.

For more information, see Rockafellar and Uryasev [48], [49], and Cornuejols and Tütüncü, [51], at “Portfolio Optimization” on page A-5.

Mean Absolute-Deviation

The mean-absolute deviation (MAD) for a portfolio $x \in X$ is defined as

$$MAD(x) = \frac{1}{S} \sum_{s=1}^S |(y_s - m)^T x|$$

where:

y_s are asset returns with scenarios $s = 1, \dots, S$ (S collection of n vectors).

$f(x, y)$ is the loss function for a portfolio x and asset return y .

m is the mean of asset returns (n vector).

such that

$$m = \frac{1}{S} \sum_{s=1}^S y_s$$

For more information, see Konno and Yamazaki [50] at “Portfolio Optimization” on page A-5.

See Also

Portfolio | PortfolioCVaR | PortfolioMAD

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51

More About

- “PortfolioCVaR Object” on page 5-17
- Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8
- “Default Portfolio Problem” on page 5-15
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object

The final element for a complete specification of a portfolio optimization problem is the set of feasible portfolios, which is called a portfolio set. A portfolio set $X \subset R^n$ is specified by construction as the intersection of sets formed by a collection of constraints on portfolio weights. A portfolio set necessarily and sufficiently must be a nonempty, closed, and bounded set.

When setting up your portfolio set, ensure that the portfolio set satisfies these conditions. The most basic or “default” portfolio set requires portfolio weights to be nonnegative (using the lower-bound constraint) and to sum to 1 (using the budget constraint). The most general portfolio set handled by the portfolio optimization tools can have any of these constraints and which are properties for the PortfolioCVaR object:

- Linear inequality constraints
- Linear equality constraints
- 'Simple' Bound constraints
- 'Conditional' Bound constraints
- Budget constraints
- Conditional budget constraints
- Group constraints
- Group ratio constraints
- Average turnover constraints
- One-way turnover constraints
- Cardinality constraints

Linear Inequality Constraints

Linear inequality constraints are general linear constraints that model relationships among portfolio weights that satisfy a system of inequalities. Linear inequality constraints take the form

$$A_I x \leq b_I$$

where:

x is the portfolio (n vector).

A_I is the linear inequality constraint matrix (n_I -by- n matrix).

b_I is the linear inequality constraint vector (n_I vector).

n is the number of assets in the universe and n_I is the number of constraints.

PortfolioCVaR object properties to specify linear inequality constraints are:

- `AInequality` for A_I
- `bInequality` for b_I
- `NumAssets` for n

The default is to ignore these constraints.

Linear Equality Constraints

Linear equality constraints are general linear constraints that model relationships among portfolio weights that satisfy a system of equalities. Linear equality constraints take the form

$$A_E x = b_E$$

where:

x is the portfolio (n vector).

A_E is the linear equality constraint matrix (n_E -by- n matrix).

b_E is the linear equality constraint vector (n_E vector).

n is the number of assets in the universe and n_E is the number of constraints.

PortfolioCVaR object properties to specify linear equality constraints are:

- `AEquality` for A_E
- `bEquality` for b_E
- `NumAssets` for n

The default is to ignore these constraints.

'Simple' Bound Constraints

'Simple' Bound constraints are specialized linear constraints that confine portfolio weights to fall either above or below specific bounds. Since every portfolio set must be bounded, it is often a good practice, albeit not necessary, to set explicit bounds for the portfolio problem. To obtain explicit bounds for a given portfolio set, use the `estimateBounds` function. Bound constraints take the form

$$l_B \leq x \leq u_B$$

where:

x is the portfolio (n vector).

l_B is the lower-bound constraint (n vector).

u_B is the upper-bound constraint (n vector).

n is the number of assets in the universe.

PortfolioCVaR object properties to specify bound constraints are:

- `LowerBound` for l_B
- `UpperBound` for u_B
- `NumAssets` for n

The default is to ignore these constraints.

The default portfolio optimization problem (see “Default Portfolio Problem” on page 5-15) has $l_B = 0$ with u_B set implicitly through a budget constraint.

'Conditional' Bound Constraints

'Conditional' Bound constraints, also called semicontinuous constraints, are mixed-integer linear constraints that confine portfolio weights to fall either above or below specific bounds *if* the asset is selected; otherwise, the value of the asset is zero. Use `setBounds` to specify bound constraints with a 'Conditional' `BoundType`. To mathematically formulate this type of constraints, a binary variable v_i is needed. $v_i = 0$ indicates that asset i is not selected and v_i indicates that the asset was selected. Thus

$$l_i v_i \leq x_i \leq u_i v_i$$

where

x is the portfolio (n vector).

l is the conditional lower-bound constraint (n vector).

u is the conditional upper-bound constraint (n vector).

n is the number of assets in the universe.

`PortfolioCVaR` object properties to specify the bound constraint are:

- `LowerBound` for l_B
- `UpperBound` for u_B
- `NumAssets` for n

The default is to ignore this constraint.

Budget Constraints

Budget constraints are specialized linear constraints that confine the sum of portfolio weights to fall either above or below specific bounds. The constraints take the form

$$l_S \leq 1^T x \leq u_S$$

where:

x is the portfolio (n vector).

1 is the vector of ones (n vector).

l_S is the lower-bound budget constraint (scalar).

u_S is the upper-bound budget constraint (scalar).

n is the number of assets in the universe.

`PortfolioCVaR` object properties to specify budget constraints are:

- LowerBudget for l_S
- UpperBudget for u_S
- NumAssets for n

The default is to ignore this constraint.

The default portfolio optimization problem (see “Default Portfolio Problem” on page 5-15) has $l_S = u_S = 1$, which means that the portfolio weights sum to 1. If the portfolio optimization problem includes possible movements in and out of cash, the budget constraint specifies how far portfolios can go into cash. For example, if $l_S = 0$ and $u_S = 1$, then the portfolio can have 0-100% invested in cash. If cash is to be a portfolio choice, set RiskFreeRate (r_0) to a suitable value (see “Portfolio Problem Specification” on page 5-3 and “Working with a Riskless Asset” on page 5-46).

Conditional Budget Constraints

Conditional budget constraints is a logical constraint that bounds the maximum amount of the portfolio that can be invested in assets exceeding a given threshold. This constraint supports the “Undertakings for Collective Investment in Transferable Securities” on page 15-1452 (UCITS) directive. Use `setConditionalBudget` to set conditional budget constraints. To mathematically formulate this type of constraints, a binary variable w_i is needed. If $w_i = 0$ the asset is below the specified threshold, otherwise the asset is above that threshold which is represented as

$$w_i = 0 \rightarrow x_i \leq \alpha_i$$

$$\sum_i x_i w_i \leq \gamma$$

where:

x is an n vector representing the portfolio.

α is a scalar or n representing the threshold above which assets are accounted for in the conditional budget (scalar or n vector).

γ is a scalar representing the maximum amount of the portfolio that can be invested in assets exceeding α .

Group Constraints

Group constraints are specialized linear constraints that enforce “membership” among groups of assets. The constraints take the form

$$l_G \leq Gx \leq u_G$$

where:

x is the portfolio (n vector).

l_G is the lower-bound group constraint (n_G vector).

u_G is the upper-bound group constraint (n_G vector).

G is the matrix of group membership indexes (n_G -by- n matrix).

Each row of G identifies which assets belong to a group associated with that row. Each row contains either 0s or 1s with 1 indicating that an asset is part of the group or 0 indicating that the asset is not part of the group.

PortfolioCVaR object properties to specify group constraints are:

- GroupMatrix for G
- LowerGroup for l_G
- UpperGroup for u_G
- NumAssets for n

The default is to ignore these constraints.

Group Ratio Constraints

Group ratio constraints are specialized linear constraints that enforce relationships among groups of assets. The constraints take the form

$$l_{Ri}(G_Bx)_i \leq (G_Ax)_i \leq u_{Ri}(G_Bx)_i$$

for $i = 1, \dots, n_R$ where:

x is the portfolio (n vector).

l_R is the vector of lower-bound group ratio constraints (n_R vector).

u_R is the vector matrix of upper-bound group ratio constraints (n_R vector).

G_A is the matrix of base group membership indexes (n_R -by- n matrix).

G_B is the matrix of comparison group membership indexes (n_R -by- n matrix).

n is the number of assets in the universe and n_R is the number of constraints.

Each row of G_A and G_B identifies which assets belong to a base and comparison group associated with that row.

Each row contains either 0s or 1s with 1 indicating that an asset is part of the group or 0 indicating that the asset is not part of the group.

PortfolioCVaR object properties to specify group ratio constraints are:

- GroupA for G_A
- GroupB for G_B
- LowerRatio for l_R
- UpperRatio for u_R
- NumAssets for n

The default is to ignore these constraints.

Average Turnover Constraints

Turnover constraint is a linear absolute value constraint that ensures estimated optimal portfolios differ from an initial portfolio by no more than a specified amount. Although portfolio turnover is defined in many ways, the turnover constraints implemented in Financial Toolbox computes portfolio turnover as the average of purchases and sales. Average turnover constraints take the form

$$\frac{1}{2} \mathbf{1}^T |x - x_0| \leq \tau$$

where:

x is the portfolio (n vector).

$\mathbf{1}$ is the vector of ones (n vector).

x_0 is the initial portfolio (n vector).

τ is the upper bound for turnover (scalar).

n is the number of assets in the universe.

PortfolioCvAR object properties to specify the average turnover constraint are:

- Turnover for τ
- InitPort for x_0
- NumAssets for n

The default is to ignore this constraint.

One-way Turnover Constraints

One-way turnover constraints ensure that estimated optimal portfolios differ from an initial portfolio by no more than specified amounts according to whether the differences are purchases or sales. The constraints take the forms

$$\mathbf{1}^T \times \max\{0, x - x_0\} \leq \tau_B$$

$$\mathbf{1}^T \times \max\{0, x_0 - x\} \leq \tau_S$$

where:

x is the portfolio (n vector)

$\mathbf{1}$ is the vector of ones (n vector).

x_0 is the Initial portfolio (n vector).

τ_B is the upper bound for turnover constraint on purchases (scalar).

τ_S is the upper bound for turnover constraint on sales (scalar).

To specify one-way turnover constraints, use the following properties in the Portfolio, PortfolioCvAR, or PortfolioMAD object:

- `BuyTurnover` for τ_B
- `SellTurnover` for τ_S
- `InitPort` for x_0

The default is to ignore this constraint.

Note The average turnover constraint (see “Average Turnover Constraints” on page 5-13) with τ is not a combination of the one-way turnover constraints with $\tau = \tau_B = \tau_S$.

Cardinality Constraints

Cardinality constraint limits the number of assets in the optimal allocation for an `PortfolioCVaR` object. Use `setMinMaxNumAssets` to specify the 'MinNumAssets' and 'MaxNumAssets' constraints. To mathematically formulate this type of constraints, a binary variable v_i is needed. $v_i = 0$ indicates that asset i is not selected and $v_i = 1$ indicates that the asset was selected. Thus

$$\text{MinNumAssets} \leq \sum_{i=1}^{\text{NumAssets}} v_i \leq \text{MaxNumAssets}$$

The default is to ignore this constraint.

See Also

`PortfolioCVaR`

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Default Portfolio Problem” on page 5-15
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Default Portfolio Problem

The default portfolio optimization problem has a risk and return proxy associated with a given problem, and a portfolio set that specifies portfolio weights to be nonnegative and to sum to 1. The lower bound combined with the budget constraint is sufficient to ensure that the portfolio set is nonempty, closed, and bounded. The default portfolio optimization problem characterizes a long-only investor who is fully invested in a collection of assets.

- For mean-variance portfolio optimization, it is sufficient to set up the default problem. After setting up the problem, data in the form of a mean and covariance of asset returns are then used to solve portfolio optimization problems.
- For conditional value-at-risk portfolio optimization, the default problem requires the additional specification of a probability level that must be set explicitly. Generally, “typical” values for this level are 0.90 or 0.95. After setting up the problem, data in the form of scenarios of asset returns are then used to solve portfolio optimization problems.
- For MAD portfolio optimization, it is sufficient to set up the default problem. After setting up the problem, data in the form of scenarios of asset returns are then used to solve portfolio optimization problems.

See Also

PortfolioCVaR

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

PortfolioCVaR Object Workflow

The PortfolioCVaR object workflow for creating and modeling a CVaR portfolio is:

1 Create a CVaR Portfolio.

Create a `PortfolioCVaR` object for conditional value-at-risk (CVaR) portfolio optimization. For more information, see “Creating the PortfolioCVaR Object” on page 5-22.

2 Define asset returns and scenarios.

Evaluate scenarios for portfolio asset returns, including assets with missing data and financial time series data. For more information, see “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36.

3 Specify the CVaR Portfolio Constraints.

Define the constraints for portfolio assets such as linear equality and inequality, bound, budget, conditional budget, group, group ratio, turnover constraints, 'Conditional' `BoundType`, and `MinNumAssets`, `MaxNumAssets` constraints. For more information, see “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51 and “Working with 'Conditional' `BoundType`, `MinNumAssets`, and `MaxNumAssets` Constraints Using PortfolioCVaR Objects” on page 5-72.

4 Validate the CVaR Portfolio.

Identify errors for the portfolio specification. For more information, see “Validate the CVaR Portfolio Problem” on page 5-81.

5 Estimate the efficient portfolios and frontiers.

Analyze the efficient portfolios and efficient frontiers for a CVaR portfolio. For more information, see “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85 and “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105.

6 Postprocess the results.

Use the efficient portfolios and efficient frontiers results to set up trades. For more information, see “Postprocessing Results to Set Up Tradable Portfolios” on page 5-115.

See Also

Related Examples

- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “Portfolio Optimization Theory” on page 5-3
- “Choosing and Controlling the Solver for PortfolioCVaR Optimizations” on page 5-98

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

PortfolioCVaR Object

In this section...

“PortfolioCVaR Object Properties and Functions” on page 5-17
 “Working with PortfolioCVaR Objects” on page 5-17
 “Setting and Getting Properties” on page 5-18
 “Displaying PortfolioCVaR Objects” on page 5-18
 “Saving and Loading PortfolioCVaR Objects” on page 5-18
 “Estimating Efficient Portfolios and Frontiers” on page 5-18
 “Arrays of PortfolioCVaR Objects” on page 5-19
 “Subclassing PortfolioCVaR Objects” on page 5-20
 “Conventions for Representation of Data” on page 5-20

PortfolioCVaR Object Properties and Functions

The `PortfolioCVaR` object implements conditional value-at-risk (CVaR) portfolio optimization. Every property and function of the `PortfolioCVaR` object is public, although some properties and functions are hidden. See `PortfolioCVaR` for the properties and functions of a `PortfolioCVaR` object. The `PortfolioCVaR` object is a value object where every instance of the object is a distinct version of the object. Since the `PortfolioCVaR` object is also a MATLAB object, it inherits the default functions associated with MATLAB objects.

Working with PortfolioCVaR Objects

The `PortfolioCVaR` object and its functions are an interface for conditional value-at-risk portfolio optimization. So, almost everything you do with the `PortfolioCVaR` object can be done using the functions. The basic workflow is:

- 1 Design your portfolio problem.
- 2 Use `PortfolioCVaR` to create the `PortfolioCVaR` object or use the various set functions to set up your portfolio problem.
- 3 Use estimate functions to solve your portfolio problem.

In addition, functions are available to help you view intermediate results and to diagnose your computations. Since MATLAB features are part of a `PortfolioCVaR` object, you can save and load objects from your workspace and create and manipulate arrays of objects. After settling on a problem, which, in the case of CVaR portfolio optimization, means that you have either scenarios, data, or moments for asset returns, a probability level, and a collection of constraints on your portfolios, use `PortfolioCVaR` to set the properties for the `PortfolioCVaR` object.

`PortfolioCVaR` lets you create an object from scratch or update an existing object. Since the `PortfolioCVaR` object is a value object, it is easy to create a basic object, then use functions to build upon the basic object to create new versions of the basic object. This is useful to compare a basic problem with alternatives derived from the basic problem. For details, see “Creating the `PortfolioCVaR` Object” on page 5-22.

Setting and Getting Properties

You can set properties of a `PortfolioCVaR` object using either the `PortfolioCVaR` object or various `set` functions.

Note Although you can also set properties directly, it is not recommended since error-checking is not performed when you set a property directly.

The `PortfolioCVaR` object supports setting properties with name-value pair arguments such that each argument name is a property and each value is the value to assign to that property. For example, to set the `LowerBound`, `Budget`, and `ProbabilityLevel` properties in an existing `PortfolioCVaR` object `p`, use the syntax:

```
p = PortfolioCVaR(p, 'LowerBound', 0, 'Budget', 1, 'ProbabilityLevel', 0.95);
```

In addition to the `PortfolioCVaR` object, which lets you set individual properties one at a time, groups of properties are set in a `PortfolioCVaR` object with various “set” and “add” functions. For example, to set up an average turnover constraint, use the `setTurnover` function to specify the bound on portfolio turnover and the initial portfolio. To get individual properties from a `PortfolioCVaR` object, obtain properties directly or use an assortment of “get” functions that obtain groups of properties from a `PortfolioCVaR` object. The `PortfolioCVaR` object and set functions have several useful features:

- The `PortfolioCVaR` object and set functions try to determine the dimensions of your problem with either explicit or implicit inputs.
- The `PortfolioCVaR` object and set functions try to resolve ambiguities with default choices.
- The `PortfolioCVaR` object and set functions perform scalar expansion on arrays when possible.
- The CVaR functions try to diagnose and warn about problems.

Displaying PortfolioCVaR Objects

The `PortfolioCVaR` object uses the default display functions provided by MATLAB, where `display` and `disp` display a `PortfolioCVaR` object and its properties with or without the object variable name.

Saving and Loading PortfolioCVaR Objects

Save and load `PortfolioCVaR` objects using the MATLAB `save` and `load` commands.

Estimating Efficient Portfolios and Frontiers

Estimating efficient portfolios and efficient frontiers is the primary purpose of the CVaR portfolio optimization tools. An efficient portfolio are the portfolios that satisfy the criteria of minimum risk for a given level of return and maximum return for a given level of risk. A collection of “estimate” and “plot” functions provide ways to explore the efficient frontier. The “estimate” functions obtain either efficient portfolios or risk and return proxies to form efficient frontiers. At the portfolio level, a collection of functions estimates efficient portfolios on the efficient frontier with functions to obtain efficient portfolios:

- At the endpoints of the efficient frontier
- That attain targeted values for return proxies
- That attain targeted values for risk proxies
- Along the entire efficient frontier

These functions also provide purchases and sales needed to shift from an initial or current portfolio to each efficient portfolio. At the efficient frontier level, a collection of functions plot the efficient frontier and estimate either risk or return proxies for efficient portfolios on the efficient frontier. You can use the resultant efficient portfolios or risk and return proxies in subsequent analyses.

Arrays of PortfolioCVaR Objects

Although all functions associated with a `PortfolioCVaR` object are designed to work on a scalar `PortfolioCVaR` object, the array capabilities of MATLAB enables you to set up and work with arrays of `PortfolioCVaR` objects. The easiest way to do this is with the `repmat` function. For example, to create a 3-by-2 array of `PortfolioCVaR` objects:

```
p = repmat(PortfolioCVaR, 3, 2);
disp(p)
```

3×2 `PortfolioCVaR` array with properties:

```
BuyCost
SellCost
RiskFreeRate
ProbabilityLevel
Turnover
BuyTurnover
SellTurnover
NumScenarios
Name
NumAssets
AssetList
InitPort
AInequality
bInequality
AEquality
bEquality
LowerBound
UpperBound
LowerBudget
UpperBudget
GroupMatrix
LowerGroup
UpperGroup
GroupA
GroupB
LowerRatio
UpperRatio
MinNumAssets
MaxNumAssets
ConditionalBudgetThreshold
ConditionalUpperBudget
BoundType
```

After setting up an array of `PortfolioCVaR` objects, you can work on individual `PortfolioCVaR` objects in the array by indexing. For example:

```
p(i,j) = PortfolioCVaR(p(i,j), ... );
```

This example calls `PortfolioCVaR` for the (i,j) element of a matrix of `PortfolioCVaR` objects in the variable `p`.

If you set up an array of `PortfolioCVaR` objects, you can access properties of a particular `PortfolioCVaR` object in the array by indexing so that you can set the lower and upper bounds `lb` and `ub` for the (i,j,k) element of a 3-D array of `PortfolioCVaR` objects with

```
p(i,j,k) = setBounds(p(i,j,k), lb, ub);
```

and, once set, you can access these bounds with

```
[lb, ub] = getBounds(p(i,j,k));
```

`PortfolioCVaR` object functions work on only one `PortfolioCVaR` object at a time.

Subclassing PortfolioCVaR Objects

You can subclass the `PortfolioCVaR` object to override existing functions or to add new properties or functions. To do so, create a derived class from the `PortfolioCVaR` class. This gives you all the properties and functions of the `PortfolioCVaR` class along with any new features that you choose to add to your subclassed object. The `PortfolioCVaR` class is derived from an abstract class called `AbstractPortfolio`. Because of this, you can also create a derived class from `AbstractPortfolio` that implements an entirely different form of portfolio optimization using properties and functions of the `AbstractPortfolio` class.

Conventions for Representation of Data

The CVaR portfolio optimization tools follow these conventions regarding the representation of different quantities associated with portfolio optimization:

- Asset returns or prices for scenarios are in matrix form with samples for a given asset going down the rows and assets going across the columns. In the case of prices, the earliest dates must be at the top of the matrix, with increasing dates going down.
- Portfolios are in vector or matrix form with weights for a given portfolio going down the rows and distinct portfolios going across the columns.
- Constraints on portfolios are formed in such a way that a portfolio is a column vector.
- Portfolio risks and returns are either scalars or column vectors (for multiple portfolio risks and returns).

See Also

`PortfolioCVaR`

Related Examples

- “Creating the `PortfolioCVaR` Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51

- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Creating the PortfolioCVaR Object

In this section...

“Syntax” on page 5-22

“PortfolioCVaR Problem Sufficiency” on page 5-22

“PortfolioCVaR Function Examples” on page 5-23

To create a fully specified CVaR portfolio optimization problem, instantiate the `PortfolioCVaR` object using `PortfolioCVaR`. For information on the workflow when using `PortfolioCVaR` objects, see “PortfolioCVaR Object Workflow” on page 5-16.

Syntax

Use `PortfolioCVaR` to create an instance of an object of the `PortfolioCVaR` class. You can use `PortfolioCVaR` object in several ways. To set up a portfolio optimization problem in a `PortfolioCVaR` object, the simplest syntax is:

```
p = PortfolioCVaR;
```

This syntax creates a `PortfolioCVaR` object, `p`, such that all object properties are empty.

The `PortfolioCVaR` object also accepts collections of argument name-value pair arguments for properties and their values. The `PortfolioCVaR` object accepts inputs for public properties with the general syntax:

```
p = PortfolioCVaR('property1', value1, 'property2', value2, ... );
```

If a `PortfolioCVaR` object already exists, the syntax permits the first (and only the first argument) of `PortfolioCVaR` to be an existing object with subsequent argument name-value pair arguments for properties to be added or modified. For example, given an existing `PortfolioCVaR` object in `p`, the general syntax is:

```
p = PortfolioCVaR(p, 'property1', value1, 'property2', value2, ... );
```

Input argument names are not case-sensitive, but must be completely specified. In addition, several properties can be specified with alternative argument names (see “Shortcuts for Property Names” on page 5-26). The `PortfolioCVaR` object tries to detect problem dimensions from the inputs and, once set, subsequent inputs can undergo various scalar or matrix expansion operations that simplify the overall process to formulate a problem. In addition, a `PortfolioCVaR` object is a value object so that, given portfolio `p`, the following code creates two objects, `p` and `q`, that are distinct:

```
q = PortfolioCVaR(p, ...)
```

PortfolioCVaR Problem Sufficiency

A CVaR portfolio optimization problem is completely specified with the `PortfolioCVaR` object if the following three conditions are met:

- You must specify a collection of asset returns or prices known as scenarios such that all scenarios are finite asset returns or prices. These scenarios are meant to be samples from the underlying probability distribution of asset returns. This condition can be satisfied by the `setScenarios` function or with several canned scenario simulation functions.

- The set of feasible portfolios must be a nonempty compact set, where a compact set is closed and bounded. You can satisfy this condition using an extensive collection of properties that define different types of constraints to form a set of feasible portfolios. Since such sets must be bounded, either explicit or implicit constraints can be imposed and several tools, such as the `estimateBounds` function, provide ways to ensure that your problem is properly formulated.
- You must specify a probability level to locate the level of tail loss above which the conditional value-at-risk is to be minimized. This condition can be satisfied by the `setProbabilityLevel` function.

Although the general sufficient conditions for CVaR portfolio optimization go beyond the first three conditions, the `PortfolioCVaR` object handles all these additional conditions.

PortfolioCVaR Function Examples

If you create a `PortfolioCVaR` object, `p`, with no input arguments, you can display it using `disp`:

```
p = PortfolioCVaR;
disp(p)
```

PortfolioCVaR with properties:

```

        BuyCost: []
        SellCost: []
    RiskFreeRate: []
ProbabilityLevel: []
        Turnover: []
    BuyTurnover: []
    SellTurnover: []
    NumScenarios: []
            Name: []
        NumAssets: []
        AssetList: []
        InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: []
    UpperBound: []
    LowerBudget: []
    UpperBudget: []
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
        GroupA: []
        GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
            BoundType: []

```

The approaches listed provide a way to set up a portfolio optimization problem with the `PortfolioCVaR` object. The custom set functions offer additional ways to set and modify collections of properties in the `PortfolioCVaR` object.

Using the `PortfolioCVaR` Function for a Single-Step Setup

You can use the `PortfolioCVaR` object to directly set up a “standard” portfolio optimization problem. Given scenarios of asset returns in the variable `AssetScenarios`, this problem is completely specified as follows:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR('Scenarios', AssetScenarios, ...
    'LowerBound', 0, 'LowerBudget', 1, 'UpperBudget', 1, ...
    'ProbabilityLevel', 0.95)

p =

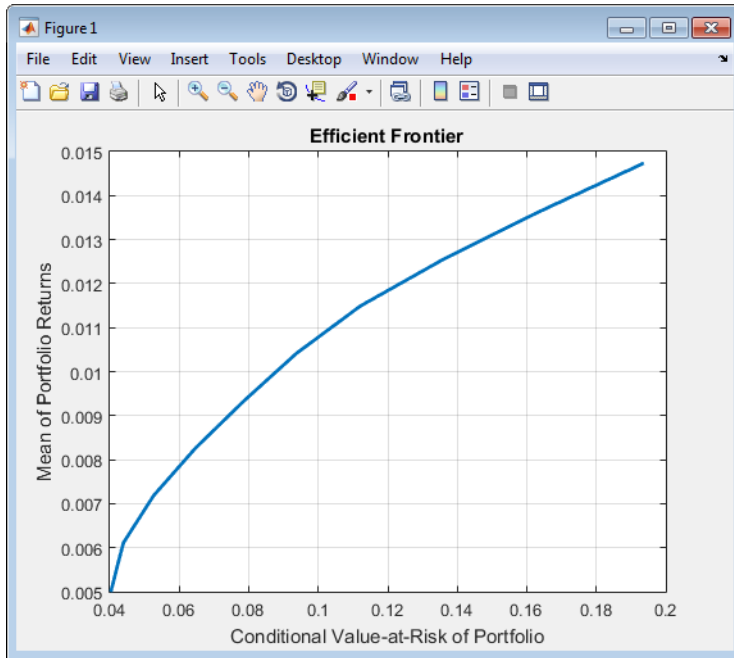
    PortfolioCVaR with properties:

        BuyCost: []
        SellCost: []
        RiskFreeRate: []
    ProbabilityLevel: 0.9500
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
        NumScenarios: 20000
            Name: []
        NumAssets: 4
        AssetList: []
        InitPort: []
        AInequality: []
        bInequality: []
        AEquality: []
        bEquality: []
        LowerBound: [4×1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
            GroupA: []
            GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
    ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
            BoundType: []
```

The `LowerBound` property value undergoes scalar expansion since `AssetScenarios` provides the dimensions of the problem.

You can use dot notation with the function `plotFrontier`.

```
p.plotFrontier
```



Using the PortfolioCVaR Function with a Sequence of Steps

An alternative way to accomplish the same task of setting up a “standard” CVaR portfolio optimization problem, given `AssetScenarios` variable is:

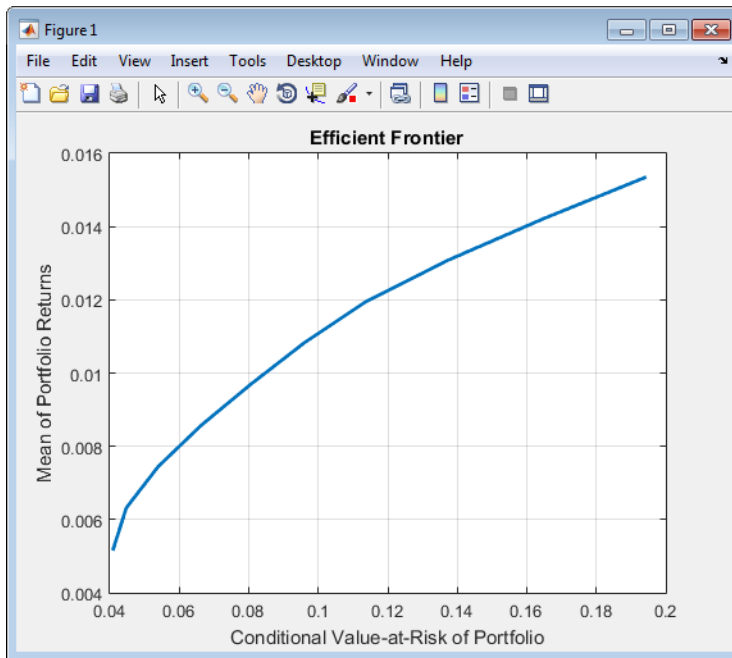
```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = PortfolioCVaR(p, 'LowerBound', 0);
p = PortfolioCVaR(p, 'LowerBudget', 1, 'UpperBudget', 1);
p = setProbabilityLevel(p, 0.95);

plotFrontier(p)
```



This way works because the calls to the are in this particular order. In this case, the call to initialize `AssetScenarios` provides the dimensions for the problem. If you were to do this step last, you would have to explicitly dimension the `LowerBound` property as follows:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = PortfolioCVaR(p, 'LowerBound', zeros(size(m)));
p = PortfolioCVaR(p, 'LowerBudget', 1, 'UpperBudget', 1);
p = setProbabilityLevel(p, 0.95);
p = setScenarios(p, AssetScenarios);
```

Note If you did not specify the size of `LowerBound` but, instead, input a scalar argument, the `PortfolioCVaR` object assumes that you are defining a single-asset problem and produces an error at the call to set asset scenarios with four assets.

Shortcuts for Property Names

The `PortfolioCVaR` object has shorter argument names that replace longer argument names associated with specific properties of the `PortfolioCVaR` object. For example, rather than enter `'ProbabilityLevel'`, the `PortfolioCVaR` object accepts the case-insensitive name `'plevel'` to set the `ProbabilityLevel` property in a `PortfolioCVaR` object. Every shorter argument name

corresponds with a single property in the PortfolioCvAR object. The one exception is the alternative argument name 'budget', which signifies both the LowerBudget and UpperBudget properties. When 'budget' is used, then the LowerBudget and UpperBudget properties are set to the same value to form an equality budget constraint.

Shortcuts for Property Names

Shortcut Argument Name	Equivalent Argument / Property Name
ae	AEquality
ai	AInequality
assetnames or assets	AssetList
be	bEquality
bi	bInequality
budget	UpperBudget and LowerBudget
group	GroupMatrix
lb	LowerBound
n or num	NumAssets
level, problevel, or plevel	ProbabilityLevel
rfr	RiskFreeRate
scenario or assetscenarios	Scenarios
ub	UpperBound

For example, this call to the PortfolioCvAR object uses these shortcuts for properties:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCvAR('scenario', AssetScenarios, 'lb', 0, 'budget', 1, 'plevel', 0.95);
plotFrontier(p)
```

Direct Setting of Portfolio Object Properties

Although not recommended, you can set properties directly using dot notation, however no error-checking is done on your inputs:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCvAR;
```

```
p = setScenarios(p, AssetScenarios);  
p.ProbabilityLevel = 0.95;  
  
p.LowerBudget = 1;  
p.UpperBudget = 1;  
p.LowerBound = zeros(size(m));  
  
plotFrontier(p)
```

Note Scenarios cannot be assigned directly using dot notation to a `PortfolioCVaR` object. Scenarios must always be set through either the `PortfolioCVaR` object, the `setScenarios` function, or any of the scenario simulation functions.

See Also

`PortfolioCVaR` | `estimateBounds`

Related Examples

- “Common Operations on the `PortfolioCVaR` Object” on page 5-29
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “`PortfolioCVaR` Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “`PortfolioCVaR` Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Common Operations on the PortfolioCVaR Object

In this section...

“Naming a PortfolioCVaR Object” on page 5-29

“Configuring the Assets in the Asset Universe” on page 5-29

“Setting Up a List of Asset Identifiers” on page 5-29

“Truncating and Padding Asset Lists” on page 5-31

Naming a PortfolioCVaR Object

To name a PortfolioCVaR object, use the Name property. Name is informational and has no effect on any portfolio calculations. If the Name property is nonempty, Name is the title for the efficient frontier plot generated by `plotFrontier`. For example, if you set up an asset allocation fund, you could name the PortfolioCVaR object Asset Allocation Fund:

```
p = PortfolioCVaR('Name','Asset Allocation Fund');
disp(p.Name)
```

```
Asset Allocation Fund
```

Configuring the Assets in the Asset Universe

The fundamental quantity in the PortfolioCVaR object is the number of assets in the asset universe. This quantity is maintained in the NumAssets property. Although you can set this property directly, it is usually derived from other properties such as the number of assets in the scenarios or the initial portfolio. In some instances, the number of assets may need to be set directly. This example shows how to set up a PortfolioCVaR object that has four assets:

```
p = PortfolioCVaR('NumAssets', 4);
disp(p.NumAssets)
```

```
4
```

After setting the NumAssets property, you cannot modify it (unless no other properties are set that depend on NumAssets). The only way to change the number of assets in an existing PortfolioCVaR object with a known number of assets is to create a new PortfolioCVaR object.

Setting Up a List of Asset Identifiers

When working with portfolios, you must specify a universe of assets. Although you can perform a complete analysis without naming the assets in your universe, it is helpful to have an identifier associated with each asset as you create and work with portfolios. You can create a list of asset identifiers as a cell vector of character vectors in the property AssetList. You can set up the list using the next two methods.

Setting Up Asset Lists Using the PortfolioCVaR Function

Suppose that you have a PortfolioCVaR object, p, with assets with symbols 'AA', 'BA', 'CAT', 'DD', and 'ETR'. You can create a list of these asset symbols in the object using the PortfolioCVaR object:

```
p = PortfolioCVaR('assetlist', { 'AA', 'BA', 'CAT', 'DD', 'ETR' });
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

Notice that the property `AssetList` is maintained as a cell array that contains character vectors, and that it is necessary to pass a cell array into the `PortfolioCVaR` object to set `AssetList`. In addition, notice that the property `NumAssets` is set to 5 based on the number of symbols used to create the asset list:

```
disp(p.NumAssets)
```

```
5
```

Setting Up Asset Lists Using the `setAssetList` Function

You can also specify a list of assets using the `setAssetList` function. Given the list of asset symbols 'AA', 'BA', 'CAT', 'DD', and 'ETR', you can use `setAssetList` with:

```
p = PortfolioCVaR;
p = setAssetList(p, { 'AA', 'BA', 'CAT', 'DD', 'ETR' });
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

`setAssetList` also enables you to enter symbols directly as a comma-separated list without creating a cell array of character vectors. For example, given the list of assets symbols 'AA', 'BA', 'CAT', 'DD', and 'ETR', use `setAssetList`:

```
p = PortfolioCVaR;
p = setAssetList(p, 'AA', 'BA', 'CAT', 'DD', 'ETR');
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

`setAssetList` has many additional features to create lists of asset identifiers. If you use `setAssetList` with just a `PortfolioCVaR` object, it creates a default asset list according to the name specified in the hidden public property `defaultforAssetList` (which is 'Asset' by default). The number of asset names created depends on the number of assets in the property `NumAssets`. If `NumAssets` is not set, then `NumAssets` is assumed to be 1.

For example, if a `PortfolioCVaR` object `p` is created with `NumAssets = 5`, then this code fragment shows the default naming behavior:

```
p = PortfolioCVaR('numassets',5);
p = setAssetList(p);
disp(p.AssetList)
```

```
'Asset1'      'Asset2'      'Asset3'      'Asset4'      'Asset5'
```

Suppose that your assets are, for example, ETFs and you change the hidden property `defaultforAssetList` to 'ETF', you can then create a default list for ETFs:

```
p = PortfolioCVaR('numassets',5);
p.defaultforAssetList = 'ETF';
p = setAssetList(p);
disp(p.AssetList)
```

```
'ETF1'      'ETF2'      'ETF3'      'ETF4'      'ETF5'
```


Truncating and Padding Asset Lists

If the `NumAssets` property is already set and you pass in too many or too few identifiers, the `PortfolioCVaR` object, and the `setAssetList` function truncate or pad the list with numbered default asset names that use the name specified in the hidden public property `defaultforAssetList`. If the list is truncated or padded, a warning message indicates the discrepancy. For example, assume that you have a `PortfolioCVaR` object with five ETFs and you only know the first three CUSIPs '921937835', '922908769', and '922042775'. Use this syntax to create an asset list that pads the remaining asset identifiers with numbered 'UnknownCUSIP' placeholders:

```
p = PortfolioCVaR('numassets',5);
p.defaultforAssetList = 'UnknownCUSIP';
p = setAssetList(p, '921937835', '922908769', '922042775');
disp(p.AssetList)

Warning: Input list of assets has 2 too few identifiers. Padding with numbered assets.
> In PortfolioCVaR.setAssetList at 118
'921937835' '922908769' '922042775' 'UnknownCUSIP4' 'UnknownCUSIP5'
```

Alternatively, suppose that you have too many identifiers and need only the first four assets. This example illustrates truncation of the asset list using the `PortfolioCVaR` object:

```
p = PortfolioCVaR('numassets',4);
p = PortfolioCVaR(p, 'assetlist', { 'AGG', 'EEM', 'MDY', 'SPY', 'VEU' });
disp(p.AssetList)

Warning: AssetList has 1 too many identifiers. Using first 4 assets.
> In PortfolioCVaR.checkarguments at 399
In PortfolioCVaR.PortfolioCVaR>PortfolioCVaR at 195
'AGG' 'EEM' 'MDY' 'SPY'
```

The hidden public property `uppercaseAssetList` is a Boolean flag to specify whether to convert asset names to uppercase letters. The default value for `uppercaseAssetList` is `false`. This example shows how to use the `uppercaseAssetList` flag to force identifiers to be uppercase letters:

```
p = PortfolioCVaR;
p.uppercaseAssetList = true;
p = setAssetList(p, { 'aa', 'ba', 'cat', 'dd', 'etr' });
disp(p.AssetList)

'AA' 'BA' 'CAT' 'DD' 'ETR'
```

See Also

`PortfolioCVaR` | `setAssetList` | `setInitPort` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Setting Up an Initial or Current Portfolio” on page 5-33
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17

- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Setting Up an Initial or Current Portfolio

In many applications, creating a new optimal portfolio requires comparing the new portfolio with an initial or current portfolio to form lists of purchases and sales. The `PortfolioCVaR` object property `InitPort` lets you identify an initial or current portfolio. The initial portfolio also plays an essential role if you have either transaction costs or turnover constraints. The initial portfolio need not be feasible within the constraints of the problem. This can happen if the weights in a portfolio have shifted such that some constraints become violated. To check if your initial portfolio is feasible, use the `checkFeasibility` function described in “Validating CVaR Portfolios” on page 5-82. Suppose that you have an initial portfolio in `x0`, then use the `PortfolioCVaR` object to set up an initial portfolio:

```
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = PortfolioCVaR('InitPort', x0);
disp(p.InitPort)

0.3000
0.2000
0.2000
0
```

As with all array properties, you can set `InitPort` with scalar expansion. This is helpful to set up an equally weighted initial portfolio of, for example, 10 assets:

```
p = PortfolioCVaR('NumAssets', 10, 'InitPort', 1/10);
disp(p.InitPort)

0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
```

To clear an initial portfolio from your `PortfolioCVaR` object, use either the `PortfolioCVaR` object or the `setInitPort` function with an empty input for the `InitPort` property. If transaction costs or turnover constraints are set, it is not possible to clear the `InitPort` property in this way. In this case, to clear `InitPort`, first clear the dependent properties and then clear the `InitPort` property.

The `InitPort` property can also be set with `setInitPort` which lets you specify the number of assets if you want to use scalar expansion. For example, given an initial portfolio in `x0`, use `setInitPort` to set the `InitPort` property:

```
p = PortfolioCVaR;
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = setInitPort(p, x0);
disp(p.InitPort)

0.3000
0.2000
0.2000
0
```

To create an equally weighted portfolio of four assets, use `setInitPort`:

```
p = PortfolioCVaR;
p = setInitPort(p, 1/4, 4);
disp(p.InitPort)

0.2500
0.2500
0.2500
0.2500
```

`PortfolioCVaR` object functions that work with either transaction costs or turnover constraints also depend on the `InitPort` property. So, the set functions for transaction costs or turnover constraints permit the assignment of a value for the `InitPort` property as part of their implementation. For details, see “Working with Average Turnover Constraints Using `PortfolioCVaR` Object” on page 5-75, “Working with One-Way Turnover Constraints Using `PortfolioCVaR` Object” on page 5-78, and “Working with Transaction Costs” on page 5-47. If either transaction costs or turnover constraints are used, then the `InitPort` property must have a nonempty value. Absent a specific value assigned through the `PortfolioCVaR` object or various set functions, the `PortfolioCVaR` object sets `InitPort` to 0 and warns if `BuyCost`, `SellCost`, or `Turnover` properties are set. This example shows what happens if you specify an average turnover constraint with an initial portfolio:

```
p = PortfolioCVaR('Turnover', 0.3, 'InitPort', [ 0.3; 0.2; 0.2; 0.0 ]);
disp(p.InitPort)

0.3000
0.2000
0.2000
0
```

In contrast, this example shows what happens if an average turnover constraint is specified without an initial portfolio:

```
p = PortfolioCVaR('Turnover', 0.3);
disp(p.InitPort)
```

```
Warning: InitPort and NumAssets are empty and either transaction costs or turnover constraints specified.
Will set NumAssets = 1 and InitPort = 0.
> In PortfolioCVaR.checkarguments at 322
    In PortfolioCVaR.PortfolioCVaR>PortfolioCVaR.PortfolioCVaR at 195
    0
```

See Also

`PortfolioCVaR` | `setAssetList` | `setInitPort` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Creating the `PortfolioCVaR` Object” on page 5-22
- “Common Operations on the `PortfolioCVaR` Object” on page 5-29
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Asset Returns and Scenarios Using PortfolioCVaR Object

In this section...

“How Stochastic Optimization Works” on page 5-36
 “What Are Scenarios?” on page 5-36
 “Setting Scenarios Using the PortfolioCVaR Function” on page 5-37
 “Setting Scenarios Using the setScenarios Function” on page 5-38
 “Estimating the Mean and Covariance of Scenarios” on page 5-38
 “Simulating Normal Scenarios” on page 5-39
 “Simulating Normal Scenarios from Returns or Prices” on page 5-39
 “Simulating Normal Scenarios with Missing Data” on page 5-40
 “Simulating Normal Scenarios from Time Series Data” on page 5-42
 “Simulating Normal Scenarios with Mean and Covariance” on page 5-44

How Stochastic Optimization Works

The CVaR of a portfolio is a conditional expectation. (For the definition of the CVaR function, see “Risk Proxy” on page 5-5.) Therefore, the CVaR portfolio optimization problem is a stochastic optimization problem. Given a sample of scenarios, the conditional expectation that defines the sample CVaR of the portfolio can be expressed as a finite sum, a weighted average of losses. The weights of the losses depend on their relative magnitude; for a confidence level α , only the worst $(1 - \alpha) \times 100\%$ losses get a positive weight. As a function of the portfolio weights, the CVaR of the portfolio is a convex function (see [48], [49] Rockafellar & Uryasev at “Portfolio Optimization” on page A-5). It is also a nonsmooth function, but its edges are less sharp as the sample size increases.

There are reformulations of the CVaR portfolio optimization problem (see [48], [49] at Rockafellar & Uryasev) that result in a linear programming problem, which can be solved either with standard linear programming techniques or with stochastic programming solvers. The `PortfolioCVaR` object, however, does not reformulate the problem in such a manner. The `PortfolioCVaR` object computes the CVaR as a nonlinear function. The convexity of the CVaR, as a function of the portfolio weights and the dull edges when the number of scenarios is large, make the CVaR portfolio optimization problem tractable, in practice, for certain nonlinear programming solvers, such as `fmincon` from Optimization Toolbox. The problem can also be solved using a cutting-plane method (see Kelley [45] at “Portfolio Optimization” on page A-5). For more information, see Algorithms section of `setSolver`. To learn more about the workflow when using `PortfolioCVaR` objects, see “PortfolioCVaR Object Workflow” on page 5-16.

What Are Scenarios?

Since conditional value-at-risk portfolio optimization works with scenarios of asset returns to perform the optimization, several ways exist to specify and simulate scenarios. In many applications with CVaR portfolio optimization, asset returns may have distinctly nonnormal probability distributions with either multiple modes, binning of returns, truncation of distributions, and so forth. In other applications, asset returns are modeled as the result of various simulation methods that might include Monte-Carlo simulation, quasi-random simulation, and so forth. Often, the underlying probability distribution for risk factors may be multivariate normal but the resultant transformations are sufficiently nonlinear to result in distinctively nonnormal asset returns.

For example, this occurs with bonds and derivatives. In the case of bonds with a nonzero probability of default, such scenarios would likely include asset returns that are -100% to indicate default and some values slightly greater than -100% to indicate recovery rates.

Although the `PortfolioCvAR` object has functions to simulate multivariate normal scenarios from either data or moments (`simulateNormalScenariosByData` and `simulateNormalScenariosByMoments`), the usual approach is to specify scenarios directly from your own simulation functions. These scenarios are entered directly as a matrix with a sample for all assets across each row of the matrix and with samples for an asset down each column of the matrix. The architecture of the CVaR portfolio optimization tools references the scenarios through a function handle so scenarios that have been set cannot be accessed directly as a property of the `PortfolioCvAR` object.

Setting Scenarios Using the PortfolioCvAR Function

Suppose that you have a matrix of scenarios in the `AssetScenarios` variable. The scenarios are set through the `PortfolioCvAR` object with:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCvAR('Scenarios', AssetScenarios);

disp(p.NumAssets)
disp(p.NumScenarios)

4

20000
```

Notice that the `PortfolioCvAR` object determines and fixes the number of assets in `NumAssets` and the number of scenarios in `NumScenarios` based on the scenario's matrix. You can change the number of scenarios by calling the `PortfolioCvAR` object with a different scenario matrix. However, once the `NumAssets` property has been set in the object, you cannot enter a scenario matrix with a different number of assets. The `getScenarios` function lets you recover scenarios from a `PortfolioCvAR` object. You can also obtain the mean and covariance of your scenarios using `estimateScenarioMoments`.

Although not recommended for the casual user, an alternative way exists to recover scenarios by working with the function handle that points to scenarios in the `PortfolioCvAR` object. To access some or all the scenarios from a `PortfolioCvAR` object, the hidden property `localScenarioHandle` is a function handle that points to a function to obtain scenarios that have already been set. To get scenarios directly from a `PortfolioCvAR` object `p`, use

```
scenarios = p.localScenarioHandle([], []);
```

and to obtain a subset of scenarios from rows `startrow` to `endrow`, use

```
scenarios = p.localScenarioHandle(startrow, endrow);
```

where $1 \leq \text{startrow} \leq \text{endrow} \leq \text{numScenarios}$.

Setting Scenarios Using the setScenarios Function

You can also set scenarios using `setScenarios`. For example, given the mean and covariance of asset returns in the variables `m` and `C`, the asset moment properties can be set:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);

disp(p.NumAssets)
disp(p.NumScenarios)

4

20000
```

Estimating the Mean and Covariance of Scenarios

This example shows how to use `estimateScenarioMoments` function to obtain estimates for the mean and covariance of scenarios in a `PortfolioCVaR` object.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
[mean, covar] = estimateScenarioMoments(p)

mean = 4×1

    0.0042
    0.0082
    0.0094
```



```
0.0147
```

```
covar = 4x4
```

```
0.0005    0.0003    0.0002   -0.0000
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
-0.0000    0.0010    0.0028    0.0100
```

Simulating Normal Scenarios

As a convenience, the two functions (`simulateNormalScenariosByData` and `simulateNormalScenariosByMoments`) exist to simulate scenarios from data or moments under an assumption that they are distributed as multivariate normal random asset returns.

Simulating Normal Scenarios from Returns or Prices

This example shows how to use the function `simulateNormalScenariosByData` to simulate multivariate normal scenarios when given either return or price data. Either returns or prices are stored as matrices with samples going down the rows and assets going across the columns. In addition, returns or prices can be stored in a `table` or `timetable` (see “Simulating Normal Scenarios from Time Series Data” on page 5-42).

To illustrate using `simulateNormalScenariosByData`, generate random samples of 120 observations of asset returns for four assets from the mean and covariance of asset returns in the variables `m` and `C` with `portsim`. The default behavior of `portsim` creates simulated data with estimated mean and covariance identical to the input moments `m` and `C`. In addition to a return series created by `portsim` in the variable `X`, a price series is created in the variable `Y`.

```
m = [ 0.0042; 0.0083; 0.01; 0.15 ];
C = [ 0.005333 0.00034 0.00016 0;
      0.00034 0.002408 0.0017 0.000992;
      0.00016 0.0017 0.0048 0.0028;
      0 0.000992 0.0028 0.010208 ];
```

```
X = portsim(m', C, 120);
Y = ret2tick(X);
```

Note that Portfolio optimization requires that you use total returns and not just price returns. So, "returns" should be total returns and "prices" should be total return prices.

Given asset returns and prices in variables `X` and `Y`, this sequence of code demonstrates equivalent ways to simulate multivariate normal scenarios for the `PortfolioCvAR` object. Assume a `PortfolioCvAR` object created in `p` that uses the asset returns in `X` uses `simulateNormalScenariosByData`:

```
p = PortfolioCvAR;
p = simulateNormalScenariosByData(p, X, 20000);

[passetmean, passetcovar] = estimateScenarioMoments(p)
```

```
passetmean = 4×1

    0.0042
    0.0082
    0.0094
    0.1498

passetcovar = 4×4

    0.0052    0.0003    0.0001   -0.0000
    0.0003    0.0024    0.0017    0.0010
    0.0001    0.0017    0.0048    0.0028
   -0.0000    0.0010    0.0028    0.0100
```

The moments that you obtain from this simulation will likely differ from the moments listed here because the scenarios are random samples from the estimated multivariate normal probability distribution of the input returns X .

The default behavior of `simulateNormalScenariosByData` is to work with asset returns. If, instead, you have asset prices as in the variable Y , `simulateNormalScenariosByData` accepts a name-value pair argument name `'DataFormat'` with a corresponding value set to `'prices'` to indicate that the input to the function is in the form of asset prices and not returns (the default value for the `'DataFormat'` argument is `'returns'`). The following code simulates scenarios with the asset price data in Y for the `PortfolioCVaR` object q :

```
p = PortfolioCVaR;
p = simulateNormalScenariosByData(p, Y, 20000, 'dataformat', 'prices');

[passetmean, passetcovar] = estimateScenarioMoments(p)

passetmean = 4×1

    0.0046
    0.0086
    0.0097
    0.1501

passetcovar = 4×4

    0.0054    0.0003    0.0002    0.0000
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0029
    0.0000    0.0010    0.0029    0.0103
```

Simulating Normal Scenarios with Missing Data

This example shows how to simulate normal scenarios when you have missing data indicated by `NaN` values in your return or price data. Although “Multivariate Normal Regression” on page 9-2 goes into detail about regression with missing data, the `simulateNormalScenariosByData` function has a name-value pair argument name `'MissingData'` that indicates with a Boolean value whether to use the missing data capabilities of Financial Toolbox™. The default value for `'MissingData'` is

false which removes all samples with NaN values. If, however, 'MissingData' is set to true, simulateNormalScenariosByData uses the ECM algorithm to estimate asset moments. The following code shows how this works on price data with missing values.

```
m = [ 0.0042; 0.0083; 0.01; 0.15 ];
C = [ 0.005333 0.00034 0.00016 0;
      0.00034 0.002408 0.0017 0.000992;
      0.00016 0.0017 0.0048 0.0028;
      0 0.000992 0.0028 0.010208 ];
```

```
X = portsim(m', C, 120);
Y = ret2tick(X);
Y(1:20,1) = NaN;
Y(1:12,4) = NaN;
```

Notice that the prices above in Y have missing values in the first and fourth series.

```
p = PortfolioCvAR;
p = simulateNormalScenariosByData(p, Y, 20000, 'dataformat', 'prices');

q = PortfolioCvAR;
q = simulateNormalScenariosByData(q, Y, 20000, 'dataformat', 'prices', 'missingdata', true);
```

```
[passetmean, passetcovar] = estimateScenarioMoments(p)
```

```
passetmean = 4×1
```

```
0.0014
0.0095
0.0162
0.1587
```

```
passetcovar = 4×4
```

```
0.0050    0.0002   -0.0002    0.0001
0.0002    0.0025    0.0014    0.0006
-0.0002    0.0014    0.0038    0.0018
0.0001    0.0006    0.0018    0.0087
```

```
[qassetmean, qassetcovar] = estimateScenarioMoments(q)
```

```
qassetmean = 4×1
```

```
0.0024
0.0086
0.0096
0.1544
```

```
qassetcovar = 4×4
```

```
0.0067    0.0003   -0.0003    0.0002
0.0003    0.0032    0.0022    0.0010
-0.0003    0.0022    0.0064    0.0027
0.0002    0.0010    0.0027    0.0119
```

The first `PortfolioCVaR` object, `p`, contains scenarios obtained from price data in `Y` where `NaN` values are discarded and the second `PortfolioCVaR` object, `q`, contains scenarios obtained from price data in `Y` that accommodate missing values. Each time you run this example, you get different estimates for the moments in `p` and `q`.

Simulating Normal Scenarios from Time Series Data

This example shows how the `simulateNormalScenariosByData` function accepts asset returns or prices stored in a table or timetable. The `simulateNormalScenariosByData` function implicitly works with matrices of data or data in a `table` or `timetable` object using the same rules for whether the data are returns or prices.

To illustrate, use `array2timetable` to create a timetable for 14 assets from `CAPMuniverse` and the use the timetable to simulate scenarios for `PortfolioCVaR`.

```
load CAPMuniverse
time = datetime(Dates,'ConvertFrom','datenum');
stockTT = array2timetable(Data,'RowTimes',time, 'VariableNames', Assets);
stockTT.Properties

ans =
    TimetableProperties with properties:

        Description: ''
        UserData: []
    DimensionNames: {'Time' 'Variables'}
    VariableNames: {'AAPL' 'AMZN' 'CSCO' 'DELL' 'EBAY' 'GOOG' 'HPQ' 'IBM' 'INTC'
    VariableTypes: ["double" "double" "double" "double" "double" "double"
    VariableDescriptions: {}
    VariableUnits: {}
    VariableContinuity: []
        RowTimes: [1471x1 datetime]
        StartTime: 03-Jan-2000
        SampleRate: NaN
        TimeStep: NaN
        Events: []
    CustomProperties: No custom properties are set.
    Use addprop and rmprop to modify CustomProperties.
```

% Notice that GOOG has missing data, because it was not listed before Aug 2004
`head(stockTT, 5)`

Time	AAPL	AMZN	CSCO	DELL	EBAY	GOOG	HPQ
03-Jan-2000	0.088805	0.1742	0.008775	-0.002353	0.12829	NaN	0.000000
04-Jan-2000	-0.084331	-0.08324	-0.05608	-0.08353	-0.093805	NaN	-0.000000
05-Jan-2000	0.014634	-0.14877	-0.003039	0.070984	0.066875	NaN	-0.000000
06-Jan-2000	-0.086538	-0.060072	-0.016619	-0.038847	-0.012302	NaN	-0.000000
07-Jan-2000	0.047368	0.061013	0.0587	-0.037708	-0.000964	NaN	0.000000

Use the `'MissingData'` option offered by `PortfolioCVaR` to account for the missing data.

```
p = PortfolioCVaR;
p = simulateNormalScenariosByData(p, stockTT, 20000, 'missingdata', true);
[passetmean, passetcovar] = estimateScenarioMoments(p)
```

```
passetmean = 14x1
```

```
0.0012
0.0007
-0.0004
-0.0000
0.0016
0.0043
-0.0000
0.0001
0.0001
-0.0003
0.0001
0.0003
0.0001
0.0001
:
```

```
passetcovar = 14x14
```

```
0.0013    0.0005    0.0006    0.0005    0.0005    0.0003    0.0005    0.0003    0.0006    0.0006
0.0005    0.0024    0.0007    0.0005    0.0010    0.0005    0.0005    0.0003    0.0006    0.0006
0.0006    0.0007    0.0013    0.0006    0.0007    0.0004    0.0006    0.0004    0.0008    0.0008
0.0005    0.0005    0.0006    0.0009    0.0006    0.0002    0.0005    0.0003    0.0006    0.0006
0.0005    0.0010    0.0007    0.0006    0.0018    0.0007    0.0005    0.0003    0.0006    0.0006
0.0003    0.0005    0.0004    0.0002    0.0007    0.0013    0.0002    0.0002    0.0002    0.0002
0.0005    0.0005    0.0006    0.0005    0.0005    0.0002    0.0010    0.0003    0.0005    0.0005
0.0003    0.0003    0.0004    0.0003    0.0003    0.0002    0.0003    0.0005    0.0004    0.0004
0.0006    0.0006    0.0008    0.0006    0.0006    0.0002    0.0005    0.0004    0.0011    0.0011
0.0004    0.0004    0.0005    0.0004    0.0005    0.0002    0.0003    0.0002    0.0005    0.0005
0.0005    0.0006    0.0008    0.0005    0.0007    0.0003    0.0005    0.0004    0.0007    0.0007
0.0006    0.0011    0.0008    0.0006    0.0011    0.0011    0.0006    0.0004    0.0007    0.0007
0.0002    0.0002    0.0002    0.0002    0.0002    0.0001    0.0002    0.0002    0.0002    0.0002
-0.0000    -0.0000    -0.0000    -0.0000    -0.0000    -0.0000    -0.0000    0.0000    -0.0000    -0.0000
:
```

Use the name-value input 'DataFormat' to handle return or price data and 'MissingData' to ignore or use samples with missing values. In addition, `simulateNormalScenariosByData` extracts asset names or identifiers from a table or timetable if the argument 'GetAssetList' is set to true (the default value is false). If the 'GetAssetList' value is true, the identifiers are used to set the `AssetList` property of the `PortfolioCVaR` object. Thus, repeating the formation of the `PortfolioCVaR` object `p` with the 'GetAssetList' flag set to true extracts the column names from the timetable object:

```
p = simulateNormalScenariosByData(p, stockTT, 20000, 'missingdata', true, 'GetAssetList', true);
disp(p.AssetList)
```

```
{'AAPL'}    {'AMZN'}    {'CSCO'}    {'DELL'}    {'EBAY'}    {'GOOG'}    {'HPQ'}    {'IBM'}
```

If you set the 'GetAssetList' flag set to true and your input data is in a matrix, `simulateNormalScenariosByData` uses the default labeling scheme from `setAssetList` as described in “Setting Up a List of Asset Identifiers” on page 5-29.

Simulating Normal Scenarios with Mean and Covariance

This example shows how to use the `simulateNormalScenariosByMoments` function to simulate multivariate normal scenarios when given the mean and covariance of asset returns. The mean can be either a row or column vector and the covariance matrix must be a symmetric positive-semidefinite matrix. Various rules for scalar expansion apply.

To illustrate using `simulateNormalScenariosByMoments` with a `PortfolioCVaR` object, start with moments in `m` and `C` and generate 20,000 scenarios.

```
m = [ 0.0042; 0.0083; 0.01; 0.15 ];
C = [ 0.005333 0.00034 0.00016 0;
      0.00034 0.002408 0.0017 0.000992;
      0.00016 0.0017 0.0048 0.0028;
      0 0.000992 0.0028 0.010208 ];

p = PortfolioCVaR;
p = simulateNormalScenariosByMoments(p, m, C, 20000);
[passetmean, passetcovar] = estimateScenarioMoments(p)

passetmean = 4×1

    0.0042
    0.0082
    0.0094
    0.1497

passetcovar = 4×4

    0.0052    0.0003    0.0001   -0.0000
    0.0003    0.0024    0.0017    0.0010
    0.0001    0.0017    0.0048    0.0028
   -0.0000    0.0010    0.0028    0.0100
```

`simulateNormalScenariosByMoments` performs scalar expansion on arguments for the moments of asset returns. If `NumAssets` has not already been set, a scalar argument is interpreted as a scalar with `NumAssets` set to 1. `simulateNormalScenariosByMoments` provides an additional optional argument to specify the number of assets so that scalar expansion works with the correct number of assets. In addition, if either a scalar or vector is input for the covariance of asset returns, a diagonal matrix is formed such that a scalar expands along the diagonal and a vector becomes the diagonal.

See Also

`PortfolioCVaR` | `setCosts` | `setProbabilityLevel` | `setScenarios` | `estimatePortVaR` | `simulateNormalScenariosByMoments` | `simulateNormalScenariosByData`

Related Examples

- “Working with a Riskless Asset” on page 5-46
- “Working with Transaction Costs” on page 5-47

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with a Riskless Asset

The `PortfolioCVaR` object has a separate `RiskFreeRate` property that stores the rate of return of a riskless asset. Thus, you can separate your universe into a riskless asset and a collection of risky assets. For example, assume that your riskless asset has a return in the scalar variable `r0`, then the property for the `RiskFreeRate` is set using the `PortfolioCVaR` object:

```
r0 = 0.01/12;  
  
p = PortfolioCVaR;  
p = PortfolioCVaR('RiskFreeRate', r0);  
disp(p.RiskFreeRate)  
  
8.3333e-04
```

Note If your portfolio problem has a budget constraint such that your portfolio weights must sum to 1, then the riskless asset is irrelevant.

See Also

`PortfolioCVaR` | `setCosts` | `setProbabilityLevel` | `setScenarios` | `estimatePortVaR` | `simulateNormalScenariosByMoments` | `simulateNormalScenariosByData`

Related Examples

- “Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36
- “Working with Transaction Costs” on page 5-47
- “Creating the `PortfolioCVaR` Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for `PortfolioCVaR` Object” on page 5-85
- “Estimate Efficient Frontiers for `PortfolioCVaR` Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “`PortfolioCVaR` Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “`PortfolioCVaR` Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with Transaction Costs

The difference between net and gross portfolio returns is transaction costs. The net portfolio return proxy has distinct proportional costs to purchase and to sell assets which are maintained in the `PortfolioCVaR` object properties `BuyCost` and `SellCost`. Transaction costs are in units of total return and, as such, are proportional to the price of an asset so that they enter the model for net portfolio returns in return form. For example, suppose that you have a stock currently priced \$40 and your usual transaction costs are 5 cents per share. Then the transaction cost for the stock is $0.05/40 = 0.00125$ (as defined in “Net Portfolio Returns” on page 5-4). Costs are entered as positive values and credits are entered as negative values.

Setting Transaction Costs Using the `PortfolioCVaR` Function

To set up transaction costs, you must specify an initial or current portfolio in the `InitPort` property. If the initial portfolio is not set when you set up the transaction cost properties, `InitPort` is 0. The properties for transaction costs can be set using the `PortfolioCVaR` object. For example, assume that purchase and sale transaction costs are in the variables `bc` and `sc` and an initial portfolio is in the variable `x0`, then transaction costs are set:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = PortfolioCVaR('BuyCost', bc, 'SellCost', sc, 'InitPort', x0);
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
```

```
0.0013
0.0013
0.0013
0.0013
0.0013
```

```
0.0013
0.0070
0.0013
0.0013
0.0024
```

```
0.4000
0.2000
0.2000
0.1000
0.1000
```

Setting Transaction Costs Using the `setCosts` Function

You can also set the properties for transaction costs using `setCosts`. Assume that you have the same costs and initial portfolio as in the previous example. Given a `PortfolioCVaR` object `p` with an initial portfolio already set, use `setCosts` to set up transaction costs:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
```

```
p = PortfolioCVaR('InitPort', x0);
p = setCosts(p, bc, sc);
```

```
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
0.0013
0.0013
0.0013
0.0013
0.0013
0.0013
0.0070
0.0013
0.0013
0.0024
0.4000
0.2000
0.2000
0.1000
0.1000
```

You can also set up the initial portfolio's `InitPort` value as an optional argument to `setCosts` so that the following is an equivalent way to set up transaction costs:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
```

```
p = PortfolioCVaR;
p = setCosts(p, bc, sc, x0);
```

```
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
0.0013
0.0013
0.0013
0.0013
0.0013
0.0013
0.0070
0.0013
0.0013
```

```

0.0013
0.0024

0.4000
0.2000
0.2000
0.1000
0.1000

```

Setting Transaction Costs with Scalar Expansion

Both the `PortfolioCVaR` object and `setCosts` function implement scalar expansion on the arguments for transaction costs and the initial portfolio. If the `NumAssets` property is already set in the `PortfolioCVaR` object, scalar arguments for these properties are expanded to have the same value across all dimensions. In addition, `setCosts` lets you specify `NumAssets` as an optional final argument. For example, assume that you have an initial portfolio `x0` and you want to set common transaction costs on all assets in your universe. You can set these costs in any of these equivalent ways:

```

x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = PortfolioCVaR('InitPort', x0, 'BuyCost', 0.002, 'SellCost', 0.002);

```

or

```

x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = PortfolioCVaR('InitPort', x0);
p = setCosts(p, 0.002, 0.002);

```

or

```

x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = PortfolioCVaR;
p = setCosts(p, 0.002, 0.002, x0);

```

To clear costs from your `PortfolioCVaR` object, use either the `PortfolioCVaR` object or `setCosts` with empty inputs for the properties to be cleared. For example, you can clear sales costs from the `PortfolioCVaR` object `p` in the previous example:

```

p = PortfolioCVaR(p, 'SellCost', []);

```

See Also

`PortfolioCVaR` | `setCosts` | `setProbabilityLevel` | `setScenarios` | `estimatePortVaR` | `simulateNormalScenariosByMoments` | `simulateNormalScenariosByData`

Related Examples

- “Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36
- “Working with a Riskless Asset” on page 5-46
- “Creating the `PortfolioCVaR` Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for `PortfolioCVaR` Object” on page 5-85

- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with CVaR Portfolio Constraints Using Defaults

The final element for a complete specification of a portfolio optimization problem is the set of feasible portfolios, which is called a portfolio set. A portfolio set $X \subset R^n$ is specified by construction as the intersection of sets formed by a collection of constraints on portfolio weights. A portfolio set necessarily and sufficiently must be a nonempty, closed, and bounded set.

When setting up your portfolio set, ensure that the portfolio set satisfies these conditions. The most basic or “default” portfolio set requires portfolio weights to be nonnegative (using the lower-bound constraint) and to sum to 1 (using the budget constraint). For information on the workflow when using `PortfolioCVaR` objects, see “PortfolioCVaR Object Workflow” on page 5-16.

Setting Default Constraints for Portfolio Weights Using PortfolioCVaR Object

The “default” CVaR portfolio problem has two constraints on portfolio weights:

- Portfolio weights must be nonnegative.
- Portfolio weights must sum to 1.

Implicitly, these constraints imply that portfolio weights are no greater than 1, although this is a superfluous constraint to impose on the problem.

Setting Default Constraints Using the PortfolioCVaR Function

Given a portfolio optimization problem with `NumAssets` = 20 assets, use the `PortfolioCVaR` object to set up a default problem and explicitly set bounds and budget constraints:

```
p = PortfolioCVaR('NumAssets', 20, 'LowerBound', 0, 'Budget', 1);
disp(p)
```

PortfolioCVaR with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
ProbabilityLevel: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: []
Name: []
NumAssets: 20
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [20×1 double]
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
```

```
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []
```

Setting Default Constraints Using the `setDefaultConstraints` Function

An alternative approach is to use the `setDefaultConstraints` function. If the number of assets is already known in a `PortfolioCVaR` object, use `setDefaultConstraints` with no arguments to set up the necessary bound and budget constraints. Suppose that you have 20 assets to set up the portfolio set for a default problem:

```
p = PortfolioCVaR('NumAssets', 20);
p = setDefaultConstraints(p);
disp(p)
```

PortfolioCVaR with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
ProbabilityLevel: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: []
Name: []
NumAssets: 20
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [20×1 double]
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: [20×1 categorical]
```

If the number of assets is unknown, `setDefaultConstraints` accepts `NumAssets` as an optional argument to form a portfolio set for a default problem. Suppose that you have 20 assets:

```
p = PortfolioCVaR;
p = setDefaultConstraints(p, 20);
disp(p)
```

PortfolioCVaR with properties:

```

        BuyCost: []
        SellCost: []
        RiskFreeRate: []
    ProbabilityLevel: []
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
        NumScenarios: []
            Name: []
        NumAssets: 20
        AssetList: []
        InitPort: []
        AInequality: []
        bInequality: []
        AEquality: []
        bEquality: []
        LowerBound: [20×1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
            GroupA: []
            GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
    ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: [20×1 categorical]
```

See Also

`PortfolioCVaR` | `setDefaultConstraints` | `setBounds` | `setBudget` |
`setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` |
`setTurnover` | `setOneWayTurnover`

Related Examples

- “Working with ‘Simple’ Bound Constraints Using PortfolioCVaR Object” on page 5-55
- “Working with Budget Constraints Using PortfolioCVaR Object” on page 5-58
- “Working with Group Constraints Using PortfolioCVaR Object” on page 5-62
- “Working with Group Ratio Constraints Using PortfolioCVaR Object” on page 5-65
- “Working with Linear Equality Constraints Using PortfolioCVaR Object” on page 5-68

- “Working with Linear Inequality Constraints Using PortfolioCVaR Object” on page 5-70
- “Working with Average Turnover Constraints Using PortfolioCVaR Object” on page 5-75
- “Working with One-Way Turnover Constraints Using PortfolioCVaR Object” on page 5-78
- “Adding Constraints to Satisfy UCITS Directive” on page 4-457
- “Creating the PortfolioCVaR Object” on page 5-22
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with 'Simple' Bound Constraints Using PortfolioCvAR Object

'Simple' bound constraints are optional linear constraints that maintain upper and lower bounds on portfolio weights (see "'Simple' Bound Constraints" on page 5-9). Although every portfolio set must be bounded, it is not necessary to specify a portfolio set with explicit bound constraints. For example, you can create a portfolio set with an implicit upper bound constraint or a portfolio set with average turnover constraints. The bound constraints have properties `LowerBound` for the lower-bound constraint and `UpperBound` for the upper-bound constraint. Set default values for these constraints using the `setDefaultConstraints` function (see "Setting Default Constraints for Portfolio Weights Using Portfolio Object" on page 4-58).

Setting 'Simple' Bounds Using the PortfolioCvAR Function

The properties for bound constraints are set through the `PortfolioCvAR` object. Suppose that you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. The bound constraints for a balanced fund are set with:

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];
p = PortfolioCvAR('LowerBound', lb, 'UpperBound', ub, 'BoundType', 'Simple');
disp(p.NumAssets)
disp(p.LowerBound)
disp(p.UpperBound)
```

2

0.5000
0.2500

0.7500
0.5000

To continue with this example, you must set up a budget constraint. For details, see "Working with Budget Constraints Using Portfolio Object" on page 4-65.

Setting 'Simple' Bounds Using the setBounds Function

You can also set the properties for bound constraints using `setBounds`. Suppose that you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. Given a `PortfolioCvAR` object `p`, use `setBounds` to set the bound constraints:

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];
p = PortfolioCvAR;
p = setBounds(p, lb, ub, 'BoundType', 'Simple');
disp(p.NumAssets)
disp(p.LowerBound)
disp(p.UpperBound)
```

2

0.5000

```
0.2500
```

```
0.7500
```

```
0.5000
```

Setting 'Simple' Bounds Using the PortfolioCVaR Function or setBounds Function

Both the PortfolioCVaR object and setBounds function implement scalar expansion on either the LowerBound or UpperBound properties. If the NumAssets property is already set in the PortfolioCVaR object, scalar arguments for either property expand to have the same value across all dimensions. In addition, setBounds lets you specify NumAssets as an optional argument. Suppose that you have a universe of 500 assets and you want to set common bound constraints on all assets in your universe. Specifically, you are a long-only investor and want to hold no more than 5% of your portfolio in any single asset. You can set these bound constraints in any of these equivalent ways:

```
p = PortfolioCVaR('NumAssets', 500, 'LowerBound', 0, 'UpperBound', 0.05, 'BoundType', 'Simple');
```

or

```
p = PortfolioCVaR('NumAssets', 500);  
p = setBounds(p, 0, 0.05, 'BoundType', 'Simple');
```

or

```
p = PortfolioCVaR;  
p = setBounds(p, 0, 0.05, 'NumAssets', 500, 'BoundType', 'Simple');
```

To clear bound constraints from your PortfolioCVaR object, use either the PortfolioCVaR object or setBounds with empty inputs for the properties to be cleared. For example, to clear the upper-bound constraint from the PortfolioCVaR object p in the previous example:

```
p = PortfolioCVaR(p, 'UpperBound', []);
```

See Also

PortfolioCVaR | setDefaultConstraints | setBounds | setBudget |
setConditionalBudget | setGroups | setGroupRatio | setEquality | setInequality |
setTurnover | setOneWayTurnover

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioCVaR Objects” on page 5-72

- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with Budget Constraints Using PortfolioCVaR Object

The budget constraint is an optional linear constraint that maintains upper and lower bounds on the sum of portfolio weights (see “Budget Constraints” on page 5-10). Budget constraints have properties `LowerBudget` for the lower budget constraint and `UpperBudget` for the upper budget constraint. If you set up a CVaR portfolio optimization problem that requires portfolios to be fully invested in your universe of assets, you can set `LowerBudget` to be equal to `UpperBudget`. These budget constraints can be set with default values equal to 1 using `setDefaultConstraints` (see “Setting Default Constraints Using the PortfolioCVaR Function” on page 5-51).

Setting Budget Constraints Using the PortfolioCVaR Function

The properties for the budget constraint can also be set using the `PortfolioCVaR` object. Suppose that you have an asset universe with many risky assets and a riskless asset and you want to ensure that your portfolio never holds more than 1% cash, that is, you want to ensure that you are 99-100% invested in risky assets. The budget constraint for this portfolio can be set with:

```
p = PortfolioCVaR('LowerBudget', 0.99, 'UpperBudget', 1);  
disp(p.LowerBudget)  
disp(p.UpperBudget)  
  
0.9900  
  
1
```

Setting Budget Constraints Using the setBudget Function

You can also set the properties for a budget constraint using `setBudget`. Suppose that you have a fund that permits up to 10% leverage which means that your portfolio can be from 100% to 110% invested in risky assets. Given a `PortfolioCVaR` object `p`, use `setBudget` to set the budget constraints:

```
p = PortfolioCVaR;  
p = setBudget(p, 1, 1.1);  
disp(p.LowerBudget)  
disp(p.UpperBudget)  
  
1  
  
1.1000
```

If you were to continue with this example, then set the `RiskFreeRate` property to the borrowing rate to finance possible leveraged positions. For details on the `RiskFreeRate` property, see “Working with a Riskless Asset” on page 5-46. To clear either bound for the budget constraint from your `PortfolioCVaR` object, use either the `PortfolioCVaR` object or `setBudget` with empty inputs for the properties to be cleared. For example, clear the upper-budget constraint from the `PortfolioCVaR` object `p` in the previous example with:

```
p = PortfolioCVaR(p, 'UpperBudget', []);
```

See Also

`PortfolioCVaR` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with Conditional Budget Constraints Using PortfolioCVaR Object

The conditional budget constraint supports the Undertakings for Collective Investment in Transferable Securities (UCITS) directive for `PortfolioCVaR` objects. The UCITS asset allocation regulation states that funds can only invest up to 10% in a single issuer, and that investments in excess of 5% must not exceed 40% of the total portfolio. This is known as the 5/10/40 rule. For more information, see “Undertakings for Collective Investment in Transferable Securities” on page 15-1452 and “Adding Constraints to Satisfy UCITS Directive” on page 4-457.

Setting Conditional Budget Constraints Using the PortfolioCVaR Function

The properties for the conditional budget constraint can also be set using the `PortfolioCVaR` object. Suppose that you have an asset universe with many risky assets and a riskless asset and you want to ensure that your portfolio never holds more than 1% cash, that is, you want to ensure that you are 99-100% invested in risky assets. The conditional budget constraint for this portfolio can be set with:

```
p = PortfolioCVaR('ConditionalBudgetThreshold', 0.99, 'ConditionalUpperBudget', 1);  
disp(p.ConditionalBudgetThreshold)  
disp(p.ConditionalUpperBudget)  
  
0.9900  
  
1
```

Setting Conditional Budget Constraints Using the setConditionalBudget Function

You can also set the properties for a conditional budget constraint using `setConditionalBudget`. Suppose that you have a fund that permits up to 10% leverage which means that your portfolio can be from 100% to 110% invested in risky assets. Given a `PortfolioCVaR` object `p`, use `setConditionalBudget` to set the conditional budget constraints:

```
p = PortfolioCVaR;  
p = setConditionalBudget(p, .40, 1.5);  
disp(p.threshold)  
disp(p.upperBudget)  
  
.40  
  
1.5000
```

See Also

`Portfolio` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover` | `setTrackingPort` | `setTrackingError`

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Group Constraints Using PortfolioCVaR Object

Group constraints are optional linear constraints that group assets together and enforce bounds on the group weights (see “Group Constraints” on page 5-11). Although the constraints are implemented as general constraints, the usual convention is to form a group matrix that identifies membership of each asset within a specific group with Boolean indicators (either `true` or `false` or with 1 or 0) for each element in the group matrix. Group constraints have properties `GroupMatrix` for the group membership matrix, `LowerGroup` for the lower-bound constraint on groups, and `UpperGroup` for the upper-bound constraint on groups.

Setting Group Constraints Using the PortfolioCVaR Function

The properties for group constraints are set through the `PortfolioCVaR` object. Suppose that you have a portfolio of five assets and want to ensure that the first three assets constitute no more than 30% of your portfolio, then you can set group constraints:

```
G = [ 1 1 1 0 0 ];
p = PortfolioCVaR('GroupMatrix', G, 'UpperGroup', 0.3);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.UpperGroup)

5

1      1      1      0      0

0.3000
```

The group matrix `G` can also be a logical matrix so that the following code achieves the same result.

```
G = [ true true true false false ];
p = PortfolioCVaR('GroupMatrix', G, 'UpperGroup', 0.3);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.UpperGroup)

5

1      1      1      0      0

0.3000
```

Setting Group Constraints Using the setGroups and addGroups Functions

You can also set the properties for group constraints using `setGroups`. Suppose that you have a portfolio of five assets and want to ensure that the first three assets constitute no more than 30% of your portfolio. Given a `PortfolioCVaR` object `p`, use `setGroups` to set the group constraints:

```
G = [ true true true false false ];
p = PortfolioCVaR;
p = setGroups(p, G, [], 0.3);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.UpperGroup)
```



```

5
1      1      1      0      0
0.3000

```

In this example, you would set the `LowerGroup` property to be empty (`[]`).

Suppose that you want to add another group constraint to make odd-numbered assets constitute at least 20% of your portfolio. Set up an augmented group matrix and introduce infinite bounds for unconstrained group bounds or use the `addGroups` function to build up group constraints. For this example, create another group matrix for the second group constraint:

```

p = PortfolioCvAR;
G = [ true true true false false ]; % group matrix for first group constraint
p = setGroups(p, G, [], 0.3);
G = [ true false true false true ]; % group matrix for second group constraint
p = addGroups(p, G, 0.2);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.LowerGroup)
disp(p.UpperGroup)

5
1      1      1      0      0
1      0      1      0      1

-Inf
0.2000

0.3000
Inf

```

`addGroups` determines which bounds are unbounded so you only need to focus on the constraints that you want to set.

The `PortfolioCvAR` object, `setGroups`, and `addGroups` implement scalar expansion on either the `LowerGroup` or `UpperGroup` properties based on the dimension of the group matrix in the property `GroupMatrix`. Suppose that you have a universe of 30 assets with 6 asset classes such that assets 1–5, assets 6–12, assets 13–18, assets 19–22, assets 23–27, and assets 28–30 constitute each of your asset classes and you want each asset class to fall from 0% to 25% of your portfolio. Let the following group matrix define your groups and scalar expansion define the common bounds on each group:

```

p = PortfolioCvAR;
G = blkdiag(true(1,5), true(1,7), true(1,6), true(1,4), true(1,5), true(1,3));
p = setGroups(p, G, 0, 0.25);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.LowerGroup)
disp(p.UpperGroup)

30

Columns 1 through 16

1      1      1      1      1      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      1      1      1      1      1      1      1      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      1      1      1
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0

Columns 17 through 30

0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0      0

```

```

1      1      0      0      0      0      0      0      0      0      0      0      0      0
0      0      1      1      1      1      0      0      0      0      0      0      0      0
0      0      0      0      0      0      1      1      1      1      1      0      0      0
0      0      0      0      0      0      0      0      0      0      0      1      1      1

0
0
0
0
0
0

0.2500
0.2500
0.2500
0.2500
0.2500
0.2500

```

See Also

PortfolioCVaR | setDefaultConstraints | setBounds | setBudget |
 setConditionalBudget | setGroups | setGroupRatio | setEquality | setInequality |
 setTurnover | setOneWayTurnover

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with Group Ratio Constraints Using PortfolioCVaR Object

Group ratio constraints are optional linear constraints that maintain bounds on proportional relationships among groups of assets (see “Group Ratio Constraints” on page 5-12). Although the constraints are implemented as general constraints, the usual convention is to specify a pair of group matrices that identify membership of each asset within specific groups with Boolean indicators (either `true` or `false` or with 1 or 0) for each element in each of the group matrices. The goal is to ensure that the ratio of a base group compared to a comparison group fall within specified bounds. Group ratio constraints have properties:

- `GroupA` for the base membership matrix
- `GroupB` for the comparison membership matrix
- `LowerRatio` for the lower-bound constraint on the ratio of groups
- `UpperRatio` for the upper-bound constraint on the ratio of groups

Setting Group Ratio Constraints Using the PortfolioCVaR Function

The properties for group ratio constraints are set using `PortfolioCVaR` object. For example, assume that you want the ratio of financial to nonfinancial companies in your portfolios to never go above 50%. Suppose that you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). To set group ratio constraints:

```
GA = [ 1 1 1 0 0 0 ]; % financial companies
GB = [ 0 0 0 1 1 1 ]; % nonfinancial companies
p = PortfolioCVaR('GroupA', GA, 'GroupB', GB, 'UpperRatio', 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

6

```
1      1      1      0      0      0
```

```
0      0      0      1      1      1
```

0.5000

Group matrices `GA` and `GB` in this example can be logical matrices with `true` and `false` elements that yield the same result:

```
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = PortfolioCVaR('GroupA', GA, 'GroupB', GB, 'UpperRatio', 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

6

```
1      1      1      0      0      0
```

```
0      0      0      1      1      1
```

0.5000

Setting Group Ratio Constraints Using the `setGroupRatio` and `addGroupRatio` Functions

You can also set the properties for group ratio constraints using `setGroupRatio`. For example, assume that you want the ratio of financial to nonfinancial companies in your portfolios to never go above 50%. Suppose that you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). Given a `PortfolioCVaR` object `p`, use `setGroupRatio` to set the group constraints:

```
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = PortfolioCVaR;
p = setGroupRatio(p, GA, GB, [], 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

6

```
1      1      1      0      0      0
0      0      0      1      1      1
```

0.5000

In this example, you would set the `LowerRatio` property to be empty (`[]`).

Suppose that you want to add another group ratio constraint to ensure that the weights in odd-numbered assets constitute at least 20% of the weights in nonfinancial assets your portfolio. You can set up augmented group ratio matrices and introduce infinite bounds for unconstrained group ratio bounds, or you can use the `addGroupRatio` function to build up group ratio constraints. For this example, create another group matrix for the second group constraint:

```
p = PortfolioCVaR;
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = setGroupRatio(p, GA, GB, [], 0.5);

GA = [ true false true false true false ]; % odd-numbered companies
GB = [ false false false true true true ]; % nonfinancial companies
p = addGroupRatio(p, GA, GB, 0.2);
```

```
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.LowerRatio)
disp(p.UpperRatio)
```

6

```
1      1      1      0      0      0
1      0      1      0      1      0

0      0      0      1      1      1
0      0      0      1      1      1
```

-Inf
0.2000

0.5000
Inf

Notice that `addGroupRatio` determines which bounds are unbounded so you only need to focus on the constraints you want to set.

The `PortfolioCVaR` object, `setGroupRatio`, and `addGroupRatio` implement scalar expansion on either the `LowerRatio` or `UpperRatio` properties based on the dimension of the group matrices in `GroupA` and `GroupB` properties.

See Also

`PortfolioCVaR` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Creating the `PortfolioCVaR` Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for `PortfolioCVaR` Object” on page 5-85
- “Estimate Efficient Frontiers for `PortfolioCVaR` Object” on page 5-105
- “Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “`PortfolioCVaR` Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “`PortfolioCVaR` Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with Linear Equality Constraints Using PortfolioCVaR Object

Linear equality constraints are optional linear constraints that impose systems of equalities on portfolio weights (see “Linear Equality Constraints” on page 5-9). Linear equality constraints have properties `AEquality`, for the equality constraint matrix, and `bEquality`, for the equality constraint vector.

Setting Linear Equality Constraints Using the PortfolioCVaR Function

The properties for linear equality constraints are set using the `PortfolioCVaR` object. Suppose that you have a portfolio of five assets and want to ensure that the first three assets are 50% of your portfolio. To set this constraint:

```
A = [ 1 1 1 0 0 ];  
b = 0.5;  
p = PortfolioCVaR('AEquality', A, 'bEquality', b);  
disp(p.NumAssets)  
disp(p.AEquality)  
disp(p.bEquality)
```

5

1 1 1 0 0

0.5000

Setting Linear Equality Constraints Using the setEquality and addEquality Functions

You can also set the properties for linear equality constraints using `setEquality`. Suppose that you have a portfolio of five assets and want to ensure that the first three assets are 50% of your portfolio. Given a `PortfolioCVaR` object `p`, use `setEquality` to set the linear equality constraints:

```
A = [ 1 1 1 0 0 ];  
b = 0.5;  
p = PortfolioCVaR;  
p = setEquality(p, A, b);  
disp(p.NumAssets)  
disp(p.AEquality)  
disp(p.bEquality)
```

5

1 1 1 0 0

0.5000

Suppose that you want to add another linear equality constraint to ensure that the last three assets also constitute 50% of your portfolio. You can set up an augmented system of linear equalities or use `addEquality` to build up linear equality constraints. For this example, create another system of equalities:

```
p = PortfolioCVaR;  
A = [ 1 1 1 0 0 ];      % first equality constraint
```

```

b = 0.5;
p = setEquality(p, A, b);

A = [ 0 0 1 1 1 ];    % second equality constraint
b = 0.5;
p = addEquality(p, A, b);

disp(p.NumAssets)
disp(p.AEquality)
disp(p.bEquality)

5

1      1      1      0      0
0      0      1      1      1

0.5000
0.5000

```

The PortfolioCVaR object, `setEquality`, and `addEquality` implement scalar expansion on the `bEquality` property based on the dimension of the matrix in the `AEquality` property.

See Also

`PortfolioCVaR` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with Linear Inequality Constraints Using PortfolioCVaR Object

Linear inequality constraints are optional linear constraints that impose systems of inequalities on portfolio weights (see “Linear Inequality Constraints” on page 5-8). Linear inequality constraints have properties `AInequality` for the inequality constraint matrix, and `bInequality` for the inequality constraint vector.

Setting Linear Inequality Constraints Using the PortfolioCVaR Function

The properties for linear inequality constraints are set using the `PortfolioCVaR` object. Suppose that you have a portfolio of five assets and you want to ensure that the first three assets are no more than 50% of your portfolio. To set up these constraints:

```
A = [ 1 1 1 0 0 ];  
b = 0.5;  
p = PortfolioCVaR('AInequality', A, 'bInequality', b);  
disp(p.NumAssets)  
disp(p.AInequality)  
disp(p.bInequality)
```

5

1 1 1 0 0

0.5000

Setting Linear Inequality Constraints Using the `setInequality` and `addInequality` Functions

You can also set the properties for linear inequality constraints using `setInequality`. Suppose that you have a portfolio of five assets and you want to ensure that the first three assets constitute no more than 50% of your portfolio. Given a `PortfolioCVaR` object `p`, use `setInequality` to set the linear inequality constraints:

```
A = [ 1 1 1 0 0 ];  
b = 0.5;  
p = PortfolioCVaR;  
p = setInequality(p, A, b);  
disp(p.NumAssets)  
disp(p.AInequality)  
disp(p.bInequality)
```

5

1 1 1 0 0

0.5000

Suppose that you want to add another linear inequality constraint to ensure that the last three assets constitute at least 50% of your portfolio. You can set up an augmented system of linear inequalities or use the `addInequality` function to build up linear inequality constraints. For this example, create another system of inequalities:


```

p = PortfolioCVaR;
A = [ 1 1 1 0 0 ];    % first inequality constraint
b = 0.5;
p = setInequality(p, A, b);

A = [ 0 0 -1 -1 -1 ];    % second inequality constraint
b = -0.5;
p = addInequality(p, A, b);

disp(p.NumAssets)
disp(p.AInequality)
disp(p.bInequality)

5

1      1      1      0      0
0      0     -1     -1     -1

0.5000
-0.5000

```

The PortfolioCVaR object, `setInequality`, and `addInequality` implement scalar expansion on the `bInequality` property based on the dimension of the matrix in the `AInequality` property.

See Also

`PortfolioCVaR` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioCVaR Objects

When any one, or any combination of 'Conditional' BoundType, MinNumAssets, or MaxNumAssets constraints are active, the portfolio problem is formulated by adding NumAssets binary variables, where 0 indicates not invested, and 1 is invested. For example, to explain the 'Conditional' BoundType and MinNumAssets and MaxNumAssets constraints, assume that your portfolio has a universe of 100 assets that you want to invest:

- 'Conditional' BoundType (also known as semicontinuous constraints), set by `setBounds`, is often used in situations where you do not want to invest small values. A standard example is a portfolio optimization problem where many small allocations are not attractive because of transaction costs. Instead, you prefer fewer instruments in the portfolio with larger allocations. This situation can be handled using 'Conditional' BoundType constraints for a PortfolioCVaR object.

For example, the weight you invest in each asset is either 0 or between $[0.01, 0.5]$. Generally, a semicontinuous variable x is a continuous variable between bounds $[lb, ub]$ that also can assume the value 0, where $lb > 0$, $lb \leq ub$. Applying this to portfolio optimization requires that very small or large positions should be avoided, that is values that fall in $(0, lb)$ or are more than ub .

- MinNumAssets and MaxNumAssets (also known as cardinality constraints), set by `setMinMaxNumAssets`, limit the number of assets in a PortfolioCVaR object. For example, if you have 100 assets in your portfolio and you want the number of assets allocated in the portfolio to be from 40 through 60. Using MinNumAssets and MaxNumAssets you can limit the number of assets in the optimized portfolio, which allows you to limit transaction and operational costs or to create an index tracking portfolio.

Setting 'Conditional' BoundType Constraints Using the setBounds Function

Use `setBounds` with a 'conditional' BoundType to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i=1, \dots, \text{NumAssets}$:

```
p = PortfolioCVaR;
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3)
```

p =

PortfolioCVaR with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
ProbabilityLevel: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: []
Name: []
NumAssets: 3
AssetList: []
InitPort: []
```

```

AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [3×1 double]
UpperBound: [3×1 double]
LowerBudget: []
UpperBudget: []
GroupMatrix: []
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: [3×1 categorical]

```

Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function

You can also set the `MinNumAssets` and `MaxNumAssets` properties to define a limit on the number of assets invested using `setMinMaxNumAssets`. For example, by setting `MinNumAssets=MaxNumAssets=2`, only two of the three assets are invested in the portfolio.

```

p = PortfolioCVaR;
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3)
p = setMinMaxNumAssets(p, 2, 2)

```

p =

PortfolioCVaR with properties:

```

BuyCost: []
SellCost: []
RiskFreeRate: []
ProbabilityLevel: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: []
Name: []
NumAssets: 3
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [3×1 double]
UpperBound: [3×1 double]
LowerBudget: []
UpperBudget: []
GroupMatrix: []

```

```
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: 2
MaxNumAssets: 2
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: [3×1 categorical]
```

See Also

`PortfolioCVaR` | `setBounds` | `setMinMaxNumAssets` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Working with 'Simple' Bound Constraints Using PortfolioCVaR Object” on page 5-55
- “Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with Average Turnover Constraints Using PortfolioCVaR Object

The turnover constraint is an optional linear absolute value constraint (see “Average Turnover Constraints” on page 5-13) that enforces an upper bound on the average of purchases and sales. The turnover constraint can be set using the `PortfolioCVaR` object or the `setTurnover` function. The turnover constraint depends on an initial or current portfolio, which is assumed to be zero if not set when the turnover constraint is set. The turnover constraint has properties `Turnover`, for the upper bound on average turnover, and `InitPort`, for the portfolio against which turnover is computed.

Setting Average Turnover Constraints Using the PortfolioCVaR Function

The properties for the turnover constraints are set using the `PortfolioCVaR` object. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and you want to ensure that average turnover is no more than 30%. To set this turnover constraint:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioCVaR('Turnover', 0.3, 'InitPort', x0);
disp(p.NumAssets)
disp(p.Turnover)
disp(p.InitPort)
```

```
10
```

```
0.3000
```

```
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

Note if the `NumAssets` or `InitPort` properties are not set before or when the turnover constraint is set, various rules are applied to assign default values to these properties (see “Setting Up an Initial or Current Portfolio” on page 5-33).

Setting Average Turnover Constraints Using the setTurnover Function

You can also set properties for portfolio turnover using `setTurnover` to specify both the upper bound for average turnover and an initial portfolio. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and want to ensure that average turnover is no more than 30%. Given a `PortfolioCVaR` object `p`, use `setTurnover` to set the turnover constraint with and without the initial portfolio being set previously:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioCVaR('InitPort', x0);
p = setTurnover(p, 0.3);
```

```
disp(p.NumAssets)
```

```
disp(p.Turnover)
disp(p.InitPort)
```

```
10
```

```
0.3000
```

```
0.1200
```

```
0.0900
```

```
0.0800
```

```
0.0700
```

```
0.1000
```

```
0.1000
```

```
0.1500
```

```
0.1100
```

```
0.0800
```

```
0.1000
```

```
or
```

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
```

```
p = PortfolioCVaR;
```

```
p = setTurnover(p, 0.3, x0);
```

```
disp(p.NumAssets)
```

```
disp(p.Turnover)
```

```
disp(p.InitPort)
```

```
10
```

```
0.3000
```

```
0.1200
```

```
0.0900
```

```
0.0800
```

```
0.0700
```

```
0.1000
```

```
0.1000
```

```
0.1500
```

```
0.1100
```

```
0.0800
```

```
0.1000
```

`setTurnover` implements scalar expansion on the argument for the initial portfolio. If the `NumAssets` property is already set in the `PortfolioCVaR` object, a scalar argument for `InitPort` expands to have the same value across all dimensions. In addition, `setTurnover` lets you specify `NumAssets` as an optional argument. To clear turnover from your `PortfolioCVaR` object, use the `PortfolioCVaR` object or `setTurnover` with empty inputs for the properties to be cleared.

See Also

`PortfolioCVaR` | `setDefaultConstraints` | `setBounds` | `setBudget` |
`setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` |
`setTurnover` | `setOneWayTurnover`

Related Examples

- “Creating the `PortfolioCVaR` Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51

- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Working with One-Way Turnover Constraints Using PortfolioCVaR Object

One-way turnover constraints are optional constraints (see “One-way Turnover Constraints” on page 5-13) that enforce upper bounds on net purchases or net sales. One-way turnover constraints can be set using the `PortfolioCVaR` object or the `setOneWayTurnover` function. One-way turnover constraints depend upon an initial or current portfolio, which is assumed to be zero if not set when the turnover constraints are set. One-way turnover constraints have properties `BuyTurnover`, for the upper bound on net purchases, `SellTurnover`, for the upper bound on net sales, and `InitPort`, for the portfolio against which turnover is computed.

Setting One-Way Turnover Constraints Using the PortfolioCVaR Function

The Properties for the one-way turnover constraints are set using the `PortfolioCVaR` object. Suppose that you have an initial portfolio with 10 assets in a variable `x0` and you want to ensure that turnover on purchases is no more than 30% and turnover on sales is no more than 20% of the initial portfolio. To set these turnover constraints:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioCVaR('BuyTurnover', 0.3, 'SellTurnover', 0.2, 'InitPort', x0);
disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)
```

10

0.3000

0.2000

0.1200

0.0900

0.0800

0.0700

0.1000

0.1000

0.1500

0.1100

0.0800

0.1000

If the `NumAssets` or `InitPort` properties are not set before or when the turnover constraint is set, various rules are applied to assign default values to these properties (see “Setting Up an Initial or Current Portfolio” on page 5-33).

Setting Turnover Constraints Using the setOneWayTurnover Function

You can also set properties for portfolio turnover using `setOneWayTurnover` to specify to the upper bounds for turnover on purchases (`BuyTurnover`) and sales (`SellTurnover`) and an initial portfolio. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and want to ensure that turnover on purchases is no more than 30% and that turnover on sales is no more than 20% of the initial portfolio. Given a `PortfolioCVaR` object `p`, use `setOneWayTurnover` to set the turnover constraints with and without the initial portfolio being set previously:


```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioCvAR('InitPort', x0);
p = setOneWayTurnover(p, 0.3, 0.2);
```

```
disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)
```

```
10
0.3000
0.2000
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

or

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioCvAR;
p = setOneWayTurnover(p, 0.3, 0.2, x0);
disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)
```

```
10
0.3000
0.2000
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

`setOneWayTurnover` implements scalar expansion on the argument for the initial portfolio. If the `NumAssets` property is already set in the `PortfolioCvAR` object, a scalar argument for `InitPort` expands to have the same value across all dimensions. In addition, `setOneWayTurnover` lets you specify `NumAssets` as an optional argument. To remove one-way turnover from your `PortfolioCvAR` object, use the `PortfolioCvAR` object or `setOneWayTurnover` with empty inputs for the properties to be cleared.

See Also

`PortfolioCVaR` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Validate the CVaR Portfolio Problem” on page 5-81
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Validate the CVaR Portfolio Problem

In this section...

“Validating a CVaR Portfolio Set” on page 5-81

“Validating CVaR Portfolios” on page 5-82

Sometimes, you may want to validate either your inputs to, or outputs from, a portfolio optimization problem. Although most error checking that occurs during the problem setup phase catches most difficulties with a portfolio optimization problem, the processes to validate CVaR portfolio sets and portfolios are time consuming and are best done offline. So, the portfolio optimization tools have specialized functions to validate CVaR portfolio sets and portfolios. For information on the workflow when using `PortfolioCVaR` objects, see “PortfolioCVaR Object Workflow” on page 5-16.

Validating a CVaR Portfolio Set

Since it is necessary and sufficient that your CVaR portfolio set must be a nonempty, closed, and bounded set to have a valid portfolio optimization problem, the `estimateBounds` function lets you examine your portfolio set to determine if it is nonempty and, if nonempty, whether it is bounded. Suppose that you have the following CVaR portfolio set which is an empty set because the initial portfolio at θ is too far from a portfolio that satisfies the budget and turnover constraint:

```
p = PortfolioCVaR('NumAssets', 3, 'Budget', 1);
p = setTurnover(p, 0.3, 0);
```

If a CVaR portfolio set is empty, `estimateBounds` returns NaN bounds and sets the `isbounded` flag to []:

```
[lb, ub, isbounded] = estimateBounds(p)
```

```
lb =
```

```
NaN
NaN
NaN
```

```
ub =
```

```
NaN
NaN
NaN
```

```
isbounded =
```

```
[]
```

Suppose that you create an unbounded CVaR portfolio set as follows:

```
p = PortfolioCVaR('AInequality', [1 -1; 1 1], 'bInequality', 0);
[lb, ub, isbounded] = estimateBounds(p)
```

```
lb =
```

```
-Inf
-Inf
```

```
ub =  
  
1.0e-008 *  
  
-0.3712  
    Inf  
  
isbounded =  
  
0
```

In this case, `estimateBounds` returns (possibly infinite) bounds and sets the `isbounded` flag to `false`. The result shows which assets are unbounded so that you can apply bound constraints as necessary.

Finally, suppose that you created a CVaR portfolio set that is both nonempty and bounded. `estimateBounds` not only validates the set, but also obtains tighter bounds which are useful if you are concerned with the actual range of portfolio choices for individual assets in your portfolio set:

```
p = PortfolioCVaR;  
p = setBudget(p, 1,1);  
p = setBounds(p, [ -0.1; 0.2; 0.3; 0.2 ], [ 0.5; 0.3; 0.9; 0.8 ]);  
  
[lb, ub, isbounded] = estimateBounds(p)  
  
lb =  
  
-0.1000  
0.2000  
0.3000  
0.2000  
  
ub =  
  
0.3000  
0.3000  
0.7000  
0.6000  
  
isbounded =  
  
1
```

In this example, all but the second asset has tighter upper bounds than the input upper bound implies.

Validating CVaR Portfolios

Given a CVaR portfolio set specified in a `PortfolioCVaR` object, you often want to check if specific portfolios are feasible with respect to the portfolio set. This can occur with, for example, initial portfolios and with portfolios obtained from other procedures. The `checkFeasibility` function determines whether a collection of portfolios is feasible. Suppose that you perform the following portfolio optimization and want to determine if the resultant efficient portfolios are feasible relative to a modified problem.

First, set up a problem in the `PortfolioCVaR` object `p`, estimate efficient portfolios in `pwgt`, and then confirm that these portfolios are feasible relative to the initial problem:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontier(p);

checkFeasibility(p, pwgt)

```

```
ans =
```

```

1      1      1      1      1      1      1      1      1      1

```

Next, set up a different portfolio problem that starts with the initial problem with an additional a turnover constraint and an equally weighted initial portfolio:

```

q = setTurnover(p, 0.3, 0.25);
checkFeasibility(q, pwgt)

```

```
ans =
```

```

0      0      0      1      1      0      0      0      0      0

```

In this case, only two of the 10 efficient portfolios from the initial problem are feasible relative to the new problem in `PortfolioCVaR` object `q`. Solving the second problem using `checkFeasibility` demonstrates that the efficient portfolio for `PortfolioCVaR` object `q` is feasible relative to the initial problem:

```

qwgt = estimateFrontier(q);
checkFeasibility(p, qwgt)

```

```
ans =
```

```

1      1      1      1      1      1      1      1      1      1

```

See Also

`PortfolioCVaR` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36

- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object

There are two ways to look at a portfolio optimization problem that depends on what you are trying to do. One goal is to estimate efficient portfolios and the other is to estimate efficient frontiers. The “Obtaining Portfolios Along the Entire Efficient Frontier” on page 5-86 example focuses on the former goal and “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105 focuses on the latter goal. For information on the workflow when using PortfolioCVaR objects, see “PortfolioCVaR Object Workflow” on page 5-16.

Obtaining Portfolios Along the Entire Efficient Frontier

The most basic way to obtain optimal portfolios is to obtain points over the entire range of the efficient frontier.

Given a portfolio optimization problem in a `PortfolioCVaR` object, the `estimateFrontier` function computes efficient portfolios spaced evenly according to the return proxy from the minimum to maximum return efficient portfolios. The number of portfolios estimated is controlled by the hidden property `defaultNumPorts` which is set to 10. A different value for the number of portfolios estimated is specified as an input argument to `estimateFrontier`. This example shows the default number of efficient portfolios over the entire range of the efficient frontier.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontier(p);
disp(pwgt)
```

0.8651	0.6963	0.5325	0.3655	0.1949	0.0220	0	0	0
0.0600	0.1580	0.2539	0.3560	0.4660	0.5742	0.4571	0.3116	0.1599
0.0318	0.0451	0.0509	0.0554	0.0577	0.0666	0.0505	0.0253	0.0077
0.0430	0.1006	0.1627	0.2230	0.2814	0.3372	0.4924	0.6631	0.8324

If you want only four portfolios, you can use `estimateFrontier` with `NumPorts` specified as 4.

```
pwgt = estimateFrontier(p, 4);
disp(pwgt)
```

0.8651	0.3655	0	0
0.0600	0.3560	0.4571	0
0.0318	0.0554	0.0505	0
0.0430	0.2230	0.4924	1.0000

Starting from the initial portfolio, `estimateFrontier` also returns purchases and sales to get from your initial portfolio to each efficient portfolio on the efficient frontier. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontier(p);

display(pwgt)

pwgt = 4×10
```


0.8651	0.6963	0.5325	0.3655	0.1949	0.0220	0	0	0
0.0600	0.1580	0.2539	0.3560	0.4660	0.5742	0.4571	0.3116	0.1599
0.0318	0.0451	0.0509	0.0554	0.0577	0.0666	0.0505	0.0253	0.0077
0.0430	0.1006	0.1627	0.2230	0.2814	0.3372	0.4924	0.6631	0.8324

```
display(pbuy)
```

```
pbuy = 4×10
```

0.5651	0.3963	0.2325	0.0655	0	0	0	0	0
0	0	0	0.0560	0.1660	0.2742	0.1571	0.0116	0
0	0	0	0	0	0	0	0	0
0	0.0006	0.0627	0.1230	0.1814	0.2372	0.3924	0.5631	0.7324

```
display(psell)
```

```
psell = 4×10
```

0	0	0	0	0.1051	0.2780	0.3000	0.3000	0.3000	0.3
0.2400	0.1420	0.0461	0	0	0	0	0	0.1401	0.3
0.1682	0.1549	0.1491	0.1446	0.1423	0.1334	0.1495	0.1747	0.1923	0.2
0.0570	0	0	0	0	0	0	0	0	

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

PortfolioCVaR | estimateFrontier | estimateFrontierLimits |
 estimateFrontierByReturn | estimatePortReturn | estimateFrontierByRisk |
 estimatePortRisk | estimateFrontierByRisk | setSolver

Related Examples

- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- [Getting Started with Portfolio Optimization](#) (4 min 13 sec)
- [CVaR Portfolio Optimization](#) (4 min 56 sec)

Obtaining Endpoints of the Efficient Frontier

Often when using a `PortfolioCVaR` object, you might be interested in the endpoint portfolios for the efficient frontier. Suppose that you want to determine the range of returns from minimum to maximum to refine a search for a portfolio with a specific target return. Use the `estimateFrontierLimits` function to obtain the endpoint portfolios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);
pwgt = estimateFrontierLimits(p);

disp(pwgt)
```

0.8651	0
0.0600	0
0.0318	0
0.0430	1.0000

The `estimatePortMoments` function shows the range of risks and returns for efficient portfolios. Note that the endpoints of the efficient frontier depend upon the `Scenarios` in the `PortfolioCVaR` object. If you change the `Scenarios`, you are likely to obtain different endpoints.

Starting from an initial portfolio, `estimateFrontierLimits` also returns purchases and sales to get from the initial portfolio to the endpoint portfolios on the efficient frontier. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontierLimits(p);
```

```
display(pwgt)
```

```
pwgt = 4×2
```

```
0.8673    0
0.0523    0
0.0414    0
0.0390    1.0000
```

```
display(pbuy)
```

```
pbuy = 4×2
```

```
0.5673    0
0          0
0          0
0          0.9000
```

```
display(psell)
```

```
psell = 4×2
```

```
0          0.3000
0.2477    0.3000
0.1586    0.2000
0.0610    0
```

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

[PortfolioCVaR](#) | [estimateFrontier](#) | [estimateFrontierLimits](#) | [estimateFrontierByReturn](#) | [estimatePortReturn](#) | [estimateFrontierByRisk](#) | [estimatePortRisk](#) | [estimateFrontierByRisk](#) | [setSolver](#)

Related Examples

- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Obtaining Efficient Portfolios for Target Returns

This example shows how to obtain efficient portfolios that have targeted portfolio returns using the `estimateFrontierByReturn` function.

The `estimateFrontierByReturn` function accepts one or more target portfolios returns and obtains efficient portfolios with the specified returns. For example, assume that you have a universe of four assets where you want to obtain efficient portfolios with target portfolio returns of 6%, 9%, and 12%:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontierByReturn(p, [0.07, 0.10, .12]);
display(pwgt)
```

```
pwgt = 4×3

    0.7435    0.3656    0.1088
    0.1257    0.3411    0.4946
    0.0510    0.0782    0.0939
    0.0799    0.2151    0.3027
```

Sometimes, you can request a return for which no efficient portfolio exists. Based on the previous code, suppose that you want a portfolio with a 4% return (which is the return of the first asset). A portfolio that is fully invested in the first asset, however, is inefficient. `estimateFrontierByReturn` warns if your target returns are outside the range of efficient portfolio returns and replaces it with the endpoint portfolio of the efficient frontier closest to your target return. The best way to avoid this situation is to bracket your target portfolio returns with `estimateFrontierLimits` and `estimatePortReturn` (see “Obtaining Endpoints of the Efficient Frontier” on page 5-89 and “Obtaining CVaR Portfolio Risks and Returns” on page 5-105).

```
pret = estimatePortReturn(p, p.estimateFrontierLimits);
display(pret)
```

```
pret = 2×1

    0.0657
    0.1790
```

This result indicates that efficient portfolios have returns that range from 6.5% to 17.8%. Note, your results for this code may be different due to the random generation of scenarios.

If you have an initial portfolio, `estimateFrontierByReturn` also returns purchases and sales to get from your initial portfolio to the target portfolios on the efficient frontier. For example, given an initial portfolio in `pwgt0`, to obtain purchases and sales with target returns of 7%, 10%, and 12%:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontierByReturn(p, [0.07, 0.10, .12]);
```

```
display(pwgt)
```

```
pwgt = 4×3
```

0.7435	0.3656	0.1088
0.1257	0.3411	0.4946
0.0510	0.0782	0.0939
0.0799	0.2151	0.3027

```
display(pbuy)
```

```
pbuy = 4×3
```

0.4435	0.0656	0
0	0.0411	0.1946
0	0	0
0	0.1151	0.2027

```
display(psell)
```

```
psell = 4×3
```

0	0	0.1912
0.1743	0	0
0.1490	0.1218	0.1061
0.0201	0	0

If you do not have an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

`PortfolioCVaR` | `estimateFrontier` | `estimateFrontierLimits` |
`estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` |
`estimatePortRisk` | `estimateFrontierByRisk` | `setSolver`

Related Examples

- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36

- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Obtaining Efficient Portfolios for Target Risks

This example shows how to obtain efficient portfolios that have targeted portfolio risks using the `estimateFrontierByRisk` function.

The `estimateFrontierByRisk` function accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks. Suppose that you have a universe of four assets where you want to obtain efficient portfolios with target portfolio risks of 12%, 14%, and 16%.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.9);

pwgt = estimateFrontierByRisk(p, [0.12, 0.14, 0.16]);

display(pwgt)

pwgt = 4x3

    0.3390    0.2367    0.1349
    0.3551    0.4087    0.4720
    0.0772    0.0875    0.0927
    0.2287    0.2670    0.3004
```

Sometimes, you can request a risk for which no efficient portfolio exists. Based on the previous code, suppose that you want a portfolio with 6% risk (individual assets in this universe have risks ranging from 7% to 42.5%). It turns out that a portfolio with 6% risk cannot be formed with these four assets.. `estimateFrontierByRisk` warns if your target risks are outside the range of efficient portfolio risks and replaces it with the endpoint of the efficient frontier closest to your target risk. The best way to avoid this situation is to bracket your target portfolio risks with `estimateFrontierLimits` and `estimatePortRisk` (see “Obtaining Endpoints of the Efficient Frontier” on page 5-89 and “Obtaining CVaR Portfolio Risks and Returns” on page 5-105).

```
prsk = estimatePortRisk(p, p.estimateFrontierLimits);
display(prsk)

prsk = 2x1

    0.0707
    0.4338
```

This result indicates that efficient portfolios have risks that range from 7% to 42.5%. Note, your results for this code may be different due to the random generation of scenarios.

Starting with an initial portfolio, `estimateFrontierByRisk` also returns purchases and sales to get from your initial portfolio to the target portfolios on the efficient frontier. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales from the example with target risks of 12%, 14%, and 16%:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];  
p = setInitPort(p, pwgt0);  
[pwgt, pbuy, psell] = estimateFrontierByRisk(p, [0.12, 0.14, 0.16]);
```

```
display(pwgt)
```

```
pwgt = 4×3
```

0.3390	0.2367	0.1349
0.3551	0.4087	0.4720
0.0772	0.0875	0.0927
0.2287	0.2670	0.3004

```
display(pbuy)
```

```
pbuy = 4×3
```

0.0390	0	0
0.0551	0.1087	0.1720
0	0	0
0.1287	0.1670	0.2004

```
display(psell)
```

```
psell = 4×3
```

0	0.0633	0.1651
0	0	0
0.1228	0.1125	0.1073
0	0	0

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

`PortfolioCVaR` | `estimateFrontier` | `estimateFrontierLimits` |
`estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` |
`estimatePortRisk` | `estimateFrontierByRisk` | `setSolver`

Related Examples

- “Estimate Efficient Portfolios for Entire Frontier for `PortfolioCVaR` Object” on page 5-85
- “Creating the `PortfolioCVaR` Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Estimate Efficient Frontiers for `PortfolioCVaR` Object” on page 5-105
- “Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36

- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Choosing and Controlling the Solver for PortfolioCVaR Optimizations

When solving portfolio optimizations for a `PortfolioCVaR` object, you are solving nonlinear optimization problems with either nonlinear objective or nonlinear constraints. You can use 'TrustRegionCP' (default), 'ExtendedCP', or 'cuttingplane' solvers that implement Kelley's cutting plane method (see Kelley [45] at "Portfolio Optimization" on page A-5). Alternatively, you can use `fmincon` and all variations of `fmincon` from Optimization Toolbox are supported. When using `fmincon` as the `solverType`, 'sqp' is the default algorithm for `fmincon`.

Using 'TrustRegionCP', 'ExtendedCP', and 'cuttingplane' SolverTypes

The 'TrustRegionCP', 'ExtendedCP', and 'cuttingplane' solvers have options to control the number iterations and stopping tolerances. Moreover, these solvers use `linprog` as the master solver, and all `linprog` options are supported using `optimoptions` structures. All these options are set using `setSolver`.

For example, you can use `setSolver` to increase the number of iterations for 'TrustRegionCP':

```
p = PortfolioCVaR;
p = setSolver(p, 'TrustRegionCP', 'MaxIterations', 2000);
display(p.solverType)
display(p.solverOptions)
```

```
trustregioncp
      MaxIterations: 2000
  AbsoluteGapTolerance: 1.0000e-07
  RelativeGapTolerance: 1.0000e-05
  NonlinearScalingFactor: 1000
  ObjectiveScalingFactor: 1000
    MainSolverOptions: [1x1 optim.options.Linprog]
              Display: 'off'
          CutGeneration: 'basic'
  MaxIterationsInactiveCut: 30
        ActiveCutTolerance: 1.0000e-07
            ShrinkRatio: 0.7500
  TrustRegionStartIteration: 2
            DeltaLimit: 1
```

To change the main solver algorithm to 'interior-point', with no display, use `setSolver` to modify 'MainSolverOptions':

```
p = PortfolioCVaR;
options = optimoptions('linprog','Algorithm','interior-point','Display','off');
p = setSolver(p,'TrustRegionCP','MainSolverOptions',options);
display(p.solverType)
display(p.solverOptions)
display(p.solverOptions.MainSolverOptions.Algorithm)
display(p.solverOptions.MainSolverOptions.Display)
```

```
trustregioncp
      MaxIterations: 1000
  AbsoluteGapTolerance: 1.0000e-07
  RelativeGapTolerance: 1.0000e-05
  NonlinearScalingFactor: 1000
  ObjectiveScalingFactor: 1000
    MaainSolverOptions: [1x1 optim.options.Linprog]
```

```

            Display: 'off'
            CutGeneration: 'basic'
MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
        ShrinkRatio: 0.7500
TrustRegionStartIteration: 2
    DeltaLimit: 1

```

```

interior-point
off

```

Using 'fmincon' SolverType

Unlike Optimization Toolbox which uses the `interior-point` algorithm as the default algorithm for `fmincon`, the portfolio optimization for a `PortfolioCVaR` object uses the `sqp` algorithm. For details about `fmincon` and constrained nonlinear optimization algorithms and options, see “Constrained Nonlinear Optimization Algorithms”.

To modify `fmincon` options for CVaR portfolio optimizations, use `setSolver` to set the hidden properties `solverType` and `solverOptions` to specify and control the solver. (Note that you can see the default options by creating a dummy `PortfolioCVaR` object, using `p = PortfolioCVaR` and then type `p.solverOptions`.) Since these solver properties are hidden, you cannot set them using the `PortfolioCVaR` object. The default for the `fmincon` solver is to use the `sqp` algorithm objective function, gradients turned on, and no displayed output, so you do not need to use `setSolver` to specify the `sqp` algorithm.

```

p = PortfolioCVaR;
p = setSolver(p, 'fmincon');
display(p.solverOptions)

```

`fmincon` options:

```

Options used by current Algorithm ('sqp'):
(Other available algorithms: 'active-set', 'interior-point', 'sqp-legacy', 'trust-region-refl

```

Set properties:

```

        Algorithm: 'sqp'
        ConstraintTolerance: 1.0000e-08
            Display: 'off'
        OptimalityTolerance: 1.0000e-08
SpecifyConstraintGradient: 1
SpecifyObjectiveGradient: 1
        StepTolerance: 1.0000e-08

```

Default properties:

```

FiniteDifferenceStepSize: 'sqrt(eps)'
FiniteDifferenceType: 'forward'
MaxFunctionEvaluations: '100*numberOfVariables'
        MaxIterations: 400
ObjectiveLimit: -1.0000e+20
        OutputFcn: []
        PlotFcn: []
ScaleProblem: 0
        TypicalX: 'ones(numberOfVariables,1)'
UseParallel: 0

```

If you want to specify additional options associated with the `fmincon` solver, `setSolver` accepts these options as name-value pair arguments. For example, if you want to use `fmincon` with the 'active-set' algorithm and with no displayed output, use `setSolver` with:

```
p = PortfolioCVaR;
p = setSolver(p, 'fmincon', 'Algorithm', 'active-set', 'Display', 'off');
display(p.solverOptions.Algorithm)
display(p.solverOptions.Display)

active-set
off
```

Alternatively, `setSolver` accepts an `optimoptions` object from Optimization Toolbox as the second argument. For example, you can change the algorithm to 'trust-region-reflective' with no displayed output as follows:

```
p = PortfolioCVaR;
options = optimoptions('fmincon', 'Algorithm', 'trust-region-reflective', 'Display', 'off');
p = setSolver(p, 'fmincon', options);
display(p.solverOptions.Algorithm)
display(p.solverOptions.Display)

trust-region-reflective
off
```

Using the Mixed Integer Nonlinear Programming (MINLP) Solver

The mixed integer nonlinear programming (MINLP) solver, configured using `setSolverMINLP`, enables you to specify associated solver options for portfolio optimization for a `PortfolioCVaR` object. The MINLP solver is used when any one, or any combination of 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active. In this case, the portfolio problem is formulated by adding `NumAssets` binary variables, where 0 indicates not invested, and 1 is invested. For more information on using 'Conditional' `BoundType`, see `setBounds`. For more information on specifying `MinNumAssets` and `MaxNumAssets`, see `setMinMaxNumAssets`.

When using the `estimate` functions with a `PortfolioCVaR` object where 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active, the mixed integer nonlinear programming (MINLP) solver is automatically used.

Solver Guidelines for PortfolioCVaR Objects

The following table provides guidelines for using `setSolver` and `setSolverMINLP`.

PortfolioCVaR Problem	PortfolioCVaR Function	Type of Optimization Problem	Main Solver	Helper Solver
PortfolioCVaR without active 'Conditional' BoundType, MinNumAssets, and MaxNumAssets	estimateFrontierByRisk	Optimizing a portfolio for a certain risk level introduces a nonlinear constraint. Therefore, this problem has a linear objective with linear and nonlinear constraints.	'TrustRegionCP', 'ExtendedCP', 'fmincon', or 'cuttingplane' using setSolver	'linprog' using setSolver
PortfolioCVaR without active 'Conditional' BoundType, MinNumAssets, and MaxNumAssets	estimateFrontierByReturn	Nonlinear objective with linear constraints	'TrustRegionCP', 'ExtendedCP', 'fmincon', or 'cuttingplane' using setSolver	'linprog' using setSolver
PortfolioCVaR without active 'Conditional' BoundType, MinNumAssets, and MaxNumAssets	estimateFrontierLimits	Nonlinear or linear objective with linear constraints	For 'min': nonlinear objective, 'TrustRegionCP', 'ExtendedCP', 'fmincon', or 'cuttingplane' using setSolver For 'max': linear objective, 'linprog' using setSolver	Not applicable

PortfolioCVaR Problem	PortfolioCVaR Function	Type of Optimization Problem	Main Solver	Helper Solver
PortfolioCVaR with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierByRisk	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'fmincon' is used when the estimate functions reduce the problem into NLP. This solver is configured through setSolver.
PortfolioCVaR with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierByReturn	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'fmincon' is used when the estimate functions reduce the problem into NLP. This solver is configured through setSolver

PortfolioCvAR Problem	PortfolioCvAR Function	Type of Optimization Problem	Main Solver	Helper Solver
PortfolioCvAR with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierLimits	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'fmincon' is used when the estimate functions reduce the problem into NLP. This solver is configured through setSolver

See Also

PortfolioCvAR | estimateFrontier | estimateFrontierLimits | estimateFrontierByReturn | estimatePortReturn | estimateFrontierByRisk | estimatePortRisk | estimateFrontierByRisk | setConditionalBudget | setSolver | setSolverMINLP

Related Examples

- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCvAR Object” on page 5-85
- “Creating the PortfolioCvAR Object” on page 5-22
- “Working with CvAR Portfolio Constraints Using Defaults” on page 5-51
- “Estimate Efficient Frontiers for PortfolioCvAR Object” on page 5-105
- “Asset Returns and Scenarios Using PortfolioCvAR Object” on page 5-36
- “Troubleshooting CvAR Portfolio Optimization Results” on page 5-121
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Hedging Using CvAR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CvAR Portfolio” on page 5-137
- “Mixed-Integer CvAR Portfolio Optimization Problem” on page 5-141
- “Adding Constraints to Satisfy UCITS Directive” on page 4-457

More About

- “PortfolioCvAR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCvAR Object Workflow” on page 5-16
- “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81

External Websites

- [Getting Started with Portfolio Optimization \(4 min 13 sec\)](#)
- [CVaR Portfolio Optimization \(4 min 56 sec\)](#)

Estimate Efficient Frontiers for PortfolioCVaR Object

In this section...

“Obtaining CVaR Portfolio Risks and Returns” on page 5-105

“Obtaining Portfolio Standard Deviation and VaR” on page 5-106

Whereas “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85 focused on estimation of efficient portfolios, this topic focuses on the estimation of efficient frontiers. For information on the workflow when using PortfolioCVaR objects, see “PortfolioCVaR Object Workflow” on page 5-16.

Obtaining CVaR Portfolio Risks and Returns

This example shows how to use the functions `estimatePortReturn` and `estimatePortRisk` with a `PortfolioCVaR` object to provide estimates for the return (or return proxy), risk (or the risk proxy). These functions have the same input syntax but have different combinations of outputs.

Suppose that you have this following portfolio optimization problem that gave you a collection of portfolios along the efficient frontier in `pwgt`:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = setInitPort(p, pwgt0);
pwgt = estimateFrontier(p);
```

Given `pwgt0` and `pwgt`, use the portfolio risk and return estimation functions to obtain risks and returns for your initial portfolio and the portfolios on the efficient frontier:

```
prsk0 = estimatePortRisk(p, pwgt0);
pret0 = estimatePortReturn(p, pwgt0);
prsk = estimatePortRisk(p, pwgt);
pret = estimatePortReturn(p, pwgt);
```

You obtain these risks and returns:

```
display(prsk0)
```

```
prsk0 =
0.0577
```

```
display(pret0)
```

```
pret0 =  
0.0071
```

```
display(prsk)
```

```
prsk = 10×1
```

```
0.0404  
0.0439  
0.0526  
0.0646  
0.0786  
0.0937  
0.1121  
0.1363  
0.1638  
0.1932
```

```
display(pret)
```

```
pret = 10×1
```

```
0.0050  
0.0061  
0.0072  
0.0083  
0.0093  
0.0104  
0.0115  
0.0126  
0.0136  
0.0147
```

Obtaining Portfolio Standard Deviation and VaR

This example shows how to compute standard deviations of portfolio returns and the value-at-risk of portfolios for a `PortfolioCVaR` object with the functions `estimatePortStd` and `estimatePortVaR`. These functions work with any `PortfolioCVaR` portfolios, not necessarily efficient portfolios.

For example, the following code obtains five portfolios (`pwgt`) on the efficient frontier and also has an initial portfolio in `pwgt0`. Various portfolio statistics are computed that include the return, risk, standard deviation, and value-at-risk. The listed estimates are for the initial portfolio in the first row followed by estimates for each of the five efficient portfolios in subsequent rows.

```
m = [ 0.0042; 0.0083; 0.01; 0.15 ];  
C = [ 0.005333 0.00034 0.00016 0;  
0.00034 0.002408 0.0017 0.000992;  
0.00016 0.0017 0.0048 0.0028;  
0 0.000992 0.0028 0.010208 ];
```

```

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioCVaR('initport', pwgt0);
p = simulateNormalScenariosByMoments(p, m, C, 20000);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.9);

pwgt = estimateFrontier(p, 5);

pret = estimatePortReturn(p, [pwgt0, pwgt]);
prsk = estimatePortRisk(p, [pwgt0, pwgt]);
pstd = estimatePortStd(p, [pwgt0, pwgt]);
pvar = estimatePortVaR(p, [pwgt0, pwgt]);

[pret, prsk, pstd, pvar]

ans = 6×4

    0.0206    0.0454    0.0377    0.0276
    0.0949    0.0215    0.0658   -0.0096
    0.1086    0.0219    0.0738   -0.0130
    0.1223    0.0229    0.0821   -0.0161
    0.1360    0.0246    0.0907   -0.0189
    0.1497    0.0272    0.0999   -0.0205

```

See Also

PortfolioCVaR | estimatePortReturn | plotFrontier | estimatePortStd | estimatePortVaR

Related Examples

- “Plotting the Efficient Frontier for a PortfolioCVaR Object” on page 5-109
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Postprocessing Results to Set Up Tradable Portfolios” on page 5-115
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

- CVaR Portfolio Optimization (4 min 56 sec)

Plotting the Efficient Frontier for a PortfolioCVaR Object

This example shows how to use the `plotFrontier` function creates a plot of the efficient frontier for a given portfolio optimization problem. This function accepts several types of inputs and generates a plot with an optional possibility to output the estimates for portfolio risks and returns along the efficient frontier. `plotFrontier` has four different ways that it can be used. In addition to a plot of the efficient frontier, if you have an initial portfolio in the `InitPort` property, `plotFrontier` also displays the return versus risk of the initial portfolio on the same plot. If you have a well-posed portfolio optimization problem set up in a `PortfolioCVaR` object and you use `plotFrontier`, you get a plot of the efficient frontier with the default number of portfolios on the frontier (the default number is currently 10 and is maintained in the hidden property `defaultNumPorts`).

Create Efficient Frontier Plot

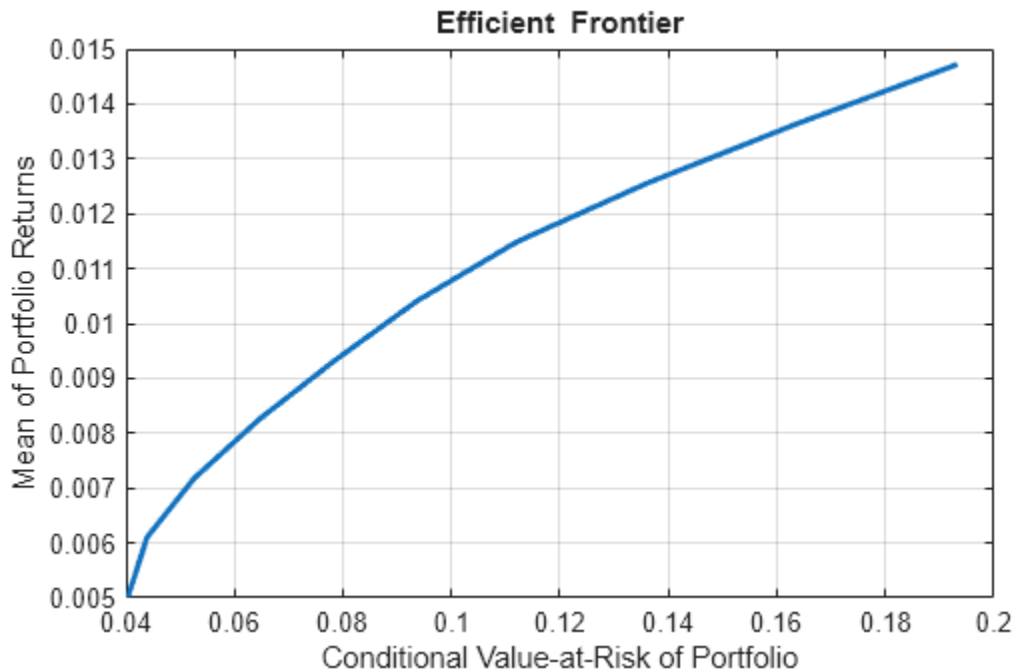
To plot the efficient frontier for a `PortfolioCVaR` object, use `plotFrontier`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

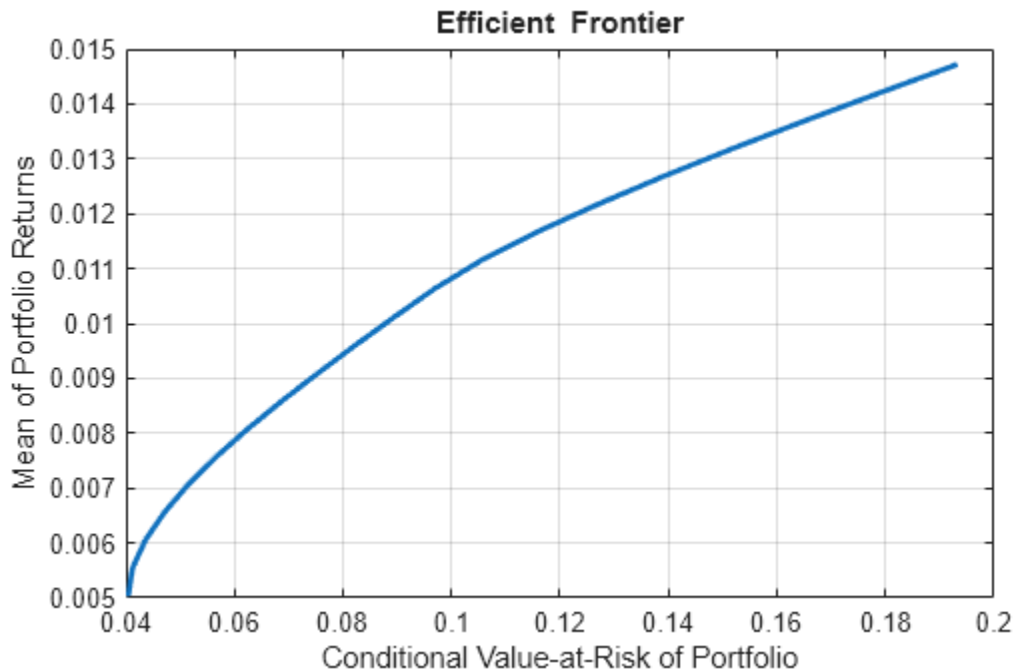
plotFrontier(p)
```



The `Name` property appears as the title of the efficient frontier plot if you set it in the `PortfolioCVaR` object. Without an explicit name, the title on the plot would be "Efficient Frontier." If you want to obtain a specific number of portfolios along the efficient frontier, use `plotFrontier` with the number of portfolios that you want.

Suppose that you have this `PortfolioCVaR` object and you want to plot 20 portfolios along the efficient frontier and to obtain 20 risk and return values for each portfolio:

```
[prsk, pret] = plotFrontier(p, 20);
```

```
display([pret, prsk])
```

```
0.0050    0.0404
0.0055    0.0412
0.0060    0.0435
0.0066    0.0471
0.0071    0.0515
0.0076    0.0567
0.0081    0.0625
0.0086    0.0689
0.0091    0.0756
0.0096    0.0825
0.0101    0.0896
0.0106    0.0970
0.0111    0.1055
0.0117    0.1156
0.0122    0.1269
0.0127    0.1390
0.0132    0.1519
0.0137    0.1653
0.0142    0.1791
0.0147    0.1932
```

Plotting Existing Efficient Portfolios

If you already have efficient portfolios from any of the "estimateFrontier" functions (see "Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object" on page 5-85), pass them into `plotFrontier` directly to plot the efficient frontier

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
```

```

    0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

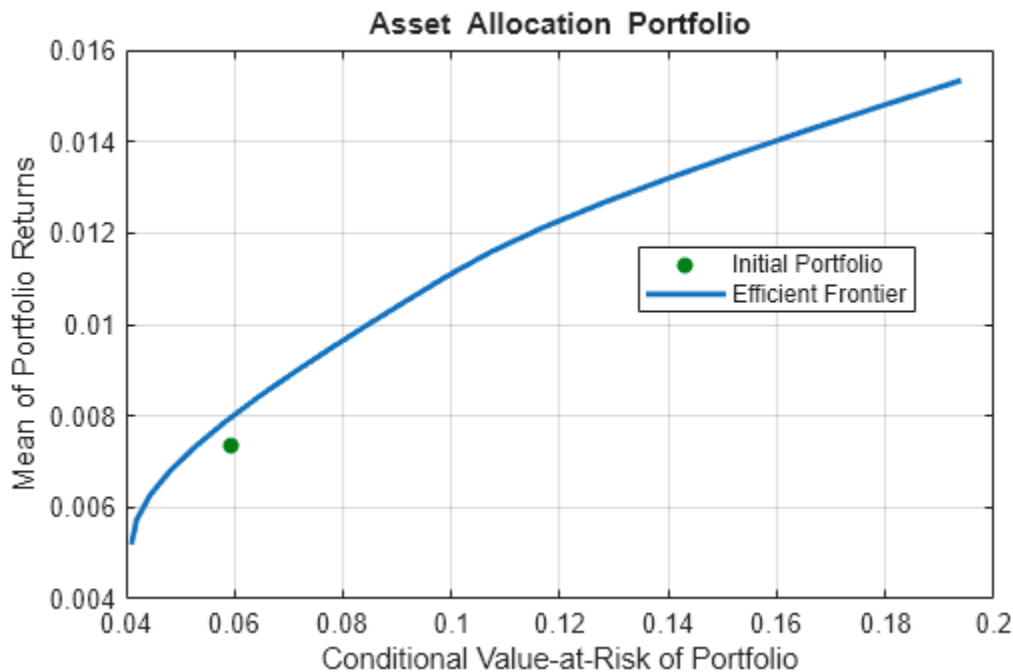
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioCVaR('Name', 'Asset Allocation Portfolio', 'InitPort', pwgt0);

p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontier(p, 20);
plotFrontier(p, pwgt)

```



Plotting Existing Efficient Portfolio Risks and Returns

If you already have efficient portfolio risks and returns, you can use the interface to `plotFrontier` to pass them into `plotFrontier` to obtain a plot of the efficient frontier:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

AssetScenarios = mvnrnd(m, C, 20000);

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioCVaR('Name', 'Asset Allocation Portfolio', 'InitPort', pwgt0);

```

```

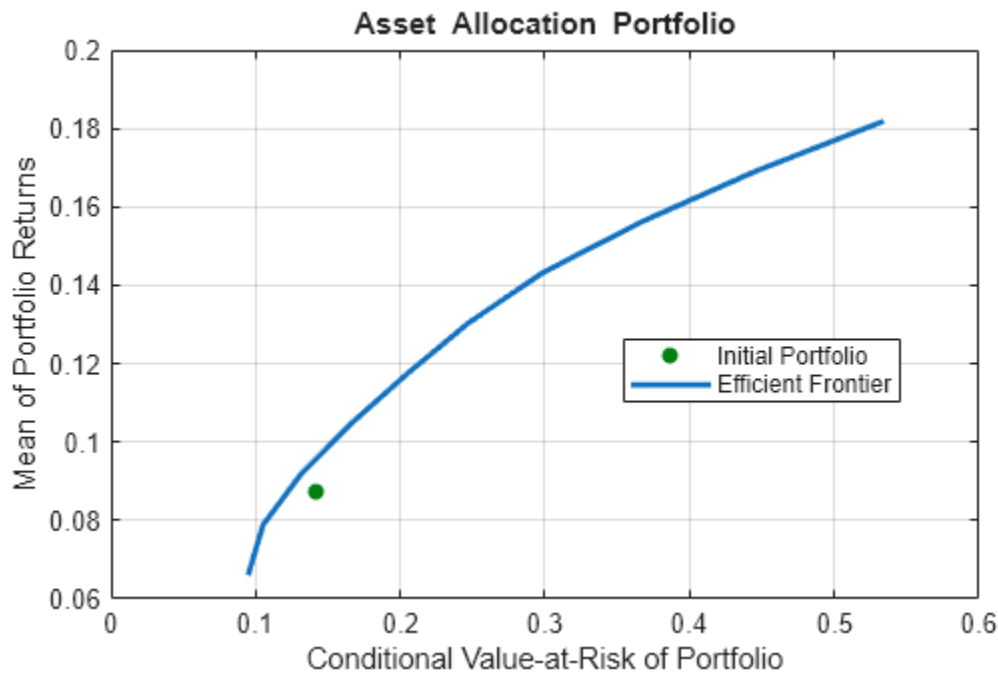
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontier(p);

pret= estimatePortReturn(p, pwgt);
prsk = estimatePortRisk(p, pwgt);

plotFrontier(p, prsk, pret)

```



See Also

PortfolioCVaR | estimatePortReturn | plotFrontier | estimatePortStd | estimatePortVaR

Related Examples

- "Estimate Efficient Frontiers for PortfolioCVaR Object" on page 5-105
- "Creating the PortfolioCVaR Object" on page 5-22
- "Working with CVaR Portfolio Constraints Using Defaults" on page 5-51
- "Asset Returns and Scenarios Using PortfolioCVaR Object" on page 5-36
- "Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object" on page 5-85
- "Postprocessing Results to Set Up Tradable Portfolios" on page 5-115
- "Hedging Using CVaR Portfolio Optimization" on page 5-125
- "Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio" on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Postprocessing Results to Set Up Tradable Portfolios

This example shows how to use your results for efficient portfolios or estimates for expected portfolio risks and returns to set up trades to move toward an efficient portfolio. For information on the workflow when using PortfolioCVaR objects, see "PortfolioCVaR Object Workflow" on page 5-16.

Suppose that you set up a portfolio optimization problem and obtained portfolios on the efficient frontier. Use the `dataset` object to form a blotter that lists your portfolios with the names for each asset. For example, suppose that you want to obtain five portfolios along the efficient frontier. You can set up a blotter with weights multiplied by 100 to view the allocations for each portfolio:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioCVaR;
p = setAssetList(p, 'Bonds','Large-Cap Equities','Small-Cap Equities','Emerging Equities');
p = setInitPort(p, pwgt0);
p = simulateNormalScenariosByMoments(p, m, C, 20000);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.9);

pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end

Blotter = dataset([100*pwgt],pnames,'obsnames',p.AssetList);
display(Blotter)
```

```
Blotter =
```

	Port1	Port2	Port3	Port4	Port5
Bonds	77.826	42.77	7.2095	0	0
Large-Cap Equities	10.549	30.253	50.951	26.764	0
Small-Cap Equities	4.3487	7.1893	9.8049	10.837	0
Emerging Equities	7.2764	19.788	32.034	62.399	100

This result indicates that you would invest primarily in bonds at the minimum-risk/minimum-return end of the efficient frontier (Port1), and that you would invest completely in emerging equity at the maximum-risk/maximum-return end of the efficient frontier (Port5). You can also select a particular efficient portfolio, for example, suppose that you want a portfolio with 15% risk and you add purchase and sale weights outputs obtained from the "estimateFrontier" functions to set up a trade blotter:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioCVaR;
p = setAssetList(p, 'Bonds','Large-Cap Equities','Small-Cap Equities','Emerging Equities');

p = setInitPort(p, pwgt0);
p = simulateNormalScenariosByMoments(p, m, C, 20000);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.9);

[pwgt, pbuy, psell] = estimateFrontierByRisk(p, 0.15);

Blotter = dataset([100*[pwgt0, pwgt, pbuy, psell]], ...
{'Initial','Weight', 'Purchases','Sales'},'obsnames',p.AssetList);
display(Blotter)

Blotter =
```

	Initial	Weight	Purchases	Sales
Bonds	30	21.084	0	8.9159
Large-Cap Equities	30	41.374	11.374	0
Small-Cap Equities	20	9.2259	0	10.774
Emerging Equities	10	28.316	18.316	0

If you have prices for each asset (in this example, they can be ETFs), add them to your blotter and then use the tools of the `dataset` object to obtain shares and shares to be traded. For an example, see “Asset Allocation Case Study” on page 4-180.

See Also

`PortfolioCVaR` | `estimateScenarioMoments` | `checkFeasibility`

Related Examples

- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- [Getting Started with Portfolio Optimization \(4 min 13 sec\)](#)
- [CVaR Portfolio Optimization \(4 min 56 sec\)](#)

Working with Other Portfolio Objects

This example shows how to examine portfolio optimization problems according to different combinations of return and risk proxies. The `PortfolioCVaR` object is for CVaR portfolio optimization. The `Portfolio` object is for mean-variance portfolio optimization. Sometimes, you might want to examine portfolio optimization problems according to different combinations of return and risk proxies. A common example is that you want to do a CVaR portfolio optimization and then want to work primarily with moments of portfolio returns. Suppose that you set up a CVaR portfolio optimization problem with:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

rng('default')
p = PortfolioCVaR;
p = setAssetList(p, 'Bonds', 'Large-Cap Equities', 'Small-Cap Equities', 'Emerging Equities');
p = setInitPort(p, pwgt0);
p = simulateNormalScenariosByMoments(p, m, C, 20000);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.9);
```

To work with the same problem in a mean-variance framework, you can use the scenarios from the `PortfolioCVaR` object to set up a `Portfolio` object so that `p` contains a CVaR optimization problem and `q` contains a mean-variance optimization problem based on the same data.

```
q = Portfolio('AssetList', p.AssetList);
q = estimateAssetMoments(q, p.getScenarios);
q = setDefaultConstraints(q);

pwgt = estimateFrontier(p);
qwgt = estimateFrontier(q);
```

Since each object has a different risk proxy, it is not possible to compare results side by side. To obtain means and standard deviations of portfolio returns, you can use the functions associated with each object to obtain:

```
pret = estimatePortReturn(p, pwgt);
pstd = estimatePortStd(p, pwgt);
qret = estimatePortReturn(q, qwgt);
qstd = estimatePortStd(q, qwgt);
```

```
[pret, qret]
```

```
ans = 10×2
```

```
0.0676    0.0591
0.0800    0.0724
0.0923    0.0857
0.1047    0.0991
0.1171    0.1124
0.1295    0.1257
```


0.1419	0.1390
0.1543	0.1524
0.1666	0.1657
0.1790	0.1790

[pstd, qstd]

ans = 10×2

0.0787	0.0761
0.0905	0.0823
0.1089	0.0984
0.1312	0.1206
0.1557	0.1462
0.1816	0.1736
0.2132	0.2051
0.2531	0.2466
0.2979	0.2944
0.3462	0.3462

To produce comparable results, you can use the returns or risks from one portfolio optimization as target returns or risks for the other portfolio optimization.

```
qwgt = estimateFrontierByReturn(q, pret);
qret = estimatePortReturn(q, qwgt);
qstd = estimatePortStd(q, qwgt);
```

[pret, qret]

ans = 10×2

0.0676	0.0676
0.0800	0.0800
0.0923	0.0923
0.1047	0.1047
0.1171	0.1171
0.1295	0.1295
0.1419	0.1419
0.1543	0.1543
0.1666	0.1666
0.1790	0.1790

[pstd, qstd]

ans = 10×2

0.0787	0.0787
0.0905	0.0905
0.1089	0.1089
0.1312	0.1312
0.1557	0.1557
0.1816	0.1815
0.2132	0.2132
0.2531	0.2531
0.2979	0.2979

0.3462 0.3462

Now it is possible to compare standard deviations of portfolio returns from either type of portfolio optimization.

See Also

PortfolioCVaR | Portfolio

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16
- “Portfolio Object Workflow” on page 4-18

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Troubleshooting CVaR Portfolio Optimization Results

PortfolioCVaR Object Destroyed When Modifying

If a PortfolioCVaR object is destroyed when modifying, remember to pass an existing object into the PortfolioCVaR object if you want to modify it, otherwise it creates a new object. See "Creating the PortfolioCVaR Object" on page 5-22 for details.

Matrix Incompatibility and "Non-Conformable" Errors

If you get matrix incompatibility or "non-conformable" errors, the representation of data in the tools follows a specific set of basic rules described in "Conventions for Representation of Data" on page 5-20.

CVaR Portfolio Optimization Warns About "Max Iterations"

If the 'cuttingplane' solver displays the following warning:

```
Warning: Max iterations reached. Consider modifying the solver options, or using fmincon.
> In @PortfolioCVaR\private\cvar_cuttingplane_solver at 255
   In @PortfolioCVaR\private\cvar_optim_min_risk at 85
   In PortfolioCVaR.estimateFrontier at 69
```

this warning indicates that some of the reported efficient portfolios may not be accurate enough.

This warning is usually related to portfolios in the lower-left end of the efficient frontier. The cutting plane solver may have gotten very close to the solution, but there may be too many portfolios with very similar risks and returns in that neighborhood, and the solver runs out of iterations before reaching the desired accuracy.

To correct this problem, you can use `setSolver` to make any of these changes:

- Increase the maximum number of iterations ('MaxIter').
- Relax the stopping tolerances ('AbsTol' and/or 'RelTol').
- Use a different main solver algorithm ('MainSolverOptions').
- Alternatively, you can try the 'fmincon' solver.

When the default maximum number of iterations of the 'cuttingplane' solver is reached, the solver usually needs many more iterations to reach the accuracy required by the default stopping tolerances. You may want to combine increasing the number of iterations (e.g., multiply by 5) with relaxing the stopping tolerances (for example, multiply by 10 or 100). Since the CVaR is a stochastic optimization problem, the accuracy of the solution is relative to the scenario sample, so a looser stopping tolerance may be acceptable. Keep in mind that the solution time may increase significantly when you increase the number of iterations. For example, doubling the number of iterations more than doubles the solution time. Sometimes using a different main solver (for example, switching to 'interior-point' if you are using the default 'simplex') can get the 'cuttingplane' solver to converge without changing the maximum number of iterations.

Alternatively, the 'fmincon' solver may be faster than the 'cuttingplane' solver for problems where cutting plane reaches the maximum number of iterations.

CVaR Portfolio Optimization Errors with “Could Not Solve” Message

If the 'cuttingplane' solver generates the following error:

```
Error using cvar_cuttingplane_solver (line 251)
Could not solve the problem. Consider modifying the solver options, or using fmincon.

Error in cvar_optim_by_return (line 100)
    [x,~,~,exitflag] = cvar_cuttingplane_solver(...)

Error in PortfolioCVaR/estimateFrontier (line 80)
    pwgt = cvar_optim_by_return(obj, r(2:end-1), obj.NumAssets, ...
```

this error means that the main solver failed to solve one of the main problems. The error may be due to numerical instability or other problem-specific situation.

To correct this problem, you can use `setSolver` to make any of these changes:

- Modify the main solver options ('MainSolverOptions'), for example, change the algorithm ('Algorithm') or the termination tolerance ('TolFun').
- Alternatively, you can try the 'fmincon' solver.

Missing Data Estimation Fails

If asset return data has missing or NaN values, the `simulateNormalScenariosByData` function with the 'missingdata' flag set to true may fail with either too many iterations or a singular covariance. To correct this problem, consider this:

- If you have asset return data with no missing or NaN values, you can compute a covariance matrix that may be singular without difficulties. If you have missing or NaN values in your data, the supported missing data feature requires that your covariance matrix must be positive-definite, that is, nonsingular.
- `simulateNormalScenariosByData` uses default settings for the missing data estimation procedure that might not be appropriate for all problems.

In either case, you might want to estimate the moments of asset returns separately with either the ECM estimation functions such as `ecmmle` or with your own functions.

cvar_optim_transform Errors

If you obtain optimization errors such as:

```
Error using cvar_optim_transform (line 276)
Portfolio set appears to be either empty or unbounded. Check constraints.

Error in PortfolioCVaR/estimateFrontier (line 64)
    [AI, bI, AE, bE, lB, uB, f0, f, x0] = cvar_optim_transform(obj);
```

or

```
Error using cvar_optim_transform (line 281)
Cannot obtain finite lower bounds for specified portfolio set.
```

```
Error in PortfolioCVaR/estimateFrontier (line 64)
    [AI, bI, AE, bE, lB, uB, f0, f, x0] = cvar_optim_transform(obj);
```

Since the portfolio optimization tools require a bounded portfolio set, these errors (and similar errors) can occur if your portfolio set is either empty and, if nonempty, unbounded. Specifically, the portfolio optimization algorithm requires that your portfolio set have at least a finite lower bound.

The best way to deal with these problems is to use the validation functions in “Validate the CVaR Portfolio Problem” on page 5-81. Specifically, use `estimateBounds` to examine your portfolio set, and use `checkFeasibility` to ensure that your initial portfolio is either feasible and, if infeasible, that you have sufficient turnover to get from your initial portfolio to the portfolio set.

Tip To correct this problem, try solving your problem with larger values for turnover and gradually reduce to the value that you want.

Efficient Portfolios Do Not Make Sense

If you obtain efficient portfolios that, do not seem to make sense, this can happen if you forget to set specific constraints or you set incorrect constraints. For example, if you allow portfolio weights to fall between 0 and 1 and do not set a budget constraint, you can get portfolios that are 100% invested in every asset. Although it may be hard to detect, the best thing to do is to review the constraints you have set with display of the `PortfolioCVaR` object. If you get portfolios with 100% invested in each asset, you can review the display of your object and quickly see that no budget constraint is set. Also, you can use `estimateBounds` and `checkFeasibility` to determine if the bounds for your portfolio set make sense and to determine if the portfolios you obtained are feasible relative to an independent formulation of your portfolio set.

See Also

`PortfolioCVaR` | `estimateScenarioMoments` | `checkFeasibility`

Related Examples

- “Troubleshooting Portfolio Optimization Results” on page 4-145
- “Postprocessing Results to Set Up Tradable Portfolios” on page 5-115
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)

- CVaR Portfolio Optimization (4 min 56 sec)

Hedging Using CVaR Portfolio Optimization

This example shows how to model two hedging strategies using CVaR portfolio optimization with a `PortfolioCVaR` object. First, you simulate the price movements of a stock by using a `gbm` object with `simByEuler`. Then you use CVaR portfolio optimization to estimate the efficient frontier of the portfolios for the returns at the horizon date. Finally, you compare the CVaR portfolio to a mean-variance portfolio to demonstrate the differences between these two types of risk measures.

Monte Carlo Simulation of Asset Scenarios

Scenarios are required to define and evaluate the CVaR portfolio. These scenarios can be generated in multiple ways, as well as obtained from historical observations. This example uses a Monte Carlo simulation of a geometric Brownian motion to generate the scenarios. The Monte Carlo simulation produces trials of a stock price after one year. These stock prices provide the returns of five different investment strategies that define the assets in the CVaR portfolio. This scenario generation strategy can be further generalized to simulations with more stocks, instruments, and strategies.

Define a stock profile.

```
% Price at time 0
Price_0 = 200;

% Drift (annualized)
Drift = 0.08;

% Volatility (annualized)
Vol = 0.4;

% Valuation date
Valuation = datetime(2012,1,1);

% Investment horizon date
Horizon = datetime(2013,1,1);

% Risk-free rate
RiskFreeRate = 0.03;
```

Simulate the price movements of the stock from the valuation date to the horizon date using a `gbm` object with `simByEuler`.

```
% Number of trials for the Monte Carlo simulation
NTRIALS = 100000;

% Length (in years) of the simulation
T = date2time(Valuation, Horizon, 1, 1);

% Number of periods per trial (approximately 100 periods per year)
NPERIODS = round(100*T);

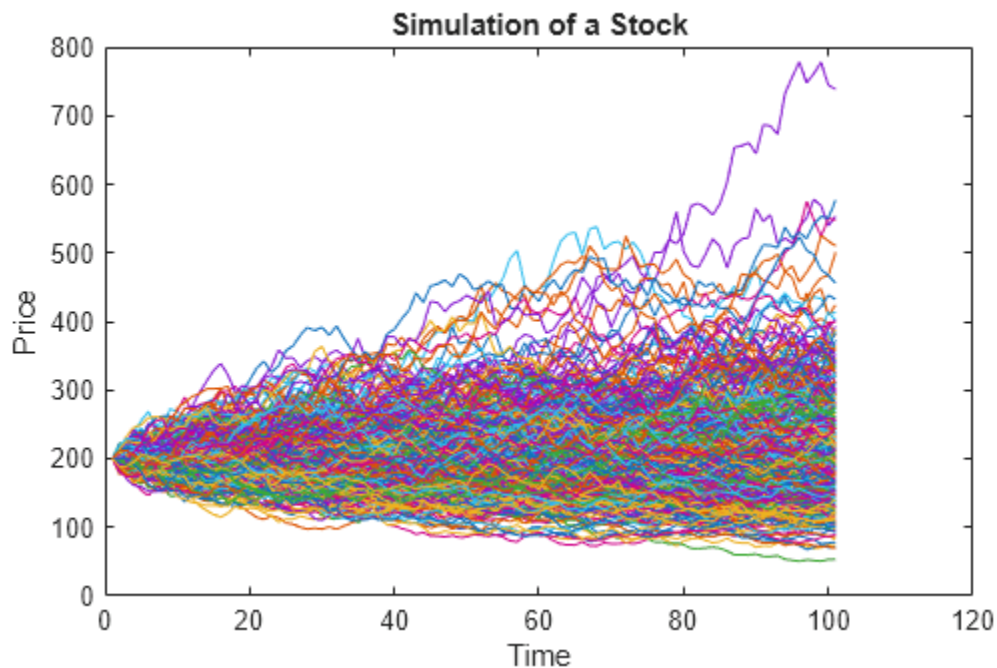
% Length (in years) of each time step per period
dt = T/NPERIODS;

% Instantiate the gbm object
StockGBM = gbm(Drift, Vol, 'StartState', Price_0);
```

```
% Run the simulation
Paths = StockGBM.simByEuler(NPERIODS, 'NTRIALS', NTRIALS, ...
    'DeltaTime', dt, 'Antithetic', true);
```

Plot the simulation of the stock. For efficiency, plot only some scenarios.

```
plot(squeeze(Paths(:, :, 1:500)));
title('Simulation of a Stock');
xlabel('Time');
ylabel('Price');
```



Calculate the prices of different put options using a Black-Scholes model with the `blsprice` function.

```
% Put option with strike at 50% of current underlying price
Strike50 = 0.50*Price_0;
[~, Put50] = blsprice(Price_0, Strike50, RiskFreeRate, T, Vol);

% Put option with strike at 75% of current underlying price
Strike75 = 0.75*Price_0;
[~, Put75] = blsprice(Price_0, Strike75, RiskFreeRate, T, Vol);

% Put option with strike at 90% of current underlying price
Strike90 = 0.90*Price_0;
[~, Put90] = blsprice(Price_0, Strike90, RiskFreeRate, T, Vol);

% Put option with strike at 95% of current underlying price
Strike95 = 0.95*Price_0; % Same as strike
[~, Put95] = blsprice(Price_0, Strike95, RiskFreeRate, T, Vol);
```

The goal is to find the efficient portfolio frontier for the returns at the horizon date. Hence, obtain the scenarios from the trials of the Monte Carlo simulation at the end of the simulation period.

```
Price_T = squeeze(Paths(end, 1, :));
```


Generate the scenario matrix using five strategies. The first strategy is a stock-only strategy; the rest of the strategies are the stock hedged with put options at different strike price levels. To compute the returns from the prices obtained by the Monte Carlo simulation for the stock-only strategy, divide the change in the stock price ($\text{Price}_T - \text{Price}_0$) by the initial price Price_0 . To compute the returns of the stock with different put options, first compute the "observed price" at the horizon date, that is, the stock price with the acquired put option taken into account. If the stock price at the horizon date is less than the strike price, the observed price is the strike price. Otherwise, the observed price is the true stock price. This observed price is represented by the formula

$$\text{Observed price} = \max\{\text{Price}_T, \text{Strike}\}.$$

You can then compute the "initial cost" of the stock with the put option, which is given by the initial stock price Price_0 plus the price of the put option. Finally, to compute the return of the put option strategies, divide the observed price minus the initial cost by the initial cost.

```
AssetScenarios = zeros(NTRIALS, 5);

% Strategy 1: Stock only
AssetScenarios(:, 1) = (Price_T - Price_0) ./ Price_0;

% Strategy 2: Put option cover at 50%
AssetScenarios(:, 2) = (max(Price_T, Strike50) - (Price_0 + Put50)) ./ ...
    (Price_0 + Put50);

% Strategy 2: Put option cover at 75%
AssetScenarios(:, 3) = (max(Price_T, Strike75) - (Price_0 + Put75)) ./ ...
    (Price_0 + Put75);

% Strategy 2: Put option cover at 90%
AssetScenarios(:, 4) = (max(Price_T, Strike90) - (Price_0 + Put90)) ./ ...
    (Price_0 + Put90);

% Strategy 2: Put option cover at 95%
AssetScenarios(:, 5) = (max(Price_T, Strike95) - (Price_0 + Put95)) ./ ...
    (Price_0 + Put95);
```

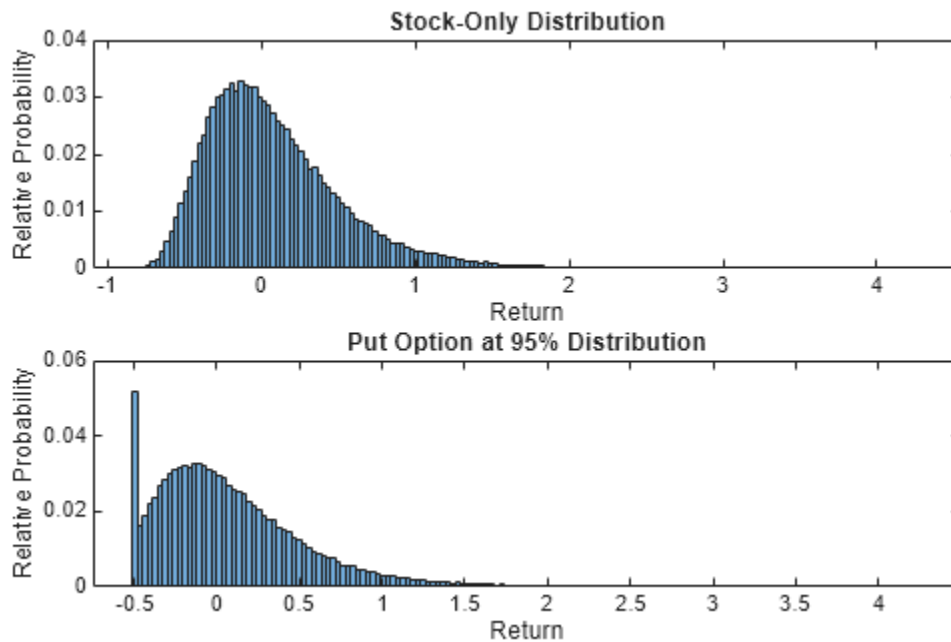
The portfolio weights associated with each of the five assets previously defined represent the percentage of the total wealth to invest in each strategy. For example, consider a portfolio with the weights $[0.5 \ 0 \ 0.5 \ 0]$. The weights indicate that the best allocation is to invest 50% in the stock-only strategy and the remaining 50% in a put option at 75%.

Plot the distribution of the returns for the stock-only strategy and the put option at the 95% strategy. Notice that the returns are not normally distributed. For a mean-variance portfolio, a lack of symmetry in the plotted returns usually indicates poor mean-variance portfolio performance since variance, as a risk measure, is not sensitive to skewed distributions.

```
% Create histogram
figure;

% Stock only
subplot(2,1,1);
histogram(AssetScenarios(:,1), 'Normalization', 'probability')
title('Stock-Only Distribution')
xlabel('Return')
ylabel('Relative Probability')
```

```
% Put option cover
subplot(2,1,2);
histogram(AssetScenarios(:,2),'Normalization','probability')
title('Put Option at 95% Distribution')
xlabel('Return')
ylabel('Relative Probability')
```



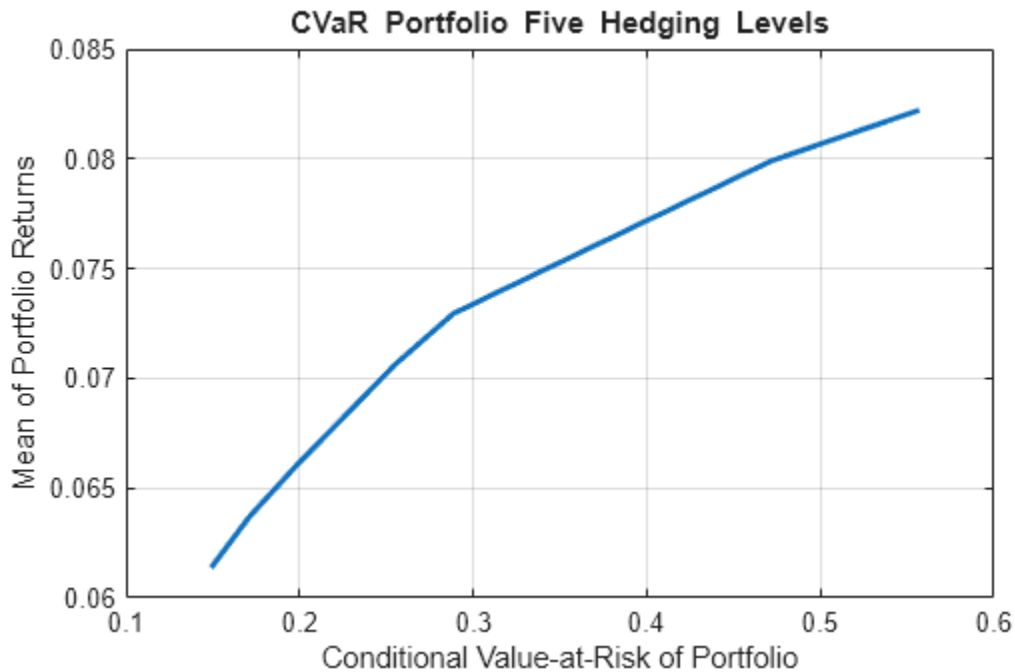
CVaR Efficient Frontier

Create a PortfolioCVaR object using the AssetScenarios from the simulation.

```
p = PortfolioCVaR('Name','CVaR Portfolio Five Hedging Levels',...
    'AssetList',{'Stock','Hedge50','Hedge75','Hedge90','Hedge95'},...
    'Scenarios', AssetScenarios, 'LowerBound', 0, ...
    'Budget', 1, 'ProbabilityLevel', 0.95);
```

```
% Estimate the efficient frontier to obtain portfolio weights
pwgt = estimateFrontier(p);
```

```
% Plot the efficient frontier
figure;
plotFrontier(p, pwgt);
```



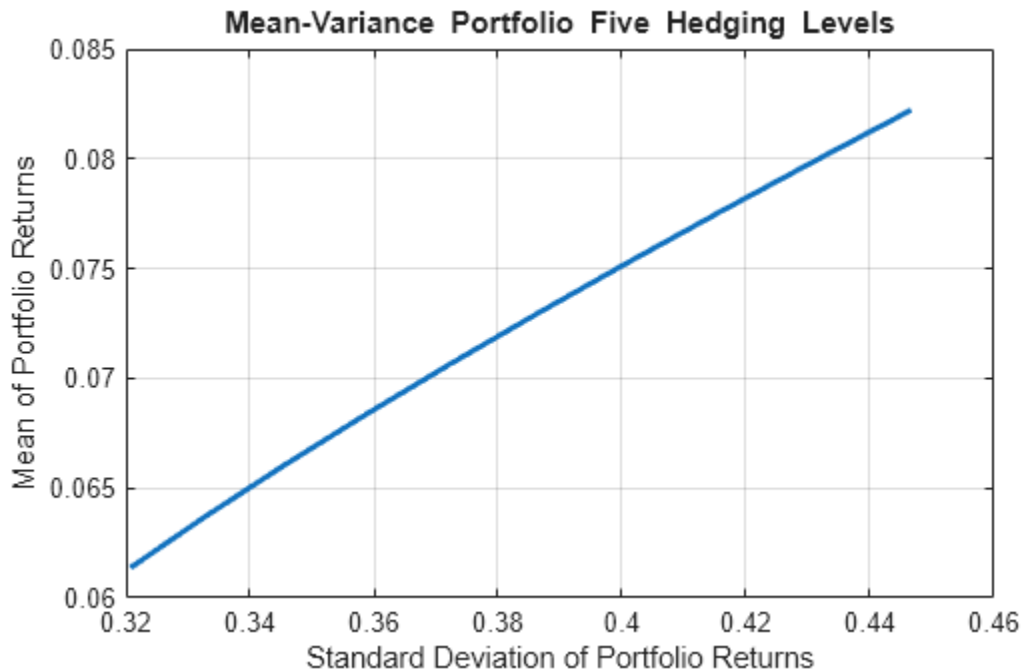
CVaR and Mean-Variance Efficient Frontier Comparison

Create the `Portfolio` object. Use `AssetScenarios` from the simulation to estimate the assets moments. Notice that, unlike for the `PortfolioCVaR` object, only an estimate of the assets moments is required to fully specify a mean-variance portfolio.

```
pmv = Portfolio('Name', 'Mean-Variance Portfolio Five Hedging Levels', ...
    'AssetList', {'Stock', 'Hedge50', 'Hedge75', 'Hedge90', 'Hedge95'});
pmv = estimateAssetMoments(pmv, AssetScenarios);
pmv = setDefaultConstraints(pmv);

% Estimate the efficient frontier to obtain portfolio weights
pwgtmv = estimateFrontier(pmv);

% Plot the efficient frontier
figure;
plotFrontier(pmv, pwgtmv);
```



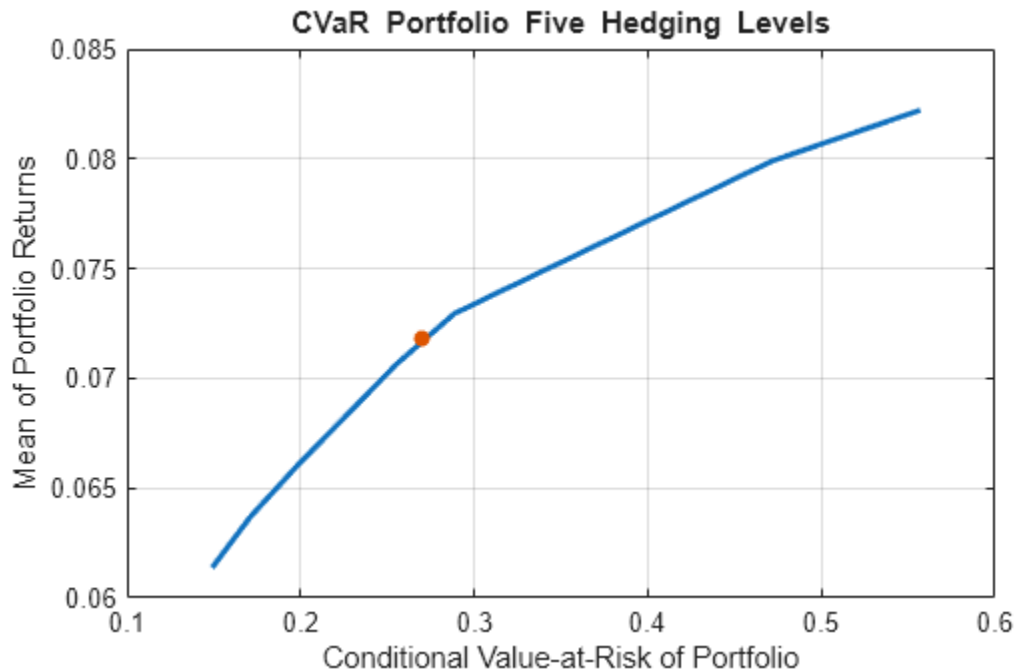
Select a target return. In this case, the target return is the midpoint return on the CVaR portfolio efficient frontier.

```
% Achievable levels of return
pretlimits = estimatePortReturn(p, estimateFrontierLimits(p));
TargetRet = mean(pretlimits); % Target half way up the frontier
```

Plot the efficient CVaR portfolio for the return TargetRet on the CVaR efficient frontier.

```
% Obtain risk level at target return
pwgtTarget = estimateFrontierByReturn(p, TargetRet); % CVaR efficient portfolio
priskTarget = estimatePortRisk(p, pwgtTarget);
```

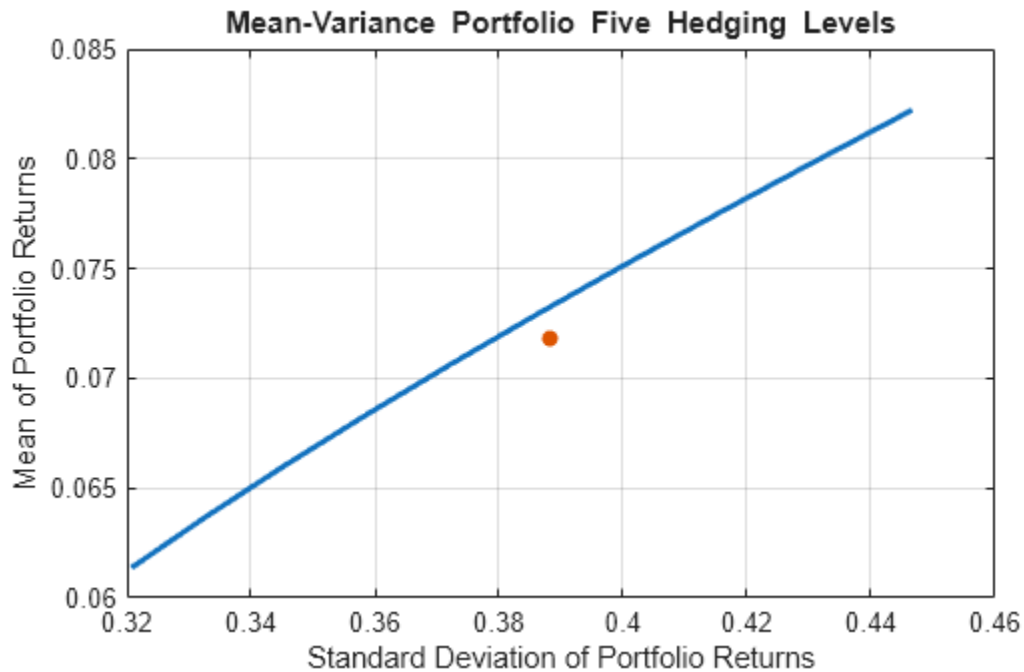
```
% Plot point onto CVaR efficient frontier
figure;
plotFrontier(p, pwgt);
hold on
scatter(priskTarget, TargetRet, [], 'filled');
hold off
```



Plot the efficient CVaR portfolio for the return `TargetRet` on the mean-variance efficient frontier. Notice that the efficient CVaR portfolio is below the mean-variance efficient frontier.

```
% Obtain the variance for the efficient CVaR portfolio
pmvretTarget = estimatePortReturn(pmv,pwgtTarget); % Should be TargetRet
pmvriskTarget = estimatePortRisk(pmv,pwgtTarget); % Risk proxy is variance

% Plot efficient CVaR portfolio onto mean-variance frontier
figure;
plotFrontier(pmv,pwgtmv);
hold on
scatter(pmvriskTarget,pmvretTarget,[],'filled');
hold off
```

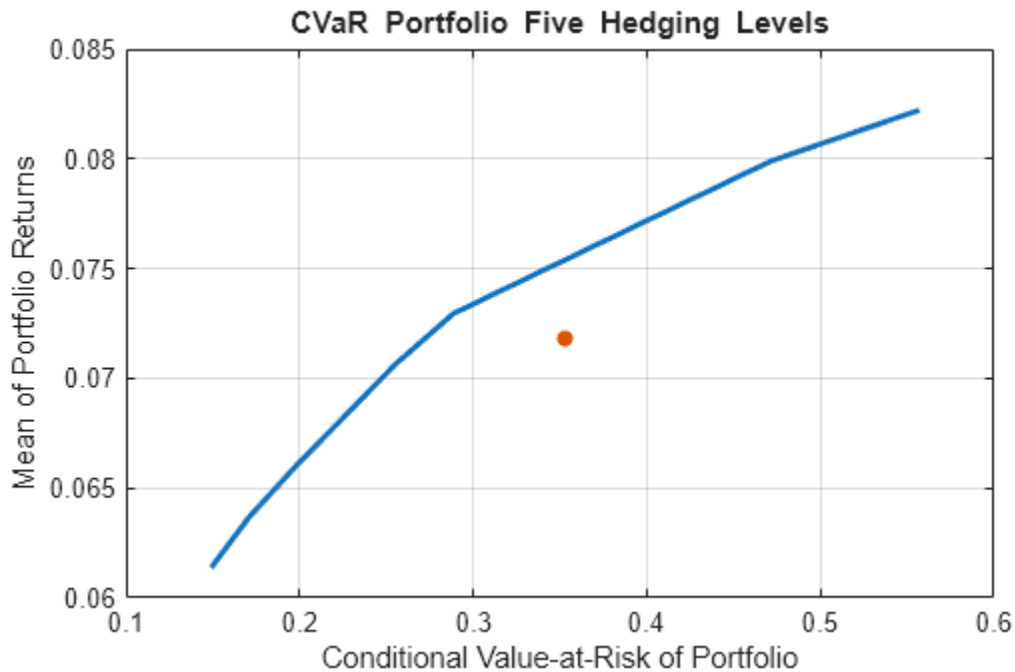


Plot the efficient mean-variance portfolio for the return `TargetRet` on the CVaR efficient frontier. Notice that the efficient mean-variance portfolio is below the CVaR efficient frontier.

```
% Obtain the mean-variance efficient portfolio at target return
pwgtmvTarget = estimateFrontierByReturn(pmv,TargetRet);

% Obtain the CVaR risk for the mean-variance efficient portfolio
pretTargetCVaR = estimatePortReturn(p,pwgtmvTarget); % Should be TargetRet
priskTargetCVaR = estimatePortRisk(p,pwgtmvTarget); % Risk proxy is CVaR

% Plot mean-variance efficient portfolio onto the CVaR frontier
figure;
plotFrontier(p,pwgt);
hold on
scatter(priskTargetCVaR,pretTargetCVaR,[],'filled');
hold off
```



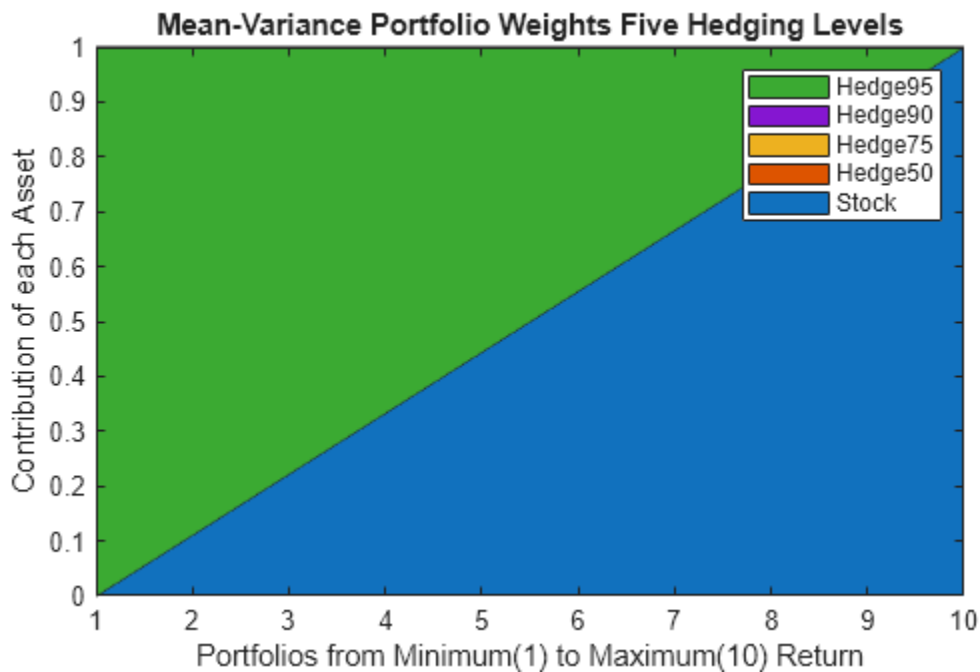
Since mean-variance and CVaR are two different risk measures, this example illustrates that the efficient portfolio for one type of risk measure is not efficient for the other.

CVaR and Mean-Variance Portfolio Weights Comparison

Examine the portfolio weights of the portfolios that make up each efficient frontier to obtain a more detailed comparison between the mean-variance and CVaR efficient frontiers.

Plot the weights associated with the mean-variance portfolio efficient frontier.

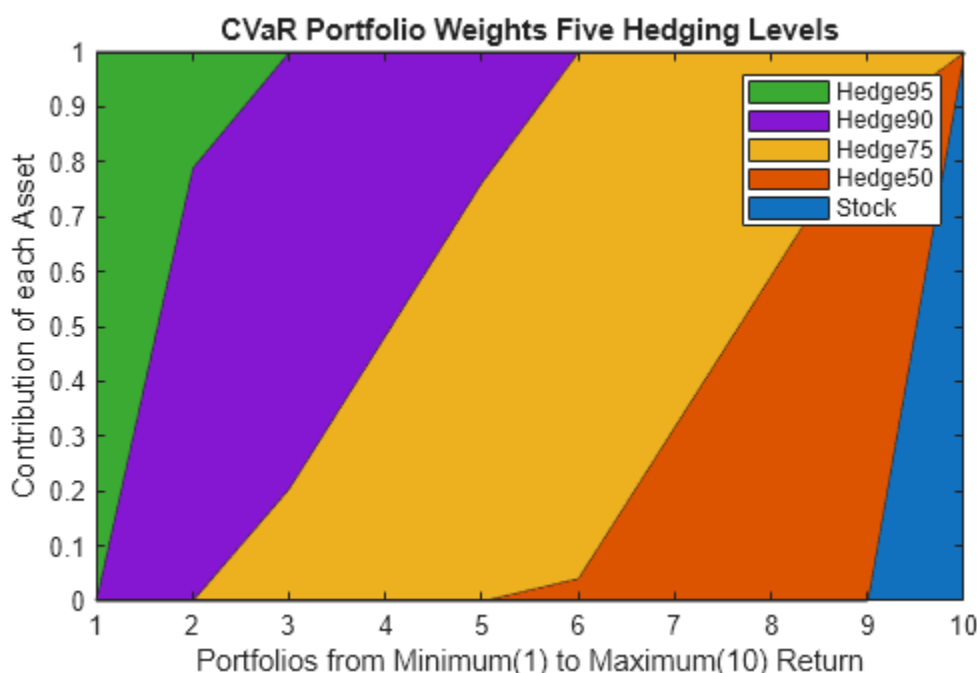
```
% Plot the mean-variance portfolio weights
figure;
area(pwgtmv');
legend(pmv.AssetList);
axis([1 10 0 1])
title('Mean-Variance Portfolio Weights Five Hedging Levels');
xlabel('Portfolios from Minimum(1) to Maximum(10) Return');
ylabel('Contribution of each Asset');
```



The weights associated with the mean-variance portfolios on the efficient frontier use only two strategies, 'Stock' and 'Hedge95'. This behavior is an effect of the correlations among the five assets in the portfolio. Because the correlations between the assets are close to 1, the standard deviation of the portfolio is almost a linear combination of the standard deviation of the assets. Hence, because assets with larger returns are associated with assets with larger variance, a linear combination of only the assets with the smallest and largest returns is observed in the efficient frontier.

Plot the weights associated with the CVaR portfolio efficient frontier.

```
% Plot the CVaR portfolio weights
figure;
area(pwgt');
legend(p.AssestList);
axis([1 10 0 1])
title('CVaR Portfolio Weights Five Hedging Levels');
xlabel('Portfolios from Minimum(1) to Maximum(10) Return');
ylabel('Contribution of each Asset');
```

For both types of portfolios, mean-variance and CVaR, the portfolio with the maximum expected return is the one that allocates all the weight to the stock-only strategy. This makes sense because no put options are acquired, which translates into larger returns. Also, the portfolio with minimum variance is the same for both risk measures. This portfolio is the one that allocates everything to 'Hedge95' because that strategy limits the possible losses the most. The real differences between the two types of portfolios are observed for the return levels between the minimum and maximum. There, in contrast to the efficient portfolios obtained using variance as the measure of risk, the weights of the CVaR portfolios range among all five possible strategies.

See Also

`PortfolioCVaR` | `getScenarios` | `setScenarios` | `estimateScenarioMoments` | `simulateNormalScenariosByMoments` | `simulateNormalScenariosByData` | `setCosts` | `checkFeasibility`

Related Examples

- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio” on page 5-137

More About

- “PortfolioCVaR Object” on page 5-17

- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio

Create a `PortfolioCVaR` object and incorporate a list of assets from `CAPMUniverse.mat`. Use `simulateNormalScenariosByData` to simulate the scenarios for each of the assets. These portfolio constraints require fully invested long-only portfolios (nonnegative weights that must sum to 1).

```
rng(1) % Set the seed for reproducibility.
load CAPMuniverse
```

```
p = PortfolioCVaR('AssetList',Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:,1:12), 20000, 'missingdata',true);
p = setProbabilityLevel(p, 0.95);
p = setDefaultConstraints(p);
disp(p)
```

PortfolioCVaR with properties:

```

    BuyCost: []
    SellCost: []
    RiskFreeRate: []
    ProbabilityLevel: 0.9500
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
    NumScenarios: 20000
    Name: []
    NumAssets: 12
    AssetList: {'AAPL' 'AMZN' 'CSCO' 'DELL' 'EBAY' 'GOOG' 'HPQ' 'IBM' 'JPM' 'KO' 'MSFT' 'PFE' 'TSLA'}
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [12x1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
    ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
    BoundType: [12x1 categorical]
```

To obtain the portfolio that maximizes the reward-to-risk ratio (which is equivalent to the Sharpe ratio for mean-variance portfolios), search on the efficient frontier iteratively for the portfolio that minimizes the negative of the reward-to-risk ratio:

$$- \frac{\text{portfolio return} - \text{risk free rate}}{\text{portfolio CVaR}}.$$

To do so, use the `sratio` function, defined in the Local Functions on page 5-139 section, to return the negative reward-to-risk ratio for a target return. Then, pass this function to `fminbnd`. `fminbnd` iterates through the possible return values and evaluates their associated reward-to-risk ratio. `fminbnd` returns the optimal return for which the maximum reward-to-risk ratio is achieved (or that minimizes the negative of the reward-to-risk ratio).

```
% Obtain the minimum and maximum returns of the portfolio.
pwgtLimits = estimateFrontierLimits(p);
retLimits = estimatePortReturn(p,pwgtLimits);
minret = retLimits(1);
maxret = retLimits(2);

% Search on the frontier iteratively. Find the return that minimizes the
% negative of the reward-to-risk ratio.
fhandle = @(ret) iterative_local_obj(ret,p);
options = optimset('Display', 'off', 'TolX', 1.0e-8);
optret = fminbnd(fhandle, minret, maxret, options);

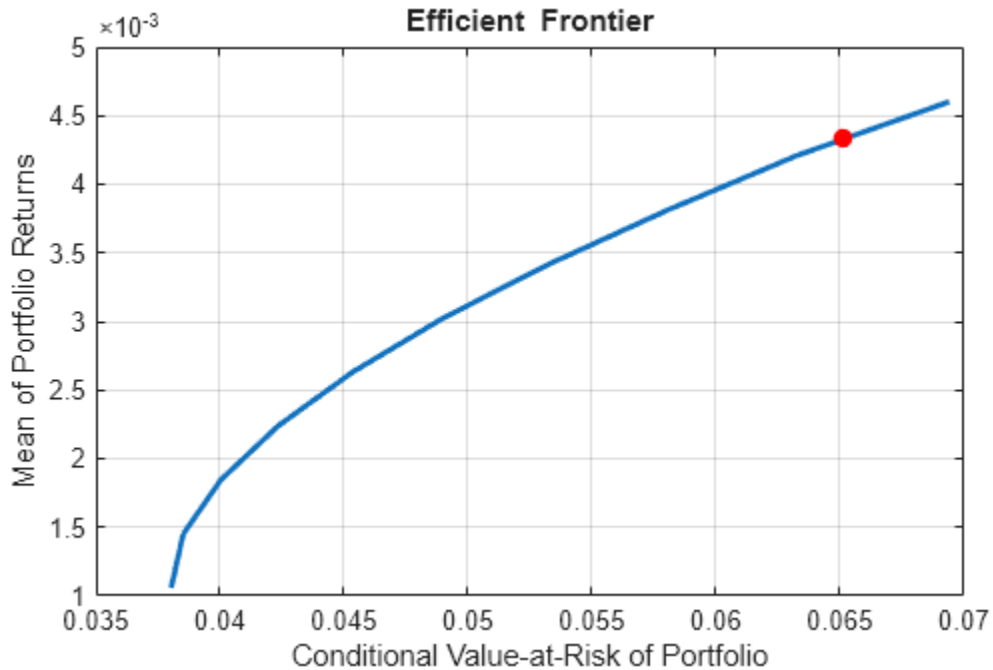
% Obtain the portfolio weights associated with the return that achieves
% the maximum reward-to-risk ratio.
pwgt = estimateFrontierByReturn(p,optret)

pwgt = 12x1

    0.0885
         0
         0
         0
         0
    0.9115
         0
         0
         0
         0
         0
         0
         0
         0
         :
```

Use `plotFrontier` to plot the efficient frontier and `estimatePortRisk` to estimate the maximum reward-to-risk ratio portfolio.

```
plotFrontier(p);
hold on
% Compute the risk level for the maximum reward-to-risk ratio portfolio.
optrsk = estimatePortRisk(p,pwgt);
scatter(optrsk,optret,50,'red','filled')
hold off
```



Local Functions

This local function that computes the negative of the reward-to-risk ratio for a target return level.

```
function sratio = iterative_local_obj(ret, obj)
% Set the objective function to the negative of the reward-to-risk ratio.

risk = estimatePortRisk(obj, estimateFrontierByReturn(obj, ret));

if ~isempty(obj.RiskFreeRate)
    sratio = -(ret - obj.RiskFreeRate)/risk;
else
    sratio = -ret/risk;
end

end
```

See Also

PortfolioCVaR | getScenarios | setScenarios | estimateScenarioMoments | simulateNormalScenariosByMoments | simulateNormalScenariosByData | setCosts | checkFeasibility

Related Examples

- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121
- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85

- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Mixed-Integer CVaR Portfolio Optimization Problem

This example shows how to solve a CVaR portfolio optimization problem with constraints in the number of selected assets or conditional (semicontinuous) bounds. To solve this problem, you can use a `PortfolioCVaR` object along with different mixed integer nonlinear programming (MINLP) solvers.

CVaR Portfolio

Load the returns data in `CAPMuniverse.mat`. Then, create a `PortfolioCVaR` object with default constraints and a long-only portfolio whose weights sum to 1. For this example, you can define the feasible region of weights X as

$$X = \left\{ x \mid \sum_{i=1}^n x_i = 1, x_i \geq 0 \right\}.$$

```
% Load data
load CAPMuniverse.mat

% Create portfolio with default constraints
p = PortfolioCVaR(ProbabilityLevel=0.95);
p = simulateNormalScenariosByData(p,Data(:,1:12),1000,missingdata=true);
p = setDefaultConstraints(p);
```

Include binary variables for this scenario by setting conditional (semicontinuous) bounds. Conditional bounds are those such that $x_i = 0$ or $x_i \geq \ell_i$. In this example, $\ell_i = 0.1$ for all assets.

```
% Set conditional bounds
condLB = 0.1;
condUB = 0.5;
p = setBounds(p,condLB,condUB,BoundType="conditional");
```

Use `estimateFrontier` to estimate a set of portfolios on the efficient frontier. The efficient frontier is a curve that shows the trade-off between the return and risk achieved by Pareto-optimal portfolios. For a given return level, the portfolio on the efficient frontier is the one that minimizes the risk while maintaining the desired return. Conversely, for a given risk level, the portfolio on the efficient frontier is the one that maximizes return while maintaining the desired risk level.

```
% Compute efficient frontier
p = setSolverMINLP(p,'TrustRegionCP',DeltaLowerBound=condLB);
pwgt = estimateFrontier(p)

pwgt = 12x10
```

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0.1000	0.1090	0.1000	0.1000	0	0	0	0.0000	0	0
0	0	0	0.1000	0.1662	0.2634	0.3369	0.4051	0.4647	0.5
0.1328	0.2387	0.3331	0.3263	0.3622	0.3543	0.3701	0.3854	0.4287	0.5
0.1000	0	0	0	0	0	0	0	0	0
0.4132	0.3887	0.2755	0.2552	0.2288	0.1812	0.1110	0	0	0
0	0	0	0	0	0	0	0	0	0
0.2540	0.2636	0.2914	0.2185	0.2427	0.2011	0.1821	0.2095	0.1067	0
0	0	0	0	0	0	0	0	0	0

```

0 0 0 0 0 0 0 0 0
:

% Compute risk and returns of the portfolios on the efficient frontier
rsk = estimatePortRisk(p,pwgt)

rsk = 10x1

    0.0368
    0.0377
    0.0398
    0.0423
    0.0453
    0.0485
    0.0520
    0.0558
    0.0598
    0.0647

ret = estimatePortReturn(p,pwgt)

ret = 10x1

    0.0009
    0.0012
    0.0014
    0.0016
    0.0019
    0.0021
    0.0023
    0.0026
    0.0028
    0.0030

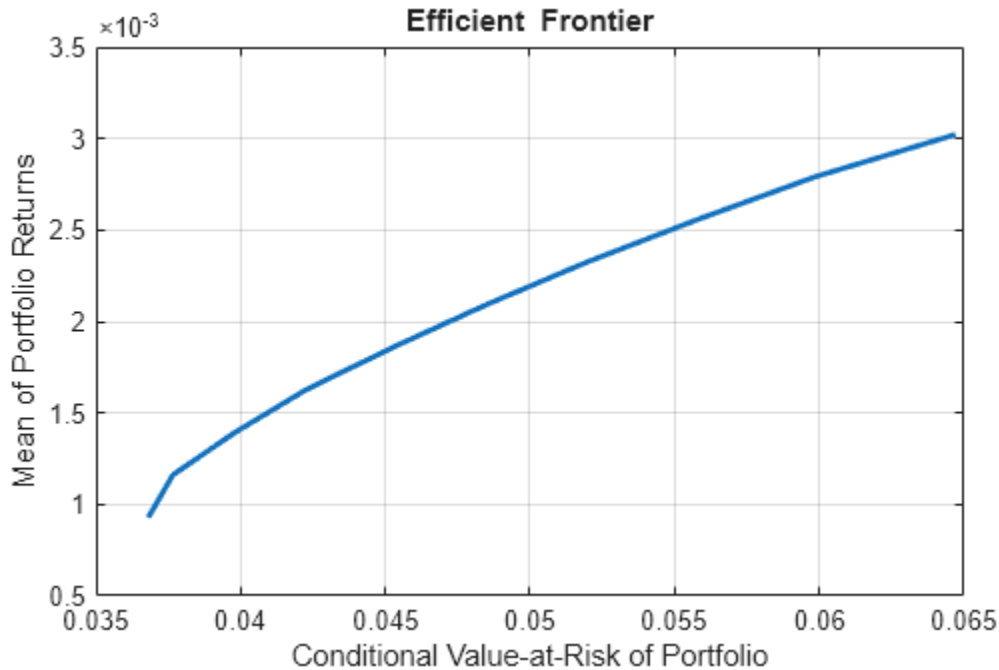
```

Plot the weights from the frontier estimation using `plotFrontier`. The resulting curve is piece-wise concave with possible vertical jumps (discontinuities) between the concave intervals.

```

% Plot efficient frontier
plotFrontier(p,pwgt)

```

Changing MINLP Solvers

In the previous section, you use the default solver for `estimateFrontier`. However, you can solve mixed-integer portfolio problems using any of the three algorithms supported by `setSolverMINLP`: `OuterApproximation`, `ExtendedCP`, and `TrustRegionCP`. Furthermore, the `OuterApproximation` algorithm accepts an additional name-value argument (`ExtendedFormulation`) for `Portfolio` problems, which reformulates problems with quadratic functions to work in an extended space that usually decreases the computation time. All algorithms, including the extended formulation variation of the `OuterApproximation` algorithm, return the same values within numerical accuracy. The available solvers are:

- `OuterApproximation` — The default algorithm, which is robust and usually faster than `ExtendedCP`
- `OuterApproximation` with `ExtendedFormulation` set to `true` — A robust algorithm that is usually faster than other algorithms, but only supported for `Portfolio` object problems
- `ExtendedCP` — The most robust solver, but usually the slowest
- `TrustRegionCP` — The fastest algorithm, but one that is less robust and may provide suboptimal solutions

For more information on solvers for mixed-integer portfolio problems, see “Choose MINLP Solvers for Portfolio Problems” on page 4-140.

To change the MINLP solvers, use `setSolverMINLP`.

```
% Select 'TrustRegionCP' as solver
p_OA = setSolverMINLP(p, 'OuterApproximation');
pwgt_OA = estimateFrontier(p_OA);
rsk_OA = estimatePortRisk(p, pwgt_OA);
ret_OA = estimatePortReturn(p, pwgt_OA);

% Select 'ExtendedCP' as solver using 'midway' cuts as 'CutGeneration'
```

```
p_ECP = setSolverMINLP(p, 'ExtendedCP', CutGeneration="midway");
pwgt_ECP = estimateFrontier(p_ECP);
rskECP = estimatePortRisk(p, pwgt_ECP);
retECP = estimatePortReturn(p, pwgt_ECP);
```

Compare the returns and risks obtained by the portfolios on the efficient frontier from the different solvers. They are all the same within a numerical accuracy that is an absolute difference $\leq 10^{-4}$.

```
retTable = table(retOA, ret, retECP, 'VariableNames', {'OA', 'TR', 'ECP'})
```

```
retTable=10×3 table
      OA      TR      ECP
-----
0.00092881  0.00092876  0.00092876
0.0011614   0.0011613   0.0011613
0.001394    0.0013939   0.0013939
0.0016265   0.0016265   0.0016265
0.0018591   0.0018591   0.0018591
0.0020917   0.0020917   0.0020917
0.0023243   0.0023243   0.0023243
0.0025569   0.0025569   0.0025569
0.0027894   0.0027894   0.0027894
0.003022    0.003022    0.003022
```

```
rskTable = table(rskOA, rsk, rskECP, 'VariableNames', {'OA', 'TR', 'ECP'})
```

```
rskTable=10×3 table
      OA      TR      ECP
-----
0.036808    0.036808    0.036808
0.037658    0.037658    0.037658
0.039808    0.039808    0.039808
0.042256    0.042256    0.042256
0.04527     0.045269    0.045269
0.048489    0.048488    0.048488
0.051963    0.051963    0.051963
0.055783    0.055783    0.055783
0.059828    0.059828    0.059828
0.064705    0.064705    0.064705
```

```
% Compare risks from the different OuterApproximation formulations
norm(rskTable.OA-rskTable.TR, Inf) <= 1e-4
```

```
ans = logical
      1
```

See Also

PortfolioCVaR

Related Examples

- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16
- “Choose MINLP Solvers for Portfolio Problems” on page 4-140

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

Bond Portfolio CVaR Optimization Using Diebold-Li Model

This example shows how to perform bond portfolio optimization by using the Diebold-Li dynamic factor model of the yield curve to simulate and compute bond returns.

For observation date t and time to maturity τ , the Diebold-Li model of the yield is given by

$$y_t(\tau) = L_t + S_t \left(\frac{1 - e^{-\lambda\tau}}{\lambda\tau} \right) + C_t \left(\frac{1 - e^{-\lambda\tau}}{\lambda\tau} - e^{-\lambda\tau} \right),$$

where:

- L_t is the long-term factor (level).
- S_t is the short-term factor (slope).
- C_t is the medium-term factor (curvature).
- λ determines the maturity at which the loading curve is maximized and governs the exponential decay rate of the model.

State-Space Formulation of Diebold-Li Model

In this example, you use the model from the “Apply State-Space Methodology to Analyze Diebold-Li Yield Curve Model” (Econometrics Toolbox) example to calculate the monthly returns. The associated `ssm` (Econometrics Toolbox) model is described in Load Yield Curve Data and `ssm` Model on page 5-147.

The level, slope, and curvature factors of the Diebold-Li model follow the state transition equation given by

$$\begin{bmatrix} L_t - \mu_L \\ S_t - \mu_S \\ C_t - \mu_C \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} L_{t-1} - \mu_L \\ S_{t-1} - \mu_S \\ C_{t-1} - \mu_C \end{bmatrix} + \begin{bmatrix} \eta_{Lt} \\ \eta_{St} \\ \eta_{Ct} \end{bmatrix}$$

where μ is the vector of means for each factor, and the corresponding observation equation is

$$\begin{bmatrix} y_{\tau_1 t} \\ y_{\tau_2 t} \\ \vdots \\ y_{\tau_N t} \end{bmatrix} = \begin{bmatrix} 1 & \frac{1 - e^{-\lambda\tau_1}}{\lambda\tau_1} & \frac{1 - e^{-\lambda\tau_1}}{\lambda\tau_1} - e^{-\lambda\tau_1} \\ 1 & \frac{1 - e^{-\lambda\tau_2}}{\lambda\tau_2} & \frac{1 - e^{-\lambda\tau_2}}{\lambda\tau_2} - e^{-\lambda\tau_2} \\ \vdots & \vdots & \vdots \\ 1 & \frac{1 - e^{-\lambda\tau_N}}{\lambda\tau_N} & \frac{1 - e^{-\lambda\tau_N}}{\lambda\tau_N} - e^{-\lambda\tau_N} \end{bmatrix} \begin{bmatrix} L_t \\ S_t \\ C_t \end{bmatrix} + \begin{bmatrix} \varepsilon_{\tau_1 t} \\ \varepsilon_{\tau_2 t} \\ \vdots \\ \varepsilon_{\tau_N t} \end{bmatrix}$$

You can represent these two sets of equations in the following vector representation

$$\begin{aligned} (f_t - \mu) &= A(f_{t-1} - \mu) + \eta_t \\ y_t &= A f_t + \varepsilon_t \end{aligned}$$

If you define the latent states x_t as the mean-adjusted factors

$$x_t = f_t - \mu$$

and define the deflated yields $\tilde{y}_t$ as

$$\tilde{y}_t = y_t - \Lambda\mu$$

then you can substitute x_t and $\tilde{y}_t$ into the preceding equations to obtain the following Diebold-Li state-space system:

$$x_t = Ax_{t-1} + \eta_t$$

$$\tilde{y}_t = \Lambda x_t + \varepsilon_t$$

Load Yield Curve Data and ssm Model

The yield data consists of 29 years of monthly, unsmoothed Fama-Bliss US Treasury zero-coupon yield measurements. The time series in the data represents maturities of 3, 6, 9, 12, 15, 18, 21, 24, 30, 36, 48, 60, 72, 84, 96, 108, and 120 months. The data expresses the yields in percents and records them at the end of each month, beginning January 1972 and ending December 2000, for a total of 348 monthly curves of 17 maturities each.

Load the Diebold-Li data set and extract the yield series.

```
load("Data_DieboldLi.mat")
yieldsMaturity = maturities';
yields = DataTable{:,1:17};
```

Load the Diebold-Li yield curve model.

```
load("EstimatedDieboldLiSSM.mat")
```

Simulate Bond Returns

To simulate bond returns for the following investment period $(t + 1)$, follow these steps:

- 1 Price bonds at time period t by using the zero-coupon bond yield curve, also known as the zero curve. Since some bonds have cash flows whose time to maturity do not match one of the tenors in the zero curve, you must use interpolation.
- 2 Simulate the $t + 1$ zero curve by using the period t factor estimates and their covariance matrix.
- 3 Price bonds at time period $t + 1$ by using the simulated zero curves.
- 4 Compute simulated returns by using the following formula for each bond i :

$$\frac{(P_{i(t+1)} - P_{it})}{P_{it}}$$

Start by loading the bonds information.

```
load('BondsCouponsAndMaturities')
couponRate = bondsTable.CouponRate;
bondsMaturity = bondsTable.Maturity;
```

Define the purchase date as the date at the end of the time horizon.

```
settle = DataTable.Time(end);
```

Generate bonds cash flow amounts and time mapping.

```
[CFAmounts,CFDates] = cfamounts(couponRate,settle,bondsMaturity);  
[allCF,allDates] = cfport(CFAmounts,CFDates);
```

Interpolate existing yields to obtain yields at cash flow times.

```
% Compute yields times  
yieldsDates = settle + calmonths(yieldsMaturity);  
yieldsTimes = yearfrac(settle,yieldsDates);  
yieldsT = yields(end,:)/100;  
  
% Interpolate to obtain yields at cash flow times  
CFTimes = yearfrac(settle,allDates);  
CFYields = interp1(yieldsTimes,yieldsT,CFTimes,[],"extrap");
```

Compute prices assuming continuous compounding.

```
% Compute prices from cash flow yields  
DF = exp(-CFYields.*CFTimes);  
bondsPrices = (allCF*DF)';
```

To simulate the yield curve at time period $t + 1$, first obtain the period t state mean and covariance matrix by using `smooth` (Econometrics Toolbox) with the deflated yields $\tilde{y}_t$. The final element of the output structure corresponds to the mean and covariance of the smoothed estimate at time period t .

```
% Deflate the yields  
intercept = (EstMdlSSM.C*mu')';  
deflatedYields = yields - intercept;  
  
% Obtain period t state mean and covariance  
[~,~,output] = smooth(EstMdlSSM,deflatedYields);  
MeanT = output(end).SmoothedStates;  
CovT = output(end).SmoothedStatesCov;  
CovT = (CovT + CovT')/2; % Ensures covariance is symmetric
```

Then, set the states initial mean and covariance of `EstMdlSSM` to the state mean and covariance at time period t .

```
% Set the initial state mean and covariance to the estimates at T  
EstMdlSSM.Mean0 = MeanT;  
EstMdlSSM.Cov0 = CovT;
```

Simulate 100,000 one-month-ahead deflated yields from the state-space model and then inflate the yields.

```
% Simulate 100,000 yields  
rng('default')  
simDeflatedYields = simulate(EstMdlSSM,1,NumPaths=1e5);  
simDeflatedYields = squeeze(simDeflatedYields(:,:,:));  
simDeflatedYields = simDeflatedYields';  
  
% Inflate yields  
simYields = (simDeflatedYields + intercept)/100;
```

Use simulated yields to compute simulation of prices at time $t + 1$.

```
% Set new settle date  
simSettle = settle + calmonths(1);
```

```

% Interpolate simulated yields at cash flow times
simCFTimes = yearfrac(simSettle,allDates);
simCFYields = interp1(yieldsTimes,simYields',simCFTimes',[],'extrap');

% Compute prices from cash flow yields
simDF = exp(-simCFYields.*simCFTimes');
simBondsPrices = (allCF*simDF)';

% Compute simulated returns
simMonthlyReturns = (simBondsPrices - bondsPrices)./bondsPrices;

```

The resulting returns simulation allows you to analyze the distribution of the one-month-ahead returns of the bonds. You can plot the distribution of the one-month-ahead distribution of the bonds with 3-month, 10-year, and 20-year maturities.

```

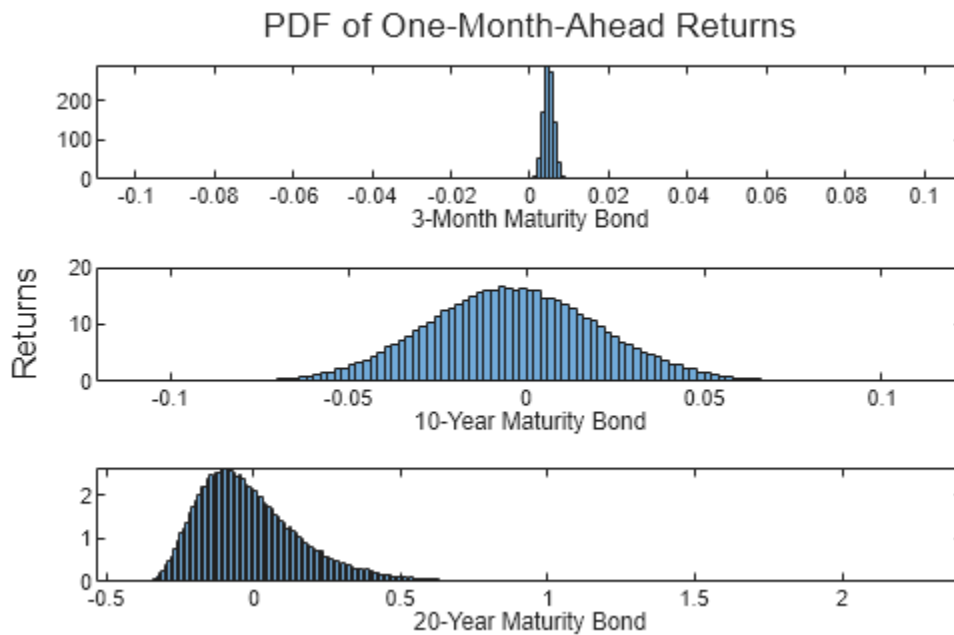
figure
t = tiledlayout(3,1);
title(t,'PDF of One-Month-Ahead Returns')
ylabel(t,'Returns')
bins = -0.1:0.001:0.1;

% 3-month maturity bond
nexttile
histogram(simMonthlyReturns(:,3),bins,'Normalization','pdf')
xlabel('3-Month Maturity Bond')

% 24-month maturity
nexttile
histogram(simMonthlyReturns(:,68),'Normalization','pdf')
xlabel('10-Year Maturity Bond')

% 120-month maturity
nexttile
histogram(simMonthlyReturns(:,90),'Normalization','pdf')
xlabel('20-Year Maturity Bond')

```



As the maturity increases, the variance of the returns increases and the distribution gets positively skewed.

Solve CVaR Bond Portfolio Optimization

Now use the simulated returns (`simMonthlyReturns`) to solve conditional value-at-risk (CVaR) optimization problems of fixed-income portfolios.

To solve the CVaR problem, use the `PortfolioCVaR` object. The optimal portfolio must invest in at least 10 types of bonds and no more than 30. At least 5% of the portfolio value must be invested in any chosen bond. This example also requires that you invest the entire capital in the portfolio.

```
% 95% CVaR Portfolio
p = PortfolioCVaR(Scenarios=simMonthlyReturns,ProbabilityLevel=0.95);

% Semi-continuous bounds
p = setBounds(p,0.05,0.5,BoundType="conditional");

% Cardinality constraint
p = setMinMaxNumAssets(p,10,30);

% Fully-invested portfolio
p = setBudget(p,1,1);

Use setInequality to add a constraint to the portfolio duration to match a given benchmark
portfolio duration.

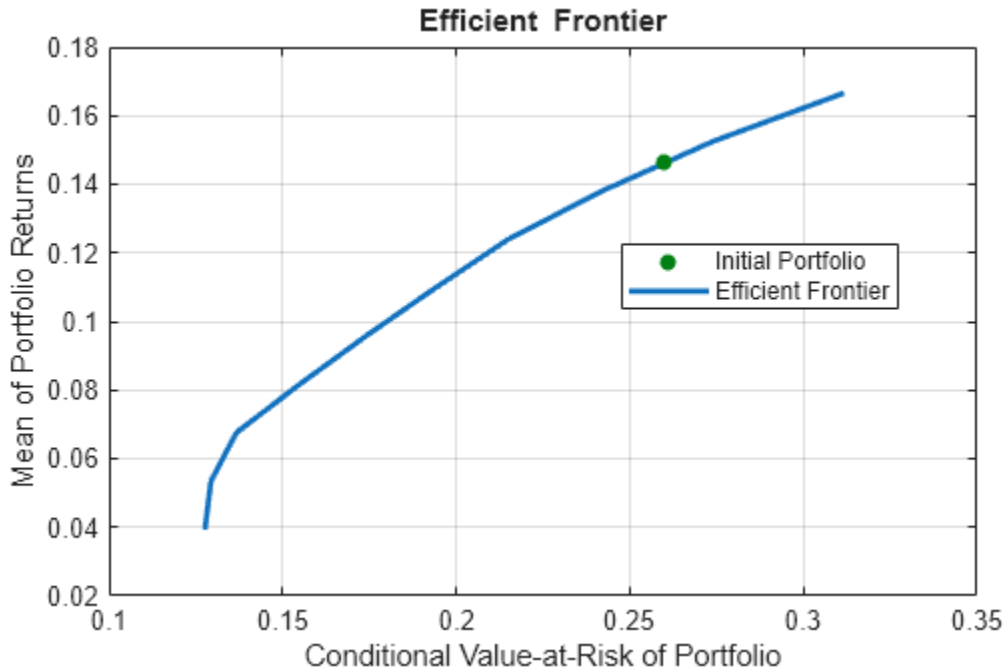
% Duration constraint
targetDuration = 10;
durationTol = 1;
bondsDuration = bnddurp(bondsPrices,couponRate,settle,bondsMaturity);
p = setInequality(p,[bondsDuration';-bondsDuration'], ...
    [targetDuration+durationTol;durationTol-targetDuration]);
```


Use `setCosts` to account for the impact of transaction costs when computing the return of the portfolio.

```
% Transaction costs
cost = 3e-4; % 3 bps
w0 = zeros(p.NumAssets,1);
w0([1 2 3 4 16 18 19 20]) = 0.05;
w0(110) = 0.1;
w0(112) = 0.5;
p = setCosts(p,cost,cost,w0);
```

Use `estimateFrontier` to compute the efficient frontier and then use `plotFrontier` to plot the efficient frontier of the portfolio problem.

```
% Plot efficient frontier
figure
pwgt = estimateFrontier(p);
plotFrontier(p,pwgt)
```



The plot shows that the minimum risk portfolio achieves a CVaR of 12.91% with an associated return of 4.10%, while the maximum return portfolio has a CVaR of 30.93% and a return of 15.94%.

```
% Table of efficient portfolio non-zero weights
idxNonZero = sum(pwgt >= 1e-4,2) >= 1;
nonZeroWeightsTable = array2table(pwgt(idxNonZero,:), ...
    RowNames=string(bondsMaturity(idxNonZero)), ...
    VariableNames="Portfolio"+string(1:10))
```

```
nonZeroWeightsTable=28x10 table
```

	Portfolio1	Portfolio2	Portfolio3	Portfolio4	Portfolio5	Portfolio6
2001-02-15	0	0	0	0.062484	0.050084	0
2001-02-28	0	0	0	0	-1.3553e-20	0.05021

2001-03-31	0	0	0	0	0.05	0
2001-04-30	0	0	0	0	0.05	0
2001-05-15	0	0	0	0	-1.3553e-20	0
2001-05-31	0	0	0	0	0	0
2001-06-30	0	0	0	0	0	0
2002-01-31	0	0	0	0	0	0.05
2002-02-28	0	0	0	0	0	0.05
2002-03-31	0	0	0	0	0	0.05
2002-04-30	0	0	0	0	1.4745e-17	0.05
2007-11-15	0	0	0	0.05	0	0
2008-02-15	0	0	0.05	0	0	0
2008-05-15	0	0	0.05	0.05	0	0
2008-11-15	0	0.05	0.05	0.05	0.05	0
2009-05-15	0.05	0.05	0.05	0.05	0.05	0.05
:						

The table shows how the least risky portfolios invest more on bonds with a maturity of approximately 9 years and, as the return levels increase, the investment concentrates on bonds with longer maturities.

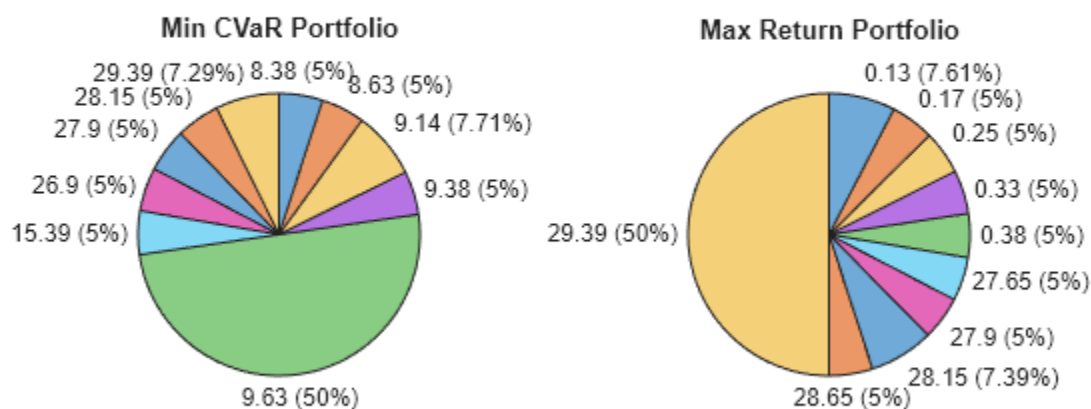
```
% Compare weights of min risk and max return portfolios
labels = yearfrac(settle,bondsMaturity);
labels = round(labels,2);
```

```
% Plot pie charts
figure
t = tiledlayout(1,2);
title(t,'Optimal Bonds Selection','Years to Maturity')
```

```
% Min CVaR portfolio
nexttile
idxMinCVaR = pwgt(:,1) >= 1e-4;
piechart(pwgt(idxMinCVaR,1),labels(idxMinCVaR))
title('Min CVaR Portfolio')
```

```
% Max return portfolio
nexttile
idxMaxRet = pwgt(:,end) >= 1e-4;
piechart(pwgt(idxMaxRet,end),labels(idxMaxRet))
title('Max Return Portfolio')
```

Optimal Bonds Selection Years to Maturity



See Also

PortfolioCVaR

Related Examples

- “Creating the PortfolioCVaR Object” on page 5-22
- “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
- “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Hedging Using CVaR Portfolio Optimization” on page 5-125
- “Troubleshooting CVaR Portfolio Optimization Results” on page 5-121

More About

- “PortfolioCVaR Object” on page 5-17
- “Portfolio Optimization Theory” on page 5-3
- “PortfolioCVaR Object Workflow” on page 5-16
- “Choose MINLP Solvers for Portfolio Problems” on page 4-140

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- CVaR Portfolio Optimization (4 min 56 sec)

MAD Portfolio Optimization Tools

- “Portfolio Optimization Theory” on page 6-3
- “Supported Constraints for Portfolio Optimization Using PortfolioMAD Object” on page 6-8
- “Default Portfolio Problem” on page 6-15
- “PortfolioMAD Object Workflow” on page 6-16
- “PortfolioMAD Object” on page 6-17
- “Creating the PortfolioMAD Object” on page 6-22
- “Common Operations on the PortfolioMAD Object” on page 6-29
- “Setting Up an Initial or Current Portfolio” on page 6-33
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35
- “Working with a Riskless Asset” on page 6-45
- “Working with Transaction Costs” on page 6-46
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Working with ‘Simple’ Bound Constraints Using PortfolioMAD Object” on page 6-54
- “Working with Budget Constraints Using PortfolioMAD Object” on page 6-57
- “Working with Conditional Budget Constraints Using PortfolioMAD Object” on page 6-59
- “Working with Group Constraints Using PortfolioMAD Object” on page 6-61
- “Working with Group Ratio Constraints Using PortfolioMAD Object” on page 6-64
- “Working with Linear Equality Constraints Using PortfolioMAD Object” on page 6-67
- “Working with Linear Inequality Constraints Using PortfolioMAD Object” on page 6-69
- “Working with ‘Conditional’ BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects” on page 6-71
- “Working with Average Turnover Constraints Using PortfolioMAD Object” on page 6-74
- “Working with One-Way Turnover Constraints Using PortfolioMAD Object” on page 6-77
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Obtaining Portfolios Along the Entire Efficient Frontier” on page 6-85
- “Obtaining Endpoints of the Efficient Frontier” on page 6-87
- “Mixed-Integer MAD Portfolio Optimization Problem” on page 6-89
- “Obtaining Efficient Portfolios for Target Returns” on page 6-93
- “Obtaining Efficient Portfolios for Target Risks” on page 6-96
- “Choosing and Controlling the Solver for PortfolioMAD Optimizations” on page 6-99
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Plotting the Efficient Frontier for a PortfolioMAD Object” on page 6-108
- “Postprocessing Results to Set Up Tradable Portfolios” on page 6-113
- “Working with Other Portfolio Objects” on page 6-115

- “Troubleshooting MAD Portfolio Optimization Results” on page 6-118

Portfolio Optimization Theory

In this section...

“Portfolio Optimization Problems” on page 6-3
 “Portfolio Problem Specification” on page 6-3
 “Return Proxy” on page 6-4
 “Risk Proxy” on page 6-5

Portfolio Optimization Problems

Portfolio optimization problems involve identifying portfolios that satisfy three criteria:

- Minimize a proxy for risk.
- Match or exceed a proxy for return.
- Satisfy basic feasibility requirements.

Portfolios are points from a feasible set of assets that constitute an asset universe. A portfolio specifies either holdings or weights in each individual asset in the asset universe. The convention is to specify portfolios in terms of weights, although the portfolio optimization tools work with holdings as well.

The set of feasible portfolios is necessarily a nonempty, closed, and bounded set. The proxy for risk is a function that characterizes either the variability or losses associated with portfolio choices. The proxy for return is a function that characterizes either the gross or net benefits associated with portfolio choices. The terms “risk” and “risk proxy” and “return” and “return proxy” are interchangeable. The fundamental insight of Markowitz (see “Portfolio Optimization” on page A-5) is that the goal of the portfolio choice problem is to seek minimum risk for a given level of return and to seek maximum return for a given level of risk. Portfolios satisfying these criteria are efficient portfolios and the graph of the risks and returns of these portfolios forms a curve called the efficient frontier.

Portfolio Problem Specification

To specify a portfolio optimization problem, you need the following:

- Proxy for portfolio return (μ)
- Proxy for portfolio risk (Σ)
- Set of feasible portfolios (X), called a portfolio set

Financial Toolbox has three objects to solve specific types of portfolio optimization problems:

- The `Portfolio` object supports mean-variance portfolio optimization (see Markowitz [46], [47] at “Portfolio Optimization” on page A-5). This object has either gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set.
- The `PortfolioCVaR` object implements what is known as conditional value-at-risk portfolio optimization (see Rockafellar and Uryasev [48], [49] at “Portfolio Optimization” on page A-5), which is generally referred to as CVaR portfolio optimization. CVaR portfolio optimization works

with the same return proxies and portfolio sets as mean-variance portfolio optimization but uses conditional value-at-risk of portfolio returns as the risk proxy.

- The `PortfolioMAD` object implements what is known as mean-absolute deviation portfolio optimization (see Konno and Yamazaki [50] at “Portfolio Optimization” on page A-5), which is generally referred to as MAD portfolio optimization. MAD portfolio optimization works with the same return proxies and portfolio sets as mean-variance portfolio optimization but uses mean-absolute deviation portfolio returns as the risk proxy.

Return Proxy

The proxy for portfolio return is a function $\mu: X \rightarrow R$ on a portfolio set $X \subset R^n$ that characterizes the rewards associated with portfolio choices. Usually, the proxy for portfolio return has two general forms, gross and net portfolio returns. Both portfolio return forms separate the risk-free rate r_0 so that the portfolio $x \in X$ contains only risky assets.

Regardless of the underlying distribution of asset returns, a collection of S asset returns $y_1, \dots, y_S$ has a mean of asset returns

$$m = \frac{1}{S} \sum_{s=1}^S y_s,$$

and (sample) covariance of asset returns

$$C = \frac{1}{S-1} \sum_{s=1}^S (y_s - m)(y_s - m)^T.$$

These moments (or alternative estimators that characterize these moments) are used directly in mean-variance portfolio optimization to form proxies for portfolio risk and return.

Gross Portfolio Returns

The gross portfolio return for a portfolio $x \in X$ is

$$\mu(x) = r_0 + (m - r_0 \mathbf{1})^T x,$$

where:

r_0 is the risk-free rate (scalar).

m is the mean of asset returns (n vector).

If the portfolio weights sum to 1, the risk-free rate is irrelevant. The properties in the `Portfolio` object to specify gross portfolio returns are:

- `RiskFreeRate` for r_0
- `AssetMean` for m

Net Portfolio Returns

The net portfolio return for a portfolio $x \in X$ is

$$\mu(x) = r_0 + (m - r_0 \mathbf{1})^T x - b^T \max\{0, x - x_0\} - s^T \max\{0, x_0 - x\},$$

where:

r_0 is the risk-free rate (scalar).

m is the mean of asset returns (n vector).

b is the proportional cost to purchase assets (n vector).

s is the proportional cost to sell assets (n vector).

You can incorporate fixed transaction costs in this model also. Though in this case, it is necessary to incorporate prices into such costs. The properties in the `Portfolio` object to specify net portfolio returns are:

- `RiskFreeRate` for r_0
- `AssetMean` for m
- `InitPort` for x_0
- `BuyCost` for b
- `SellCost` for s

Risk Proxy

The proxy for portfolio risk is a function $\sigma: X \rightarrow R$ on a portfolio set $X \subset R^n$ that characterizes the risks associated with portfolio choices.

Variance

The variance of portfolio returns for a portfolio $x \in X$ is

$$\text{Variance}(x) = x^T C x$$

where C is the covariance of asset returns (n -by- n positive-semidefinite matrix).

The property in the `Portfolio` object to specify the variance of portfolio returns is `AssetCovar` for C .

Although the risk proxy in mean-variance portfolio optimization is the variance of portfolio returns, the square root, which is the standard deviation of portfolio returns, is often reported and displayed. Moreover, this quantity is often called the “risk” of the portfolio. For details, see Markowitz [46], [47] at (“Portfolio Optimization” on page A-5).

Conditional Value-at-Risk

The conditional value-at-risk for a portfolio $x \in X$, which is also known as expected shortfall, is defined as

$$\text{CVaR}_\alpha(x) = \frac{1}{1-\alpha} \int_{f(x,y) \geq \text{VaR}_\alpha(x)} f(x,y) p(y) dy,$$

where:

α is the probability level such that $0 < \alpha < 1$.

$f(x, y)$ is the loss function for a portfolio x and asset return y .

$p(y)$ is the probability density function for asset return y .

VaR_α is the value-at-risk of portfolio x at probability level α .

The value-at-risk is defined as

$$VaR_\alpha(x) = \min\{\gamma: \Pr[f(x, Y) \leq \gamma] \geq \alpha\}.$$

An alternative formulation for CVaR has the form:

$$CVaR_\alpha(x) = VaR_\alpha(x) + \frac{1}{1-\alpha} \int_{R^n} \max\{0, (f(x, y) - VaR_\alpha(x))\} p(y) dy$$

The choice for the probability level α is typically 0.9 or 0.95. Choosing α implies that the value-at-risk $VaR_\alpha(x)$ for portfolio x is the portfolio return such that the probability of portfolio returns falling below this level is $(1 - \alpha)$. Given $VaR_\alpha(x)$ for a portfolio x , the conditional value-at-risk of the portfolio is the expected loss of portfolio returns above the value-at-risk return.

Note Value-at-risk is a positive value for losses so that the probability level α indicates the probability that portfolio returns are below the negative of the value-at-risk.

To describe the probability distribution of returns, the `PortfolioCVaR` object takes a finite sample of return scenarios y_s , with $s = 1, \dots, S$. Each y_s is an n vector that contains the returns for each of the n assets under the scenario s . This sample of S scenarios is stored as a scenario matrix of size S -by- n . Then, the risk proxy for CVaR portfolio optimization, for a given portfolio $x \in X$ and $\alpha \in (0, 1)$, is computed as

$$CVaR_\alpha(x) = VaR_\alpha(x) + \frac{1}{(1-\alpha)S} \sum_{s=1}^S \max\{0, -y_s^T x - VaR_\alpha(x)\}$$

The value-at-risk, $VaR_\alpha(x)$, is estimated whenever the CVaR is estimated. The loss function is $f(x, y_s) = -y_s^T x$, which is the portfolio loss under scenario s .

Under this definition, VaR and CVaR are sample estimators for VaR and CVaR based on the given scenarios. Better scenario samples yield more reliable estimates of VaR and CVaR.

For more information, see Rockafellar and Uryasev [48], [49], and Cornuejols and Tütüncü, [51], at "Portfolio Optimization" on page A-5.

Mean Absolute-Deviation

The mean-absolute deviation (MAD) for a portfolio $x \in X$ is defined as

$$MAD(x) = \frac{1}{S} \sum_{s=1}^S |(y_s - m)^T x|$$

where:

y_s are asset returns with scenarios $s = 1, \dots, S$ (S collection of n vectors).

$f(x,y)$ is the loss function for a portfolio x and asset return y .

m is the mean of asset returns (n vector).

such that

$$m = \frac{1}{S} \sum_{s=1}^S y_s$$

For more information, see Konno and Yamazaki [50] at “Portfolio Optimization” on page A-5.

See Also

Portfolio | PortfolioCVaR | PortfolioMAD

Related Examples

- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50

More About

- Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8
- “Default Portfolio Problem” on page 6-15
- “PortfolioMAD Object Workflow” on page 6-16

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object

The final element for a complete specification of a portfolio optimization problem is the set of feasible portfolios, which is called a portfolio set. A portfolio set $X \subset R^n$ is specified by construction as the intersection of sets formed by a collection of constraints on portfolio weights. A portfolio set necessarily and sufficiently must be a nonempty, closed, and bounded set.

When setting up your portfolio set, ensure that the portfolio set satisfies these conditions. The most basic or “default” portfolio set requires portfolio weights to be nonnegative (using the lower-bound constraint) and to sum to 1 (using the budget constraint). The most general portfolio set handled by the portfolio optimization tools can have any of these constraints and which are properties for the PortfolioMAD object:

- Linear inequality constraints
- Linear equality constraints
- 'Simple' Bound constraints
- 'Conditional' Bound constraints
- Budget constraints
- Conditional budget constraints
- Group constraints
- Group ratio constraints
- Average turnover constraints
- One-way turnover constraints
- Cardinality constraints

Linear Inequality Constraints

Linear inequality constraints are general linear constraints that model relationships among portfolio weights that satisfy a system of inequalities. Linear inequality constraints take the form

$$A_I x \leq b_I$$

where:

x is the portfolio (n vector).

A_I is the linear inequality constraint matrix (n_I -by- n matrix).

b_I is the linear inequality constraint vector (n_I vector).

n is the number of assets in the universe and n_I is the number of constraints.

PortfolioMAD object properties to specify linear inequality constraints are:

- AInequality for A_I
- bInequality for b_I
- NumAssets for n

The default is to ignore these constraints.

Linear Equality Constraints

Linear equality constraints are general linear constraints that model relationships among portfolio weights that satisfy a system of equalities. Linear equality constraints take the form

$$A_E x = b_E$$

where:

x is the portfolio (n vector).

A_E is the linear equality constraint matrix (n_E -by- n matrix).

b_E is the linear equality constraint vector (n_E vector).

n is the number of assets in the universe and n_E is the number of constraints.

PortfolioMAD object properties to specify linear equality constraints are:

- `AEquality` for A_E
- `bEquality` for b_E
- `NumAssets` for n

The default is to ignore these constraints.

'Simple' Bound Constraints

'Simple' Bound constraints are specialized linear constraints that confine portfolio weights to fall either above or below specific bounds. Since every portfolio set must be bounded, it is often a good practice, albeit not necessary, to set explicit bounds for the portfolio problem. To obtain explicit bounds for a given portfolio set, use the `estimateBounds` function. Bound constraints take the form

$$l_B \leq x \leq u_B$$

where:

x is the portfolio (n vector).

l_B is the lower-bound constraint (n vector).

u_B is the upper-bound constraint (n vector).

n is the number of assets in the universe.

PortfolioMAD object properties to specify bound constraints are:

- `LowerBound` for l_B
- `UpperBound` for u_B
- `NumAssets` for n

The default is to ignore these constraints.

The default portfolio optimization problem (see “Default Portfolio Problem” on page 6-15) has $l_B = 0$ with u_B set implicitly through a budget constraint.

'Conditional' Bound Constraints

'Conditional' Bound constraints, also called semicontinuous constraints, are mixed-integer linear constraints that confine portfolio weights to fall either above or below specific bounds *if* the asset is selected; otherwise, the value of the asset is zero. Use `setBounds` to specify bound constraints with a 'Conditional' `BoundType`. To mathematically formulate this type of constraints, a binary variable v_i is needed. $v_i = 0$ indicates that asset i is not selected and v_i indicates that the asset was selected. Thus

$$l_i v_i \leq x_i \leq u_i v_i$$

where

x is the portfolio (n vector).

l is the conditional lower-bound constraint (n vector).

u is the conditional upper-bound constraint (n vector).

n is the number of assets in the universe.

`PortfolioMAD` object properties to specify the bound constraint are:

- `LowerBound` for l_B
- `UpperBound` for u_B
- `NumAssets` for n

The default is to ignore this constraint.

Budget Constraints

Budget constraints are specialized linear constraints that confine the sum of portfolio weights to fall either above or below specific bounds. The constraints take the form

$$l_S \leq 1^T x \leq u_S$$

where:

x is the portfolio (n vector).

1 is the vector of ones (n vector).

l_S is the lower-bound budget constraint (scalar).

u_S is the upper-bound budget constraint (scalar).

n is the number of assets in the universe.

`PortfolioMAD` object properties to specify budget constraints are:

- LowerBudget for l_S
- UpperBudget for u_S
- NumAssets for n

The default is to ignore this constraint.

The default portfolio optimization problem (see “Default Portfolio Problem” on page 6-15) has $l_S = u_S = 1$, which means that the portfolio weights sum to 1. If the portfolio optimization problem includes possible movements in and out of cash, the budget constraint specifies how far portfolios can go into cash. For example, if $l_S = 0$ and $u_S = 1$, then the portfolio can have 0-100% invested in cash. If cash is to be a portfolio choice, set RiskFreeRate (r_0) to a suitable value (see “Return Proxy” on page 6-4 and “Working with a Riskless Asset” on page 6-45).

Conditional Budget Constraints

Conditional budget constraints is a logical constraint that bounds the maximum amount of the portfolio that can be invested in assets exceeding a given threshold. This constraint supports the “Undertakings for Collective Investment in Transferable Securities” on page 15-1452 (UCITS) directive. Use `setConditionalBudget` to set conditional budget constraints. To mathematically formulate this type of constraints, a binary variable w_i is needed. If $w_i = 0$ the asset is below the specified threshold, otherwise the asset is above that threshold which is represented as

$$w_i = 0 \rightarrow x_i \leq \alpha_i$$

$$\sum_i x_i w_i \leq \gamma$$

where:

x is an n vector representing the portfolio.

α is a scalar or n representing the threshold above which assets are accounted for in the conditional budget (scalar or n vector).

γ is a scalar representing the maximum amount of the portfolio that can be invested in assets exceeding α .

Group Constraints

Group constraints are specialized linear constraints that enforce “membership” among groups of assets. The constraints take the form

$$l_G \leq Gx \leq u_G$$

where:

x is the portfolio (n vector).

l_G is the lower-bound group constraint (n_G vector).

u_G is the upper-bound group constraint (n_G vector).

G is the matrix of group membership indexes (n_G -by- n matrix).

Each row of G identifies which assets belong to a group associated with that row. Each row contains either 0s or 1s with 1 indicating that an asset is part of the group or 0 indicating that the asset is not part of the group.

PortfolioMAD object properties to specify group constraints are:

- GroupMatrix for G
- LowerGroup for l_G
- UpperGroup for u_G
- NumAssets for n

The default is to ignore these constraints.

Group Ratio Constraints

Group ratio constraints are specialized linear constraints that enforce relationships among groups of assets. The constraints take the form

$$l_{Ri}(G_Bx)_i \leq (G_Ax)_i \leq u_{Ri}(G_Bx)_i$$

for $i = 1, \dots, n_R$ where:

x is the portfolio (n vector).

l_R is the vector of lower-bound group ratio constraints (n_R vector).

u_R is the vector matrix of upper-bound group ratio constraints (n_R vector).

G_A is the matrix of base group membership indexes (n_R -by- n matrix).

G_B is the matrix of comparison group membership indexes (n_R -by- n matrix).

n is the number of assets in the universe and n_R is the number of constraints.

Each row of G_A and G_B identifies which assets belong to a base and comparison group associated with that row.

Each row contains either 0s or 1s with 1 indicating that an asset is part of the group or 0 indicating that the asset is not part of the group.

PortfolioMAD object properties to specify group ratio constraints are:

- GroupA for G_A
- GroupB for G_B
- LowerRatio for l_R
- UpperRatio for u_R
- NumAssets for n

The default is to ignore these constraints.

Average Turnover Constraints

Turnover constraint is a linear absolute value constraint that ensures estimated optimal portfolios differ from an initial portfolio by no more than a specified amount. Although portfolio turnover is defined in many ways, the turnover constraints implemented in Financial Toolbox compute portfolio turnover as the average of purchases and sales. Average turnover constraints take the form

$$\frac{1}{2} \mathbf{1}^T |x - x_0| \leq \tau$$

where:

x is the portfolio (n vector).

$\mathbf{1}$ is the vector of ones (n vector).

x_0 is the initial portfolio (n vector).

τ is the upper bound for turnover (scalar).

n is the number of assets in the universe.

PortfolioMAD object properties to specify the average turnover constraint are:

- `Turnover` for τ
- `InitPort` for x_0
- `NumAssets` for n

The default is to ignore this constraint.

One-way Turnover Constraints

One-way turnover constraints ensure that estimated optimal portfolios differ from an initial portfolio by no more than specified amounts according to whether the differences are purchases or sales. The constraints take the forms

$$\mathbf{1}^T \times \max\{0, x - x_0\} \leq \tau_B$$

$$\mathbf{1}^T \times \max\{0, x_0 - x\} \leq \tau_S$$

where:

x is the portfolio (n vector)

$\mathbf{1}$ is the vector of ones (n vector).

x_0 is the Initial portfolio (n vector).

τ_B is the upper bound for turnover constraint on purchases (scalar).

τ_S is the upper bound for turnover constraint on sales (scalar).

To specify one-way turnover constraints, use the following properties in the PortfolioMAD object:

- `BuyTurnover` for τ_B
- `SellTurnover` for τ_S
- `InitPort` for x_0

The default is to ignore this constraint.

Note The average turnover constraint (see “Working with Average Turnover Constraints Using PortfolioMAD Object” on page 6-74) with τ is not a combination of the one-way turnover constraints with $\tau = \tau_B = \tau_S$.

Cardinality Constraints

Cardinality constraint limits the number of assets in the optimal allocation for an `PortfolioMAD` object. Use `setMinMaxNumAssets` to specify the 'MinNumAssets' and 'MaxNumAssets' constraints. To mathematically formulate this type of constraints, a binary variable v_i is needed. $v_i = 0$ indicates that asset i is not selected and $v_i = 1$ indicates that the asset was selected. Thus

$$\text{MinNumAssets} \leq \sum_{i=1}^{\text{NumAssets}} v_i \leq \text{MaxNumAssets}$$

The default is to ignore this constraint.

See Also

`Portfolio` | `PortfolioCVaR` | `PortfolioMAD`

Related Examples

- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50

More About

- “PortfolioMAD Object” on page 6-17
- “Default Portfolio Problem” on page 6-15
- “PortfolioMAD Object Workflow” on page 6-16

Default Portfolio Problem

The default portfolio optimization problem has a risk and return proxy associated with a given problem, and a portfolio set that specifies portfolio weights to be nonnegative and to sum to 1. The lower bound combined with the budget constraint is sufficient to ensure that the portfolio set is nonempty, closed, and bounded. The default portfolio optimization problem characterizes a long-only investor who is fully invested in a collection of assets.

- For mean-variance portfolio optimization, it is sufficient to set up the default problem. After setting up the problem, data in the form of a mean and covariance of asset returns are then used to solve portfolio optimization problems.
- For conditional value-at-risk portfolio optimization, the default problem requires the additional specification of a probability level that must be set explicitly. Generally, “typical” values for this level are 0.90 or 0.95. After setting up the problem, data in the form of scenarios of asset returns are then used to solve portfolio optimization problems.
- For MAD portfolio optimization, it is sufficient to set up the default problem. After setting up the problem, data in the form of scenarios of asset returns are then used to solve portfolio optimization problems.

See Also

[Portfolio](#) | [PortfolioCVaR](#) | [PortfolioMAD](#)

Related Examples

- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50

More About

- “PortfolioMAD Object” on page 6-17
- Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8
- “PortfolioMAD Object Workflow” on page 6-16

PortfolioMAD Object Workflow

The PortfolioMAD object workflow for creating and modeling a MAD portfolio is:

1 Create a MAD Portfolio.

Create a PortfolioMAD object for mean-absolute deviation (MAD) portfolio optimization. For more information, see “Creating the PortfolioMAD Object” on page 6-22.

2 Define asset returns and scenarios.

Evaluate scenarios for portfolio asset returns, including assets with missing data and financial time series data. For more information, see “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35.

3 Specify the MAD Portfolio Constraints.

Define the constraints for portfolio assets such as linear equality and inequality, bound, conditional budget, budget, group, group ratio, and turnover constraints, 'Conditional' BoundType, and MinNumAssets, MaxNumAssets constraints. For more information, see “Working with MAD Portfolio Constraints Using Defaults” on page 6-50 and “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects” on page 6-71.

4 Validate the MAD Portfolio.

Identify errors for the portfolio specification. For more information, see “Validate the MAD Portfolio Problem” on page 6-80.

5 Estimate the efficient portfolios and frontiers.

Analyze the efficient portfolios and efficient frontiers for a portfolio. For more information, see “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84 and “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105.

6 Postprocess the results.

Use the efficient portfolios and efficient frontiers results to set up trades. For more information, see “Postprocessing Results to Set Up Tradable Portfolios” on page 6-113.

See Also

More About

- “Portfolio Optimization Theory” on page 6-3
- “Choosing and Controlling the Solver for PortfolioMAD Optimizations” on page 6-99

PortfolioMAD Object

In this section...
“PortfolioMAD Object Properties and Functions” on page 6-17 “Working with PortfolioMAD Objects” on page 6-17 “Setting and Getting Properties” on page 6-18 “Displaying PortfolioMAD Objects” on page 6-18 “Saving and Loading PortfolioMAD Objects” on page 6-18 “Estimating Efficient Portfolios and Frontiers” on page 6-18 “Arrays of PortfolioMAD Objects” on page 6-19 “Subclassing PortfolioMAD Objects” on page 6-20 “Conventions for Representation of Data” on page 6-20

PortfolioMAD Object Properties and Functions

The `PortfolioMAD` object implements mean absolute-deviation (MAD) portfolio optimization and is derived from the abstract class `AbstractPortfolio`. Every property and function of the `PortfolioMAD` object is public, although some properties and functions are hidden. The `PortfolioMAD` object is a value object where every instance of the object is a distinct version of the object. Since the `PortfolioMAD` object is also a MATLAB object, it inherits the default functions associated with MATLAB objects.

Working with PortfolioMAD Objects

The `PortfolioMAD` object and its functions are an interface for mean absolute-deviation portfolio optimization. So, almost everything you do with the `PortfolioMAD` object can be done using the functions. The basic workflow is:

- 1 Design your portfolio problem.
- 2 Use `PortfolioMAD` to create the `PortfolioMAD` object or use the various set functions to set up your portfolio problem.
- 3 Use estimate functions to solve your portfolio problem.

In addition, functions are available to help you view intermediate results and to diagnose your computations. Since MATLAB features are part of a `PortfolioMAD` object, you can save and load objects from your workspace and create and manipulate arrays of objects. After settling on a problem, which, in the case of MAD portfolio optimization, means that you have either scenarios, data, or moments for asset returns, and a collection of constraints on your portfolios, use `PortfolioMAD` to set the properties for the `PortfolioMAD` object.

`PortfolioMAD` lets you create an object from scratch or update an existing object. Since the `PortfolioMAD` object is a value object, it is easy to create a basic object, then use functions to build upon the basic object to create new versions of the basic object. This is useful to compare a basic problem with alternatives derived from the basic problem. For details, see “Creating the `PortfolioMAD` Object” on page 6-22.

Setting and Getting Properties

You can set properties of a `PortfolioMAD` object using either `PortfolioMAD` or various `set` functions.

Note Although you can also set properties directly, it is not recommended since error-checking is not performed when you set a property directly.

The `PortfolioMAD` object supports setting properties with name-value pair arguments such that each argument name is a property and each value is the value to assign to that property. For example, to set the `LowerBound` and `Budget` properties in an existing `PortfolioMAD` object `p`, use the syntax:

```
p = PortfolioMAD(p, 'LowerBound', 0, 'Budget', 1);
```

In addition to the `PortfolioMAD` object, which lets you set individual properties one at a time, groups of properties are set in a `PortfolioMAD` object with various “set” and “add” functions. For example, to set up an average turnover constraint, use the `setTurnover` function to specify the bound on portfolio turnover and the initial portfolio. To get individual properties from a `PortfolioMAD` object, obtain properties directly or use an assortment of “get” functions that obtain groups of properties from a `PortfolioMAD` object. The `PortfolioMAD` object and `set` functions have several useful features:

- The `PortfolioMAD` object and `set` functions try to determine the dimensions of your problem with either explicit or implicit inputs.
- The `PortfolioMAD` object and `set` functions try to resolve ambiguities with default choices.
- The `PortfolioMAD` object and `set` functions perform scalar expansion on arrays when possible.
- The `PortfolioMAD` functions try to diagnose and warn about problems.

Displaying PortfolioMAD Objects

The `PortfolioMAD` object uses the default display function provided by MATLAB, where `display` and `disp` display a `PortfolioMAD` object and its properties with or without the object variable name.

Saving and Loading PortfolioMAD Objects

Save and load `PortfolioMAD` objects using the MATLAB `save` and `load` commands.

Estimating Efficient Portfolios and Frontiers

Estimating efficient portfolios and efficient frontiers is the primary purpose of the MAD portfolio optimization tools. An efficient portfolio are the portfolios that satisfy the criteria of minimum risk for a given level of return and maximum return for a given level of risk. A collection of “estimate” and “plot” functions provide ways to explore the efficient frontier. The “estimate” functions obtain either efficient portfolios or risk and return proxies to form efficient frontiers. At the portfolio level, a collection of functions estimates efficient portfolios on the efficient frontier with functions to obtain efficient portfolios:

- At the endpoints of the efficient frontier
- That attain targeted values for return proxies
- That attain targeted values for risk proxies
- Along the entire efficient frontier

These functions also provide purchases and sales needed to shift from an initial or current portfolio to each efficient portfolio. At the efficient frontier level, a collection of functions plot the efficient frontier and estimate either risk or return proxies for efficient portfolios on the efficient frontier. You can use the resultant efficient portfolios or risk and return proxies in subsequent analyses.

Arrays of PortfolioMAD Objects

Although all functions associated with a PortfolioMAD object are designed to work on a scalar PortfolioMAD object, the array capabilities of MATLAB enable you to set up and work with arrays of PortfolioMAD objects. The easiest way to do this is with the `repmat` function. For example, to create a 3-by-2 array of PortfolioMAD objects:

```
p = repmat(PortfolioMAD, 3, 2);
disp(p)
```

3×2 PortfolioMAD array with properties:

```
BuyCost
SellCost
RiskFreeRate
Turnover
BuyTurnover
SellTurnover
NumScenarios
Name
NumAssets
AssetList
InitPort
AInequality
bInequality
AEquality
bEquality
LowerBound
UpperBound
LowerBudget
UpperBudget
GroupMatrix
LowerGroup
UpperGroup
GroupA
GroupB
LowerRatio
UpperRatio
MinNumAssets
MaxNumAssets
ConditionalBudgetThreshold
ConditionalUpperBudget
BoundType
```

After setting up an array of PortfolioMAD objects, you can work on individual PortfolioMAD objects in the array by indexing. For example:

```
p(i,j) = PortfolioMAD(p(i,j), ... );
```

This example calls `PortfolioMAD` for the (i,j) element of a matrix of `PortfolioMAD` objects in the variable `p`.

If you set up an array of `PortfolioMAD` objects, you can access properties of a particular `PortfolioMAD` object in the array by indexing so that you can set the lower and upper bounds `lb` and `ub` for the (i,j,k) element of a 3-D array of `PortfolioMAD` objects with

```
p(i,j,k) = setBounds(p(i,j,k),lb, ub);
```

and, once set, you can access these bounds with

```
[lb, ub] = getBounds(p(i,j,k));
```

`PortfolioMAD` object functions work on only one `PortfolioMAD` object at a time.

Subclassing PortfolioMAD Objects

You can subclass the `PortfolioMAD` object to override existing functions or to add new properties or functions. To do so, create a derived class from the `PortfolioMAD` class. This gives you all the properties and functions of the `PortfolioMAD` class along with any new features that you choose to add to your subclassed object. The `PortfolioMAD` class is derived from an abstract class called `AbstractPortfolio`. Because of this, you can also create a derived class from `AbstractPortfolio` that implements an entirely different form of portfolio optimization using properties and functions of the `AbstractPortfolio` class.

Conventions for Representation of Data

The MAD portfolio optimization tools follow these conventions regarding the representation of different quantities associated with portfolio optimization:

- Asset returns or prices for scenarios are in matrix form with samples for a given asset going down the rows and assets going across the columns. In the case of prices, the earliest dates must be at the top of the matrix, with increasing dates going down.
- Portfolios are in vector or matrix form with weights for a given portfolio going down the rows and distinct portfolios going across the columns.
- Constraints on portfolios are formed in such a way that a portfolio is a column vector.
- Portfolio risks and returns are either scalars or column vectors (for multiple portfolio risks and returns).

See Also

`PortfolioMAD`

Related Examples

- “Creating the `PortfolioMAD` Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50

More About

- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Creating the PortfolioMAD Object

In this section...

“Syntax” on page 6-22

“PortfolioMAD Problem Sufficiency” on page 6-22

“PortfolioMAD Function Examples” on page 6-23

To create a fully specified MAD portfolio optimization problem, instantiate the `PortfolioMAD` object using `PortfolioMAD`. For information on the workflow when using `PortfolioMAD` objects, see “PortfolioMAD Object Workflow” on page 6-16.

Syntax

Use `PortfolioMAD` to create an instance of an object of the `PortfolioMAD` class. You can use the `PortfolioMAD` object in several ways. To set up a portfolio optimization problem in a `PortfolioMAD` object, the simplest syntax is:

```
p = PortfolioMAD;
```

This syntax creates a `PortfolioMAD` object, `p`, such that all object properties are empty.

The `PortfolioMAD` object also accepts collections of argument name-value pair arguments for properties and their values. The `PortfolioMAD` object accepts inputs for public properties with the general syntax:

```
p = PortfolioMAD('property1', value1, 'property2', value2, ... );
```

If a `PortfolioMAD` object already exists, the syntax permits the first (and only the first argument) of `PortfolioMAD` to be an existing object with subsequent argument name-value pair arguments for properties to be added or modified. For example, given an existing `PortfolioMAD` object in `p`, the general syntax is:

```
p = PortfolioMAD(p, 'property1', value1, 'property2', value2, ... );
```

Input argument names are not case-sensitive, but must be completely specified. In addition, several properties can be specified with alternative argument names (see “Shortcuts for Property Names” on page 6-26). The `PortfolioMAD` object tries to detect problem dimensions from the inputs and, once set, subsequent inputs can undergo various scalar or matrix expansion operations that simplify the overall process to formulate a problem. In addition, a `PortfolioMAD` object is a value object so that, given portfolio `p`, the following code creates two objects, `p` and `q`, that are distinct:

```
q = PortfolioMAD(p, ...)
```

PortfolioMAD Problem Sufficiency

A MAD portfolio optimization problem is completely specified with the `PortfolioMAD` object if the following three conditions are met:

- You must specify a collection of asset returns or prices known as scenarios such that all scenarios are finite asset returns or prices. These scenarios are meant to be samples from the underlying probability distribution of asset returns. This condition can be satisfied by the `setScenarios` function or with several canned scenario simulation functions.

- The set of feasible portfolios must be a nonempty compact set, where a compact set is closed and bounded. You can satisfy this condition using an extensive collection of properties that define different types of constraints to form a set of feasible portfolios. Since such sets must be bounded, either explicit or implicit constraints can be imposed and several tools, such as the `estimateBounds` function, provide ways to ensure that your problem is properly formulated.

Although the general sufficient conditions for MAD portfolio optimization go beyond these conditions, the `PortfolioMAD` object handles all these additional conditions.

PortfolioMAD Function Examples

If you create a `PortfolioMAD` object, `p`, with no input arguments, you can display it using `disp`:

```
p = PortfolioMAD;
disp(p)
```

PortfolioMAD with properties:

```

        BuyCost: []
        SellCost: []
    RiskFreeRate: []
        Turnover: []
    BuyTurnover: []
    SellTurnover: []
    NumScenarios: []
        Name: []
    NumAssets: []
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: []
    UpperBound: []
    LowerBudget: []
    UpperBudget: []
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
        GroupA: []
        GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
        BoundType: []
```

The approaches listed provide a way to set up a portfolio optimization problem with the `PortfolioMAD` object. The custom set functions offer additional ways to set and modify collections of properties in the `PortfolioMAD` object.

Using the PortfolioMAD Function for a Single-Step Setup

You can use the PortfolioMAD object to directly set up a “standard” portfolio optimization problem. Given scenarios of asset returns in the variable AssetScenarios, this problem is completely specified as follows:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD('Scenarios', AssetScenarios, ...
'LowerBound', 0, 'LowerBudget', 1, 'UpperBudget', 1)
```

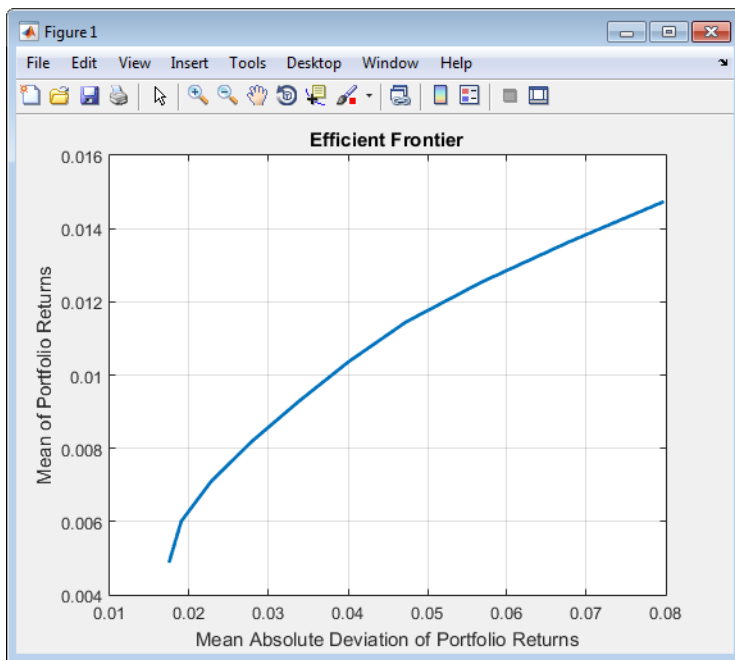
PortfolioMAD with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: 20000
Name: []
NumAssets: 4
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [4×1 double]
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []
```

The LowerBound property value undergoes scalar expansion since AssetScenarios provides the dimensions of the problem.

You can use dot notation with the function plotFrontier.

```
p.plotFrontier
```



Using the PortfolioMAD Function with a Sequence of Steps

An alternative way to accomplish the same task of setting up a “standard” MAD portfolio optimization problem, given AssetScenarios variable is:

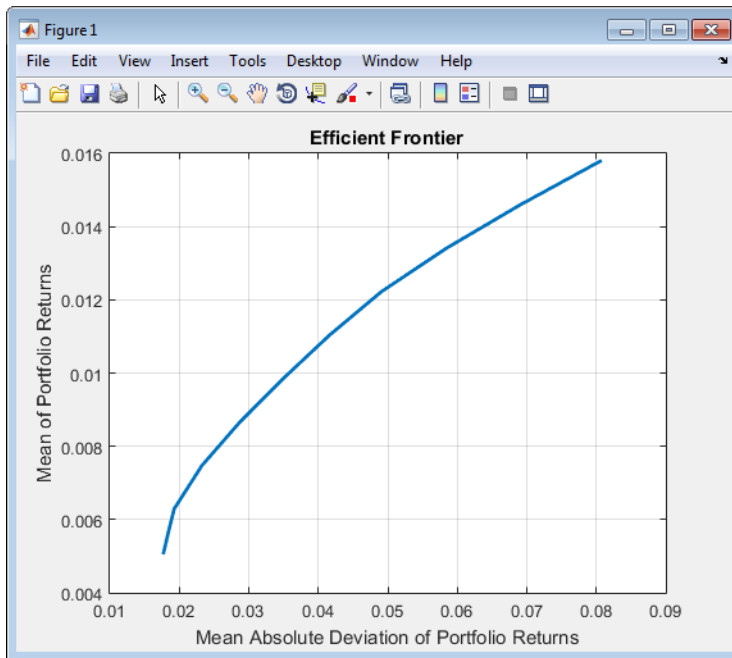
```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = PortfolioMAD(p, 'LowerBound', 0);
p = PortfolioMAD(p, 'LowerBudget', 1, 'UpperBudget', 1);

plotFrontier(p);
```



This way works because the calls to the `PortfolioMAD` object are in this particular order. In this case, the call to initialize `AssetScenarios` provides the dimensions for the problem. If you were to do this step last, you would have to explicitly dimension the `LowerBound` property as follows:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = PortfolioMAD(p, 'LowerBound', zeros(size(m)));
p = PortfolioMAD(p, 'LowerBudget', 1, 'UpperBudget', 1);
p = setScenarios(p, AssetScenarios);
```

Note If you did not specify the size of `LowerBound` but, instead, input a scalar argument, the `PortfolioMAD` object assumes that you are defining a single-asset problem and produces an error at the call to set asset scenarios with four assets.

Shortcuts for Property Names

The `PortfolioMAD` object has shorter argument names that replace longer argument names associated with specific properties of the `PortfolioMAD` object. For example, rather than enter `'AInequality'`, the `PortfolioMAD` object accepts the case-insensitive name `'ai'` to set the `AInequality` property in a `PortfolioMAD` object. Every shorter argument name corresponds with a single property in the `PortfolioMAD` object. The one exception is the alternative argument name

'budget', which signifies both the LowerBudget and UpperBudget properties. When 'budget' is used, then the LowerBudget and UpperBudget properties are set to the same value to form an equality budget constraint.

Shortcuts for Property Names

Shortcut Argument Name	Equivalent Argument / Property Name
ae	AEquality
ai	AInequality
assetnames or assets	AssetList
be	bEquality
bi	bInequality
budget	UpperBudget and LowerBudget
group	GroupMatrix
lb	LowerBound
n or num	NumAssets
rfr	RiskFreeRate
scenario or assetscenarios	Scenarios
ub	UpperBound

For example, this call to PortfolioMAD uses these shortcuts for properties:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD('scenario', AssetScenarios, 'lb', 0, 'budget', 1);
plotFrontier(p);
```

Direct Setting of Portfolio Object Properties

Although not recommended, you can set properties directly using dot notation, however no error-checking is done on your inputs:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;

p = setScenarios(p, AssetScenarios);
```

```
p.LowerBudget = 1;  
p.UpperBudget = 1;  
p.LowerBound = zeros(size(m));  
  
plotFrontier(p);
```

Note Scenarios cannot be assigned directly using dot notation to a `PortfolioMAD` object. Scenarios must always be set through either the `PortfolioMAD` object, the `setScenarios` function, or any of the scenario simulation functions.

See Also

`PortfolioMAD` | `estimateBounds`

Related Examples

- “Common Operations on the `PortfolioMAD` Object” on page 6-29
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50

More About

- “`PortfolioMAD` Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “`PortfolioMAD` Object Workflow” on page 6-16

Common Operations on the PortfolioMAD Object

In this section...

“Naming a PortfolioMAD Object” on page 6-29
 “Configuring the Assets in the Asset Universe” on page 6-29
 “Setting Up a List of Asset Identifiers” on page 6-29
 “Truncating and Padding Asset Lists” on page 6-31

Naming a PortfolioMAD Object

To name a PortfolioMAD object, use the Name property. Name is informational and has no effect on any portfolio calculations. If the Name property is nonempty, Name is the title for the efficient frontier plot generated by `plotFrontier`. For example, if you set up an asset allocation fund, you could name the PortfolioMAD object Asset Allocation Fund:

```
p = PortfolioMAD('Name', 'Asset Allocation Fund');
disp(p.Name);
Asset Allocation Fund
```

Configuring the Assets in the Asset Universe

The fundamental quantity in the PortfolioMAD object is the number of assets in the asset universe. This quantity is maintained in the NumAssets property. Although you can set this property directly, it is usually derived from other properties such as the number of assets in the scenarios or the initial portfolio. In some instances, the number of assets may need to be set directly. This example shows how to set up a PortfolioMAD object that has four assets:

```
p = PortfolioMAD('NumAssets', 4);
disp(p.NumAssets)
```

4

After setting the NumAssets property, you cannot modify it (unless no other properties are set that depend on NumAssets). The only way to change the number of assets in an existing PortfolioMAD object with a known number of assets is to create a new PortfolioMAD object.

Setting Up a List of Asset Identifiers

When working with portfolios, you must specify a universe of assets. Although you can perform a complete analysis without naming the assets in your universe, it is helpful to have an identifier associated with each asset as you create and work with portfolios. You can create a list of asset identifiers as a cell vector of character vectors in the property AssetList. You can set up the list using the next two methods.

Setting Up Asset Lists Using the PortfolioMAD Function

Suppose that you have a PortfolioMAD object, p, with assets with symbols 'AA', 'BA', 'CAT', 'DD', and 'ETR'. You can create a list of these asset symbols in the object using PortfolioMAD:

```
p = PortfolioMAD('assetlist', { 'AA', 'BA', 'CAT', 'DD', 'ETR' });
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

Notice that the property `AssetList` is maintained as a cell array that contains character vectors, and that it is necessary to pass a cell array into `PortfolioMAD` to set `AssetList`. In addition, notice that the property `NumAssets` is set to 5 based on the number of symbols used to create the asset list:

```
disp(p.NumAssets)
```

```
5
```

Setting Up Asset Lists Using the `setAssetList` Function

You can also specify a list of assets using the `setAssetList` function. Given the list of asset symbols 'AA', 'BA', 'CAT', 'DD', and 'ETR', you can use `setAssetList` with:

```
p = PortfolioMAD;  
p = setAssetList(p, { 'AA', 'BA', 'CAT', 'DD', 'ETR' });  
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

`setAssetList` also enables you to enter symbols directly as a comma-separated list without creating a cell array of character vectors. For example, given the list of assets symbols 'AA', 'BA', 'CAT', 'DD', and 'ETR', use `setAssetList`:

```
p = PortfolioMAD;  
p = setAssetList(p, 'AA', 'BA', 'CAT', 'DD', 'ETR');  
disp(p.AssetList)
```

```
'AA'      'BA'      'CAT'      'DD'      'ETR'
```

`setAssetList` has many additional features to create lists of asset identifiers. If you use `setAssetList` with just a `PortfolioMAD` object, it creates a default asset list according to the name specified in the hidden public property `defaultforAssetList` (which is 'Asset' by default). The number of asset names created depends on the number of assets in the property `NumAssets`. If `NumAssets` is not set, then `NumAssets` is assumed to be 1.

For example, if a `PortfolioMAD` object `p` is created with `NumAssets = 5`, then this code fragment shows the default naming behavior:

```
p = PortfolioMAD('numassets',5);  
p = setAssetList(p);  
disp(p.AssetList)
```

```
'Asset1'      'Asset2'      'Asset3'      'Asset4'      'Asset5'
```

Suppose that your assets are, for example, ETFs and you change the hidden property `defaultforAssetList` to 'ETF', you can then create a default list for ETFs:

```
p = PortfolioMAD('numassets',5);  
p.defaultforAssetList = 'ETF';  
p = setAssetList(p);  
disp(p.AssetList)
```

```
'ETF1'      'ETF2'      'ETF3'      'ETF4'      'ETF5'
```

Truncating and Padding Asset Lists

If the `NumAssets` property is already set and you pass in too many or too few identifiers, the `PortfolioMAD` object, and the `setAssetList` function truncate or pad the list with numbered default asset names that use the name specified in the hidden public property `defaultforAssetList`. If the list is truncated or padded, a warning message indicates the discrepancy. For example, assume that you have a `PortfolioMAD` object with five ETFs and you only know the first three CUSIPs '921937835', '922908769', and '922042775'. Use this syntax to create an asset list that pads the remaining asset identifiers with numbered 'UnknownCUSIP' placeholders:

```
p = PortfolioMAD('numassets',5);
p.defaultforAssetList = 'UnknownCUSIP';
p = setAssetList(p, '921937835', '922908769', '922042775');
disp(p.AssetList)
```

Warning: Input list of assets has 2 too few identifiers. Padding with numbered assets.

```
> In PortfolioMAD.setAssetList at 121
    Columns 1 through 4

    '921937835'    '922908769'    '922042775'    'UnknownCUSIP4'

    Column 5

    'UnknownCUSIP5'
```

Alternatively, suppose that you have too many identifiers and need only the first four assets. This example illustrates truncation of the asset list using the `PortfolioMAD` object:

```
p = PortfolioMAD('numassets',4);
p = PortfolioMAD(p, 'assetlist', { 'AGG', 'EEM', 'MDY', 'SPY', 'VEU' });
disp(p.AssetList)
```

Warning: AssetList has 1 too many identifiers. Using first 4 assets.

```
> In PortfolioMAD.checkarguments at 410
    In PortfolioMAD.PortfolioMAD>PortfolioMAD.PortfolioMAD at 187
    'AGG'    'EEM'    'MDY'    'SPY'
```

The hidden public property `uppercaseAssetList` is a Boolean flag to specify whether to convert asset names to uppercase letters. The default value for `uppercaseAssetList` is `false`. This example shows how to use the `uppercaseAssetList` flag to force identifiers to be uppercase letters:

```
p = PortfolioMAD;
p.uppercaseAssetList = true;
p = setAssetList(p, { 'aa', 'ba', 'cat', 'dd', 'etr' });
disp(p.AssetList)
```

```
'AA'    'BA'    'CAT'    'DD'    'ETR'
```

See Also

`PortfolioMAD` | `setAssetList` | `setInitPort` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Setting Up an Initial or Current Portfolio” on page 6-33
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Setting Up an Initial or Current Portfolio

In many applications, creating a new optimal portfolio requires comparing the new portfolio with an initial or current portfolio to form lists of purchases and sales. The `PortfolioMAD` object property `InitPort` lets you identify an initial or current portfolio. The initial portfolio also plays an essential role if you have either transaction costs or turnover constraints. The initial portfolio need not be feasible within the constraints of the problem. This can happen if the weights in a portfolio have shifted such that some constraints become violated. To check if your initial portfolio is feasible, use the `checkFeasibility` function described in “Validating MAD Portfolios” on page 6-81. Suppose that you have an initial portfolio in `x0`, then use the `PortfolioMAD` object to set up an initial portfolio:

```
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = PortfolioMAD('InitPort', x0);
disp(p.InitPort)
```

```
0.3000
0.2000
0.2000
0
```

As with all array properties, you can set `InitPort` with scalar expansion. This is helpful to set up an equally weighted initial portfolio of, for example, 10 assets:

```
p = PortfolioMAD('NumAssets', 10, 'InitPort', 1/10);
disp(p.InitPort)
```

```
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
0.1000
```

To clear an initial portfolio from your `PortfolioMAD` object, use either the `PortfolioMAD` object or the `setInitPort` function with an empty input for the `InitPort` property. If transaction costs or turnover constraints are set, it is not possible to clear the `InitPort` property in this way. In this case, to clear `InitPort`, first clear the dependent properties and then clear the `InitPort` property.

The `InitPort` property can also be set with `setInitPort` which lets you specify the number of assets if you want to use scalar expansion. For example, given an initial portfolio in `x0`, use `setInitPort` to set the `InitPort` property:

```
p = PortfolioMAD;
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = setInitPort(p, x0);
disp(p.InitPort)
```

```
0.3000
0.2000
0.2000
0
```

To create an equally weighted portfolio of four assets, use `setInitPort`:

```
p = PortfolioMAD;  
p = setInitPort(p, 1/4, 4);  
disp(p.InitPort)  
  
0.2500  
0.2500  
0.2500  
0.2500
```

`PortfolioMAD` object functions that work with either transaction costs or turnover constraints also depend on the `InitPort` property. So, the set functions for transaction costs or turnover constraints permit the assignment of a value for the `InitPort` property as part of their implementation. For details, see “Working with Average Turnover Constraints Using `PortfolioMAD` Object” on page 6-74, “Working with One-Way Turnover Constraints Using `PortfolioMAD` Object” on page 6-77, and “Working with Transaction Costs” on page 6-46. If either transaction costs or turnover constraints are used, then the `InitPort` property must have a nonempty value. Absent a specific value assigned through the `PortfolioMAD` object or various set functions, the `PortfolioMAD` object sets `InitPort` to 0 and warns if `BuyCost`, `SellCost`, or `Turnover` properties are set. This example shows what happens if you specify an average turnover constraint with an initial portfolio:

```
p = PortfolioMAD('Turnover', 0.3, 'InitPort', [ 0.3; 0.2; 0.2; 0.0 ]);  
disp(p.InitPort)  
  
0.3000  
0.2000  
0.2000  
0
```

In contrast, this example shows what happens if an average turnover constraint is specified without an initial portfolio:

```
p = PortfolioMAD('Turnover', 0.3);  
disp(p.InitPort)  
  
Warning: InitPort and NumAssets are empty and either transaction costs or  
turnover constraints specified. Will set NumAssets = 1 and InitPort = 0.  
> In PortfolioMAD.checkarguments at 446  
In PortfolioMAD.PortfolioMAD>PortfolioMAD.PortfolioMAD at 190  
0
```

See Also

`PortfolioMAD` | `setAssetList` | `setInitPort` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Common Operations on the `PortfolioMAD` Object” on page 6-29
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Asset Returns and Scenarios Using `PortfolioMAD` Object” on page 6-35

More About

- “`PortfolioMAD` Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “`PortfolioMAD` Object Workflow” on page 6-16

Asset Returns and Scenarios Using PortfolioMAD Object

In this section...

“How Stochastic Optimization Works” on page 6-35
 “What Are Scenarios?” on page 6-35
 “Setting Scenarios Using the PortfolioMAD Function” on page 6-36
 “Setting Scenarios Using the setScenarios Function” on page 6-37
 “Estimating the Mean and Covariance of Scenarios” on page 6-37
 “Simulating Normal Scenarios” on page 6-38
 “Simulating Normal Scenarios from Returns or Prices” on page 6-38
 “Simulating Normal Scenarios with Missing Data” on page 6-39
 “Simulating Normal Scenarios from Time Series Data” on page 6-41
 “Simulating Normal Scenarios with Mean and Covariance” on page 6-43

How Stochastic Optimization Works

The MAD of a portfolio is mean-absolute deviation. For the definition of the MAD function, see “Risk Proxy” on page 6-5. Although analytic solutions for MAD exist for a few probability distributions, an alternative is to compute the expectation for MAD with samples from the probability distribution of asset returns. These samples are called scenarios and, given a collection of scenarios, the portfolio optimization problem becomes a stochastic optimization problem.

As a function of the portfolio weights, the MAD of the portfolio is a convex non-smooth function (see Konno and Yamazaki [50] at “Portfolio Optimization” on page A-5). The PortfolioMAD object computes MAD as this nonlinear function which can be handled by the solver `fmincon` Optimization Toolbox. The nonlinear programming solver `fmincon` has several algorithms that can be selected with the `setSolver` function, the two algorithms that work best in practice are `'sqp'` and `'active-set'`.

There are reformulations of the MAD portfolio optimization problem (see Konno and Yamazaki [50] at “Portfolio Optimization” on page A-5) that result in a linear programming problem, which can be solved either with standard linear programming techniques or with stochastic programming solvers. The PortfolioMAD object, however, does not reformulate the problem in such a manner. The PortfolioMAD object computes the MAD as a nonlinear function. The convexity of the MAD, as a function of the portfolio weights and the dull edges when the number of scenarios is large, make the MAD portfolio optimization problem tractable, in practice, for certain nonlinear programming solvers, such as `fmincon` from Optimization Toolbox. To learn more about the workflow when using PortfolioMAD objects, see “PortfolioMAD Object Workflow” on page 6-16.

What Are Scenarios?

Since mean absolute deviation portfolio optimization works with scenarios of asset returns to perform the optimization, several ways exist to specify and simulate scenarios. In many applications with MAD portfolio optimization, asset returns may have distinctly nonnormal probability distributions with either multiple modes, binning of returns, truncation of distributions, and so forth. In other applications, asset returns are modeled as the result of various simulation methods that might include Monte-Carlo simulation, quasi-random simulation, and so forth. Often, the underlying

probability distribution for risk factors may be multivariate normal but the resultant transformations are sufficiently nonlinear to result in distinctively nonnormal asset returns.

For example, this occurs with bonds and derivatives. In the case of bonds with a nonzero probability of default, such scenarios would likely include asset returns that are -100% to indicate default and some values slightly greater than -100% to indicate recovery rates.

Although the `PortfolioMAD` object has functions to simulate multivariate normal scenarios from either data or moments (`simulateNormalScenariosByData` and `simulateNormalScenariosByMoments`), the usual approach is to specify scenarios directly from your own simulation functions. These scenarios are entered directly as a matrix with a sample for all assets across each row of the matrix and with samples for an asset down each column of the matrix. The architecture of the MAD portfolio optimization tools references the scenarios through a function handle so scenarios that have been set cannot be accessed directly as a property of the `PortfolioMAD` object.

Setting Scenarios Using the PortfolioMAD Function

Suppose that you have a matrix of scenarios in the `AssetScenarios` variable. The scenarios are set through the `PortfolioMAD` object with:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD('Scenarios', AssetScenarios);

disp(p.NumAssets)
disp(p.NumScenarios)

4
20000
```

Notice that the `PortfolioMAD` object determines and fixes the number of assets in `NumAssets` and the number of scenarios in `NumScenarios` based on the scenario's matrix. You can change the number of scenarios by calling the `PortfolioMAD` object with a different scenario matrix. However, once the `NumAssets` property has been set in the object, you cannot enter a scenario matrix with a different number of assets. The `getScenarios` function lets you recover scenarios from a `PortfolioMAD` object. You can also obtain the mean and covariance of your scenarios using `estimateScenarioMoments`.

Although not recommended for the casual user, an alternative way exists to recover scenarios by working with the function handle that points to scenarios in the `PortfolioMAD` object. To access some or all the scenarios from a `PortfolioMAD` object, the hidden property `localScenarioHandle` is a function handle that points to a function to obtain scenarios that have already been set. To get scenarios directly from a `PortfolioMAD` object `p`, use


```
scenarios = p.localScenarioHandle([], []);
```

and to obtain a subset of scenarios from rows `startrow` to `endrow`, use

```
scenarios = p.localScenarioHandle(startrow, endrow);
```

where $1 \leq \text{startrow} \leq \text{endrow} \leq \text{numScenarios}$.

Setting Scenarios Using the `setScenarios` Function

You can also set scenarios using `setScenarios`. For example, given the mean and covariance of asset returns in the variables `m` and `C`, the asset moment properties can be set:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);

disp(p.NumAssets)
disp(p.NumScenarios)

4

20000
```

Estimating the Mean and Covariance of Scenarios

This example shows how to use `estimateScenarioMoments` function to obtain estimates for the mean and covariance of scenarios in a `PortfolioMAD` object.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
[mean, covar] = estimateScenarioMoments(p)

mean = 4×1
```

```
0.0042
0.0082
0.0094
0.0147

covar = 4x4

0.0005    0.0003    0.0002   -0.0000
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
-0.0000    0.0010    0.0028    0.0100
```

Simulating Normal Scenarios

As a convenience, the two functions (`simulateNormalScenariosByData` and `simulateNormalScenariosByMoments`) exist to simulate scenarios from data or moments under an assumption that they are distributed as multivariate normal random asset returns.

Simulating Normal Scenarios from Returns or Prices

This example shows how to use the `simulateNormalScenariosByData` function with the `PortfolioMAD` object to simulate multivariate normal scenarios when you are given either return or price data. Either returns or prices are stored as matrices with samples going down the rows and assets going across the columns. In addition, returns or prices can be stored in a `table` or `timetable` (see “Simulating Normal Scenarios from Time Series Data” on page 6-41).

To illustrate using `simulateNormalScenariosByData`, generate random samples of 120 observations of asset returns for four assets from the mean and covariance of asset returns in the variables `m` and `C` with `portsim`. The default behavior of `portsim` creates simulated data with estimated mean and covariance identical to the input moments `m` and `C`. In addition to a return series created by `portsim` in the variable `X`, a price series is created in the variable `Y`:

```
m = [ 0.0042; 0.0083; 0.01; 0.15 ];
C = [ 0.005333 0.00034 0.00016 0;
0.00034 0.002408 0.0017 0.000992;
0.00016 0.0017 0.0048 0.0028;
0 0.000992 0.0028 0.010208 ];

X = portsim(m', C, 120);
Y = ret2tick(X);
```

Note that Portfolio optimization requires that you use total returns and not just price returns. So, “returns” should be total returns and “prices” should be total return prices.

Given asset returns and prices in variables `X` and `Y` from above, this sequence of code demonstrates equivalent ways to simulate multivariate normal scenarios for the `PortfolioMAD` object. Assume a `PortfolioMAD` object created in `p` that uses the asset returns in `X` uses `simulateNormalScenariosByData`:

```
p = PortfolioMAD;
p = simulateNormalScenariosByData(p, X, 20000);
```

```
[passetmean, passetcovar] = estimateScenarioMoments(p)
```

```
passetmean = 4×1
```

```
0.0042
0.0082
0.0094
0.1498
```

```
passetcovar = 4×4
```

```
0.0052    0.0003    0.0001   -0.0000
0.0003    0.0024    0.0017    0.0010
0.0001    0.0017    0.0048    0.0028
-0.0000    0.0010    0.0028    0.0100
```

The moments that you obtain from this simulation will likely differ from the moments listed here because the scenarios are random samples from the estimated multivariate normal probability distribution of the input returns X .

The default behavior of `simulateNormalScenariosByData` is to work with asset returns. If, instead, you have asset prices as in the variable Y , `simulateNormalScenariosByData` accepts a name-value pair argument name `'DataFormat'` with a corresponding value set to `'prices'` to indicate that the input to the function is in the form of asset prices and not returns (the default value for the `'DataFormat'` argument is `'returns'`). This example simulates scenarios with the asset price data in Y for the `PortfolioMAD` object q :

```
p = PortfolioMAD;
p = simulateNormalScenariosByData(p, Y, 20000, 'dataformat', 'prices');
```

```
[passetmean, passetcovar] = estimateScenarioMoments(p)
```

```
passetmean = 4×1
```

```
0.0046
0.0086
0.0097
0.1501
```

```
passetcovar = 4×4
```

```
0.0054    0.0003    0.0002    0.0000
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0029
0.0000    0.0010    0.0029    0.0103
```

Simulating Normal Scenarios with Missing Data

This example shows how to simulate normal scenarios when you have missing data indicated by `NaN` values in your return or price data. Although “Multivariate Normal Regression” on page 9-2 goes

into detail about regression with missing data, the `simulateNormalScenariosByData` function has a name-value pair argument name `'MissingData'` that indicates with a Boolean value whether to use the missing data capabilities of Financial Toolbox™. The default value for `'MissingData'` is `false` which removes all samples with NaN values. If, however, `'MissingData'` is set to `true`, `simulateNormalScenariosByData` uses the ECM algorithm to estimate asset moments. The following code shows how this works on price data with missing values.

```
m = [ 0.0042; 0.0083; 0.01; 0.15 ];
C = [ 0.005333 0.00034 0.00016 0;
      0.00034 0.002408 0.0017 0.000992;
      0.00016 0.0017 0.0048 0.0028;
      0 0.000992 0.0028 0.010208 ];
```

```
X = portsim(m', C, 120);
Y = ret2tick(X);
Y(1:20,1) = NaN;
Y(1:12,4) = NaN;
```

Notice that the prices above in `Y` have missing values in the first and fourth series.

```
p = PortfolioMAD;
p = simulateNormalScenariosByData(p, Y, 20000, 'dataformat', 'prices');

q = PortfolioMAD;
q = simulateNormalScenariosByData(q, Y, 20000, 'dataformat', 'prices', 'missingdata', true);
```

```
[passetmean, passetcovar] = estimateScenarioMoments(p)
```

```
passetmean = 4×1
```

```
0.0014
0.0095
0.0162
0.1587
```

```
passetcovar = 4×4
```

```
0.0050    0.0002   -0.0002    0.0001
0.0002    0.0025    0.0014    0.0006
-0.0002    0.0014    0.0038    0.0018
0.0001    0.0006    0.0018    0.0087
```

```
[qassetmean, qassetcovar] = estimateScenarioMoments(q)
```

```
qassetmean = 4×1
```

```
0.0024
0.0086
0.0096
0.1544
```

```
qassetcovar = 4×4
```

```
0.0067    0.0003   -0.0003    0.0002
0.0003    0.0032    0.0022    0.0010
```

```
-0.0003    0.0022    0.0064    0.0027
 0.0002    0.0010    0.0027    0.0119
```

The first `PortfolioMAD` object, `p`, contains scenarios obtained from price data in `Y` where `NaN` values are discarded and the second `PortfolioCVaR` object, `q`, contains scenarios obtained from price data in `Y` that accommodate missing values. Each time you run this example, you get different estimates for the moments in `p` and `q`.

Simulating Normal Scenarios from Time Series Data

This example shows how the `simulateNormalScenariosByData` function accepts asset returns or prices stored in a table or timetable. The `simulateNormalScenariosByData` function implicitly works with matrices of data or data in a `table` or `timetable` object using the same rules for whether the data are returns or prices.

To illustrate, use `array2timetable` to create a timetable for 14 assets from `CAPMuniverse` and then use the timetable to simulate scenarios for `PortfolioMAD`.

```
load CAPMuniverse
time = datetime(Dates,'ConvertFrom','datenum');
stockTT = array2timetable(Data,'RowTimes',time, 'VariableNames', Assets);
stockTT.Properties

ans =
    TimetableProperties with properties:

        Description: ''
        UserData: []
    DimensionNames: {'Time' 'Variables'}
    VariableNames: {'AAPL' 'AMZN' 'CSCO' 'DELL' 'EBAY' 'GOOG' 'HPQ' 'IBM' 'INTC'
    VariableTypes: ["double" "double" "double" "double" "double" "double"]
    VariableDescriptions: {}
    VariableUnits: {}
    VariableContinuity: []
        RowTimes: [1471x1 datetime]
        StartTime: 03-Jan-2000
        SampleRate: NaN
        TimeStep: NaN
        Events: []
    CustomProperties: No custom properties are set.
    Use addprop and rmprop to modify CustomProperties.
```

% Notice that GOOG has missing data, because it was not listed before Aug 2004
`head(stockTT, 5)`

Time	AAPL	AMZN	CSCO	DELL	EBAY	GOOG	HPQ
03-Jan-2000	0.088805	0.1742	0.008775	-0.002353	0.12829	NaN	0.0
04-Jan-2000	-0.084331	-0.08324	-0.05608	-0.08353	-0.093805	NaN	-0.0
05-Jan-2000	0.014634	-0.14877	-0.003039	0.070984	0.066875	NaN	-0.0
06-Jan-2000	-0.086538	-0.060072	-0.016619	-0.038847	-0.012302	NaN	-0.0
07-Jan-2000	0.047368	0.061013	0.0587	-0.037708	-0.000964	NaN	0.0

Use the 'MissingData' option offered by PortfolioMAD to account for the missing data.

```
p = PortfolioMAD;
p = simulateNormalScenariosByData(p, stockTT, 20000, 'missingdata', true);
[passetmean, passetcovar] = estimateScenarioMoments(p)
```

```
passetmean = 14×1
```

```
0.0012
0.0007
-0.0004
-0.0000
0.0016
0.0043
-0.0000
0.0001
0.0001
-0.0003
0.0001
0.0003
0.0001
0.0001
:
```

```
passetcovar = 14×14
```

```
0.0013    0.0005    0.0006    0.0005    0.0005    0.0003    0.0005    0.0003    0.0006    0.0006
0.0005    0.0024    0.0007    0.0005    0.0010    0.0005    0.0005    0.0003    0.0006    0.0006
0.0006    0.0007    0.0013    0.0006    0.0007    0.0004    0.0006    0.0004    0.0008    0.0008
0.0005    0.0005    0.0006    0.0009    0.0006    0.0002    0.0005    0.0003    0.0006    0.0006
0.0005    0.0010    0.0007    0.0006    0.0018    0.0007    0.0005    0.0003    0.0006    0.0006
0.0003    0.0005    0.0004    0.0002    0.0007    0.0013    0.0002    0.0002    0.0002    0.0002
0.0005    0.0005    0.0006    0.0005    0.0005    0.0002    0.0010    0.0003    0.0005    0.0005
0.0003    0.0003    0.0004    0.0003    0.0003    0.0002    0.0003    0.0005    0.0004    0.0004
0.0006    0.0006    0.0008    0.0006    0.0006    0.0002    0.0005    0.0004    0.0011    0.0011
0.0004    0.0004    0.0005    0.0004    0.0005    0.0002    0.0003    0.0002    0.0005    0.0005
0.0005    0.0006    0.0008    0.0005    0.0007    0.0003    0.0005    0.0004    0.0007    0.0007
0.0006    0.0011    0.0008    0.0006    0.0011    0.0011    0.0006    0.0004    0.0007    0.0007
0.0002    0.0002    0.0002    0.0002    0.0002    0.0001    0.0002    0.0002    0.0002    0.0002
-0.0000    -0.0000    -0.0000    -0.0000    -0.0000    -0.0000    -0.0000    0.0000    -0.0000    -0.0000
:
```

Use the name-value input 'DataFormat' to handle return or price data and 'MissingData' to ignore or use samples with missing values. In addition, `simulateNormalScenariosByData` extracts asset names or identifiers from a table or timetable if the argument 'GetAssetList' is set to true (the default value is false). If the 'GetAssetList' value is true, the identifiers are used to set the `AssetList` property of the `PortfolioMAD` object. Thus, repeating the formation of the `PortfolioMAD` object `p` with the 'GetAssetList' flag set to true extracts the column names from the timetable object:

```
p = simulateNormalScenariosByData(p, stockTT, 20000, 'missingdata', true, 'GetAssetList', true);
disp(p.AssetList)
```

```
{'AAPL'}    {'AMZN'}    {'CSCO'}    {'DELL'}    {'EBAY'}    {'GOOG'}    {'HPQ'}    {'IBM'}
```

If you set the 'GetAssetList' flag set to `true` and your input data is in a matrix, `simulateNormalScenariosByData` uses the default labeling scheme from `setAssetList` as described in “Setting Up a List of Asset Identifiers” on page 5-29.

Simulating Normal Scenarios with Mean and Covariance

This example shows how to use the `simulateNormalScenariosByMoments` function to simulate multivariate normal scenarios when given the mean and covariance of asset returns. The mean can be either a row or column vector and the covariance matrix must be a symmetric positive-semidefinite matrix. Various rules for scalar expansion apply.

To illustrate using `simulateNormalScenariosByMoments` with a `PortfolioMAD` object, start with moments in `m` and `C` and generate 20,000 scenarios.

```
m = [ 0.0042; 0.0083; 0.01; 0.15 ];
C = [ 0.005333 0.00034 0.00016 0;
      0.00034 0.002408 0.0017 0.000992;
      0.00016 0.0017 0.0048 0.0028;
      0 0.000992 0.0028 0.010208 ];
```

```
p = PortfolioMAD;
p = simulateNormalScenariosByMoments(p, m, C, 20000);
[passetmean, passetcovar] = estimateScenarioMoments(p)
```

```
passetmean = 4×1
```

```
0.0042
0.0082
0.0094
0.1497
```

```
passetcovar = 4×4
```

```
0.0052    0.0003    0.0001   -0.0000
0.0003    0.0024    0.0017    0.0010
0.0001    0.0017    0.0048    0.0028
-0.0000    0.0010    0.0028    0.0100
```

`simulateNormalScenariosByMoments` performs scalar expansion on arguments for the moments of asset returns. If `NumAssets` has not already been set, a scalar argument is interpreted as a scalar with `NumAssets` set to 1. `simulateNormalScenariosByMoments` provides an additional optional argument to specify the number of assets so that scalar expansion works with the correct number of assets. In addition, if either a scalar or vector is input for the covariance of asset returns, a diagonal matrix is formed such that a scalar expands along the diagonal and a vector becomes the diagonal.

See Also

`PortfolioMAD` | `setCosts` | `setScenarios` | `simulateNormalScenariosByMoments` | `simulateNormalScenariosByData`

Related Examples

- “Working with a Riskless Asset” on page 6-45
- “Working with Transaction Costs” on page 6-46
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with a Riskless Asset

The PortfolioMAD object has a separate RiskFreeRate property that stores the rate of return of a riskless asset. Thus, you can separate your universe into a riskless asset and a collection of risky assets. For example, assume that your riskless asset has a return in the scalar variable `r0`, then the property for the RiskFreeRate is set using the PortfolioMAD object:

```
r0 = 0.01/12;

p = PortfolioMAD;
p = PortfolioMAD('RiskFreeRate', r0);
disp(p.RiskFreeRate)

8.3333e-04
```

Note If your portfolio problem has a budget constraint such that your portfolio weights must sum to 1, then the riskless asset is irrelevant.

See Also

PortfolioMAD | setCosts | setScenarios | simulateNormalScenariosByMoments | simulateNormalScenariosByData

Related Examples

- “Working with Transaction Costs” on page 6-46
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with Transaction Costs

The difference between net and gross portfolio returns is transaction costs. The net portfolio return proxy has distinct proportional costs to purchase and to sell assets which are maintained in the `PortfolioMAD` object properties `BuyCost` and `SellCost`. Transaction costs are in units of total return and, as such, are proportional to the price of an asset so that they enter the model for net portfolio returns in return form. For example, suppose that you have a stock currently priced \$40 and your usual transaction costs are 5 cents per share. Then the transaction cost for the stock is $0.05/40 = 0.00125$ (as defined in “Net Portfolio Returns” on page 6-4). Costs are entered as positive values and credits are entered as negative values.

Setting Transaction Costs Using the `PortfolioMAD` Function

To set up transaction costs, you must specify an initial or current portfolio in the `InitPort` property. If the initial portfolio is not set when you set up the transaction cost properties, `InitPort` is 0. The properties for transaction costs can be set using the `PortfolioMAD` object. For example, assume that purchase and sale transaction costs are in the variables `bc` and `sc` and an initial portfolio is in the variable `x0`, then transaction costs are set:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = PortfolioMAD('BuyCost', bc, 'SellCost', sc, 'InitPort', x0);
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
0.0013
0.0013
0.0013
0.0013
0.0013
0.0013
0.0070
0.0013
0.0013
0.0024
0.4000
0.2000
0.2000
0.1000
0.1000
```

Setting Transaction Costs Using the `setCosts` Function

You can also set the properties for transaction costs using `setCosts`. Assume that you have the same costs and initial portfolio as in the previous example. Given a `PortfolioMAD` object `p` with an initial portfolio already set, use `setCosts` to set up transaction costs:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
```

```
p = PortfolioMAD('InitPort', x0);
p = setCosts(p, bc, sc);
```

```
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
0.0013
0.0013
0.0013
0.0013
0.0013
0.0013
0.0070
0.0013
0.0013
0.0024
0.4000
0.2000
0.2000
0.1000
0.1000
```

You can also set up the initial portfolio's `InitPort` value as an optional argument to `setCosts` so that the following is an equivalent way to set up transaction costs:

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
```

```
p = PortfolioMAD;
p = setCosts(p, bc, sc, x0);
```

```
disp(p.NumAssets)
disp(p.BuyCost)
disp(p.SellCost)
disp(p.InitPort)
```

```
5
0.0013
0.0013
0.0013
0.0013
0.0013
0.0013
0.0070
0.0013
0.0013
```

```
0.0013
0.0024

0.4000
0.2000
0.2000
0.1000
0.1000
```

Setting Transaction Costs with Scalar Expansion

Both the `PortfolioMAD` object and `setCosts` function implement scalar expansion on the arguments for transaction costs and the initial portfolio. If the `NumAssets` property is already set in the `PortfolioMAD` object, scalar arguments for these properties are expanded to have the same value across all dimensions. In addition, `setCosts` lets you specify `NumAssets` as an optional final argument. For example, assume that you have an initial portfolio `x0` and you want to set common transaction costs on all assets in your universe. You can set these costs in any of these equivalent ways:

```
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = PortfolioMAD('InitPort', x0, 'BuyCost', 0.002, 'SellCost', 0.002);
```

or

```
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = PortfolioMAD('InitPort', x0);
p = setCosts(p, 0.002, 0.002);
```

or

```
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
p = PortfolioMAD;
p = setCosts(p, 0.002, 0.002, x0);
```

To clear costs from your `PortfolioMAD` object, use either the `PortfolioMAD` object or `setCosts` with empty inputs for the properties to be cleared. For example, you can clear sales costs from the `PortfolioMAD` object `p` in the previous example:

```
p = PortfolioMAD(p, 'SellCost', []);
```

See Also

`PortfolioMAD` | `setCosts` | `setScenarios` | `simulateNormalScenariosByMoments` | `simulateNormalScenariosByData`

Related Examples

- “Working with a Riskless Asset” on page 6-45
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84

- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with MAD Portfolio Constraints Using Defaults

The final element for a complete specification of a portfolio optimization problem is the set of feasible portfolios, which is called a portfolio set. A portfolio set $X \subset R^n$ is specified by construction as the intersection of sets formed by a collection of constraints on portfolio weights. A portfolio set necessarily and sufficiently must be a nonempty, closed, and bounded set.

When setting up your portfolio set, ensure that the portfolio set satisfies these conditions. The most basic or “default” portfolio set requires portfolio weights to be nonnegative (using the lower-bound constraint) and to sum to 1 (using the budget constraint). For information on the workflow when using `PortfolioMAD` objects, see “PortfolioMAD Object Workflow” on page 6-16.

Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object

The “default” MAD portfolio problem has two constraints on portfolio weights:

- Portfolio weights must be nonnegative.
- Portfolio weights must sum to 1.

Implicitly, these constraints imply that portfolio weights are no greater than 1, although this is a superfluous constraint to impose on the problem.

Setting Default Constraints Using the PortfolioMAD Function

Given a portfolio optimization problem with `NumAssets` = 20 assets, use the `PortfolioMAD` object to set up a default problem and explicitly set bounds and budget constraints:

```
p = PortfolioMAD('NumAssets', 20, 'LowerBound', 0, 'Budget', 1);
disp(p)
```

PortfolioMAD with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: []
Name: []
NumAssets: 20
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [20×1 double]
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
LowerGroup: []
```

```

        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: []

```

Setting Default Constraints Using the `setDefaultConstraints` Function

An alternative approach is to use the `setDefaultConstraints` function. If the number of assets is already known in a `PortfolioMAD` object, use `setDefaultConstraints` with no arguments to set up the necessary bound and budget constraints. Suppose that you have 20 assets to set up the portfolio set for a default problem:

```

p = PortfolioMAD('NumAssets', 20);
p = setDefaultConstraints(p);
disp(p)

```

PortfolioMAD with properties:

```

        BuyCost: []
        SellCost: []
        RiskFreeRate: []
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
        NumScenarios: []
        Name: []
        NumAssets: 20
        AssetList: []
        InitPort: []
        AInequality: []
        bInequality: []
        AEquality: []
        bEquality: []
        LowerBound: [20x1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: [20x1 categorical]

```

If the number of assets is unknown, `setDefaultConstraints` accepts `NumAssets` as an optional argument to form a portfolio set for a default problem. Suppose that you have 20 assets:

```
p = PortfolioMAD;  
p = setDefaultConstraints(p, 20);  
disp(p)
```

PortfolioMAD with properties:

```
        BuyCost: []  
        SellCost: []  
RiskFreeRate: []  
        Turnover: []  
    BuyTurnover: []  
    SellTurnover: []  
    NumScenarios: []  
        Name: []  
    NumAssets: 20  
    AssetList: []  
    InitPort: []  
    AInequality: []  
    bInequality: []  
    AEquality: []  
    bEquality: []  
    LowerBound: [20×1 double]  
    UpperBound: []  
    LowerBudget: 1  
    UpperBudget: 1  
    GroupMatrix: []  
    LowerGroup: []  
    UpperGroup: []  
        GroupA: []  
        GroupB: []  
    LowerRatio: []  
    UpperRatio: []  
    MinNumAssets: []  
    MaxNumAssets: []  
ConditionalBudgetThreshold: []  
    ConditionalUpperBudget: []  
        BoundType: [20×1 categorical]
```

See Also

PortfolioMAD | setDefaultConstraints | setBounds | setBudget | setConditionalBudget
| setGroups | setGroupRatio | setEquality | setInequality | setTurnover |
setOneWayTurnover

Related Examples

- “Working with ‘Simple’ Bound Constraints Using PortfolioMAD Object” on page 6-54
- “Working with Budget Constraints Using PortfolioMAD Object” on page 6-57
- “Working with Group Constraints Using PortfolioMAD Object” on page 6-61
- “Working with Group Ratio Constraints Using PortfolioMAD Object” on page 6-64
- “Working with Linear Equality Constraints Using PortfolioMAD Object” on page 6-67
- “Working with Linear Inequality Constraints Using PortfolioMAD Object” on page 6-69
- “Working with Average Turnover Constraints Using PortfolioMAD Object” on page 6-74
- “Working with One-Way Turnover Constraints Using PortfolioMAD Object” on page 6-77

- “Adding Constraints to Satisfy UCITS Directive” on page 4-457
- “Creating the PortfolioMAD Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with 'Simple' Bound Constraints Using PortfolioMAD Object

'Simple' bound constraints are optional linear constraints that maintain upper and lower bounds on portfolio weights (see "'Simple' Bound Constraints" on page 6-9). Although every portfolio set must be bounded, it is not necessary to specify a portfolio set with explicit bound constraints. For example, you can create a portfolio set with an implicit upper bound constraint or a portfolio set with average turnover constraints. The bound constraints have properties `LowerBound` for the lower-bound constraint and `UpperBound` for the upper-bound constraint. Set default values for these constraints using the `setDefaultConstraints` function (see "Setting Default Constraints for Portfolio Weights Using Portfolio Object" on page 4-58).

Setting 'Simple' Bounds Using the PortfolioMAD Function

The properties for bound constraints are set through the `PortfolioMAD` object. Suppose that you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. The bound constraints for a balanced fund are set with:

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];
p = PortfolioMAD('LowerBound', lb, 'UpperBound', ub, 'BoundType', 'Simple');
disp(p.NumAssets)
disp(p.LowerBound)
disp(p.UpperBound)

2

0.5000
0.2500

0.7500
0.5000
```

To continue with this example, you must set up a budget constraint. For details, see "Working with Budget Constraints Using Portfolio Object" on page 4-65.

Setting 'Simple' Bounds Using the setBounds Function

You can also set the properties for bound constraints using `setBounds`. Suppose that you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. Given a `PortfolioMAD` object `p`, use `setBounds` to set the bound constraints:

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];
p = PortfolioMAD;
p = setBounds(p, lb, ub, 'BoundType', 'Simple');
disp(p.NumAssets)
disp(p.LowerBound)
disp(p.UpperBound)

2

0.5000
```

```
0.2500
```

```
0.7500
```

```
0.5000
```

Setting 'Simple' Bounds Using the PortfolioMAD Function or setBounds Function

Both the PortfolioMAD object and setBounds function implement scalar expansion on either the LowerBound or UpperBound properties. If the NumAssets property is already set in the PortfolioMAD object, scalar arguments for either property expand to have the same value across all dimensions. In addition, setBounds lets you specify NumAssets as an optional argument. Suppose that you have a universe of 500 assets and you want to set common bound constraints on all assets in your universe. Specifically, you are a long-only investor and want to hold no more than 5% of your portfolio in any single asset. You can set these bound constraints in any of these equivalent ways:

```
p = PortfolioMAD('NumAssets', 500, 'LowerBound', 0, 'UpperBound', 0.05, 'BoundType', 'Simple');
```

or

```
p = PortfolioMAD('NumAssets', 500);
p = setBounds(p, 0, 0.05, 'BoundType', 'Simple');
```

or

```
p = PortfolioMAD;
p = setBounds(p, 0, 0.05, 'NumAssets', 500, 'BoundType', 'Simple');
```

To clear bound constraints from your PortfolioMAD object, use either the PortfolioMAD object or setBounds with empty inputs for the properties to be cleared. For example, to clear the upper-bound constraint from the PortfolioMAD object p in the previous example:

```
p = PortfolioMAD(p, 'UpperBound', []);
```

See Also

PortfolioMAD | setDefaultConstraints | setBounds | setBudget | setConditionalBudget | setGroups | setGroupRatio | setEquality | setInequality | setTurnover | setOneWayTurnover

Related Examples

- “Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50
- “Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148
- “Creating the PortfolioMAD Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35
- “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects” on page 6-71

- “Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with Budget Constraints Using PortfolioMAD Object

The budget constraint is an optional linear constraint that maintains upper and lower bounds on the sum of portfolio weights (see “Budget Constraints” on page 5-10). Budget constraints have properties `LowerBudget` for the lower budget constraint and `UpperBudget` for the upper budget constraint. If you set up a MAD portfolio optimization problem that requires portfolios to be fully invested in your universe of assets, you can set `LowerBudget` to be equal to `UpperBudget`. These budget constraints can be set with default values equal to 1 using `setDefaultConstraints` (see “Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50).

Setting Budget Constraints Using the PortfolioMAD Function

The properties for the budget constraint can also be set using the `PortfolioMAD` object. Suppose that you have an asset universe with many risky assets and a riskless asset and you want to ensure that your portfolio never holds more than 1% cash, that is, you want to ensure that you are 99-100% invested in risky assets. The budget constraint for this portfolio can be set with:

```
p = PortfolioMAD('LowerBudget', 0.99, 'UpperBudget', 1);
disp(p.LowerBudget)
disp(p.UpperBudget)

0.9900

1
```

Setting Budget Constraints Using the setBudget Function

You can also set the properties for a budget constraint using `setBudget`. Suppose that you have a fund that permits up to 10% leverage which means that your portfolio can be from 100% to 110% invested in risky assets. Given a `PortfolioMAD` object `p`, use `setBudget` to set the budget constraints:

```
p = PortfolioMAD;
p = setBudget(p, 1, 1.1);
disp(p.LowerBudget)
disp(p.UpperBudget)

1

1.1000
```

If you were to continue with this example, then set the `RiskFreeRate` property to the borrowing rate to finance possible leveraged positions. For details on the `RiskFreeRate` property, see “Working with a Riskless Asset” on page 6-45. To clear either bound for the budget constraint from your `PortfolioMAD` object, use either the `PortfolioMAD` object or `setBudget` with empty inputs for the properties to be cleared. For example, clear the upper-budget constraint from the `PortfolioMAD` object `p` in the previous example with:

```
p = PortfolioMAD(p, 'UpperBudget', []);
```

See Also

`PortfolioMAD` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50
- “Creating the PortfolioMAD Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with Conditional Budget Constraints Using PortfolioMAD Object

The conditional budget constraint supports the Undertakings for Collective Investment in Transferable Securities (UCITS) directive for PortfolioMAD objects. The UCITS asset allocation regulation states that funds can only invest up to 10% in a single issuer, and that investments in excess of 5% must not exceed 40% of the total portfolio. This is known as the 5/10/40 rule. For more information, see “Undertakings for Collective Investment in Transferable Securities” on page 15-1452 and “Adding Constraints to Satisfy UCITS Directive” on page 4-457.

Setting Conditional Budget Constraints Using the PortfolioMAD Function

The properties for the conditional budget constraint can also be set using the PortfolioMAD object. Suppose that you have an asset universe with many risky assets and a riskless asset and you want to ensure that your portfolio never holds more than 1% cash, that is, you want to ensure that you are 99-100% invested in risky assets. The conditional budget constraint for this portfolio can be set with:

```
p = PortfolioMAD('ConditionalBudgetThreshold', 0.99, 'ConditionalUpperBudget', 1);
disp(p.ConditionalBudgetThreshold)
disp(p.ConditionalUpperBudget)

0.9900

1
```

Setting Conditional Budget Constraints Using the setConditionalBudget Function

You can also set the properties for a conditional budget constraint using setConditionalBudget. Suppose that you have a fund that permits up to 10% leverage which means that your portfolio can be from 100% to 110% invested in risky assets. Given a PortfolioMAD object p, use setConditionalBudget to set the conditional budget constraints:

```
p = PortfolioMAD;
p = setConditionBudget(p, .40, 1.5);
disp(p.threshold)
disp(p.upperBudget)

.40

1.5000
```

See Also

Portfolio | setDefaultConstraints | setBounds | setBudget | setGroups | setGroupRatio | setEquality | setInequality | setTurnover | setOneWayTurnover | setTrackingPort | setTrackingError

Related Examples

- “Creating the Portfolio Object” on page 4-25

- “Working with Portfolio Constraints Using Defaults” on page 4-58
- “Validate the Portfolio Problem for Portfolio Object” on page 4-94
- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Constraint Specification Using a Portfolio Object” on page 3-19
- “Asset Allocation Case Study” on page 4-180
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Using Social Performance Measure” on page 4-264
- “Diversify Portfolios Using Custom Objective” on page 4-331

More About

- “Portfolio Object” on page 4-20
- “Portfolio Optimization Theory” on page 4-4
- “Portfolio Object Workflow” on page 4-18
- “Setting Up a Tracking Portfolio” on page 4-40

External Websites

- Getting Started with Portfolio Optimization (4 min 13 sec)
- MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)

Working with Group Constraints Using PortfolioMAD Object

Group constraints are optional linear constraints that group assets together and enforce bounds on the group weights (see “Group Constraints” on page 6-11). Although the constraints are implemented as general constraints, the usual convention is to form a group matrix that identifies membership of each asset within a specific group with Boolean indicators (either `true` or `false` or with 1 or 0) for each element in the group matrix. Group constraints have properties `GroupMatrix` for the group membership matrix, `LowerGroup` for the lower-bound constraint on groups, and `UpperGroup` for the upper-bound constraint on groups.

Setting Group Constraints Using the PortfolioMAD Function

The properties for group constraints are set through the `PortfolioMAD` object. Suppose that you have a portfolio of five assets and want to ensure that the first three assets constitute no more than 30% of your portfolio, then you can set group constraints:

```
G = [ 1 1 1 0 0 ];
p = PortfolioMAD('GroupMatrix', G, 'UpperGroup', 0.3);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.UpperGroup)
```

5

1 1 1 0 0

0.3000

The group matrix `G` can also be a logical matrix so that the following code achieves the same result.

```
G = [ true true true false false ];
p = PortfolioMAD('GroupMatrix', G, 'UpperGroup', 0.3);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.UpperGroup)
```

5

1 1 1 0 0

0.3000

Setting Group Constraints Using the setGroups and addGroups Functions

You can also set the properties for group constraints using `setGroups`. Suppose that you have a portfolio of five assets and want to ensure that the first three assets constitute no more than 30% of your portfolio. Given a `PortfolioMAD` object `p`, use `setGroups` to set the group constraints:

```
G = [ true true true false false ];
p = PortfolioMAD;
p = setGroups(p, G, [], 0.3);
disp(p.NumAssets);
disp(p.GroupMatrix);
disp(p.UpperGroup);
```

```

5
1      1      1      0      0
0.3000

```

In this example, you would set the `LowerGroup` property to be empty (`[]`).

Suppose that you want to add another group constraint to make odd-numbered assets constitute at least 20% of your portfolio. Set up an augmented group matrix and introduce infinite bounds for unconstrained group bounds or use the `addGroups` function to build up group constraints. For this example, create another group matrix for the second group constraint:

```

p = PortfolioMAD;
G = [ true true true false false ]; % group matrix for first group constraint
p = setGroups(p, G, [], 0.3);
G = [ true false true false true ]; % group matrix for second group constraint
p = addGroups(p, G, 0.2);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.LowerGroup)
disp(p.UpperGroup)

5
1      1      1      0      0
1      0      1      0      1

-Inf
0.2000

0.3000
Inf

```

`addGroups` determines which bounds are unbounded so you only need to focus on the constraints that you want to set.

The `PortfolioMAD` object, `setGroups`, and `addGroups` implement scalar expansion on either the `LowerGroup` or `UpperGroup` properties based on the dimension of the group matrix in the property `GroupMatrix`. Suppose that you have a universe of 30 assets with 6 asset classes such that assets 1–5, assets 6–12, assets 13–18, assets 19–22, assets 23–27, and assets 28–30 constitute each of your asset classes and you want each asset class to fall from 0% to 25% of your portfolio. Let the following group matrix define your groups and scalar expansion define the common bounds on each group:

```

p = PortfolioMAD;
G = blkdiag(true(1,5), true(1,7), true(1,6), true(1,4), true(1,5), true(1,3));
p = setGroups(p, G, 0, 0.25);
disp(p.NumAssets)
disp(p.GroupMatrix)
disp(p.LowerGroup)
disp(p.UpperGroup)

30

Columns 1 through 13

1      1      1      1      1      0      0      0      0      0      0      0      0
0      0      0      0      0      1      1      1      1      1      1      1      0
0      0      0      0      0      0      0      0      0      0      0      0      1
0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0

Columns 14 through 26

0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0

```

```

1      1      1      1      1      0      0      0      0      0      0      0      0
0      0      0      0      0      1      1      1      1      0      0      0      0
0      0      0      0      0      0      0      0      0      1      1      1      1
0      0      0      0      0      0      0      0      0      0      0      0      0

Columns 27 through 30

0      0      0      0
0      0      0      0
0      0      0      0
0      0      0      0
1      0      0      0
0      1      1      1

0
0
0
0
0
0
0

0.2500
0.2500
0.2500
0.2500
0.2500
0.2500

```

See Also

PortfolioMAD | setDefaultConstraints | setBounds | setBudget | setConditionalBudget | setGroups | setGroupRatio | setEquality | setInequality | setTurnover | setOneWayTurnover

Related Examples

- “Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50
- “Creating the PortfolioMAD Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with Group Ratio Constraints Using PortfolioMAD Object

Group ratio constraints are optional linear constraints that maintain bounds on proportional relationships among groups of assets (see “Group Ratio Constraints” on page 6-12). Although the constraints are implemented as general constraints, the usual convention is to specify a pair of group matrices that identify membership of each asset within specific groups with Boolean indicators (either `true` or `false` or with 1 or 0) for each element in each of the group matrices. The goal is to ensure that the ratio of a base group compared to a comparison group fall within specified bounds. Group ratio constraints have properties:

- `GroupA` for the base membership matrix
- `GroupB` for the comparison membership matrix
- `LowerRatio` for the lower-bound constraint on the ratio of groups
- `UpperRatio` for the upper-bound constraint on the ratio of groups

Setting Group Ratio Constraints Using the PortfolioMAD Function

The properties for group ratio constraints are set using `PortfolioMAD` object. For example, assume that you want the ratio of financial to nonfinancial companies in your portfolios to never go above 50%. Suppose that you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). To set group ratio constraints:

```
GA = [ 1 1 1 0 0 0 ]; % financial companies
GB = [ 0 0 0 1 1 1 ]; % nonfinancial companies
p = PortfolioMAD('GroupA', GA, 'GroupB', GB, 'UpperRatio', 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

6

```
1      1      1      0      0      0
0      0      0      1      1      1
```

0.5000

Group matrices `GA` and `GB` in this example can be logical matrices with `true` and `false` elements that yield the same result:

```
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = PortfolioMAD('GroupA', GA, 'GroupB', GB, 'UpperRatio', 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

6

```
1      1      1      0      0      0
0      0      0      1      1      1
```

0.5000

Setting Group Ratio Constraints Using the `setGroupRatio` and `addGroupRatio` Functions

You can also set the properties for group ratio constraints using `setGroupRatio`. For example, assume that you want the ratio of financial to nonfinancial companies in your portfolios to never go above 50%. Suppose that you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). Given a `PortfolioMAD` object `p`, use `setGroupRatio` to set the group constraints:

```
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = PortfolioMAD;
p = setGroupRatio(p, GA, GB, [], 0.5);
disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.UpperRatio)
```

```
6

1      1      1      0      0      0
0      0      0      1      1      1

0.5000
```

In this example, you would set the `LowerRatio` property to be empty (`[]`).

Suppose that you want to add another group ratio constraint to ensure that the weights in odd-numbered assets constitute at least 20% of the weights in nonfinancial assets your portfolio. You can set up augmented group ratio matrices and introduce infinite bounds for unconstrained group ratio bounds, or you can use the `addGroupRatio` function to build up group ratio constraints. For this example, create another group matrix for the second group constraint:

```
p = PortfolioMAD;
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = setGroupRatio(p, GA, GB, [], 0.5);

GA = [ true false true false true false ]; % odd-numbered companies
GB = [ false false false true true true ]; % nonfinancial companies
p = addGroupRatio(p, GA, GB, 0.2);

disp(p.NumAssets)
disp(p.GroupA)
disp(p.GroupB)
disp(p.LowerRatio)
disp(p.UpperRatio)
```

```
6

1      1      1      0      0      0
1      0      1      0      1      0

0      0      0      1      1      1
0      0      0      1      1      1

-Inf
0.2000

0.5000
Inf
```

Notice that `addGroupRatio` determines which bounds are unbounded so you only need to focus on the constraints you want to set.

The `PortfolioMAD` object, `setGroupRatio`, and `addGroupRatio` implement scalar expansion on either the `LowerRatio` or `UpperRatio` properties based on the dimension of the group matrices in `GroupA` and `GroupB` properties.

See Also

`PortfolioMAD` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Setting Default Constraints for Portfolio Weights Using `PortfolioMAD` Object” on page 6-50
- “Creating the `PortfolioMAD` Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for `PortfolioMAD` Object” on page 6-84
- “Estimate Efficient Frontiers for `PortfolioMAD` Object” on page 6-105
- “Asset Returns and Scenarios Using `PortfolioMAD` Object” on page 6-35

More About

- “`PortfolioMAD` Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “`PortfolioMAD` Object Workflow” on page 6-16

Working with Linear Equality Constraints Using PortfolioMAD Object

Linear equality constraints are optional linear constraints that impose systems of equalities on portfolio weights (see “Linear Equality Constraints” on page 6-9). Linear equality constraints have properties `AEquality`, for the equality constraint matrix, and `bEquality`, for the equality constraint vector.

Setting Linear Equality Constraints Using the PortfolioMAD Function

The properties for linear equality constraints are set using the `PortfolioMAD` object. Suppose that you have a portfolio of five assets and want to ensure that the first three assets are 50% of your portfolio. To set this constraint:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioMAD('AEquality', A, 'bEquality', b);
disp(p.NumAssets)
disp(p.AEquality)
disp(p.bEquality)
```

5

1 1 1 0 0

0.5000

Setting Linear Equality Constraints Using the setEquality and addEquality Functions

You can also set the properties for linear equality constraints using `setEquality`. Suppose that you have a portfolio of five assets and want to ensure that the first three assets are 50% of your portfolio. Given a `PortfolioMAD` object `p`, use `setEquality` to set the linear equality constraints:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioMAD;
p = setEquality(p, A, b);
disp(p.NumAssets)
disp(p.AEquality)
disp(p.bEquality)
```

5

1 1 1 0 0

0.5000

Suppose that you want to add another linear equality constraint to ensure that the last three assets also constitute 50% of your portfolio. You can set up an augmented system of linear equalities or use `addEquality` to build up linear equality constraints. For this example, create another system of equalities:

```
p = PortfolioMAD;
A = [ 1 1 1 0 0 ];      % first equality constraint
```

```
b = 0.5;
p = setEquality(p, A, b);

A = [ 0 0 1 1 1 ];    % second equality constraint
b = 0.5;
p = addEquality(p, A, b);

disp(p.NumAssets)
disp(p.AEquality)
disp(p.bEquality)

5

1      1      1      0      0
0      0      1      1      1

0.5000
0.5000
```

The PortfolioMAD object, setEquality, and addEquality implement scalar expansion on the bEquality property based on the dimension of the matrix in the AEquality property.

See Also

PortfolioMAD | setDefaultConstraints | setBounds | setBudget | setConditionalBudget | setGroups | setGroupRatio | setEquality | setInequality | setTurnover | setOneWayTurnover

Related Examples

- “Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50
- “Creating the PortfolioMAD Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with Linear Inequality Constraints Using PortfolioMAD Object

Linear inequality constraints are optional linear constraints that impose systems of inequalities on portfolio weights (see “Linear Inequality Constraints” on page 6-8). Linear inequality constraints have properties `AInequality` for the inequality constraint matrix, and `bInequality` for the inequality constraint vector.

Setting Linear Inequality Constraints Using the PortfolioMAD Function

The properties for linear inequality constraints are set using the `PortfolioMAD` object. Suppose that you have a portfolio of five assets and you want to ensure that the first three assets are no more than 50% of your portfolio. To set up these constraints:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioMAD('AInequality', A, 'bInequality', b);
disp(p.NumAssets)
disp(p.AInequality)
disp(p.bInequality)
```

5

1 1 1 0 0

0.5000

Setting Linear Inequality Constraints Using the setInequality and addInequality Functions

You can also set the properties for linear inequality constraints using `setInequality`. Suppose that you have a portfolio of five assets and you want to ensure that the first three assets constitute no more than 50% of your portfolio. Given a `PortfolioMAD` object `p`, use `setInequality` to set the linear inequality constraints:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioMAD;
p = setInequality(p, A, b);
disp(p.NumAssets)
disp(p.AInequality)
disp(p.bInequality)
```

5

1 1 1 0 0

0.5000

Suppose that you want to add another linear inequality constraint to ensure that the last three assets constitute at least 50% of your portfolio. You can set up an augmented system of linear inequalities or use the `addInequality` function to build up linear inequality constraints. For this example, create another system of inequalities:

```
p = PortfolioMAD;
A = [ 1 1 1 0 0 ];    % first inequality constraint
b = 0.5;
p = setInequality(p, A, b);

A = [ 0 0 -1 -1 -1 ];    % second inequality constraint
b = -0.5;
p = addInequality(p, A, b);

disp(p.NumAssets)
disp(p.AInequality)
disp(p.bInequality)

5

1      1      1      0      0
0      0     -1     -1     -1

0.5000
-0.5000
```

The PortfolioMAD object, setInequality, and addInequality implement scalar expansion on the bInequality property based on the dimension of the matrix in the AInequality property.

See Also

PortfolioMAD | setDefaultConstraints | setBounds | setBudget | setConditionalBudget | setGroups | setGroupRatio | setEquality | setInequality | setTurnover | setOneWayTurnover

Related Examples

- “Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50
- “Creating the PortfolioMAD Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects

When any one, or any combination of 'Conditional' BoundType, MinNumAssets, or MaxNumAssets constraints are active, the portfolio problem is formulated by adding NumAssets binary variables, where 0 indicates not invested, and 1 is invested. For example, to explain the 'Conditional' BoundType and MinNumAssets and MaxNumAssets constraints, assume that your portfolio has a universe of 100 assets that you want to invest:

- 'Conditional' BoundType (also known as semicontinuous constraints), set by `setBounds`, is often used in situations where you do not want to invest small values. A standard example is a portfolio optimization problem where many small allocations are not attractive because of transaction costs. Instead, you prefer fewer instruments in the portfolio with larger allocations. This situation can be handled using 'Conditional' BoundType constraints for a PortfolioMAD object.

For example, the weight you invest in each asset is either 0 or between $[0.01, 0.5]$. Generally, a semicontinuous variable x is a continuous variable between bounds $[lb, ub]$ that also can assume the value 0, where $lb > 0$, $lb \leq ub$. Applying this to portfolio optimization requires that very small or large positions should be avoided, that is values that fall in $(0, lb)$ or are more than ub .

- MinNumAssets and MaxNumAssets (also known as cardinality constraints), set by `setMinMaxNumAssets`, limit the number of assets in a PortfolioMAD object. For example, if you have 100 assets in your portfolio and you want the number of assets allocated in the portfolio to be from 40 through 60. Using MinNumAssets and MaxNumAssets you can limit the number of assets in the optimized portfolio, which allows you to limit transaction and operational costs or to create an index tracking portfolio.

Setting 'Conditional' BoundType Constraints Using the setBounds Function

Use `setBounds` with a 'conditional' BoundType to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i=1, \dots, \text{NumAssets}$:

```
p = PortfolioMAD;
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3)
```

```
p =
```

PortfolioMAD with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: []
Name: []
NumAssets: 3
AssetList: []
InitPort: []
AInequality: []
```

```
bInequality: []
  AEquality: []
  bEquality: []
  LowerBound: [3×1 double]
  UpperBound: [3×1 double]
LowerBudget: []
UpperBudget: []
GroupMatrix: []
  LowerGroup: []
  UpperGroup: []
    GroupA: []
    GroupB: []
  LowerRatio: []
  UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
  BoundType: [3×1 categorical]
```

Setting the Limits on the Number of Assets Invested Using the `setMinMaxNumAssets` Function

You can also set the `MinNumAssets` and `MaxNumAssets` properties to define a limit on the number of assets invested using `setMinMaxNumAssets`. For example, by setting `MinNumAssets=MaxNumAssets=2`, only two of the three assets are invested in the portfolio.

```
p = PortfolioMAD;
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3)
p = setMinMaxNumAssets(p, 2, 2)
```

`p =`

PortfolioMAD with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: []
  Name: []
  NumAssets: 3
  AssetList: []
  InitPort: []
AInequality: []
bInequality: []
  AEquality: []
  bEquality: []
  LowerBound: [3×1 double]
  UpperBound: [3×1 double]
LowerBudget: []
UpperBudget: []
GroupMatrix: []
  LowerGroup: []
  UpperGroup: []
```

```
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: 2
        MaxNumAssets: 2
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: [3×1 categorical]
```

See Also

`PortfolioMAD` | `setBounds` | `setMinMaxNumAssets` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50
- “Working with 'Simple' Bound Constraints Using PortfolioMAD Object” on page 6-54
- “Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148
- “Creating the PortfolioMAD Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with Average Turnover Constraints Using PortfolioMAD Object

The turnover constraint is an optional linear absolute value constraint (see “Average Turnover Constraints” on page 6-13) that enforces an upper bound on the average of purchases and sales. The turnover constraint can be set using the `PortfolioMAD` object or the `setTurnover` function. The turnover constraint depends on an initial or current portfolio, which is assumed to be zero if not set when the turnover constraint is set. The turnover constraint has properties `Turnover`, for the upper bound on average turnover, and `InitPort`, for the portfolio against which turnover is computed.

Setting Average Turnover Constraints Using the PortfolioMAD Function

The properties for the turnover constraints are set using the `PortfolioMAD` object. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and you want to ensure that average turnover is no more than 30%. To set this turnover constraint:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioMAD('Turnover', 0.3, 'InitPort', x0);
disp(p.NumAssets)
disp(p.Turnover)
disp(p.InitPort)
```

```
10
```

```
0.3000
```

```
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

Note if the `NumAssets` or `InitPort` properties are not set before or when the turnover constraint is set, various rules are applied to assign default values to these properties (see “Setting Up an Initial or Current Portfolio” on page 6-33).

Setting Average Turnover Constraints Using the setTurnover Function

You can also set properties for portfolio turnover using `setTurnover` to specify both the upper bound for average turnover and an initial portfolio. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and want to ensure that average turnover is no more than 30%. Given a `PortfolioMAD` object `p`, use `setTurnover` to set the turnover constraint with and without the initial portfolio being set previously:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioMAD('InitPort', x0);
p = setTurnover(p, 0.3);
```

```
disp(p.NumAssets)
```

```
disp(p.Turnover)
disp(p.InitPort)
```

```
10
```

```
0.3000
```

```
0.1200
```

```
0.0900
```

```
0.0800
```

```
0.0700
```

```
0.1000
```

```
0.1000
```

```
0.1500
```

```
0.1100
```

```
0.0800
```

```
0.1000
```

```
or
```

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
```

```
p = PortfolioMAD;
```

```
p = setTurnover(p, 0.3, x0);
```

```
disp(p.NumAssets)
```

```
disp(p.Turnover)
```

```
disp(p.InitPort)
```

```
10
```

```
0.3000
```

```
0.1200
```

```
0.0900
```

```
0.0800
```

```
0.0700
```

```
0.1000
```

```
0.1000
```

```
0.1500
```

```
0.1100
```

```
0.0800
```

```
0.1000
```

`setTurnover` implements scalar expansion on the argument for the initial portfolio. If the `NumAssets` property is already set in the `PortfolioMAD` object, a scalar argument for `InitPort` expands to have the same value across all dimensions. In addition, `setTurnover` lets you specify `NumAssets` as an optional argument. To clear turnover from your `PortfolioMAD` object, use the `PortfolioMAD` object or `setTurnover` with empty inputs for the properties to be cleared.

See Also

`PortfolioMAD` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50
- “Creating the PortfolioMAD Object” on page 6-22

- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with One-Way Turnover Constraints Using PortfolioMAD Object

One-way turnover constraints are optional constraints (see “One-way Turnover Constraints” on page 6-13) that enforce upper bounds on net purchases or net sales. One-way turnover constraints can be set using the PortfolioMAD object or the `setOneWayTurnover` function. One-way turnover constraints depend upon an initial or current portfolio, which is assumed to be zero if not set when the turnover constraints are set. One-way turnover constraints have properties `BuyTurnover`, for the upper bound on net purchases, `SellTurnover`, for the upper bound on net sales, and `InitPort`, for the portfolio against which turnover is computed.

Setting One-Way Turnover Constraints Using the PortfolioMAD Function

The Properties for the one-way turnover constraints are set using the PortfolioMAD object. Suppose that you have an initial portfolio with 10 assets in a variable `x0` and you want to ensure that turnover on purchases is no more than 30% and turnover on sales is no more than 20% of the initial portfolio. To set these turnover constraints:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioMAD('BuyTurnover', 0.3, 'SellTurnover', 0.2, 'InitPort', x0);
disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)
```

```
10
0.3000
0.2000
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

If the `NumAssets` or `InitPort` properties are not set before or when the turnover constraint is set, various rules are applied to assign default values to these properties (see “Setting Up an Initial or Current Portfolio” on page 6-33).

Setting Turnover Constraints Using the setOneWayTurnover Function

You can also set properties for portfolio turnover using `setOneWayTurnover` to specify to the upper bounds for turnover on purchases (`BuyTurnover`) and sales (`SellTurnover`) and an initial portfolio. Suppose that you have an initial portfolio of 10 assets in a variable `x0` and want to ensure that turnover on purchases is no more than 30% and that turnover on sales is no more than 20% of the initial portfolio. Given a PortfolioMAD object `p`, use `setOneWayTurnover` to set the turnover constraints with and without the initial portfolio being set previously:

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioMAD('InitPort', x0);
p = setOneWayTurnover(p, 0.3, 0.2);

disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)

or

x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioMAD;
p = setOneWayTurnover(p, 0.3, 0.2, x0);
disp(p.NumAssets)
disp(p.BuyTurnover)
disp(p.SellTurnover)
disp(p.InitPort)

10

0.3000

0.2000

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

`setOneWayTurnover` implements scalar expansion on the argument for the initial portfolio. If the `NumAssets` property is already set in the `PortfolioMAD` object, a scalar argument for `InitPort` expands to have the same value across all dimensions. In addition, `setOneWayTurnover` lets you specify `NumAssets` as an optional argument. To remove one-way turnover from your `PortfolioMAD` object, use the `PortfolioMAD` object or `setOneWayTurnover` with empty inputs for the properties to be cleared.

See Also

`PortfolioMAD` | `setDefaultConstraints` | `setBounds` | `setBudget` | `setConditionalBudget` | `setGroups` | `setGroupRatio` | `setEquality` | `setInequality` | `setTurnover` | `setOneWayTurnover`

Related Examples

- “Setting Default Constraints for Portfolio Weights Using `PortfolioMAD` Object” on page 6-50
- “Creating the `PortfolioMAD` Object” on page 6-22
- “Validate the MAD Portfolio Problem” on page 6-80
- “Estimate Efficient Portfolios Along the Entire Frontier for `PortfolioMAD` Object” on page 6-84
- “Estimate Efficient Frontiers for `PortfolioMAD` Object” on page 6-105
- “Asset Returns and Scenarios Using `PortfolioMAD` Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Validate the MAD Portfolio Problem

In this section...

“Validating a MAD Portfolio Set” on page 6-80

“Validating MAD Portfolios” on page 6-81

Sometimes, you may want to validate either your inputs to, or outputs from, a portfolio optimization problem. Although most error checking that occurs during the problem setup phase catches most difficulties with a portfolio optimization problem, the processes to validate MAD portfolio sets and portfolios are time consuming and are best done offline. So, the portfolio optimization tools have specialized functions to validate MAD portfolio sets and portfolios. For information on the workflow when using `PortfolioMAD` objects, see “PortfolioMAD Object Workflow” on page 6-16.

Validating a MAD Portfolio Set

Since it is necessary and sufficient that your MAD portfolio set must be a nonempty, closed, and bounded set to have a valid portfolio optimization problem, the `estimateBounds` function lets you examine your portfolio set to determine if it is nonempty and, if nonempty, whether it is bounded. Suppose that you have the following MAD portfolio set which is an empty set because the initial portfolio at θ is too far from a portfolio that satisfies the budget and turnover constraint:

```
p = PortfolioMAD('NumAssets', 3, 'Budget', 1);
p = setTurnover(p, 0.3, 0);
```

If a MAD portfolio set is empty, `estimateBounds` returns NaN bounds and sets the `isbounded` flag to []:

```
[lb, ub, isbounded] = estimateBounds(p)
```

```
lb =
```

```
NaN
NaN
NaN
```

```
ub =
```

```
NaN
NaN
NaN
```

```
isbounded =
```

```
[]
```

Suppose that you create an unbounded MAD portfolio set as follows:

```
p = PortfolioMAD('AInequality', [1 -1; 1 1 ], 'bInequality', 0);
[lb, ub, isbounded] = estimateBounds(p)
```

```
lb =
```

```
-Inf
-Inf
```

```

ub =

    1.0e+008 *
    -0.3712
         Inf

isbounded =

    0

```

In this case, `estimateBounds` returns (possibly infinite) bounds and sets the `isbounded` flag to `false`. The result shows which assets are unbounded so that you can apply bound constraints as necessary.

Finally, suppose that you created a `PortfolioMAD` object that is both nonempty and bounded. `estimateBounds` not only validates the set, but also obtains tighter bounds which are useful if you are concerned with the actual range of portfolio choices for individual assets in your portfolio:

```

p = PortfolioMAD;
p = setBudget(p, 1,1);
p = setBounds(p, [ -0.1; 0.2; 0.3; 0.2 ], [ 0.5; 0.3; 0.9; 0.8 ]);

[lb, ub, isbounded] = estimateBounds(p)

lb =

    -0.1000
     0.2000
     0.3000
     0.2000

ub =

     0.3000
     0.3000
     0.7000
     0.6000

isbounded =

     1

```

In this example, all but the second asset has tighter upper bounds than the input upper bound implies.

Validating MAD Portfolios

Given a MAD portfolio set specified in a `PortfolioMAD` object, you often want to check if specific portfolios are feasible with respect to the portfolio set. This can occur with, for example, initial portfolios and with portfolios obtained from other procedures. The `checkFeasibility` function determines whether a collection of portfolios is feasible. Suppose that you perform the following portfolio optimization and want to determine if the resultant efficient portfolios are feasible relative to a modified problem.

First, set up a problem in the `PortfolioMAD` object `p`, estimate efficient portfolios in `pwgt`, and then confirm that these portfolios are feasible relative to the initial problem:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;
```

```
AssetScenarios = mvnrnd(m, C, 20000);
```

```
p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
```

```
pwgt = estimateFrontier(p);
```

```
checkFeasibility(p, pwgt)
```

```
ans =
```

```
1      1      1      1      1      1      1      1      1      1
```

Next, set up a different portfolio problem that starts with the initial problem with an additional a turnover constraint and an equally weighted initial portfolio:

```
q = setTurnover(p, 0.3, 0.25);
checkFeasibility(q, pwgt)
```

```
ans =
```

```
0      0      1      1      1      0      0      0      0      0
```

In this case, only two of the 10 efficient portfolios from the initial problem are feasible relative to the new problem in `PortfolioMAD` object `q`. Solving the second problem using `checkFeasibility` demonstrates that the efficient portfolio for `PortfolioMAD` object `q` is feasible relative to the initial problem:

```
qwgt = estimateFrontier(q);
checkFeasibility(p, qwgt)
```

```
ans =
```

```
1      1      1      1      1      1      1      1      1      1
```

See Also

`PortfolioMAD` | `estimateBounds` | `checkFeasibility`

Related Examples

- “Creating the `PortfolioMAD` Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Portfolios Along the Entire Frontier for `PortfolioMAD` Object” on page 6-84
- “Estimate Efficient Frontiers for `PortfolioMAD` Object” on page 6-105
- “Asset Returns and Scenarios Using `PortfolioMAD` Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object

There are two ways to look at a portfolio optimization problem that depends on what you are trying to do. One goal is to estimate efficient portfolios and the other is to estimate efficient frontiers. The “Obtaining Portfolios Along the Entire Efficient Frontier” on page 6-85 example focuses on the former goal and “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105 focuses on the latter goal. For information on the workflow when using PortfolioMAD objects, see “PortfolioMAD Object Workflow” on page 6-16.

Obtaining Portfolios Along the Entire Efficient Frontier

The most basic way to obtain optimal portfolios is to obtain points over the entire range of the efficient frontier.

Given a portfolio optimization problem in a `PortfolioMAD` object, the `estimateFrontier` function computes efficient portfolios spaced evenly according to the return proxy from the minimum to maximum return efficient portfolios. The number of portfolios estimated is controlled by the hidden property `defaultNumPorts` which is set to 10. A different value for the number of portfolios estimated is specified as an input argument to `estimateFrontier`. This example shows the default number of efficient portfolios over the entire range of the efficient frontier.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
pwgt = estimateFrontier(p);
disp(pwgt)
```

0.8823	0.7156	0.5497	0.3854	0.2199	0.0530	0	0	0
0.0427	0.1427	0.2404	0.3378	0.4365	0.5342	0.4710	0.3338	0.1681
0.0392	0.0434	0.0490	0.0518	0.0553	0.0628	0.0404	0.0029	0
0.0358	0.0982	0.1608	0.2250	0.2883	0.3500	0.4886	0.6633	0.8319

If you want only four portfolios, you can use `estimateFrontier` with `NumPorts` specified as 4.

```
pwgt = estimateFrontier(p, 4);
disp(pwgt)
```

0.8823	0.3854	0	0
0.0427	0.3378	0.4710	0
0.0392	0.0518	0.0404	0
0.0358	0.2250	0.4886	1.0000

Starting from the initial portfolio, `estimateFrontier` also returns purchases and sales to get from your initial portfolio to each efficient portfolio on the efficient frontier. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontier(p);

display(pwgt)

pwgt = 4×10
```

0.8823	0.7156	0.5497	0.3854	0.2199	0.0530	0	0	0
--------	--------	--------	--------	--------	--------	---	---	---

0.0427	0.1427	0.2404	0.3378	0.4365	0.5342	0.4710	0.3338	0.1681	
0.0392	0.0434	0.0490	0.0518	0.0553	0.0628	0.0404	0.0029	0	
0.0358	0.0982	0.1608	0.2250	0.2883	0.3500	0.4886	0.6633	0.8319	1.0

```
display(pbuy)
```

```
pbuy = 4×10
```

0.5823	0.4156	0.2497	0.0854	0	0	0	0	0	
0	0	0	0.0378	0.1365	0.2342	0.1710	0.0338	0	
0	0	0	0	0	0	0	0	0	
0	0	0.0608	0.1250	0.1883	0.2500	0.3886	0.5633	0.7319	0.9

```
display(psell)
```

```
psell = 4×10
```

0	0	0	0	0.0801	0.2470	0.3000	0.3000	0.3000	0.3
0.2573	0.1573	0.0596	0	0	0	0	0	0.1319	0.3
0.1608	0.1566	0.1510	0.1482	0.1447	0.1372	0.1596	0.1971	0.2000	0.2
0.0642	0.0018	0	0	0	0	0	0	0	

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

`PortfolioMAD` | `estimateFrontier` | `estimateFrontierLimits` |
`estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` |
`estimatePortRisk` | `estimateFrontierByRisk` | `setSolver`

Related Examples

- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Obtaining Endpoints of the Efficient Frontier

Often when using a `PortfolioMAD` object, you might be interested in the endpoint portfolios for the efficient frontier. Suppose that you want to determine the range of returns from minimum to maximum to refine a search for a portfolio with a specific target return. Use the `estimateFrontierLimits` function to obtain the endpoint portfolios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
pwgt = estimateFrontierLimits(p);

disp(pwgt)

    0.8823         0
    0.0427         0
    0.0392         0
    0.0358    1.0000
```

Note that the endpoints of the efficient frontier depend upon the `Scenarios` in the `PortfolioMAD` object. If you change the `Scenarios`, you are likely to obtain different endpoints.

Starting from an initial portfolio, `estimateFrontierLimits` also returns purchases and sales to get from the initial portfolio to the endpoint portfolios on the efficient frontier. For example, given an initial portfolio in `pwgt0`, you can obtain purchases and sales:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontierLimits(p);

display(pwgt)
```

```
pwgt = 4x2
    0.8862      0
    0.0416      0
    0.0354      0
    0.0368    1.0000
```

```
display(pbuy)
```

```
pbuy = 4x2
    0.5862      0
     0          0
     0          0
     0    0.9000
```

```
display(psell)
```

```
psell = 4x2
     0    0.3000
    0.2584 0.3000
    0.1646 0.2000
    0.0632      0
```

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

[PortfolioMAD](#) | [estimateFrontier](#) | [estimateFrontierLimits](#) |
[estimateFrontierByReturn](#) | [estimatePortReturn](#) | [estimateFrontierByRisk](#) |
[estimatePortRisk](#) | [estimateFrontierByRisk](#) | [setSolver](#)

Related Examples

- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Mixed-Integer MAD Portfolio Optimization Problem

This example shows how to solve a MAD portfolio optimization problem with constraints in the number of selected assets or conditional (semicontinuous) bounds. To solve this problem, you can use a `PortfolioMAD` object along with different mixed integer nonlinear programming (MINLP) solvers.

MAD Portfolio

Load the returns data in `CAPMuniverse.mat`. Then, create a `PortfolioMAD` object with default constraints and a long-only portfolio whose weights sum to 1. For this example, you can define the feasible region of weights X as

$$X = \left\{ x \mid \sum_{i=1}^n x_i = 1, x_i \geq 0 \right\}.$$

```
% Load data
load CAPMuniverse.mat
```

```
% Create MAD portfolio with default constraints
p = PortfolioMAD;
p = simulateNormalScenariosByData(p,Data(:,1:12),1000,missingdata=true);
p = setDefaultConstraints(p);
```

Include binary variables for this scenario by setting conditional (semicontinuous) bounds. Conditional bounds are those such that $x_i = 0$ or $x_i \geq \ell_i$. In this example, $\ell_i = 0.1$ for all assets.

```
% Set conditional bounds
condLB = 0.1;
condUB = 0.5;
p = setBounds(p,condLB,condUB,BoundType="conditional");
```

Use `estimateFrontier` to estimate a set of portfolios on the efficient frontier. The efficient frontier is a curve that shows the trade-off between the return and risk achieved by Pareto-optimal portfolios. For a given return level, the portfolio on the efficient frontier is the one that minimizes the risk while maintaining the desired return. Conversely, for a given risk level, the portfolio on the efficient frontier is the one that maximizes return while maintaining the desired risk level.

```
% Compute efficient frontier
p = setSolverMINLP(p,'TrustRegionCP',DeltaLowerBound=condLB);
pwgt = estimateFrontier(p)
```

```
pwgt = 12x10
```

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0.1000	0.1283	0.1973	0.2316	0.3102	0.4375	0.5
0.1709	0.2574	0.3502	0.3371	0.3992	0.4272	0.4812	0.5000	0.4625	0.5
0	0.1000	0.1000	0.1000	0.1000	0	0	0	0	0
0.5000	0.3436	0.2380	0.1933	0.1000	0.1000	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0.3291	0.2990	0.3117	0.2696	0.2726	0.2755	0.2872	0.1898	0.1000	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

```

:
% Compute risk and returns of the portfolios on the efficient frontier
rsk = estimatePortRisk(p,pwgt);
ret = estimatePortReturn(p,pwgt);

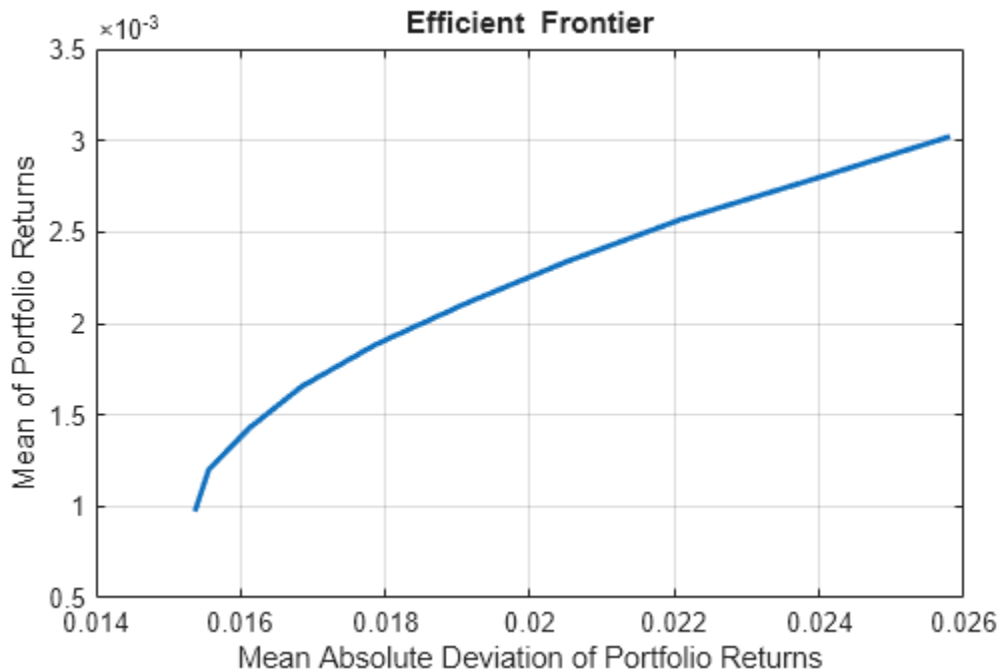
```

Plot the weights from the frontier estimation with `plotFrontier`. The resulting curve is piece-wise concave with possible vertical jumps (discontinuities) between the concave intervals.

```

% Plot efficient frontier
plotFrontier(p,pwgt)

```



Changing MINLP Solvers

In the previous section, you use the default solver for `estimateFrontier`. However, you can solve mixed-integer portfolio problems using any of these three algorithms supported by `setSolverMINLP`: `OuterApproximation`, `ExtendedCP`, and `TrustRegionCP`. Furthermore, the `OuterApproximation` algorithm accepts an additional name-value argument (`ExtendedFormulation`) for `Portfolio` problems, which reformulates problems with quadratic functions to work in an extended space that usually decreases the computation time. All algorithms, including the extended formulation variation of the `OuterApproximation` algorithm, return the same values within numerical accuracy. The available solvers are:

- `OuterApproximation` — The default algorithm, which is robust and usually faster than `ExtendedCP`
- `OuterApproximation` with `ExtendedFormulation` set to `true` — A robust algorithm that is usually faster than other algorithms, but supported only for `Portfolio` object problems
- `ExtendedCP` — The most robust solver, but usually the slowest
- `TrustRegionCP` — The fastest algorithm, but one that is less robust and may provide suboptimal solutions

For more information on solvers for mixed-integer portfolio problems, see “Choose MINLP Solvers for Portfolio Problems” on page 4-140.

To change the MINLP solvers, use `setSolverMINLP`.

```
% Select 'TrustRegionCP' as solver
p_OA = setSolverMINLP(p, 'OuterApproximation');
pwgt_OA = estimateFrontier(p_OA);
rskOA = estimatePortRisk(p, pwgt_OA);
retOA = estimatePortReturn(p, pwgt_OA);

% Select 'ExtendedCP' as solver using 'midway' cuts as 'CutGeneration'
p_ECP = setSolverMINLP(p, 'ExtendedCP', CutGeneration="midway");
pwgt_ECP = estimateFrontier(p_ECP);
rskECP = estimatePortRisk(p, pwgt_ECP);
retECP = estimatePortReturn(p, pwgt_ECP);
```

Compare the returns and risks obtained by the portfolios on the efficient frontier from the different solvers. They are all the same within a numerical accuracy where the absolute difference $\leq 10^{-4}$.

```
retTable = table(retOA, ret, retECP, 'VariableNames', {'OA', 'TR', 'ECP'})
```

```
retTable=10×3 table
      OA      TR      ECP
      _____
0.00097434  0.00097297  0.0009746
0.0012019  0.0012006  0.0012021
0.0014294  0.0014283  0.0014296
0.0016569  0.001656  0.0016571
0.0018844  0.0018837  0.0018846
0.0021119  0.0021113  0.0021121
0.0023395  0.002339  0.0023395
0.002567  0.0025667  0.002567
0.0027945  0.0027944  0.0027945
0.003022  0.003022  0.003022
```

```
rskTable = table(rskOA, rsk, rskECP, 'VariableNames', {'OA', 'TR', 'ECP'})
```

```
rskTable=10×3 table
      OA      TR      ECP
      _____
0.015368  0.015368  0.015368
0.01556  0.015558  0.01556
0.016123  0.01612  0.016124
0.016849  0.016846  0.01685
0.017864  0.01786  0.017864
0.019127  0.019123  0.019127
0.020516  0.020513  0.020517
0.022074  0.022072  0.022074
0.023968  0.023965  0.023968
0.025807  0.025807  0.025807
```

```
% Compare risks from the different OuterApproximation formulations
norm(rskTable.OA-rskTable.TR, Inf) <= 1e-4
```

```
ans = logical  
    1
```

See Also

`PortfolioMAD` | `estimateFrontier` | `estimateFrontierLimits` |
`estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` |
`estimatePortRisk` | `estimateFrontierByRisk`

Related Examples

- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16
- “Choose MINLP Solvers for Portfolio Problems” on page 4-140

Obtaining Efficient Portfolios for Target Returns

This example shows how to obtain efficient portfolios that have targeted portfolio returns using the `estimateFrontierByReturn` function.

The `estimateFrontierByReturn` function accepts one or more target portfolio returns and obtains efficient portfolios with the specified returns. For example, assume that you have a universe of four assets where you want to obtain efficient portfolios with target portfolio returns of 7%, 10%, and 12%:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontierByReturn(p, [0.07, 0.10, .12]);
display(pwgt)

pwgt = 4×3

    0.7489    0.3770    0.1264
    0.1202    0.3235    0.4641
    0.0466    0.0771    0.0963
    0.0843    0.2224    0.3132
```

Sometimes, you can request a return for which no efficient portfolio exists. Based on the previous code, suppose that you want a portfolio with a 4% return (which is the return of the first asset). A portfolio that is fully invested in the first asset, however, is inefficient. `estimateFrontierByReturn` warns if your target returns are outside the range of efficient portfolio returns and replaces it with the endpoint portfolio of the efficient frontier closest to your target return. The best way to avoid this situation is to bracket your target portfolio returns with `estimateFrontierLimits` and `estimatePortReturn` (see “Obtaining Endpoints of the Efficient Frontier” on page 6-87 and “Obtaining MAD Portfolio Risks and Returns” on page 6-105).

```
pret = estimatePortReturn(p, p.estimateFrontierLimits);
display(pret)

pret = 2×1

    0.0594
    0.1790
```

This result indicates that efficient portfolios have returns that range from 6.5% to 17.8%. Note, your results for this code may be different due to the random generation of scenarios.

If you have an initial portfolio, `estimateFrontierByReturn` also returns purchases and sales to get from your initial portfolio to the target portfolios on the efficient frontier. For example, given an initial portfolio in `pwgt0`, to obtain purchases and sales with target returns of 7%, 10%, and 12%:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];  
p = setInitPort(p, pwgt0);  
[pwgt, pbuy, psell] = estimateFrontierByReturn(p, [0.07, 0.10, .12]);
```

```
display(pwgt)
```

```
pwgt = 4×3
```

0.7489	0.3770	0.1264
0.1202	0.3235	0.4641
0.0466	0.0771	0.0963
0.0843	0.2224	0.3132

```
display(pbuy)
```

```
pbuy = 4×3
```

0.4489	0.0770	0
0	0.0235	0.1641
0	0	0
0	0.1224	0.2132

```
display(psell)
```

```
psell = 4×3
```

0	0	0.1736
0.1798	0	0
0.1534	0.1229	0.1037
0.0157	0	0

If you do not have an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

`PortfolioMAD` | `estimateFrontier` | `estimateFrontierLimits` |
`estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` |
`estimatePortRisk` | `estimateFrontierByRisk` | `setSolver`

Related Examples

- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Obtaining Efficient Portfolios for Target Risks

This example shows how to obtain efficient portfolios that have targeted portfolio risks using the `estimateFrontierByRisk` function.

The `estimateFrontierByRisk` function accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks. Suppose that you have a universe of four assets where you want to obtain efficient portfolios with target portfolio risks of 12%, 14%, and 16%.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontierByRisk(p, [0.12, 0.14, 0.16]);

display(pwgt)

pwgt = 4×3

    0.1926    0.0402         0
    0.4288    0.5146    0.4455
    0.0903    0.1020    0.1006
    0.2883    0.3432    0.4539
```

Sometimes, you can request a risk for which no efficient portfolio exists. Based on the previous code, suppose that you want a portfolio with 6% risk (individual assets in this universe have risks ranging from 7% to 42.5%). It turns out that a portfolio with 6% risk cannot be formed with these four assets. `estimateFrontierByRisk` warns if your target risks are outside the range of efficient portfolio risks and replaces it with the endpoint of the efficient frontier closest to your target risk. The best way to avoid this situation is to bracket your target portfolio risks with `estimateFrontierLimits` and `estimatePortRisk` (see “Obtaining Endpoints of the Efficient Frontier” on page 6-87 and “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105).

```
prsk = estimatePortRisk(p, p.estimateFrontierLimits);
display(prsk)

prsk = 2×1

    0.0607
    0.2756
```

This result indicates that efficient portfolios have risks that range from 7% to 42.5%. Note, your results for this code may be different due to the random generation of scenarios.

Starting with an initial portfolio, `estimateFrontierByRisk` also returns purchases and sales to get from your initial portfolio to the target portfolios on the efficient frontier. For example, given an initial

portfolio in `pwgt0`, you can obtain purchases and sales from the example with target risks of 12%, 14%, and 16%:

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontierByRisk(p, [0.12, 0.14, 0.16]);
```

```
display(pwgt)
```

```
pwgt = 4×3
```

```
    0.1926    0.0402         0
    0.4288    0.5146    0.4455
    0.0903    0.1020    0.1006
    0.2883    0.3432    0.4539
```

```
display(pbuy)
```

```
pbuy = 4×3
```

```
         0         0         0
    0.1288    0.2146    0.1455
         0         0         0
    0.1883    0.2432    0.3539
```

```
display(psell)
```

```
psell = 4×3
```

```
    0.1074    0.2598    0.3000
         0         0         0
    0.1097    0.0980    0.0994
         0         0         0
```

If you do not specify an initial portfolio, the purchase and sale weights assume that your initial portfolio is 0.

See Also

`PortfolioMAD` | `estimateFrontier` | `estimateFrontierLimits` |
`estimateFrontierByReturn` | `estimatePortReturn` | `estimateFrontierByRisk` |
`estimatePortRisk` | `estimateFrontierByRisk` | `setSolver`

Related Examples

- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Choosing and Controlling the Solver for PortfolioMAD Optimizations

When solving portfolio optimizations for a `PortfolioMAD` object, you are solving nonlinear optimization problems with either nonlinear objective or nonlinear constraints. You can use 'TrustRegionCP' (default) or 'ExtendedCP' solvers that implement Kelley's cutting plane method (see Kelley [45] at "Portfolio Optimization" on page A-5). Alternatively, you can use `fmincon` and all variations of `fmincon` from Optimization Toolbox are supported. When using `fmincon` as the solverType, 'sqp' is the default algorithm for `fmincon`.

Using 'TrustRegionCP' and 'ExtendedCP' SolverTypes

The 'TrustRegionCP' and 'ExtendedCP' solvers have options to control the number iterations and stopping tolerances. Moreover, these solvers use `linprog` as the primary solver, and all `linprog` options are supported using `optimoptions` structures. All these options are set using `setSolver`.

For example, you can use `setSolver` to increase the number of iterations for 'TrustRegionCP':

```
p = PortfolioMAD;
p = setSolver(p, 'TrustRegionCP', 'MaxIterations', 2000);
display(p.solverType)
display(p.solverOptions)
```

```
trustregioncp
      MaxIterations: 2000
      AbsoluteGapTolerance: 1.0000e-07
      RelativeGapTolerance: 1.0000e-05
      NonlinearScalingFactor: 1000
      ObjectiveScalingFactor: 1000
      MainSolverOptions: [1x1 optim.options.Linprog]
          Display: 'off'
      CutGeneration: 'basic'
      MaxIterationsInactiveCut: 30
      ActiveCutTolerance: 1.0000e-07
      ShrinkRatio: 0.7500
      TrustRegionStartIteration: 2
      DeltaLimit: 1
```

To change the primary solver algorithm to 'interior-point', with no display, use `setSolver` to modify 'MainSolverOptions':

```
p = PortfolioMAD;
options = optimoptions('linprog','Algorithm','interior-point','Display','off');
p = setSolver(p,'TrustRegionCP','MainSolverOptions',options);
display(p.solverType)
display(p.solverOptions)
display(p.solverOptions.MainSolverOptions.Algorithm)
display(p.solverOptions.MainSolverOptions.Display)
```

```
trustregioncp
      MaxIterations: 1000
      AbsoluteGapTolerance: 1.0000e-07
      RelativeGapTolerance: 1.0000e-05
      NonlinearScalingFactor: 1000
      ObjectiveScalingFactor: 1000
      MainSolverOptions: [1x1 optim.options.Linprog]
```

```
            Display: 'off'
            CutGeneration: 'basic'
MaxIterationsInactiveCut: 30
            ActiveCutTolerance: 1.0000e-07
            ShrinkRatio: 0.7500
TrustRegionStartIteration: 2
            DeltaLimit: 1
```

```
interior-point
off
```

Using 'fmincon' SolverType

Unlike Optimization Toolbox which uses the 'interior-point' algorithm as the default algorithm for `fmincon`, the portfolio optimization for a `PortfolioMAD` object uses the 'sqp' algorithm as the default. For details about `fmincon` and constrained nonlinear optimization algorithms and options, see “Constrained Nonlinear Optimization Algorithms”.

To modify `fmincon` options for MAD portfolio optimizations, use `setSolver` to set the hidden properties `solverType` and `solverOptions` to specify and control the solver. Since these solver properties are hidden, you cannot set them using the `PortfolioMAD` object. The default for the `fmincon` solver is the 'sqp' algorithm and no displayed output, so you do not need to use `setSolver` to specify the 'sqp' algorithm for `fmincon`.

```
p = PortfolioMAD;
p = setSolver(p, 'fmincon');
display(p.solverOptions.Algorithm)
display(p.solverOptions.Display)
```

```
sqp
off
```

If you want to specify additional options associated with the `fmincon` solver, `setSolver` accepts these options as name-value pair arguments. For example, if you want to use `fmincon` with the 'active-set' algorithm and with displayed output, use `setSolver` with:

```
p = PortfolioMAD;
p = setSolver(p, 'fmincon', 'Algorithm', 'active-set', 'Display', 'final');
display(p.solverOptions)
```

`fmincon` options:

```
Options used by current Algorithm ('active-set'):
(Other available algorithms: 'interior-point', 'sqp', 'sqp-legacy', 'trust-region-reflective')
```

Set properties:

```
Algorithm: 'active-set'
Display: 'final'
```

Default properties:

```
ConstraintTolerance: 1.0000e-06
FiniteDifferenceStepSize: 'sqrt(eps)'
FiniteDifferenceType: 'forward'
FunctionTolerance: 1.0000e-06
MaxFunctionEvaluations: '100*numberOfVariables'
MaxIterations: 400
OptimalityTolerance: 1.0000e-06
OutputFcn: []
PlotFcn: []
```



```
SpecifyConstraintGradient: 0
SpecifyObjectiveGradient: 0
StepTolerance: 1.0000e-06
TypicalX: 'ones(numberOfVariables,1)'
UseParallel: 0
```

Alternatively, the `setSolver` function accepts an `optimoptions` object as the second argument. For example, you can change the algorithm to 'active-set' with no displayed output as follows:

```
p = PortfolioMAD;
options = optimoptions('fmincon', 'Algorithm', 'active-set', 'Display', 'off');
p = setSolver(p, 'fmincon', options);
display(p.solverOptions.Algorithm)
display(p.solverOptions.Display)

active-set
off
```

Using the Mixed Integer Nonlinear Programming (MINLP) Solver

The mixed integer nonlinear programming (MINLP) solver, configured using `setSolverMINLP`, enables you to specify associated solver options for portfolio optimization for a `PortfolioMAD` object. The MINLP solver is used when any one, or any combination of 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active, the portfolio problem is formulated by adding `NumAssets` binary variables, where 0 indicates not invested, and 1 is invested. For more information on using 'Conditional' `BoundType`, see `setBounds`. For more information on specifying `MinNumAssets` and `MaxNumAssets`, see `setMinMaxNumAssets`.

When using the estimate functions with a `PortfolioMAD` object where 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active, the mixed integer nonlinear programming (MINLP) solver is automatically used.

Solver Guidelines for PortfolioMAD Objects

The following table provides guidelines for using `setSolver` and `setSolverMINLP`.

PortfolioMAD Problem	PortfolioMAD Function	Type of Optimization Problem	Main Solver	Helper Solver
PortfolioMAD without active 'Conditional' <code>BoundType</code> , <code>MinNumAssets</code> , <code>MaxNumAssets</code> , or conditional budget constraints	<code>estimateFrontierByRisk</code>	Optimizing a portfolio for a certain risk level introduces a nonlinear constraint. Therefore, this problem has a linear objective with linear and nonlinear constraints.	'TrustRegionCP', 'ExtendedCP', or 'fmincon' using <code>setSolver</code>	'linprog' using <code>setSolver</code>

PortfolioMAD Problem	PortfolioMAD Function	Type of Optimization Problem	Main Solver	Helper Solver
PortfolioMAD without active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierByReturn	Nonlinear objective with linear constraints	'TrustRegionCP', 'ExtendedCP', or 'fmincon' using setSolver	'linprog' using setSolver
PortfolioMAD without active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierLimits	Nonlinear or linear objective with linear constraints	For 'min': nonlinear objective, 'TrustRegionCP', 'ExtendedCP', or 'fmincon' using setSolver For 'max': linear objective, 'linprog' using setSolver	Not applicable
PortfolioMAD with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierByRisk	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'fmincon' is used when the estimate functions reduce the problem into NLP. This solver is configured through setSolver.

PortfolioMAD Problem	PortfolioMAD Function	Type of Optimization Problem	Main Solver	Helper Solver
PortfolioMAD with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierByReturn	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'fmincon' is used when the estimate functions reduce the problem into NLP. This solver is configured through setSolver
PortfolioMAD with active 'Conditional' BoundType, MinNumAssets, MaxNumAssets, or conditional budget constraints	estimateFrontierLimits	The problem is formulated by introducing NumAssets binary variables to indicate whether the corresponding asset is invested or not. Therefore, it requires a mixed integer nonlinear programming solver. Three types of MINLP solvers are offered, see setSolverMINLP.	Mixed integer nonlinear programming solver (MINLP) using setSolverMINLP	'fmincon' is used when the estimate functions reduce the problem into NLP. This solver is configured through setSolver

See Also

PortfolioMAD | estimateFrontier | estimateFrontierLimits |
 estimateFrontierByReturn | estimatePortReturn | estimateFrontierByRisk |
 estimatePortRisk | estimateFrontierByRisk | setSolver | setSolverMINLP

Related Examples

- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

- “Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190
- “Mixed-Integer MAD Portfolio Optimization Problem” on page 6-89

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16
- “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81

Estimate Efficient Frontiers for PortfolioMAD Object

In this section...

“Obtaining MAD Portfolio Risks and Returns” on page 6-105

“Obtaining the PortfolioMAD Standard Deviation” on page 6-106

Whereas “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84 focused on estimation of efficient portfolios, this section focuses on the estimation of efficient frontiers. For information on the workflow when using PortfolioMAD objects, see “PortfolioMAD Object Workflow” on page 6-16.

Obtaining MAD Portfolio Risks and Returns

This example shows how to use the functions `estimatePortReturn` and `estimatePortRisk` with a PortfolioMAD object to provide estimates for the return (or return proxy), risk (or the risk proxy). These functions have the same input syntax but have different combinations of outputs.

Suppose that you have this following portfolio optimization problem that gave you a collection of portfolios along the efficient frontier in `pwgt`:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;
```

```
AssetScenarios = mvnrnd(m, C, 20000);
```

```
p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
```

```
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
```

```
p = setInitPort(p, pwgt0);
pwgt = estimateFrontier(p)
```

```
pwgt = 4×10
```

0.8823	0.7156	0.5497	0.3854	0.2199	0.0530	0	0	0
0.0427	0.1427	0.2404	0.3378	0.4365	0.5342	0.4710	0.3338	0.1681
0.0392	0.0434	0.0490	0.0518	0.0553	0.0628	0.0404	0.0029	0
0.0358	0.0982	0.1608	0.2250	0.2883	0.3500	0.4886	0.6633	0.8319

Given `pwgt0` and `pwgt`, use the portfolio risk and return estimation functions to obtain risks and returns for your initial portfolio and the portfolios on the efficient frontier:

```
prsk0 = estimatePortRisk(p, pwgt0);
pret0 = estimatePortReturn(p, pwgt0);
prsk = estimatePortRisk(p, pwgt);
pret = estimatePortReturn(p, pwgt);
```

You obtain these risks and returns:

```
display(prsk0)
```

```
prsk0 =  
0.0252
```

```
display(pret0)
```

```
pret0 =  
0.0071
```

```
display(prsk)
```

```
prsk = 10×1
```

```
0.0175  
0.0190  
0.0228  
0.0279  
0.0338  
0.0401  
0.0473  
0.0568  
0.0678  
0.0796
```

```
display(pret)
```

```
pret = 10×1
```

```
0.0049  
0.0060  
0.0071  
0.0082  
0.0093  
0.0104  
0.0115  
0.0125  
0.0136  
0.0147
```

Obtaining the PortfolioMAD Standard Deviation

This example shows how to compute standard deviations of portfolio returns for a `PortfolioMAD` object with the function `estimatePortStd`. These functions work with any `PortfolioMAD` portfolios, not necessarily efficient portfolios.

For example, the following code obtains five portfolios (`pwgt`) on the efficient frontier and also has an initial portfolio in `pwgt0`. Various portfolio statistics are computed that include the return, risk, and standard deviation. The listed estimates are for the initial portfolio in the first row followed by estimates for each of the five efficient portfolios in subsequent rows.

```
m = [ 0.0042; 0.0083; 0.01; 0.15 ];  
C = [ 0.005333 0.00034 0.00016 0;
```

```

0.00034 0.002408 0.0017 0.000992;
0.00016 0.0017 0.0048 0.0028;
0 0.000992 0.0028 0.010208 ];

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioMAD('initport', pwgt0);
p = simulateNormalScenariosByMoments(p, m, C, 20000);
p = setDefaultConstraints(p);

pwgt = estimateFrontier(p, 5);

pret = estimatePortReturn(p, [pwgt0, pwgt]);
prsk = estimatePortRisk(p, [pwgt0, pwgt]);
pstd = estimatePortStd(p, [pwgt0, pwgt]);

[pret, prsk, pstd]

ans = 6×3

    0.0206    0.0300    0.0377
    0.0196    0.0321    0.0403
    0.0521    0.0365    0.0459
    0.0847    0.0480    0.0603
    0.1172    0.0629    0.0790
    0.1497    0.0796    0.0999

```

See Also

PortfolioMAD | estimatePortReturn | plotFrontier | estimatePortStd

Related Examples

- “Plotting the Efficient Frontier for a PortfolioMAD Object” on page 6-108
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35
- “Postprocessing Results to Set Up Tradable Portfolios” on page 6-113

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Plotting the Efficient Frontier for a PortfolioMAD Object

This example shows how to use the `plotFrontier` function creates a plot of the efficient frontier for a given portfolio optimization problem. This function accepts several types of inputs and generates a plot with an optional possibility to output the estimates for portfolio risks and returns along the efficient frontier. `plotFrontier` has four different ways that it can be used. In addition to a plot of the efficient frontier, if you have an initial portfolio in the `InitPort` property, `plotFrontier` also displays the return versus risk of the initial portfolio on the same plot. If you have a well-posed portfolio optimization problem set up in a `PortfolioMAD` object and you use `plotFrontier`, you get a plot of the efficient frontier with the default number of portfolios on the frontier (the default number is currently 10 and is maintained in the hidden property `defaultNumPorts`).

Create Efficient Frontier Plot

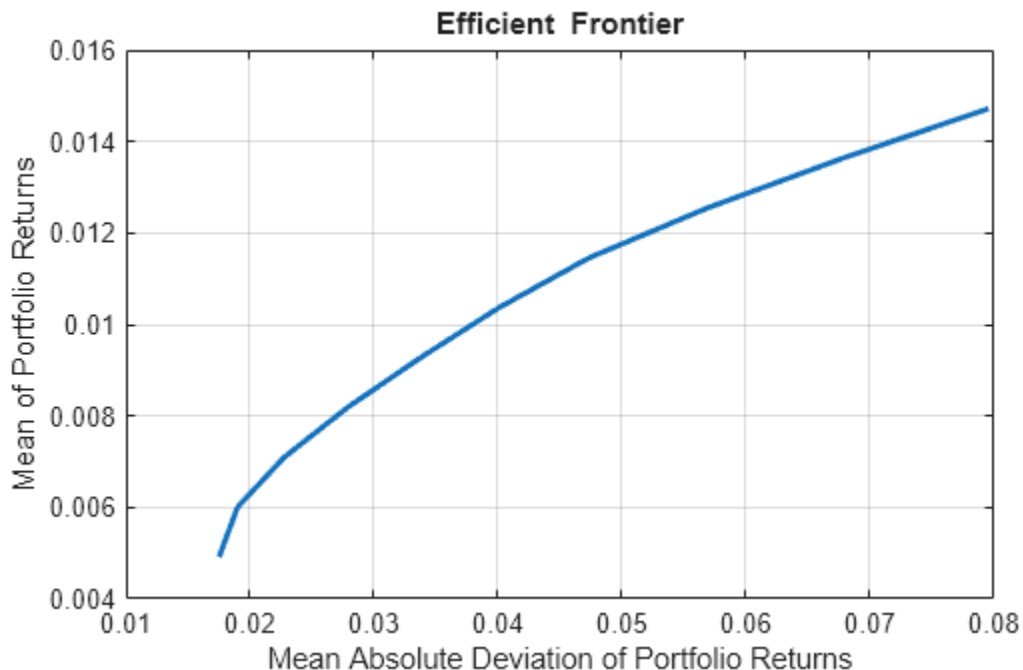
To plot the efficient frontier for a `PortfolioMAD` object, use `plotFrontier`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

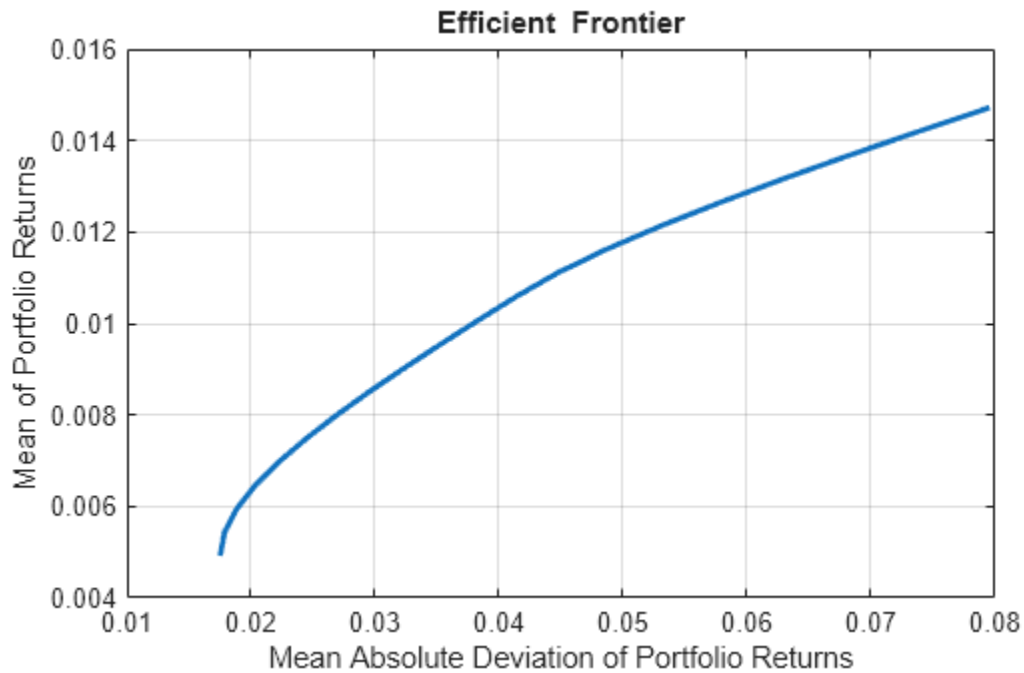
plotFrontier(p)
```



The `Name` property appears as the title of the efficient frontier plot if you set it in the `PortfolioMAD` object. Without an explicit name, the title on the plot would be "Efficient Frontier." If you want to obtain a specific number of portfolios along the efficient frontier, use `plotFrontier` with the number of portfolios that you want.

Suppose that you have this `PortfolioMAD` object and you want to plot 20 portfolios along the efficient frontier and to obtain 20 risk and return values for each portfolio:

```
[prsk, pret] = plotFrontier(p, 20);
```



```
display([pret, prsk])
```

0.0049	0.0175
0.0054	0.0179
0.0060	0.0189
0.0065	0.0204
0.0070	0.0223
0.0075	0.0245
0.0080	0.0270
0.0085	0.0297
0.0090	0.0325
0.0096	0.0354
0.0101	0.0384
0.0106	0.0414
0.0111	0.0447
0.0116	0.0487
0.0121	0.0531
0.0127	0.0579
0.0132	0.0630
0.0137	0.0684
0.0142	0.0739
0.0147	0.0796

Plotting Existing Efficient Portfolios

If you already have efficient portfolios from any of the "estimateFrontier" functions (see "Estimate Efficient Frontiers for PortfolioMAD Object" on page 6-105), pass them into `plotFrontier` directly to plot the efficient frontier

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

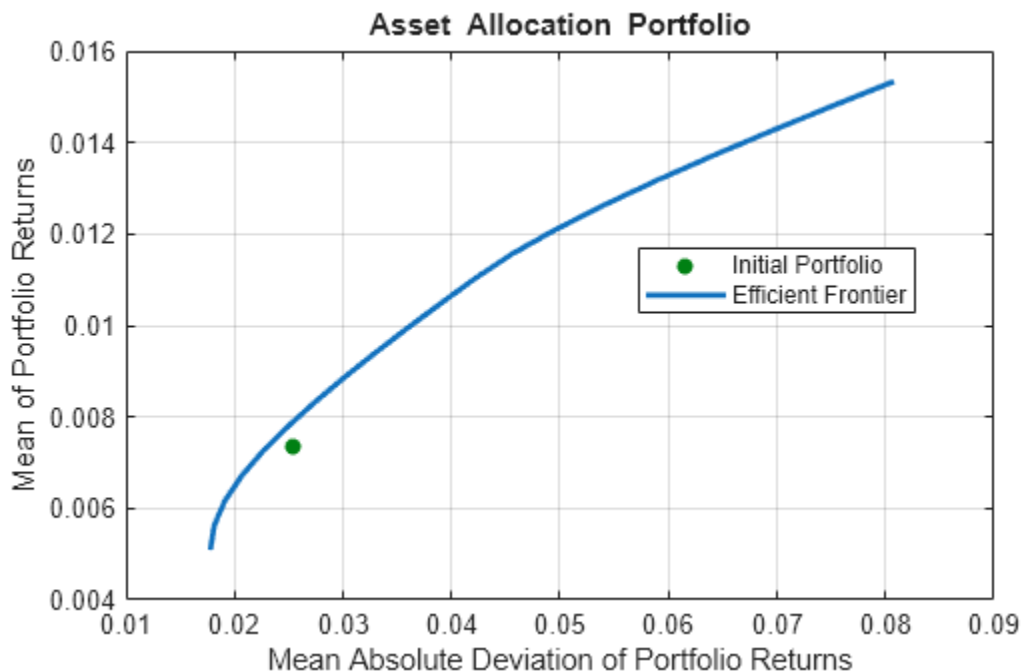
AssetScenarios = mvnrnd(m, C, 20000);

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioMAD('Name', 'Asset Allocation Portfolio', 'InitPort', pwgt0);

p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontier(p, 20);
plotFrontier(p, pwgt)
```



Plotting Existing Efficient Portfolio Risks and Returns

If you already have efficient portfolio risks and returns, you can use the interface to `plotFrontier` to pass them into `plotFrontier` to obtain a plot of the efficient frontier:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
```

```

0.00408 0.0289 0.0204 0.0119;
0.00192 0.0204 0.0576 0.0336;
0 0.0119 0.0336 0.1225 ];

AssetScenarios = mvnrnd(m, C, 20000);

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioMAD('Name', 'Asset Allocation Portfolio', 'InitPort', pwgt0);

p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontier(p);

pret= estimatePortReturn(p, pwgt)

pret = 10×1

    0.0592
    0.0729
    0.0865
    0.1001
    0.1137
    0.1274
    0.1410
    0.1546
    0.1682
    0.1819

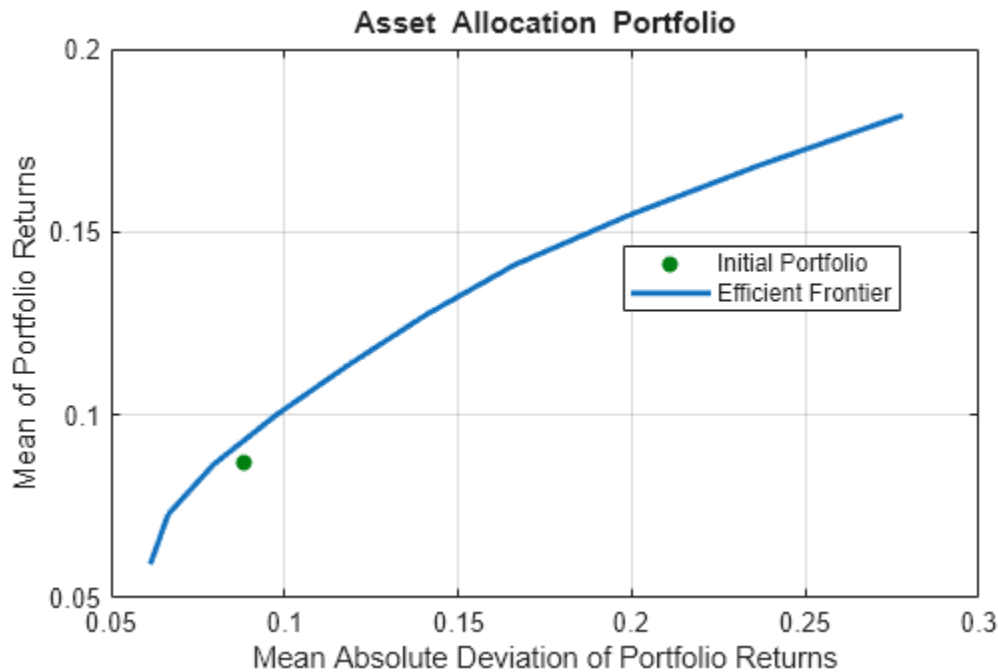
prsk = estimatePortRisk(p, pwgt)

prsk = 10×1

    0.0614
    0.0665
    0.0796
    0.0978
    0.1185
    0.1407
    0.1662
    0.1992
    0.2367
    0.2781

plotFrontier(p, prsk, pret)

```



See Also

PortfolioMAD | estimatePortReturn | plotFrontier | estimatePortStd

Related Examples

- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35
- “Postprocessing Results to Set Up Tradable Portfolios” on page 6-113

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Postprocessing Results to Set Up Tradable Portfolios

This example shows how to use your results for efficient portfolios or estimates for expected portfolio risks and returns to set up trades to move toward an efficient portfolio. For information on the workflow when using PortfolioMAD objects, see "PortfolioMAD Object Workflow" on page 6-16.

Suppose that you set up a portfolio optimization problem and obtained portfolios on the efficient frontier. Use the `dataset` object to form a blotter that lists your portfolios with the names for each asset. For example, suppose that you want to obtain five portfolios along the efficient frontier. You can set up a blotter with weights multiplied by 100 to view the allocations for each portfolio:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

p = PortfolioMAD;
p = setAssetList(p, 'Bonds', 'Large-Cap Equities', 'Small-Cap Equities', 'Emerging Equities');
p = setInitPort(p, pwgt0);
p = simulateNormalScenariosByMoments(p, m, C, 20000);
p = setDefaultConstraints(p);

pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end

Blotter = dataset([100*pwgt], pnames, 'obsnames', p.AssetList);
display(Blotter)
```

```
Blotter =
```

	Port1	Port2	Port3	Port4	Port5
Bonds	88.232	50.89	13.581	0	0
Large-Cap Equities	4.2697	25.147	45.859	30.299	0
Small-Cap Equities	3.9151	6.7146	9.6576	9.5949	0
Emerging Equities	3.5828	17.249	30.903	60.106	100

This result indicates that you would invest primarily in bonds at the minimum-risk/minimum-return end of the efficient frontier (Port1), and that you would invest completely in emerging equity at the maximum-risk/maximum-return end of the efficient frontier (Port5). You can also select a particular efficient portfolio, for example, suppose that you want a portfolio with 15% risk and you add purchase and sale weights outputs obtained from the "estimateFrontier" functions to set up a trade blotter:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
```

```
p = PortfolioMAD;
p = setAssetList(p, 'Bonds', 'Large-Cap Equities', 'Small-Cap Equities', 'Emerging Equities');

p = setInitPort(p, pwgt0);
p = simulateNormalScenariosByMoments(p, m, C, 20000);
p = p.setDefaultConstraints;

[pwgt, pbuy, psell] = estimateFrontierByRisk(p, 0.15);

Blotter = dataset([100*[pwgt0, pwgt, pbuy, psell]], ...
{'Initial', 'Weight', 'Purchases', 'Sales'}, 'obsnames', p.AssetList);
display(Blotter)
```

Blotter =

	Initial	Weight	Purchases	Sales
Bonds	30	0	0	30
Large-Cap Equities	30	50.597	20.597	0
Small-Cap Equities	20	12.416	0	7.5843
Emerging Equities	10	36.987	26.987	0

If you have prices for each asset (in this example, they can be ETFs), add them to your blotter and then use the tools of the `dataset` object to obtain shares and shares to be traded. For an example, see “Asset Allocation Case Study” on page 4-180.

See Also

`PortfolioMAD` | `estimateScenarioMoments` | `checkFeasibility`

Related Examples

- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Working with Other Portfolio Objects

This example shows how to examine portfolio optimization problems according to different combinations of return and risk proxies. The `PortfolioMAD` object is for MAD portfolio optimization. The `Portfolio` object is for mean-variance portfolio optimization. Sometimes, you might want to examine portfolio optimization problems according to different combinations of return and risk proxies. A common example is that you want to do a MAD portfolio optimization and then want to work primarily with moments of portfolio returns. Suppose that you set up a MAD portfolio optimization problem with:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];

rng('default')
p = PortfolioMAD;
p = setAssetList(p, 'Bonds', 'Large-Cap Equities', 'Small-Cap Equities', 'Emerging Equities');
p = setInitPort(p, pwgt0);
p = simulateNormalScenariosByMoments(p, m, C, 20000);
p = setDefaultConstraints(p);
```

To work with the same problem in a mean-variance framework, you can use the scenarios from the `PortfolioMAD` object to set up a `Portfolio` object so that `p` contains a MAD optimization problem and `q` contains a mean-variance optimization problem based on the same data.

```
q = Portfolio('AssetList', p.AssetList);
q = estimateAssetMoments(q, p.getScenarios);
q = setDefaultConstraints(q);

pwgt = estimateFrontier(p);
qwgt = estimateFrontier(q);
```

Since each object has a different risk proxy, it is not possible to compare results side by side. To obtain means and standard deviations of portfolio returns, you can use the functions associated with each object to obtain:

```
pret = estimatePortReturn(p, pwgt);
pstd = estimatePortStd(p, pwgt);
qret = estimatePortReturn(q, qwgt);
qstd = estimatePortStd(q, qwgt);
```

```
[pret, qret]
```

```
ans = 10×2
```

```
0.0594    0.0591
0.0727    0.0724
0.0860    0.0857
0.0993    0.0991
0.1126    0.1124
0.1259    0.1257
0.1391    0.1390
```

```

0.1524    0.1524
0.1657    0.1657
0.1790    0.1790

```

```
[pstd, qstd]
```

```
ans = 10×2
```

```

0.0761    0.0761
0.0825    0.0823
0.0988    0.0984
0.1210    0.1206
0.1465    0.1462
0.1739    0.1736
0.2054    0.2051
0.2469    0.2466
0.2945    0.2944
0.3462    0.3462

```

To produce comparable results, you can use the returns or risks from one portfolio optimization as target returns or risks for the other portfolio optimization.

```

qwgt = estimateFrontierByReturn(q, pret);
qret = estimatePortReturn(q, qwgt);
qstd = estimatePortStd(q, qwgt);

```

```
[pret, qret]
```

```
ans = 10×2
```

```

0.0594    0.0594
0.0727    0.0727
0.0860    0.0860
0.0993    0.0993
0.1126    0.1126
0.1259    0.1259
0.1391    0.1391
0.1524    0.1524
0.1657    0.1657
0.1790    0.1790

```

```
[pstd, qstd]
```

```
ans = 10×2
```

```

0.0761    0.0761
0.0825    0.0825
0.0988    0.0988
0.1210    0.1210
0.1465    0.1465
0.1739    0.1739
0.2054    0.2054
0.2469    0.2469
0.2945    0.2945
0.3462    0.3462

```


Now it is possible to compare standard deviations of portfolio returns from either type of portfolio optimization.

See Also

PortfolioMAD | Portfolio

Related Examples

- “Creating the Portfolio Object” on page 4-25
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

Troubleshooting MAD Portfolio Optimization Results

PortfolioMAD Object Destroyed When Modifying

If a PortfolioMAD object is destroyed when modifying, remember to pass an existing object into the PortfolioMAD object if you want to modify it, otherwise it creates a new object. See “Creating the PortfolioMAD Object” on page 6-22 for details.

Matrix Incompatibility and "Non-Conformable" Errors

If you get matrix incompatibility or "non-conformable" errors, the representation of data in the tools follows a specific set of basic rules described in “Conventions for Representation of Data” on page 6-20.

Missing Data Estimation Fails

If asset return data has missing or NaN values, the `simulateNormalScenariosByData` function with the 'missingdata' flag set to true may fail with either too many iterations or a singular covariance. To correct this problem, consider this:

- If you have asset return data with no missing or NaN values, you can compute a covariance matrix that may be singular without difficulties. If you have missing or NaN values in your data, the supported missing data feature requires that your covariance matrix must be positive-definite, that is, nonsingular.
- `simulateNormalScenariosByData` uses default settings for the missing data estimation procedure that might not be appropriate for all problems.

In either case, you might want to estimate the moments of asset returns separately with either the ECM estimation functions such as `ecmmle` or with your own functions.

mad_optim_transform Errors

If you obtain optimization errors such as:

```
Error using mad_optim_transform (line 276)
Portfolio set appears to be either empty or unbounded. Check constraints.
```

```
Error in PortfolioMAD/estimateFrontier (line 64)
    [AI, bI, AE, bE, lB, uB, f0, f, x0] = mad_optim_transform(obj);
```

or

```
Error using mad_optim_transform (line 281)
Cannot obtain finite lower bounds for specified portfolio set.
```

```
Error in PortfolioMAD/estimateFrontier (line 64)
    [AI, bI, AE, bE, lB, uB, f0, f, x0] = mad_optim_transform(obj);
```

Since the portfolio optimization tools require a bounded portfolio set, these errors (and similar errors) can occur if your portfolio set is either empty and, if nonempty, unbounded. Specifically, the portfolio optimization algorithm requires that your portfolio set have at least a finite lower bound. The best way to deal with these problems is to use the validation methods in “Validate the MAD Portfolio Problem” on page 6-80. Specifically, use `estimateBounds` to examine your portfolio set, and use `checkFeasibility` to ensure that your initial portfolio is either feasible and, if infeasible, that you have sufficient turnover to get from your initial portfolio to the portfolio set.

Tip To correct this problem, try solving your problem with larger values for turnover and gradually reduce to the value that you want.

Efficient Portfolios Do Not Make Sense

If you obtain efficient portfolios that, do not seem to make sense, this can happen if you forget to set specific constraints or you set incorrect constraints. For example, if you allow portfolio weights to fall between 0 and 1 and do not set a budget constraint, you can get portfolios that are 100% invested in every asset. Although it may be hard to detect, the best thing to do is to review the constraints you have set with display of the PortfolioMAD object. If you get portfolios with 100% invested in each asset, you can review the display of your object and quickly see that no budget constraint is set. Also, you can use `estimateBounds` and `checkFeasibility` to determine if the bounds for your portfolio set make sense and to determine if the portfolios you obtained are feasible relative to an independent formulation of your portfolio set.

See Also

PortfolioMAD | `estimateScenarioMoments` | `checkFeasibility`

Related Examples

- “Troubleshooting Portfolio Optimization Results” on page 4-145
- “Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148
- “Postprocessing Results to Set Up Tradable Portfolios” on page 6-113
- “Creating the PortfolioMAD Object” on page 6-22
- “Working with MAD Portfolio Constraints Using Defaults” on page 6-50
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

More About

- “PortfolioMAD Object” on page 6-17
- “Portfolio Optimization Theory” on page 6-3
- “PortfolioMAD Object Workflow” on page 6-16

